

TEXTBOOK

Martin H. Trauth

MATLAB[®] Recipes for Earth Sciences

Fifth Edition

EXTRAS ONLINE

 Springer

Springer Textbooks in Earth Sciences, Geography and Environment

The Springer Textbooks series publishes a broad portfolio of textbooks on Earth Sciences, Geography and Environmental Science. Springer textbooks provide comprehensive introductions as well as in-depth knowledge for advanced studies. A clear, reader-friendly layout and features such as end-of-chapter summaries, work examples, exercises, and glossaries help the reader to access the subject. Springer textbooks are essential for students, researchers and applied scientists.

More information about this series at <http://www.springer.com/series/15201>

Martin H. Trauth

MATLAB® Recipes for Earth Sciences

Fifth Edition

Martin H. Trauth
Institute of Geosciences
University of Potsdam
Potsdam, Germany

ISSN 2510-1307 ISSN 2510-1315 (electronic)
Springer Textbooks in Earth Sciences, Geography and Environment
ISBN 978-3-030-38440-1 ISBN 978-3-030-38441-8 (eBook)
<https://doi.org/10.1007/978-3-030-38441-8>

MATLAB® and Simulink® are registered trademarks of The MathWorks, Inc., 1 Apple Hill Drive, Natick, MA 01760-2098, USA, <http://www.mathworks.com>.

1st–4th editions: © Springer-Verlag Berlin Heidelberg 2006, 2007, 2010, 2015

5th edition: © Springer Nature Switzerland AG 2021

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Preface to the Fifth Edition

The book *MATLAB Recipes for Earth Sciences* is designed to help undergraduate and postgraduate students, doctoral students, post-doctoral researchers, and professionals find quick solutions for common data analysis problems in earth sciences. It provides a minimal amount of theoretical background and demonstrates the application of all described methods through the use of examples.

The book contains MATLAB scripts, or *M-files*, that can be used to solve typical problems in earth sciences by simple statistics, time-series analysis, geostatistics, and image processing, and also demonstrates the application of selected advanced techniques of data analysis such as nonlinear time-series analysis, adaptive filtering, bootstrapping, and terrain analysis. The book's supplementary electronic material includes recipes with all the MATLAB commands featured in the book and the example data. The MATLAB codes can be easily modified for use with the reader's own data and projects.

The MATLAB software, which has been in existence for more than 40 years, is used since it not only provides numerous ready-to-use algorithms for most methods of data analysis but also allows the existing routines to be modified and expanded, or new software to be developed. MATLAB is also used because, unlike free alternatives, it offers a warranty, complete and up-to-date documentation, professional support with a guaranteed response time of less than one working day, and the ability to certify computer code. However, most methods are introduced with very simple codes before any MATLAB-specific functions are used, so that users of free alternatives to MATLAB will also appreciate the book.

This revised and updated Fifth Edition includes new sections on error analysis, aligning stratigraphic sequences, calculating vegetation indices, log-ratio transformations to overcome the closed-sum problem of compositional data, graphical representation and statistics of spherical data. Many of the pre-existing sections have also been expanded and rewritten, and numerous new example data sets have been included. This takes into account new developments in MATLAB, such as the Live Editor and App Designer, and the datastore family of functions for importing mixed data. Some MATLAB algorithms have also been modified, for example, to take into account changes in satellite data and in data sets used for digital terrain models.

In order to derive the maximum benefit from this book, the reader will need to have access to the MATLAB software and be able to execute the recipes while reading the book. The MATLAB recipes display various graphs on the screen that are not shown in the printed book. The tutorial-style book does, however, contain numerous figures making it possible to go through the text without actually running MATLAB on a computer. I have developed the recipes using MATLAB 9.9 Release R2020b, but most of them will also work with earlier software releases. While undergraduates participating in a course on data analysis might go through the entire book, more experienced readers may choose to refer to only one particular method in order to solve a specific problem. The concept of the book and the contents of its chapters are therefore outlined below, in order to make it easier to use for readers with a variety of different requirements.

- *Chapter 1*—This chapter introduces some fundamental concepts of samples and populations. It also links the various types of data, and questions to be answered from the data, to the methods described in the succeeding chapters.
- *Chapter 2*—A tutorial-style introduction to MATLAB is designed for earth scientists. Readers already familiar with the software are advised to proceed directly to the succeeding chapters. The Fifth Edition now includes new text on floating-point numbers, string and table arrays, the datastore family of functions, publishing and sharing code using the new Live Editor, and creating graphical user interfaces using the new App Designer.
- *Chapters 3 and 4*—Fundamentals in univariate and bivariate statistics. These two chapters contain basic concepts in statistics and also introduce advanced topics such as resampling schemes and cross-validation. Readers who are already familiar with basic statistics might want to skip these chapters. The Fifth Edition now also includes an introduction to error analysis.
- *Chapters 5 and 6*—Readers who wish to work with time series are recommended to read both of these chapters. Time-series analysis and signal processing are closely linked. A good knowledge of statistics is required to work successfully with these methods. The Fifth Edition now includes a paragraph on aliasing and the importance of the Nyquist frequency, a revised version of the sections on Lomb–Scargle and wavelet power spectral analysis using recently introduced MATLAB functions, a revised section on detecting and removing jumps in time series, a new section on aligning stratigraphic sequences, and a revised section on recurrence plots and recurrence quantification analysis.
- *Chapters 7 and 8*—I recommend reading through both of these chapters since the processing methods used for spatial data and for images have much in common. Moreover, spatial data and images are often combined in earth sciences, for instance, when projecting satellite images onto digital elevation models. The Fifth Edition now includes updated sections on digital elevation models and satellite imagery, since some of the data sets used have recently been modified. It also includes a new section on calculating vegetation indices from multispectral data, acquired with either hand-held cameras or satellite-borne sensors.

- *Chapter 9*—Data sets in earth sciences often have many variables and many data points. Multivariate methods are applied to a great variety of large data sets, including satellite imagery data. Any reader particularly interested in multivariate methods is advised to read Chaps. 3 and 4 before proceeding to this chapter. The Fifth Edition includes a rewritten section on principal component analysis and an entirely new section on the use of Aitchison's log-ratio transformation to avoid the closed-sum problem in compositional data. Most of the older sections have been revised.
- *Chapter 10*—Methods to analyze circular and spherical data are widely used in earth sciences. Structural geologists, for example, measure and analyze the orientation of slickensides (or striae) on a fault plane, and statistical analysis of circular data is used in paleomagnetic applications. Microstructural investigations include the analysis of grain shapes and quartz *c*-axis orientations in thin sections. This chapter has been fundamentally revised and extended, new example data are used, and two completely new sections on spherical data have been added.

The book has benefited from the comments of many people, in particular, my contributing authors Robin Gebbers and Norbert Marwan, as well as from many colleagues and friends worldwide who have read the book, pointed out mistakes, and made suggestions for improvement. I have great difficulty expressing my gratitude to Ed Manning in an appropriate manner; he has been improving the readability of the book for more than a decade since the First Edition was rightly criticized for numerous linguistic and typographic errors. I also very much appreciate the expertise and patience of Elisabeth Sillmann at *blaetterwaldDesign*, who has supported and advised me with the graphic design since the First Edition. I am very grateful to Annett Büttner, Doerthe Mennecke-Buehler, Marion Schneider, Helen Rachner, Kay Stoll, Claus-Dieter Bachem, Agata Weissmann, Femina Joshi Arul Thas, Solaimuthu Thambidurai, Karthik Raj Selvaraj, Nagaraj Ramamoorthy, and their team at *Springer* for their continuing interest and support in publishing my books. I would also like to thank Brunhilde Schulz, Andreas Bohlen, Robert Laudien and their team at *UP Transfer GmbH* for organizing short courses on the book at the universities of Kiel, Bremen, Bratislava, Ghent, Munich, Nairobi, Köln, Stockholm, Amsterdam, Aberystwyth, and Potsdam, as well as at UA Barcelona, UC London, Brown University Providence, the BGR Hannover, the BGI Bayreuth, the NHM Vienna, and GNS Science Wellington over almost two decades.

I also acknowledge Emily Devine, John Elhilow, Naomi Fernandes and Alexa Sanchez from the *Book Program* at *The MathWorks, Inc.*, Mihaela Jarema, Sebastian Gross, Cedric Esmarch, Steve Schäfer, Martin Strathemann, Andreas Himmeldorf, and their teams at *The MathWorks GmbH Deutschland*, Steve Eddins for dealing with my many questions about the history of MATLAB and for helping in the use of the Image Processing Toolbox, and Conor McGovern, Friedrich Hempel, Vijay Sridhar, Jim David, Martijn Aben, Oliver Jährgig, and Christoph Stockhammer at MathWorks for their patience in dealing with the large number of questions that I have sent to the Support section of MathWorks over the past few

months. I think the excellent support provided by the company more than justifies any expenditure that I have made on MATLAB licenses over the past 25 years or more.

I would also like to thank the *NASA/GSFC/METI/ERSDAC/JAROS* and *U.S./Japan ASTER Science Team*, and the director Mike Abrams, for allowing me to include the ASTER images in this book. I am grateful to Stuart W. Frye at the NASA Goddard Space Flight Center (GSFC) and his team for allowing me to include EO-1 data in the book, as well as for fruitful discussions while working on the section about Hyperion images.

Potsdam, Germany
November 2020

Martin H. Trauth

Contents

1	Data Analysis in Earth Sciences	1
1.1	Introduction	1
1.2	Data Collection	2
1.3	Types of Data	3
1.4	Methods of Data Analysis	6
	Recommended Reading	8
2	Introduction to MATLAB	9
2.1	MATLAB in Earth Sciences	9
2.2	Getting Started	10
2.3	The Syntax	12
2.4	Array Manipulation	17
2.5	Data Types in MATLAB	23
2.6	Data Storage and Handling	38
2.7	Control Flow	47
2.8	Scripts and Functions	52
2.9	Basic Visualization Tools	56
2.10	Generating Code to Recreate Graphics	59
2.11	Publishing and Sharing MATLAB Code	61
2.12	Creating Graphical User Interfaces	64
	Recommended Reading	68
3	Univariate Statistics	69
3.1	Introduction	69
3.2	Empirical Distributions	70
3.3	Examples of Empirical Distributions	75
3.4	Theoretical Distributions	86
3.5	Examples of Theoretical Distributions	94
3.6	Hypothesis Testing	99
3.7	The t -Test	101
3.8	The F -Test	106
3.9	The χ^2 -Test	110
3.10	The Kolmogorov-Smirnov Test	114

3.11	Mann-Whitney Test	117
3.12	The Ansari-Bradley Test	124
3.13	Distribution Fitting	131
3.14	Error Analysis	135
	Recommended Reading	143
4	Bivariate Statistics	145
4.1	Introduction	145
4.2	Correlation Coefficients	146
4.3	Classical Linear Regression Analysis	155
4.4	Analyzing the Residuals	158
4.5	Bootstrap Estimates of the Regression Coefficients	161
4.6	Jackknife Estimates of the Regression Coefficients	162
4.7	Cross Validation	164
4.8	Reduced Major Axis Regression	165
4.9	Curvilinear Regression	166
4.10	Nonlinear and Weighted Regression	169
4.11	Classical Linear Regression of Log-Transformed Data	173
	Recommended Reading	175
5	Time-Series Analysis	177
5.1	Introduction	177
5.2	Generating Signals	178
5.3	Auto-Spectral and Cross-Spectral Analysis	183
5.4	Examples of Auto-Spectral and Cross-Spectral Analysis	187
5.5	Interpolating and Analyzing Unevenly-Spaced Data	196
5.6	Evolutionary Power Spectrum	200
5.7	Lomb-Scargle Power Spectrum	205
5.8	Wavelet Power Spectrum	210
5.9	Detecting Abrupt Transitions in Time Series	216
5.10	Aligning Stratigraphic Sequences	226
5.11	Nonlinear Time-Series Analysis (by N. Marwan)	237
	Recommended Reading	255
6	Signal Processing	259
6.1	Introduction	259
6.2	Generating Signals	260
6.3	Linear Time-Invariant Systems	261
6.4	Convolution, Deconvolution and Filtering	264
6.5	Comparing Functions for Filtering Data Series	268
6.6	Recursive and Nonrecursive Filters	270
6.7	Impulse Response	272
6.8	Frequency Response	274
6.9	Filter Design	281

6.10	Adaptive Filtering	284
	Recommended Reading	291
7	Spatial Data	293
7.1	Types of Spatial Data	293
7.2	The Global Geography Database GSHHG	294
7.3	The 1 Arc-Minute Gridded Global Relief Data ETOPO1	298
7.4	The 30 Arc-Seconds Elevation Model GTOPO30	301
7.5	The Shuttle Radar Topography Mission SRTM	304
7.6	Exporting 3D Graphics to Create Interactive Documents	309
7.7	Gridding and Contouring	314
7.8	Comparison of Methods and Potential Artifacts	320
7.9	Statistics of Point Distributions	327
7.10	Analysis of Digital Elevation Models (by R. Gebbers)	335
7.11	Geostatistics and Kriging (by R. Gebbers)	345
	Recommended Reading	362
8	Image Processing	365
8.1	Introduction	365
8.2	Data Storage	366
8.3	Importing, Processing and Exporting Images	371
8.4	Importing, Processing and Exporting LANDSAT Images	378
8.5	Importing and Georeferencing TERRA ASTER Images	382
8.6	Processing and Exporting EO-1 Hyperion Images	390
8.7	Digitizing from the Screen	396
8.8	Image Enhancement, Correction and Rectification	398
8.9	Color-Intensity Transects Across Varved Sediments	406
8.10	Grain Size Analysis from Microscope Images	411
8.11	Quantifying Charcoal in Microscope Images	419
8.12	Shape-Based Object Detection in Images	423
8.13	The Normalized Difference Vegetation Index	430
	Recommended Reading	437
9	Multivariate Statistics	439
9.1	Introduction	439
9.2	Principal Component Analysis	441
9.3	Independent Component Analysis (by N. Marwan)	458
9.4	Discriminant Analysis	464
9.5	Cluster Analysis	470
9.6	Multiple Linear Regression	475
9.7	Aitchison's Log-Ratio Transformation	484
	Recommended Reading	488

10	Directional Data	491
10.1	Introduction	491
10.2	Graphical Representation of Circular Data	493
10.3	Empirical Distributions of Circular Data	495
10.4	Theoretical Distributions of Circular Data	499
10.5	Test for Randomness of Circular Data	502
10.6	Test for the Significance of a Mean Direction	503
10.7	Test for the Difference between Two Sets of Directions	506
10.8	Graphical Representation of Spherical Data	509
10.9	Statistics of Spherical Data	513
	Recommended Reading	516



1.1 Introduction

Earth scientists make observations and gather data about the natural processes that operate on planet Earth. They formulate and test hypotheses on the forces that have acted on a particular region to create its structure and also make predictions about future changes to the planet. All of these steps in exploring the Earth involve the acquisition and analysis of numerical data. An earth scientist therefore needs to have a firm understanding of statistical and numerical methods as well as the ability to utilize relevant computer software packages, in order to be able to analyze the acquired data.

This book introduces some of the most important methods of data analysis employed in earth sciences and illustrates their use through examples using the MATLAB[®] software package. These examples can then be used as recipes for the analysis of the reader's own data, after having learned their application with synthetic data. This introductory chapter deals with data acquisition (Sect. 1.2), the various types of data (Sect. 1.3) and the appropriate methods for analyzing earth science data (Sect. 1.4). We therefore first explore the characteristics of typical data sets and subsequently investigate the various ways of analyzing data using MATLAB.

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_1) contains supplementary material, which is available to authorized users.

1.2 Data Collection

Most data sets in earth sciences have a very limited sample size and also contain a significant number of uncertainties. Such data sets are typically used to describe rather large natural phenomena, such as a granite body, a large landslide or a widespread sedimentary unit. The methods described in this book aim to find a way of predicting the characteristics of a larger *population* from a much smaller *sample* (Fig. 1.1). An appropriate sampling strategy is the first step towards obtaining a good data set. The development of a successful strategy for field sampling requires decisions on the *sample size* and the *spatial sampling scheme*.

The sample size includes the sample volume, the sample weight and the number of samples collected in the field. The sample weights or volumes can be critical factors if the samples are later analyzed in a laboratory and most statistical methods also have a minimum requirement for the sample size. The sample size also affects the number of subsamples that can be collected from a single sample. If the population is heterogeneous then the sample needs to be large enough to represent the population's

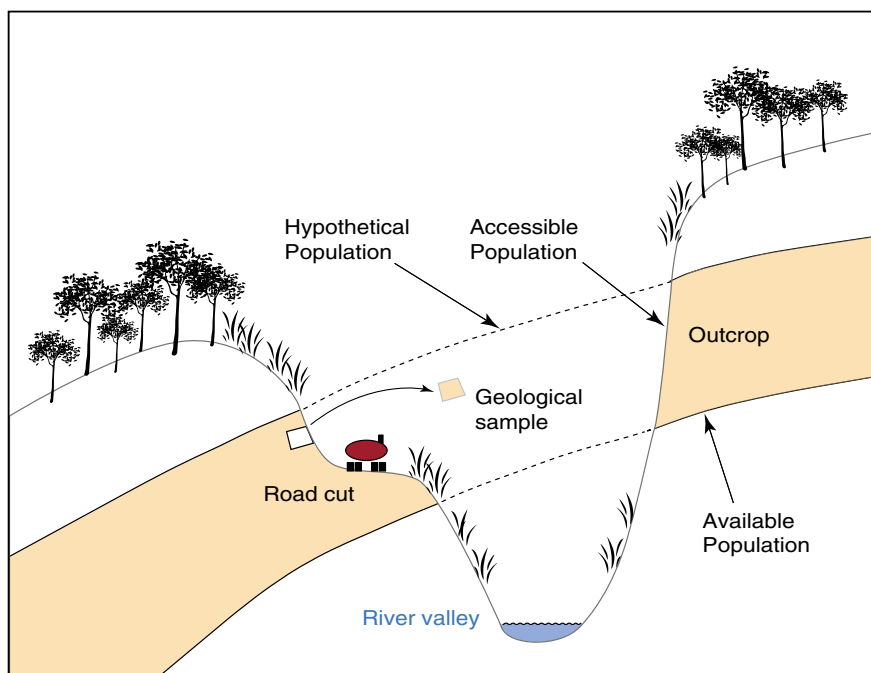


Fig. 1.1 Samples and populations. Deep valley incision has eroded parts of a sandstone unit (*hypothetical population*). The remaining sandstone (*available population*) can only be sampled from outcrops, i.e. road cuts and quarries (*accessible population*). Note the difference between a statistical sample as a representative of a population and a geological sample as a piece of rock. Modified from Swan and Sandilands (1995).

variability, but on the other hand samples should be as small as possible in order to minimize the time and costs involved in their analysis. The collection of smaller pilot samples is recommended prior to defining a suitable sample size.

The design of the spatial sampling scheme is dependent on the availability of outcrops or other material suitable for sampling (Fig. 1.2). Sampling in quarries typically leads to clustered data, whereas sampling along road cuts, shoreline cliffs or steep gorges results in one-dimensional traverse sampling schemes. A more uniform sampling pattern can be designed where there is 100% exposure or if there are no financial limitations. A regular sampling scheme results in a gridded distribution of sample locations, whereas a uniform sampling strategy includes the random location of a sampling point within a grid square. Although these sampling schemes might be expected to provide superior methods for sampling collection, evenly-spaced sampling locations tend to miss small-scale variations in the area, such as thin mafic dykes within a granite body or the spatially-restricted occurrence of a particular type of fossil.

The correct sampling strategy will depend on the objectives of the investigation, the type of analysis required and the desired level of confidence in the results. Having chosen a suitable sampling strategy, the quality of the sample can be influenced by a number of factors resulting in the samples not being truly representative of the larger population. Chemical or physical alteration, contamination by other material or displacement by natural and anthropogenic processes may all result in erroneous results and interpretations. It is therefore recommended that the quality of the samples, the method of data analysis employed and the validity of the conclusions drawn from the analysis be checked at each stage of the investigation.

1.3 Types of Data

Most earth science data sets consist of numerical measurements, although some information can also be represented by a list of names such as fossils and minerals (Fig. 1.3). The available methods for data analysis may require certain types of data in earth sciences. These are

- *Nominal data*—Information in earth sciences is sometimes presented as a list of names, e.g. the various fossil species collected from a limestone bed or the minerals identified in a thin section. In some studies, these data are converted into a binary representation, i.e. *one* for present and *zero* for absent. Special statistical methods are available for the analysis of such data sets.
- *Ordinal data*—These are numerical data representing observations that can be ranked, but in which the intervals along the scale are irregularly spaced. The hardness scale of German mineralogist Friedrich Mohs (1773–1839) is one example of an ordinal scale. The hardness value indicates the material's resistance to scratching. Diamond has a hardness of 10, whereas the value for talc is

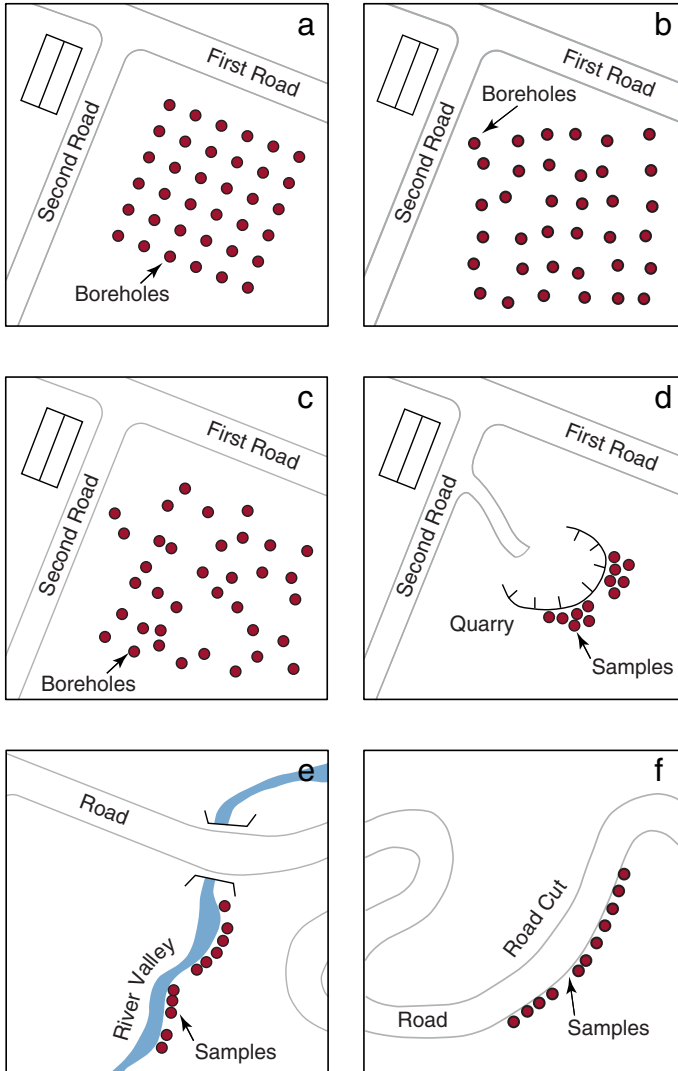


Fig. 1.2 Sampling schemes. **a** Regular sampling on an evenly-spaced rectangular grid, **b** uniform sampling by obtaining samples randomly located within regular grid squares, **c** random sampling using uniformly-distributed xy coordinates, **d** clustered sampling constrained by limited access in a quarry, and **e** traverse sampling along **e** river valleys and **f** road cuts. Modified from Swan and Sandilands (1995).

1, but in terms of absolute hardness diamond (hardness 10) is four times harder than corundum (hardness 9) and six times harder than topaz (hardness 8). The Modified Mercalli Scale, which attempts to categorize the effects of earthquakes,

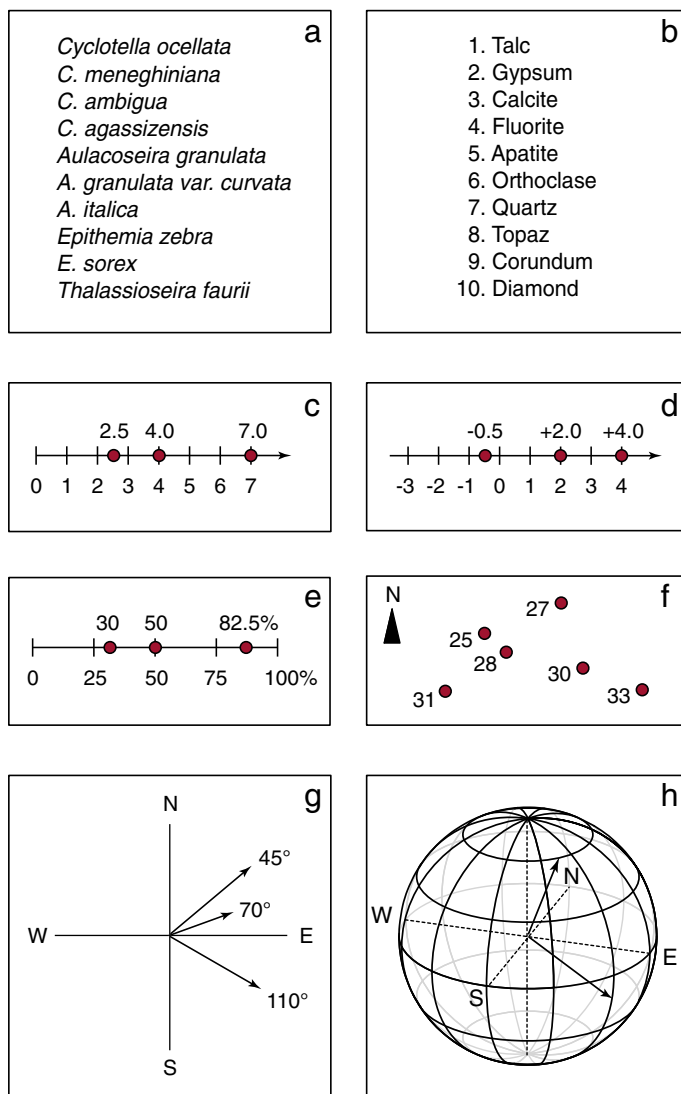


Fig. 1.3 Types of earth science data. **a** Nominal data, **b** ordinal data, **c** ratio data, **d** interval data, **e** closed data, **f** spatial data, and **g–h** directional data. All of these data types are described in this book. Modified from Swan and Sandilands (1995).

is another example of an ordinal scale; it ranks earthquakes from intensity I (barely felt) to XII (total destruction) (Richter 1958).

- **Ratio data**—These data are characterized by a constant length of successive intervals, therefore offering a great advantage over ordinal data. The zero point is the natural termination of the data scale, and this type of data allows for either

discrete or continuous data sampling. Examples of such data sets include length or weight data.

- *Interval data*—These are ordered data that have a constant length of successive intervals, but in which the data scale is not terminated by zero. Temperatures C and F represent an example of this data type even though arbitrary zero points exist for both scales. This type of data may be sampled continuously or in discrete intervals.

In addition to these standard data types, earth scientists frequently encounter special kinds of data such as

- *Closed data*—These data are expressed as proportions and add up to a fixed total such as 100%. Compositional data represent the majority of closed data, such as element compositions of rock samples.
- *Spatial data*—These are collected in a 2D or 3D study area. The spatial distribution of a certain fossil species, the spatial variation in thickness of a sandstone bed and the distribution of tracer concentrations in groundwater are examples of this type of data, which is likely to be the most important data type in earth sciences.
- *Directional data*—These data are expressed in angles. Examples include the strike and dip of bedding, the orientation of elongated fossils or the flow direction of lava. This is another very common type of data in earth sciences.

Most of these different types of data require specialized methods of analysis, some of which are outlined in the next section.

1.4 Methods of Data Analysis

Data analysis uses precise characteristics of small samples to hypothesize about the general phenomenon of interest. Which particular method is used to analyze the data depends on the data type and the project requirements. The various methods available include:

- *Univariate methods*—Each variable is assumed to be independent of the others and is explored individually. The data are presented as a list of numbers representing a series of points on a scaled line. Univariate statistical methods include the collection of information about the variable, such as the minimum and maximum values, the average, and the dispersion about the average. This information is then used to attempt to infer the underlying processes responsible for the variations in the data. Examples are the effects of chemical weathering on the sodium content of volcanic glass shards, or the influence of specific environmental factors on the sizes of snail shells within a sediment layer.

- *Bivariate methods*—Two variables are investigated together to detect relationships between these two parameters. For example, the correlation coefficient may be calculated to investigate whether there is a linear relationship between two variables. Alternatively, the bivariate regression analysis may be used to find an equation that describes the relationship between the two variables. An example of a bivariate plot is the *Harker Diagram*, which is one of the oldest methods of visualizing geochemical data from igneous rocks and simply plots the concentrations of oxides of chemical elements against SiO_2 (Harker 1909).
- *Time-series analysis*—These methods investigate data sequences as a function of time. The time series is decomposed into a long-term trend, a systematic (periodic, cyclic, rhythmic) component and an irregular (random, stochastic) component. A widely used technique to describe cyclic components of a time series is that of spectral analysis. Examples of the application of these techniques include the investigation of cyclic climatic variations in sedimentary rocks, or the analysis of seismic data.
- *Signal processing*—This includes all techniques for manipulating a signal to minimize the effects of noise in order to correct all kinds of unwanted distortions or to separate various components of interest. It includes the design and realization of filters, and their application to the data. These methods are widely used in combination with time-series analysis, e.g. to increase the signal-to-noise ratio in climate time series, digital images or geophysical data.
- *Spatial analysis*—This is the analysis of parameters in 2D or 3D space and hence two or three of the required parameters are coordinate numbers. These methods include descriptive tools to investigate the spatial patterns of geographically distributed data. Other techniques involve spatial regression analysis to detect spatial trends. Also included are 2D and 3D interpolation techniques, which help to estimate surfaces representing the predicted continuous distribution of the variable throughout the area. Examples are drainage-system analysis, the identification of old landscape forms and lineament analysis in tectonically active regions.
- *Image processing*—The processing and analysis of images has become increasingly important in earth sciences. These methods involve importing and exporting, compressing and decompressing, and displaying images. Image processing also aims to enhance images for improved intelligibility, and to manipulate images in order to increase the signal-to-noise ratio. Advanced techniques are used to extract specific features or analyze shapes and textures, such as for counting mineral grains or fossils in microscope images. Another important application of image processing is in the use of satellite remote sensing to map certain types of rocks, soils and vegetation, as well as other parameters such as soil moisture, rock weathering and erosion.
- *Multivariate analysis*—These methods involve the observation and analysis of more than one statistical variable at a time. Since the graphical representation of multidimensional data sets is difficult, most of these methods include dimension reduction. Multivariate methods are widely used on geochemical data, for instance in tephrochronology where volcanic ash layers are correlated by geochemical fingerprinting of glass shards. Another important usage is in the

comparison of species assemblages in ocean and lake sediments for the reconstruction of paleoenvironments.

- *Analysis of directional data*—Methods to analyze circular and spherical data are widely used in earth sciences. Structural geologists measure and analyze the orientation of slickensides (or striae) on a fault plane, circular statistical methods are common in paleomagnetic studies, and microstructural investigations include the analysis of grain shapes and quartz *c*-axis orientations in thin sections.

Some of these methods of data analysis require the application of numerical methods such as interpolation techniques. While the following text deals mainly with statistical techniques it also introduces several numerical methods commonly used in earth sciences.

Recommended Reading

- Araghinejad S (2014) Data-driven modeling: using MATLAB® in water resources and environmental engineering. Springer, Berlin, Heidelberg, New York
- Borradaile G (2003) Statistics of earth science data: their distribution in time, space and orientation. Springer, Berlin, Heidelberg, New York
- Carr JR (1994) Numerical analysis for the geological sciences. Prentice Hall, Englewood Cliffs, New Jersey
- Davis JC (2002) Statistics and data analysis in geology, 3rd edn. Wiley, New York
- Gonzalez RC, Woods RE, Eddins SL (2009) Digital image processing using MATLAB, 2nd ed. Gatesmark Publishing, LLC
- Haneberg WC (2004) Computational geosciences with mathematica. Springer, Berlin, Heidelberg, New York
- Harker A (1909) The natural history of igneous rocks. Macmillan, New York
- Holzbecher E (2012) Environmental modeling using MATLAB, 2nd edn. Springer, Berlin, Heidelberg, New York
- Menke W, Menke J (2012) Environmental data analysis with MATLAB. Springer, Berlin, Heidelberg, New York
- Middleton GV (1999) Data analysis in the earth sciences using MATLAB. Prentice Hall, New Jersey
- Press WH, Teukolsky SA, Vetterling WT, Flannery BP (2007) Numerical recipes: the art of scientific computing, 3rd edn. Cambridge University Press, Cambridge
- Quarteroni A, Saleri F, Gervasio P (2014) Scientific computing with MATLAB and Octave, 4th edn. Springer, Berlin, Heidelberg, New York
- Richter CF (1958) Elementary seismology. WH Freeman and Company, San Francisco
- Swan ARH, Sandilands M (1995) Introduction to geological data analysis. Blackwell Sciences, Oxford
- Wilks DS (2011) Statistical methods in the atmospheric sciences, 3rd edn. Academic Press, Elsevier, Amsterdam



2.1 MATLAB in Earth Sciences

MATLAB® is a software package developed by The MathWorks, Inc., founded by Cleve Moler, Jack Little and Steve Bangert in 1984, which has its headquarters in Natick, Massachusetts (<http://www.mathworks.com>). MATLAB was designed to perform mathematical calculations, to analyze and visualize data, and to facilitate the writing of new software programs. The advantage of this software is that it combines comprehensive math and graphics functions with a powerful high-level language. Since MATLAB contains a large library of ready-to-use routines for a wide range of applications, the user can solve technical computing problems much more quickly than with traditional programming languages, such as C and FORTRAN. The standard library of functions can be significantly expanded by add-on toolboxes, which are collections of functions for special purposes such as image processing, creating map displays, performing geospatial data analysis or solving partial differential equations.

During the last few years MATLAB has become an increasingly popular tool in earth sciences. It has been used for finite element modeling, processing of seismic data, analyzing satellite imagery, and for the generation of digital elevation models from satellite data. The continuing popularity of the software is also apparent in published scientific literature, and many conference presentations have also made reference to MATLAB. Universities and research institutions have recognized the need for MATLAB training for staff and students, and many earth science departments across the world now offer MATLAB courses for undergraduates.

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_2) contains supplementary material, which is available to authorized users.

The MathWorks, Inc. provides classroom kits for teachers at a reasonable price and it is also possible for students to purchase a low-cost edition of the software. This student version provides an inexpensive way for students to improve their MATLAB skills. In addition, there are campus licenses (*Total Academic Headcount License*) with numerous toolboxes, that can be purchased by universities. Employees and students can then download and install MATLAB for free, use it independently of a license server, and participate in an extensive online training program called *MATLAB Academy*.

The following sections contain a tutorial-style introduction to MATLAB, covering the setup on the computer (Sect. 2.2), the MATLAB syntax (Sects. 2.3 and 2.4), data input and output (Sects. 2.5 and 2.6), programming (Sects. 2.7 and 2.8), and visualization (Sect. 2.9). Advanced sections are also included on generating M-files to recreate graphics (Sect. 2.10), on publishing and sharing MATLAB code (Sect. 2.11), and on creating graphical user interfaces (Sect. 2.12). The reader is recommended to go through the entire chapter in order to obtain a good knowledge of the software before proceeding to the following chapters of the book. A more detailed introduction can be found in the *MATLAB Primer* (MathWorks 2020a) which is available in print form, online and as PDF file.

In this book we use MATLAB Version 9.9 (Release 2020b), the Bioinformatics Toolbox Version 4.15, the Image Processing Toolbox Version 11.2, the Mapping Toolbox Version 5.0, the Signal Processing Toolbox Version 8.5, Simulink 3D Animation Version 9.1, the Statistics and Machine Learning Toolbox Version 12.0, and the Wavelet Toolbox Version 5.5.

2.2 Getting Started

The software package comes with extensive documentation, tutorials and examples. The first three chapters of the book *MATLAB Primer* (MathWorks 2020a) are directed at beginners. The chapters on programming, creating graphical user interfaces (GUIs) and development environments are aimed at more advanced users. Since *MATLAB Primer* provides all the information required to use the software, this introduction concentrates on the most relevant software components and tools used in the following chapters of this book.

After the installation of MATLAB, the software is launched by clicking the application's shortcut icon. We can also launch the software without using the MATLAB Desktop by typing

```
cd /Applications/MATLAB_R2020b.app/bin
matlab -nojvm -nodisplay -nosplash
```

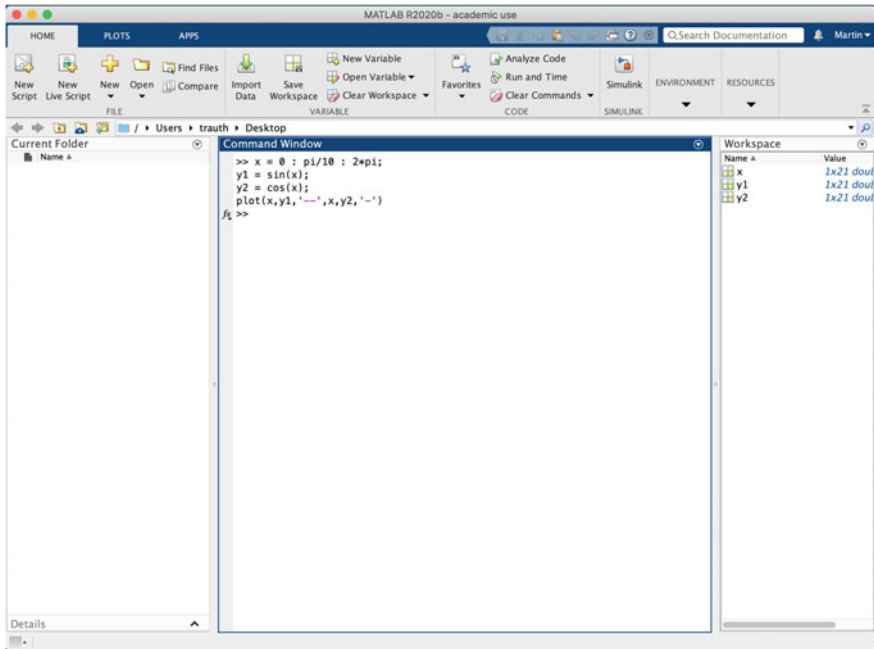


Fig. 2.1 Screenshot of the MATLAB default desktop layout including the *Current Folder* (left in the figure), the *Command Window* (center), and the *Workspace* (right) panels. This book uses only the Command Window and the built-in *Editor*, which can be called up by typing `edit` after the prompt. All information provided by the other panels can also be accessed through the Command Window.

in the operating system prompt, using `./matlab` instead of `matlab` on computers with the operating system macOS. If we launch the software with its desktop then it comes up with several window panels (Fig. 2.1). The default desktop layout includes the *Current Folder* panel that lists the files in the directory currently being used. The *Command Window* presents the interface between the software and the user, i.e. it accepts MATLAB commands typed after the prompt `>>`. The *Workspace* panel lists the variables in the MATLAB workspace, which is empty when starting a new software session. In this book we mainly use the Command Window and the built-in *Editor*, which can be launched by typing

`edit`

By default, the software stores all of your MATLAB-related files in the startup folder named *MATLAB*. Alternatively, you can create a personal working directory in which to store your MATLAB-related files. You should then make this new directory the working directory using the *Current Folder* panel or the *Folder Browser* at the top of the MATLAB desktop. The software uses a *Search Path* to find MATLAB-related files, which are organized in directories on the hard disk.

The default search path includes only the *MATLAB_R2020b* directory that has been created by the installer in the applications folder and the default working directory *MATLAB*. To see which directories are in the search path or to add new directories, select *Set Path* from the *Home* toolstrip of the MATLAB desktop, and use the *Set Path* dialog box. The modified search path is saved in the software preferences on your hard disk and the software will then in future direct MATLAB to use your custom path.

2.3 The Syntax

The name MATLAB stands for *matrix laboratory*. The classic object handled by MATLAB is a *matrix*, i.e. a rectangular two-dimensional *array* of numbers. A simple 1-by-1 array is a *scalar*. Arrays with one column or row are *vectors*, time series or other one-dimensional data fields. An *m*-by-*n* array can be used for a digital elevation model or a grayscale image. Red, green and blue (RGB) color images are usually stored as three-dimensional arrays, i.e. the colors red, green and blue are represented by an *m*-by-*n*-by-3 array.

Before proceeding, we need to clear the workspace by typing

```
clear
```

after the prompt in the Command Window. Clearing the workspace is always recommended before working on a new MATLAB project to avoid name conflicts with previous projects. We can also go a step further, close all Figure Windows using `close all` and clear the content of the Command Window using `clc`. It is therefore recommended that a new MATLAB project should always start with the line

```
clear, close all, clc
```

Entering matrices or arrays in MATLAB is easy. To enter an arbitrary array, type

```
A = [2 4 3 7; 9 3 -1 2; 1 9 3 7; 6 6 3 -2]
```

which first defines a variable *A*, then lists the elements of the array in square brackets. The rows of *A* are separated by semicolons, whereas the elements of a row are separated by blank spaces, or alternatively, by commas. After pressing *return*, MATLAB displays the array

```
A =  
    2    4    3    7  
    9    3   -1    2  
    1    9    3    7  
    6    6    3   -2
```

Displaying the elements of *A* could be problematic for very large arrays such as digital elevation models consisting of thousands or millions of elements. To suppress the display of an array or the result of an operation in general, the line should be ended with a semicolon.

```
A = [2 4 3 7; 9 3 -1 2; 1 9 3 7; 6 6 3 -2];
```

The array *A* is now stored in the workspace and we can carry out some basic operations with it, such as computing the sum of elements,

```
sum(A)
```

which results in the display

```
ans =  
    18    22     8    14
```

Since we did not specify an output variable, such as *A* for the array entered above, MATLAB uses a default variable *ans*, short for *answer* or *most recent answer*, to store the results of the calculation. In general, we should define variables since the next computation without a new variable name will overwrite the contents of *ans*.

The above example illustrates an important point about MATLAB: the software prefers to work with the columns of arrays. The four results of `sum(A)` are obviously the sums of the elements in each of the four columns of *A*. To sum all elements of *A* and store the result in a scalar *b*, we simply need to type

```
b = sum(sum(A));
```

which first sums the columns of the array and then the elements of the resulting vector. We now have two variables, *A* and *b*, stored in the workspace. We can easily check this by typing

```
whos
```

which is one the most frequently-used MATLAB commands. The software then lists all variables in the workspace, together with information about their sizes or dimensions, number of bytes, classes and attributes (see Sect. 2.5 for more details).

Name	Size	Bytes	Class	Attributes
A	4×4	128	double	
ans	1×4	32	double	
b	1×1	8	double	

Note that by default MATLAB is case sensitive, i.e. A and a can define two different variables. In this context, it is recommended that capital letters be used for arrays that have two dimensions or more and lower-case letters for one-dimensional arrays (or vectors) and for scalars. However, it is also common to use variables with mixed large and small letters. This is particularly important when using descriptive variable names, i.e. variables whose names contain information concerning their meaning or purpose, such as the variable `CatchmentSize`, rather than a single-character variable `a`. We could now delete the contents of the variable `ans` by typing

```
clear ans
```

Next, we will learn how specific array elements can be accessed or exchanged. Typing

```
A(3,2)
```

simply yields the array element located in the third row and second column, which is 9. The array indexing therefore follows the rule (*row, column*). We can use this to replace single or multiple array elements. As an example we type

```
A(3,2) = 30
```

to replace the element `A(3,2)` by 30 and to display the entire array.

```
A =
     2     4     3     7
     9     3    -1     2
     1    30     3     7
     6     6     3    -2
```

If we wish to replace several elements at one time, we can use the *colon operator*. Typing

```
A(3,1:4) = [1 3 3 5]
```

or

```
A(3,:) = [1 3 3 5]
```

replaces all elements of the third row of the array `A`. The colon operator also has several other uses in MATLAB, for instance as a shortcut for entering array elements such as

```
c = 0 : 10
```

which creates a vector, or a one-dimensional array with a single row, containing all integers from 0 to 10. The resultant MATLAB response is

```
c =  
    0    1    2    3    4    5    6    7    8    9   10
```

Note that this statement creates 11 elements, i.e. the integers from 1 to 10 and the zero. A common error when indexing arrays is to ignore the zero and therefore expect 10 elements instead of 11 in our example. We can check this from the output of `whos`.

Name	Size	Bytes	Class	Attributes
A	4×4	128	double	
ans	1×1	8	double	
b	1×1	8	double	
c	1×11	88	double	

The above command creates only integers, i.e. the interval between the array elements is one unit. However, an arbitrary interval can be defined, for example 0.5 units. This is later used to create evenly-spaced time vectors for time-series analysis. Typing

```
c = 1 : 0.5 : 10
```

results in the display

```
c =  
  
Columns 1 through 6  
  
    1.0000    1.5000    2.0000    2.5000    3.0000    3.5000  
  
Columns 7 through 12  
  
    4.0000    4.5000    5.0000    5.5000    6.0000    6.5000
```


Columns 13 through 18

```
7.0000    7.5000    8.0000    8.5000    9.0000    9.5000
```

Column 19

```
10.0000
```

which autowraps the lines that are longer than the width of the Command Window. The display of the values of a variable can be interrupted by pressing *Ctrl+C* (*Control+C*) on the keyboard. This interruption affects only the output in the Command Window, whereas the actual command is processed before displaying the result.

MATLAB provides standard arithmetic operators for addition, +, and subtraction, -. The asterisk, *, denotes matrix multiplication involving inner products between rows and columns. For instance, to multiply the matrix A by a new matrix B

```
B = [4 2 6 5; 7 8 5 6; 2 1 -8 -9; 3 1 2 3];
```

we type

```
C = A * B'
```

where ' is the complex conjugate transpose, which turns rows into columns and columns into rows. This generates the output

```
C =
    69    103   -79    37
    46     94    11    34
    53     76   -64    27
    44     93    12    24
```

In linear algebra, matrices are used to keep track of the coefficients of linear transformations. The multiplication of two matrices represents the combination of two linear transformations into a single transformation. Matrix multiplication is not commutative, i.e. $A*B'$ and $B*A'$ yield different results in most cases. Similarly, MATLAB allows matrix divisions representing different transformations, with / as the operator for right-hand matrix division and \ as the operator for left-hand division. Finally, the software also allows powers of matrices, ^.

In earth sciences, however, matrices are often simply used as two-dimensional arrays of numerical data rather than a matrix *sensu stricto* representing a linear transformation. Arithmetic operations on such arrays are carried out element-by-element. While this does not make any difference in addition and subtraction, it does affect multiplicative operations. MATLAB uses a dot, ., as part of the notation for these operations. As an example multiplying A and B element-by-element is performed by typing

```
C = A .* B
```

which generates the output

```
C =  
      8      8     18     35  
     63     24     -5     12  
      2      3    -24    -45  
     18      6      6     -6
```

2.4 Array Manipulation

MATLAB provides a wide range of functions with which to manipulate arrays (or matrices). This section introduces the most important functions for array manipulation, which are used later in the book. We first clear the workspace and create two arrays, A and B, by typing

```
clear, close all, clc  
  
A = [2 4 3; 9 3 -1]  
B = [1 9 3; 6 6 3]
```

which yields

```
A =  
      2      4      3  
      9      3     -1  
  
B =  
      1      9      3  
      6      6      3
```

When we work with arrays, we sometimes need to concatenate two or more arrays into a single array. We can use either `cat(dim,A,B)` with `dim=1` to concatenate the arrays A and B along the first dimension (i.e. along the rows). Alternatively, we can use the function `vertcat` to concatenate the arrays A and B vertically. By typing either

```
C = cat(1,A,B)
```

or

```
C = vertcat(A,B)
```

we obtain (in both cases)

C =

```

2   4   3
9   3  -1
1   9   3
6   6   3

```

Similarly, we can concatenate arrays horizontally, i.e. concatenate the arrays along the second dimension (i.e. along the columns) by typing

```
D = cat(2,A,B)
```

or using the function `horzcat` instead

```
D = horzcat(A,B)
```

which both yield

D =

```

2   4   3   1   9   3
9   3  -1   6   6   3

```

When working with satellite images we often concatenate three spectral bands of satellite images into three-dimensional arrays of the colors red, green and blue (RGB) (Sects. 2.5 and 8.4). We again use `cat(dim,A,B)` with `dim = 3` to concatenate the arrays A and B along the third dimension by typing

```
E = cat(3,A,B)
```

which yields

```
E(:,:,1) =
```

```

2   4   3
9   3  -1

```

```
E(:,:,2) =
```

```

1   9   3
6   6   3

```

Typing

```
whos
```

yields

Name	Size	Bytes	Class	Attributes
A	2×3	48	double	
B	2×3	48	double	
C	4×3	96	double	
D	2×6	96	double	
E	2×3×2	96	double	

indicating that we have now created a three-dimensional array, as the size 2-by-3-by-2 suggests. Alternatively, we can use

```
size(E)
```

which yields

```
ans =  
  
     2     3     2
```

to see that the array has 2 rows, 3 columns, and 2 layers in the third dimension. Using `length` instead of `size`,

```
length(A)
```

yields

```
ans =  
  
     3
```

which tells us the dimension of the largest array only. Hence `length` is normally used to determine the length of a one-dimensional array (or vector), such the evenly-spaced time axis `c` that was created in Sect. 2.3.

MATLAB uses a matrix-style indexing of arrays with the (1,1) element being located in the upper-left corner of arrays. Other types of data that are to be imported into MATLAB may follow a different indexing convention. As an example, digital terrain models (introduced in Sects. 7.3–7.5) often have a different way of indexing and therefore need to be flipped in an up-down direction or, in other words, about a horizontal axis. Alternatively, we can flip arrays in a left-right direction (i.e. about a vertical axis). We can do this by using `flipud` for flipping in an up-down direction and `fliplr` for flipping in a left-right direction

```
F = flipud(A)  
F = fliplr(A)
```

yielding

```
F =  
    9    3   -1  
    2    4    3
```

```
F =  
    3    4    2  
   -1    3    9
```

In more complex examples we can use `circshift(A,k,dim)` to circularly shift (i.e. rotate) arrays by `k` positions along the dimension `dim`. As an example we can shift the array `A` by one position along the 2nd dimension (i.e. along the rows) by typing

```
G = circshift(A,1,2)
```

which yields

```
G =  
    3    2    4  
   -1    9    3
```

We can also use `reshape(A,[m n])` to completely reshape the array. The result is an `m`-by-`n` array `H` whose elements are taken column-wise from `A`. As an example we create a 3-by-2 array from `A` by typing

```
H = reshape(A,[3 2])
```

which yields

```
H =  
    2    3  
    9    3  
    4   -1
```

Another important way to manipulate arrays is sorting their elements. As an example we can use `sort(C,dim,mode)` with `dim = 1` and `mode='ascend'` for sorting the elements of `C` in ascending order along the first array dimension (i.e. the rows). Typing

```
I = sort(C,1,'ascend')
```

yields

I =

```
1  3  -1
2  4   3
6  6   3
9  9   3
```

The function `sortrows(C,column)` with `column = 2` sorts the rows of `C` according to the second column. Typing

```
J = sortrows(C,2)
```

yields

J =

```
9  3  -1
2  4   3
6  6   3
1  9   3
```

Array manipulation also includes the comparison of arrays, for example by checking whether elements in `A(i,j)` are also found in `B` using `ismember`. Typing

A, B

```
K = ismember(A,B)
```

yields

A =

```
2  4   3
9  3  -1
```

B =

```
1  9   3
6  6   3
```

K =

2×3 logical array

```
0  0  1
1  1  0
```

K is a logical array, i.e. an array of logical values 0 (*false*) or 1 (*true*). The array $L(i,j)$ is zero if $A(i,j)$ is not in B , and one if $A(i,j)$ is in B . We can also locate elements within A for which a statement is true. For example we can locate elements with values less than zero and replace them with NaNs by typing

```
L = A;
L(find(L<0)) = NaN
```

or, more briefly

```
L(L<0) = NaN
```

which yields (in both cases)

```
L =
     2     4     3
     9     3    NaN
```

This is very useful when working with digital elevation models, where values below sea level are not relevant. Alternatively, we can replace data voids other than NaNs such as -32768, which are often used with digital terrain models (Sects. [7.3–7.5](#)). We can then determine which elements of an array are NaNs by typing

```
M = isnan(L)
```

which yields

```
M =
2×3 logical array
     0     0     0
     0     0     1
```

where NaNs are indicated by ones and non-NaN values are indicated by zeros. Which of the elements in array A are unique can be determined by typing

```
N = unique(A)
```

which yields

```
N =  
-1  
 2  
 3  
 4  
 9
```

The value of 3 occurs twice in A and the number of elements in N is therefore one less than in A.

2.5 Data Types in MATLAB

On a computer we describe numbers by a finite (e.g. 32 or 64) number of zeros and ones, the so-called binary system. The number of representable numbers is therefore finite, in contrast to real numbers. In a 64 bit system there are 2^{64} or $\sim 1.8447 \cdot 10^{19}$ different numbers. These *floating-point numbers* are arranged so that the density is greatest around zero and decreases in the direction of minus infinity and plus infinity to support a trade-off between precision and range.

To demonstrate the difference between real numbers and floating-point numbers, consider the value of $x = 4/3$ which you can write as 1.3333333... with an infinitely repeating decimal 3. Try 4/3 in MATLAB:

```
clear, close all, clc  
  
format long, 4/3
```

which yields

```
ans =  
  
1.333333333333333
```

The command `format long` changes the standard output format from a scaled fixed point format with 5 digits to 15 digits to display all available decimal digits for the double-precision floating-point format. We should change it back from `long` to `short` again later. As we can see MATLAB rounds the value of $4/3$ to the next floating-point number. Real numbers are continuous whereas floating-point numbers are not. The result of your calculation can therefore fall within the gap between two floating-point numbers; it is then rounded to the next floating-point number and therefore not exact.

Converting decimals to binaries (and back) is not difficult. As an example, let us convert 25.1 to a binary representation. This number has an integer part of 25 and a fractional part of 0.1 to be converted separately. First let us first convert the integer part:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
0	0	0	1	1	0	0	1	binary number
								(power of 2)
			16	8			1	= 25 decimal number
								(power of 10)

As we can see we divide the number 25 into a sum of powers of two instead of powers of ten in the decimal system. Alternatively you can calculate the binary representation by using

```

25/2 = 12 remainder 1
12/2 = 6 remainder 0
6/2 = 3 remainder 0
3/2 = 1 remainder 1
1/2 = 0 remainder 1 (results in a zero therefore stop)

```

The remainders are the binaries, which are arranged in reverse order so that the first remainder becomes the least significant digit (*LSD*) and the last remainder becomes the most significant digit (*MSD*), hence 11001 or, as a normalized floating-point number (mantissa between 1 and 2, e.g. 1.ffff), it is $1.1001 \cdot 2^4$.

Let us now convert 0.1 into a binary representation. Unfortunately, 0.1 is one of the many examples that has an infinitely repeating binary representation. This is because 0.1 lies between two floating-point numbers. To the base of 10 it is $1.0 \cdot 10^{-1}$, but to the base of 2 it is

.000110011001100110011001100110011001100110011...

which is calculated by multiplying 0.1 by two, taking the decimal as the digit, taking the fraction as the starting point for the next step, repeating until you either get to 0 or a periodic number, and then reading the binary number starting from the top:

```

0.1 * 2 = 0.2 -> 0
0.2 * 2 = 0.4 -> 0
0.4 * 2 = 0.8 -> 0
0.8 * 2 = 1.6 -> 1
0.6 * 2 = 1.2 -> 1
0.2 * 2 = 0.4 -> 0
0.4 * 2 = 0.8 -> 0
0.8 * 2 = 1.6 -> 1
0.6 * 2 = 1.2 -> 1

```

The result is .00011(0011) periodic, or as a normalized floating-point number it is $1.10011001\dots \cdot 2^{-4}$. Merging the integer with the fractional part we get

```
11001.000110011001100110011001100110011001100110011...
```

as the binary representation of 25.1. Here is a simple MATLAB script that does the same. First we create the decimal representation of the floating-point number:

```
clear
```

```
x = 25.1;
```

Then we separate the integer and fractional parts of 25.1 by typing

```
xi = round(25.1,0,'decimals');
xf = x - xi
```

Here we see some floating-point issues already, since the difference between 25.1 and 0.1 is not exactly 0.1 if we switch to `format long`:

```
xf =

    0.1000000000000001
```

We therefore use

```
xf = 0.1
```

instead. First, we convert the integer part,

```
i = 0;
bi = 0;
while xi ~= 0
    i = i + 1;
    xi = xi / 2;
    bi(i) = 2*(xi - floor(xi));
    xi = xi - 0.5*bi(i);
end
bi = fliplr(bi);
bistr = num2str(bi,'%u');
```

then the fractional part

```
i = 0;
bf = 0;
```

```

while xf ~= 0
    i = i + 1;
    xf = xf * 2;
    bf(i) = floor(xf);
    xf = xf - bf(i);
end
bfstr = num2str(bf, '%u');

```

before merging the two character strings into a single binary representation of 25.1

```
bstr = [bistr, '.', bfstr]
```

which yields

```

bstr =

'11001.0001100110011001100110011001100110011001100110011001101'

```

The second while loop should actually never stop since 0.1 does not have a finite binary representation. Because of the limited accuracy of floating-point numbers, however, `xf` indeed reaches zero and therefore the while loop does come to an end. Ready-to-use functions exist to do this conversion, such as the MATLAB `dec2bin` function, but this unfortunately only converts the integer part. Others can be found in the MATLAB File Exchange.

In MATLAB we use `eps(x)` to calculate the *floating-point relative accuracy*, in order to estimate the distance from `x` to the next largest double-precision number. As an example, you can calculate the distance from 1 to the next largest double-precision floating-point number by typing

```
eps(1)
```

which yields

```

ans =

2.220446049250313e-16

```

and to calculate the distance from 5 to the next largest floating-point number we type

```
eps(5)
```

which yields

```
ans =
```

```
8.881784197001252e-16
```

demonstrating the decreasing density of floating-point numbers with increasing distance from zero. The range of available floating-point numbers in a computer is limited to between `realmin` and `realmax`, the smallest and largest possible floating-point numbers. Values outside this range are displayed as `-Inf` and `+Inf`. Using

```
realmin
```

```
realmax
```

yields

```
ans =
```

```
2.225073858507201e-308
```

```
ans =
```

```
1.797693134862316e+308
```

There are problems associated with our work with floating-point numbers. Commutativity of addition [$a+b = b+a$] and multiplication [$a \cdot b = b \cdot a$] still holds, while associativity [$(a+(b+c)) = (a+b)+c$] does not, as we can see from the following example. Typing

```
(0.5 + 0.2) - 0.7
```

```
0.5 + (0.2 - 0.7)
```

yields

```
ans =
```

```
0
```

```
ans =
```

```
5.551115123125783e-17
```

but would be the same (zero) with real numbers. Distributivity [$a \cdot (b+c) = a \cdot b + a \cdot c$] is also violated, as the following example demonstrates. Typing

```
0.2 * (0.5 - 0.7) + 0.04
```

```
0.2 * 0.5 - 0.2 * 0.7 + 0.04
```

yields different values

```
ans =

    6.938893903907228e-18

ans =

    2.081668171172169e-17
```

whereas the correct value is again zero. Here is a slightly more complex example wherein b is halved at every step. With real numbers this would never stop but with floating-point numbers the halving stops when b gets smaller than the floating-point relative accuracy, which is then considered to be zero.

```
i = 1;
a = 1;
b = 1;
while a + b ~= a;
    b = b/2;
    i = i+1;
end
b
i
```

which yields

```
b =

    1.110223024625157e-16

i =

    54
```

suggesting that the while loop indeed terminates after 54 loops.

The following example helps to explain that the violation of associativity, combined with overflow (larger than `realmax`) or underflow (smaller than `realmin`), can really affect our work. The result of $((1+x)-1)/x$ should be 1, but typing

```
x = 1.e-15;
((1+x)-1)/x
```

yields

```
ans =

    1.110223024625157
```

i.e. ~ 0.11 or $\sim 11\%$ more than the correct value of one.

Let us now look at some examples of arrays in order to familiarize ourselves with the different data types in MATLAB. The default data type or *class* in MATLAB is *double precision* or *double*, which stores data in a 64-bit array of floating-point numbers. A double-precision array allows the sign of a number to be stored (bit 63), together with the exponent (bits 62 to 52), and roughly 16 significant decimal digits (bits 51 to 0). For the first example we create a 3-by-4 array of random numbers with double precision by typing

```
clear

format short

rng(0)
A = rand(3,4)
```

We use the function `rand` that generates uniformly distributed pseudorandom numbers within the open interval $[0,1]$. To obtain identical data values, we use `rng(0)` to reset the random number generator by using the integer 0 as *seed* (see Chap. 3 for more details on random number generators and types of distributions). Since we did not use a semicolon here we get the output

```
A =

    0.8147    0.9134    0.2785    0.9649
    0.9058    0.6324    0.5469    0.1576
    0.1270    0.0975    0.9575    0.9706
```

By default, the output is in a scaled fixed point format with 5 digits, e.g. 0.8147 for the (1,1) element of A. Typing

```
format long
```

switches to a fixed point format with 16 digits for double precision. Recalling A by typing

```
A
```

yields the output

```
A =

Columns 1 through 2

    0.814723686393179    0.913375856139019
    0.905791937075619    0.632359246225410
    0.126986816293506    0.097540404999410
```

Columns 3 through 4

```
0.278498218867048  0.964888535199277
0.546881519204984  0.157613081677548
0.957506835434298  0.970592781760616
```

which autowraps those lines that are longer than the width of the Command Window. The command `format` does not affect how the computations are carried out, i.e. the precision of the computation results remains unchanged. The precision is, however, affected by converting the data type from *double* to 32-bit *single precision*. Typing

```
B = single(A)
```

yields

```
B =
```

```
3×4 single matrix
```

```
0.8147237  0.9133759  0.2784982  0.9648885
0.9057919  0.6323593  0.5468815  0.1576131
0.1269868  0.0975404  0.9575068  0.9705928
```

Although we have switched to `format long`, only 8 digits are displayed. The command `whos` lists the variables `A` and `B` with information on their sizes or dimensions, number of bytes, and classes

Name	Size	Bytes	Class	Attributes
A	3×4	96	double	
B	3×4	48	single	

The default class `double` is used in all MATLAB operations in which the physical memory of the computer is not a limiting factor, whereas `single` is used when working with large data sets. The double-precision variable `A`, whose size is 3-by-4 elements, requires $3 \cdot 4 \cdot 64 = 768$ bits or $768/8 = 96$ bytes of memory, whereas `B` requires only 48 bytes and so has half the memory requirement of `A`. Introducing at least one complex number to `A` doubles the memory requirement since both real and imaginary parts are double precision, by default. Switching back to `format short` and typing

```
format short
A(1,3) = 4i + 3
```

yields

```
A =

Columns 1 through 2

    0.8147 + 0.0000i    0.9134 + 0.0000i
    0.9058 + 0.0000i    0.6324 + 0.0000i
    0.1270 + 0.0000i    0.0975 + 0.0000i

Columns 3 through 4

    3.0000 + 4.0000i    0.9649 + 0.0000i
    0.5469 + 0.0000i    0.1576 + 0.0000i
    0.9575 + 0.0000i    0.9706 + 0.0000i
```

and the variable listing is now

Name	Size	Bytes	Class	Attributes
A	3x4	192	double	complex
B	3x4	48	single	

indicating the class double and the attribute complex.

MATLAB also works with even smaller data types such as 1-bit, 8-bit and 16-bit data, in order to save memory. These data types are used to store digital elevation models or images (see Chaps. 7 and 8). For example m -by- n pixel RGB true color images are usually stored as three-dimensional arrays, i.e. the three colors are represented by an m -by- n -by-3 array (see Chap. 8 for more details on RGB composites and true color images). Such multi-dimensional arrays can be generated by concatenating three two-dimensional arrays representing the m -by- n pixels of an image. First, we generate a 100-by-100 array of uniformly distributed random numbers in the range [0,1]. We then multiply the random numbers by 255 to get values between 0 and 255.

```
clear

rng(0)
I1 = 255 * rand(100,100);
I2 = 255 * rand(100,100);
I3 = 255 * rand(100,100);
```


The command `cat` concatenates the three two-dimensional arrays (8 bits each) into a three-dimensional array ($3 \cdot 8 \text{ bits} = 24 \text{ bits}$).

```
I = cat(3,I1,I2,I3);
```

Since RGB images are represented by integer values between 0 and 255 for each color, we convert the 64-bit double-precision values to unsigned 8-bit integers using `uint8` (Sect. 8.2). The function `uint8` rounds the values in `I` to the nearest integer. Any values that are outside the range `[0,255]` are assigned to the nearest endpoint (0 or 255).

```
I = uint8(I);
```

Typing `whos` then yields

Name	Size	Bytes	Class	Attributes
I	100x100x3	30000	uint8	
I1	100x100	80000	double	
I2	100x100	80000	double	
I3	100x100	80000	double	

Since 8 bits can be used to store 256 different values, this data type can be used to store integer values between 0 and 255, whereas using `int8` to create signed 8-bit integers generates values between -128 and $+127$. The value of zero requires one bit and there is therefore no space left in which to store $+128$. Finally, `imshow` can be used to display the three-dimensional array as a true color image.

```
imshow(I)
```

In the following chapters we will often have to save character strings (for example location or sample names) in arrays. MATLAB provides a number of ways for storing text data, for example in character arrays and string arrays. The older of the two data types is a *character array*. We can as an example create such an array named `location_1` containing the districts of northern Potsdam. Typing

```
clear

locations_1 = ...
    ['Bornim';...
    'Nedlitz';...
    'Bornstedt';...
    'Sacrow';...
    'Eiche';...
    'Grube';...
    'Golm']
```

causes the error message

```
Error using vertcat
Dimensions of arrays being concatenated are not consistent.
```

In this data type the number of characters in each line must be the same, a clear disadvantage of character arrays. Since the district names have a different number of letters, we have to fill each character string to the same number of letters using spaces by typing

```
locations_1 = ...
    ['Bornim   ';...
     'Nedlitz  ';...
     'Bornstedt';...
     'Sacrow   ';...
     'Eiche    ';...
     'Grube    ';...
     'Golm     ']
```

Since the MATLAB R2016b release we can also store multiple lines of text in *string arrays*. The sequences of characters can then have different lengths without having to pad with spaces. The difference from character arrays is the use of double quotes to enclose the pieces of text.

```
locations_2 = ...
    ['Bornim';...
     'Nedlitz';...
     'Bornstedt';...
     'Sacrow';...
     'Eiche';...
     'Grube';...
     'Golm']
```

Typing `whos` shows the different appearances of the two types of arrays in the workspace:

Name	Size	Bytes	Class	Attributes
locations_1	7×9	126	char	
locations_2	7×1	460	string	

As we can see the size of the character array `location_1` corresponds to the number of district names times the number of characters in each individual name. Character

arrays are 16-bit arrays, i.e. $2^{16} = 65,536$ different characters can be stored in each element of such an array. The character string `location_1` therefore requires $7 \cdot 9 \cdot 16 = 1,008$ bits or $1,008/8 = 126$ bytes of memory. Things are a little more complicated for string arrays. The characters are still stored as 16-bit characters but each string array has a header, followed by more headers for each element in the array, followed by a buffer with the actual character data. The buffer is ultimately used to accommodate the different lengths of character strings, but depends on the platform used and may change in later versions of MATLAB.

We next look at the possibilities that MATLAB offers for storing mixed data types in a single array. The oldest of these data types are *structure arrays*, which are multi-dimensional arrays with elements accessed by textual field designators. These arrays are data containers that are particularly useful in storing any kind of information about a sample in a single variable. As an example we can generate a structure array `sample_1` that includes the image array `I` defined in the previous example

```
clear

rng(0)
I = uint8(255*rand(100,100,3));
```

as well as other types of information about a sample, such as the name of the sampling location, the date of sampling, and geochemical measurements, stored in a 10-by-10 array.

```
sample_1.location = 'Plougasnou';
sample_1.date = date;
sample_1.image = I;
sample_1.geochemistry = rand(10,10);
```

The first layer location of the structure array `sample_1` contains a character array with the location name `Plougasnou`. The second layer `date` contains a character string generated by the function `date` that yields a string containing the current date in `dd-mm-yyyy` format. We access this particular layer in `sample_1` by typing

```
sample_1.date
```

which yields

```
ans =

    '07-Oct-2019'
```

as an example. The third layer of `sample_1` contains the image, while the fourth layer contains a 10-by-10 array of uniformly-distributed pseudorandom numbers. All layers of `sample_1` can be listed by typing

```
sample_1
```

resulting in the output

```
sample_1 =

    struct with fields:

        location: 'Plougasnou'
         date: '07-Oct-2019'
        image: [100×100×3 uint8]
    geochemistry: [10×10 double]
```

This represents a list of the layers `location`, `date`, `image` and `geochemistry` within the structure array `sample_1`. Some variables are listed in full, whereas larger data arrays are only represented by their size. In the list of the layers within the structure array `sample_1`, the array `image` is characterized by its size `100×100×3` and the class `uint8`. The variable `geochemistry` in the last layer of the structure array contains a 10-by-10 array of double-precision numbers. The command

```
whos sample_1
```

does not list the layers in `sample_1` but the name of the variable, the number of bytes and the class `struct` of the variable.

```
Name      Size  Bytes  Class  Attributes

sample_1  1×1   31546   struct
```

MATLAB also has *cell arrays* as an alternative to structure arrays. Both types of arrays are very similar and contain different types and sizes of data. The most important difference between the two is that a structure array uses *named fields*, whereas a cell array uses *numerically-indexed cells*. Structure arrays are often used in applications where the organization of the data is particularly important. Cell arrays are often used when processing large data sets in count-controlled loops (Sect. 2.7).

As an example of cell arrays we use the same data collection as in structure arrays, with the layers of the structure array as the cells in the cell array. The cell array is created by enclosing the location name *Plougasnou*, the date, the image I

and the 10-by-10 array of uniformly-distributed pseudorandom numbers in curly brackets.

```
C = {'Plougasnou' date I rand(10,10)}
```

Typing

```
C
```

lists the contents of the cell array

```
C =  
  
1×4 cell array  
  
Columns 1 through 2  
  
{'Plougasnou'} {'07-Oct-2019'}  
  
Columns 3 through 4  
  
{100×100×3 uint8} {10×10 double}
```

which contains the location name and date. The image and the array of random numbers are too large to be displayed in the Command Window, but the dimensions and class of the data are displayed instead. We access a particular cell in C, e.g. the cell 2, by typing

```
C{2}
```

which yields

```
ans =  
  
'07-Oct-2019'
```

We can also access the other cells of the cell array in a similar manner.

Another very convenient way to store mixed data types in a single array involves the use of *table arrays*. This data type was introduced with the MATLAB R2013b release and is very useful when reading large datasets (see Sect. 2.6). Table arrays store column-oriented or tabular data, with named variables stored in individual columns. We can use this type of data to allocate space for variables whose values will be filled in later. As an example, we create a simple table with two columns. The first column again contains the names of the districts of northern Potsdam:

```
clear

Districts = ...
    {'Bornim';...
    'Nedlitz';...
    'Bornstedt';...
    'Sacrow';...
    'Eiche';...
    'Grube';...
    'Golm'};
```

The second column contains the number of inhabitants in December 2018 (source <https://www.potsdam.de/>).

```
Inhabitants = [3429
               192
               13895
               140
               5308
               433
               4310];
```

The function `table` then concatenates the two columns to a single table

```
potsdamnorth = table(Districts,Inhabitants)
```

which yields

```
potsdamnorth =
7x2 table

    Districts    Inhabitants
    _____    _____
    {'Bornim'   }    3429
    {'Nedlitz'  }    192
    {'Bornstedt'}    13895
    {'Sacrow'   }    140
    {'Eiche'    }    5308
    {'Grube'    }    433
    {'Golm'     }    4310
```

As we can see the heterogeneous data are collected in single array and displayed as a table. This data type is very useful when working with experimental data, e.g. large data arrays with sample IDs that consist of combinations of letters with both integer and floating-point numbers.

2.6 Data Storage and Handling

This section deals with how to store, import, and export data with MATLAB. Many of the data formats typically used in earth sciences have to be converted before being analyzed with MATLAB. Alternatively, the software provides several import routines to read many binary data formats in earth sciences, such as those used to store digital elevation models and satellite data. Newer functions in MATLAB are capable of importing mixed data types such as text, integers, and floating-point numbers, from a single file into the workspace.

A computer generally stores data as *binary digits* or *bits*. A bit is analogous to a two-way switch with two states, *on* = 1 and *off* = 0. The bits are joined together to form larger groups, such as *bytes* consisting of 8 bits, in order to store more complex types of data. Such groups of bits are then used to encode data, e.g. numbers or characters. Unfortunately, different computer systems and software use different schemes for encoding data. Exchanging binary data between different computer platforms and software can therefore be difficult. Binary data can be stored in relatively small files if both partners are using similar systems of data exchange. The transfer rate for binary data is generally faster than that for the exchange of other file formats.

Various formats for exchanging data have been developed during recent decades. The classic example for the establishment of a data format that can be used with different computer platforms and software is the American Standard Code for Information Interchange (ASCII) that was first published in 1963 by the American Standards Association (ASA). As a 7-bit code, ASCII consists of $2^7 = 128$ characters (codes 0 to 127). Whereas ASCII-1963 was lacking lower-case letters, in the ASCII-1967 update lower-case letters as well as various control characters such as escape and line feed, and various symbols such as brackets and mathematical operators, were also included. Since then, a number of variants appeared in order to facilitate the exchange of text written in non-English languages, such as the expanded ASCII containing 255 codes, e.g. the Latin-1 encoding.

The simplest way to exchange data between a certain piece of software and MATLAB is using the ASCII format. Although the newer versions of MATLAB provide various import routines for file types such as Microsoft Excel binaries, most data arrive in the form of ASCII files. Consider a simple data set stored in a table such as

SampleID	Percent C	Percent S
101	0.3657	0.0636
102	0.2208	0.1135
103	0.5353	0.5191
104	0.5009	0.5216
105	0.5415	-999
106	0.501	-999

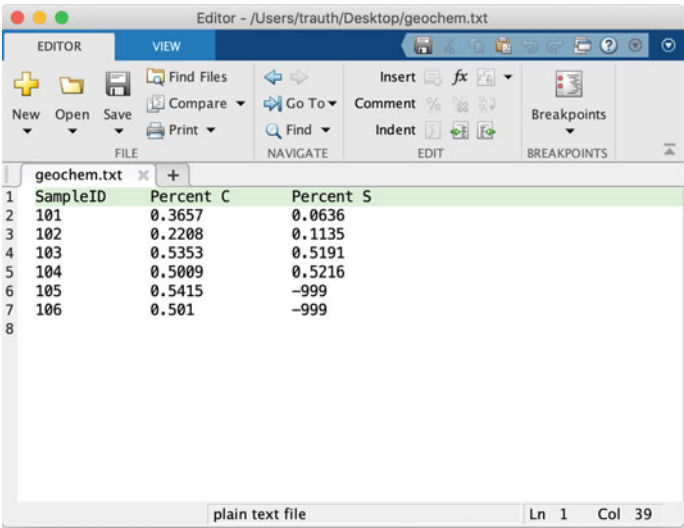


Fig. 2.2 Screenshot of MATLAB Editor showing the content of the file `geochem.txt`. The first line of the text needs to be commented by a percent sign at the beginning of the line, followed by the actual data array. The -999 values need to be replaced by NaNs.

The first row contains the names of the variables and the columns provide the percentages of carbon and sulfur in each sample. The absurd value -999 indicates missing data in the data set. Two things have to be changed to convert this table into MATLAB format. First, MATLAB uses NaN as the representation for *Not-a-Number* that can be used to mark missing data or gaps. Second, a percent sign, %, should be added at the beginning of the first line. The percent sign is used to indicate nonexecutable text within the body of a program. This text is normally used to include comments in the code.

%SampleID	Percent C	Percent S
101	0.3657	0.0636
102	0.2208	0.1135
103	0.5353	0.5191
104	0.5009	0.5216
105	0.5415	NaN
106	0.501	NaN

MATLAB will ignore any text appearing after the percent sign and continue processing on the next line. After editing this table in a text editor, such as the *MATLAB Editor*, it can be saved as ASCII text file `geochem.txt` in the current working directory (Fig. 2.2). The MATLAB workspace should first be cleared by typing


```
clear, close all, clc
```

after the prompt in the Command Window. MATLAB can now import the data from this file with the `load` command.

```
load geochem.txt
```

MATLAB then loads the contents of the file and assigns the array to a variable `geochem` specified by the filename *geochem.txt*. Typing

```
whos
```

yields

Name	Size	Bytes	Class	Attributes
geochem	6x3	144	double	

The command `save` now allows workspace variables to be stored in a binary format.

```
save geochem_new.mat
```

MAT-files are double-precision binary files using *.mat* as extension. The advantage of these binary MAT-files is that they are independent of the computer platforms running different floating-point formats. The command

```
save geochem_new.mat geochem
```

saves only the variable `geochem` instead of the entire workspace. The option `-ascii`, for example

```
save geochem_new.txt geochem -ascii
```

again saves the variable `geochem`, but in an ASCII file named *geochem_new.txt* in a floating-point format with 8 digits:

1.0100000e+02	3.6570000e-01	6.3600000e-02
1.0200000e+02	2.2080000e-01	1.1350000e-01
1.0300000e+02	5.3530000e-01	5.1910000e-01
1.0400000e+02	5.0090000e-01	5.2160000e-01
1.0500000e+02	5.4150000e-01	NaN
1.0600000e+02	5.0100000e-01	NaN

In contrast to the binary file *geochem_new.mat*, this ASCII file can be viewed and edited using the MATLAB Editor or any other text editor.

Such data files, especially those that are produced by electronic instruments, can look much more complicated than the example file *geochem.txt* with a single header line. In Chaps. 7 and 8 we will read some of these complicated and extensive files, which are either binary or text files and usually have long headers describing the contents of the files. At this point, let us have a look at a variant of text files that contains not only one or more header lines but also unusual data types such as date and time, in a non-decimal format. We use the function `textscan` to perform this task. The MATLAB workspace should first be cleared by typing

```
clear
```

after the prompt in the Command Window. MATLAB can now import the data from the file *geochem.txt* using the `textscan` command.

```
fid = fopen('geochem.txt');
C = textscan(fid, '%u %f %f', ...
    'Headerlines', 1, ...
    'CollectOutput', 1);
fclose(fid);
```

This script opens the file *geochem.txt* for *read only* access using `fopen` and defines the file identifier `fid`, which is then used to read the text from the file using `textscan` and to write it into the cell array `C`. The character string `%u %f %f` defines the conversion specifiers enclosed in single quotation marks, where `%u` stands for the 32-bit unsigned integer output class and `%f` stands for a 64-bit double-precision floating-point number. The parameter `Headerlines` is set to 1, which means that a single header line is ignored while reading the file. If the parameter `CollectOutput` is 1 (i.e. is true), `textscan` concatenates output cells with the same data type into a single array. The function `fclose` closes the file defined by `fid`. The array `C` is a cell array, which is a data type with indexed containers called cells (see Sect. 2.5). The advantage of this data type is that it can store data of various types and sizes, such as character strings, double-precision numbers, and images in a single variable such as `C`. Typing

```
C
```

yields

```
C =
1×2 cell array
{6×1 uint32} {6×2 double}
```

indicating that `C` contains a 6-by-1 32-bit unsigned integer array, which is the sample ID, and a 6-by-1 double-precision array, which represents the percentages of carbon and sulfur in each sample. We can access the contents of the cells in `C` by typing

```
data1 = C{1}
data2 = C{2}
```

which yields

```
data1 =

6×1 uint32 column vector

    101
    102
    103
    104
    105
    106

data2 =

    0.3657    0.0636
    0.2208    0.1135
    0.5353    0.5191
    0.5009    0.5216
    0.5415         NaN
    0.5010         NaN
```

We now concatenate the two cells into one double-precision array `data`. First, we have to change the class of `C{1}` into `double` or the class of the entire array `data` will be `uint32`. Typing

```
data(:,1) = double(C{1})
data(:,2:3) = C{2}
```

yields

```
data =

    101.0000    0.3657    0.0636
    102.0000    0.2208    0.1135
    103.0000    0.5353    0.5191
    104.0000    0.5009    0.5216
    105.0000    0.5415         NaN
    106.0000    0.5010         NaN
```

The format of the data is as expected.

The next examples demonstrate how to read the file *geophys.txt*, which contains a single header line but also the date (in an *MM/DD/YY* format) and time (in an *HH:MM:SS.SS* format). We again use `textscan` to read the file,

```
clear

fid = fopen('geophys.txt');
data = textscan(fid, '%u %f %f %f %s %s', ...
    'Headerlines', 1);
fclose(fid);
```

where we skip the header, read the first column (the sample ID) as a 32-bit unsigned integer (`uint32`) with specifier `%u`, the next three columns *X*, *Y*, and *Z* as 64-bit double-precision floating-point numbers (`double`) with specifier `%f`, and then the date and time as character strings with specifier `%s`. We then convert the date and time to serial numbers, where a serial date number of 1 corresponds to *Jan-1-0000*. The year *0000* is merely a reference point and is not intended to be interpreted as a real year.

```
data_date_serial = datenum(data{5});
data_time_serial = datenum(data{6});
```

Finally, we can convert the date and time serial numbers into a data and time array by typing

```
data_date = datevec(data_date_serial)
data_time = datevec(data_time_serial)
```

which yields

```
data_date =

    2013    11    18     0     0     0
    2013    11    18     0     0     0
    2013    11    18     0     0     0
    2013    11    18     0     0     0
    2013    11    18     0     0     0

data_time =

    1.0e+03 *

    2.0130    0.0010    0.0010    0.0100    0.0230    0.0091
    2.0130    0.0010    0.0010    0.0100    0.0230    0.0102
```

```

2.0130  0.0010  0.0010  0.0100  0.0230  0.0504
2.0130  0.0010  0.0010  0.0100  0.0240  0.0051
2.0130  0.0010  0.0010  0.0100  0.0240  0.0233

```

The first three columns of `data_date` contain the year, month and day. The fourth to sixth columns of `data_time` contain the hour, minute and second.

We can also write data to a formatted text file using `fprintf`. As an example we again load the data from `geochem.txt` after we have commented out the first line and have replaced -999 with NaN. Instead of using `load geochem.txt`, we can type

```

clear

data = load('geochem.txt');

```

to load the contents of the text file into a double-precision array `data`. We write the data to a new text file `geochem_formatted.txt` using `fprintf`. Since the function `fprintf` writes all elements of the array `data` to the file in column order we need to transpose the data before we save it.

```

data = data';

```

We first open the file using the permission `w` for *writing*, and discard the existing contents. We then write `data` to this file using the formatting operators `%u` for unsigned integers and `%6.4f` for fixed-point numbers with a field width of six characters and four digits after the decimal point. The control character `\n` denotes a new line after each line of three numbers.

```

fid = fopen('geochem_formatted.txt','w');
fprintf(fid,'%u %6.4f %6.4f\n',data);
fclose(fid);

```

We can view the contents of the file by typing

```

edit geochem_formatted.txt

```

which opens the file `geochem_formatted.txt`

```

101  0.3657  0.0636
102  0.2208  0.1135
103  0.5353  0.5191
104  0.5009  0.5216
105  0.5415    NaN
106  0.5010    NaN

```

in the MATLAB Editor. The format of the data is as expected.

The function `datastore` can be used to handle large collections of data with mixed data types. As an example, the modern multi-sensor core loggers (MSCL) used for automatic determination of physical and chemical properties of drill cores generates vast quantities of data within a very short time. An array with 80 variables, each with 50,000 measured values, is not unusual. In addition, the corresponding files contain mixtures of character strings for core section names, measurement dates and times, and comments from the laboratory technician who carried out the measurements, together with the actual numeric measurements recorded.

As an example we import the content of the file *geoxrf.txt* into the MATLAB like this in the editor:

Location	Core	Depth	K	Si
ChewBahir	2A	0.30	975	1576
ChewBahir	2A	0.60	2785	9706
ChewBahir	2A	0.90	5469	9572
ChewBahir	2A	0:12	9575	4554
ChewBahir	2A	0:15	9649	8003

As we see the first line contains the header of the data. The first and second columns contain the location name and the name of the drill core. The third column contains the depth in the core. The fourth and fifth columns contain numerical values representing X-ray fluorescence counts for potassium and silicon. The columns are separated by one or more spaces. We use the function `datastore` to create the datastore `data` without actually importing the data.

```
clear

data = datastore('geoxrf.txt');
```

Typing

```
data
```

yields a list of properties of the data. We can read all of the data from the datastore using

```
dataxrf = readall(data)
```

which yields

```
dataxrf =
```

```
5×5 table
```

Location	Core	Depth	K	Si
{ 'ChewBahir' }	{ '2A' }	0.3	975	1576
{ 'ChewBahir' }	{ '2A' }	0.6	2785	9706
{ 'ChewBahir' }	{ '2A' }	0.9	5469	9572
{ 'ChewBahir' }	{ '2A' }	0.12	9575	4854
{ 'ChewBahir' }	{ '2A' }	0.15	9649	8003

If the file is too large to be loaded into memory, we can import only part of it. First we define the variables Depth and K that we want to load

```
data.SelectedVariableNames = {'Depth','K'}
```

and then we use `readall` again by typing

```
dataxrf = readall(data)
```

which yields

```
dataxrf =
```

```
5×2 table
```

Depth	K
0.3	975
0.6	2785
0.9	5469
0.12	9575
0.15	9649

Now we can use the properties from tables and their associated functions to convert the data into simple double arrays, e.g. by typing

```
depth = dataxrf.Depth
potassium = dataxrf.K
```

which yields

```
depth =
```

```
0.3000
0.6000
0.9000
0.1200
0.1500
```

```
potassium =
```

```
975
2785
5469
9575
9649
```

Alternatively, we can use the function `table2array` to convert the table to a double precision array by typing

```
dataxrfarray = table2array(dataxrf)
```

which yields

```
dataxrfarray =
```

```
1.0e+03 *
0.0003  0.9750
0.0006  2.7850
0.0009  5.4690
0.0001  9.5750
0.0001  9.6490
```

2.7 Control Flow

Control flow in computer science helps to control the order in which computer code is evaluated. The most important kinds of control flow statements are count-controlled loops such as `for` loops and conditional statements such as `if-then` constructs. Since in this book we do not deal with the programming capabilities of MATLAB in any depth, the following introduction to the basics of control flow is rather brief and omits certain important aspects of efficient programming, such as the pre-allocation of

memory prior to using for loops, and instructions on how the use of for loops can be avoided by vectorization of the MATLAB code. This introduction is instead limited to the two most important kinds of control flow statements: the aforementioned for loops and the if-then constructs. Readers interested in MATLAB as a programming environment are advised to read the more detailed chapters on control flow in the MATLAB documentation (MathWorks 2020a, c).

The for loops, as the first example of a MATLAB language statement, execute a series of commands between for end end a specified number of times. As an example we use such a loop to multiply the elements of an array A by 10, round the result to the nearest integer, and store the result in B.

```
clear, close all, clc

rng(0)
A = rand(10,1)
for i = 1 : 10
    B(i,1) = round(10 * A(i));
end
B
```

which yields

```
A =

    0.8147
    0.9058
    0.1270
    0.9134
    0.6324
    0.0975
    0.2785
    0.5469
    0.9575
    0.9649
```

```
B =
```

```
8
9
1
9
6
1
3
```

```

5
10
10

```

The result is as expected. We can expand the experiment by using a nested for loop to create a 2D array B.

```

rng(0)
A = rand(10,3)
for i = 1 : 10
    for j = 1 : 3
        B(i,j) = round(10 * A(i,j));
    end
end
B

```

which yields

```

A =

    0.8147    0.1576    0.6557
    0.9058    0.9706    0.0357
    0.1270    0.9572    0.8491
    0.9134    0.4854    0.9340
    0.6324    0.8003    0.6787
    0.0975    0.1419    0.7577
    0.2785    0.4218    0.7431
    0.5469    0.9157    0.3922
    0.9575    0.7922    0.6555
    0.9649    0.9595    0.1712

```

```

B =

     8     2     7
     9    10     0
     1    10     8
     9     5     9
     6     8     7
     1     1     8
     3     4     7
     5     9     4
    10     8     7
    10    10     2

```

This book tries to make all of the recipes independent of the actual dimensions of the data. This is achieved by the consistent use of `size` and `length` to determine the size of the data instead of using fixed numbers such as the 10 and 3 in the above example (Sect. 2.4).

```
rng(0)
A = rand(10,3)
for i = 1 : size(A,1)
    for j = 1 : size(A,2)
        B(i,j) = round(10 * A(i,j));
    end
end
B
```

When working with larger data sets with many variables one might occasionally wish to automate array manipulations such as those described in Sect. 2.4. Let us assume, for example, that we want to replace all NaNs in all variables in the memory with -999. We first create a collection of four variables, each of which contains a single NaN.

```
clear

rng(0)
A = rand(3,3); A(2,1) = NaN
BC = rand(2,4); BC(2,2) = NaN
DE = rand(1,2); DE(1,1) = NaN
FG = rand(3,2); FG(2,2) = NaN
```

We list the variables in the workspace using `whos` and store this list in `variables`.

```
variables = whos;
```

We then use a `for` loop to store the content of each variable in `v` using `eval` and then locate the NaNs in `v` using `isnan` (Sect. 2.4) and replace them with -999. The function `eval` executes a MATLAB expression stored in a text string. We assign the value of `v` to the variable in the base workspace and then clear the variables `i`, `v` and `variables`, which are no longer needed.

```
for i = 1 : size(variables,1)
    v = eval(variables{i});
    v(isnan(v)==1) = -999;
    assignin('base',variables{i},v);
    eval(variables{i})
end

clear i v variables
```

Comparing the variables before and after the replacement of the NaNs with -999 reveals that the script works well and that we have successfully manipulated our data.

The second important statements to control the flow of a script (apart from for loops) are if-then constructs, which evaluate an expression and then execute a group of instructions if the expression is true. As an example we compare the value of two scalars A and B.

```
clear

A = 1
B = 2
if A < B
    disp('A is less than B')
end
```

which yields

```
A is less than B
```

The script first evaluates whether A is less than B and, if it is, displays the message A is less than B in the Command Window. We can expand the if-then construct by introducing else, which provides an alternative statement if the expression is not true.

```
A = 1
B = 2
if A < B
    disp('A is less than B')
else
    disp('A is not less than B')
end
```

which yields

A is less than B

Alternatively, we can use `elseif` to introduce a second expression to be evaluated.

```
A = 1
B = 2
if A < B
    disp('A is less than B')
elseif A >= B
    disp('A is not less than B')
end
```

The `for` loops and `if-then` constructs are extensively used in the following chapters of the book. For other aspects of programming, please refer to the MATLAB documentation (MathWorks [2020a](#), [c](#)).

2.8 Scripts and Functions

MATLAB is a powerful programming language. All files containing MATLAB code use `.m` as an extension and are therefore called *M-files*. These files contain ASCII text and can be edited using a standard text editor. However, the built-in Editor color-highlights various syntax elements such as comments in green, keywords such as *if*, *for* and *end* in blue, and character strings in purple. This syntax highlighting facilitates MATLAB coding.

MATLAB uses two types of M-files: scripts and functions. Whereas scripts are a series of commands that operate on data in the workspace, functions are true algorithms with input and output variables. The advantages and disadvantages of both types of M-file will now be illustrated by an example. We first start the Editor by typing

```
edit
```

This opens a new window named *untitled*. Next, we generate a simple MATLAB script by typing a series of commands to calculate the average of the elements of a data array `x`.

```
[m,n] = size(x);
if m == 1
    m = n;
end
sum(x)/m
```

The first line of the `if-then` construct yields the dimensions of the variable `x` using the command `size`. In our example `x` should be either a column vector, i.e. an array with a single column and dimensions $(m,1)$, or a row vector, i.e. an array with a single row and dimensions $(1,n)$. The `if` statement evaluates a logical expression and executes a group of commands if this expression is true. The `end` keyword terminates the last group of commands. In the example the `if-then` construct picks either `m` or `n`, depending on whether `m==1` is false or true. Here, the double equal sign `==` makes element by element comparisons between the variables (or numbers) to the left and right of the equal signs and returns an array of the same size, made up of elements set to logical 1 where the relationship is true and to logical 0 where it is not true. In our example `m==1` returns 1 if `m` equals 1 and 0 if `m` equals any other value. The last line of the `if-then` construct computes the average by dividing the sum of elements by `m` or `n`. We do not use a semicolon here in order to allow the output of the result. We can now save our new M-file as *average.m* and type

```
clear, close all, clc
```

```
x = [3 6 2 -3 8];
```

in the Command Window to define an example array `x`. We then type

```
average
```

without the extension *.m* to run our script and obtain the average of the elements of the array `x` as output.

```
ans =
```

```
3.2000
```

After typing

```
whos
```

we see that the workspace now contains

Name	Size	Bytes	Class	Attributes
ans	1x1	8	double	
m	1x1	8	double	
n	1x1	8	double	
x	1x5	40	double	

The listed variables are the example array `x` and the output of the function `size`, `m` and `n`. The result of the operation is stored in the variable `ans`. Since the default variable `ans` might be overwritten during one of the succeeding operations, we need to define a different variable. Typing

```
a = average
```

however, results in the error message

```
Attempt to execute SCRIPT average as a function:  
/Users/trauth/Desktop/average.m
```

We can obviously not assign a variable to the output of a script. Moreover, all variables defined and used in the script appear in the workspace; in our example these are the variables `m` and `n`. Scripts contain sequences of commands that are applied to variables in the workspace. MATLAB functions, however, allow inputs and outputs to be defined. They do not automatically import variables from the workspace. To convert the above script into a function we need to introduce the following modifications (Fig. 2.3):

```
function y = average(x)  
%AVERAGE Average value.  
% AVERAGE(X) is the average of the elements in the array X.  
  
% By Martin Trauth, June 27, 2014  
  
[m,n] = size(x);  
if m == 1  
    m = n;  
end  
y = sum(x)/m;
```

The first line now contains the keyword `function`, the function name `average`, the input `x` and the output `y`. The next two lines contain comments, as indicated by the percent sign, separated by an empty line. The second comment line contains the author's name and the version of the M-file. The rest of the file contains the actual operations. The last line now defines the value of the output variable `y`, and this line is terminated by a semicolon to suppress the display of the result in the Command Window. Next we type

```
help average
```

which displays the first block of contiguous comment lines. The first executable statement (or blank line in our example) effectively ends the help section and therefore the output of `help`. Now we are independent of the variable names used in our function. The workspace can now be cleared and a new data vector defined.

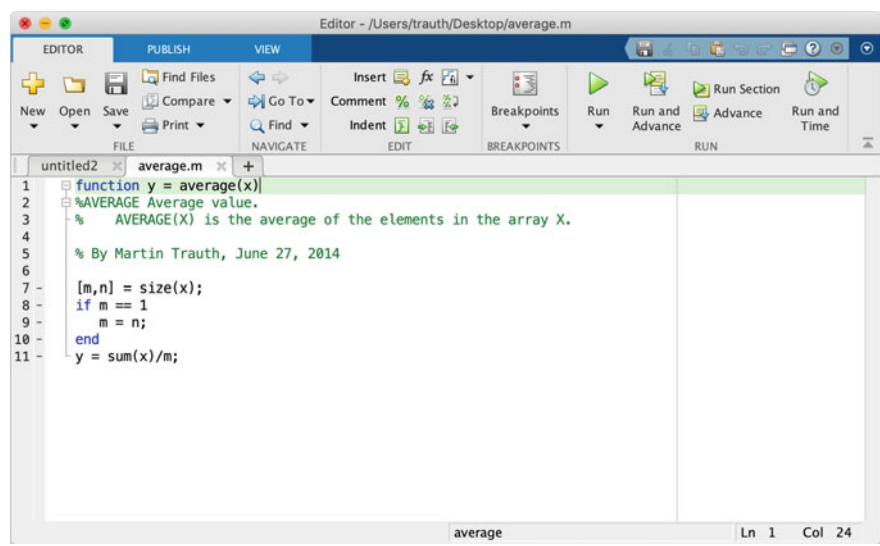


Fig. 2.3 Screenshot of the MATLAB *Editor* showing the function `average`. The function starts with a line containing the keyword `function`, the name of the function `average`, the input variable `x`, and the output variable `y`. The subsequent lines contain the output for `help average`, the copyright and version information, and also the actual MATLAB code for computing the average using this function.

```
clear

data = [3 6 2 -3 8];
```

Our function can then be run by the statement

```
result = average(data);
```

This clearly illustrates the advantages of functions compared to scripts. Typing

```
whos

results in
```

Name	Size	Bytes	Class	Attributes
data	1×5	40	double	
result	1×1	8	double	

revealing that all variables used in the function do not appear in the workspace. Only the input and output as defined by the user are stored in the workspace. The M-files can therefore be applied to data as if they were real functions, whereas

scripts contain sequences of commands that are applied to the variables in the workspace. If we want variables such as `m` and `n` to also appear in the memory they must be defined as *global* variables in both the function and the workspace, otherwise they are considered to be *local* variables. We therefore add one line to the function `average` with the command `global`:

```
function y = average(x)
% AVERAGE Average value.
% AVERAGE(X) is the average of the elements in the array X.

% By Martin Trauth, June 27, 2014

global m n
[m,n] = size(x);
if m == 1
    m = n;
end
y = sum(x)/m;
```

We now type

```
global m n
```

in the Command Window. After running the function as described in the previous example we find the two variables `m` and `n` in the workspace. We have therefore transferred the variables `m` and `n` between the function `average` and the workspace.

2.9 Basic Visualization Tools

MATLAB provides numerous routines for displaying data as graphics. This section introduces the most important graphics functions. The graphics can be modified, printed and exported to be edited with graphics software other than MATLAB. The simplest function producing a graph of a variable `y` versus another variable `x` is `plot`. First, we define two one-dimensional arrays `x` and `y`, where `y` is the sine of `x`. The array `x` contains values between 0 and 2π with $\pi/10$ increments, whereas `y` is the element-by-element sine of `x`.

```
clear, close all, clc

x = 0 : pi/10 : 2*pi;
y = sin(x);
```

These two commands result in two one-dimensional arrays with 21 elements each, i.e. two 1-by-21 arrays. Since the two arrays x and y have the same length, we can use `plot` to produce a linear 2D graph of y against x .

```
plot(x,y)
```

This command opens a *Figure Window* named *Figure 1* with a gray background, an x -axis ranging from 0 to 7, a y -axis ranging from -1 to $+1$, and a blue line. We may wish to plot two different curves in a single plot, for example the sine and the cosine of x in different colors. The command

```
x = 0 : pi/10 : 2*pi;  
y1 = sin(x);  
y2 = cos(x);  
  
plot(x,y1,'--',x,y2,'-')
```

creates a dashed blue line displaying the sine of x and a solid red line representing the cosine of this array (Fig. 2.4). If we create another plot, the window *Figure 1* will be cleared and a new graph displayed. The command `figure`, however, can be used to create a new figure object in a new window.

```
plot(x,y1,'--')  
figure  
plot(x,y2,'-')
```

Instead of plotting both lines in one graph simultaneously, we can also plot the sine wave, hold the graph and then plot the second curve. The command `hold` is particularly important for displaying data while using different plot functions, for example if we wish to display the sine of x as a line plot and the cosine of x as a bar plot.

```
plot(x,y1,'r--')  
hold on  
bar(x,y2,'b')  
hold off
```

This command plots $y1$ versus x as a dashed red line using `'r--'`, whereas $y2$ versus x is shown as a group of blue vertical bars. Alternatively, we can plot both graphics in the same Figure Window but in different plots using `subplot`. The syntax `subplot(m,n,p)` divides the Figure Window into an m -by- n array of display regions and makes the p th display region active.

```
subplot(2,1,1), plot(x,y1,'r--')  
subplot(2,1,2), bar(x,y2,'b')
```

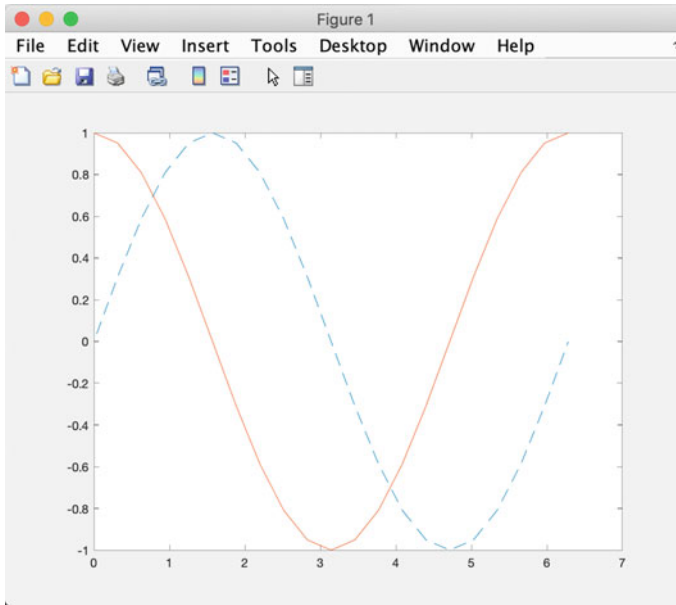


Fig. 2.4 Screenshot of the MATLAB *Figure Window* showing two curves in different colors and line types. The Figure Window allows editing of all elements of the graph after selecting *Edit Plot* from the *Tools* menu. Double clicking on the graphics elements opens an options window for modifying the appearance of the graphics. The graphics can be exported using *Save as* from the *File* menu. The command *Generate Code* from the *File* menu creates MATLAB code from an edited graph.

For example the Figure Window is divided into two rows and one column. The 2D linear plot is displayed in the upper half of the Figure Window and the bar plot appears in the lower half. It is recommended that all Figure Windows be closed before proceeding to the next example. Subsequent plots would replace the graph in the lower display region only, or in other words, the last generated graph in a Figure Window. Alternatively, the command

```
clf
```

clears the current figure. This command can be used in larger MATLAB scripts after using the function `subplot` for multiple plots in a Figure Window.

An important modification to graphics is the scaling of the axis. By default, MATLAB uses axis limits close to the minima and maxima of the data. Using the command `axis`, however, allows the scale settings to be changed. The syntax for this command is simply `axis([xmin xmax ymin ymax])`. The command

```
plot(x,y1,'r--')  
axis([0 pi -1 1])
```

sets the limits of the x -axis to 0 and π , whereas the limits of the y -axis are set to the default values -1 and $+1$. Important options of `axis` are

```
plot(x,y1,'r--')  
axis square
```

which makes the x -axis and y -axis the same length, and

```
plot(x,y1,'r--')  
axis equal
```

which makes the individual tick mark increments on the x -axis and y -axis the same length. The function `grid` adds a grid to the current plot, whereas the functions `title`, `xlabel` and `ylabel` allow a title to be defined and labels to be applied to the x - and y -axes.

```
plot(x,y1,'r--')  
title('My first plot')  
xlabel('x-axis')  
ylabel('y-axis')  
grid
```

These are a few examples how MATLAB functions can be used to edit the plot in the Command Window. More graphics functions will be introduced in the following chapters of this book.

2.10 Generating Code to Recreate Graphics

MATLAB supports various ways of editing all objects in a graph interactively using a computer mouse. First, the *Edit Plot* mode of the Figure Window needs to be activated by clicking on the arrow icon or selecting *Edit Plot* from the *Tools* menu. The Figure Window also contains some other options, such as *Rotate 3D*, *Zoom* or *Insert Legend*. The various objects in a graph, however, are selected by double-clicking on the specific component, which opens the *Property Inspector*. The Property Inspector allows changes to be made to many features (or properties) of the graph such as axes, lines, patches and text objects.

The *Generate Code* option enables us to automatically generate the MATLAB code of a figure to recreate a similar graph with different data. We use a simple plot to illustrate the use of the Property Editor and the Generate Code option to recreate a graph.

```
clear, close all, clc
```

```
x = 0 : pi/10 : 2*pi;  
y1 = sin(x);  
plot(x,y1)
```

The default layout of the graph is that of Fig. 2.4. Clicking on the arrow icon in the *Figure Toolbar* enables the Edit Plot mode. The selection handles of the graph appear, identifying the objects that are activated. Double-clicking an object in a graph opens the Property Inspector.

As an example we can use the Property Inspector to change various properties of the graph. Double-clicking the gray background of the Figure Window gives access to properties such as Window Appearance, where we can change the color to, for example, [0.68 0.92 1]. Similarly we can change the appearance of the blue line, e.g. by defining 15 point squares as markers and changing the line width to 2 points. After having made all necessary changes to the graph, the corresponding commands can even be exported by selecting Generate Code from the File menu of the Figure Window. As an example, the following code may appear in the MATLAB Editor:

```
function createfigure(X1, Y1)  
%CREATEFIGURE(X1, Y1)  
% X1: vector of x data  
% Y1: vector of y data  
  
% Auto-generated by MATLAB on 27-Jun-2014 13:28:13  
  
% Create figure  
figure1 = figure('Color',[0.68 0.92 1]);  
  
% Create axes  
axes1 = axes('Parent',figure1,'ColorOrderIndex',2);  
box(axes1,'on');  
hold(axes1,'on');  
  
% Create plot  
plot(X1,Y1,...  
     'MarkerSize',15,...  
     'Marker','square',...  
     'LineWidth',2);
```

We can then rename the function createfigure to mygraph and save the file as *mygraph.m*.

```
function mygraph(X1, Y1)
%MYGRAPH(X1,Y1)
% X1: vector of x data
% Y1: vector of y data
(cont'd)
```

The automatically-generated graphics function illustrates how graphics are organized in MATLAB. The function `figure` first opens a Figure Window. Using `axes` then establishes a coordinate system, and using `plot` draws the actual line object. The Figure section in the function reminds us that the light-blue background color of the Figure Window is represented by the RGB color coding `[0.68 0.92 1]`. The Plot section reveals the square marker symbol used and the line width of 2 points.

The newly-created function `mygraph` can now be used to plot a different data set.

```
clear

x = 0 : pi/10 : 2*pi;
y2 = cos(x);
mygraph(x,y2)
```

The figure shows a new plot with the same layout as the previous plot. The *Generate Code* function of MATLAB can therefore be used to create templates for graphics that can be used to generate plots of multiple data sets using the same layout.

Another way of modifying graphs for presentations or publications is to export the figure and import it into a different software such as CorelDraw or Adobe Illustrator. MATLAB graphics are exported by selecting the command *Save as* from the File menu or by using the command `print`. This function exports the graphics, either as a raster image (e.g. JPEG or GIF) or as a vector file (e.g. EPS or PDF), into the working directory (see Chap. 8 for more details on graphic file formats). In practice, the user should check the various combinations of export file formats and the graphics software used for final editing of the graphics. Readers interested in advanced visualization techniques with MATLAB are directed to the sister book *Collecting, Processing and Presenting Geoscientific Information—Second Edition* (Trauth and Sillmann 2018).

2.11 Publishing and Sharing MATLAB Code

Another useful feature of the software is the option to publish reports on MATLAB projects in various file formats such as HTML, XML, LaTeX and many others. This feature enables you to share your results with colleagues who may or may not have

the MATLAB software. The published code includes formatted commentary on the code, the actual MATLAB code, and all results of running the code including the output to the Command Window and all graphics created or modified by the code. To illustrate the use of the publishing feature we create a simple example of a commented MATLAB code to compute the sine and cosine of a time vector and display the results as two separate figures.

We start the Editor by typing `edit` in the Command Window, which opens a new window named `untitled`. An M-file to be published starts with a document title at the top of the file, followed by some comments that describe the contents and the version of the script. The subsequent contents of the file include sections of MATLAB code and comments, separated by the double percent signs `%%`. Whereas single percent signs `%` are known (from Sects. 2.6 and 2.8) to initiate comments in MATLAB, we now use double percent signs `%%` that indicate the start of new code sections in the Editor. The *code sections* feature, previously also known as *code cells* or *cell mode*, is a feature in MATLAB that enables you to evaluate blocks of commands called *sections* by using the buttons *Run*, *Run and Advance*, *Run Section*, *Advance*, and *Run and Time* on the Editor Toolstrip to evaluate either the entire script or parts of the script.

```
%% Example for Publishing M-Files
% This M-file illustrates the use of the publishing
% feature of MATLAB.
% By Martin Trauth, June 27, 2014

%% Sine Wave
% We define a time vector t and compute the sine y1 of t.
% The results are displayed as linear 2D graph y1 against x.
x = 0 : pi/10 : 2*pi;
y1 = sin(x);
plot(x,y1)
title('My first plot')
xlabel('x-axis')
ylabel('y-axis')

%% Cosine Wave
% Now we compute the cosine y2 of the same time vector and
% display the results.
y2 = sin(x);
plot(x,y2)
title('My first plot')
xlabel('x-axis')
ylabel('y-axis')

%%
% The last comment is separated by the double percent sign
% without text. This creates a comment in a separate cell
% without a subheader.
```

We save the M-file as *myproject.m* and click the *Publish* button in the Publish Toolstrip. The entire script is now evaluated and the Figure Windows pop up while the script is running. Finally, a window opens up that shows the contents of the published M-file. The document title and subheaders are shown in an orange font whereas the comments are in black fonts. The file includes a list of contents with jump links to proceed to the chapters of the file. The MATLAB commands are displayed on gray backgrounds but the graphics are embedded in the file without the gray default background of Figure Windows. The resulting HTML file can be easily included on a course or project webpage. Alternatively, the HTML file and included graphics can be saved as a PDF-file and shared with students or colleagues.

In the MATLAB R2016a release the Live Editor was introduced as a new way of creating, editing and running MATLAB code. Using this editor the results of a MATLAB script are displayed together with the code that produced them. A Live Script can include equations, images, hyperlinks and formatted text, to be shared with others as an interactive document, as the webpage of MathWorks suggests. The design of a Live Script looks very similar to that of published MATLAB code from the first half of this section, but you can run MATLAB commands included in the Live Script and the graphics are automatically updated.

As an example we can again use the script *myproject.m* from above. After opening the file in the classic Editor, we click on the *Save* button, select *Save As...* and save the file in the format *MATLAB Live Code files (*.mlx)*. The appearance of the script immediately changes to the design known from publishing. At first no graphics appear, but if we click on *Run* in the toolbar of the Editor, the Live Script is executed and all graphics appear embedded in the script.

The Live Editor uses three separate toolbars: *Live Editor*, *Insert* and *View*. The Live Editor toolbar contains all important functions for creating, formatting and executing Live Scripts. The Insert toolbar helps to structure the Live Script and to insert objects. These objects include, for example, images that allow us to include geological maps of the study area or outcrop photos. In addition, hyperlinks can be integrated, for example to link external material with the Live Script.

A very comfortable and useful tool is the built-in WYSIWYG (*What You See Is What You Get*) LaTeX editor. With it we can not only insert complicated equations into our Live Script, but also extend the Live Script through almost unlimited possibilities of designing text with LaTeX, for example by using complicated variable names with indices and other mathematical features. Including LaTeX together with the WYSIWYG editor is a very clever move from MathWorks, as the Live Editor itself has only a very limited selection of paragraph and character styles.

In the latest version of the Live Editor available at the time of writing this edition of the book, there was also a selection of controls such as sliders, check boxes and buttons that can be built into the Live Script. One thing that is missing is the possibility of setting writing permissions for text and code with a password. This would make it possible to actually create an interactive book that allows the reader to change only certain parameters in the code, but not the descriptive text. MathWorks advises that the Live Editor will be further developed and extended with additional features, but will never compete with modern text processing software. In

summary the new Live Editor is a real alternative to the classic Editor, but the comfortable user interface also sometimes leads to minor delays and inaccuracies when writing code in a hurry.

2.12 Creating Graphical User Interfaces

Almost all the methods of data analysis presented in this book are in the form of MATLAB scripts, i.e. series of commands that operate on data in the workspace (Sect. 2.8). Only in a few cases are the algorithms implemented in functions such as `canc` for adaptive filtering (Sect. 6.10) or `minput` for digitizing from the screen (Sect. 8.7). The MATLAB commands provided by MathWorks, however, are mostly functions, i.e. algorithms with input and output variables. The most convenient variants of these functions are those with a *graphical user interface* (GUI). A GUI in MATLAB is a graphical display in one or more windows containing controls (or components) that enable the user to perform interactive tasks without typing commands in the Command Window or writing a script in the Editor. These components include pull-down menus, push buttons, sliders, text input fields and more. The GUI can read and write data files as well as performing many types of computation and displaying the results in graphics.

The manual entitled *MATLAB App Building* (MathWorks 2020b) provides a comprehensive guide to the creation of GUIs with MATLAB. This manual, however, demonstrates a rather complex example with many interactive elements, rather than providing the simplest possible example of a GUI. The following text therefore provides a very simple example of a GUI that computes and displays a Gaussian function for a mean and a standard deviation that can be defined by the user. Creating such a simple GUI with MATLAB requires two steps: the first step involves designing the layout of the GUI, and the second step involves adding functions to the components of the GUI. One way to create a graphical user interface with MATLAB is using the *GUI Design Environment* (GUIDE). We start GUIDE by typing

```
guide
```

in the Command Window. Calling GUIDE opens the *GUIDE Quick Start* dialog where we can choose to open a previously created GUI or create a new one from a template. From the dialog we choose the GUIDE template *Blank GUI (Default)* and click *OK*, after which the *GUIDE Layout Editor* starts. First, we enable *Show names in component palette* in the *GUIDE Preferences* under the *File* menu and click *OK*. We then select *Grid and Rulers* from the *Tools* menu and enable *Show rulers*. The *GUIDE Layout Editor* displays an empty layout with dimensions of 670-by-388 pixels. We resize the layout to 500-by-300 pixels by clicking and dragging the lower right corner of the GUI.

Next, we place components such as static text, edit text, and axes onto the GUI by choosing the corresponding controls from the component palette. In our example we place two *Edit Text* areas on the left side of the GUI, along with a *Static Text*

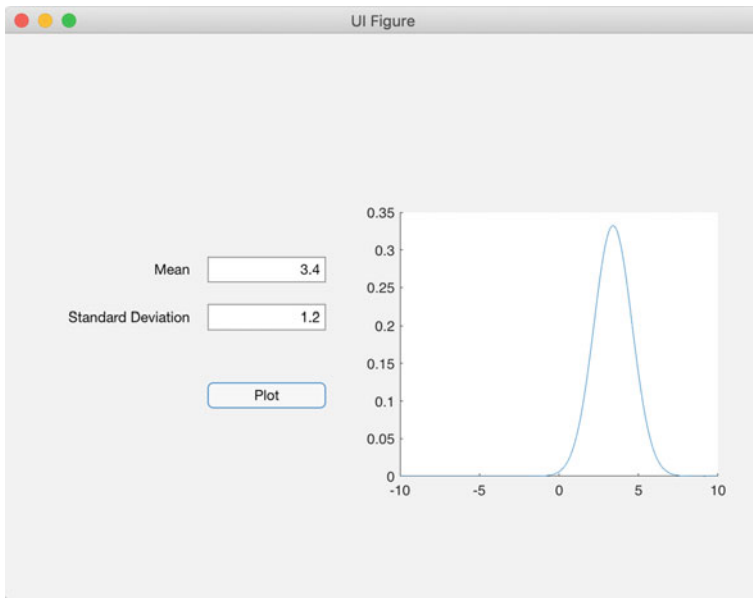


Fig. 2.5 Screenshot of the graphical user interface (GUI) `gaussiantool` for plotting a Gaussian function with a given mean and standard deviation. The GUI allows the values of the mean and standard deviation to be changed in order to update the graphics on the right. The GUI has been created using the MATLAB *App Designer*.

area containing the title *Mean*, with *Standard Deviation* above it. Double clicking the static text areas, the *Property Inspector* comes up in which we can modify the properties of the components. We change the *String* of the static text areas to *Mean* and *Standard Deviation*. We can also change other properties, such as the *Font-Name*, *FontSize*, *BackgroundColor*, and *HorizontalAlignment* of the text. Instead of the default *Edit Text* content of the edit text areas we choose 0 for the mean and 1 for the standard deviation text area. We then place an axis with dimensions of 250-by-200 pixels to the right of the GUI. Next, we save and activate the GUI by selecting *Run* from the *Tools* menu. GUIDE displays a dialog box with the question *Activating will save changes ...?*, where we click *Yes*. In the following *Save As* dialog box, we define a *FIG*-file name such as `gaussiantool.fig`.

GUIDE then saves this figure file together with the corresponding MATLAB code in a second file named `gaussiantool.m`. Furthermore, the MATLAB code is opened in the Editor and the default GUI is opened in a Figure Window with no menu or toolbar (Fig. 2.5). As we can see, GUIDE has automatically programmed the code of our GUI layout, including an initialization code at the beginning of the file that we should not edit. This code is included in the main routine named `gaussiantool`. The file also contains other functions called by `gaussiantool`, for instance the function `gaussiantool_Opening_Fcn` (executed before `gaussiantool` is made visible), `gaussiantool_OutputFcn` (sending output to the command line, not used here), `edit1_CreateFcn` and `edit2_CreateFcn` (initializing the edit text areas

when they are created), and `edit1_Callback` and `edit2_Callback` (accepting text input and returning this input either as text or as a double-precision number).

We now add code to our GUI `gaussiantool`. First, we add initial values for the global variables `mmean` and `mstd` in the opening function `gaussiantool_Opening_Fcn` by adding the following lines after the last comment line marked by `%` in the first column:

```
global mmean mstd
mmean = 0;
mstd = 1;
```

The two variables must be global because they are used in the callbacks that we edit next (as in Sect. 2.8). The first of these callbacks `edit1_Callback` gets three more lines of code after the last comment line:

```
global mmean
mmean = str2double(get(hObject,'String'));
calculating_gaussian(hObject, eventdata, handles)
```

The first line defines the global variable `mmean`, which is then obtained by converting the text input into double precision with `str2double` in the second line. The function `edit1_Callback` then calls the function `calculating_gaussian`, which is a new function at the end of the file. This function computes and displays the Gaussian function with a mean value of `mmean` and a standard deviation of `mstd`.

```
function calculating_gaussian(hObject, eventdata, handles)
global mmean mstd
x = -10 : 0.1 : 10;
y = normpdf(x, mmean, mstd);
plot(x,y)
```

The second callback `edit2_Callback` picks the value of the standard deviation `mstd` from the second Edit Text area, which is then also used by the function `calculating_gaussian`.

```
global mstd
mstd = str2double(get(hObject,'String'));
calculating_gaussian(hObject, eventdata, handles)
```

After saving the file `gaussiantool.m` we can run the new GUI by typing

```
gaussiantool
```

in the Command Window. The GUI starts where we can change the values of the mean and the standard deviation, then press return. The plot on the right is updated with each press of the return key. Using

```
edit gaussiantool
guide gaussiantool
```

we can open the GUI code and Figure Window for further edits.

The second way to create a graphical user interface with MATLAB is using the *App Designer*. We start the App Designer by typing

```
appdesigner
```

in the Command Window. As before, we drag and drop components such as two *Edit Text* areas on the left side of the GUI, which contains static text areas that we change to *Mean* and *Standard Deviation*. Below these we place a *Button* whose label we change to *Plot*. We then place an axis to the right of the GUI. Next, we save and activate the GUI by selecting *Run* from the toolbar of the App Designer. In the following *Save As* dialog box, we define a name such as *gaussiantool.mlapp*.

The App Designer then saves the app and the default GUI is opened in a Figure Window entitled *UI Figure*, with no menu or toolbar. As before with GUIDE, the App Designer has automatically programmed the code of our GUI layout, which we can view by switching from the *Design View* to the *Code View* in the user interface of the tool. In contrast to GUIDE, the App Designer allows us to edit only those parts of the code that are highlighted in white, while all lines highlighted in gray are not editable.

First we change the function *MeanEditFieldValueChanged* by defining a global variable *mmean*. This variable is then changed in the callback function *app.-MeanEditField.Value*, where we change the variable value to *mmean*. The function now looks like this:

```
% Value changed function: MeanEditField
function MeanEditFieldValueChanged(app,event)
global mmean
mmean = app.MeanEditField.Value;
end
```

We change the function to enter the standard deviation accordingly:

```
% Value changed function: StandardDeviationEditField
function StandardDeviationEditFieldValueChanged(app,event)
global mstd
mstd = app.StandardDeviationEditField.Value;
end
```

Finally we modify the code for the push button:

```
% Button pushed function: PlotButton
function PlotButtonPushed(app, event)
global mmean mstd
x = -10 : 0.1 : 10;
y = normpdf(x, mmean, mstd);
plot(app.UIAxes,x,y)
```

The function first defines the two global variables `mmean` and `mstd`, then creates an array `x` running from -10 to $+10$ in steps of 0.1 . It then computes and displays the Gaussian function with a mean value of `mmean` and a standard deviation of `mstd`. After saving the file *gaussiantool.mlapp* we can run the new GUI by typing

```
gaussiantool
```

in the Command Window. The GUI starts where we can change the values of the mean and the standard deviation, then press *Plot*. The plot on the right is updated with each press of the *Plot* button. Using

```
appdesigner gaussiantool
```

we can open the GUI for further edits. Such GUIs allow a very direct and intuitive handling of functions in MATLAB that can also include animations such as the one used in *canctool* (Sect. 6.10), and the display of an audio-video signal. On the other hand, however, GUIs always require an interaction with the user, who needs to click push buttons, move sliders and edit text input fields while the data is being analyzed. The automatic processing of large quantities of data is therefore more often carried out using scripts and functions with no graphical user interface.

Recommended Reading

- Attaway S (2018) MATLAB: a practical introduction to programming and problem solving. Butterworth-Heinemann, Oxford Waltham
- Hahn BH, Valentine DT (2017) Essential MATLAB for engineers and scientists, 6th edn. Academic Press, London, Oxford, Boston, New York, San Diego
- MathWorks (2020a) MATLAB primer. The MathWorks Inc., Natick, MA
- MathWorks (2020b) MATLAB App Building. The MathWorks Inc., Natick, MA
- MathWorks (2020c) MATLAB programming fundamentals. The MathWorks Inc., Natick, MA
- Quarteroni A, Saleri F, Gervasio P (2014) Scientific computing with MATLAB and Octave, 4th edn. Springer, Berlin, Heidelberg, New York
- Trauth MH, Sillmann E (2018) Collecting, processing and presenting geoscientific information, MATLAB® and design recipes for earth sciences, 2nd edn. Springer, Berlin Heidelberg, New York

3.1 Introduction

The statistical properties of a single parameter are investigated by means of univariate analysis. Such a parameter could, for example, be the organic carbon content of deep-sea sediments, the sizes of grains in a sandstone layer, or the ages of sanidine crystals in a volcanic ash. Both the number and the size of *samples* that we collect from a larger *population* are often limited by financial and logistical constraints. The methods of univariate statistics assist us to draw from the sample conclusions that apply to the population as a whole. For univariate analysis we use the Statistics and Machine Learning Toolbox (MathWorks 2020), which contains all the necessary routines.

We first need to describe the characteristics of the sample using statistical parameters, and to compute an *empirical distribution* (*descriptive statistics*) (Sects. 3.2 and 3.3). A brief introduction is provided to the most important statistical parameters (such as the *measures of central tendency and dispersion*), followed by MATLAB examples. We then select a *theoretical distribution* that shows similar characteristics to the empirical distribution (Sects. 3.4 and 3.5). A suite of theoretical distributions is introduced and their potential applications outlined prior to using MATLAB tools to explore these distributions. We then try to draw conclusions from the sample that can be applied to the larger population of interest (*hypothesis testing*) (Sects. 3.6–3.12). Section 3.13 introduces methods used to fit distributions to our own data sets. The final section in this chapter (Sect. 3.14) provides a very brief introduction to the basics of error analysis.

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_3) contains supplementary material, which is available to authorized users.

3.2 Empirical Distributions

Let us assume that we have collected a number of measurements x_i from a specific object. The collection of data, or sample, as a subset of the population of interest, can be written as a vector x , or one-dimensional array

$$x = (x_1, x_2, \dots, x_N)$$

containing a total of N observations. The vector x may contain a large number of data points and it may consequently be difficult to understand its properties. Descriptive statistics are therefore often used to summarize the characteristics of the data. The statistical properties of the data set may be used to define an empirical distribution, which can then be compared to a theoretical one.

The most straightforward way of investigating the sample characteristics is to display the data in a graphical form. Plotting all of the data points along a single axis does not reveal a great deal of information about the data set. However, the density of the points along the scale does provide some information about the characteristics of the data. A widely-used graphical display of univariate data is the *histogram* (Fig. 3.1). A histogram is a bar plot of a frequency distribution that is organized in intervals or *classes*. Such histogram plots provide valuable information on the characteristics of the data, such as the *central tendency*, the *dispersion* and the *general shape* of the distribution. However, quantitative measures provide a more accurate way of describing the data set than the graphical form. In purely quantitative terms, the *mean* and the *median* define the central tendency of the data set, while the data dispersion is expressed in terms of the *range* and the *standard deviation*.

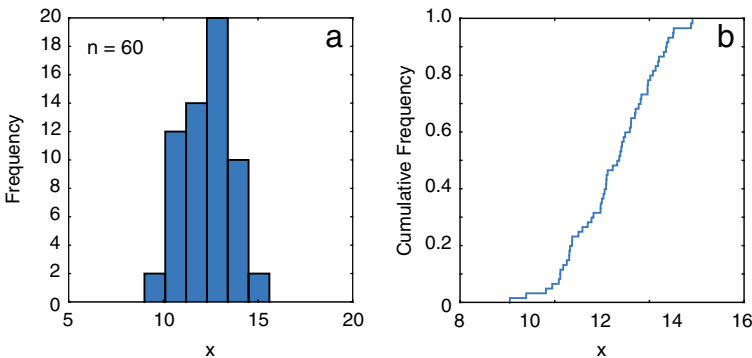


Fig. 3.1 Graphical representation of an empirical frequency distribution. **a** In a *histogram* the frequencies are organized in *nbin* classes and plotted as a bar plot. **b** The *cumulative distribution plot* of a frequency distribution displays the totals of all observations less than or equal to a certain value. This plot is normalized to a total number of observations of one.

Measures of Central Tendency

Parameters of central tendency or location represent the most important measures for characterizing an empirical distribution (Fig. 3.2). These values help locate the data on a linear scale. They represent a typical or best value that describes the data. The most popular indicator of central tendency is the *arithmetic mean*, which is the sum of all data points divided by the number of observations:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

The arithmetic mean of a univariate data set can also be referred to the mean or the average. The sample mean is used as an estimate of the population mean μ for the underlying theoretical distribution. The arithmetic mean is, however, sensitive to outliers, i.e. extreme values that may be very different from the majority of the data, and the *median* is therefore often used as an alternative measure of central tendency. The median is the x -value that is in the middle of the data set, i.e. 50% of the observations are smaller than the median and 50% are larger. The median of a data set sorted in ascending order is defined as

$$\tilde{x} = x_{(N+1)/2}$$

if N is odd and

$$\tilde{x} = (x_{(N/2)} + x_{(N/2)+1})/2$$

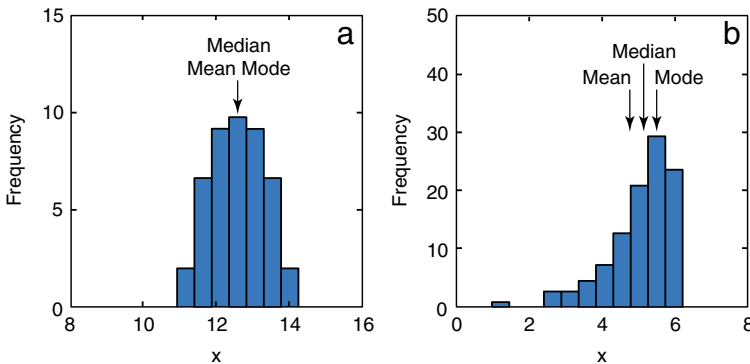


Fig. 3.2 Measures of *central tendency*. **a** In an unimodal symmetric distribution the mean, the median and the mode are all identical. **b** In a skewed distribution the median lies between the mean and the mode. The mean is highly sensitive to outliers, whereas the median and the mode are little influenced by extremely high or low values.

if N is even. Although outliers also affect the median, their absolute values do not influence it. *Quantiles* are a more general way of dividing the data sample into groups containing equal numbers of observations. For example the three *quartiles* divide the data into four groups, the four *quintiles* divide the observations in five groups and the 99 *percentiles* define one hundred groups.

The third important measure for central tendency is the *mode*. The mode is the most frequent x -value or—if the data are grouped in classes—the center of the class with the largest number of observations. The data set has no mode if there are no values that appear more frequently than any of the other values. Frequency distributions with a single mode are called *unimodal*, but there may also be two modes (*bimodal*), three modes (*trimodal*) or four or more modes (*multimodal*) (Fig. 3.3).

The mean, median and mode are used when several quantities add together to produce a total, whereas the *geometric mean* is often used if these quantities are multiplied. Let us assume that the population of an organism increases by 10% in the first year, 25% in the second year, and then 60% in the last year. The average rate of increase is not the arithmetic mean, since the original number of individuals has increased by a factor (not a sum) of 1.1 after one year, 1.25 after the second year, and 1.6 after the third year. The average growth of the population is therefore calculated by the geometric mean:

$$\bar{x}_G = (x_1 \cdot x_2 \cdot \dots \cdot x_N)^{1/N}$$

The average growth of these values is 1.3006 suggesting an approximate per annum growth in the population of 30%. The arithmetic mean would result in an erroneous value of 1.3167 or approximately 32% annual growth. The geometric mean is also a useful measure of central tendency for skewed or log-normally distributed data, in which the logarithms of the observations follow a Gaussian or normal distribution. The geometric mean, however, is not used for data sets containing negative values. Finally, the harmonic mean

$$\bar{x}_H = N / \left(\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_N} \right)$$

is also used to derive a mean value for asymmetric or log-normally distributed data, as is the geometric mean, but neither is robust to outliers. The harmonic mean is a better average when the numbers are defined in relation to a particular unit. The commonly quoted example is for averaging velocities. The harmonic mean is also used to calculate the mean of sample sizes.

Measures of Dispersion

Another important property of a distribution is the dispersion. Some of the parameters that can be used to quantify dispersion are illustrated in Fig. 3.3. The

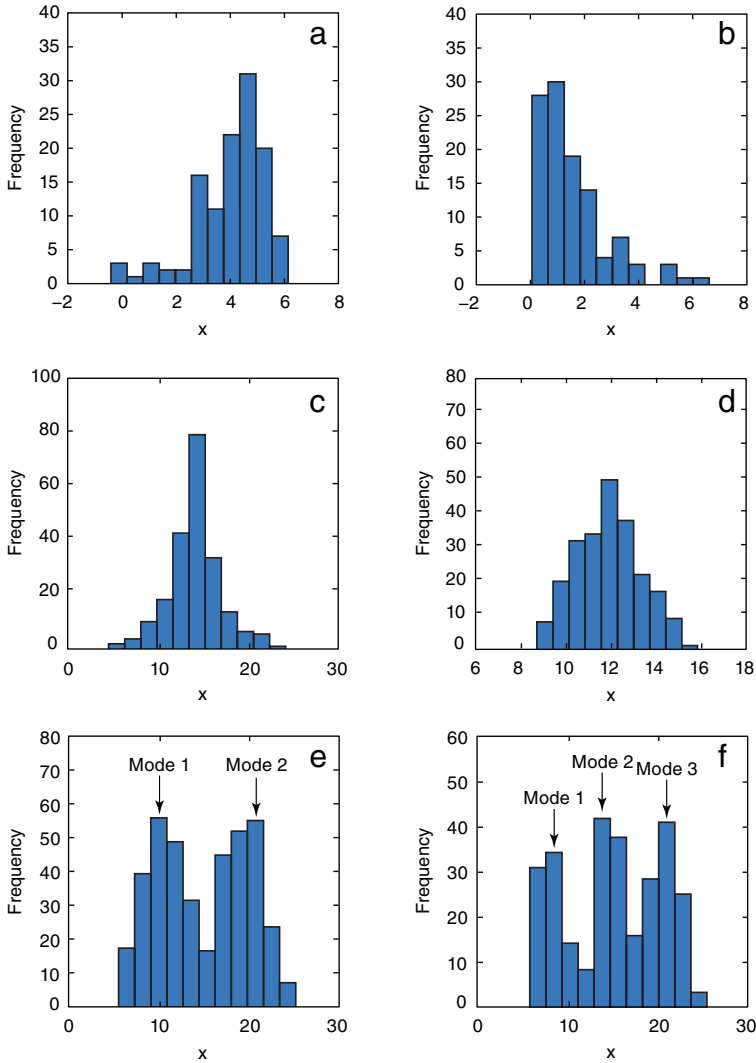


Fig. 3.3 Dispersion and shape of a distribution. **a–b** Unimodal distributions showing a negative or positive skew. **c–d** Distributions showing a high or low kurtosis. **e–f** Bimodal and trimodal distributions showing two or three modes.

simplest way to describe the dispersion of a data set is by the range, which is the difference between the highest and lowest values in the data set, given by

$$\Delta x = x_{\max} - x_{\min}$$

Since the range is defined by the two extreme data points it is very susceptible to outliers and is therefore not a reliable measure of dispersion in most cases. The interquartile range of the data (i.e. the middle 50% of the data) can be used in an attempt to overcome this problem.

A more useful measure for dispersion is the *standard deviation*.

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

The standard deviation is the average deviation of each data point from the mean. The standard deviation of an empirical distribution is often used as an estimate of the population standard deviation σ . The formula for the population standard deviation uses N instead of $N-1$ as the denominator. The sample standard deviation s is computed with $N-1$ instead of N since it uses the sample mean instead of the unknown population mean. The sample mean, however, is computed from the data x_i , which reduces the number of degrees of freedom by one. The *degrees of freedom* are the number of values in a distribution that are free to be varied. Dividing the average deviation of the data from the mean by N would therefore underestimate the population standard deviation σ .

The *variance* is the third important measure of dispersion. The variance is simply the square of the standard deviation.

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$$

Although the variance has the disadvantage of not having the same dimensions as the original data, it is extensively used in many applications instead of the standard deviation.

In addition, both *skewness* and *kurtosis* can be used to describe the shape of a frequency distribution (Fig. 3.3). Skewness is a measure of the asymmetry of the tails of a distribution. The most popular way to compute the asymmetry of a distribution is by Pearson's (1857–1936) mode skewness:

$$skewness = (mean - mode)/standard\ deviation$$

A negative skew indicates that the distribution is spread out more to the left of the mean value, assuming values increasing towards the right along the axis. The sample mean is in this case smaller than the mode. Distributions with positive skewness have large tails that extend towards the right. The skewness of the symmetric normal distribution is zero. Although Pearson's measure is a useful one, the following formula by Fisher for calculating the skewness is often used instead, including in the relevant MATLAB function.

$$skewness = \frac{\sum_{i=1}^N \frac{(x_i - \bar{x})^3}{s^3}}{N}$$

The second important measure for the shape of a distribution is the *kurtosis*. Again, numerous formulas are available for computing the kurtosis. MATLAB uses the following formula:

$$kurtosis = \sum_{i=1}^N \frac{(x_i - \bar{x})^4}{s^4}$$

The kurtosis is a measure of whether the data are peaked or flat relative to a normal distribution. A high kurtosis indicates that the distribution has a distinct peak near the mean, whereas a distribution characterized by a low kurtosis shows a flat top near the mean and broad tails. Higher peakedness in a distribution results from rare extreme deviations, whereas a low kurtosis is caused by frequent moderate deviations. A normal distribution has a kurtosis of three, and some definitions of kurtosis therefore subtract three from the above term in order to set the kurtosis of the normal distribution to zero.

3.3 Examples of Empirical Distributions

As an example we can analyze the data contained in the file *organicmatter_one.txt*. This file contains the organic carbon content C_{org} of lake sediments in weight percentage (wt%). In order to load the data, we type

```
clear, close all, clc
corg = load('organicmatter_one.txt');
```

The data file contains 60 measurements of C_{org} that can be displayed by

```
plot(corg,zeros(1,length(corg)),'o')
```

This graph shows some of the characteristics of the data. The organic carbon content of the samples ranges between 9 and 15 wt%, with most of the data clustering between 12 and 13 wt%. Values below 10 wt% and above 14 wt% are rare. While this kind of representation of the data undoubtedly has its advantages, histograms are a much more convenient way to display univariate data (Fig. 3.1). Histograms divide the range of the data into *nbin* equal intervals (also called bins or classes), count the number of observations *n* in each bin, and display the frequency distribution of observations as a bar plot. The bins are either defined by their edges *e* or their centers *v*. There is no fixed rule for the correct number of bins; the most suitable number will depend on the application (for example the statistical method) that is used. Ideally, the number of bins *n* should lie between 5 and 15; it should closely reflect the underlying distribution and should not result in any empty bins,

i.e. classes with no counts. In practice, the square root of the total number of observations `length(corg)`, rounded to the nearest integer using `round`, is often used as the number of bins. In our example `nbin` can be calculated using

```
nbin = round(sqrt(length(corg)))
```

which yields

```
nbin =  
      8
```

The eight bin centers `v` are calculated by typing

```
vmin = min(corg) + 0.5*range(corg)/nbin;  
vmax = max(corg) - 0.5*range(corg)/nbin;  
vwth = range(corg)/nbin;  
v = vmin : vwth : vmax;
```

which yields

```
v =  
  
Columns 1 through 4  
    9.7383    10.3814    11.0245    11.6676  
  
Columns 5 through 8  
   12.3107   12.9537   13.5968   14.2399
```

The smallest bin center `vmin` equals the minimum value in the data `corg` plus half of the range of the data `corg`, divided by the number of bins `nbin`. Similarly, the largest bin center `vmax` equals the maximum value of the data `corg` minus half of the range of the data `corg`, divided by `nbin`. The bin width `ewth` is calculated by dividing the range of the data `corg` by `nbin`. The corresponding nine bin edges `e` are calculated using

```
emin = min(corg);  
emax = max(corg);  
ewth = range(corg)/nbin;  
e = emin : ewth : emax
```

which yields

```
e =
Columns 1 through 4
    9.4168    10.0598    10.7029    11.3460
Columns 5 through 8
    11.9891    12.6322    13.2753    13.9184
Column 9
    14.5615
```

The smallest and largest bin edges, `emin` and `emax`, correspond to the minimum and maximum values in `corg`. The width of the bins `ewth`, as defined by the bin edges, is equal to `vwth`. Binning can be performed using a `for` loop in combination with an `if-then` construct. Values in `corg` that fall exactly on an edge `e` are herein assigned to the higher bin. Values falling exactly on the edge of the last bin are assigned to the last bin.

```
for i = 1 : nbin
    if i < nbin
        corgb = corg(emin+(i-1)*ewth<=corg & emin+i*ewth>corg);
        n(i) = length(corgb);
    else
        corgb = corg(emin+(i-1)*ewth<=corg & emin+i*ewth>=corg);
        n(i) = length(corgb);
    end
end
```

The result of the binning is stored in the frequency distribution `n`, which we can display with

```
n
```

which yields

```
n =
    2     2    10     7    14     9    12     4
```

Instead of using the above script we can also use the functions `histcount` and `histogram` to determine the bin centers, edges and counts. Typing

```
histogram(corg)
```

displays a histogram with six classes, using an automatic binning algorithm. The algorithm returns bins with a uniform width, chosen to cover the range of elements in `corg` and to reveal the underlying shape of the distribution. The MATLAB Help

lists and explains various such methods for automatic binning. The function `histogram` also returns a structure array `h`

```
h = histogram(corg);
```

which yields

```
h =  
Histogram with properties:  
    Data: [60×1 double]  
  Values: [2 6 13 20 15 4]  
  NumBins: 6  
 BinEdges: [9 10 11 12 13 14 15]  
 BinWidth: 1  
 BinLimits: [9 15]  
Normalization: 'count'  
  FaceColor: 'auto'  
 EdgeColor: [0 0 0]
```

Show all properties

listing the properties of the histogram. As an example we can access the bin centers `v` and the frequency distribution `n` by typing

```
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)  
n = h.Values
```

while the Figure Window is still active, which yields

```
v =  
    9.5000    10.5000    11.5000    12.5000    13.5000    14.5000  
  
n =  
     2     6    13    20    15     4
```

Alternatively, we can use the bin edges `e` defined above to display the bar plot of the frequency distribution `n` by typing

```
h = histogram(corg,e);  
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)  
n = h.Values
```

which yields

```
v =
Columns 1 through 4
    9.7383    10.3814    11.0245    11.6676
Columns 5 through 8
    12.3107    12.9537    13.5968    14.2399
n =
    2     2    10     7    14     9    12     4
```

as well as the histogram plot, as shown in Fig. 3.1a. We can also use the function `histcount` to determine `n` and `e` without plotting the histogram. Typing

```
[n,e] = histcounts(corg,e)
v = diff(e(1:2)) * 0.5 + e(1:end-1)
```

yields

```
n =
    2     2    10     7    14     9    12     4
e =
Columns 1 through 5
    9.4168    10.0598    10.7029    11.3460    11.9891
Columns 6 through 9
    12.6322    13.2753    13.9184    14.5615
v =
Columns 1 through 5
    9.7383    10.3814    11.0245    11.6676    12.3107
Columns 6 through 8
    12.9537    13.5968    14.2399
```

The functions `histogram` and `histcount` provide numerous ways of binning the data, of normalizing the data, and of displaying the histogram. In the following sections, we often have to normalize the frequency distribution such that the total number of observations `sum(n)` is one. This can be achieved by typing


```
h = histogram(corg, 'Normalization', 'probability')
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)
n = h.Values
```

which yields

```
v =
Columns 1 through 5
    9.5000    10.5000    11.5000    12.5000    13.5000
Column 6
    14.5000
n =
Columns 1 through 5
    0.0333    0.1000    0.2167    0.3333    0.2500
Column 6
    0.0667
```

As an alternative way of plotting the data, the empirical cumulative distribution function can be displayed using `cdfplot` (Fig. 3.1b).

```
cdfplot(corg)
```

The most important parameters describing the distribution are the measures for central tendency and the dispersion about the average. The most popular measure for central tendency is the arithmetic mean.

```
mean(corg)
ans =
    12.3448
```

Since this measure is very susceptible to outliers, we can take the median as an alternative measure of central tendency,

```
median(corg)
ans =
    12.4712
```

which does not differ by very much in this particular example. However, we will see later that this difference can be significant for distributions that are not symmetric. A more general parameter to define fractions of the data less than, or equal to, a certain value is the quantile. Some of the quantiles have special names, such as the three quartiles dividing the distribution into four equal parts, 0–25%, 25–50%, 50–75% and 75–100% of the total number of observations. We use the function `quantile` to compute the three quartiles:

```
quantile(corg,[.25 .50 .75])  
  
ans =  
  
    11.4054    12.4712    13.2965
```

Less than 25% of the data values are therefore lower than 11.4054, 25% are between 11.4054 and 12.4712, another 25% are between 12.4712 and 13.2965, and the remaining 25% are higher than 13.2965.

The third parameter in this context is the mode, which is the midpoint of the interval with the highest frequency. The MATLAB function `mode` to identify the most frequent value in a sample is unlikely to provide a good estimate of the peak in continuous probability distributions, such as the one in `corg`. Furthermore, the `mode` function is not suitable for finding peaks in distributions that have multiple modes. In these cases it is better to compute a histogram and calculate the peak of that histogram. We can use the function `find` to locate the class that has the largest number of observations.

```
v(find(n == max(n)))
```

or simply

```
v(n == max(n))  
  
ans =  
  
    12.3107
```

Both statements are identical and identify the largest element in `n`. The index of this element is then used to display the midpoint of the corresponding class `v`. If there are several elements in `n` with similar values this statement returns several solutions, suggesting that the distribution has several modes. The median, quartiles, minimum, and maximum of a data set can be summarized and displayed in a *box and whisker plot*.

```
boxplot(corg)
```

The boxes have lines at the lower quartile, the median, and the upper quartile values. The whiskers are lines extending from each end of the boxes to show the extent or range of the rest of the data.

The most popular measures for dispersion are range, variance and standard deviation. We have already used the range to define the midpoints of the classes. The range is the difference between the highest and lowest value in the data set.

```
range(corg)
ans =
5.1447
```

The variance is the average of the squared deviation of each number from the mean of a data set.

```
var(corg)
ans =
1.3595
```

The standard deviation is the square root of the variance.

```
std(corg)
ans =
1.1660
```

Note that, by default, the functions `var` and `std` calculate the sample variance and sample standard deviation, providing an unbiased estimate of the dispersion of the population. When using skewness to describe the shape of the distribution, we observe a slightly negative skew.

```
skewness(corg)
ans =
-0.2529
```

Finally, the peakedness of the distribution is described by the kurtosis. The result from the function `kurtosis`,

```
kurtosis(corg)
ans =
2.4670
```

suggests that our distribution is slightly flatter than a Gaussian distribution since its kurtosis is less than three.

Most of these functions have corresponding versions for data sets containing gaps, such as `nanmean` and `nanstd`, that treat NaNs as missing values. To illustrate the use of these functions we introduce a gap into our data set and compute the mean using both `mean` and `nanmean` for comparison.

```
corg(25,1) = NaN;
mean(corg)
ans =
    NaN
nanmean(corg)
ans =
    12.3371
```

In this example the function `mean` follows the rule that all operations with NaNs result in NaNs, whereas the function `nanmean` simply skips the missing value and computes the mean of the remaining data.

As a second example we now explore a data set characterized by a significant skew. The data represent 120 microprobe analyses on glass shards hand-picked from a volcanic ash. The volcanic glass has been affected by chemical weathering at an initial stage and the shards therefore exhibit glass hydration and sodium depletion in some sectors. We can study the distribution of sodium (in wt%) in the 120 analyses using the same procedure as above. The data are stored in the file *sodiumcontent_one.txt*.

```
clear, close all, clc
sodium = load('sodiumcontent_one.txt');
```

As a first step, it is always recommended to display the data as a histogram. The square root of 120 suggests 11 classes, and we therefore display the data by typing

```
h = histogram(sodium,11)
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)
n = h.Values
```

Since the distribution has a negative skew, the mean, the median and the mode differ significantly from each other.

```
mean(sodium)
ans =
    5.6628
median(sodium)
```

```
ans =
    5.9741

v(find(n == max(n)))

ans =
    6.5800
```

The mean of the data is lower than the median, which is in turn lower than the mode. We can observe a strong negative skewness, as expected from our data.

```
skewness(sodium)

ans =
   -1.1086
```

We now introduce a significant outlier to the data and explore its effect on the statistics of the sodium content. For this we will use a different data set that is better suited to this example than the previous data set. The new data set contains higher sodium values of around 17 wt% and is stored in the file *sodiumcontent_two.txt*.

```
clear, close all, clc

sodium = load('sodiumcontent_two.txt');
```

This data set contains only 50 measurements, in order to better illustrate the effects of an outlier. We can use the same script used in the previous example to display the data in a histogram with seven classes, and to compute the number of observations n in each of the classes v .

```
h = histogram(sodium,7)
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)
n = h.Values

v =

Columns 1 through 4

    13.5500    14.2500    14.9500    15.6500

Columns 5 through 7

    16.3500    17.0500    17.7500
```

```

n =
    1  3  3  6  6 15 16

mean(sodium)

ans =

    16.6379

median(sodium)

ans =

    16.9739

v(find(n == max(n)))

ans =

    17.7500

```

The mean of the data is 16.6379, the median is 16.9739 and the mode is 17.7500. We now introduce a single, very low value of 7.0 wt% in addition to the 50 measurements contained in the original data set.

```

sodium(51,1) = 7.0;

h = histogram(sodium,11)
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1)
n = h.Values

v =

Columns 1 through 4
    7.5500    8.6500    9.7500   10.8500

Columns 5 through 8
   11.9500   13.0500   14.1500   15.2500

Columns 9 through 11
   16.3500   17.4500   18.5500

n =

Columns 1 through 8
     1     0     0     0     0     1     3     8

Columns 9 through 11
     9    27     2

```

The histogram of this data set using a larger number of classes illustrates the distortion produced in the frequency distribution by this single outlier, showing several empty classes. The influence of this outlier on the sample statistics is also substantial.

```
mean(sodium)

ans =

    16.4490

median(sodium)

ans =

    16.9722

v(find(n == max(n)))

ans =

    17.4500
```

The most significant change observed is in the mean (16.4490), which is substantially lower due to the presence of the outlier. This example clearly demonstrates the sensitivity of the mean to outliers. In contrast, the median of 16.9722 is relatively unaffected.

3.4 Theoretical Distributions

We have now described the empirical frequency distribution of our sample. A histogram is a convenient way to depict the frequency distribution of the variable x . If we sample the variable sufficiently often and the output ranges are narrow, we obtain a very smooth version of the histogram. An infinite number of measurements $N \rightarrow \infty$ and an infinitely small class width produce the random variable's *probability density function* (PDF). The probability distribution density $f(x)$ defines the probability that the variable has a value equal to x . The integral of $f(x)$ is normalized to unity, i.e. the total number of observations is one. The *cumulative distribution function* (CDF) is the sum of the frequencies of a discrete PDF or the integral of a continuous PDF. The cumulative distribution function $F(x)$ is the probability that the variable will have a value less than or equal to x .

As a next step, we need to find appropriate theoretical distributions that fit the empirical distributions described in the previous section. This section therefore introduces the most important theoretical distributions and describes their application.

Uniform Distribution

A *uniform* or *rectangular distribution* is a distribution that has a constant probability (Fig. 3.4). The corresponding probability density function is

$$f(x) = 1/N = \text{const.}$$

where the random variable x has any of N possible values. The cumulative distribution function is

$$F(x) = x \cdot 1/N$$

The probability density function is normalized to unity

$$\sum_{-\infty}^{+\infty} f(x) dx = 1$$

i.e. the sum of all probabilities is one. The maximum value of the cumulative distribution function is therefore one.

$$F(x)_{\max} = 1$$

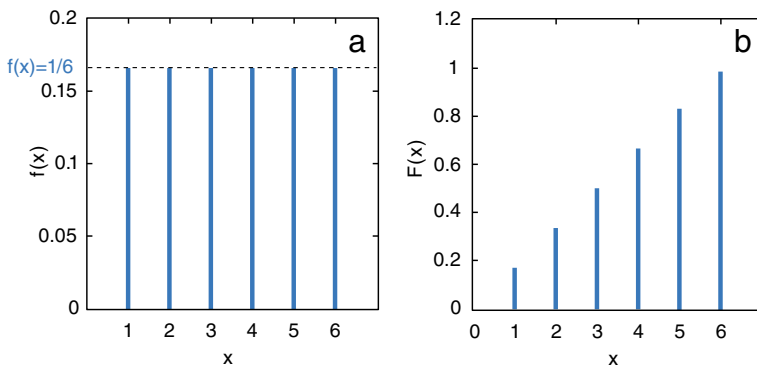


Fig. 3.4 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a uniform distribution with $N = 6$. The 6 discrete values of the variable x have the same probability of $1/6$.

An example is a rolling die with $N = 6$ faces. A discrete variable such as the faces of a die can only take a countable number of values x . The probability for each face is $1/6$. The probability density function of this distribution is

$$f(x) = 1/6$$

The corresponding cumulative distribution function is

$$F(x) = x \cdot 1/6$$

where x takes only discrete values, $x = 1, 2, \dots, 6$.

Binomial or Bernoulli Distribution

A *binomial* or *Bernoulli distribution*, named after the Swiss scientist Jakob Bernoulli (1654–1705), gives the discrete probability of x successes out of N trials, with a probability p of success in any given trial (Fig. 3.5). The probability density function of a binomial distribution is

$$f(x) = \binom{N}{x} p^x (1-p)^{N-x}$$

The cumulative distribution function is

$$F(x) = \sum_{i=1}^x \binom{N}{i} p^i (1-p)^{N-i}$$

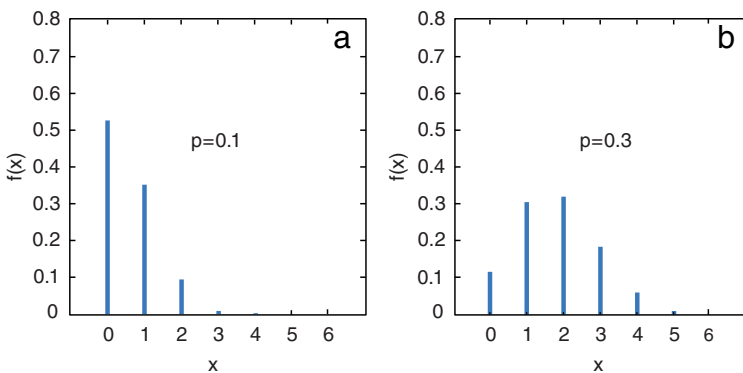


Fig. 3.5 Probability density function $f(x)$ of a binomial distribution, which gives the probability p of x successes out of $N = 6$ trials, with probability **a** $p = 0.1$ and **b** $p = 0.3$ of success in any given trial.

where

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

The binomial distribution has two parameters N and p . An example for the application of this distribution is to determine the likely outcome of drilling for oil. Let us assume that the probability of drilling success is 0.1 or 10%. The probability of $x = 3$ successful wells out of a total number of $N = 10$ wells is

$$f(3) = \binom{10}{3} 0.1^3 (1 - 0.1)^{10-3} = 0.057 \approx 6\%$$

The probability of exactly 3 successful wells out of 10 trials is therefore approximately 6% in this example.

Poisson Distribution

When the number of trials is $N \rightarrow \infty$ and the success probability is $p \rightarrow 0$, the binomial distribution approaches a *Poisson distribution* with a single parameter $\lambda = N \cdot p$ (Fig. 3.6) (Poisson 1837). This works well for $N > 100$ and $p < 0.05$ (or 5%). We therefore use the Poisson distribution for processes characterized by extremely low occurrence, e.g. earthquakes, volcanic eruptions, storms and floods. The probability density function is

$$f(x) = \frac{e^{-\lambda} \lambda^x}{x!}$$

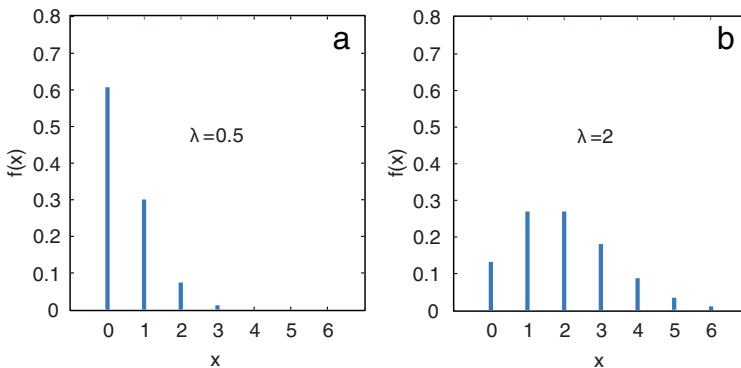


Fig. 3.6 Probability density function $f(x)$ of a Poisson distribution with different values for λ : **a** $\lambda = 0.5$, and **b** $\lambda = 2$.

and the cumulative distribution function is

$$F(x) = \sum_{i=0}^x \frac{e^{-\lambda} \lambda^i}{i!}$$

The single parameter λ describes both the mean and the variance of this distribution.

Normal or Gaussian Distribution

When $p = 0.5$ (symmetric, no skew) and $N \rightarrow \infty$, the binomial distribution approaches a *normal* or *Gaussian distribution* defined by the mean μ and standard deviation σ (Fig. 3.7). The probability density function of a normal distribution is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

and the cumulative distribution function is

$$F(x) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^x \exp\left(-\frac{1}{2}\left(\frac{y-\mu}{\sigma}\right)^2\right) dy$$

The normal distribution is therefore used when the mean is both the most frequent and the most likely value. The probability of deviations is equal in either direction and decreases with increasing distance from the mean.

The *standard normal distribution* is a special member of the normal distribution family that has a mean of *zero* and a standard deviation of *one*. We can transform

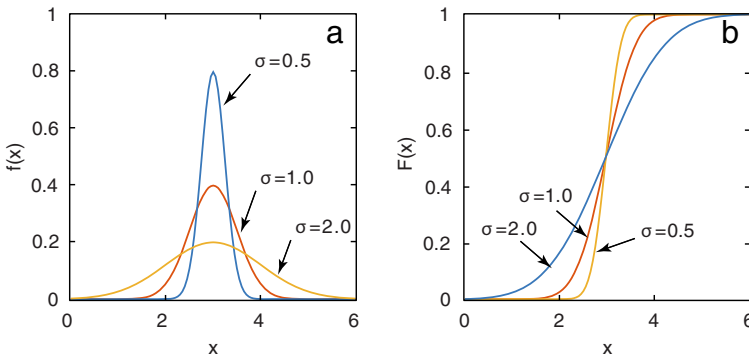


Fig. 3.7 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a Gaussian or normal distribution with a mean $\mu = 3$ and various values for standard deviation σ .

the equation for a normal distribution by substituting $z = (x - \mu)/\sigma$. The probability density function of this distribution is

$$f(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right)$$

This definition of the normal distribution is often called the *z distribution*.

Logarithmic Normal or Log-Normal Distribution

The *logarithmic normal* or *log-normal distribution* is used when the data have a lower limit, e.g. mean-annual precipitation or the frequency of earthquakes (Fig. 3.8). In such cases distributions are usually characterized by significant skewness, which is best described by a logarithmic normal distribution. The probability density function of this distribution is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}x} \exp\left(-\frac{1}{2}\left(\frac{\ln x - \mu}{\sigma}\right)^2\right)$$

and the cumulative distribution function is

$$F(x) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^x \frac{1}{y} \exp\left(-\frac{1}{2}\left(\frac{\ln y - \mu}{\sigma}\right)^2\right) dy$$

where $x > 0$. The distribution can be described by two parameters: the mean μ and the standard deviation σ . The formulas for the mean and the standard deviation, however, are different from the ones used for normal distributions. In practice, the values of x are logarithmized, the mean and the standard deviation are computed using the formulas for a normal distribution, and the empirical distribution is then compared with a normal distribution.

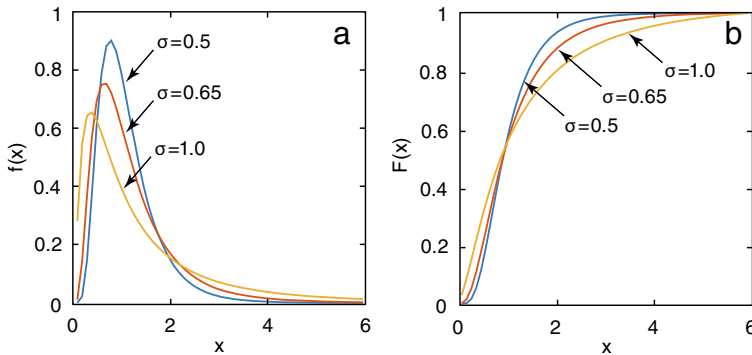


Fig. 3.8 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a logarithmic normal distribution with a mean $\mu = 0$ and with various values for σ .

Student's t Distribution

The *Student's t distribution* was first introduced by William Gosset (1876–1937) who needed a distribution for small samples (Fig. 3.9). Gosset was an employee of the Irish Guinness Brewery and was not allowed to publish research results. For that reason he published his t distribution under the pseudonym *Student* (Student 1908). The probability density function is

$$f(x) = \frac{\Gamma\left(\frac{\Phi+1}{2}\right)}{\Gamma\left(\frac{\Phi}{2}\right)} \frac{1}{\sqrt{\Phi\pi}} \frac{1}{\left(1 + \frac{x^2}{\Phi}\right)^{\frac{\Phi+1}{2}}}$$

where Γ is the Gamma function

$$\Gamma(x) = \lim_{n \rightarrow \infty} \frac{n! n^{x-1}}{x(x+1)(x+2)\dots(x+n-1)}$$

which can be written as

$$\Gamma(x) = \int_0^{\infty} e^{-y} y^{x-1} dy$$

if $x > 0$. The single parameter Φ of the t distribution is the number of degrees of freedom. In the analysis of univariate data this distribution has $n - 1$ degrees of freedom, where n is the sample size. As $\Phi \rightarrow \infty$, the t distribution converges towards the standard normal distribution. Since the t distribution approaches the

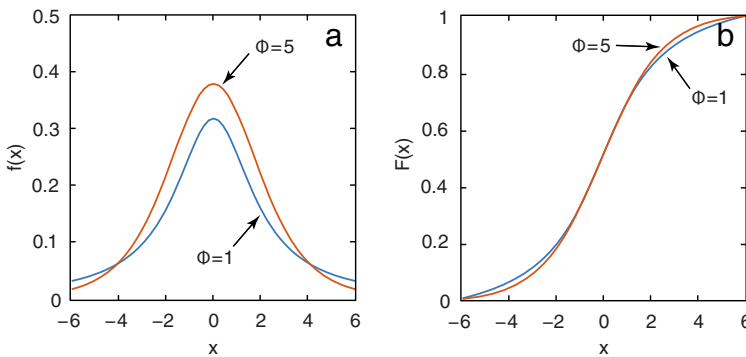


Fig. 3.9 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a Student's t distribution with two different values for Φ .

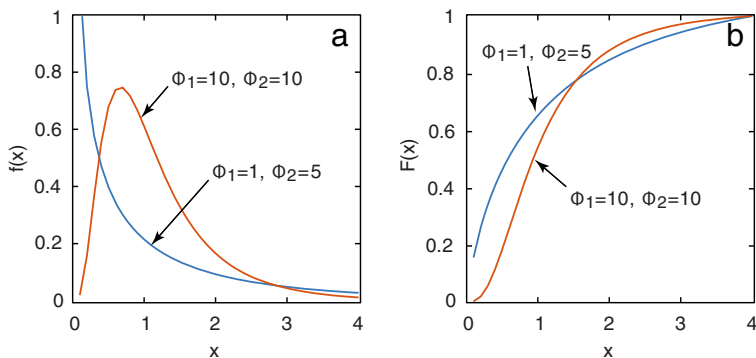


Fig. 3.10 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a Fisher's F distribution with different values for Φ_1 and Φ_2 .

normal distribution for $\Phi > 30$, it is rarely used for distribution fitting. However, the t distribution is used for hypothesis testing using the t -test (Sect. 3.7).

Fisher's F Distribution

The F distribution was named after the statistician Sir Ronald Fisher (1890–1962). It is used for hypothesis testing using the F -test (Sect. 3.8). The F distribution has a relatively complex probability density function (Fig. 3.10):

$$f(x) = \frac{\Gamma\left(\frac{\Phi_1 + \Phi_2}{2}\right) \left(\frac{\Phi_1}{\Phi_2}\right)^{\frac{\Phi_1}{2}}}{\Gamma(\Phi_1/2) \Gamma(\Phi_2/2)} x^{\frac{\Phi_1 - 2}{2}} \left(1 + \frac{\Phi_1}{\Phi_2} x\right)^{-\frac{\Phi_1 + \Phi_2}{2}}$$

where $x > 0$ and Γ is again the Gamma function. The two parameters Φ_1 and Φ_2 are the numbers of degrees of freedom.

χ^2 or Chi-Squared Distribution

The χ^2 distribution was introduced by Helmert (1876) and Pearson (1900). This distribution has important applications in statistical hypothesis testing (Sect. 3.9). The probability density function of the χ^2 distribution is

$$f(x) = \frac{1}{2^{\Phi/2} \Gamma(\Phi/2)} x^{\frac{\Phi - 2}{2}} e^{-\frac{x}{2}}$$

where $x > 0$, otherwise $f(x) = 0$; Γ is again the Gamma function. Once again, Φ is the number of degrees of freedom (Fig. 3.11).

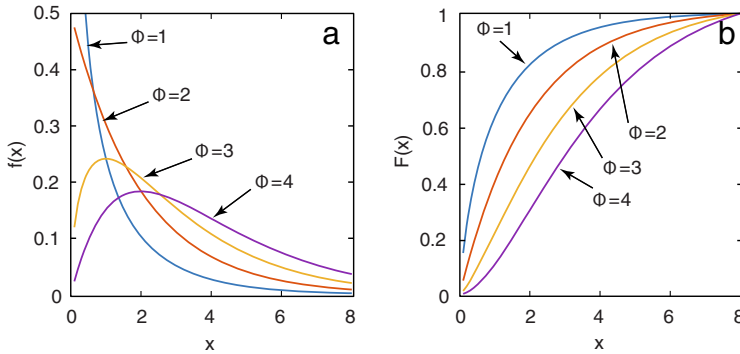


Fig. 3.11 **a** Probability density function $f(x)$, and **b** cumulative distribution function $F(x)$, of a χ^2 distribution with different values for Φ .

3.5 Examples of Theoretical Distributions

The function `randtool` is a tool for simulating discrete data sets with statistics similar to our data set. This function creates a histogram of *random numbers* from the distributions in the Statistics and Machine Learning Toolbox (MathWorks 2020). The random numbers that have been generated using this tool can then be exported into the workspace. We start the *graphical user interface* (GUI) of the function by typing

```
clear, close all, clc
randtool
```

after the prompt. We can now create a data set similar to the one in the file *organicmatter_one.txt*. The 60 measurements have a mean of 12.3448 wt% and a standard deviation of 1.1660 wt%. The GUI uses *Mu* for μ (the mean of a population) and *Sigma* for σ (the standard deviation). After choosing *Normal* for a *Gaussian* distribution and 60 for the number of samples we get a histogram similar to the one in the first example (Sect. 3.3). This synthetic distribution based on 60 measurements represents a rough estimate of the true normal distribution. If we increase the sample size the histogram looks much more like a true Gaussian distribution.

Instead of simulating discrete distributions we can use the probability density function (PDF) or the cumulative distribution function (CDF) to compute a theoretical distribution. MATLAB also provides a GUI-based function for generating PDFs and CDFs with a specific statistic, which is called `disttool`.

```
disttool
```

We choose `Normal` as the distribution and `PDF` as the function type, and then define the mean as `mu = 12.3448` and the standard deviation as `sigma = 1.1660`. Although the function `disttool` is GUI-based, it uses non-GUI functions such as `normpdf` and `normcdf` to calculate the probability density function and the cumulative distribution function. The MATLAB Help gives an overview of the available theoretical distributions. As an example we can use the functions `normpdf(x,mu,sigma)` and `normcdf(x,mu,sigma)` to compute the PDF and CDF of a Gaussian distribution with `mu = 12.3448` and `sigma = 1.1660`, evaluated for the values in `x`, to compare the results with those from our sample data set.

```
clear, close all, clc

mu = 12.3448;
sigma = 1.1660;

x = 5 : 0.001 : 20;
pdf = normpdf(x,mu,sigma);
cdf = normcdf(x,mu,sigma);
plot(x,pdf,x,cdf)
```

We can use these functions to familiarize ourselves with the properties of distributions. This will be important when we deal with the testing of hypotheses in the following sections. The test statistics used there follow the theoretical frequency distributions introduced in the previous sections of this chapter. In particular, the integral (or in the discrete case, the sum) of the theoretical distribution within a certain range $a \leq x \leq b$ is of great importance as it helps in calculating the probability that a measurement will fall within this range.

As an example we can calculate the probability that a measurement will fall within the range of $\mu - \sigma \leq x \leq \mu + \sigma$, where μ is the mean and σ is the standard deviation of a Gaussian distribution. Using the PDF of the Gaussian distribution with `mu = 12.3448` and `sigma = 1.1660`, we find

```
pdf = pdf/sum(pdf);
sum(pdf(find(x>mu-sigma,1,'first'):...
        find(x<mu+sigma,1,'last')))
```

which yields

```
ans =

    0.6827
```

or $\sim 68\%$ after normalizing the PDF to unity. The expression `find(x>mu-sigma,1,'first')` returns the first index of `x` where `x` is larger than `mu-sigma`. Similarly, `find(x<mu+sigma,1,'last')` returns the last index of `x` where `x` is smaller than `mu+sigma`. Using


```
sum(pdf(find(x>mu-sigma):find(x<mu+sigma)))
```

instead yields

```
ans =  
0
```

since the value of $\mu - \sigma$ (11.1788) falls between two values of x , 11.178 and 11.179, similar to the value of $\mu + \sigma$ (13.5108) falling between 13.510 and 13.511. Both, $\text{find}(x == \mu - \sigma)$ and $\text{find}(x == \mu + \sigma)$ return an empty matrix and hence the sum of the $\mu - \sigma \leq x \leq \mu + \sigma$ range is zero. Alternatively, we can calculate the probability of the $-\sigma \leq x \leq +\sigma$ range from the CDF by typing

```
cdf(find(x<mu+sigma,1,'last'))-...  
cdf(find(x>mu-sigma,1,'first'))
```

which yields about the same probability of

```
ans =  
0.6825
```

or $\sim 68\%$. Again,

```
cdf(find(x==mu+sigma))-cdf(find(x==mu-sigma))
```

yields no result

```
ans =  
1×0 empty double row vector
```

for the same reason as before. Conversely, we can also calculate the x -values of the $\mu \pm \sigma$ range of our PDF using the inverse of the cumulative normal distribution function with $\text{norminv}(p, \mu, \sigma)$ by typing

```
norminv((1-0.6827)/2,mu,sigma)  
norminv(1-(1-0.6827)/2,mu,sigma)
```

which yields

```
ans =
    11.1788

ans =
    13.5108
```

Here, the values for p are calculated from the complement of $\sim 68\%$, which is $\sim 32\%$, halved on both tails of the Gaussian distribution, e.g. $(1-0.6827)/2$ and $1-(1-0.6827)/2$.

The standard deviation σ of the Gaussian distribution is important for the definition of confidence intervals. In many examples, however, the confidence of one sigma ($\mu \pm 1\sigma$) or $\sim 68\%$ that the true value falls within the $\mu \pm 1\sigma$ range is not sufficient and higher confidence intervals such as two sigma ($\mu \pm 2\sigma$) and three sigma ($\mu \pm 3\sigma$) intervals are therefore also used. We can calculate the corresponding probabilities that the true value falls within the $\mu \pm 2\sigma$ range and the $\mu \pm 3\sigma$ range by typing

```
sum(pdf(find(x>mu-2*sigma,1,'first'):...
    find(x<mu+2*sigma,1,'last'))))
sum(pdf(find(x>mu-3*sigma,1,'first'):...
    find(x<mu+3*sigma,1,'last'))))
```

which yields

```
ans =
    0.9545

ans =
    0.9973
```

or $\sim 95\%$ and $\sim 99\%$. Again, using `norminv` we can calculate the upper and lower bounds of the two sigma ($\mu \pm 2\sigma$) range

```
norminv(0.05/2,mu,sigma)
norminv(1-0.05/2,mu,sigma)
```

which yields

```
ans =
    10.0595
```

```
ans =
    14.6301
```

and the three sigma ($\mu \pm 3\sigma$) range

```
norminv(0.01/2,mu,sigma)
norminv(1-0.01/2,mu,sigma)
```

which yields

```
ans =
    9.3414

ans =
    15.3482
```

Instead of using `normpdf` and `normcdf`, we can use the general function `makedist` to create a probability distribution object for any type of distribution. As an example we create a probability distribution object for a Gaussian distribution by typing

```
clear
mu = 12.3448;
sigma = 1.1660;
pd = makedist('normal',mu,sigma)
```

which yields

```
pd =
    NormalDistribution

    Normal distribution
        mu = 12.3448
        sigma = 1.166
```

When we type `whos` we can then see that we have created an object with name `pd` and class `prob.NormalDistribution`. Using this object we can determine the $\mu \pm 1\sigma$ range using the probabilities $(1-0.6827)/2$ and $1-(1-0.6827)/2$ with the inverse cumulative distribution function `icdf` of the probability distribution object `pd`:

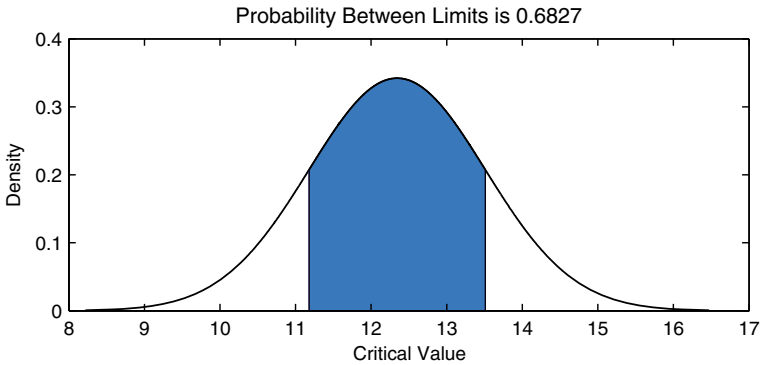


Fig. 3.12 Plot of a standard normal distribution between specified limits. As an example, the shaded area displays the $\mu \pm \sigma$ range of a Gaussian distribution with a mean $\mu = 12.3448$ and a standard deviation $\sigma = 1.1660$.

```
y = icdf(pd,[(1-0.6827)/2,1-(1-0.6827)/2])
```

which yields

```
y =
11.1788 13.5108
```

We can calculate the $\mu \pm 2\sigma$ and $\mu \pm 3\sigma$ ranges in a similar manner. The function `normspec`

```
p = normspec(y,mu,sigma,'inside')
```

which yields

```
p =
0.6827
```

plots the PDF and shades the region inside the $\mu \pm 1\sigma$ range (Fig. 3.12).

3.6 Hypothesis Testing

The following sections are concerned with methods used to draw conclusions from the statistical sample that can then be applied to the larger population of interest (*hypothesis testing*) (Sects. 3.6–3.12). All hypothesis tests share the same concept

and terminology. The *null hypothesis* is an assertion about the population describing the absence of a statistically significant characteristic or effect, whereas an *alternative hypothesis* is a contrasting assertion. The p -value of a hypothesis test is the probability, under the null hypothesis, of observing larger values for the test statistic than those calculated from the sample. The *significance level* α is the threshold of probability that controls the outcome of the tests. If the p -value is smaller than α , the null hypothesis can be rejected; the outcome of the test is regarded as *significant* if $p < 0.05$, or *highly significant* if $p < 0.01$.

A hypothesis test can be performed either as a *one-tailed* (one-sided) or *two-tailed* (two-sided) test. The term *tail* derives from the tailing off the data to the far left or far right of a probability density function as, for instance, in the standard normal distribution used in the Mann-Whitney and Ansari-Bradley tests (Sects. 3.11 and 3.12). As an example, the Mann-Whitney test compares the medians of two data sets. The one-tailed Mann-Whitney test is used to test against the alternative hypothesis that the median of the first sample is either smaller or larger than the median of the second sample at a significance level of 5% (or 0.05). The two-tailed Mann-Whitney test is used when the medians are not equal at a 5% significance level, i.e. when it makes no difference which of the medians is larger. In this case, the significance level is halved, i.e. 2.5% is used instead of 5%.

We can display the standard normal distribution for the one-tailed test by typing

```
clear, close all, clc

pd = makedist('normal',0,1);
y = icdf(pd,[0,.95]);
[p,h] = normspec(y,0,1,'outside');
```

and obtain a plot with one blue tail to the right, with the 5% area shaded (Fig. 3.13a). Similarly, we can display the standard normal distribution for the two-tailed test by typing

```
clear, close all, clc

pd = makedist('normal',0,1);
y = icdf(pd,[.025,.975]);
[p,h] = normspec(y,0,1,'outside');
```

and obtain a plot with two blue tails, one to the left and one to the right, with the 2.5% areas shaded (Fig. 3.13b).

Note that we cannot prove the null hypothesis, in other words *not guilty* is not the same as *innocent*. In practice, we design hypotheses based on our data, test them, and then continue to work with those we could not show to be false. The inherent possibility to prove a hypothesis to be false is therefore an important requirement for our hypotheses.

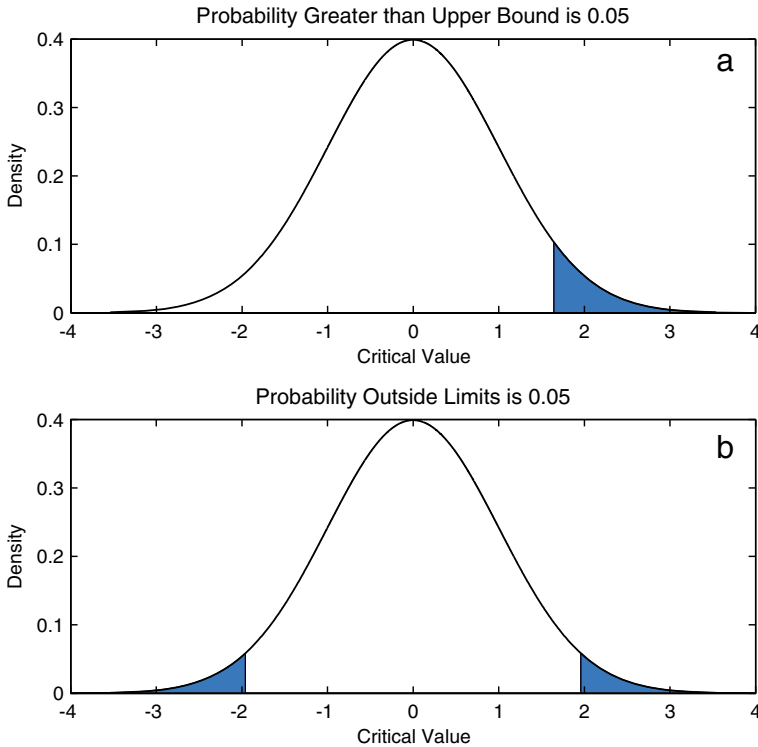


Fig. 3.13 Plot of a standard normal distribution with **a** one, or **b** two, critical regions shown as shaded areas that contain a total of 5% of the area under the curve.

The next sections introduce the most important hypothesis tests for earth science applications: the two-sample t -test to compare the means of two data sets, the two-sample F -test to compare the variances of two data sets, and the χ^2 -test and Kolmogorov-Smirnov test to compare distributions (Sects. 3.7–3.10). The Mann-Whitney and Ansari-Bradley tests are alternatives to the t -test and F -test for comparing the medians and dispersions of two data sets without requiring a normality assumption for the underlying population (Sects. 3.11 and 3.12).

3.7 The t -Test

The Student's t -test by William Sealy Gosset (Student 1908) compares the means of two distributions. The *one-sample t -test* is used to test the hypothesis that the mean of a Gaussian-distributed population has a value specified in the null hypothesis. The *two-sample t -test* is employed to test the hypothesis that the means of two Gaussian distributions are identical. In the following text the two-sample t -test is

introduced to demonstrate hypothesis testing. Let us assume that two independent sets of n_a and n_b measurements have been carried out on the same object, for instance measurements on two sets of rock samples taken from two separate outcrops. The t -test can be used to determine whether both samples come from the same population, e.g. the same lithologic unit (*null hypothesis*) or from two different populations (*alternative hypothesis*). Both sample distributions must be Gaussian and the variances for the two sets of measurements should be similar. The appropriate test statistic for the difference between the two means is then

$$t = \frac{|\bar{a} - \bar{b}|}{\sqrt{\frac{n_a + n_b}{n_a n_b} \cdot \frac{(n_a - 1) \cdot s_a^2 + (n_b - 1) \cdot s_b^2}{n_a + n_b - 2}}}$$

where n_a and n_b are the sample sizes, and s_a^2 and s_b^2 are the variances of the two samples a and b . The null hypothesis can be rejected if the measured t -value is higher than the critical t -value, which depends on the number of degrees of freedom $\Phi = n_a + n_b - 2$ and the significance level α . The one-tailed test is used to test against the alternative hypothesis that the mean of the first sample is either smaller or larger than the mean of the second sample at a significance level of 5% (or 0.05). The one-tailed test would require modification of the above equation by replacing the absolute value of the difference between the means with the actual difference between the means. The two-tailed t -test is used when the means are not equal at a 5% significance level, i.e. when it makes no difference which of the means is larger. In this case, the significance level is halved, i.e. 2.5% is used to compute the critical t -value.

We can now load two example data sets from two independent series of measurements. The first example shows the performance of the two-sample t -test on two distributions with means of 25.5 and 25.3 and standard deviations of 1.3 and 1.5, respectively.

```
clear, close all, clc
load('organicmatter_two.mat');
```

The binary file *organicmatter_two.mat* contains two data sets *corg1* and *corg2*. We first plot both histograms in a single graph.

```
histogram(corg1, 'FaceColor', 'b'), hold on
histogram(corg2, 'FaceColor', 'r'), hold off
```

We then compute the sample sizes, the means and the standard deviations.

```
na = length(corg1); nb = length(corg2);
ma = mean(corg1); mb = mean(corg2);
sa = std(corg1); sb = std(corg2);
```

We then calculate the *t*-value using the translation of the equation for the *t*-test statistic into MATLAB code.

```
tcalc = abs((ma-mb))/sqrt(((na+nb)/(na*nb)) * ...
    (((na-1)*sa^2+(nb-1)*sb^2)/(na+nb-2)))

tcalc =

    0.7279
```

We can now compare the calculated *tcalc* value of 0.7279 with the critical *tcrit* value. This can be accomplished using the function *tinv*, which yields the inverse of the *t* distribution function with *na+nb-2* degrees of freedom at the 5% significance level. This is a two-sample *t*-test, i.e. the means are not equal. Computing the two-tailed critical *tcrit* value by entering $1-0.05/2$ yields the upper (positive) *tcrit* value, which we compare with the absolute value of the difference between the means.

```
tcrit = tinv(1-0.05/2,na+nb-2)

tcrit =

    1.9803
```

Since the *tcalc* value calculated from the data is smaller than the critical *tcrit* value we cannot reject the null hypothesis without another cause. We conclude therefore that the two means are identical at a 5% significance level. Alternatively, we can apply the function *ttest2*(*x,y,alpha*) to the two independent samples *corg1* and *corg2* at an *alpha*=0.05 or a 5% significance level. The command

```
[h,p,ci,stats] = ttest2(corg1,corg2,0.05)
```

yields

```
h =

    0

p =

    0.4681

ci =

   -0.3028
    0.6547
```



```
stats =
    struct with fields:
        tstat: 0.7279
        df: 118
        sd: 1.3241
```

The result $h=0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.4681 or $\sim 47\%$ (which is much greater than the significance level of 0.05 or 5%) suggests that the chances of observing more extreme t -values than the values in this example from similar experiments would be 4,681 in 10,000. The 95% confidence interval on the mean is $[-0.3028, 0.6547]$, which includes the theoretical (and hypothesized) difference between the means of $25.5 - 25.3 = 0.2$.

The second synthetic example shows the performance of the two-sample t -test in an example with very different means of 24.3 and 25.5, while the standard deviations are again 1.3 and 1.5, respectively.

```
clear, close all, clc
load('organicmatter_three.mat');
```

This file again contains two data sets `corg1` and `corg2`. As before, we plot both histograms in a single graph.

```
histogram(corg1,'FaceColor','b'), hold on
histogram(corg2,'FaceColor','r'), hold off
```

We then compute the sample sizes, the means and the standard deviations.

```
na = length(corg1); nb = length(corg2);
ma = mean(corg1); mb = mean(corg2);
sa = std(corg1); sb = std(corg2);
```

We then calculate the t -value using the translation of the equation for the t -test statistic into MATLAB code.

```
tcalc = abs((ma-mb))/sqrt(((na+nb)/(na*nb)) * ...
    (((na-1)*sa^2+(nb-1)*sb^2)/(na+nb-2)))
tcalc =
    4.7364
```

We can now compare the calculated `tcalc` value of 4.7364 with the critical `tcrit` value. Again, this can be accomplished using the function `tinverse` at a 5% significance level. The function `tinverse` yields the inverse of the *t* distribution function with `na-nb-2` degrees of freedom at the 5% significance level. This is again a two-sample *t*-test, i.e. the means are not equal. Computing the two-tailed critical `tcrit` value by entering `1-0.05/2` yields the upper (positive) `tcrit` value that we compare with the absolute value of the difference between the means.

```
tcrit = tinverse(1-0.05/2,na+nb-2)
```

```
tcrit =
```

```
1.9803
```

Since the `tcalc` value calculated from the data is now larger than the critical `tcrit` value, we can reject the null hypothesis and conclude that the means are not identical at a 5% significance level. Alternatively, we can apply the function `ttest2` (`x,y,alpha`) to the two independent samples `corg1` and `corg2` at an `alpha=0.05` or a 5% significance level. The command

```
[h,p,ci,stats] = ttest2(corg1,corg2,0.05)
```

yields

```
h =
```

```
1
```

```
p =
```

```
6.1183e-06
```

```
ci =
```

```
0.7011
```

```
1.7086
```

```
stats =
```

```
struct with fields:
```

```
tstat: 4.7364
```

```
df: 118
```

```
sd: 1.3933
```

The result `h=1` suggests that we can reject the null hypothesis. The *p*-value is extremely low and very close to zero suggesting that the null hypothesis is very unlikely to be true. The 95% confidence interval on the mean is `[0.7011,1.7086]`, which again includes the theoretical difference between the means of $25.5 - 24.3 = 1.2$.

3.8 The F -Test

The two-sample F -test by Snedecor and Cochran (1989) compares the variances sa^2 and sb^2 of two distributions, where $sa^2 > sb^2$. An example is the comparison of the natural heterogeneity of two samples based on replicated measurements. The sample sizes na and nb should be above 30. Both the sample and population distributions must be Gaussian. The appropriate test statistic with which to compare the variances is then

$$\hat{F} = \frac{s_a^2}{s_b^2}$$

The two variances are significantly different (i.e. we can reject the null hypothesis without another cause) if the measured F value is higher than the critical F value, which will in turn depend on the number of degrees of freedom $\Phi a = na - 1$ and $\Phi b = nb - 1$, respectively, and the significance level α . The one-sample F -test, in contrast, virtually performs a χ^2 -test of the hypothesis that the data come from a normal distribution with a specific variance (see Sect. 3.9). We first apply the two-sample F -test to two distributions with very similar standard deviations of 1.2550 and 1.2097.

```
clear, close all, clc
load('organicmatter_four.mat');
```

The quantity F is the quotient of the larger variance divided by the smaller variance. We can now compute the standard deviations, where

```
s1 = std(corg1)
s2 = std(corg2)
```

yields

```
s1 =
    1.2550
s2 =
    1.2097
```

The F distribution has two parameters, $df1$ and $df2$, which represent the number of observations in each of the distributions reduced by one, where

```
df1 = length(corg1) - 1
df2 = length(corg2) - 1
```

yields

```
df1 =
    59
df2 =
    59
```

We then sort the standard deviations by their absolute values,

```
if s1 > s2
  slarger = s1
  ssmaller = s2
else
  slarger = s2
  ssmaller = s1
end
```

and get

```
slarger =
    1.2550
ssmaller =
    1.2097
```

We now compare the calculated F value with the critical F value. This can be accomplished using the function `finv` at a significance level of 0.05 (or 5%). The function `finv` returns the inverse of the F distribution function with `df1` and `df2` degrees of freedom, at the 5% significance level. This is a two-tailed test and we must therefore divide the p -value of 0.05 by two. Typing

```
Fcalc = slarger^2 / ssmaller^2
Fcrit = finv(1-0.05/2,df1,df2)
```

yields

```
Fcalc =
    1.0762

Fcrit =
    1.6741
```

Since the F value calculated from the data is smaller than the critical F value, we cannot reject the null hypothesis without another cause. We therefore conclude that the variances are identical at a 5% significance level. Alternatively, we can apply the function `vartest2(x,y,alpha)` to the two independent samples `corg1` and `corg2` at an $\alpha=0.05$ or a 5% significance level. MATLAB also provides a one-sample variance test `vartest(x,variance)` analogous to the one-sample t -test discussed in the previous section. The one-sample variance test, however, virtually performs a χ^2 -test of the hypothesis that the data in the vector x come from a normal distribution with a variance defined by `variance`. The χ^2 -test is introduced in the next section. The command

```
[h,p,ci,stats] = vartest2(corg1,corg2,0.05)
```

yields

```
h =
    0

p =
    0.7787

ci =
    0.6429
    1.8018

stats =
    struct with fields:
        fstat: 1.0762
        df1: 59
        df2: 59
```

The result $h=0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.7787 or $\sim 78\%$ (which is much greater than the significance level) means that the chances of observing more

extreme values of F than the value in this example from similar experiments would be 7,787 in 10,000. A 95% confidence interval is $[-0.6429, 1.8018]$, which includes the theoretical (and hypothesized) ratio $\text{var}(\text{corg1})/\text{var}(\text{corg2})$ of $1.2550^2/1.2097^2 = 1.0762$.

We now apply this test to two distributions with very different standard deviations, 1.8799 and 1.2939.

```
clear, close all, clc  
  
load('organicmatter_five.mat');
```

We again compare the calculated F_{calc} value with the critical F_{crit} value at a 5% significance level, using the function `finv` to compute F_{crit} .

```
s1 = std(corg1);  
s2 = std(corg2);  
  
df1 = length(corg1) - 1;  
df2 = length(corg2) - 1;  
  
if s1 > s2  
    slarger = s1;  
    ssmaller = s2;  
else  
    slarger = s2;  
    ssmaller = s1;  
end  
  
Fcalc = slarger^2 / ssmaller^2  
  
Fcrit = finv(1-0.05/2, df1, df2)
```

and get

```
Fcalc =  
  
    3.4967  
  
Fcrit =  
  
    1.6741
```

Since the F_{calc} value calculated from the data is now larger than the critical F_{crit} value, we can reject the null hypothesis. The variances are therefore different at a 5% significance level.

Alternatively, we can apply the function `vartest2(x,y,alpha)`, performing a two-sample F -test on the two independent samples `corg1` and `corg2` at an $\alpha=0.05$ or a 5% significance level. Typing

```
[h,p,ci,stats] = vartest2(corg1,corg2,0.05)
```

yields

```
h =
    1
p =
    3.4153e-06
ci =
    2.0887
    5.8539
stats =
    struct with fields:
        fstat: 3.4967
        df1: 59
        df2: 59
```

The result $h=1$ suggests that we can reject the null hypothesis. The p -value is extremely low and very close to zero suggesting that the null hypothesis is very unlikely. The 95% confidence interval is [2.0887, 5.8539], which again includes the theoretical ratio $\text{var}(\text{corg1})/\text{var}(\text{corg2})$ of $5.0717^2/1.4504^2 = 3.4967$.

3.9 The χ^2 -Test

The χ^2 -test introduced by Pearson (1900) involves a comparison of distributions, allowing two distributions to be tested for derivation from the same population. This test is independent of the distribution that is being used and can therefore be used to test the hypothesis that the observations were drawn from a specific theoretical distribution.

Let us assume that we have a data set that consists of multiple chemical measurements from a sedimentary unit. We could use the χ^2 -test to test the null hypothesis that these measurements can be described by a Gaussian distribution with a typical central value and a random dispersion around it. The n data are grouped in K classes, where n should be above 30. The frequencies within the

classes O_k should not be lower than four and should certainly never be zero. The appropriate test statistic is then

$$\hat{\chi}^2 = \sum_{k=1}^K \frac{(O_k - E_k)^2}{E_k}$$

where E_k are the frequencies expected from the theoretical distribution (Fig. 3.14). The null hypothesis can be rejected if the measured χ^2 value is higher than the critical χ^2 value, which depends on the number of degrees of freedom $\Phi = K - Z$, where K is the number of classes and Z is the number of parameters describing the theoretical distribution plus the number of variables (for instance, $Z = 2 + 1$ for the mean and the variance from a Gaussian distribution of a data set for a single variable, and $Z = 1 + 1$ for a Poisson distribution for a single variable).

As an example we can test the hypothesis that our organic carbon measurements contained in *organicmatter_one.txt* follow a Gaussian distribution. We must first load the data into the workspace and compute the frequency distribution `n_obs` for the data measurements using eight classes.

```
clear, close all, clc
corg = load('organicmatter_one.txt');
```

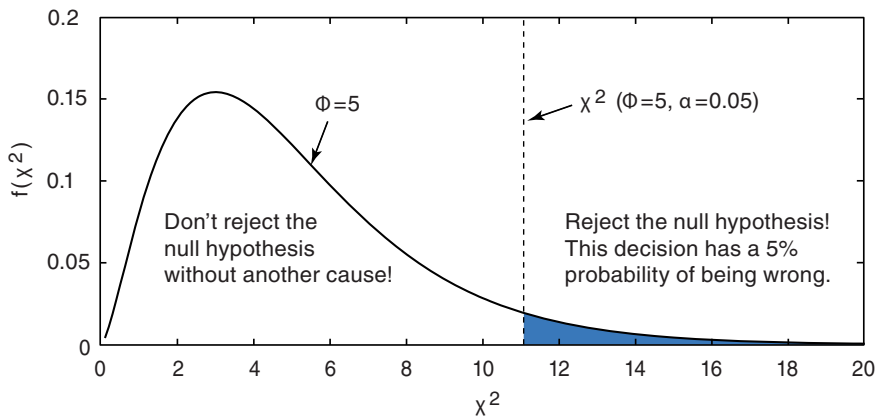


Fig. 3.14 Principles of a χ^2 -test. The alternative hypothesis that the two distributions are different can be rejected if the measured χ^2 is lower than the critical χ^2 . χ^2 depends on $\Phi = K - Z$, where K is the number of classes and Z is the number of parameters describing the theoretical distribution plus the number of variables. In the example the critical $\chi^2(\Phi = 5, \alpha = 0.05)$ is 11.0705. Since the measured $\chi^2 = 5.7602$ is below the critical χ^2 , we cannot reject the null hypothesis. In our example we can conclude that the sample distribution is not significantly different from a Gaussian distribution.


```
h = histogram(corg,8);
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1);
n_obs = h.Values;
```

We then use the function `normpdf` to create the expected frequency distribution `n_exp` with the mean and standard deviation of the data in `corg`.

```
n_exp = normpdf(v,mean(corg),std(corg));
```

The data need to be scaled so that they are similar to the original data set.

```
n_exp = n_exp / sum(n_exp);
n_exp = sum(n_obs) * n_exp;
```

The first command normalizes the observed frequencies `n_exp` to a total of one. The second command scales the expected frequencies `n_exp` to the sum of `n_obs`. We can now display both histograms for comparison.

```
subplot(1,2,1), bar(v,n_obs, 'r')
subplot(1,2,2), bar(v,n_exp, 'b')
```

An alternative way of plotting the data in `corg` is to use a normal probability plot.

```
normplot(corg)
```

The function `normplot` plots the data in `corg` with the blue + symbol superimposed by a red line representing the normal distribution. If the data in `corg` are indeed normally-distributed, they will all fall on the red line. In our example the data seem to agree well with the line except for the tails of the normal distribution.

Visual inspection of these plots reveals that the empirical distribution is similar to the theoretical distribution. It is, however, advisable to use a more quantitative approach to test the hypothesis of similarity. The χ^2 -test explores the squared differences between the observed and expected frequencies. The quantity `chi2calc` is the sum of the squared differences divided by the expected frequencies.

```
chi2calc = sum((n_obs - n_exp).^2 ./ n_exp)

chi2calc =

    5.7602
```

The critical `chi2crit` value can be calculated using `chi2inv` value. The χ^2 -test requires the number of degrees of freedom Φ . In our example we test the hypothesis that the data have a Gaussian distribution, i.e. we estimate the two parameters μ and σ . The number of degrees of freedom is $\Phi = 8 - (2 + 1) = 5$. We can now test our

hypothesis at a 5% significance level. The function `chi2inv` computes the inverse of the χ^2 CDF with parameters specified by Φ for the corresponding probabilities in p .

```
chi2crit = chi2inv(1-0.05,5)

chi2crit =

    11.0705
```

Since the critical `chi2crit` value of 11.0705 is well above the measured `chi2calc` value of 5.7602, we cannot reject the null hypothesis without another cause. We can therefore conclude that our data follow a Gaussian distribution. Alternatively, we can apply the function `chi2gof(x)` to the sample. The command

```
[h,p] = chi2gof(corg)
```

yields

```
h =

    0

p =

    0.6244

stats =

    struct with fields:
        chi2stat: 2.6136
        df: 4
        edges: [1x8 double]
              0: [8 8 5 10 10 9 10]
              E: [1x7 double]
```

The function automatically bins the data into seven classes. The result `h=0` means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.6244 or $\sim 62\%$ (which is much greater than the significance level) means that the chances of observing either the same result or a more extreme result from similar experiments in which the null hypothesis is true would be 6,244 in 10,000. The structure array `stats` contains the calculated χ^2 value, which is 2.6136 and differs from our result of 6.2489 due to the different number of classes. The array `stats` also contains the number of degrees of freedom $\Phi = 7 - (2 + 1) = 4$, the eight edges of the seven classes automatically defined by the function `chi2gof`, and the observed and expected frequencies of the distribution.

3.10 The Kolmogorov-Smirnov Test

The Kolmogorov-Smirnov (K-S) test introduced by Andrei N. Kolmogorov and Nikolai W. Smirnov is similar to the χ^2 -test in that it also involves the comparison of distributions, allowing two distributions to be tested for derivation from the same population (Kolmogorov 1933; Smirnov 1939). This test is independent of the type of distribution that is being used and can therefore be used to test the hypothesis that the observations were drawn from a specific theoretical distribution.

Let us again assume that we have a data set that consists of multiple chemical measurements from a sedimentary unit. We can use the K-S test to test the null hypothesis that these measurements can be described by a Gaussian distribution with a typical central value (the mean) and a random dispersion around the mean (the standard deviation). The appropriate test statistic is then

$$KS = \max|F_n(x) - F(x)|$$

where $F_n(x)$ is the empirical distribution function of the n measurements and $F(x)$ is the cumulative distribution function expected from the theoretical distribution. The null hypothesis can be rejected if the measured KS value is higher than the critical KS value.

As an example we can again test the hypothesis that our organic carbon measurements contained in *organicmatter_one.txt* follow a Gaussian distribution. We must first load and standardize the data to have zero mean and unit standard deviation.

```
clear, close all, clc

corg = load('organicmatter_one.txt');
corg = (corg-mean(corg))/std(corg);
```

We then compute the empirical distribution function `cn_obs` of `corg` evaluated at the points `x` by using the function `ecdf`

```
[cn_obs,x] = ecdf(corg);
```

We then use the function `normcdf` to create the cumulative distribution function expected from the theoretical distribution `cn_exp` with a mean of zero and a standard deviation of one.

```
cn_exp = normcdf(x,0,1);
```

The test statistic is the maximum difference between the two cumulative distribution functions `cn_obs` and `cn_exp`

```
kscal = max(cn_obs-normcdf(x,0,1))
```

which yields

```
kscal =
    0.0757
```

We can compare the two cumulative distribution functions in a plot by typing

```
plot(x,cn_obs,'b'), hold on
plot(x,normcdf(x,0,1), 'r')
plot(x(find((cn_obs-normcdf(x,0,1))== ...
    max(cn_obs-normcdf(x,0,1))))*...
    ones(2,1),[0 1], 'k:')
```

The vertical black line marks the location of the maximum difference between the two cumulative distributions. The critical `kscal` values are solutions of an n -th order polynomial, which can be obtained from Table 3.1 (O'Connor and Kleyner 2012). For sample sizes larger than 40 and a significance level of 0.05 (or 5%) we calculate

```
kscrit = 1.36/length(corg)^0.5
```

which yields

```
kscrit =
    0.1756
```

Since the critical `kscrit` value of 0.1756 is well above the measured `kscal` value of 0.0757, we cannot reject the null hypothesis without another cause. We can therefore conclude that our data follow a Gaussian distribution.

Alternatively, we can apply the function `kstest` to the sample. Typing

```
[h,p,kscal, kscrit] = kstest(corg)
```

yields

```
h =
    logical
    0
p =
    0.8562
```

Table 3.1 Critical values of KS for the Kolmogorov-Smirnov test (O'Connor and Kleyner 2012).

N	Level of Significance, α			
	0.10	0.05	0.02	0.01
1	0.95000	0.97500	0.99000	0.99500
2	0.77639	0.84189	0.90000	0.92929
3	0.63604	0.70760	0.78456	0.82900
4	0.56522	0.62394	0.68887	0.73424
5	0.50945	0.56328	0.62718	0.66853
6	0.46799	0.51926	0.57741	0.61661
7	0.43607	0.48342	0.53844	0.57581
8	0.40962	0.45427	0.50654	0.54179
9	0.38746	0.43001	0.47960	0.51332
10	0.36866	0.40925	0.45662	0.48893
11	0.35242	0.39122	0.43670	0.46770
12	0.33815	0.37543	0.41918	0.44905
13	0.32549	0.36143	0.40362	0.43247
14	0.31417	0.34890	0.38970	0.41762
15	0.30397	0.33760	0.37713	0.40420
16	0.29472	0.32733	0.36571	0.39201
17	0.28627	0.31796	0.35528	0.38086
18	0.27851	0.30936	0.34569	0.37062
19	0.27136	0.30143	0.33685	0.36117
20	0.26473	0.29408	0.32866	0.35241
21	0.25858	0.28724	0.32104	0.34427
22	0.25283	0.28087	0.31394	0.33666
23	0.24746	0.27490	0.30728	0.32954
24	0.24242	0.26931	0.30104	0.32286
25	0.23768	0.26404	0.29516	0.31657
26	0.23320	0.25907	0.28962	0.31064
27	0.22898	0.25438	0.28438	0.30502
28	0.22497	0.24993	0.27942	0.29971
29	0.22117	0.24571	0.27471	0.29466
30	0.21756	0.24170	0.27023	0.28987
31	0.21412	0.23788	0.26596	0.28530
32	0.21085	0.23424	0.26189	0.28094
33	0.20771	0.23076	0.25801	0.27677
34	0.20472	0.22743	0.25429	0.27279
35	0.20185	0.22425	0.26073	0.26897
36	0.19910	0.22119	0.24732	0.26532
37	0.19646	0.21826	0.24404	0.26180
38	0.19392	0.21544	0.24089	0.25843
39	0.19148	0.21273	0.23786	0.25518
40	0.18913	0.21012	0.23494	0.25205
>40	$1.22/N^{0.5}$	$1.36/N^{0.5}$	$1.51/N^{0.5}$	$1.63/N^{0.5}$

```
kscalc =  
  
0.0757  
  
kscrit =  
  
0.1723
```

The result $h=0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.8562 or $\sim 86\%$ (which is much greater than the significance level) means that the chances of observing either the

same result or a more extreme result from similar experiments in which the null hypothesis is true would be 8,562 in 10,000. The output variable `kscalc=0.0757` corresponds to `kscalc` in our experiment without using `kstest`. The output variable `kscrit=1.723` differs slightly from that in Table 3.1 since `kstest` uses a slightly more precise approximation for the critical value for sample sizes larger than 40 from Miller (1956).

3.11 Mann-Whitney Test

The Mann-Whitney test (also known as the Wilcoxon rank-sum test) introduced by Mann and Whitney (1947), can be used to determine whether two samples come from the same population, e.g. the same lithologic unit (*null hypothesis*), or from two different populations (*alternative hypothesis*). In contrast to the *t*-test, which compares the means of Gaussian distributed data, the Mann-Whitney test compares the medians without requiring a normality assumption for the underlying population, i.e. it is a non-parametric hypothesis test.

The test requires that the samples have similar dispersions. We first combine both sets of measurements (samples 1 and 2) and arrange them together in ascending order. We then sum the ranks of samples 1 and 2, where the sum of all ranks is $R = n(n + 1)/2$ with n as the total number of measurements. Published literature is full of different versions of how to calculate the test statistic. Here we use the version that can be found in Hedderich and Sachs (2012, p. 484):

$$U_1 = n_1 n_2 + \frac{n_1(n_1 + 1)}{2} - R_1$$

$$U_2 = n_1 n_2 + \frac{n_2(n_2 + 1)}{2} - R_2$$

where n_1 and n_2 are the sizes of samples 1 and 2, respectively, and R_1 and R_2 are the sums of the ranks of samples 1 and 2, respectively. The required test statistic U is the smaller of the two variables U_1 and U_2 , which we compare with a critical U value that depends on the sample sizes n_1 and n_2 , and on the significance level α . Alternatively, we can use the U value to calculate

$$\hat{z} = \frac{|U - \frac{n_1 n_2}{2}|}{\sqrt{\frac{n_1 n_2 (n_1 + n_2 + 1)}{12}}}$$

if $n_1 \geq 8$ and $n_2 \geq 8$ (Mann and Whitney 1978; Hedderich and Sachs 2012, p. 486). The null hypothesis can be rejected if the absolute measured z -value is higher than the absolute critical z -value, which depends on the significance level α (Sect. 3.4).

In practice, data sets often contain tied values, i.e. some of the values in the sample 1 and/or sample 2 are identical. In this case, the average ranks of the tied values are used instead of the true ranks. This means that the equation for the z -value must be corrected for tied values

$$\hat{z} = \frac{|U - \frac{n_1 n_2}{2}|}{\sqrt{\left(\frac{n_1 n_2}{S(S-1)}\right) \cdot \left(\frac{S^3 - S}{12} - \sum_{i=1}^r \frac{t_i^3 - t_i}{12}\right)}}$$

where $S = n_1 + n_2$, r is the number of tied values and t_i is the number of occurrences of the i -th tied value. Again, the null hypothesis can be rejected if the absolute measured z -value is higher than the absolute critical z -value, which depends on the significance level α .

The MATLAB code presented here has been tested with an example contained in the book by Hedderich and Sachs (2012, p. 489). The example uses the Mann-Whitney test to test whether two samples (data1 and data2), each consisting of eight measurements with some tied values, come from the same population (*null hypothesis*) or from two different populations (*alternative hypothesis*). We clear the workspace and define two samples, each consisting of eight measurements:

```
clear, close all, clc

data1 = [5 5 8 9 13 13 13 15];
data2 = [3 3 4 5 5 8 10 16];
```

We concatenate the two samples horizontally and sort the measurements in ascending order by typing

```
data = horzcat(data1,data2);
data = sort(data)
```

which yields the output

```
data =

Columns 1 through 8

    3     3     4     5     5     5     5     8
```

Columns 9 through 16

```
8      9      10     13     13     13     15     16
```

autowrapping those lines that are longer than the width of the Command Window. We then determine the number of measurements n_1 and n_2 in each sample, which is eight for both of the samples in our example. Then we create a linearly-spaced vector L ranging from 1 to `length(data)`.

```
n1 = length(data1);
n2 = length(data2);
L = 1 : length(data)
```

which yields

```
L =
```

Columns 1 through 8

```
1      2      3      4      5      6      7      8
```

Columns 9 through 16

```
9      10     11     12     13     14     15     16
```

We next find the unique values C in the data. The location vectors ia and ic help to find $C = \text{data}(ia)$ and to reconstruct data from $\text{data} = C(ic)$.

We then search for tied values in data . After we have initialized the variables by setting them all to zero, the tied values in ic are located using `find` and are then stored in $dties$, with the number of occurrences recorded in $nties$. Having located the tied values their ranks are averaged and stored in icf . We type

```
nties = 0;
dties = 0;
kties = 0;

j = 1;
for i = 1 : length(ia)
    f = find(ic==i);
    if length(f) > 1
        dties(j) = max(data(f));
```



```

        nties = nties+1;
        j = j + 1;
    end
    icf(f) = mean(L(f));
end
icf

```

which yields

```

icf =

Columns 1 through 4

    1.5000    1.5000    3.0000    5.5000

Columns 5 through 8

    5.5000    5.5000    5.5000    8.5000

Columns 9 through 12

    8.5000   10.0000   11.0000   13.0000

Columns 13 through 16

   13.0000   13.0000   15.0000   16.0000

```

As we can see, instead of ranks 1 and 2, the averaged ranks 1.5 and 1.5 are used for the two duplicate values of 3 in the data set. Similarly, the averaged rank 5.5 is used for the value of 5, which occurs four times in the data set, instead of the ranks 4, 5, 6 and 7. The averaged rank 8.5 is used for the value of 8 instead of ranks 8 and 9 and the averaged rank 13 is used for the value of 13 instead of ranks 12, 13 and 14. Instead of using the above code MATLAB provides the function `tiedrank` to compute ranks adjusted for tied values:

```
icf = tiedrank(data);
```

The result obtained using `tiedrank` is `icf`, which is identical to that obtained previously. Then we count the repetitions of each tied value by typing

```

for i = 1 : nties
    kties(i) = sum(data == dties(i));
end

```

and display the values of all three variables by typing

```
nties
dties
kties
```

which yields

```
nties =
      4
dties =
      3  5  8 13
kties =
      2  4  2  3
```

We see that we have four different tied values ($nties=4$), with the actual values stored in `dties` (these being the values 3, 5, 8 and 13, which occur 2, 4, 2 and 3 times in the data set, respectively). We now use `ismember` to locate the values in `data1` and `data2` in `data`. The first output from `ismember` for the sample in `data1` is `Lia1`, which contains 1 indicating that the statement that the data in `data1` are found in `data` is true, and the `Lia1` output of `ismember` for the sample in `data1` contains 0 whenever `data1` is not a row in `data`. The second output from `ismember` is the column array `locb1`, which contains the lowest index in `data` for each row in `data1` that is also a row in `data`. We now collect the corresponding ranks in `icf` and sum them up to `R1` and `R2`.

```
[Lia1,locb1] = ismember(data1,data);
r1 = icf(locb1);
R1 = sum(r1)

[Lia2,locb2] = ismember(data2,data);
r2 = icf(locb2);
R2 = sum(r2)
```

which yields

```
R1 =
    83.5000

R2 =
    52.5000
```

We then calculate the test static U of the Mann-Whitney test, which is the smaller of U_1 and U_2

```
U1 = n1*n2 + n1*(n1+1)/2 - R1
U2 = n1*n2 + n2*(n2+1)/2 - R2
U = min([U1 U2])
```

which yields

```
U1 =
    16.5000
U2 =
    47.5000
U =
    16.5000
```

We use U to calculate the measured z -value, corrected for tied values, by typing

```
t = 0;
S = n1 + n2;
for i = 1 : nties
    t(i) = (kties(i)^3 - kties(i)) / 12;
end
T = sum(t);
zcal = abs(U - n1*n2/2) / ...
    sqrt( (n1*n2/(S*(S-1))) * ((S^3-S)/12 - T))
```

which yields

```
zcalc =
    1.6473
```

This is a two-tailed Mann-Whitney test, i.e. the alternative hypothesis is that the medians are not equal, no matter which is larger. Computing the two-tailed critical z_{crit} value using the function `norminv` for the standard normal distribution (with a mean of zero and a standard deviation of one) by entering $1-0.05/2$ yields the upper (positive) z_{crit} value, which we compare with the calculated z_{calc} value

```
zcrit = norminv(1-0.05/2,0,1)
```

which yields

```
zcrit =  
1.9600
```

Since the absolute measured `zcalc` value is 1.6473, which is smaller than the critical `zcrit` value of 1.9600, we cannot reject the null hypothesis. We can therefore conclude that our samples come from the same population. Alternatively, we can use the function `ranksum` to perform a Mann-Whitney test on the same samples:

```
[p,h,stats] = ranksum(data1,data2)
```

which yields

```
p =  
0.1071  
h =  
logical  
0  
stats =  
struct with fields:  
ranksum: 83.5000
```

The result `h=0` means that we cannot reject the null hypothesis at a 5% significance level. The p -value of 0.1071 or $\sim 11\%$ (which is larger than the significance level) means that the chances of observing either the same result or a more extreme result, from similar experiments in which the null hypothesis is true, would be 11 in 100.

We can use the same script to test whether the two samples in *organicmatter_two.mat* come from the same lithological unit (*null hypothesis*) or from two different units (*alternative hypothesis*) without requiring a normality assumption for the underlying population. We clear the workspace and load the data.

```
clear  
load('organicmatter_two.mat');
```

Alternatively, we can use the function `ranksum` to perform a Mann-Whitney test on the same samples:

```
[p,h,stats] = ranksum(corg1,corg2)
```

which yields

```
p =
    0.5906

h =
    logical
    0

stats =

    struct with fields:
        zval: 0.5380
        ranksum: 3733
```

The result $h=0$ means that we cannot reject the null hypothesis at a 5% significance level without another cause. The p -value of 0.5906 or $\sim 59\%$ (which is much larger than the significance level) means that the chances of observing either the same result or a more extreme result from similar experiments in which the null hypothesis is true would be 5,906 in 10,000.

3.12 The Ansari-Bradley Test

The Ansari-Bradley test introduced by Ansari and Bradley (1960) can be used to determine whether two samples come from the same distribution (*null hypothesis*), or whether the samples come from distributions with the same median and shape but different dispersions (e.g. variances) (*alternative hypothesis*). In contrast to the F -test, which compares the dispersions of normally-distributed data, the Ansari-Bradley test compares dispersions without requiring a normality assumption for the underlying population, i.e. it is a non-parametric hypothesis test.

The test requires that the samples have similar medians, which can be achieved by subtracting the medians from the samples. The test combines both sets of measurements (samples 1 and 2) and arranges them together in ascending order. There are different ways to calculate the test statistic A_n . Here we use the one given by Hedderich and Sachs (2012, p. 463)

$$A_n = \sum_{i=1}^n \left(\frac{n+1}{2} - \left| i - \frac{n+1}{2} \right| \right) V_i$$

where the value of the indicator function V_i is 1 for values from sample 1 and 0 for values from sample 2. The test statistic is therefore equal to the sum of the absolute values of the deviations from the mean value $(n+1)/2$ (Hedderich and Sachs 2012).

For this the data are concatenated and sorted, as in the Mann-Whitney test (Sect. 3.11), and the smallest and largest values are then assigned rank 1, the second smallest and second largest values are assigned rank 2, and so forth. The smaller A_n , the larger the dispersion of the values between the two samples 1 and 2. Again, the ranking of the data may also be corrected for tied values, as was previously carried out in the Mann-Whitney test. For $n \leq 20$ we can find the critical values for the test statistic A_n in Table 1 in the open-access article by Ansari and Bradley (1960). For larger values of n we use the standard normal distribution

$$\hat{z} = \frac{A_n - \mu_{A_n}}{\sqrt{\sigma_{A_n}^2}}$$

with

$$\mu_{A_n} = \begin{cases} \frac{1}{4}n_1(n+2), & \text{if } n \text{ is even} \\ \frac{1}{4}n_1(n+1)^2/n, & \text{if } n \text{ is odd} \end{cases}$$

$$\sigma_{A_n} = \begin{cases} n_1n_2(n^2-4)/(48(n-1)), & \text{if } n \text{ is even} \\ n_1n_2(n+1)(n^2+3)/(48n^2), & \text{if } n \text{ is odd} \end{cases}$$

again from Hedderich and Sachs (2012, p. 463). The null hypothesis can be rejected if the absolute measured z -value is higher than the absolute critical z -value, which depends on the significance level α .

Our first example uses the Ansari-Bradley test to test whether two samples `data1` and `data2`, each consisting of eight measurements, come from the same distribution (*null hypothesis*), or from distributions with the same median and shape but different dispersions (*alternative hypothesis*). We first clear the workspace and define two samples.

```
clear, close all, clc

data1 = [7 14 22 36 40 48 49 52];
data2 = [3 5 6 10 17 18 20 39];
```

We next subtract the medians from both data sets as the method requires the medians to be similar.

```
data1 = data1 - median(data1);
data2 = data2 - median(data2);
```

We then concatenate the two samples horizontally and sort the measurements in ascending order by typing

```
data = horzcat(data1,data2);
data = sort(data)
```

which yields

```
data =

Columns 1 through 5
-31.0000  -24.0000  -16.0000  -10.5000  -8.5000

Columns 6 through 10
-7.5000  -3.5000  -2.0000   2.0000   3.5000

Columns 11 through 15
 4.5000   6.5000  10.0000  11.0000  14.0000

Column 16
25.5000
```

We determine the sample sizes for data1 and data2 and then calculate the size n of the concatenated samples.

```
n1 = length(data1);
n2 = length(data2);
n = n1 + n2;
```

We then determine the positions of the measurements in data1 and data2 within the concatenated and sorted vector data using ismember. The function ismember returns an array of the same size as data containing 1 (true) where the elements of data1 are in data, or 0 (false) where they are not. We type

```
V1 = ismember(data,data1)
V2 = ismember(data,data2)
```

which yields

```
V1 =

Columns 1 through 8
    1     1     1     0     0     0     0     1

Columns 9 through 16
    1     0     0     0     1     1     1     0
```

V2 =

Columns 1 through 8

0 0 0 1 1 1 1 0

Columns 9 through 16

0 1 1 1 0 0 0 1

Then we create a linearly-spaced vector L ranging from 1 to length(data).

```
L = 1 : length(data);
```

We then calculate the test static An of the Ansari-Bradley test by typing

```
An = sum(((n+1)/2 - abs(L - (n+1)/2)).* V1)
```

which yields

An =

31

According to Table 1 in Ansari and Bradley (1960), the critical value for $n_1=8$ and $n_2=8$ is 45 at a significance level of 5%. Alternatively, we can use the function `ansaribradley` to perform the Ansari-Bradley test by typing

```
[h,p,stats] = ansaribradley(data1,data2)
```

which yields

h =

0

p =

0.3507

stats =

struct with fields:

W: 31

Wstar: -1.0564

The result $h=0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.3507 or $\sim 35\%$ (which is greater than the significance level) means that the chances of observing either the same

result or a more extreme result from similar experiments in which the null hypothesis is true would be 3,507 in 10,000.

The second example demonstrates the handling of tied values in a data set with a large sample size (>50 measurements). We create such a data set of 100 measurements by using a random number generator:

```
data1 = 3.4 + rand(1,100);
data2 = 4.3 + rand(1,100);
```

We then replace some values in `data1` and `data2` to introduce replicate (or tied) values.

```
data1(1,50:55) = 2.5;
data2(1,25:28) = 2.5;
```

We first subtract the medians from both data sets as the method requires the medians to be similar.

```
data1 = data1 - median(data1);
data2 = data2 - median(data2);
```

The two samples are then concatenated horizontally and sorted in ascending order by typing

```
data = horzcat(data1,data2);
data = sort(data);
```

For the Ansari-Bradley test we now use the function `tiedrank` with `(data,0,1)`, which computes the ranks from each end so that the smallest and largest values are assigned rank 1, the next smallest and largest are assigned rank 2, and so forth.

```
icf = tiedrank(data,0,1);
```

We then calculate the test statistic of the Ansari-Bradley test, which is the sum of the ranks of those measurements that are members of `data1`.

```
An = sum(icf(ismember(data,data1)))
```

which yields

```
An =
    4903
```

Alternatively, we can again use the equation introduced above by typing

```
n1 = length(data1);
n2 = length(data2);
n = n1 + n2;

V1 = ismember(data,data1)
V2 = ismember(data,data2)

An = sum(((n+1)/2 - abs(icf - (n+1)/2)).* V1)
```

which also yields

```
An =

    4903
```

Finally, we calculate the z -value using the equations introduced above by typing

```
if mod(n,2) == 0
    display('n is even')
    muAn = 0.25 * n1 * (n+2);
    siAn = n1*n2*(n^2-4)/((48*(n-1)));
else
    display('n is odd')
    muAn = 0.25 * n1 * (n+1)^2/n;
    siAn = n1*n2*(n+1)*(n^2+3)/(48*n^2);
end

zcalc = (An - muAn)/sqrt(siAn)
```

which yields

```
zcalc =

   -0.7184
```

Since the critical z -value

```
zcrit = norminv(1-0.05/2,0,1)
```

which yields

```
zcrit =

    1.9600
```

is larger than the calculated z -value, we cannot reject the null hypothesis without another cause. Using

```
[h,p,stats] = ansaribradley(data1,data2)
```

yields a similar result

```
h =  
    0  
p =  
    0.4725  
stats =  
    struct with fields:  
        W: 4903  
        Wstar: -0.7184
```

We can use the same script to test whether the two samples in *organicmatter_four.mat* come from the same distribution (*null hypothesis*), or from distributions with the same median and shape but different dispersions (*alternative hypothesis*). We clear the workspace and load the data.

```
clear  
load('organicmatter_four.mat');
```

We use the function `ansaribradley` to perform the Ansari-Bradley test by typing

```
[h,p,stats] = ansaribradley(corg1,corg2)
```

which yields

```
h =  
    0  
p =  
    0.7448  
stats =
```

```
struct with fields:
```

```
W: 1861
```

```
Wstar: 0.3254
```

The result $h=0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. The p -value of 0.7448 or $\sim 74\%$ (which is greater than the significance level) means that the chances of observing either the same result or a more extreme result from similar experiments in which the null hypothesis is true would be 7,448 in 10,000.

3.13 Distribution Fitting

In Sect. 3.9 we computed the mean and the standard deviation of a sample and designed a normal distribution based on these two parameters. We then used the χ^2 -test to test the hypothesis that our data indeed follow a Gaussian or normal distribution. Distribution fitting functions contained in the Statistics and Machine Learning Toolbox (MathWorks 2020) provide powerful tools for estimating the distributions directly from the data. Distribution fitting functions for supported distributions all end with `fit`, as in `binofit`, or `expfit`. The function to fit normal distributions to the data is `normfit`. To demonstrate the use of this function we first generate 100 synthetic Gaussian-distributed sets of values with a mean of 6.4 and a standard deviation of 1.4.

```
clear, close all, clc

rng(0)
data = 6.4 + 1.4*randn(100,1);
```

We then define the midpoints v of nine histogram intervals with the edges e , display the results, and calculate the frequency distribution n .

```
e = 1.5 : 10.5;
h = histogram(data,e);
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1);
n = h.Values;
```

The function `normfit` yields estimates of the mean, `muhat`, and standard deviation, `sigmahat`, of the normal distribution for the observations in `data`.

```
[muhat,sigmahat] = normfit(data)

muhat =

    6.5723

sigmahat =

    1.6274
```

These values for the mean and the standard deviation are similar to the ones that we defined initially. We can now calculate the probability density function of the normal distribution with the mean `muhat` and standard deviation `sigmahat`, scale the resulting function `y` to same total number of observations in `data`, and plot the result.

```
x = 2 : 1/20 : 10;
y = normpdf(x,muhat,sigmahat);
y = trapz(v,n) * y/trapz(x,y);
bar(v,n), hold on, plot(x,y,'r'), hold off
```

Alternatively, we can use the function `mle` to fit a normal distribution (and also other distributions such as binomial or exponential distributions) to the data. The function `mle(data,'distribution',dist)` computes parameter estimates for the distribution specified by `dist`. Acceptable strings for `dist` can be obtained by typing `help mle`.

```
phat = mle(data,'distribution','normal');
```

The variable `phat` contains the values of the parameters describing the type of distribution fitted to the data. As before, we can now calculate and scale the probability density function `y`, and display the result.

```
x = 2 : 1/20 : 10;
y = normpdf(x,phat(:,1),phat(:,2));
y = trapz(v,n) * y/trapz(x,y);

bar(v,n), hold on, plot(x,y,'r'), hold off
```

In earth sciences we often encounter mixed distributions. Examples are multimodal grain size distributions (Sect. 8.10), multiple preferred paleocurrent directions (Sect. 10.6), and multimodal chemical ages in monazite reflecting multiple episodes of deformation and metamorphism in a mountain belt. Fitting Gaussian mixture distributions to the data aims to determine the means and variances of the individual

distributions that combine to produce the mixed distribution. The methods described in this section help to determine the episodes of deformation in a mountain range, or to separate the different paleocurrent directions caused by tidal flow in an ocean basin.

As a synthetic example of Gaussian mixture distributions we generate two sets of 100 random numbers, `ya` and `yb`, with means of 6.4 and 13.3, respectively, and standard deviations of 1.4 and 1.8, respectively. We then vertically concatenate the series using `vertcat` and store the 200 data values in the variable `data`.

```
clear, close all, clc

rng(0)
ya = 6.4 + 1.4*randn(100,1);
yb = 13.3 + 1.8*randn(100,1);
data = vertcat(ya,yb);
```

Plotting the histogram reveals a bimodal distribution. We can also determine the frequency distribution `n` using `histogram`.

```
e = -0.5 : 30.5;
h = histogram(data,e);
v = h.BinWidth * 0.5 + h.BinEdges(1:end-1);
n = h.Values;
```

We use the function `gmdistribution.fit(data,k)` to fit a Gaussian mixture distribution with `k` components to the data. The function fits the model by maximum likelihood, using the *Expectation-Maximization (EM) algorithm*. The EM algorithm introduced by Dempster et al. (1977) is an iterative method alternating between performing an expectation step and a maximization step. The expectation step computes an expectation of the logarithmic likelihood with respect to the current estimate of the distribution. The maximization step computes the parameters that maximize the expected logarithmic likelihood computed in the expectation step. The function `gmdistribution.fit` creates an object of the `gmdistribution` class (see Sect. 2.5 and MATLAB Help on object-oriented programming for details on objects and classes). The function `gmdistribution.fit` treats NaN values as missing data: rows of data with NaN values are excluded from the fit. We can now determine the Gaussian mixture distribution with two components in a single dimension. Typing

```
gmfit = gmdistribution.fit(data,2)
```

yields

```
gmfit =
Gaussian mixture distribution with 2 components in 1 dimensions
Component 1:
Mixing proportion: 0.514361
Mean: 13.0854
Component 2:
Mixing proportion: 0.485639
Mean: 6.4659
```

Thus we obtain the means and relative mixing proportions of both distributions. In our example both normal distributions with means of 6.4659 and 13.0854 contribute $\sim 50\%$ (~ 0.49 and ~ 0.51 , respectively) to the mixture distribution. The object `gmfit` contains several layers of information, including the mean `gmfit.mu` and the standard deviation `gmfit.Sigma` that we use to calculate the probability density function `y` of the mixed distribution.

```
x = 0 : 1/30 : 20;
y1 = normpdf(x, gmfit.mu(1,1), gmfit.Sigma(:,1));
y2 = normpdf(x, gmfit.mu(2,1), gmfit.Sigma(:,2));
```

The object `gmfit` also contains information on the relative mixing proportions of the two distributions in the layer `gmfit.PComponents`. We can use this information to scale `y1` and `y2` to the correct proportions relative to each other.

```
y1 = gmfit.PComponents(1,1) * y1/trapz(x,y1);
y2 = gmfit.PComponents(1,2) * y2/trapz(x,y2);
```

We can now superimpose the two scaled probability density functions `y1` and `y2`, and scale the result `y` to the same integral of 200 as the original data. The integral of the original data is determined using the function `trapz` to perform a trapezoidal numerical integration.

```
y = y1 + y2;
y = trapz(v,n) * y/trapz(x(1:10:end),y(1:10:end));
```

Finally, we can plot the probability density function `y` on the bar plot of the original histogram of data.

```
bar(v,n), hold on, plot(x,y,'r'), hold off
```

We can then see that the Gaussian mixture distribution closely matches the histogram of the data (Fig. 3.15).

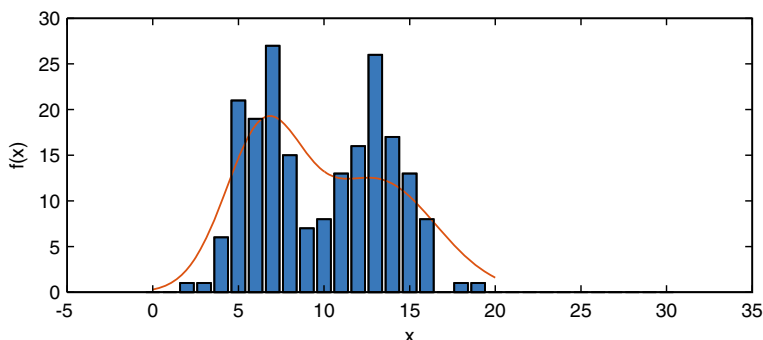


Fig. 3.15 Fitting Gaussian mixture distributions. As a synthetic example of Gaussian mixture distributions we generate two sets of 100 random numbers, with means of 6.4 and 13.3, and standard deviations of 1.4 and 1.8, respectively. The *Expectation-Maximization (EM)* algorithm is used to fit a Gaussian mixture distribution (solid line) with two components to the data (bars).

3.14 Error Analysis

The previous sections on univariate statistics provide a basis for the investigation and evaluation of measurement errors. These errors cannot be avoided but they are of the utmost importance to statistical conclusions. Unfortunately, thorough error analysis is still rare in many areas of the geosciences. This is surprising, if not alarming, because a statement that does not include associated uncertainty is of no value to the recipients of the information. If, for example, the gold content of a rock is given as 6 grams per ton without any confidence limits, an astute investor will hold back on making any commitment. If confidence limits are provided then the investor will be happy to invest, provided they do not exceed ± 2 grams per ton. A confidence interval of ± 8 grams per ton would mean that the actual gold content could be more than twice as high, or it could be zero and therefore a bad investment.

Let us look briefly at the principles of error calculation without becoming too deeply immersed in the subject. This will not be necessary because there are excellent books available on the topic (although without MATLAB examples), such as *An Introduction to Error Analysis* by Taylor (1997). First of all, it is important to understand that there are two different types of measurement errors, not only in geosciences: *random errors* and *systematic errors* (Fig. 3.16). If you try to determine the gold content of a rock with a measuring device, you will only achieve a finite measuring precision. This becomes clear when the gold content of the same sample is determined several times in a row: the results are never identical, but usually scatter around a value, e.g. 6 grams per ton. This is due to technical inaccuracies resulting from imperfections in the measuring instrument, but external influences such as temperature fluctuations and vibrations can also be a factor, as

can errors in recording the measured values (such as reading errors). In the ideal case all these disturbances will be random, so that the distribution of N measured values y_i

$$y_i = (y_1, y_2, \dots, y_N)$$

follows a Gaussian distribution (Sect. 3.4). In this case we can use the sample mean to obtain a best estimate of the gold concentration in the rock:

$$\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

The sample standard deviation s_r describes the *random error* and therefore defines the reliability of the single measurement (Fig. 3.16):

$$s_r = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{y})^2}$$

The standard deviation is the mean square deviation of the individual measurements y_i from the sample mean. Since ± 2 grams per ton is the standard deviation, we know that a single measurement will fall within the 6 ± 2 grams per ton interval with a $\sim 68\%$ probability (see Sect. 3.5). Together with the mean, the standard deviation describes our set of measurements. However, both the mean and the standard deviation of our repeat measurements are only of finite precision, depending on the number of measurements. Thus the Gaussian distribution

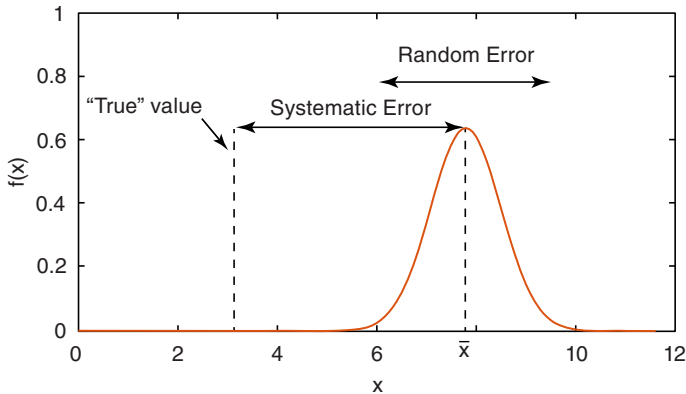


Fig. 3.16 Illustration of the difference between *random error* and *systematic error*. While random errors can be reduced by increasing the sample size, this does have any effect on systematic errors.

calculated from our measured values and displayed as a histogram is only an approximation of the Gaussian distribution that describes the actual gold content and the scatter due to measuring errors.

If we know the standard deviation sr of the single measurement, we can calculate the standard error of the mean depending on the number of measurements N :

$$s_{\bar{x}} = s/\sqrt{N}$$

The standard error of the mean provides an estimate of the error in the sample mean, with respect to the population (or *true*) mean. As we can see, the error of the mean decreases with the square root of the sample size N . Due to the square root relationship, the error first decreases rapidly with increasing N , but then very slowly, i.e. we have to greatly increase the sample size to see any effect. For example, the result of repeated gold content measurements might be given as 6 ± 1.2 grams per ton being the sample mean and the best estimate for the population (*true*) mean, and with ± 1.2 grams per ton being the standard error of the mean, for N measurements. Assuming a normal population distribution, we then know that the true mean has a $\sim 68\%$ probability of falling within the 6 ± 1.2 grams per ton interval (see Sect. 3.5).

In contrast to random errors that can be determined by evaluating repeated measurements, *systematic errors* (s_s) are much harder to detect and quantify. They are usually consistent downward or upward deviations in measured values, caused by an incorrectly calibrated instrument. The equation

$$s_s = \mu - \bar{y}$$

expresses systematic errors as the difference between the (usually unknown) population mean (or *true* value) μ and the sample mean of the measurements (Fig. 3.16).

As an example, when I was a student participant of a mapping course a professor asked us perform measurements with his geological compass. Each student did his/her best, but the professor commented each reading with *wrong*. He was obviously bored with the beginners' course and so he had simply adjusted the scale on his compass by five degrees. None of the students dared to question the accuracy of the professor's compass and all of their readings were therefore wrong because they included a 5 degree systematic error (apart from any random errors). If there was no possibility of detecting this error on the device itself, repeated measurements with another device would have allowed the systematic error to be detected and eliminated. If we know the random and systematic errors of a measurement, we can indicate the *precision* and *accuracy* of our data. Precise measurements are consistent about the mean, but the *true* value maybe different from the mean value of the measurements due to systematic errors that limit the accuracy. Increasing the sample size N increases the precision, but not the accuracy.

We now look at a simple MATLAB experiment to get an understanding of measurement uncertainties. After clearing the workspace and resetting the random number generator, we generate measurement data by randomly sampling a population with a mean of 3.4 and a standard deviation of 1.2. This could mean that the

true gold content of a rock is 3.4 grams per ton, but this value cannot be determined more precisely than ± 1.2 grams per ton due to measurement errors. Because of time and budgetary limitations let us assume that we can only make 12 measurements, which are stored in data:

```
clear, close all, clc

rng(0)
samplesize = 12;
data = 3.4 + 1.5*randn(samplesize,1)
```

which yields

```
data =

    4.2065
    6.1508
    0.0117
    4.6933
    3.8781
    1.4385
    2.7496
    3.9139
    8.7676
    7.5542
    1.3752
    7.9524
```

We display the frequency distribution of the measured values in a histogram and calculate the mean and the standard deviation by typing

```
histogram(data)
mean(data)
std(data)
```

which yields

```
ans =

    4.3910

ans =

    2.7837
```

The error in the mean, which depends on the number of measurements (in this case $N = 12$), is obtained by

```
std(data)/sqrt(samplesize)

ans =

    0.8036
```

When we specify the result there are a number of rules about how many digits to specify. First, the measured value and the measurement uncertainty must have the same number of digits. Second, the number of digits after rounding is defined by the measurement uncertainty. Third, we need to search for the first non-zero digit in the measurement uncertainty, starting from the left. If this digit is greater than 2, then this digit is the rounding digit. If this digit is 1 or 2, then the next digit to the right is the rounding digit. Fourth, the measurement uncertainty is always rounded up. In our example, the first digit of the measurement uncertainty is indeed 2 and therefore we use the first digit to the right of the decimal point, which is 7, as the rounding digit. The result of our measurements is therefore 4.4 ± 2.8 grams per ton, suggesting that the true value falls within this interval with a $\sim 68\%$ probability. As we can easily see from our synthetic example, this value deviates considerably from the original value of 3.4 ± 1.5 , defined at the beginning, which is due to the small sample size of $N = 12$. However, the *true* value of 3.4 still falls within the interval of 4.4 ± 2.8 grams per ton.

Increasing the sample size, however, dramatically improves the result, since the error of the mean depends on the number of measurements:

```
clear
rng(0)
samplesize = 1000;
data = 3.4 + 1.5*randn(samplesize,1);

histogram(data)
mean(data)
std(data)
```

which yields

```
ans =

    3.3511

ans =

    1.4984
```

The error of the mean for $N = 1,000$ measurements is

```
std(data)/sqrt(samplesize)

ans =

    0.0474
```

As expected, the value is much smaller than the corresponding value for $N = 12$. This result suggests that the true value lies in the interval 3.35 ± 0.05 grams per ton. Again, the *true* value of 3.4 still falls within the interval of 3.35 ± 0.05 grams per ton.

Next we look at the case where our value y depends on j measured variables $x_1, x_2, \dots$ and their j uncertainties $s_1, s_2, \dots$, which we use to calculate y and its uncertainty s . We can use Gauss' law of error propagation to calculate s if the uncertainties $s_1, s_2, \dots$ are not independent from each other

$$s = \sqrt{\sum_{j=1}^m \left(\frac{\partial y}{\partial x_j} \cdot s_j \right)^2 + 2 \sum_{j=1}^{m-1} \sum_{k=j+1}^m \left(\frac{\partial y}{\partial x_j} \right) \left(\frac{\partial y}{\partial x_k} \right) \cdot s(x_j, x_k)}$$

wherein $\partial y / \partial x_j$ is the partial derivative of y with respect to the j th variable x_j , s_j is the standard deviation of this variable, and $s(x_j, x_k)$ is the covariance of the uncertainties of the variables x_j and x_k (e.g. Taylor 1997, p. 212). If the variables are not dependent of each other, the covariance term is zero and the equation is reduced to

$$s = \sqrt{\sum_{j=1}^m \left(\frac{\partial y}{\partial x_j} \cdot s_j \right)^2}$$

MATLAB is a great tool with which to simulate error propagation. As an example we will use the software to estimate the uncertainty in the difference y between the measurements of two quantities x_1 and x_2 , based on their individual uncertainties s_1 and s_2 . The experiment shows the difference in uncertainty when the measurements are dependent on each other, and when they are independent of each other. We define a large sample size of 100,000,000 measurements to get very precise values for the errors.

```
clear

samplesize = 100000000;
```

To calculate the difference y between x_1 and x_2 , and the error in this difference, we use two Gaussian distributions with mean values of 6.3 and 3.4, and with standard deviations of 1.2 and 1.5, respectively. We calculate the difference between the

means of the two distributions on the assumption that the two distributions are dependent of each other.

$$\bar{y} = \bar{x}_1 - \bar{x}_2$$

which yields

$$\bar{y} = 6.3 - 3.4 = 2.9$$

with the real variables x_1 and x_2 having standard deviations of s_1 and s_2 , respectively, and a covariance of $s(x_1x_2)$. If we solve the equation for Gauss' law of error propagation from above for our example of a difference between the two means, we then obtain

$$s_f = \sqrt{s_1^2 + s_2^2 - 2 \cdot s(x_1x_2)}$$

which yields

$$s_f = \sqrt{1.5^2 + 1.2^2 - 2 \cdot 1.5 \cdot 1.2} = 0.3$$

which the error in the difference between the means where the two measurements are perfectly dependent on each other. In another MATLAB experiment, we use the same set of random numbers r to add uncertainty to the data.

```
rng(0)
r = randn(samplesize,1);
age1 = 3.4 + 1.2*r;
age2 = 6.3 + 1.5*r;
dage = age2-age1;

figure(...
    'Position',[200 200 600 400],...
    'Color',[1 1 1]);
axes(...
    'Box','on',...
    'Units','Centimeters',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',14); hold on
histogram(dage,30)

mean(dage)
std(dage)
```

which yields

```
ans =
    2.9000
```

and

```
ans =
    0.3000
```

for the difference in the means and its error. We now run the same experiment again but with the two measurements independent of each other. The covariance term in

$$s_f = \sqrt{s_1^2 + s_2^2 - 2 \cdot s(x_1 x_2)}$$

is then zero which yields

$$s_f = \sqrt{1.5^2 + 1.2^2 - 0} \approx 1.92$$

which is the error in the difference between two completely independent measurements. In another MATLAB experiment we use the different sets of random numbers to add uncertainty to the data.

```
rng(0)
age1 = 3.4 + 1.2*randn(samplesize,1);
age2 = 6.3 + 1.5*randn(samplesize,1);
dage = age2-age1;

figure(...
    'Position',[200 800 600 400],...
    'Color',[1 1 1]);
axes(...
    'Box','on',...
    'Units','Centimeters',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',14); hold on
histogram(dage,30)

mean(dage)
std(dage)
```

which yields

```
ans =  
2.8998
```

and

```
ans =  
1.9207
```

for the approximate difference between the means and its error. The small deviations from the exact values used to create the data are due to the limited sample size. As we can see from the example, we either add the same Gaussian random number to the mean or we take different, independent random numbers to simulate the error in the difference y between dependent and independent measurement errors.

Recommended Reading

- Ansari AR, Bradley RA (1960) Rank-sum tests for dispersion. *Ann Math Stat* 31:1174–1189. [Open access]
- Bernoulli J (1713) *Ars Conjectandi*. Reprinted by Ostwalds Klassiker Nr. 107–108. Leipzig 1899
- Dempster AP, Laird NM, Rubin DB (1977) Maximum likelihood from incomplete data via the EM algorithm. *J R Stat Soc Ser B (Methodol)* 39(1):1–38
- Fisher RA (1935) *Design of experiments*. Oliver and Boyd, Edinburgh
- Grabe M (2010) *Generalized Gaussian error calculus*. Springer, Berlin, Heidelberg, New York
- Hedderich J, Sachs L (2012) *Angewandte Statistik: Methodensammlung mit R*, 14th edn. Springer, Berlin Heidelberg New York
- Helmert FR (1876) Über die Wahrscheinlichkeit der Potenzsummen der Beobachtungsfehler und über einige damit im Zusammenhang stehende Fragen. *Zeitschrift für Mathematik und Physik* 21:192–218
- Kolmogorov AN (1933) On the empirical determination of a distribution function. *Italian Giornale dell'Istituto Italiano degli Attuari* 4:83–91
- Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *Ann Math Stat* 18:50–60
- MathWorks (2020) *MATLAB Statistics and machine learning toolbox—user's guide*. The MathWorks, Inc., Natick, MA
- Miller LH (1956) Table of percentage points of Kolmogorov statistics. *J Am Stat Assoc* 51:111–121
- O'Connor PDT, Kleyner A (2012) *Practical reliability engineering*, 5th edn. Wiley, New York
- Pearson K (1900) On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philos Mag* 50:157–175
- Pearson ES (1990) Student—a statistical biography of William sealy Gosset. In: Plackett RL (ed) *With the assistance of Barnard GA*, Oxford University Press, Oxford

- Poisson SD (1837) *Recherches sur la Probabilité des Jugements en Matière Criminelle et en Matière Civile, Précédées des Regles Générales du Calcul des Probabilités*. Bachelier, Imprimeur-Libraire pour les Mathématiques, Paris
- Popper K (1959) *The logic of scientific discovery*. Hutchinson & Co., London
- Smirnov NV (1939) On the estimation of the discrepancy between empirical curves of distribution for two independent samples. *Bull Moscow* 2:3–16
- Spiegel MR (2011) *Schaum's outline of statistics*, 4th edn. Schaum's Outlines, McGraw-Hill Professional, New York
- Student (1908) On the probable error of the mean. *Biometrika* 6:1–25
- Taylor JR (1997) *An introduction to error analysis: the study of uncertainties in physical measurements*, 2nd edn. University Science Books, Sausalito, California
- Wilcoxon F (1945) Individual comparisons by ranking methods. *Biometrics Bulletin* 1:80–83

4.1 Introduction

Bivariate analysis is used to obtain an understanding of the relationship between two variables, x and y . Examples are the length and width of a fossil, the sodium and potassium content of volcanic glass, and the organic matter content along a sediment core. When the two variables are measured on the same object, x is usually identified as the *independent variable* and y as the *dependent variable*. If both variables have been generated in an experiment, the variable manipulated by the experimenter is described as the independent variable. In some cases neither variable is manipulated and neither is independent.

Bivariate statistics are used to describe the strength of a relationship between two variables, either with a single parameter such as Pearson's correlation coefficient for linear relationships, or with an equation obtained by regression analysis (Fig. 4.1). The equation describing the relationship between x and y can be used to predict the y -response from any arbitrary x within the range of the original data values used for the regression analysis. This is of particular importance if one of the two parameters is difficult to measure. In such a case, the relationship between the two variables is first determined by regression analysis on a small training set of data. The regression equation can then be used to calculate the second parameter.

This chapter first introduces correlation coefficients (Sect. 4.2), and then explains the widely-used methods of linear and nonlinear regression analysis (Sects. 4.3, 4.9–4.11). A selection of other methods that are also used to assess the uncertainties in regression analysis are also explained (Sects. 4.4–4.8). All methods are illustrated by means of synthetic examples since these provide an excellent

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_4) contains supplementary material, which is available to authorized users.

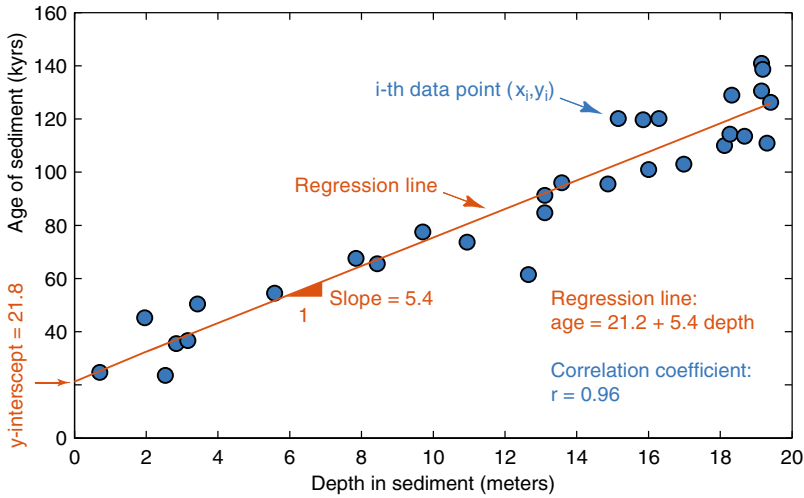


Fig. 4.1 Display of a bivariate data set. The thirty data points represent the *age* of a sediment (in kiloyears before present) at a certain *depth* (in meters) below the sediment-water interface. The combined distribution of the two variables suggests a linear relationship between *age* and *depth*, i.e. the rate of increase in the sediment age with depth is constant. A Pearson's correlation coefficient (explained in the text) of $r = 0.96$ supports a strong linear interdependency between the two variables. Linear regression yields the equation $\text{age} = 21.2 + 5.4 \text{ depth}$, indicating an increase in sediment age of 5.4 kyrs per meter of sediment depth (the slope of the regression line).

means of assessing the final outcome. We use the Statistics and Machine Learning Toolbox (MathWorks 2020), which contains all the necessary routines for bivariate analysis.

4.2 Correlation Coefficients

Correlation coefficients are often used in the early stages of bivariate analysis. They provide only a very rough estimate of a rectilinear trend in a bivariate data set. Unfortunately, the literature is full of examples where the importance of correlation coefficients is overestimated, or where outliers in the data set lead to an extremely biased estimation of the population correlation coefficient.

The most popular correlation coefficient is *Pearson's linear product-moment correlation coefficient* ρ (Pearson 1895) (Fig. 4.2). We estimate the population's correlation coefficient ρ from the sample data, i.e. we compute the sample correlation coefficient r , which is defined as

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n-1)s_x s_y}$$

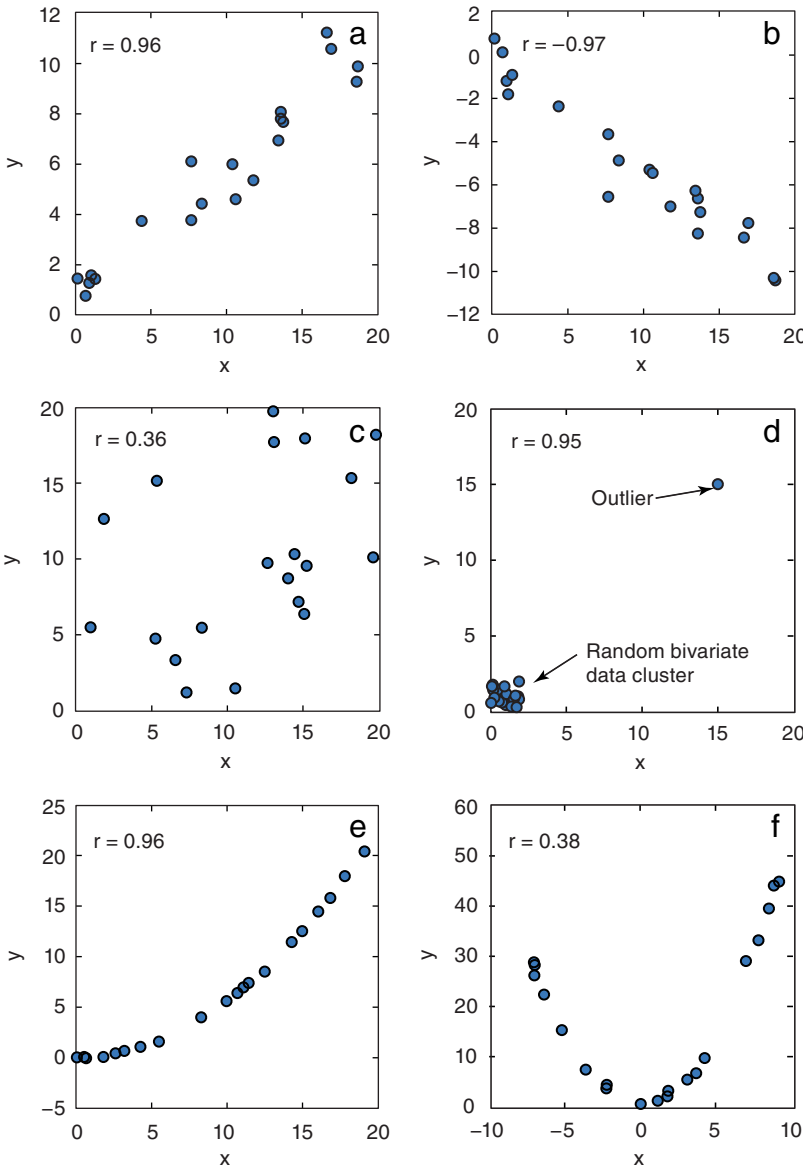


Fig. 4.2 Pearson’s correlation coefficient r for various sample data sets. **a, b** Positive and negative linear correlation, **c** random scatter with no linear correlation, **d** an outlier causing a misleading value of r , **e** curvilinear relationship causing a high r since the curve is close to a straight line, **f** curvilinear relationship clearly not described by r . Modified from Swan and Sandilands (1995).

where n is the number of pairs xy of data points, s_x and s_y are the univariate standard deviations. The numerator of Pearson's correlation coefficient is known as the *corrected sum of products* of the bivariate data set. Dividing the numerator by $(n - 1)$ yields the *covariance*

$$\text{cov}_{xy} = \frac{1}{(n - 1)} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

which is the summed product of deviations in the data from the sample means, divided by $(n - 1)$. The covariance is a widely-used measure in bivariate statistics although it has the disadvantage of being dependent on the dimensions of the data. Dividing the covariance by the univariate standard deviations removes this dependency and leads to Pearson's correlation coefficient.

A popular way to test the significance of Pearson's correlation coefficient is to determine the probability of an r -value for a random sample from a population with a $\rho = 0$. The significance of the correlation coefficient can be estimated using a t -statistic

$$t = r \sqrt{\frac{n - 2}{1 - r^2}}$$

The correlation coefficient is significant if the calculated t is greater than the critical t ($n - 2$ degrees of freedom, $\alpha = 0.05$). This test is, however, only valid if both variables are Gaussian distributed.

Pearson's correlation coefficient is very sensitive to disturbances in the bivariate data set. Several alternatives exist to Pearson's correlation coefficient, such as *Spearman's rank correlation coefficient* proposed by the English psychologist Charles Spearman (1863–1945). Spearman's coefficient can be used to measure statistical dependence between two variables without requiring a normality assumption for the underlying population, i.e. it is a non-parametric measure of correlation (Spearman 1904, 1910). Furthermore, since it uses the ranks of the values in x and y rather than the actual numerical values, it can be used to find correlations in nonlinear data, and even in non-numerical data such as fossil names or rock types in stratigraphic sequences. Having replaced the numerical values in x and y by their ranks (whereby multiple values in x and y are replaced by their respective average ranks), the sample Spearman's rank correlation coefficient is defined as

$$r_{xy} = \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

where d_i is the difference between the ranks of the two variables. Since this correlation coefficient is based on ranks rather than numerical values it is less sensitive to outliers than Pearson's correlation coefficient.

Another alternative to Pearson's correlation coefficient is the *Kendall's tau rank correlation coefficient* proposed by the British statistician Maurice Kendall (1907–1983). This is also a non-parametric measure of correlation, similar to the Spearman's rank correlation coefficient (Kendall 1938). The Kendall's tau rank correlation coefficient compares the ranks of the numerical values in x and y , which means a total of $0.5 n(n - 1)$ pairs to compare. Pairs of observations (x_i, y_i) and (x_j, y_j) are said to be concordant if the ranks for both observations are the same, and discordant if they are not. The sample Kendall's tau rank correlation coefficient is defined as

$$\tau = \frac{(P - Q)}{\frac{1}{2}n(n - 1)}$$

where P is the number of concordant pairs and Q is the number of discordant pairs. Kendall's correlation coefficient typically has a lower value than Spearman's correlation coefficient.

The following example illustrates the use of the correlation coefficients and highlights the potential pitfalls when using these measures of linear trends. It also illustrates the resampling methods that can be used to explore the confidence level of the estimate for ρ . The synthetic data consist of two variables, the age of a sediment in kiloyears (kyrs) before present and the depth below the sediment-water interface in meters. The use of synthetic data sets has the advantage that we fully understand the linear model behind the data.

The data are represented as two columns contained in file *agedepth_1.txt*. These data have been generated using a series of thirty random levels (in *meters*) below the sediment surface. The linear relationship $age = 5.6 \text{ meters} + 20$ was used to compute noise-free values for the variable *age*. This is the equation of a straight line with a slope of 5.6 and an intercept with the y -axis of 20. Some Gaussian noise with a mean of zero and a standard deviation of ten has been added to the *age* data.

```
clear, close all, clc

rng(0)
meters = 20*rand(30,1);
age = 5.6*meters + 20;
age = age + 10*randn(length(meters),1);

plot(meters,age,'o')
axis([0 20 0 140])

agedepth(:,1) = meters;
agedepth(:,2) = age;
agedepth = sortrows(agedepth,1);

save agedepth_1.txt agedepth -ascii
```

The synthetic bivariate data set can be loaded from the file *agedepth_1.txt*.

```
clear  
  
agedepth = load('agedepth_1.txt');
```

We then define two new variables, `meters` and `age`, and generate a scatter plot of the data.

```
meters = agedepth(:,1);  
age = agedepth(:,2);  
  
plot(meters,age,'o')  
axis([0 20 0 140])
```

We can observe a strong linear trend in the plot suggesting some interdependency between the two variables, `meters` and `age`. This trend can be described by Pearson's correlation coefficient r , where $r = 1$ indicates a perfect positive correlation (i.e. `age` increases with `meters`), $r = 0$ suggests no correlation, and $r = -1$ indicates a perfect negative correlation. We use the function `corrcoef` to compute Pearson's correlation coefficient.

```
corrcoef(meters,age)
```

which results in the output

```
ans =  
  
    1.0000    0.9563  
    0.9563    1.0000
```

The function `corrcoef` calculates an array (or matrix) of Pearson's correlation coefficients for all possible combinations of the two variables `age` and `meters`. The value of $r = 0.9563$ suggests that the two variables `age` and `meters` are strongly dependent on each other.

Pearson's correlation coefficient is, however, highly sensitive to outliers, as can be illustrated by the following example. Let us generate a normally-distributed cluster of thirty data points with a mean of zero and a standard deviation of one. To obtain identical data values we reset the random number generator by using the integer 10 as seed.

```
clear  
  
rng(10)  
x = randn(30,1); y = randn(30,1);  
  
plot(x,y,'o'), axis([-1 20 -1 20]);
```

As expected, the correlation coefficient for these random data is very low.

```

corrcoef(x,y)

ans =

    1.0000    0.0302
    0.0302    1.0000

```

We now introduce a single outlier to the data set in the form of an exceptionally high (x,y) value, in which $x = y$. The correlation coefficient for the bivariate data set including the outlier $(x,y) = (5,5)$ is much higher than before.

```

x(31,1) = 5; y(31,1) = 5;

plot(x,y,'o'), axis([-1 20 -1 20]);

corrcoef(x,y)

ans =

    1.0000    0.5022
    0.5022    1.0000

```

Increasing the absolute (x,y) values for this outlier results in a dramatic increase in the correlation coefficient.

```

x(31,1) = 10; y(31,1) = 10;

plot(x,y,'o'), axis([-1 20 -1 20]);

corrcoef(x,y)

ans =

    1.0000    0.7981
    0.7981    1.0000

```

which reaches a value close to $r = 1$ if the outlier has a value of $(x,y) = (20,20)$.

```

x(31,1) = 20; y(31,1) = 20;

plot(x,y,'o'), axis([-1 20 -1 20]);

corrcoef(x,y)

ans =

    1.0000    0.9403
    0.9403    1.0000

```

We can compare the sensitivity of Pearson's correlation coefficient with that of Spearman's correlation coefficient and Kendall's correlation coefficient using the function `corr`. In contrast to `corrcoef`, this function does not calculate correlation

matrices that we can use later (e.g. in Chap. 9) for calculating correlations within multivariate data sets. We type

```
r_pearson = corr(x,y, 'Type', 'Pearson')
r_spearman = corr(x,y, 'Type', 'Spearman')
r_kendall = corr(x,y, 'Type', 'Kendall')
```

which yields

```
r_pearson =
    0.9403

r_spearman =
    0.1343

r_kendall =
    0.0753
```

and observe that the alternative measures of correlation result in reasonable values, in contrast to the absurd value for Pearson's correlation coefficient that mistakenly suggests a strong interdependency between the variables. Although outliers are easy to identify in a bivariate scatter, erroneous values can easily be overlooked in large multivariate data sets (Chap. 9).

Various methods exist to calculate the significance of Pearson's correlation coefficient. The function `corrcoef` also includes the possibility of evaluating the quality of the result. The p -value is the probability of obtaining a correlation as large as the observed value by random chance, when the true correlation is zero. If the p -value is small, then the correlation coefficient r is significant.

```
[r,p] = corrcoef(x,y)

r =
    1.0000    0.9403
    0.9403    1.0000

p =
    1.0000    0.0000
    0.0000    1.0000
```

In our example the p -value is close to zero suggesting that the correlation coefficient is significant. We conclude from this experiment that this particular significance test fails to detect correlations attributed to an outlier. We therefore try an alternative t -test statistic to determine the significance of the correlation between x and y . According to this test, we can reject the null hypothesis that there is no correlation if the calculated t is larger than the critical t ($n - 2$ degrees of freedom, $\alpha = 0.05$).

```

tcalc = r(2,1) * ((length(x)-2)/(1-r(2,1)^2))^0.5
tcrit = tinv(0.95,length(x)-2)

tcalc =

    14.8746

tcrit =

    1.6991

```

This result indeed indicates that we can reject the null hypothesis and therefore there is no correlation.

As an alternative to detecting outliers, *resampling schemes* or *surrogates* such as the *bootstrap* or *jackknife* methods represent powerful tools for assessing the statistical significance of the results. These techniques are particularly useful when scanning large multivariate data sets for outliers (see Chap. 9). Resampling schemes repeatedly resample the original data set of n data points, either by choosing $n - 1$ subsamples n times (the jackknife), or by picking an arbitrary set of subsamples with n data points *with replacement* (the bootstrap). The statistics of these subsamples provide better information on the characteristics of the population than the statistical parameters (mean, standard deviation, correlation coefficients) computed from the full data set. The function `bootstrp` allows resampling of our bivariate data set, including the outlier $(x,y) = (20,20)$.

```

rng(0)
rhos1000 = bootstrp(1000,'corrcoef',x,y);

```

This command first resamples the data a thousand times; it then calculates the correlation coefficient for each new subsample and stores the result in the variable `rhos1000`. Since `corrcoef` delivers a 2-by-2 matrix (as mentioned above), `rhos1000` has the dimensions of 1000-by-4, i.e. 1000 values for each element of the 2-by-2 matrix. Plotting the histogram of the 1000 values for the second element, i.e. the correlation coefficient of (x,y) , illustrates the dispersion of this parameter with respect to the presence or absence of the outlier. Since the distribution of `rhos1000` contains many empty classes, we use a large number of bins.

```

histogram(rhos1000(:,2),30)

```

The histogram shows a cluster of correlation coefficients at around $r = 0$ that follow the normal distribution, and a strong peak close to $r = 1$ (Fig. 4.3). The interpretation of this histogram is relatively straightforward. When the subsample contains the outlier the correlation coefficient is close to one, but subsamples without the outlier yield a very low (close to zero) correlation coefficient suggesting the absence of any strong interdependence between the two variables x and y .

Bootstrapping therefore provides a simple but powerful tool for either accepting or rejecting our first estimate of the correlation coefficient for the population. The application of the above procedure to the synthetic sediment data yields a clear unimodal Gaussian distribution for the correlation coefficients of the subsamples.

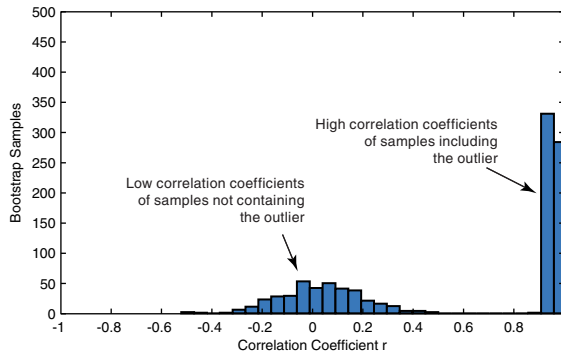


Fig. 4.3 Bootstrap result for Pearson's correlation coefficient r from 1000 subsamples. The histogram shows a roughly normally-distributed cluster of correlation coefficients at around $r = 0$, suggesting that these subsamples do not include the outlier. The strong peak close to $r = 1$, however, suggests that an outlier with high values for the two variables x and y is present in the corresponding subsamples.

```
clear

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

corrcoef(meters,age)

ans =

    1.0000    0.9563
    0.9563    1.0000

rng(0)
rhos1000 = bootstrap(1000,'corrcoef',meters,age);

histogram(rhos1000(:,2),30)
```

Most of the values for `rhos1000` fall within the interval $[0.92, 0.98]$. Since the correlation coefficients for the resampled data sets (in our example) have an approximately normal distribution, we can use their mean as a good estimate for the true correlation coefficient.

```
mean(rhos1000(:,2))

ans =

    0.9557
```

This value is similar to our first result of $r = 0.9557$, but now we have confidence in the validity of this result. In our example, however, the distribution of the bootstrap estimates of the correlations from the age-depth data is quite skewed, as the upper limit is fixed at one. Nevertheless, the bootstrap method is a valuable tool for assessing the reliability of Pearson's correlation coefficient for bivariate analysis.

4.3 Classical Linear Regression Analysis

Linear regression offers another way of describing the relationship between the two variables x and y . Whereas Pearson's correlation coefficient provides only a rough measure of a linear trend, linear models obtained by regression analysis allow the prediction of arbitrary y -values for any given value of x within the data range. Statistical testing of the significance of the linear model provides some insights into the accuracy of these predictions.

Classical regression assumes that y responds to x and that the entire dispersion in the data set is contained within the y -value (Fig. 4.4). This means that x is then the independent variable (also known as the predictor variable, or the regressor). The values of x are defined by the experimenter and are often regarded as being free of errors. An example is the location x within a sediment core from which the variable y has been measured. The dependent variable y contains errors as its magnitude cannot be determined accurately. Linear regression minimizes the deviations Δy between the data points xy and the value y predicted by the best-fit line $y = b_0 + b_1x$ using a least-squares criterion. The basic equation for a general linear model is

$$y = b_0 + b_1x$$

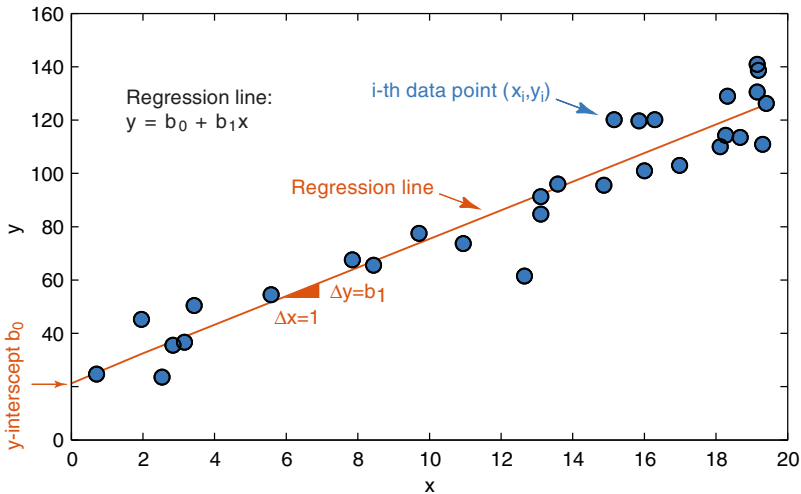


Fig. 4.4 Linear regression. Whereas classical regression minimizes the Δy deviations, reduced major axis regression minimizes the triangular area $0.5 * (\Delta x \Delta y)$ between the data points and the regression line, where Δx and Δy are the distances between the predicted and the true x - and y -values. The intercept of the line with the y -axis is b_0 , and the slope is b_1 . These two parameters define the equation of the regression line.

where b_0 and b_1 are the regression coefficients. The value of b_0 is the intercept with the y -axis and b_1 is the slope of the line. The squared sum of the Δy deviations to be minimized is

$$\sum_{i=1}^n (\Delta y_i)^2 = \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2$$

Partial differentiation of the right-hand term in the equation and setting it to zero yields a simple equation for the regression coefficient b_1 :

$$b_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

The regression line passes through the data centroid defined by the sample means, and we can therefore compute the other regression coefficient b_0 ,

$$b_0 = \bar{y} - b_1 \bar{x}$$

using the univariate sample means and the slope b_1 computed earlier.

As an example let us again load the synthetic age-depth data from the file *agedepth_1.txt*. We can define two new variables, *meters* and *age*, and generate a scatter plot of the data.

```
clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);
```

A significant linear trend in the bivariate scatter plot, together with a correlation coefficient greater than $r = 0.9$, suggests a strong linear dependence between *meters* and *age*. In geological terms this implies that the sedimentation rate was constant through time. We now try to fit a linear model to the data that will help us predict the age of the sediment at levels for which we have no age data. The function `polyfit` computes the coefficients of a polynomial $p(x)$ of a specific degree that fits the data y in a least-squared sense. In our example we fit a first degree (linear) polynomial to the data

```
p = polyfit(meters,age,1)

p =
    5.3667    21.7607
```

where p is a row vector containing the polynomial coefficients in descending powers. Since we are working with synthetic data we know the values for the slope and the intercept with the y -axis. The estimated slope (5.3667) and the intercept with the y -axis (21.7607) are in good agreement with the true values of 5.6 and 20, respectively. Both the data and the fitted line can be plotted on the same graph.

```
plot(meters,age,'o'), hold on
plot(meters,p(1)*meters+p(2),'r'), hold off
```

Instead of using the equation for the regression line we can also use the function `polyval` to calculate the y -values.

```
plot(meters,age,'o'), hold on
plot(meters,polyval(p,meters),'r'), hold off
```

Both of the functions `polyfit` and `polyval` are incorporated in the GUI function `polytool`.

```
polytool(meters,age)
```

The coefficients $p(x)$ and the equation obtained by linear regression can now be used to predict y -values for any given x -value. However, we can only do this within the depth interval for which the linear model was fitted, i.e. between 0 and 20 m. As an example the age of the sediment at a depth of 17 m is given by

```
polyval(p,17)

ans =

    112.9946
```

This result suggests that the sediment at 17 m depth has an age of ca. 113 kyrs. The goodness-of-fit of the linear model can be determined by calculating error bounds. These are obtained by using an additional output parameter s from `polyfit` as an input parameter for `polyconf` to calculate the 95% ($\alpha = 0.05$) prediction intervals.

```
[p,s] = polyfit(meters,age,1);
[p_age,delta] = polyconf(p,meters,s,'alpha',0.05);

plot(meters,age,'o',...
     meters,p_age,'g-',...
     meters,p_age+delta,'r--',...
     meters,p_age-delta,'r--')
axis([0 20 0 140]), grid on
xlabel('Depth in Sediment (meters)')
ylabel('Age of Sediment (kyrs)')
```

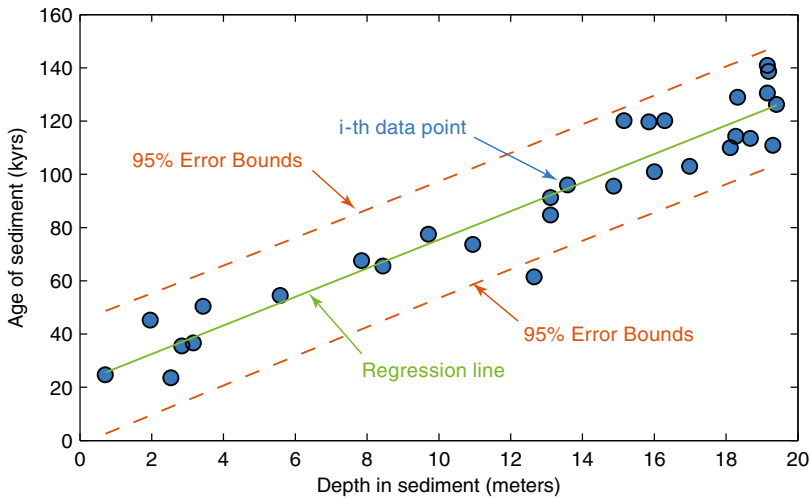


Fig. 4.5 The result of linear regression. The plot shows the original data points (circles), the regression line (solid line), and the error bounds (dashed lines) of the regression. Note that the error bounds are actually curved although they seem to be almost straight lines in the example.

The variable `delta` provides an estimate for the standard deviation of the error in predicting a future observation at x by using $p(x)$. Since the `plot` statement does not fit on one line we use an *ellipsis* (three periods, i.e. ...) followed by `return` or `enter` to indicate that the statement continues on the next line. The plot now shows the data points, and also the regression line and the error bounds of the regression (Fig. 4.5). This graph already provides some valuable information on the quality of the result. However, in many cases a better understanding of the validity of the model is required and more sophisticated methods for testing the confidence in the results are therefore introduced in the following sections.

4.4 Analyzing the Residuals

When we compare how much the predicted values vary from the actual or observed values, we are performing an analysis of the residuals. The statistics of the residuals provide valuable information on the quality of a model fitted to the data. For instance, a significant trend in the residuals suggests that the model does not fully describe the data. In such cases a more complex model, such as a polynomial of a higher degree, should be fitted to the data. Residuals are ideally purely random, i.e. they are Gaussian distributed with a mean of zero. We therefore test the hypothesis that our residuals are Gaussian distributed by visual inspection of the histogram and by employing a χ^2 -test, as introduced in Chap. 3.

```
clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

p = polyfit(meters,age,1);

res = age - polyval(p,meters);
```

Since plotting the residuals does not reveal any obvious pattern of behavior, no more complex model than a straight line should be fitted to the data.

```
plot(meters,res,'o')
```

An alternative way to plot the residuals is as a stem plot using `stem`.

```
subplot(2,1,1)
plot(meters,age,'o'), hold on
plot(meters,p(1)*meters+p(2),'r'), hold off

subplot(2,1,2)
stem(meters,res);
```

To explore the distribution of the residuals we can choose six classes and display the corresponding frequencies.

```
histogram(res,6)
```

The χ^2 -test can be used to test the hypothesis that the residuals follow a Gaussian distribution (Sect. 3.9). We use `chi2gof` to perform the χ^2 -test

```
[h,p,stats] = chi2gof(res)
```

which yields

```
h =
    0

p =
    NaN

stats =
    struct with fields:
```



```
chi2stat: 0.0099
df: 0
edges: [-28.2530 -1.0404 3.4950 17.1012]
O: [14 5 11]
E: [13.7940 5.1876 11.0184]
```

The result $h = 0$ means that we cannot reject the null hypothesis without another cause at a 5% significance level. However, the quality of the result is not very good because the sample size of 30 measurements is very small. In such an example, pooling the data could yield very low (or even zero) expected counts. Bins in either tail with expected counts less than five are pooled with neighboring bins until the count in each extreme bin is at least five. If there are bins with counts of less than five in the central part of the plot, the function `chi2gof` displays a warning that the χ^2 approximation may not be accurate. In our example we have not defined the number of bins and therefore (according to the documentation) `chi2gof` should use a default of 10 bins for pooling the data. Pooling neighboring bins, however, has reduced the actual number of bins to three. According to Sect. 3.9, the corresponding number of degrees of freedom should be $df = 3 - (2 + 1) = 0$. If there are not enough degrees of freedom to conduct the test, `chi2gof` returns a p -value of NaN, as in our example. Alternatively, we use `chi2gof` with 6 bins

```
[h,p,stats] = chi2gof(res,'NBins',6)
```

which also causes neighboring bins to be pooled. However, the bin boundaries are a bit different and we therefore get four bins, so the corresponding number of degrees of freedom is $df = 4 - (2 + 1) = 1$. The output of `chi2gof` is

```
h =
    0

p =
    0.1481

stats =
    struct with fields:
        chi2stat: 2.0922
        df: 1
        edges: [-28.2530 -5.5759 1.9832 9.5422 17.1012]
        O: [11 7 5 7]
        E: [8.8282 8.4603 7.3927 5.3187]
```

The p -value of 0.1481 indicates that the result is significant and we cannot reject the null hypothesis that the residuals follow a Gaussian distribution.

4.5 Bootstrap Estimates of the Regression Coefficients

In this section we use the *bootstrap* method to obtain a better estimate of the regression coefficients. As an example we use the function `bootstrap` with 1000 samples (Fig. 4.6).

```
clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

p = polyfit(meters,age,1);

rng(0)

p_bootstrap = bootstrap(1000,'polyfit',meters,age,1);
```

The statistic of the first coefficient, i.e. the slope of the regression line is

```
histogram(p_bootstrap(:,1),15)

median(p_bootstrap(:,1))

ans =

    5.3855
```

We use the median instead of the mean since we cannot expect the bootstrap results to be Gaussian distributed. In contrast, the statistic of the second coefficient shows a significant dispersion.

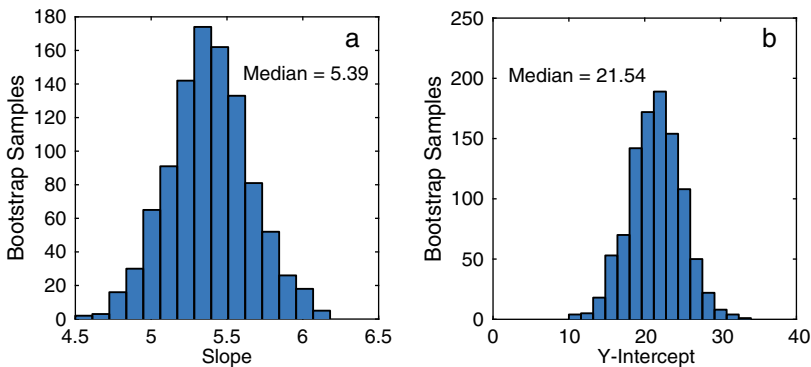


Fig. 4.6 Histogram of **a** the first (slope of the regression line) and **b** the second (y-axis intercept of the line) regression coefficient, as estimated from bootstrap resampling. The first coefficient is well constrained, but the second coefficient shows a broad scatter.

```

histogram(p_bootstrap(:,2),15)

median(p_bootstrap(:,2))

ans =

    21.5374

```

The true values, as used to simulate our data set, are 5.6 for the slope and 20 for the intercept with the y -axis, whereas the corresponding coefficients calculated using `polyfit` were 5.3667 and 21.7607 (Sect. 4.3).

4.6 Jackknife Estimates of the Regression Coefficients

The *jackknife* method is a resampling technique that is similar to the bootstrap method. From a sample with n data points, n subsamples with $n - 1$ data points are taken. The parameters of interest, e.g. the regression coefficients, are calculated for each of the subsamples. The mean and dispersion of the coefficients are then computed. The disadvantage of this method is the limited number of n subsamples: a jackknife estimate of the regression coefficients is therefore less precise than a bootstrap estimate.

The relevant code for the jackknife is easy to generate:

```

clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

p = polyfit(meters,age,1);

for i = 1 : 30
    j_meters = meters;
    j_age = age;
    j_meters(i) = [];
    j_age(i) = [];
    p(i,:) = polyfit(j_meters,j_age,1);
end

```

The jackknife for subsamples with $n - 1 = 29$ data points can be obtained by a simple for loop. The i th data point within each iteration is deleted and regression coefficients are calculated for the remaining data points. The mean of the i subsamples gives an improved estimate of the regression coefficients. As with the bootstrap result, the slope of the regression line (first coefficient) is well defined, whereas the intercept with the y -axis (second coefficient) has a large uncertainty:

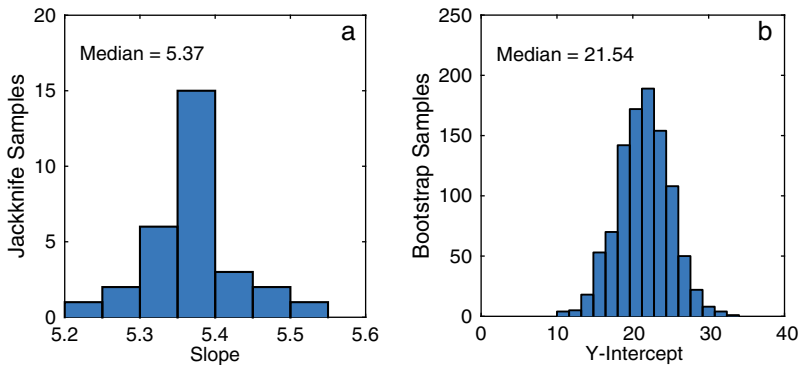


Fig. 4.7 Histogram of **a** the first (slope of the regression line) and **b** the second (y-axis intercept of the line) regression coefficient, as estimated from jackknife resampling. Note that the parameters are not as well defined as those from bootstrapping.

```
median(p(:,1))
```

```
ans =
```

```
5.3663
```

compared to 5.3855 calculated by the bootstrap method and

```
median(p(:,2))
```

```
ans =
```

```
21.7964
```

compared to 21.5374 from the bootstrap method (Sect. 4.5). The true values, as before, are 5.6 and 20. The histograms of the jackknife results from 30 subsamples (Fig. 4.7)

```
subplot(1,2,1), histogram(p(:,1)), axis square
```

```
subplot(1,2,2), histogram(p(:,2)), axis square
```

do not display such clear distributions for the coefficients as the histograms of the bootstrap estimates. As an alternative to the above method, MATLAB provides the function `jackknife` with which to perform a jackknife experiment.

```
p = jackknife('polyfit',meters,age,1);
```

```
median(p(:,1))
```

```
median(p(:,2))
```

```
ans =

    5.3663

ans =

    21.7964

subplot(1,2,1), histogram(p(:,1)), axis square
subplot(1,2,2), histogram(p(:,2)), axis square
```

The results are identical to those obtained using the code introduced above. We have seen therefore that resampling using either the jackknife or the bootstrap method is a simple and valuable way to test the quality of regression models. The next section introduces an alternative approach for quality estimation, which is much more commonly used than the resampling methods.

4.7 Cross Validation

A third method to test the quality of the result of a regression analysis involves *cross validation*. The regression line is computed by using $n - 1$ data points. The n th data point is predicted and the discrepancy between the prediction and the actual value is computed. The mean of the discrepancies between the actual and predicted values is subsequently determined.

In this example the cross validation is computed for $n = 30$ data points. The resulting 30 regression lines, each computed using $n - 1 = 29$ data points, display some dispersion in their slopes and y-axis intercepts.

```
clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

p = polyfit(meters,age,1);

for i = 1 : 30
    j_meters = meters;
    j_age = age;
    j_meters(i) = [];
    j_age(i) = [];
    p(i,:) = polyfit(j_meters,j_age,1);
    plot(meters,polyval(p(i,:),meters),'r'), hold on
```

```

    p_age(i) = polyval(p(i,:),meters(i));
    p_error(i) = p_age(i) - age(i);
end
hold off

```

The prediction error is—in the ideal case—Gaussian distributed with a mean of zero.

```

mean(p_error)

ans =

    -0.0487

```

The standard deviation is an unbiased mean of the deviations of the true data points from the predicted straight line.

```

std(p_error)

ans =

    10.9757

```

Cross validation gives valuable information on the goodness-of-fit of the regression result and can also be used for quality control in other fields, such as those of temporal and spatial prediction (Chaps. 5 and 7).

4.8 Reduced Major Axis Regression

In some examples neither variable is manipulated and both can therefore be considered to be independent. In such cases several methods are available to compute a best-fit line that minimizes the distance from both x and y . As an example the method of *reduced major axis* (RMA) minimizes the triangular area $0.5 \cdot (\Delta x \Delta y)$ between the data points and the regression line, where Δx and Δy are the distances between the predicted values of x and y and the true values of x and y (Fig. 4.4). Although this optimization appears to be complex, it can be shown that the first regression coefficient b_1 (the slope) is simply the ratio of the standard deviations of y and x .

$$b_1 = s_y / s_x$$

As with classical regression, the regression line passes through the data centroid defined by the sample mean. We can therefore compute the second regression coefficient b_0 (the y -intercept),

$$b_0 = \bar{y} - b_1 \bar{x}$$

using the univariate sample means and the slope b_1 computed earlier. Let us again load the age-depth data from the file *agedepth_1.txt* and define two variables, *meters* and *age*. It is assumed that both of the variables contain errors and that the scatter of the data can be explained by dispersions of *meters* and *age*.

```
clear, close all, clc

agedepth = load('agedepth_1.txt');

meters = agedepth(:,1);
age = agedepth(:,2);
```

The above formula is used for computing the slope of the regression line b_1 .

```
p(1,1) = std(age)/std(meters)

p =

    5.6117
```

The second coefficient b_0 , i.e. the y-axis intercept, can therefore be computed by

```
p(1,2) = mean(age) - p(1,1) * mean(meters)

p =

    5.6117    18.7037
```

The regression line can be plotted by

```
plot(meters,age,'o'), hold on
plot(meters,polyval(p,meters),'r'), hold off
```

This linear fit differs slightly from the line obtained from classical regression. Note that the regression line from RMA is *not* the bisector of the lines produced by the x - y and y - x classical linear regression analyses, i.e. those produced using either x or y as an independent variable while computing the regression lines.

4.9 Curvilinear Regression

It is apparent from our previous analysis that a linear regression model provides a good way of describing the scaling properties of the data. However, we may wish to check whether the data could be equally well described by a polynomial fit of a higher degree, for instance by a second degree polynomial:

$$y = b_0 + b_1x + b_2x^2$$

To clear the workspace and reload the original data, we type

```
clear, close all, clc
agedepth = load('agedepth_1.txt');
meters = agedepth(:,1);
age = agedepth(:,2);
```

A second degree polynomial can then be fitted by using the function `polyfit`.

```
p = polyfit(meters,age,2)

p =
    0.0589    4.1087   26.0381
```

The first coefficient is close to zero, i.e. has little influence on predictions. The second and third coefficients are similar to those obtained by linear regression. Plotting the data yields a curve that resembles a straight line.

```
plot(meters,age,'o'), hold on
plot(meters,polyval(p,meters),'r'), hold off
```

Let us compute and plot the error bounds obtained by using an optional second output parameter from `polyfit` as an input parameter to `polyval`.

```
[p,s] = polyfit(meters,age,2);
[p_age,delta] = polyval(p,meters,s);
```

As before, this code uses an interval of ± 2 s, corresponding to a 95% confidence interval. Using `polyfit` not only yields the polynomial coefficients `p`, but also a structure `s` for use with `polyval` to obtain error bounds for the predictions. The variable `delta` is an estimate of the standard deviation of the prediction error of a future observation at x by $p(x)$. We then plot the results:

```
plot(meters,age,'o',...
      meters,p_age,'g-',...
      meters,p_age+2*delta,'r',...
      meters,p_age-2*delta,'r')
axis([0 20 0 140]), grid on
xlabel('Depth in Sediment (meters)')
ylabel('Age of Sediment (kyrs)')
```

We now use another synthetic data set that we generate using a quadratic relationship between meters and age.


```

clear

rng(0)
meters = 20 * rand(30,1);
age = 1.6 * meters.^2 - 1.1 * meters + 50;
age = age + 40.* randn(length(meters),1);

plot(meters,age,'o')

agedepth(:,1) = meters;
agedepth(:,2) = age;

agedepth = sortrows(agedepth,1);

save agedepth_2.txt agedepth -ascii

```

The synthetic bivariate data set can be loaded from the file *agedepth_2.txt*.

```

clear

agedepth = load('agedepth_2.txt');

meters = agedepth(:,1);
age = agedepth(:,2);

plot(meters,age,'o')

```

Fitting a second order polynomial yields a convincing regression result.

```

p = polyfit(meters,age,2)

p =

    1.8356   -7.0653   74.1526

```

As shown above, the true values for the three coefficients are +1.8, -7.1 and +74.2, which means that there are some discrepancies between the true values and the coefficients estimated using `polyfit`. The regression curve and the error bounds can be plotted by typing (Fig. 4.8).

```

plot(meters,age,'o'), hold on
plot(meters,polyval(p,meters),'r'), hold off

[p,s] = polyfit(meters,age,2);
[p_age,delta] = polyval(p,meters,s);

plot(meters,age,'o',...
     meters,p_age,'g',...
     meters,p_age+2*delta,'r--',...
     meters,p_age-2*delta,'r--')

```

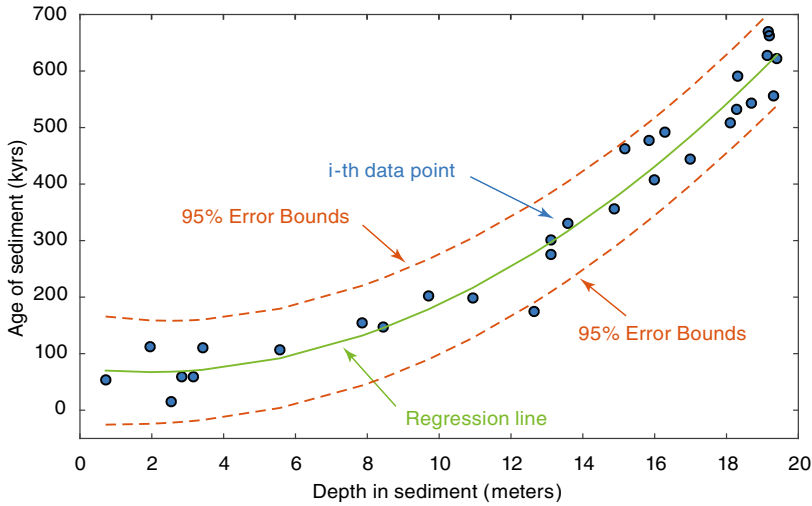


Fig. 4.8 Curvilinear regression from measurements of barium content. The plot shows the original data points (circles), the regression line for a polynomial of degree $n = 2$ (solid line), and the error bounds (dashed lines) of the regression.

```
axis([0 20 -50 700]), grid on
xlabel('Depth in Sediment (meters)')
ylabel('Age of Sediment (kyrs)')
```

The plot shows that the quadratic model for this data is a good one. The quality of the result could again be tested by exploring the residuals, by employing resampling schemes, or by cross validation. Combining regression analysis with one of these methods provides a powerful tool in bivariate data analysis, whereas Pearson's correlation coefficient should be used only as a preliminary test for linear relationships.

4.10 Nonlinear and Weighted Regression

Many bivariate data sets in earth sciences follow a more complex trend than a simple linear or curvilinear trend. Classic examples for nonlinear trends are the exponential decay of radionuclides, or the exponential growth of algae populations. In such cases MATLAB provides various tools to fit nonlinear models to the data. An easy-to-use routine to fit such models is nonlinear regression using the function `nlinfit`. To demonstrate the use of `nlinfit` we generate a bivariate data set where one variable is exponentially correlated with a second variable. We first generate evenly-spaced values between 0.5 and 3 at intervals of 0.1 and add some Gaussian noise with a

standard deviation of 0.2 to make the data unevenly spaced. The resulting 26 data points are stored in the first column of the variable `data`.

```
clear, close all, clc

rng(0)
data(:,1) = 0.5 : 0.1 : 3;
data(:,1) = data(:,1) + 0.2*randn(size(data(:,1)));
```

We can then compute the second variable, which is the exponent of the first variable multiplied by 0.2 and increased by 3. We again add Gaussian noise, this time with a standard deviation of 0.5, to the data. Finally, we can sort the data with respect to the first column and display the result.

```
data(:,2) = 3 + 0.2 * exp(data(:,1));
data(:,2) = data(:,2) + 0.5*randn(size(data(:,2)));
data = sortrows(data,1);
plot(data(:,1),data(:,2),'o')
xlabel('x-Axis'), ylabel('y-Axis')
```

Nonlinear regression is used to estimate the two coefficients of the exponential function, i.e. the multiplier 0.2 and the summand 3. The function `p = nlinfit(data(:,1),data(:,2),fun,p0)` returns a vector `p` of coefficient estimates for a nonlinear regression of the responses in `data(:,2)` to the predictors in `data(:,1)` using the model specified by `fun`. We can design a function handle model `fun` representing an exponential function with an input variable `x`, and with the vector `phi` representing the two coefficients of the exponential function.

```
model = @(phi,x)(phi(1)*exp(x) + phi(2));
```

The variable `p0` is a vector containing initial values for the coefficients and is the same length as `p`. In our example, we use `[0 0]` as the initial values of `p`.

```
p0 = [0 0];
```

We can then use `nlinfit` to estimate the coefficients `p` using the data `data`, the model `model`, and the initial values `p0`.

```
p = nlinfit(data(:,1),data(:,2),model,p0)

p =

    0.2121    2.8306
```

We can now use the resulting coefficients $p(1)$ and $p(2)$ to calculate the function values `fittedcurve` using the model and compare the results with the original data in a graphic.

```
fittedcurve_1 = p(1)*exp(data(:,1)) + p(2);

plot(data(:,1),data(:,2),'o')
hold on
plot(data(:,1),fittedcurve_1,'r')
xlabel('x-Axis'), ylabel('y-Axis')
title('Unweighted Fit')
hold off
```

As we can see from the output of `p` and the graphics, the fitted red curve describes the data reasonably well. We can now also use `nlinfit` to perform a weighted regression. Let us assume that we know the one-sigma errors of the values in `data(:,2)`. We can generate synthetic errors and store them in the third column of `data`.

```
data(:,3) = abs(randn(size(data(:,1)))));
errorbar(data(:,1),data(:,2),data(:,3),'o')
xlabel('x-Axis'), ylabel('y-Axis')
```

We can now normalize the data points so that they are weighted by the inverse of the relative errors. We therefore normalize `data(:,3)` so that the total of all errors in `data(:,3)` is one, and store the normalized errors in `data(:,4)`.

```
data(:,4) = sum(data(:,3))./data(:,3);
```

To make a weighted fit, we define the model function `model`, and then use `nlinfit` with the parameter weights.

```
model = @(phi,x)(phi(1)*exp(x) + phi(2));
p0 = [0 0];
p = nlinfit(data(:,1),data(:,2),model,p0,'Weights',data(:,4))

p =

    0.2191    2.3442
```

As before, `nlinfit` will compute weighted parameter estimates `p`. We again use the resulting coefficients $p(1)$ and $p(2)$ to calculate the function values `fittedcurve` using the model and compare the results with the original data.

```

fittedcurve_2 = p(1)*exp(data(:,1)) + p(2);
errorbar(data(:,1),data(:,2),data(:,3),'o')
hold on
plot(data(:,1),fittedcurve_2,'r')
xlabel('x-Axis'), ylabel('y-Axis')
title('Weighted Fit')
hold off

```

Comparing the coefficients p and the red curves from the weighted regression with the previous results from the unweighted regression reveals slightly different results (Fig. 4.9):

```

errorbar(data(:,1),data(:,2),data(:,3),'o')
hold on
plot(data(:,1),fittedcurve_1,'r--')
plot(data(:,1),fittedcurve_2,'r-')
xlabel('x-Axis'), ylabel('y-Axis')
title('Comparison of Unweighted and Weighted Fit')
hold off

```

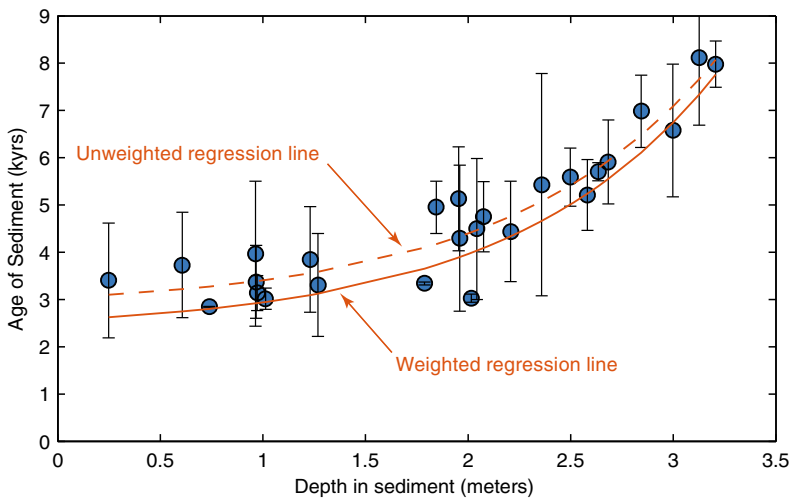


Fig. 4.9 Comparison of unweighted (dashed line) and weighted (solid line) regression result from synthetic data. The plot shows the original data points (circles), the error bars for all data points, and the regression line for an exponential model function. In the unweighted regression, the fitted curved is moved towards the first two data points ($x = -0.24823$ and $x = 0.60753$) with a large error, while in the weighted regression it is moved towards the third data point ($x = 0.73846$) with a small error.

As an example, in the unweighted regression the fitted curve is moved towards the first two data points ($x = 0.24823$ and $x = 0.60753$) with a large error, while in the weighted experiment it is moved towards the third data point ($x = 0.73846$) with a small error.

4.11 Classical Linear Regression of Log-Transformed Data

A common error in regression analysis is that bivariate data with an exponential relationship are log-transformed, then a best-fit line calculated using a least squares linear regression and the result then transformed back. Below is a simple MATLAB example to show how to do it better.

Classical regression assumes that y responds to x and that the entire dispersion in the data set is contained within the y -value (see Sect. 4.3). This means that x is the independent variable defined by the experimenter and regarded as being free of errors. Linear regression minimizes the deviations Δy between the data points xy and the value y predicted by the best-fit line $y = b_0 + b_1x$ using a least-squares criterion. The classic linear regression method makes two assumptions about the data: (1) there is a linear relationship between x and y , (2) the unknown errors around the means have a normal (Gaussian) distribution, with a similar variance for all data points. Logarithmizing the y -values violates this assumption: the errors then have a log-normal distribution and the regression therefore places less weight on the larger y -values.

To see the difference in the results we first create a synthetic data set. The y -values, stored in `data(:,2)`, have an exponential relationship with the x -values, stored in `data(:,1)`. After computing the x - and y -values we add Gaussian noise to the y -values in `data(:,2)` and sort the data.

```
clear, close all, clc

rng(0)
data(:,1) = 0.5 : 0.1 : 3;
data(:,1) = data(:,1) + 0.2*randn(size(data(:,1)));
data(:,2) = 3 + 0.2 * exp(data(:,1));
data(:,2) = data(:,2) + 0.5*randn(size(data(:,2)));
data = sortrows(data,1);
```

Here is the linear fit of the logarithmized data using `polyfit`:

```
[p1,s] = polyfit(data(:,1),log(data(:,2)),1)
```

We then use the function `nlinfit` from the Statistics and Machine Learning Toolbox to calculate the nonlinear fit without transforming the data (see Sect. 4.10):

```
model = @(phi,t)(phi(1)*exp(t) + phi(2));
p0 = [0 0];
pn = nlinfit(data(:,1),data(:,2),model,p0)
```

We can also calculate the true (noise-free) line using the exponential equation from above, before adding noise:

```
trueline(:,1) = 0.5 : 0.01 : 3;
trueline(:,2) = 3 + 0.2 * exp(trueline(:,1));
```

Displaying the data clearly shows the difference:

```
line(data(:,1),data(:,2),...
      'LineStyle','none',...
      'Color',[0 0.4453 0.7383],...
      'Marker','o',...
      'MarkerFaceColor',[0 0.4453 0.7383],...
      'MarkerEdgeColor',[0 0 0]);
line(data(:,1),pn(1)*exp(data(:,1)) + pn(2),...
      'Color',[0.8477 0.3242 0.0977])
line(data(:,1),exp(polyval(pl,data(:,1),s)),...
      'Color',[ 0.9258 0.6914 0.1250])
line(trueline(:,1),trueline(:,2),...
      'Color',[0 0 0],...
      'LineStyle',':')
legend('Data points',...
      'Nonlinear (exponential) fit',...
      'Linear fit of log-transformed data',...
      'True noise-free line',...
      'Location','Northwest',...
      'Box','off')
xlabel('x-Values')
ylabel('y-Values')
```

The yellow curve calculated by linear regression of the log-transformed data has a much lower curvature than the nonlinear fit (Fig. 4.10). The dotted black line is the noise-free curve. Of course the result of the nonlinear regression is not a perfect match with the true line due to the noise in the data used with the `nlinfit` function. The yellow line, however, is statistically incorrect because it is the result of the use of the wrong method.

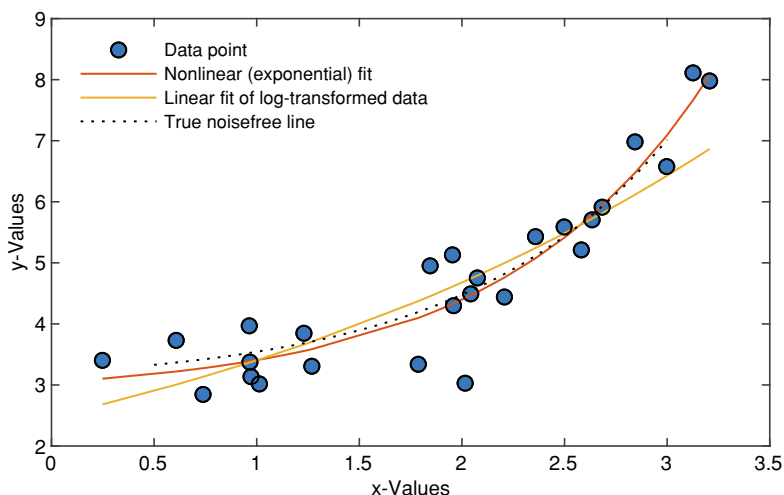


Fig. 4.10 Comparison of nonlinear regression (orange line) and classical linear regression of log-transformed data (yellow line) with an exponential relationship. The yellow curve calculated by linear regression of the log-transformed data has a much lower curvature than the nonlinear fit. The dotted black line is the noise-free curve. The yellow line is statistically incorrect because it is the result from using of the wrong method.

Recommended Reading

- Albarède F (2002) Introduction to geochemical modeling. Cambridge University Press, Cambridge
- Davis JC (2002) Statistics and data analysis in geology, 3rd edn. John Wiley and Sons, New York
- Draper NR, Smith, H (1998) Applied regression analysis. In: Wiley series in probability and statistics. Wiley, New York
- Efron B (1982) The jackknife, the bootstrap, and other resampling plans. In: Society of industrial and applied mathematics CBMS-NSF monographs, p 38
- Fisher RA (1922) The goodness of fit of regression formulae, and the distribution of regression coefficients. *J Roy Stat Soc* 85:597–612
- Kendall M (1938) A new measure of rank correlation. *Biometrika* 30:81–89
- MacTavish JN, Malone PG, Wells TL (1968) RMAR; A reduced major axis regression program designed for paleontologic data. *J Paleontol* 42(4):1076–1078
- MathWorks (2020) Statistics and machine learning toolbox—user’s guide. The MathWorks, Inc., Natick, MA
- Pearson K (1894–1898) Mathematical contributions to the theory of evolution, Part I to IV. *Philos Trans R Soc* 185–191
- Pearson K (1895) Notes on regression and inheritance in the case of two parents. *Proc R Soc Lond* 58:240–242
- Spearman C (1904) The proof and measurement of association between two things. *Am J Psychol* 15:72–101
- Spearman C (1910) Correlation calculated from faulty data. *Br J Psychol* 3:271–295



5.1 Introduction

Time-series analysis is used to investigate the temporal behavior of a variable $x(t)$. Examples include the investigations into long-term records of mountain uplift, sea-level fluctuations, orbitally-induced insolation variations and their influence on the ice-age cycles, millennium-scale variations in the atmosphere-ocean system, the effect of the El Niño/Southern Oscillation on tropical rainfall and sedimentation (Fig. 5.1), and tidal influences on noble gas emissions from bore holes. The temporal pattern of a sequence of events can be random, clustered, cyclic, or chaotic. Time-series analysis provides various tools with which to detect these temporal patterns. Understanding the underlying processes that produced the observed data allows us to predict future values of the variable. We use the Signal Processing Toolbox and the Wavelet Toolbox, which contain all the necessary routines for time-series analysis (MathWorks 2020a, b).

Section 5.2 discusses signals in general and contains a technical description of how to generate synthetic signals for time-series analysis. The use of spectral analysis to detect cyclicities in a single time series (auto-spectral analysis) and to determine the relationship between two time series as a function of frequency (cross-spectral analysis) is then demonstrated in Sects. 5.3 and 5.4. Since most time series in earth sciences have uneven time intervals, various interpolation techniques and subsequent methods of spectral analysis are required, and these are introduced in Sect. 5.5. Evolutionary power spectra to map changes in cyclicity through time are demonstrated in Sect. 5.6. An alternative technique for analyzing unevenly-spaced data is explained in Sect. 5.7. Section 5.8 introduces the very

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_5) contains supplementary material, which is available to authorized users.

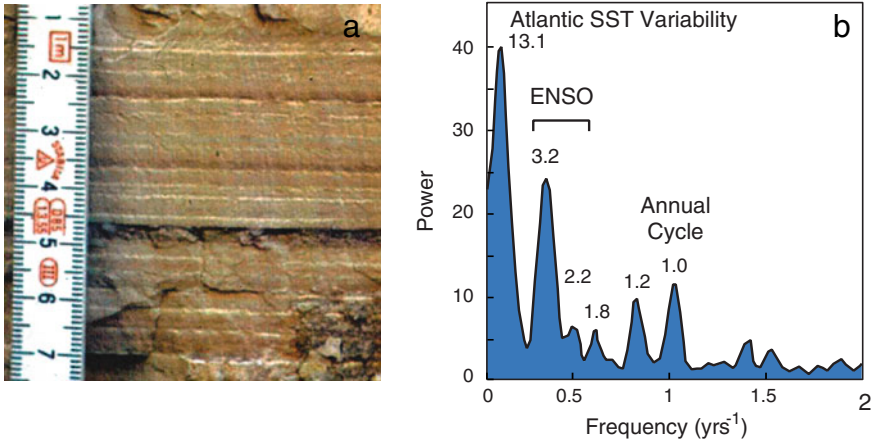


Fig. 5.1 **a** Photograph of ca. 30 kyr-old varved sediments from a lake in the Andes of north-western Argentina. The distribution of the source rocks and the inter-annual precipitation pattern in the area suggest that the reddish-brown layers reflect cyclic recurrences of enhanced precipitation, with increased erosion and sediment input into the lake. **b** The power spectrum of a red-color intensity transect across 70 varves is dominated by significant peaks at frequencies of ca. 0.076, 0.313, 0.455 and 1.0 yrs⁻¹. These cyclicities suggest a strong influence of the tropical Atlantic sea-surface temperature (SST) variability, the El Niño/Southern Oscillation (ENSO), and the annual cycle that occurred 30 kyrs ago, similar to today's cyclicities (Trauth et al. 2003).

popular wavelet power spectrum, which is able to map temporal variations in the spectra in a similar way to the method demonstrated in Sect. 5.6. Section 5.9 then introduces methods to detect, and to remove, abrupt transitions in central tendency and dispersion within time series. Section 5.10 presents methods used to align stratigraphic sequences. This chapter then closes (Sect. 5.11) with an overview of nonlinear techniques, with special attention to recurrence plots.

5.2 Generating Signals

A time series is an ordered sequence of values of a variable $x(t)$ at certain times t_k .

$$x(t_k) = x(t_1), x(t_2), \dots, x(t_N)$$

If the time interval between any two successive observations $x(t_k)$ and $x(t_{k+1})$ is constant, the time series is said to be equally spaced and the sampling interval is

$$\Delta t = t_{k+1} - t_k$$

The sampling frequency f_s is the inverse of the sampling interval Δt . We generally try to sample at regular time intervals or constant sampling frequencies, but in many earth science examples this is not possible. As an example, imagine deep-sea sediments sampled at five-centimeter intervals along a sediment core. Radiometric age determinations at certain levels in the sediment core have revealed significant fluctuations in the sedimentation rates. Despite the samples being evenly spaced along the sediment core they are not equally spaced on the time axis. Hence the quantity

$$\Delta t = T/N$$

(where T is the full length of the time series and N is the number of data points) represents only an average sampling interval. In general, a time series $x(t_k)$ can be represented as the linear sum of a periodic component $x_p(t_k)$, a long-term component or trend $x_{tr}(t_k)$, and random noise $x_n(t_k)$.

$$x(t_k) = x_p(t_k) + x_{tr}(t_k) + x_n(t_k)$$

The long-term component is a linear or higher-degree trend that can be extracted by fitting a polynomial of a certain degree and subtracting the values of this polynomial from the data (see Chap. 4). Noise removal will be described in Chap. 6. The periodic - or cyclic in a mathematically less rigorous sense—component can be approximated by a linear combination of sine (or cosine) waves that have different amplitudes A_i , frequencies f_i , and phase angles ψ_i .

$$x_p(t_k) = \sum_i A_i \cdot \sin(2\pi f_i t_k - \psi_i)$$

The phase angle ψ helps to detect temporal shifts between signals of the same frequency. Two signals x and y with the same period are out of phase unless the difference between ψ_x and ψ_y is equal to zero (Fig. 5.2).

The frequency f of a periodic signal is the inverse of the period τ . The *Nyquist frequency* f_{nyq} is half the sampling frequency f_s and represents the maximum frequency the data can produce. This frequency, defined by the US American mathematician Claude Elwood Shannon and named after the Swedish-American electrical engineer Harry Nyquist, prevents the phenomenon of *aliasing* in time-discrete systems. Aliasing means that ambiguities occur in digital signals if the sampling frequency is too low ($<f_{nyq}$). As an example audio compact disks (CDs) are sampled at frequencies of 44,100 Hz (Hertz, where 1 Hz = 1 cycle per second), but the corresponding Nyquist frequency is 22,050 Hz, which is the highest frequency a CD player can theoretically produce. The performance limitations of anti-alias filters used by CD players further reduce the frequency band and result in a cutoff frequency of around 20,050 Hz, which is the true upper frequency limit of a CD player.

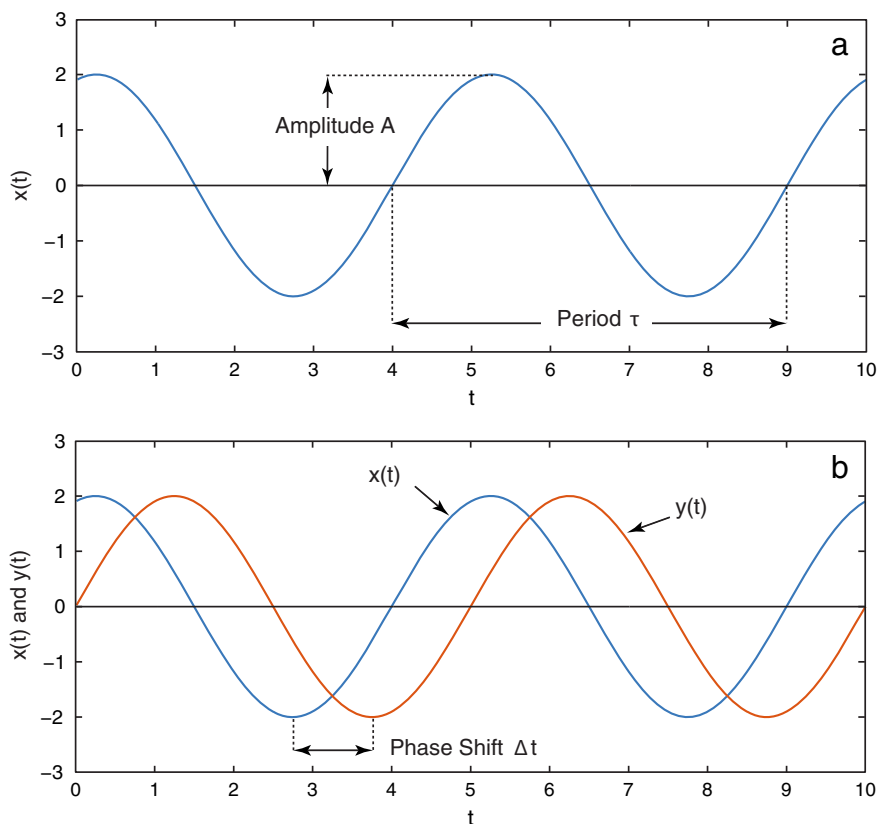


Fig. 5.2 **a** Periodic signal x , a function of time t , defined by the amplitude A , and the period τ which is the inverse of the frequency f . **b** Two signals x and y with the same period are out of phase if the difference between ψ_x and ψ_y is not equal to zero.

We can now generate synthetic signals to illustrate the use of time-series analysis tools. When using synthetic data we know in advance which features the time series contains, such as periodic or random components, and we can introduce a linear trend or gaps in the time series. The user will encounter plenty of examples of the possible effects of varying the parameter settings, as well as potential artefacts and errors that can result from the application of spectral analysis tools. We will start with simple data and then apply the methods to more complex time series. The first example illustrates how to generate a basic synthetic data series that is characteristic of earth science data. First, we create a time axis t running from 1 to 1000 in steps of one unit, i.e. the sampling frequency is also one. We then generate a simple periodic signal y : a sine wave with a period of five and an amplitude of two by typing

```
clear, close all, clc

t = 1 : 1000;
x = 2*sin(2*pi*t/5);

plot(t,x), axis([0 200 -4 4])
```

The period of $\tau = 5$ corresponds to a frequency of $f = 1/5 = 0.2$. Natural data series, however, are more complex than a simple periodic signal. The slightly more complicated signal can be generated by superimposing several periodic components with different periods. As an example we compute such a signal by adding three sine waves with the periods $\tau_1 = 50$ ($f_1 = 0.02$), $\tau_2 = 15$ ($f_2 \approx 0.07$) and $\tau_3 = 5$ ($f_3 = 0.2$). The corresponding amplitudes are $A_1 = 2$, $A_2 = 1$ and $A_3 = 0.5$.

```
t = 1 : 1000;
x = 2*sin(2*pi*t/50) + sin(2*pi*t/15) + 0.5*sin(2*pi*t/5);

plot(t,x), axis([0 200 -4 4])
```

By restricting the t -axis to the interval $[0, 200]$, only one fifth of the original data series is displayed (Fig. 5.3a). It is, however, recommended that long data series be generated, as in the example, in order to avoid edge effects when applying spectral analysis tools for the first time.

In contrast to our synthetic time series, real data also contain various disturbances such as random noise and first or higher-order trends. In order to reproduce the effects of noise, a random-number generator can be used to compute Gaussian noise with mean of zero and standard deviation of one. The seed of the algorithm should be set to zero using `rng(0)`. One thousand random numbers are then generated using the function `randn`.

```
rng(0)
n = randn(1,1000);
```

We add this noise to the original data, i.e. we generate a signal containing additive noise (Fig. 5.3b). Displaying the data illustrates the effect of noise on a periodic signal. Since in reality no record is totally free of noise it is important to familiarize oneself with the influence of noise on power spectra.

```
xn = x + n;

plot(t,x,'b-',t,xn,'r-'), axis([0 200 -4 4])
```

Signal processing methods are often applied to remove a major part of the noise, although many filtering methods make arbitrary assumptions concerning the signal-to-noise ratio. Moreover, filtering introduces artefacts and statistical dependencies into the data, which may have a profound influence on the resulting power spectra.

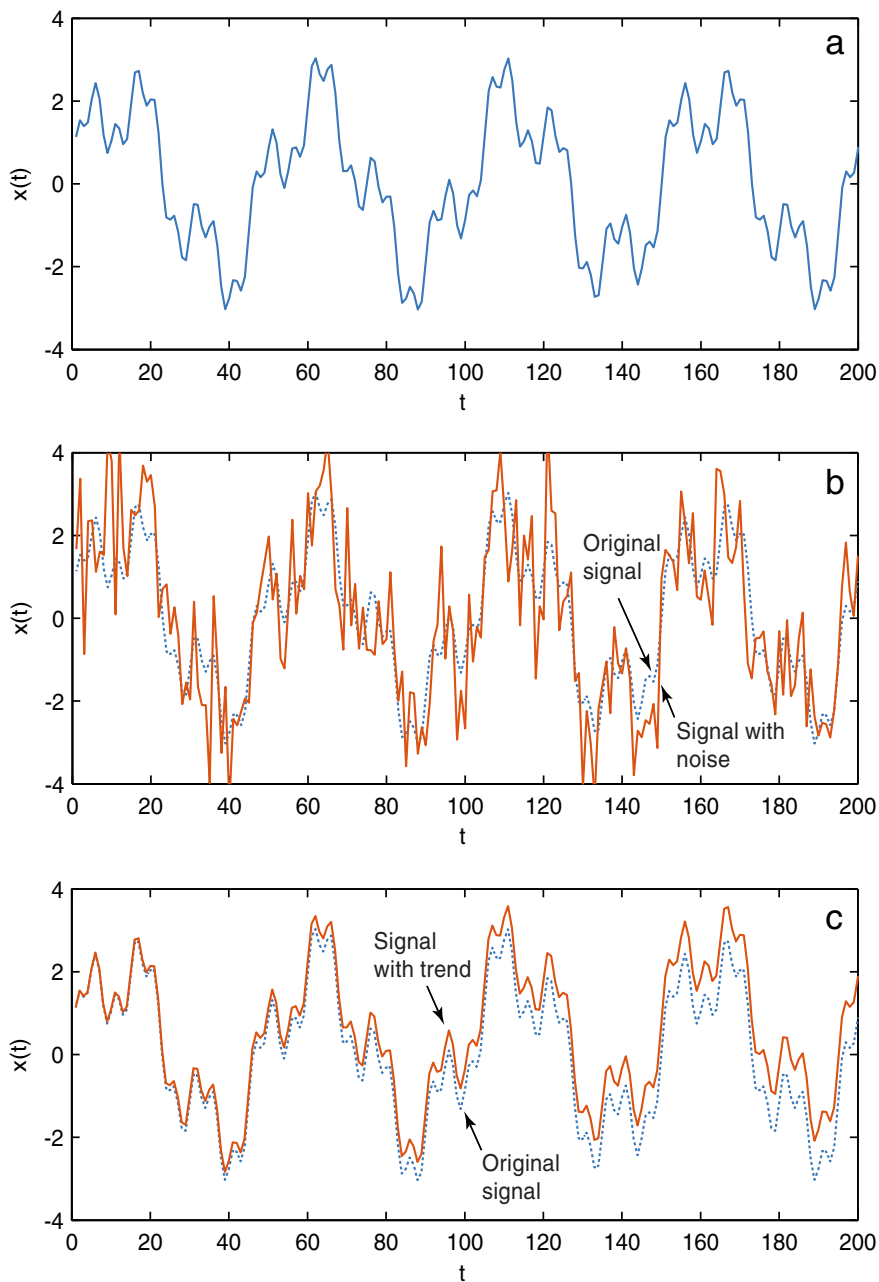


Fig. 5.3 **a** Synthetic signal with periodicities of $\tau_1 = 50$, $\tau_2 = 15$ and $\tau_3 = 5$, with different amplitudes, and **b** The same signal overprinted with Gaussian noise. **c** The time series shows a significant linear trend.

Finally, we introduce a linear long-term trend to the data by adding a straight line with a slope of 0.005 and an intercept with the y-axis of zero (Fig. 5.3c). Such trends are common in earth sciences. As an example, consider the glacial-interglacial cycles observed in marine oxygen isotope records, overprinted on a long-term cooling trend over the last six million years.

```
xt = x + 0.005*t;
plot(t,x,'b-',t,xt,'r-'), axis([0 200 -4 4])
```

In reality, more complex trends exist, such as higher-order trends or trends characterized by variations in gradient. In practice, it is recommended that such trends be eliminated by fitting polynomials to the data and subtracting the corresponding values. Our synthetic time series now contains many characteristics of a typical earth science data set. It can be used to illustrate the use of the spectral analysis tools that are introduced in the next section.

5.3 Auto-Spectral and Cross-Spectral Analysis

Auto-spectral analysis aims to describe the distribution of variance contained in a single signal $x(t)$ as a function of frequency or wavelength. A simple way to describe the variance in a signal over a time lag k is by means of the autocovariance. An unbiased estimator of the autocovariance cov_{xx} of the signal $x(t)$ with N data points sampled at constant time intervals Δt is

$$cov_{xx}(k) = \frac{1}{N-k-1} \sum_{t=1}^{N-k} (x_t - \bar{x})(x_{t+k} - \bar{x})$$

The autocovariance series clearly depends on the amplitude of $x(t)$. Normalizing the covariance by the variance s^2 of $x(t)$ yields the autocorrelation sequence. Autocorrelation involves correlating a series of data with itself as a function of a time lag k .

$$corr_{xx}(k) = \frac{cov_{xx}(k)}{cov_{xx}(0)} = \frac{cov_{xx}(k)}{s_x^2}$$

A popular method used to compute power spectra in earth sciences is the method introduced by Blackman and Tukey (1958). The *Blackman-Tukey method* uses the complex Fourier transform $X_{xx}(f)$ of the autocorrelation sequence $corr_{xx}(k)$,

$$X_{xx}(f) = \sum_{k=0}^M corr_{xx}(k) e^{i2\pi fk/f_s}$$

where M is the maximum lag and f_s the sampling frequency. The Blackman-Tukey auto-spectrum is the absolute value of the Fourier transform of the autocorrelation function. In some fields, the *power spectral density* is used as an alternative way of describing the auto-spectrum. The Blackman-Tukey power spectral density PSD is estimated by

$$PSD = \frac{X_{xx}^*(f)X_{xx}(f)}{f_s} = \frac{|X_{xx}(f)|^2}{f_s}$$

where $X_{xx}^*(f)$ is the conjugate complex of the Fourier transform of the autocorrelation function $X_{xx}(f)$ and f_s is the sampling frequency. The actual computation of the power spectrum can only be performed at a finite number of different frequencies by employing a *Fast Fourier Transformation* (FFT). The FFT is a method of computing a discrete Fourier transform with reduced execution time. Most FFT algorithms divide the transform into two portions of size $N/2$ at each step of the transformation. The transform is therefore limited to blocks with dimensions equal to a power of two. In practice, the *spectrum* is computed by using a number of frequencies that is close to the number of data points in the original signal $x(t)$.

The discrete Fourier transform is an approximation of the continuous Fourier transform. The continuous Fourier transform assumes an infinite signal but discrete real data are limited at both ends, i.e. the signal amplitude is zero beyond either end of the time series. In the time domain, a finite signal corresponds to an infinite signal multiplied by a rectangular window that has a value of one within the limits of the signal and a value of zero elsewhere. In the frequency domain, the multiplication of the time series by this window is equivalent to a convolution of the power spectrum of the signal with the spectrum of the rectangular window (see Sect. 6.4 for a definition of convolution). The spectrum of the window, however, is a $\sin(x)/x$ function, which has a main lobe and numerous side lobes on either side of the main peak, and hence all maxima in a power spectrum *leak*, i.e. they lose power on either side of the peaks (Fig. 5.4).

A popular way to overcome the problem of *spectral leakage* is by windowing, in which the sequence of data is simply multiplied by a smooth bell-shaped curve with positive values. Several window shapes are available, e.g. *Bartlett* (triangular), *Hamming* (cosinusoidal) and *Hanning* (slightly different cosinusoidal) (Fig. 5.4). The use of these windows slightly modifies the equation for the Blackman-Tukey auto-spectrum to

$$PSD_{xx}(f) = \sum_{k=0}^M corr_{xx}(k)w(k) e^{i2\pi fk/f_s}$$

where $w(k)$ is the windowing function. The Blackman-Tukey method therefore performs auto-spectral analysis in three steps: calculation of the autocorrelation sequence $corr_{xx}(k)$, windowing and, finally, computation of the discrete Fourier transform. MATLAB allows power spectral analysis to be performed with a number

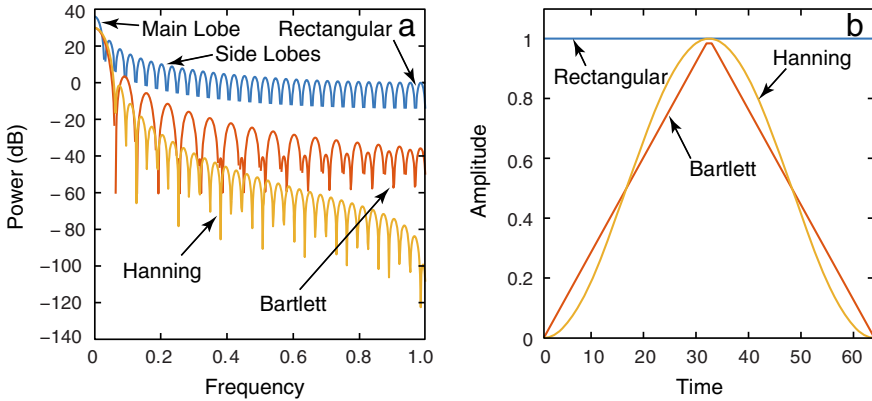


Fig. 5.4 Spectral leakage. **a** The amplitudes of the side lobes relative to that of the main lobe are reduced by multiplying the corresponding time series by **b** a smooth bell-shaped window function. A number of different windows, each with their own advantages and disadvantages, are available for use instead of the default rectangular window, including *Bartlett* (triangular) and *Hanning* (cosinusoidal) windows. Graph generated using the function `wvtool`.

of modifications to the above method. One useful modification is the Welch method (Welch 1967) (Fig. 5.5). This method involves dividing the time series into overlapping segments, computing the power spectrum for each segment, and then averaging the power spectra. The advantage of averaging the spectra is obvious: it simply improves the signal-to-noise ratio of a spectrum. The disadvantage is a loss of resolution in the spectra.

Cross-spectral analysis correlates two time series in the frequency domain. The cross-covariance is a measure of the variance between two signals over a time lag k . An unbiased estimator of the cross-covariance cov_{xy} of two signals, $x(t)$ and $y(t)$, with N data points sampled at constant time intervals Δt , is

$$cov_{xy}(k) = \frac{1}{N - k - 1} \sum_{i=1}^{N-k} (x_i - \bar{x})(y_{i+k} - \bar{y})$$

The cross-covariance series again depends on the amplitudes of $x(t)$ and $y(t)$. Normalizing the covariance by the standard deviations of $x(t)$ and $y(t)$ yields the cross-correlation sequence.

$$corr_{xy}(k) = \frac{cov_{xy}(k)}{cov_{xy}(0)} = \frac{cov_{xy}(k)}{\sigma_x \sigma_y}$$

The Blackman-Tukey method uses the complex Fourier transform $X_{xy}(f)$ of the cross-correlation sequence $corr_{xy}(k)$

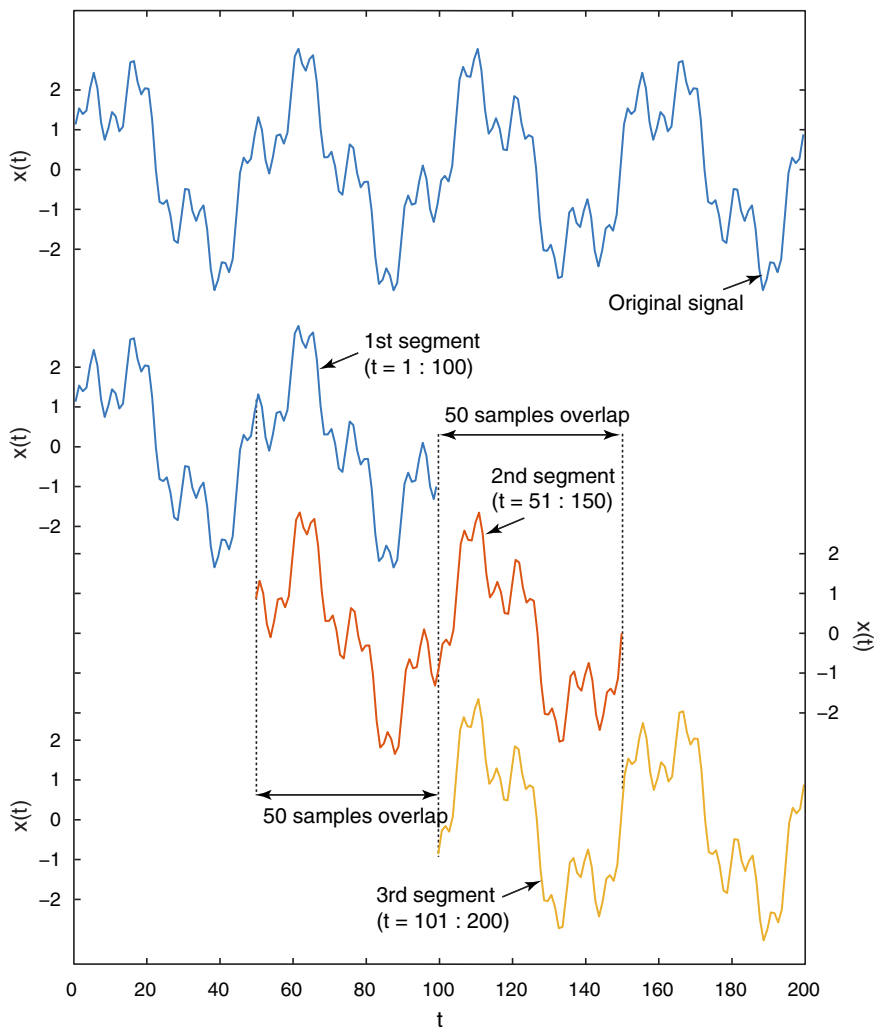


Fig. 5.5 Principle of Welch's power spectral analysis. The time series is first divided into overlapping segments; the power spectrum for each segment is then computed and all spectra are averaged to improve the signal-to-noise ratio of the power spectrum.

$$X_{xy}(f) = \sum_{k=0}^M \text{corr}_{xy}(k) e^{i2\pi f k / f_s}$$

where M is the maximum lag and f_s the sampling frequency. The absolute value of the complex Fourier transform $X_{xy}(f)$ is the cross-spectrum while the angle of $X_{xy}(f)$ represents the phase spectrum. The phase difference is important in

calculating leads and lags between two signals, a parameter often used to propose causalities between two processes documented by the signals. The correlation between two spectra can be calculated by means of the coherence:

$$C_{xy} = \frac{|X_{xy}(f)|^2}{X_{xx}(f)X_{yy}(f)}$$

The coherence is a real number between 0 and 1, where 0 indicates no correlation and 1 indicates maximum correlation between $x(t)$ and $y(t)$ at the frequency f . A significant degree of coherence is an important precondition for computing phase shifts between two signals.

5.4 Examples of Auto-Spectral and Cross-Spectral Analysis

The Signal Processing Toolbox provides numerous methods for computing spectral estimators for time series. The introduction of object-oriented programming with MATLAB has led to the launch of a new set of functions performing spectral analyses. Type `help spectrum` for more information about object-oriented spectral analysis. The non-object-oriented functions to perform spectral analyses, however, are still available. One of the oldest functions in this toolbox is `periodogram(x,window,nfft,fs)` which computes the power spectral density P_{xx} of a time series $x(t)$ using the periodogram method. This method was invented by Arthur Schuster in 1898 for studying the climate and calculates the power spectrum by performing a Fourier transform directly on a sequence without requiring prior calculation of the autocorrelation sequence. The periodogram method can therefore be considered a special case of the Blackman and Tukey (1958) method, applied with the time lag k set to unity (Muller and Macdonald 2000). At the time of its introduction in 1958, indirect computation of the power spectrum via an autocorrelation sequence was faster than calculating the Fourier transformation for the full data series $x(t)$ directly. After the introduction of the Fast Fourier Transform (FFT) by Cooley and Tukey (1965), and subsequent faster computer hardware, the higher computing speed of the Blackman-Tukey approach compared to the periodogram method became relatively unimportant.

For this next example we again use the synthetic time series x , x_n and x_t generated in Sect. 5.2 as the inputs:

```
clear, close all, clc

t = 1 : 1000; t = t';
x = 2*sin(2*pi*t/50) + sin(2*pi*t/15) + 0.5*sin(2*pi*t/5);

rng(0)
n = randn(1000,1);
xn = x + n;

xt = x + 0.005*t;
```

We then compute the periodogram by calculating the Fourier transform of the sequence x . The fastest possible Fourier transform using `fft` computes the Fourier transform for `nfft` frequencies, where `nfft` is the next power of two closest to the number of data points n in the original signal x . Since the length of the data series is $n=1000$, the Fourier transform is computed for `nfft=1024` frequencies, while the signal is padded with `nfft-n=24` zeros.

```
Xxx = fft(x,1024);
```

If `nfft` is even, as in our example, then `Xxx` is symmetric. For example, as the first $(1+nfft/2)$ points in `Xxx` are unique, the remaining points are symmetrically redundant. The power spectral density is defined as $Pxx2=(abs(Xxx).^2)/Fs$, where Fs is the sampling frequency. The function `periodogram` also scales the power spectral density by the length of the data series, i.e. it divides by $Fs=1$ and `length(x)=1000`.

```
Pxx2 = abs(Xxx).^2/1000;
```

We now drop the redundant part of the power spectrum and use only the first $(1+nfft/2)$ points. We also multiply the power spectral density by two to keep the same energy as in the symmetric spectrum, except for the first data point.

```
Pxx = [Pxx2(1); 2*Pxx2(2:512)];
```

The corresponding frequency axis runs from 0 to $Fs/2$ in $Fs/(nfft-1)$ steps, where $Fs/2$ is the Nyquist frequency. Since $Fs = 1$ in our example, the frequency axis is

```
f = 0 : 1/(1024-1) : 1/2;
```

We then plot the power spectral density `Pxx` in the Nyquist frequency range from 0 to $Fs/2$, which in our example is from 0 to 1/2. The Nyquist frequency range corresponds to the first 512 or `nfft/2` data points. We can plot the power spectral density over the frequency by typing

```
plot(f,Pxx), grid
```

The graphical output shows that there are three significant peaks at the positions of the original frequencies of the three sine waves (1/50, 1/15, and 1/5). Alternatively, we can also plot the power spectral density over the period by typing

```
plot(1./f,Pxx), axis([0 100 0 1000]), grid
```

where we observe the three periods 50, 15, and 5, as expected. Since the values on the x -axis of this plot are not evenly spaced (in contrast to those on the frequency axis), we find the long periods poorly resolved and a broad peak at a period of 50 in this graphics. The code for the power spectral density can be rewritten to make it independent of the sampling frequency,

```

Fs = 1;

t = 1/Fs : 1/Fs : 1000/Fs; t = t';
x = 2*sin(2*pi*t/50) + sin(2*pi*t/15) + 0.5*sin(2*pi*t/5);

nfft = 2^nextpow2(length(t));
Xxx = fft(x,nfft);

Pxx2 = abs(Xxx).^2 / Fs /length(x);
Pxx = [Pxx2(1); 2*Pxx2(2:512)];
f = 0 : Fs/(nfft-1) : Fs/2;

plot(f,Pxx), grid
axis([0 0.5 0 max(Pxx)])

```

where the function `nextpow2` computes the next power of two closest to the length of the time series $x(t)$. This code allows the sampling frequency to be modified and the differences in the results to be explored. We can now compare the results with those obtained using the function `periodogram(x,window,nfft,fs)`.

```
[Pxx,f] = periodogram(x,[],1024,1);
```

This function allows the windowing of the signals with various window shapes to overcome spectral leakage. However, we use the default rectangular window by choosing an empty vector `[]` for `window` to compare the results with the above experiment. The power spectrum `Pxx` is computed using an FFT of length `nfft = 1024`, which is the next power of two closest to the length of the series $x(t)$ and which is padded with zeros to make up the number of data points to the value of `nfft`. A sampling frequency `fs` of one is used within the function in order to obtain the correct frequency scaling for the f -axis. We display the results by typing

```

plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')
title('Auto-Spectrum')

```

or alternatively

```
plot(1./f,Pxx), axis([0 100 0 1000]), grid
xlabel('Period')
ylabel('Power')
title('Auto-Spectrum')
```

The graphical output is almost identical to our Blackman-Tukey plot and again shows that there are three significant peaks at the positions of the original frequencies (or periods) of the three sine waves. The same procedure can also be applied to the noisy data:

```
[Pxx,f] = periodogram(xn,[], 1024,1);

plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')
title('Auto-Spectrum')
```

Let us now increase the noise level by introducing Gaussian noise with a mean of zero and a standard deviation of five.

```
rng(0)
n = 5 * randn(size(x));
xn = x + n;

[Pxx,f] = periodogram(xn,[],1024,1);

plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')
title('Auto-Spectrum')
```

This spectrum now resembles a real data spectrum in the earth sciences and the spectral peaks are set against a significant background noise level. The peak of the highest frequency even disappears into the noise and cannot be distinguished from maxima that are attributed to noise. Both spectra can be compared on the same plot (Fig. 5.6):

```
[Pxx,f] = periodogram(x,[],1024,1);
[Pxxn,f] = periodogram(xn,[],1024,1);

subplot(1,2,1)
plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')
```

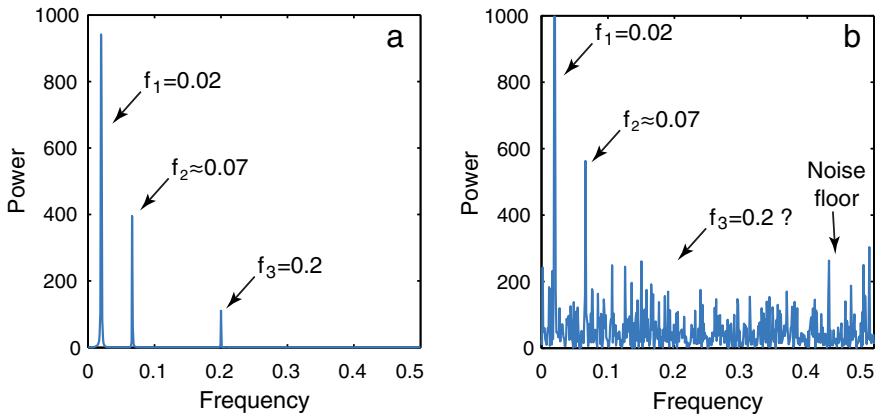


Fig. 5.6 Comparison of the auto-spectra for **a** noise-free, and **b** noisy synthetic signals with periods of $\tau_1 = 50$ ($f_1 = 0.02$), $\tau_2 = 15$ ($f_2 \approx 0.07$) and $\tau_3 = 5$ ($f_3 = 0.2$). The highest frequency peak disappears completely into the background noise and cannot be distinguished from peaks attributed to the Gaussian noise.

```
subplot(1,2,2)
plot(f,Pxxn), grid
xlabel('Frequency')
ylabel('Power')
```

Next, we explore the influence of a linear trend on a spectrum. Long-term trends are common features in earth science data. We will see that this trend is misinterpreted as a very long period by the FFT, producing a large peak with a frequency close to zero (Fig. 5.7).

```
[Pxx,f] = periodogram(x,[],1024,1);
[Pxxt,f] = periodogram(xt,[],1024,1);

subplot(1,2,1)
plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')

subplot(1,2,2)
plot(f,Pxxt), grid
xlabel('Frequency')
ylabel('Power')
```

To eliminate the long-term trend, we use the function `detrend`. This function removes linear trends, defined as either a single straight-line fit from the vector x , or a continuous, piecewise linear trend from x with one or more breakpoints defined by the user.

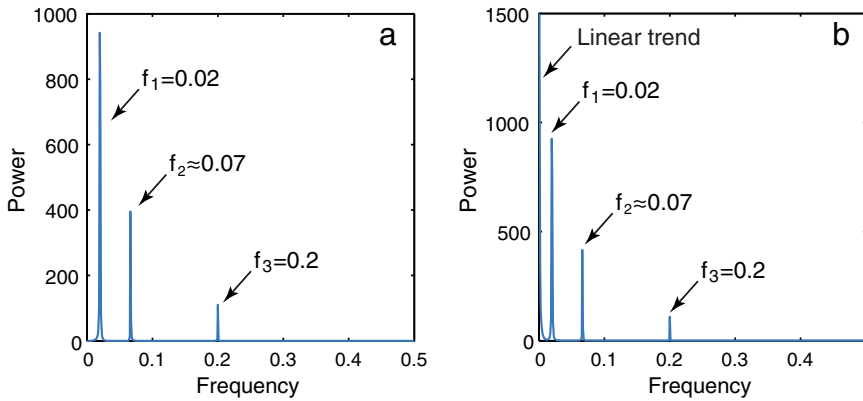


Fig. 5.7 Comparison of the auto-spectra for **a**, the original noise-free signal with periods of $\tau_1 = 50$ ($f_1 = 0.02$), $\tau_2 = 15$ ($f_2 \approx 0.07$) and $\tau_3 = 5$ ($f_3 = 0.2$), and **b**, the same signal overprinted on a linear trend. The linear trend is misinterpreted by the FFT as a very long period with a high amplitude.

```
xdt = detrend(xt);

subplot(2,1,1)
plot(t,x,'b-',t,xt,'r-'), grid
axis([0 200 -4 4])

subplot(2,1,2)
plot(t,x,'b-',t,xdt,'r-'), grid
axis([0 200 -4 4])
```

The resulting spectrum no longer shows the low-frequency peak.

```
[Pxxt,f] = periodogram(xt,[],1024,1);
[Pxxdt,f] = periodogram(xdt,[],1024,1);

subplot(1,2,1)
plot(f,Pxx), grid
xlabel('Frequency')
ylabel('Power')

subplot(1,2,2)
plot(f,Pxxdt), grid
xlabel('Frequency')
ylabel('Power')
```

Some data contain a high-order trend that can be removed by fitting a higher-order polynomial to the data and subtracting the corresponding $x(t)$ values.

We now use two sine waves with identical periodicities $\tau = 5$ (equivalent to $f = 0.2$) and amplitudes equal to two to compute the cross-spectrum of two time series. The sine waves show a relative phase shift of $t = 1$. In the argument of the second sine wave this corresponds to $2\pi/5$, which is one fifth of the full wavelength of $\tau = 5$.

```
clear

t = 1 : 1000;
x = 2*sin(2*pi*t/5);
y = 2*sin(2*pi*t/5 + 2*pi/5);

plot(t,x,'b-',t,y,'r-')
axis([0 50 -2 2]), grid
```

The cross-spectrum is computed with the function `cpsd`, which uses Welch's method for computing power spectra (Fig. 5.8). `Pxy` is complex and contains both amplitude and phase information.

```
[Pxy,f] = cpsd(x,y,[],0,1024,1);

plot(f,abs(Pxy)), grid
xlabel('Frequency')
ylabel('Power')
title('Cross-Spectrum')
```

The function `cpsd(x,y>window,noverlap,nfft,fs)` specifies the number of FFT points `nfft` used to calculate the cross power spectral density, which is 1,024 in our example. The parameter `window` is empty in our example and the default rectangular window is therefore used to obtain eight sections of `x` and `y`. The parameter `noverlap` defines the number of overlapping samples, which is zero in our example. The sampling frequency `fs` is 1 in this example. The coherence of the two signals is one for all frequencies, since we are working with noise-free data.

```
[Cxy,f] = mscohere(x,y,[],0,1024,1);

plot(f,Cxy), grid
xlabel('Frequency')
ylabel('Coherence')
title('Coherence')
```

We use the function `mscohere(x,y>window,noverlap,nfft,fs)` which specifies the number of FFT points `nfft = 1024`, the default rectangular window (`window=[]`), and no overlapping data points (`noverlap=0`). The complex part of `Pxy` is required for computing the phase shift between the two signals using the function `angle`.

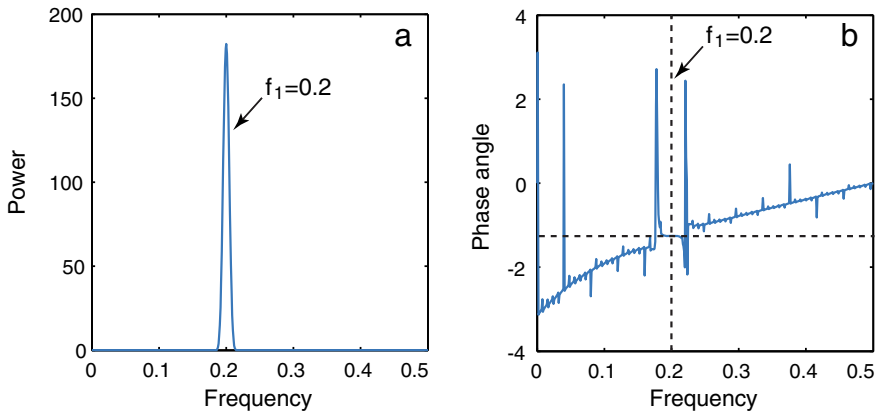


Fig. 5.8 Cross-spectrum of two sine waves with identical periodicities of $\tau = 5$ (equivalent to $f = 0.2$) and amplitudes of 2. The sine waves show a relative phase shift of $t = 1$. In the argument of the second sine wave this corresponds to $2\pi/5$, which is one fifth of the full wavelength of $\tau = 5$. **a** The magnitude shows the expected peak at $f = 0.2$. **b** The corresponding phase difference in radians at this frequency is 1.2566, which equals $(1.2566 \cdot 5)/(2 \cdot \pi) = 1.0000$, which is the relative phase shift of 1 between the two synthetic sine waves.

```

phase = angle(Pxy);

plot(f,phase), grid
xlabel('Frequency')
ylabel('Phase Angle')
title('Phase Spectrum')

```

The phase shift at a frequency of $f=0.2$ (period $\tau=5$) can be interpolated from the phase spectrum

```
interp1(f,phase,0.2)
```

which produces the output

```

ans =

-1.2566

```

The phase spectrum is normalized to one full period $\tau=2\pi$ and the phase shift of -1.2566 related to a period of 2π therefore becomes $(-1.2566 \cdot 5)/(2 \cdot \pi) = -1.0000$ related to the true period of $\tau=5$, which is the relative phase shift of 1 between the two synthetic sine waves.

We now use two sine waves with different periodicities to illustrate cross-spectral analysis. Both signals, x and y , have a periodicity of 5 but a phase shift of 1.

```
clear

t = 1 : 1000;
x = sin(2*pi*t/15) + 0.5*sin(2*pi*t/5);
y = 2*sin(2*pi*t/50) + 0.5*sin(2*pi*t/5+2*pi/5);

plot(t,x,'b-',t,y,'r-')
axis([0 100 -3 3]), grid
```

We can now compute the cross-spectrum P_{xy} , which clearly shows the common period of $\tau=5$ (or frequency of $f=0.2$).

```
[Pxy,f] = cpsd(x,y,[],0,1024,1);

plot(f, abs(Pxy)), grid
xlabel('Frequency')
ylabel('Power')
title('Cross-Spectrum')
```

The coherence shows a high value that is close to one at $f = 0.2$.

```
[Cxy,f] = mscohere(x,y,[],0,1024,1);

plot(f,Cxy), grid
xlabel('Frequency')
ylabel('Coherence')
title('Coherence')
```

The complex part of the cross-spectrum P_{xy} is required for calculating the phase shift between the two sine waves.

```
[Pxy,f] = cpsd(x,y,[],0,1024,1);
phase = angle(Pxy);

plot(f,phase), grid
```

The phase shift at a frequency of $f=0.2$ (period $\tau=5$) is

```
interp1(f,phase,0.2)
```

which produces the output

```
ans =

-1.2572
```

The phase spectrum is normalized to one full period $\tau = 2\pi$ and the phase shift of -1.2572 related to a period of 2π therefore equals $(-1.2572 \cdot 5)/(2 \cdot \pi) = -1.0004$ related to a period of 5, which is again the relative phase shift of 1 between the two synthetic sine waves.

5.5 Interpolating and Analyzing Unevenly-Spaced Data

We can now use our experience in analyzing evenly-spaced data to run a spectral analysis on unevenly-spaced data. Such data are very common in earth sciences, for example in the field of paleoceanography, where deep-sea cores are typically sampled at constant depth intervals. The transformation of evenly-spaced length-parameter data to time-parameter data in an environment with changing length-time ratios results in unevenly-spaced time series. Numerous methods exist for interpolating unevenly-spaced sequences of data or time series. The aim of these *interpolation techniques* for $x(t)$ data is to estimate the x -values for an equally-spaced t vector from the irregularly-spaced $x(t)$ actual measurements. *Linear interpolation* predicts the x -values by effectively drawing a straight line between two neighboring measurements and by calculating the x -value at the appropriate point along that line. However, this method has its limitations. It assumes linear transitions in the data, which introduces a number of artefacts (including the loss of high-frequency components of the signal) and limits the data range to that of the original measurements.

Cubic-spline interpolation is another method for interpolating data that are unevenly spaced. Cubic splines are piecewise continuous curves requiring at least four data points for each step. The method has the advantage that it preserves the high-frequency information contained in the data. However, steep gradients in the data sequence, which typically occur adjacent to extreme minima and maxima, could cause spurious amplitudes in the interpolated time series. Since all these (and other) interpolation techniques might introduce artefacts into the data, it is always advisable to (1) keep the total number of data points constant before and after interpolation, (2) report the method employed for estimating the evenly-spaced data sequence, and (3) explore the effect of interpolation on the variance of the data.

Following this brief introduction to interpolation techniques we can apply the most popular linear and cubic spline interpolation techniques to unevenly-spaced data. Having interpolated the data we can then use the spectral tools that have previously been applied to evenly-spaced data (Sects. 5.3 and 5.4). We must first load the two time series:

```
clear, close all, clc

series1 = load('series1.txt');
series2 = load('series2.txt');
```

Both synthetic data sets contain a two-column matrix with 339 rows. The first column contains ages in kiloyears (kyrs), which are unevenly spaced. The second column contains oxygen-isotope values measured on calcareous microfossils

(foraminifera). The data sets contain 100, 40 and 20 kyr cyclicities and they are overlain by Gaussian noise. In the 100 kyr frequency band, the second data series has shifted by 5 kyrs with respect to the first data series. To plot the data we type

```
plot(series1(:,1),series1(:,2))
figure
plot(series2(:,1),series2(:,2))
```

The statistics for the spacing of the first data series can be computed by

```
intv1 = diff(series1(:,1));
plot(intv1)
```

The plot shows that the spacing varies around a mean interval of 3 kyrs, with a standard deviation of ca. 1 kyr. The minimum and maximum values for the time axis

```
min(series1(:,1))
max(series1(:,1))
```

of $t_{min}=0$ and $t_{max}=997$ kyrs provide some information about the temporal range of the data. The second data series

```
intv2 = diff(series2(:,1));

plot(intv2)

min(series2(:,1))
max(series2(:,1))
```

has a similar range, from 0 to 997 kyrs. We see that both series have a mean spacing of 3 kyrs and range from 0 to $\sim 1,000$ kyrs. We now interpolate the data to an evenly-spaced time axis. While doing this, we follow the rule that the number of data points should not be increased. The new time axis runs from 0 to 996 kyrs, with 3 kyr intervals.

```
t = 0 : 3 : 996;
```

We can now interpolate the two time series to this axis with *linear* and *spline* interpolation methods, using the function `interp1`.

```
series1L = interp1(series1(:,1),series1(:,2),t,'linear');
series1S = interp1(series1(:,1),series1(:,2),t,'spline');
```

```
series2L = interp1(series2(:,1),series2(:,2),t,'linear');
series2S = interp1(series2(:,1),series2(:,2),t,'spline');
```

In the linear interpolation method the linear interpolant is the straight line between neighboring data points. In the *spline* interpolation the interpolant is a piecewise polynomial (the *spline*) between these data points. The method *spline* with *interp1* uses a piecewise cubic spline interpolation, i.e. the interpolant is a third-degree polynomial. The results are compared by plotting the first series before and after interpolation.

```
plot(series1(:,1),series1(:,2),'ko'), hold on
plot(t,series1L,'b-',t,series1S,'r-'), hold off
```

We can already observe some significant artefacts at ca. 370 kyrs. Whereas the linearly-interpolated points are always within the range of the original data, the spline interpolation method produces values that are unrealistically high or low (Fig. 5.9). The results can be compared by plotting the second data series.

```
plot(series2(:,1),series2(:,2),'ko'), hold on
plot(t,series2L,'b-',t,series2S,'r-'), hold off
```

In this series, only a few artefacts can be observed. The function *interp1* also provides an alternative to *spline*, which is *pchip*. The name *pchip* stands for *Piecewise Cubic Hermite Interpolating Polynomial* and this method performs a shape-preserving piecewise cubic interpolation. The function avoids the typical artefacts of the splines as it preserves the original shape of the data series. We can apply the function used above to calculate the power spectrum, computing the FFT for 256 data points with a sampling frequency of $1/3 \text{ kyr}^{-1}$.

```
[Pxx,f] = periodogram(series1L,[],256,1/3);

plot(f,Pxx)
xlabel('Frequency')
ylabel('Power')
title('Auto-Spectrum')
```

Significant peaks occur at frequencies of approximately 0.01, 0.025 and 0.05, corresponding approximately to the 100, 40 and 20 kyr cycles. Analysis of the second time series

```
[Pxx,f] = periodogram(series2L,[],256,1/3);

plot(f,Pxx)
xlabel('Frequency')
ylabel('Power')
title('Auto-Spectrum')
```

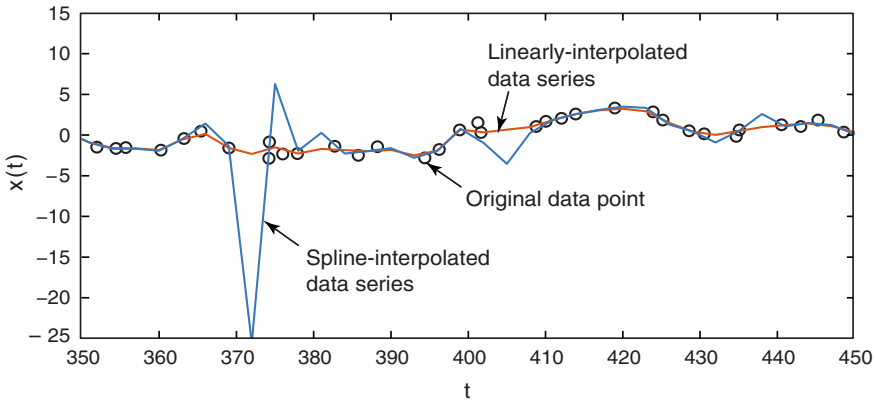


Fig. 5.9 Interpolation artefacts. Whereas the linearly interpolated points are always within the range of the original data, the spline interpolation method results in unrealistic high and low values.

also yields significant peaks at frequencies of 0.01, 0.025 and 0.05 (Fig. 5.10). We now compute the cross-spectrum for both data series.

```
[Pxy,f] = cpsd(series1L,series2L,[],128,256,1/3);

plot(f,abs(Pxy))
xlabel('Frequency')
ylabel('Power')
title('Cross-Spectrum')
```

The correlation, as indicated by the high value for the coherence, is quite convincing.

```
[Cxy,f] = mscohere(series1L,series2L,[],128,256,1/3);

plot(f,Cxy)
xlabel('Frequency')
ylabel('Magnitude Squared Coherence')
title('Coherence')
```

We can observe a fairly high coherence at frequencies of 0.01, 0.025 and 0.05. The complex part of P_{xy} is required for calculating the phase difference for each frequency.

```
phase = angle(Pxy);  
  
plot(f,phase)  
xlabel('Frequency')  
ylabel('Phase Angle')  
title('Phase spectrum')
```

The phase shift at a frequency of $f = 0.01$ is calculated using

```
interp1(f,phase,0.01)
```

which produces the output

```
ans =  
  
-0.2796
```

The phase spectrum is normalized to a full period $\tau=2\pi$ and the phase shift of -0.2796 therefore equals $(-0.2796 \cdot 100 \text{ kyrs})/(2 \cdot \pi) = -4.45 \text{ kyrs}$. This corresponds roughly to the phase shift of 5 kyrs introduced to the second data series with respect to the first series.

The Signal Processing Toolbox also contains a GUI function named `sptool` (for *Signal Processing Tool*), which is a more convenient tool for spectral analysis but is not described in any detail herein. According to the MATLAB documentation, this GUI will be discontinued in the future and you are therefore recommended to use the much more comprehensive *Signal Analyzer*, which is started with `signalAnalyzer` in the Command Window.

5.6 Evolutionary Power Spectrum

The amplitude of spectral peaks usually varies with time. This is particularly true for paleoclimate time series. Paleoclimate records usually show trends, not only in the mean and variance but also in the relative contributions of rhythmic components such as the Milankovitch cycles in marine oxygen-isotope records. Evolutionary power spectra have the ability to map such changes in the frequency domain. The *evolutionary* or *windowed power spectrum* is a modification of the method introduced in Sect. 5.3, which computes the spectrum of overlapping segments of the time series. These overlapping segments are relatively short compared to the windowed segments used by the Welch method (Sect. 5.3), which is used to increase the signal-to-noise ratio of power spectra. The evolutionary power spectrum method therefore uses the Short-Time Fourier Transform (STFT) instead of the Fast Fourier Transformation (FFT). The output from the evolutionary power spectrum is the short-term, time-localized frequency content of the signal. There are various methods that can be used to display the results.

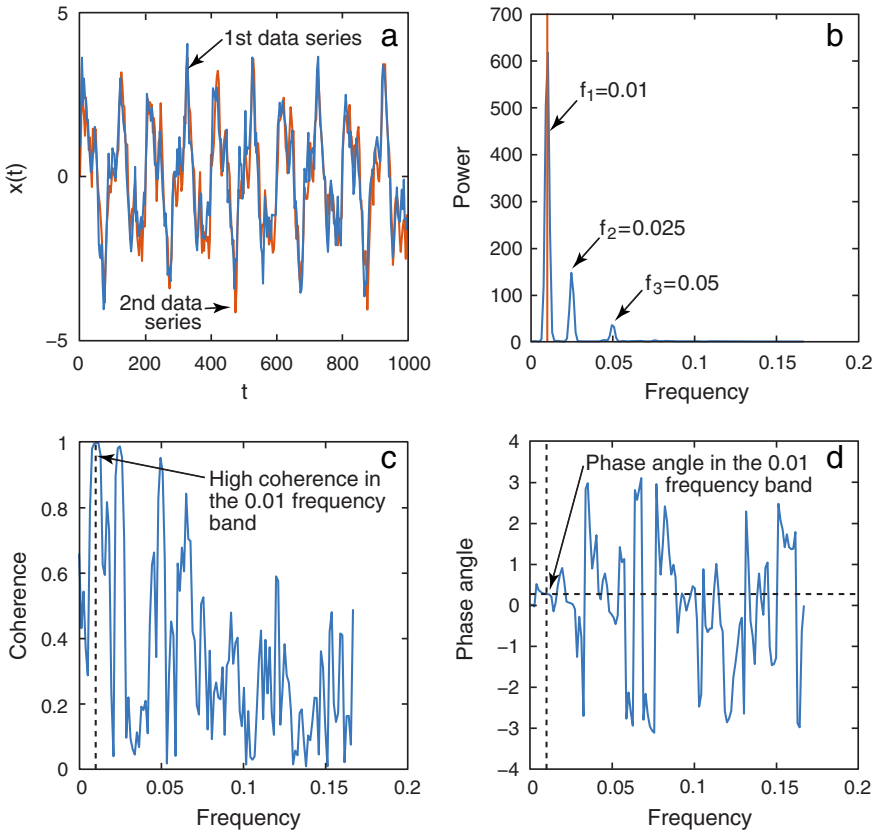


Fig. 5.10 Result from cross-spectral analysis of the two linearly-interpolated signals: **a** signals in the time domain, **b** cross-spectrum of both signals, **c** coherence of the signals in the frequency domain, and **d** phase spectrum in radians.

For instance, time and frequency can be plotted on the x - and y -axes, respectively, or vice versa, with the color of the plot being dependent on the height of the spectral peaks.

As an example we use a data set that is similar to those used in Sect. 5.5. The data series contains three main periodicities of 100, 40 and 20 kyrs and additive Gaussian noise. The amplitudes, however, change through time and this example can therefore be used to illustrate the advantage of the evolutionary power spectrum method. In our example the 40 kyr cycle appears only after ca. 450 kyrs, whereas the 100 and 20 kyr cycles are present throughout the time series. We first load from the file *series3.txt* and display the data (Fig. 5.11).

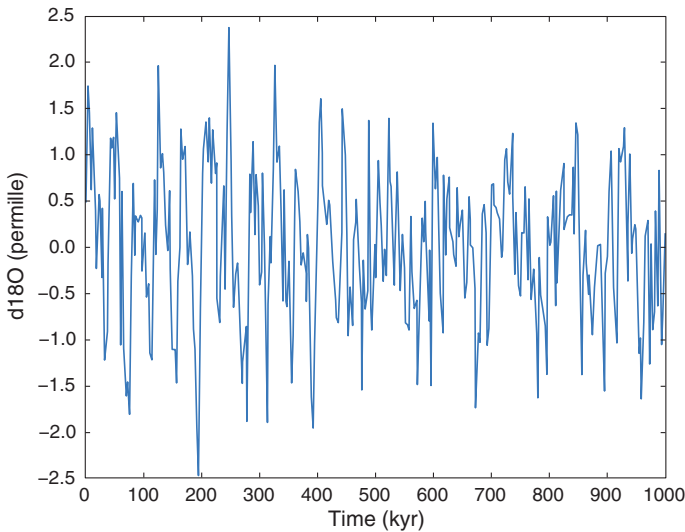


Fig. 5.11 Synthetic data set containing three main periodicities of 100, 40, and 20 kyr and additive Gaussian noise. Whereas the 100 and 20 kyr cycles are present throughout the time series, the 40 kyr cycle only appears at around 450 kyr before present.

```
clear, close all, clc

series3 = load('series3.txt');
plot(series3(:,1),series3(:,2))
xlabel('Time (kyr)')
ylabel('d18O (permille)')
title('Signal with Varying Cyclicities')
```

Since both the standard and the evolutionary power spectrum methods require evenly-spaced data, we interpolate the data to an evenly-spaced time vector t , as demonstrated in Sect. 5.5.

```
t = 0 : 3 : 1000;
series3L = interp1(series3(:,1),series3(:,2),t,'linear');
```

We then compute a non-evolutionary power spectrum for the full length of the time series (Fig. 5.12). This exercise helps us to compare the differences between the results of the standard and the evolutionary power spectrum methods.

```
[Pxx,f] = periodogram(series3L,[],1024,1/3);
plot(f,Pxx)
xlabel('Frequency')
ylabel('Power')
title('Power Spectrum')
```

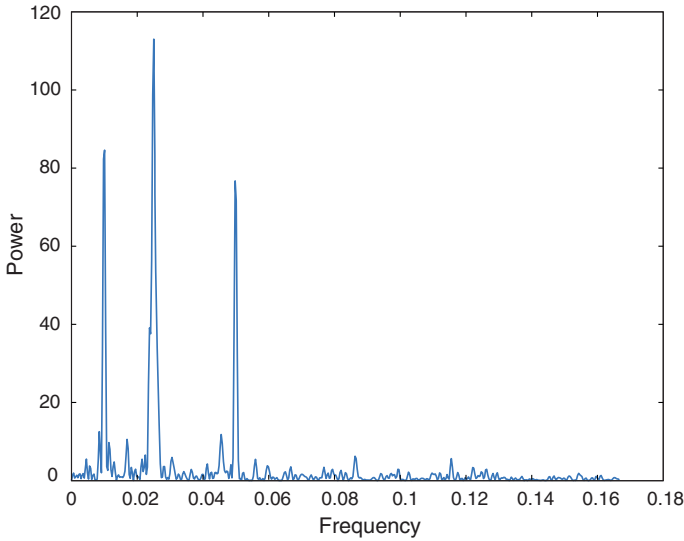


Fig. 5.12 Power spectrum for the complete time series, showing significant peaks at 100, 40 and 20 kyr. The plot, however, does not provide any information on the temporal behavior of the cyclicities.

The auto-spectrum shows significant peaks at 100, 40 and 20 kyr cyclicities, as well as some noise. The power spectrum, however, does not provide any information about fluctuations in the amplitudes of these peaks. The non-evolutionary power spectrum simply represents an average of the spectral information contained in the data.

We now use the function `spectrogram` to map the changes in the power spectrum with time. By default, the time series is divided into eight segments with a 50% overlap. Each segment is windowed with a Hamming window to suppress spectral leakage (Sect. 5.3). The function `spectrogram` uses similar input parameters to those used in `periodogram` in Sect. 5.3. We then compute the evolutionary power spectrum for a window of 64 data points with a 50 data point overlap. The STFT is computed for `nfft = 256`. Since the spacing of the interpolated time vector is 3 kyr, the sampling frequency is $1/3 \text{ kyr}^{-1}$.

```
[s,f,t] = spectrogram(series3L,64,50,256,1/3);
pcolor(f,t,(abs(s))'), shading flat
title('Evolutionary Power Spectrum')
xlabel('Frequency (1/kyr)')
ylabel('Time (kyr)')
```

We use `pcolor` to display the output of `spectrogram` in a color plot (Fig. 5.13) that displays yellow vertical stripes representing significant maxima at frequencies of 0.01 and 0.05 kyr^{-1} (i.e. every 100 and 20 kyrs). There is also a 40 kyr cycle (corresponding to a frequency of 0.025 kyr^{-1}), but this only occurs after ca. 450 kyrs, as documented by the vertical red stripe in the lower half of the graph.

To improve the visibility of the significant cycles, the colors used in the graph can be modified using the colormap editor.

`colormapeditor`

The colormap editor displays the colormap of the figure as a strip of rectangular cells. The nodes that separate regions of uniform slope in the RGB colormap can be shifted using the computer mouse, which introduces distortions in the colormap and results in modification of the spectrogram colors. For example shifting the yellow node towards the right increases the contrast between the vertical peak areas at 100, 40 and 20 kyrs, and the background.

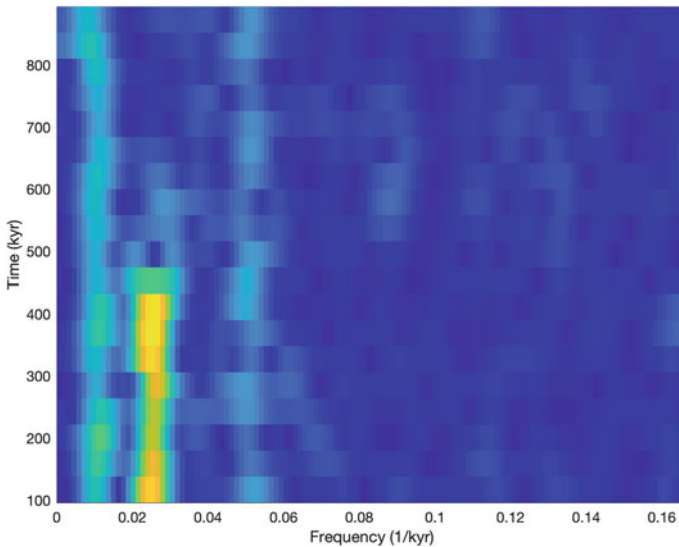


Fig. 5.13 Evolutionary power spectrum using `spectrogram`, which computes the short-time Fourier transform STFT of overlapping segments of the time series. We use a Hamming window of 64 data points and with a 50 data point overlap. The STFT is computed for `nfft=256`. Since the spacing of the interpolated time vector is 3 kyrs, the sampling frequency is $1/3 \text{ kyr}^{-1}$. We use `pcolor` to display the output of `spectrogram` in a color plot that displays yellow vertical stripes representing significant maxima at frequencies of 0.01 and 0.05 kyr^{-1} (i.e. every 100 and 20 kyrs). The plot shows the onset of the 40 kyr cycle at around 450 kyrs before present.

5.7 Lomb-Scargle Power Spectrum

The power spectrum methods introduced in the previous sections require evenly-spaced data. In earth sciences, however, time series are often unevenly spaced. Although interpolating the unevenly-spaced data to a grid of evenly-spaced times is one way to overcome this problem (Sect. 5.5), interpolation introduces numerous artefacts into the data, in both the time and frequency domains. For this reason an alternative method of time-series analysis has become increasingly popular in earth sciences, the *Lomb-Scargle algorithm* (e.g. Scargle 1981, 1982, 1989, 1990; Schulz and Stattegger 1998; Press et al. 2007).

The Lomb-Scargle algorithm only evaluates the data of the time series at the times t_i that are actually measured. Assuming a series $y(t)$ of N data points, the Lomb-Scargle normalized periodogram P_x , as a function of angular frequency $\omega = 2\pi f > 0$, is given by

$$P_x(\omega) = \frac{1}{2s^2} \left\{ \frac{\left[\sum_j (y_i - \bar{y}) \cos \omega(t_j - \tau) \right]^2}{\sum_j \cos^2 \omega(t_j - \tau)} + \frac{\left[\sum_j (y_i - \bar{y}) \sin \omega(t_j - \tau) \right]^2}{\sum_j \sin^2 \omega(t_j - \tau)} \right\}$$

where

$$\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

and

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{y})^2$$

are the arithmetic mean and the variance of the data (e.g. Scargle 1981, 1982, 1989, 1990; Press et al. 2007) (Sect. 3.2). The constant τ , which is defined by the relationship

$$\tan(2\omega\tau) = \frac{\sum_j \sin 2\omega t_j}{\sum_j \cos 2\omega t_j}$$

is an offset that makes $P_x(\omega)$ independent of shifting the t_i values by any constant amount. Scargle (1982) showed that this particular choice of the offset τ has the consequence that the solution for $P_x(\omega)$ is identical to a least-squares fit of sine and cosine functions to the data series $y(t)$:

$$y(t) = A \cos \omega t + B \sin \omega t$$

The least-squares fit of harmonic functions to data series in conjunction with spectral analysis had previously been investigated by Lomb (1972), and hence the method is called the normalized Lomb-Scargle Fourier transform. The term *normalized* refers to the factor s^2 in the dominator of the equation for the periodogram.

Scargle (1982) has shown that the Lomb-Scargle periodogram has an exponential probability distribution with a mean equal to one, assuming that the noise is Gaussian distributed. The probability that $P_x(\omega)$ will be between some positive quantity z and $z + dz$ is $\exp(-z)dz$. If we scan M independent frequencies, the probability of none of them having a value larger than z is $(1-\exp(-z))^M$. We can therefore compute the false-alarm probability of the null hypothesis (i.e. the probability that a given peak in the periodogram is not significant) (Press et al. 2007):

$$P(> z) \equiv 1 - (1 - e^{-z})^M$$

Press et al. (2007) also suggested using the Nyquist criterion (Sect. 5.2) to determine the number of independent frequencies M , assuming that the data were evenly spaced. In this case, the appropriate value for the number of independent frequencies is $M=2N$, where N is the length of the time series.

The following MATLAB code is based on the above equations for the Lomb-Scargle normalized periodogram. We first load the synthetic data that were generated to illustrate the use of the evolutionary or windowed power spectrum method in Sect. 5.6. The data contain periodicities of 100, 40 and 20 kyrs, as well as additive Gaussian noise, and are unevenly spaced about the time axis. We define two new vectors t and x that contain the original time vector and the synthetic oxygen-isotope data sampled at times t .

```
clear, close all, clc

series3 = load('series3.txt');
t = series3(:,1);
x = series3(:,2);
```

We then generate a frequency axis f . Since the Lomb-Scargle method is not able to deal with the frequency of zero (i.e. with an infinite period) we start at a frequency value that is equivalent to the spacing of the frequency vector. The variable $ofac$ is the oversampling parameter that influences the resolution of the frequency axis about the $N(\text{frequencies})=N(\text{datapoints})$ case. We also need the highest frequency f_{hi} that can be analyzed by the Lomb-Scargle algorithm: the Nyquist frequency f_{nyq} that would be obtained if the N data points were evenly spaced over the same time interval is commonly used for f_{hi} . The following code uses the input parameter $hifac$, which is defined by Press et al. (2007) as $hifac=f_{hi}/f_{nyq}$.

```

int = mean(diff(t));
ofac = 4; hifac = 1;
f = ((2*int)^(-1))/(length(x)*ofac): ...
    ((2*int)^(-1))/(length(x)*ofac): ...
    hifac*(2*int)^(-1);
f = f';

```

where `int` is the mean sampling interval. We can now compute the normalized Lomb-Scargle periodogram `px` as a function of the angular frequency `wrun`, translating the above equations for τ and P_x into MATLAB code.

```

for k = 1:length(f)
    wrun = 2*pi*f(k);
    tau = (1/2*wrun)*atan2(sum(sin(2*wrun*t)), ...
        sum(cos(2*wrun*t)));
    px(k) = 1/ (2*var(x)) * ...
        (...
        (sum((x - mean(x)).*cos(wrun*(t-tau))))).^2 ...
        /sum((cos(wrun*(t-tau))).^2) + ...
        (sum((x - mean(x)).*sin(wrun*(t-tau))))).^2 ...
        /sum((sin(wrun*(t-tau))).^2) ...
        );
end
px = px';

```

The significance level for any peak in the power spectrum `px` can now be computed. The variable `prob` indicates the false-alarm probability for the null hypothesis: a low `prob` therefore indicates a highly significant peak in the power spectrum.

```

prob = 1-(1-exp(-px)).^(2*length(x));

```

We now plot the power spectrum and the probabilities (Fig. 5.14):

```

plot(f,px)
xlabel('Frequency')
ylabel('Power')
title('Lomb-Scargle Power Spectrum')

figure
plot(f,prob)
xlabel('Frequency')
ylabel('Probability')
title('Probabilities')

```

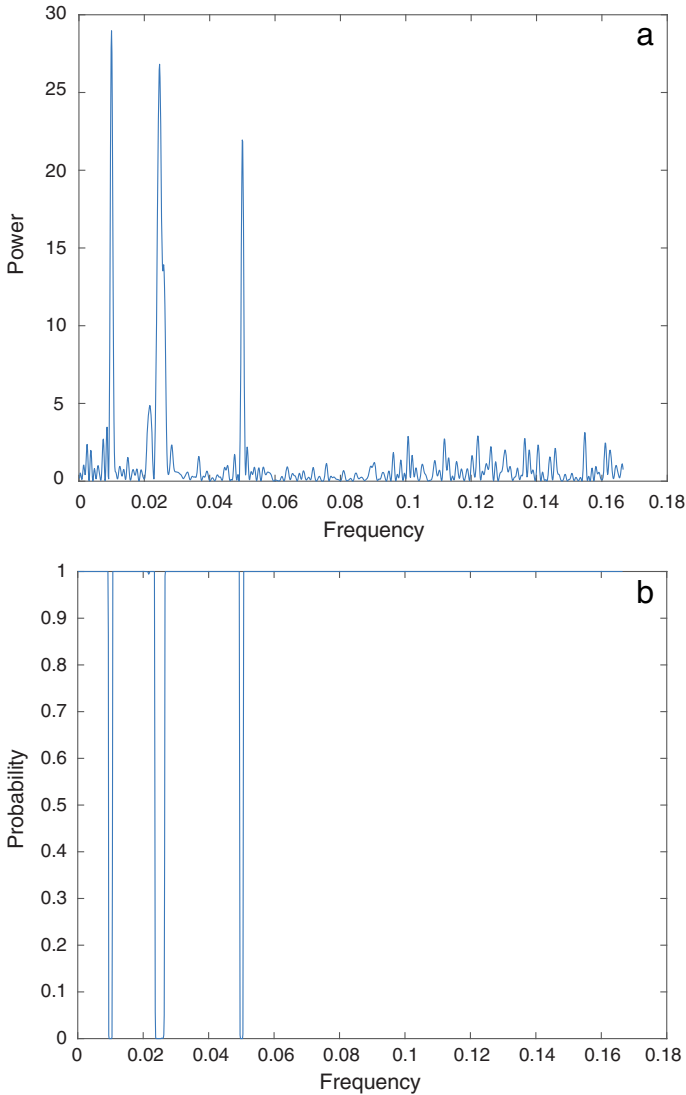


Fig. 5.14 **a** Lomb-Scargle power spectrum and **b** the false-alarm probability of the null hypothesis. The plot suggests that the 100, 40 and 20 kyr cycles are highly significant.

The two plots suggest that all three peaks are highly significant since the errors are extremely low at the cyclicities of 100, 40 and 20 kyrs.

An alternative way of displaying the significance levels was suggested by Press et al. (2007). In this method the equation for the false-alarm probability of the null hypothesis is inverted to compute the corresponding power of the significance

levels. As an example we choose a significance level of 95%. However, this number can also be replaced by a vector of several significance levels such as `signif=[0.90 0.95 0.99]`. We can now type

```
m = floor(0.5*ofac*hifac*length(x));
effm = 2*m/ofac;
signif = 0.95;
levels = log((1-signif.^(1/effm)).^(-1));
```

where `m` is the true number of independent frequencies and `effm` is the effective number of frequencies using the oversampling factor `ofac`. The second plot displays the spectral peaks and the corresponding probabilities.

```
plot(f,px)
hold on
for k = 1:length(signif)
    line(f,levels(:,k)*ones(size(f)),'LineStyle','-')
end
xlabel('Frequency')
ylabel('Power')
title('Lomb-Scargle Power Spectrum')
hold off
```

All three spectral peaks at frequencies of 0.01, 0.025 and 0.05 kyr⁻¹ exceed the 95% significant level, suggesting that they represent significant cyclicities. We have therefore obtained similar results to those obtained using the periodogram method. However, the Lomb-Scargle method has the advantage that it does not require any interpolation of unevenly-spaced data, as well as permitting quantitative significance testing.

Since the MATLAB R2014b release the Signal Processing Toolbox includes the `plomb` function to compute the Lomb-Scargle periodogram. Using an oversampling factor of four

```
[px,f] = plomb(x,t,f,'normalized');

plot(f,px)
xlabel('Frequency')
ylabel('Power')
title('Lomb-Scargle Power Spectrum')
hold off
```

yields exactly the same results. Herein we use `normalized` as the spectrum type to obtain a Lomb-Scargle normalized periodogram, which is normalized by dividing `px` by `2*var(x)` as in our MATLAB code above.

5.8 Wavelet Power Spectrum

Section 5.6 demonstrated the use of a modification to the power spectrum method for mapping changes in cyclicity through time. A similar modification could, in theory, be applied to the Lomb-Scargle method, which would have the advantage that it could then be applied to unevenly-spaced data. Both methods, however, assume that the data are composites of sine and cosine waves that are globally uniform in time and have infinite time spans. The evolutionary power spectrum method divides the time series into overlapping segments and computes the Fourier transform of these segments. To avoid spectral leakage, the data are multiplied by windows that are smooth bell-shaped curves with positive values (Sect. 5.3). The higher the temporal resolution of the evolutionary power spectrum the lower the accuracy of the result. Moreover, short time windows contain a large number of high-frequency cycles whereas the low-frequency cycles are underrepresented.

In contrast to the Fourier transform, the *wavelet transform* uses base functions (*wavelets*) that have smooth ends per se (Lau and Weng 1995; Mackenzie et al. 2001). Wavelets are small packets of waves; they are defined by a specific frequency and decay towards either end. Since wavelets can be stretched and translated in both frequency and time, with a flexible resolution, they can easily map changes in the time-frequency domain. We use the functions for wavelet analysis that are included in the Wavelet Toolbox (MathWorks 2020b). There is also, however, a very popular wavelet toolbox produced by Torrence and Compo (1998), which is freely available online from

<http://paos.colorado.edu/research/wavelets/>

A wavelet transformation mathematically decomposes a signal $y(t)$ into elementary functions $\psi_{a,b}(t)$ derived from a *mother wavelet* $\psi(t)$, by dilation and translation,

$$\psi_{a,b} = \frac{1}{(a)^{1/2}} \psi\left(\frac{t-b}{a}\right)$$

where b denotes the position (translation) and a (>0) the scale (dilation) of the wavelet (Lau and Weng 1995). The wavelet transform of the signal $y(t)$ about the mother wavelet $\psi(t)$ is defined as the convolution integral

$$W(b,a) = \frac{1}{(a)^{1/2}} \int \psi^* \left(\frac{t-b}{a} \right) y(t) dt$$

where ψ^* is the complex conjugate of ψ . There are many mother wavelets available in the literature, such as the classic *Haar* wavelet, the *Morlet* wavelet, or the *Daubechies* wavelet. The most popular wavelet in geosciences is the Morlet wavelet introduced by French geophysicist Jean Morlet (1931–2007), which is defined by

$$\psi_0(\eta) = \pi^{-1/4} \exp(i\omega_0\eta) \exp(-\eta^2/2)$$

where η is the time and ω_0 is the wave number (Torrence and Compo 1998). The wave number is the number of oscillations within the wavelet itself. We can easily compute a discrete version of the Morlet wavelet wave by translating the above equation into MATLAB code, where `eta` is the non-dimensional time and `w0` is the wave number. Changing `w0` produces wavelets with different wave numbers. Note that it is important not to use `i` for index in for loops, since it is used here for imaginary unit (Fig. 5.15).

```
clear, close all, clc

eta = -10 : 0.1 : 10;
w0 = 6;
wave = pi.^(-1/4) .* exp(i*w0*eta) .* exp(-eta.^2/2);

plot(eta,wave)
xlabel('Position')
ylabel('Scale')
title('Morlet Mother Wavelet')
```

In order to familiarize ourselves with wavelet power spectra, we calculate a continuous wavelet transform (CWT) of the synthetic data used in Sect. 5.6. We first load the synthetic data contained in file `series3.txt`, remembering that the data contain periodicities of 100, 40, and 20 kyrs, as well as additive Gaussian noise, and that they are unevenly spaced about the time axis (Figs. 5.16 and 5.17).

```
clear

series3 = load('series3.txt');
```

As for the Fourier transform (Sects. 5.3 to 5.6) and in contrast to the Lomb-Scargle algorithm (Sect. 5.7), the wavelet transform requires evenly-spaced data and we therefore interpolate the data using `interp1`.

```
t = 0 : 3 : 1000;
series3L = interp1(series3(:,1),series3(:,2),t,'linear');
```

If our time series has a trend we need to remove the trend before running the wavelet analysis (see Sect. 5.4):

```
series3L = detrend(series3L);
```

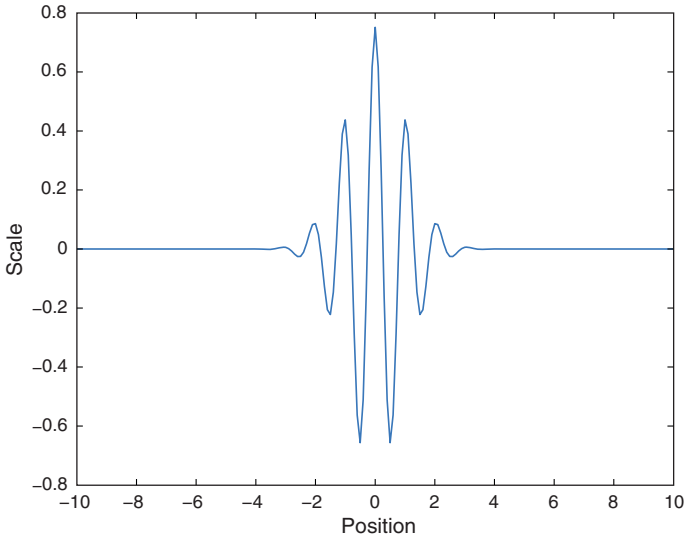


Fig. 5.15 Morlet mother wavelet with wave number 6.

We then use `cwt` with the data series `series3L` and the inverse of our sampling interval as the sampling frequency. In our example, the sampling interval is 3, therefore the sampling frequency is $1/3$. The output is the wavelet transform `wt`, the frequencies `f`, and the cone of influence `coi`, which is used to mark the area where edge effects occur in the CWT when using the new `cwt` function introduced with the MATLAB R2016b release. The `cwt` function uses an FFT-based algorithm similar to the one introduced by Torrence and Compo (1998), which was already used by the (no longer supported) function `cwtft` and is similar to the old version of `cwt` used until the 4th edition of this book (Trauth 2015). The advantage of the new `cwt` function over the old version is that it provides defaults, which allows a wavelet analysis to be run without worrying about the settings. The `cwt` function uses a *Morse* wavelet as the default setting. If we want to use a *Morlet* wavelet, we have to use the `amor` option.

```
[wt,f,coi] = cwt(series3L,1/3,'amor');
```

We now display the wavelet power spectrum. We can change the limits of the axes, the colormap and many other things. We can see cycles with frequencies of 0.01/kyr (corresponding to a period of 100 kyr), 0.025 (40 kyr) and 0.05 (20 kyr). The great thing about wavelets is that they display an evolutionary spectrum, i.e. the appearance and disappearance of cycles through time.

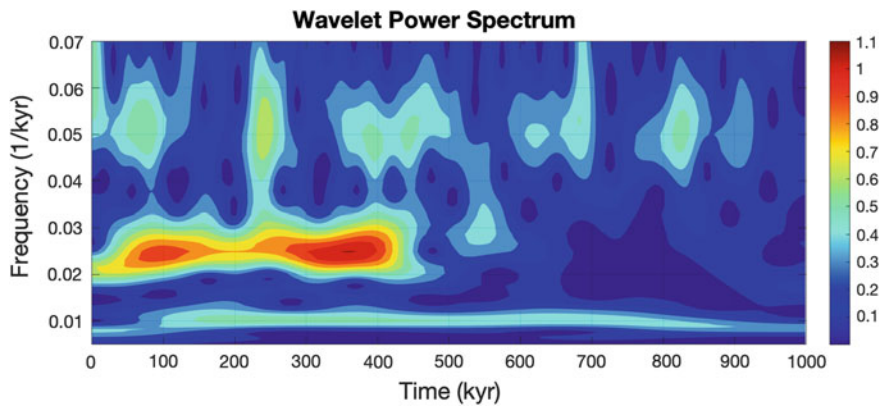


Fig. 5.16 Wavelet power spectrum for the synthetic data series contained in *series_3.txt*. The plot clearly shows significant periodicities at frequencies of 0.01, 0.025, and 0.05 kyr⁻¹ corresponding to the 100, 40, and 20 kyr cycles. The 100 kyr cycle is present throughout the entire time series, whereas the 40 kyr cycle only appears at around 450 kyrs before present. The 20 kyr cycle is relatively weak but is probably present throughout the entire time series. The wavelet power spectrum has been calculated using the continuous 1D wavelet transform `cwt`.

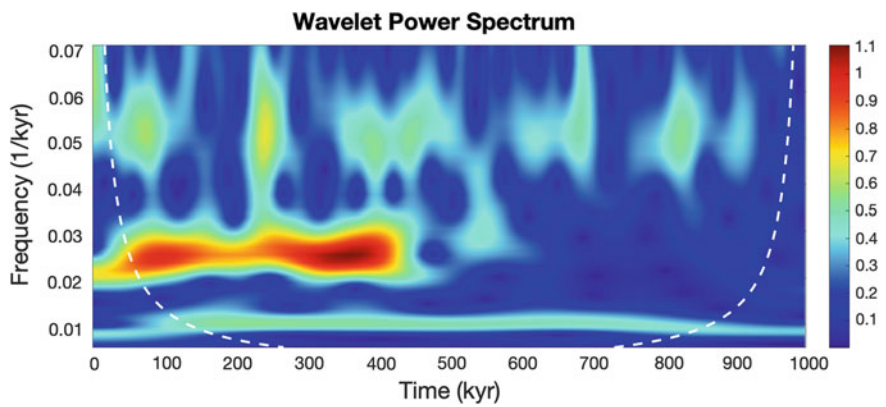


Fig. 5.17 Wavelet power spectrum for the synthetic data series contained in *series_3.txt*. The result is similar to Fig. 5.16 except that here the function `pcolor` was used to visualize the wavelet power spectrum. The cone of interest `coi` is also displayed as a dashed white line to mark the area where edge effects occur in the wavelet power spectrum. This version of the graphics is similar to the representation of the `cwt` function as it is generated without output parameters.

```

figure('Position',[100 300 800 300],...
      'Color',[1 1 1]);
contour(t,f,abs(wt),...
      'LineStyle','none',...
      'LineColor',[0 0 0],...
      'Fill','on')
xlabel('Time (kyr)')
ylabel('Frequency (1/kyr)')
title('Wavelet Power Spectrum')
set(gcf,'Colormap',jet)
set(gca,'XLim',[0 1000],...
      'YLim',[0.005 0.07],...
      'XGrid','On',...
      'YGrid','On')
colorbar

```

We now display the same plot with periods instead of frequencies on the y-axis. We can see cycles with a period of 100, 40 and 20 kyr.

```

figure('Position',[100 300 800 300],...
      'Color',[1 1 1]);
contour(t,1./f,abs(wt),...
      'LineStyle','none',...
      'LineColor',[0 0 0],...
      'Fill','on')
xlabel('Time (kyr)')
ylabel('Period (kyr)')
title('Wavelet Power Spectrum')
set(gcf,'Colormap',jet)
set(gca,'XLim',[0 1000],...
      'YLim',[10 150],...
      'XGrid','On',...
      'YGrid','On')
colorbar

```

We can also display the same plot using `pcolor` instead of `contour`:

```

figure('Position',[100 300 800 300],...
      'Color',[1 1 1]);
pcolor(t,1./f,abs(wt)), shading interp
xlabel('Time (kyr)')
ylabel('Period (kyr)')
title('Wavelet Power Spectrum')
set(gcf,'Colormap',jet)
set(gca,'XLim',[0 1000],...
      'YLim',[10 150])
colorbar

```

Alternatively, we can use the graphical output of the function `cwt`, but it is then a bit tricky to change the display of the graphics. It is, unfortunately, difficult to change the limits of the axes because they are independent of the colorful display of the wavelet transform. Browsing the code of the `cwt` function reveals that it actually uses `imagesc` to create the pseudocolor plot and then creates an independent axis system on top of that.

```
figure('Position',[100 300 800 300],...
      'Color',[1 1 1]);
cwt(series3L,years(3000),'amor')
colormap(jet)
```

The cone of influence marks the area where edge effects occur in the continuous 1D wavelet transform.

```
clear

series3 = load('series3.txt');
t = 0 : 3 : 1000;
series3L = interp1(series3(:,1),...
                  series3(:,2),t,'linear','extrap');
series3L = detrend(series3L);
[wt,f,coi] = cwt(series3L,1/3,'amor');
```

Our philosophy with this book is to reproduce algorithms for data analysis by the simplest means possible. In this instance we try to understand, and also to reproduce, the graphical output of `ctw`. According to the documentation provided, the data format of `coi` is either an array of doubles or an array of duration objects. By using frequencies as y-axes instead of periods we can simply use `line(t,coi)` to display the cone of influence by typing

```
figure('Position',[100 300 800 300],...
      'Color',[1 1 1]);
pcolor(t,f,abs(wt)), shading interp
hold on
line(t,coi,'Color','w',...
     'LineStyle','-',...
     'LineWidth',2)
xlabel('Time (kyr)')
ylabel('Frequency (1/kyr)')
title('Wavelet Power Spectrum')
set(gcf,'Colormap',jet)
set(gca,'XLim',[0 1000],...
     'YLim',[0.005 0.07],...
     'XGrid','On',...
     'YGrid','On')
colorbar
```

Compared to the windowed power spectrum method, the wavelet power spectrum clearly shows a much higher resolution on both the time and the frequency axes. Instead of dividing the time series into overlapping segments and computing the power spectrum for each segment, the wavelet transform uses short packets of waves that better map temporal changes in the cyclicities. The disadvantage of both the windowed power spectrum and the wavelet power spectrum is, however, the requirement for evenly-spaced data. The Lomb-Scargle method overcomes this problem but (as with the power spectrum method) has limitations in its ability to map temporal changes in the frequency domain.

5.9 Detecting Abrupt Transitions in Time Series

A number of methods are available to detect abrupt changes in time series in the time domain. An example of such methods suitable for use in climate time series is the *rampfit method* (Mudelsee and Stattegger 1997; Mudelsee 2000), and examples suitable for use in the frequency domain are the evolutionary *power spectrum* and the *wavelet power spectrum* (e.g. Lau and Weng 1995; Mackenzie et al. 2001). In most cases, trends and events in both time and frequency domains are detected by computing the statistical parameters of the data (e.g. measures of central tendency and dispersion) contained in a sliding window of length (or width) L . The precision of these parameters depends on the length of the window, i.e. an accurate value for the mean and the variance is obtained if L is large. However, a larger window reduces the accuracy of the estimated changes in these parameters. This problem is often referred to as *Grenander's uncertainty principle of statistics* (Grenander 1958). Performing a statistical test to assess differences in central tendency and dispersion between two different sliding windows, however, partly overcomes this problem, provided only the locations of sharp transitions in statistical parameters are required.

The classic t -test and F -test statistics are often used to compare the means and variances of two sets of measurements and could therefore be used to detect changes in the location and dispersion between two sliding windows. These two tests, however, make the basic assumption that the samples came from a population with a Gaussian distribution (Sects. 3.7 and 3.8). The non-parametric Mann-Whitney and Ansari-Bradley tests provide a solution to this problem that is independent of the distribution (Sects. 3.11 and 3.12). The Mann-Whitney test (Mann and Whitney 1947; Lepage 1971) performs a two-sided rank sum test of the null hypothesis that two samples come from continuous distributions with identical medians, against the alternative that they do not have identical medians. The Ansari-Bradley test performs a two-sided test that two independent samples come from the same distribution, against the alternative that they come from distributions that have the same medians and shapes but different dispersions (Ansari and Bradley 1960; Lepage 1971). This method has been successfully applied to records of terrigenous dust flux preserved in marine sediments offshore from subtropical West Africa, in the eastern

Mediterranean Sea, and the Arabian Sea, in order to detect trends, rhythms and events in the African Plio-Pleistocene climate (Trauth et al. 2009).

If the data are Gaussian distributed we can also use the function `findchangepts`, which was introduced in the MATLAB R2016b release to detect change points by minimizing a cost function over all possible numbers and locations of change points (Lavielle 2005; Killick et al. 2012). This function returns the largest number of significant changes in the mean, the standard deviation, and the trend of a time series, not exceeding a maximum number of permissible changes defined by the user, that minimize the sum of the residual error and an internal fixed penalty for each change. The method has been successfully applied to detect change points in lake records from the Chew Bahir in the southern Ethiopian rift (Trauth et al. 2018).

The example below demonstrates the use of these methods to detect change points on synthetic records that contain significant changes in the central tendency (mean, median, mode) and dispersion (range, variance, quantiles) in the middle of the time series (Figs. 5.18 and 5.19). The time axis runs from 0.1 to 500 kyr, at a sampling interval of 0.1 kyr. At 250 kyr the mean of the normal distributed data changes abruptly from 1.0 to 3.0, while the standard deviation remains unchanged (Fig. 5.18a).

```
clear, close all, clc

rng(10)
t = 0.1 : 0.1 : 500;
y1 = 0.1*random('norm',1,0.5,1,length(t),1);
y2 = 0.1*random('norm',3,0.5,1,length(t),1);
y = y1(1:length(t)/2);
y(length(t)/2+1:length(t)) = ...
    y2(length(t)/2+1:length(t));
```

We first use a Mann-Whitney test with paired sliding windows of three different lengths to detect any abrupt changes in the mean. We choose sliding window lengths of 300, 500, and 1,000 data points, i.e. in each step we apply the Mann-Whitney test to two samples of 150 data points, two samples of 250 data points, and two samples of 500 data points. Note that when running a Mann-Whitney test on different sets of data the length of the window needs to be adjusted to the length of the time series, as well as to the required accuracy with which the transition in the mean is to be identified.

```
w = [300 500 1000];
```

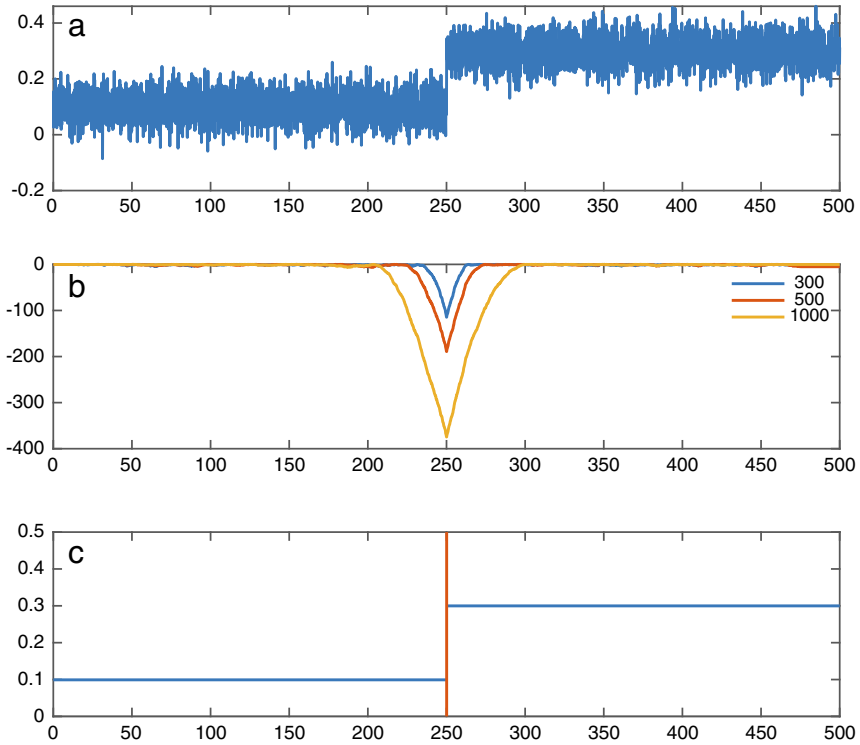


Fig. 5.18 Detecting abrupt transitions in the measures of central tendency in a synthetic dust flux record for the last 500 kyr. The data set comprises 5,000 data points, so it has a temporal resolution of 0.1 kyr. **a** Synthetic data set of Gaussian distributed noise. After 250 kyrs the mean of the data shifts towards a different value. **b** Result of a Mann-Whitney test for three different lengths of the paired sliding windows (300, 500 and 1,000 data points, equivalent to 30, 50 and 100 kyrs). The length of the window clearly influences the amplitudes and widths of the parameter maxima, whereas the location of the transition in the mean is well defined. **c** Result of change point analysis using `findchangepts`. If we allow the algorithm to find a single transition in the mean, it will be reproduced very accurately.

We use the function `ranksum` introduced in Sect. 3.11 to perform the Mann-Whitney test (Fig. 5.18a).

```
for j = 1:length(w)
    na = w(j);
    nb = w(j);
    for i = w(j)/2+1:length(y)-w(j)/2
        [p,h] = ranksum(y(i-w(j)/2:i-1),...
            y(i+1:i+w(j)/2));
        mwreal(j,i) = p;
    end
end
```

```

mwreal(j,1:w(j)/2) = ...
    mwreal(j,w(j)/2+1) * ones(1,w(j)/2);
mwreal(j,length(y)-w(j)/2+1:length(y)) = ...
    mwreal(j,length(y)-w(j)/2) * ones(1,w(j)/2);
end

```

Alternatively we can use `findchangepts`, which returns the largest number of significant changes in the mean, the standard deviation, and the trend of a time series (not exceeding a maximum number of permissible changes `maxnumchg` defined by the user) that minimize the sum of the residual error and an internal fixed penalty for each change. Here we use `findchangepts` to find a maximum number of `maxnumchg=1` change points in the mean (Fig. 5.18b).

```

maxnumchg = 1;
ipoint = findchangepts(y,...
    'Statistic','mean',...
    'MaxNumChanges',maxnumchg);
ipoint(end+1) = length(y);
m(1) = mean(y(1:ipoint(1)));
for i = 1 : length(ipoint)-1
    m(i+1) = mean(y(ipoint(i):ipoint(i+1)));
end
m(length(ipoint)+1) = mean(y(ipoint(i):end));

```

We can compare the results in a single Figure Window (Fig. 5.18):

```

axes('Position',[0.1 0.7 0.8 0.25],...
    'Box','On',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',8)
line(t,y,...
    'LineWidth',1)
axes('Position',[0.1 0.4 0.8 0.25],...
    'Box','On',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',8)
line(t,log(mwreal),...
    'LineWidth',1)
legend(num2str(w'),...
    'Box','Off')
axes('Position',[0.1 0.1 0.8 0.25],...
    'XLim',[0 500],...
    'Box','On',...

```

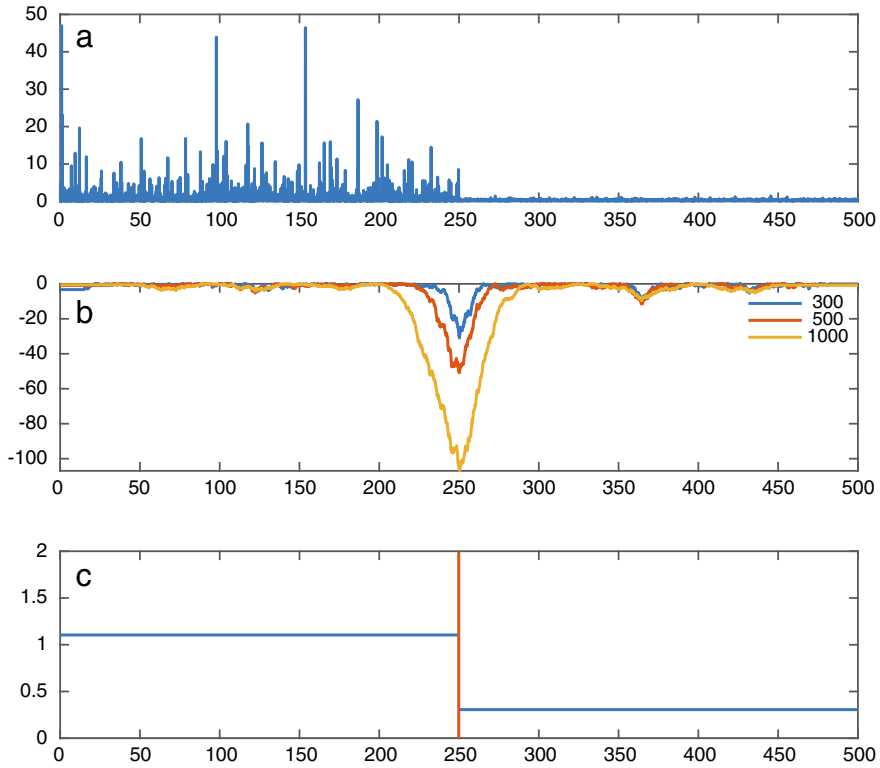


Fig. 5.19 Detecting abrupt transitions in the measures of dispersion in a synthetic dust flux record for the last 500 kyr. The data set comprises 5,000 data points, so it has a temporal resolution of 0.1 kyr. **a** Synthetic data set of Gaussian distributed noise. After 250 kyrs the standard deviation of the data shifts towards a different value. **b** Result of an Ansari-Bradley test for three different lengths of the paired sliding windows (300, 500 and 1,000 data points, equivalent to 30, 50 and 100 kyrs). The length of the window clearly influences the amplitudes and widths of the parameter maxima, whereas the location of the transition in the standard deviation is well defined. **c** Result of change point analysis using `findchangepts`. If we allow the algorithm to find a single transition in the standard deviation, it will be reproduced very accurately.

```
'LineWidth',0.6,...
'FontName','Helvetica',...
'FontSize',8); hold on
line([0 t(ipoint(1))],[m(1) m(1)],...
'LineWidth',1)
for i = 1 : length(ipoint)-1
    line([t(ipoint(i)) t(ipoint(i+1))],...
[m(i+1) m(i+1)],...
'LineWidth',1)
end
```

```

line([t(ipoint(length(ipoint))) t(end)],...
      [m(end) m(end)],...
      'LineWidth',1)
stem(t(ipoint(1:end-1)),...
      0.5*ones(size(t(ipoint),2)-1),...
      'Marker','none',...
      'Color',[0.8510 0.3255 0.0980],...
      'LineWidth',1);

```

The result from the Mann-Whitney test reveals that the length of the window influences the amplitudes and widths of the maxima of the test parameters, whereas the location of the transition in the mean is well defined (Fig. 5.18b). The results from using `findchangepts` suggest that also this method detects the change point very well if we allow the algorithm to find a single transition in the mean (Fig. 5.18c).

We next use an Ansari-Bradley test for the same three different lengths of paired sliding windows (300, 500 and 1000 data points) to detect any abrupt changes in the standard deviation. We use the function `ansaribradley` introduced in Sect. 3.12 to perform the Ansari-Bradley test. The time axis runs from 0.1 to 500 kyr at sampling intervals of 0.1 kyr. At 250 kyr the standard deviation of the log-normal distributed data changes abruptly from 0.5 to 1.3, while the mean changes from 1 to 1.5 (Fig. 5.19a).

```

clear

rng(0)
t = 0.1 : 0.1 : 500;
y1 = 0.1 * random('logn',1.5,1.3,1,length(t),1);
y2 = 0.1 * random('logn',1,0.5,1,length(t),1);
y = y1(1:length(t)/2);
y(length(t)/2+1:length(t)) = ...
    y2(length(t)/2+1:length(t));

```

We now use the Ansari-Bradley test (Fig. 5.19b):

```

w = [300 500 1000];

for j = 1:length(w)
    df1 = w(j) - 1;
    df2 = w(j) - 1;
    for i = w(j)/2+1:length(y)-w(j)/2
        [h,p] = ansaribradley(y(i-w(j)/2:i-1),...
                               y(i+1:i+w(j)/2));
        abreal(j,i) = p;
    end
    abreal(j,1:w(j)/2) = ...

```

```

    abreal(j,w(j)/2+1) * ones(1,w(j)/2);
    abreal(j,length(y)-w(j)/2+1:length(y)) = ...
    abreal(j,length(y)-w(j)/2) * ones(1,w(j)/2);
end

```

We again use `findchangepts` to detect significant changes in the standard deviation, not exceeding a maximum number of permissible changes `maxnumchg=1` defined by the user, that minimize the sum of the residual error and an internal fixed penalty for each change (Fig. 5.19c). We need to keep in mind that our data is not normally distributed but log-normally distributed: since the function `findchangepts` is actually defined for normally distributed data we cannot use the absolute values of the statistical parameters as estimators for the parameters of the distributions. However, the result of the search for change points is not affected by this.

```

maxnumchg = 1;
ipoint = findchangepts(y,...
    'Statistic','std',...
    'MaxNumChanges',maxnumchg);
ipoint(end+1) = length(y);
m(1) = mean(y(1:ipoint(1)));
for i = 1 : length(ipoint)-1
    m(i+1) = mean(y(ipoint(i):ipoint(i+1)));
end
m(length(ipoint)+1) = mean(y(ipoint(i):end));

```

We can compare the results in a single Figure Window (Fig. 5.19):

```

figure
axes('Position',[0.1 0.7 0.8 0.25],...
    'Box','On',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',8); hold on
line(t,y,...
    'LineWidth',1)
axes('Position',[0.1 0.4 0.8 0.25],...
    'Box','On',...
    'LineWidth',0.6,...
    'FontName','Helvetica',...
    'FontSize',8)
line(t,log(abreal),...
    'LineWidth',1)
legend(num2str(w'),...
    'Box','Off')
axes('Position',[0.1 0.1 0.8 0.25],...

```

```

'XLim',[0 500],...
'YLim',[0 2],...
'Box','On',...
'LineWidth',0.6,...
'FontName','Helvetica',...
'FontSize',8); hold on
line([0 t(ipoint(1))],[m(1) m(1)],...
'LineWidth',1)
for i = 1 : length(ipoint)-1
    line([t(ipoint(i)) t(ipoint(i+1))],...
[m(i+1) m(i+1)],...
'LineWidth',1)
end
line([t(ipoint(length(ipoint))) t(end)],...
[m(end) m(end)],...
'LineWidth',1)
stem(t(ipoint(1:end-1)),...
2*ones(size(t(ipoint),2)-1),...
'Marker','none',...
'Color',[0.8510 0.3255 0.0980],...
'LineWidth',1);

```

The result from the Ansari-Bradley test reveals that the length of the window influences the amplitudes and widths of the maxima of the test parameters, whereas the location of the transition in the standard deviation is well defined (Fig. 5.18b). The results from using `findchangepts` suggest that this method also detects the change point very well if we allow the algorithm to find a single transition in the standard deviation (Fig. 5.18c).

We can use the search for change points as a starting point for correcting jumps in the data. Such jumps are very common, for example when series of measurements are interrupted and then continued from the same point after a period of time, for example as a result of X-ray tubes aging. Standards are commonly used to correct such effects, but sometimes the jumps cannot be completely corrected. Jumps due to measurement errors are of course not welcome, especially if you are looking for real transitions, for example in the climate. To demonstrate the detection and removal of jumps, we generate synthetic data, a simple sine wave without noise, and add two jumps up and down.

```

clear

t = 0 : 0.1 : 10*pi;
x1 = sin(t);
x1(1,131:315) = x1(1,131:315) + 5;
x1(1,170:315) = x1(1,170:315) - 10;

```

The first way of removing jumps is to calculate the 1st derivative using `diff`, and then removing any jumps indicated by outliers using, for example, the function `filloutliers`, introduced in the MATLAB R2017a release. We then integrate the time series and add the end of the time series a single NaN that was lost while using `diff`.

```
x2 = diff(x1);
x3 = filloutliers(x2,'pchip');
x4 = cumsum(x3);
x4(end+1) = NaN;

figure('Position',[100 100 800 1000])
axes('Position',[0.1 0.8 0.8 0.15],...
     'Box','On'); hold on
plot(t,x1), grid
title('Original Data with Jumps')
axes('Position',[0.1 0.57 0.8 0.15],...
     'Box','On')
plot(t(1:end-1),x2), grid
title('1st Derivative')
axes('Position',[0.1 0.34 0.8 0.15])
plot(t(1:end-1),x3), grid
title('1st Derivative, Outliers removed')
axes('Position',[0.1 0.11 0.8 0.15],...
     'Box','On')
plot(t,x1,t,x4), grid
title('Integrated Signal')
```

This works fine for the noise-free sine wave but fails to remove jumps from a signal that is just noise, created with `x1 = randn(size(t))`. We can therefore try an alternative approach, using the function `findchangepts`.

```
clear x2 x3 x4

t = 0 : 0.1 : 10*pi;
x1 = randn(size(t));
x1(1,131:315) = x1(1,131:315) + 5;
x1(1,170:315) = x1(1,170:315) - 10;
```

We define the number of change points we expect and remove the mean (or alternatively, the median) from the segments between the change points.

```
maxnumchg = 2;
[ipoint,residual] = findchangepts(x1,...
    'MaxNumChanges',maxnumchg);
m(1) = mean(x1(1:ipoint(1)));
```



```

for i = 1 : length(ipoint)-1
    m(i+1) = mean(x1(ipoint(i):ipoint(i+1)));
end
m(length(ipoint)+1) = mean(x1(ipoint(i+1):end));

x2 = zeros(size(x1));
x2(1:ipoint(1)) = x1(1:ipoint(1)) - m(1);
for i = 1 : length(ipoint)-1
    x2(ipoint(i):ipoint(i+1)) = ...
        x1(ipoint(i):ipoint(i+1)) - m(2);
end
x2(ipoint(i+1):end) = x1(ipoint(i+1):end) - m(end);

```

We then display the results.

```

figure('Position',[100 100 800 600])
axes('Position',[0.1 0.7 0.8 0.25],...
    'Box','On'); hold on
plot(t,x1), grid, title('Original Data with Jumps')
axes('Position',[0.1 0.4 0.8 0.25],...
    'Box','On'), hold on
line(t,x1)
line([0 t(ipoint(1))],[m(1) m(1)])
for i = 1 : length(ipoint)-1
    line([t(ipoint(i)) t(ipoint(i+1))],...
        [m(i+1) m(i+1)])
end
line([t(ipoint(length(ipoint))) t(end)],...
    [m(end) m(end)])
stem(t(ipoint),6*ones(size(t(ipoint))),...
    'Marker','none',...
    'Color',[0.4667 0.6745 0.1882],...
    'BaseValue',-6)
title('Original Data with Change Points')
axes('Position',[0.1 0.1 0.8 0.25],...
    'Box','On')
plot(t,x1,t,x2)
title('Data without Jumps')

```

This approach seems to work fine for both sine waves and noise.

5.10 Aligning Stratigraphic Sequences

In geological drilling projects, two or more parallel cores with overlapping sequences are often collected in order to compensate for core losses during drilling. Correlation of the core is a time-consuming and complex process that is usually carried out by very experienced geologists. As an example, in the Chew Bahir Drilling Project in Ethiopia, which formed part of the Hominin Sites and Paleolakes Drilling Projects (HSPDP) (<http://hspdp.asu.edu>), two parallel cores, each almost 280 m long cores, were collected in late 2014 (Campisano et al. 2017). The two cores were correlated using the Correlator software, which is a free, open-source application for processing core data (<https://csdco.umn.edu/>). In this section we will first use a MATLAB script for manual correlation of core records based on visual inspection of records of the physical properties of the cored sediment. We then use dynamic time warping for automatically aligning the two records (Sakoe and Chiba 1978; Paliwal et al. 1982).

The first example is a hypothetical record of magnetic susceptibility from two 30 m long sediment cores. We first need to define the segment of the core to be correlated; this is defined by the minimum and maximum x -value (X_{Min} and X_{Max}) which could, for example, be depths below the surface in meters. We then need to decide how many tiepoints n we want to use to correlate the cores. Finally, we need to define the length (or width) w of the sliding window, for which we calculate the cross-correlation of the cores following adjustment of their depths.

```
clear, close all, clc
```

```
XMin = 0;  
XMax = 30;  
n = 5;  
w = 50;
```

We then import the sample data from the *coredata_1.txt* and *coredata_2.txt* files. These files have two columns, one for core depth and one for magnetic susceptibility.

```
data1 = load('coredata_1.txt');  
data2 = load('coredata_2.txt');
```

We can then eliminate duplicate data points by typing

```
for i = 1:size(data1)-1  
    if data1(i,1) == data1(i+1,1)  
        data1(i,1) = NaN;  
    end  
end
```

```

data1(isnan(data1(:,1))==1,:) = [];
for i = 1:size(data2)-1
    if data2(i,1) == data2(i+1,1)
        data2(i,1) = NaN;
    end
end
data2(isnan(data2(:,1))==1,:) = [];

```

We then define the segment to be correlated. This is necessary when the cores are actually longer than the interval we want to analyze.

```

data1 = data1(data1(:,1)>XMin & data1(:,1)<XMax,:);
data2 = data2(data2(:,1)>XMin & data2(:,1)<XMax,:);

```

We then display the two magnetic susceptibility records in a single Figure Window using

```

figure('Position',[150 300 1500 1100],...
    'Color','w');
axes('Position',[0.1 0.85 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Core 1');
line(data1(:,1),data1(:,2),...
    'Marker','.',...
    'MarkerSize',3,...
    'LineWidth',1,...
    'Color',[0.1 0.5 0.8])
axes('Position',[0.1 0.70 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Core 2');
line(data2(:,1),data2(:,2),...
    'Marker','.',...
    'MarkerSize',3,...
    'LineWidth',1,...
    'Color',[0.1 0.5 0.8])
axes('Position',[0.1 0.40 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Define tiepoints in core 1 and 2!');...
    'Color','r',...
    'FontSize',24);
axis off

```

We can now select our age control points, in pairs, by first clicking on a data point in the upper plot, which represents data from Core 1, and then on the lower plot, which represents data from of Core 2. When we reach the preselected number $n=5$ of control points, the Figure Window disappears.

```
for i = 1 : n
    tiepoints1(i,:) = ginput(1);
    tiepoints2(i,:) = ginput(1);
    tietext = ['Tie Point ',num2str(i),' completed.'];
    disp(tietext)
    tielabels(i,:) = sprintf('%02.0f',i);
end
tune(1,:) = [0 0];
tune(2:size(tiepoints1,1)+1,1) = tiepoints1(:,1);
tune(2:size(tiepoints1,1)+1,2) = tiepoints2(:,1);
tune = sortrows(tune);
data1t(:,1) = interp1(tune(:,1),tune(:,2),...
    data1(:,1),'linear','extrap');
data1t(:,2) = data1(:,2);
close all
```

We now tune Core 2 to Core 1 by matching the tiepoints and interpolating the whole curve linearly.

```
data1ti(:,1) = data2(:,1);
data1ti(:,2) = interp1(data1t(:,1),data1t(:,2), ...
    data2(:,1),'linear','extrap');
```

To check the quality of our fit we calculate the piecewise Pearson correlation coefficient, or one of the two alternatives by Spearman and Kendall (see Sect. 4.2) for a sliding window.

```
corrt(1,:) = length(data1ti)-w/2;
for i = w/2+1:length(data1ti)-w/2
    [p,h] = corr(data1ti(i-w/2:i+w/2,2), ...
        data2(i-w/2:i+w/2,2));
    corrt(1,i) = p;
end
corrt(1,1:w/2) = corrt(1,w/2+1) * ones(1,w/2);
corrt(1,length(data1ti)-w/2+1:length(data1ti)) = ...
corrt(1,length(data1ti)-w/2) * ones(1,w/2);
corrt = corrt';
```

Finally, we present the results in a single Figure Window. The upper two curves are the original magnetic susceptibility records for the two cores. The third and fourth curves show the first (original) record and the tuned second record. The fifth curve represents both magnetic susceptibility records in a single plot. The sixth curve represents the piecewise correlation of the first record with the tuned second record (in green). The tiepoints between the two records are marked as red stars (Fig. 5.20).

```
figure('Position',[100 100 1500 1100],...
    'Color','w');
axes('Position',[0.1 0.85 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Core 1');
line(data1(:,1),data1(:,2),...
    'Marker','.',...
    'MarkerSize',3,...
    'LineWidth',1,...
    'Color',[0.8 0.3 0.1])
axes('Position',[0.1 0.70 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Core 2');
line(data2(:,1),data2(:,2),...
    'Marker','.',...
    'MarkerSize',3,...
    'LineWidth',1,...
    'Color',[0.1 0.5 0.8])
axes('Position',[0.1 0.55 0.8 0.12],...
    'Box','On',...
    'XLim',[XMin XMax],...
    'XGrid','On');
title('Tuned core 1');
line(data1t(:,1),data1t(:,2),...
    'Marker','.',...
    'MarkerSize',3,...
    'LineWidth',1,...
    'Color',[0.8 0.3 0.1]), hold on
line(tiepoints2(:,1),tiepoints1(:,2),...
    'LineStyle','None',...
    'Marker','Pentagram',...
    'Color','r')
```

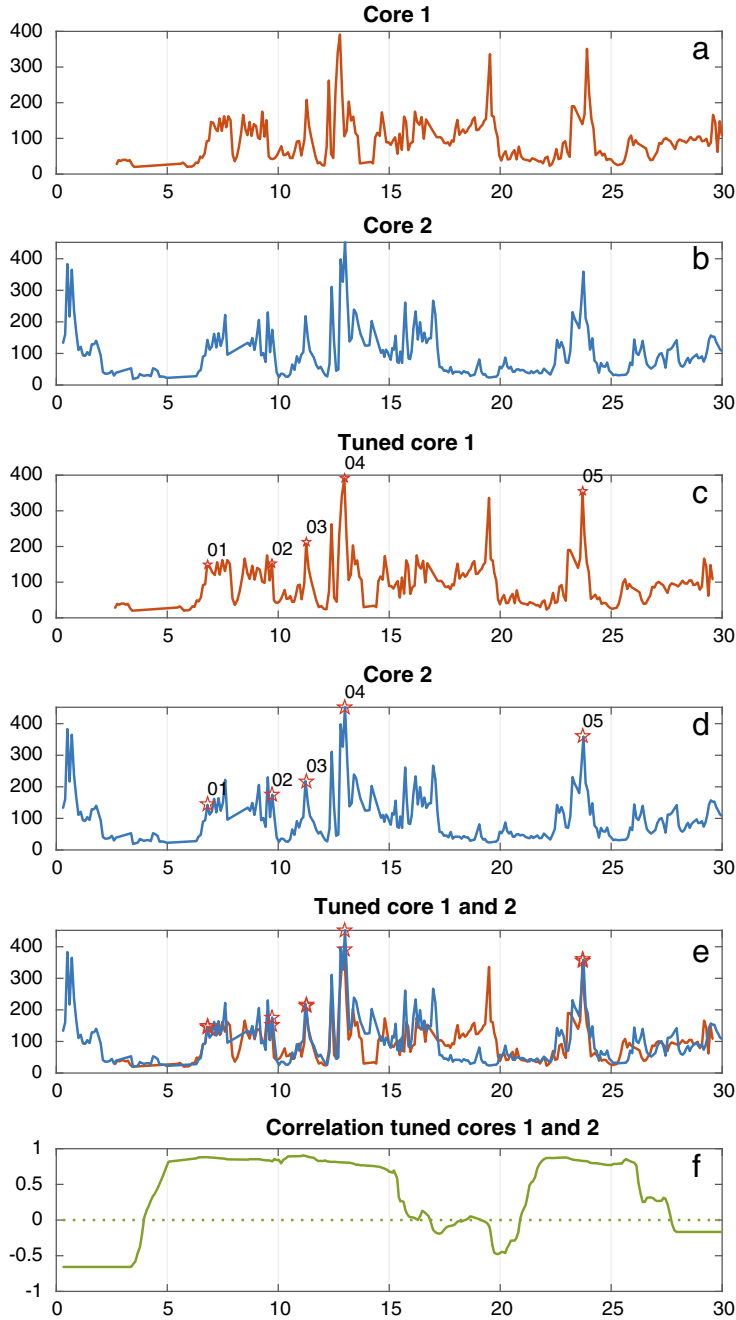


Fig. 5.20 Result of manual stratigraphic alignment of two hypothetical records of magnetic susceptibility from two 30 m long sediment cores. After matching data points in both cores the whole curve is interpolated linearly. The piecewise correlation of the aligned records helps to evaluate the result.

```

text(tiepoints2(:,1),tiepoints1(:,2)+ ...
     0.1*max(tiepoints1(:,2)),tielabels)
axes('Position',[0.1 0.40 0.8 0.12],...
     'Box','On',...
     'XLim',[XMin XMax],...
     'XGrid','On');
title('Core 2');
line(data2(:,1),data2(:,2),...
     'Marker','.',...
     'MarkerSize',3,...
     'LineWidth',1,...
     'Color',[0.1 0.5 0.8]), hold on
line(tiepoints2(:,1),tiepoints2(:,2),...
     'LineStyle','None',...
     'Marker','Pentagram',...
     'MarkerSize',10,...
     'Color','r')
text(tiepoints2(:,1),tiepoints2(:,2)+ ...
     0.1*max(tiepoints2(:,2)),tielabels)
axes('Position',[0.1 0.25 0.8 0.12],...
     'Box','On',...
     'XLim',[XMin XMax],...
     'XGrid','On');
title('Tuned core 1 and 2');
line(data1t(:,1),data1t(:,2),...
     'Marker','.',...
     'MarkerSize',3,...
     'LineWidth',1,...
     'Color',[0.8 0.3 0.1]), hold on
line(tiepoints2(:,1),tiepoints1(:,2),...
     'LineStyle','None',...
     'Marker','Pentagram',...
     'MarkerSize',10,...
     'Color','r')
line(data2(:,1),data2(:,2),...
     'Marker','.',...
     'MarkerSize',3,...
     'LineWidth',1,...
     'Color',[0.1 0.5 0.8]), hold on
line(tiepoints2(:,1),tiepoints2(:,2),...
     'LineStyle','None',...
     'Marker','Pentagram',...
     'MarkerSize',10,...
     'Color','r')

```

```

axes('Position',[0.1 0.10 0.8 0.12],...
     'Box','On',...
     'XLim',[XMin XMax],...
     'YLim',[-1 1],...
     'XGrid','On');
title('Correlation tuned cores 1 and 2');
line(data2(:,1),corrnt(:,1),...
     'Marker','.',...
     'MarkerSize',3,...
     'Color',[0.5 0.6 0.1],...
     'LineWidth',1), hold on
line(data2(:,1),zeros(length(data2(:,1)),1),...
     'Color',[0.5 0.6 0.1],...
     'LineWidth',1,...
     'LineStyle',':')

```

As we can see, the correlation is very good around the tiepoints but usually lower in between. We can improve this by increasing the number of tiepoints n . However, with a high number of tiepoints the alignment becomes arbitrary.

As an automated alternative technique for stratigraphic alignment we can use dynamic time warping, which uses similarities to match time series (Sakoe and Chiba 1978; Paliwal et al. 1982). This method is, for example, widely used in speech recognition, where speaking at different speeds hampers correlation between time series. Dynamic time warping minimizes the total Euclidean distance between two records by matching similarities in the time series (see Sect. 9.5). We first need to define the segment of the core to be correlated; this is defined by the minimum and maximum x -value ($X_{\min}$ and $X_{\max}$) which could, for example, be depth below the surface in meters. We also need to define the length (or width) w of the sliding window, for which we calculate the cross-correlation of the cores following the adjustment of their depths.

```

clear

XMin = 0;
XMax = 30;

w = 20;

```

We then import the sample data from files *coredata_1.txt* and *coredata_2.txt*.

```

data1 = load('coredata_1.txt');
data2 = load('coredata_2.txt');

```

We can then eliminate duplicate data points by typing

```

for i = 1:size(data1)-1
    if data1(i,1) == data1(i+1,1)

```



```

        data1(i,1) = NaN;
    end
end
data1(isnan(data1(:,1))==1,:) = [];
for i = 1:size(data2)-1
    if data2(i,1) == data2(i+1,1)
        data2(i,1) = NaN;
    end
end
data2(isnan(data2(:,1))==1,:) = [];

```

We next define the segment to be correlated.

```

data1 = data1(data1(:,1)>XMin & data1(:,1)<XMax,:);
data2 = data2(data2(:,1)>XMin & data2(:,1)<XMax,:);

```

We can now display the two records in a single Figure Window.

```

figure('Position',[50 950 800 300])
axes('Box','On',...
    'FontSize',14,...
    'XGrid','On',...
    'YGrid','On',...
    'YLim',[0 700]), hold on
line(data1(:,1),data1(:,2),...
    'Color',[0.8 0.3 0.1],...
    'LineWidth',1)
line(data2(:,1),data2(:,2)+200,...
    'Color',[0.1 0.5 0.8],...
    'LineWidth',1)
xlabel('Depth below surface (m)')
legend('Core 1','Core 2')
legend('boxoff')

```

We use the function `dtw` from the Signal Processing Toolbox (MathWorks 2020a) to perform dynamic time warping. Alternatively, we can use the function `samplealign` from the Bioinformatics Toolbox (MathWorks 2020c).

```

maxsamp = 100;
metric = 'euclidean';
[dist,iydata2,ixdata1] = ...
    dtw(data2(:,2),data1(:,2),maxsamp,metric);

```

We then display the aligned time series. The maximum value in `ixdata1` is the length of `data1` and the maximum value in `iydata2` is the length of `data2`. We can therefore plot the data in `data1` and `data2` over the index `ixdata1` and `ixdata2`. The

index values `ixdata1` and `iydata2` are integer values from 1 to `length(data1)` and 1 to `length(data2)`, but there are duplicate values due to the introduction of repeated values of `data1` and `data2`.

```
figure('Position',[50 550 800 300])
axes('Box','On',...
     'FontSize',14,...
     'XGrid','On',...
     'YGrid','On',...
     'YLim',[0 700]), hold on
line(iydata2,data1(ixdata1,2),...
     'Color',[0.8 0.3 0.1],...
     'LineWidth',1)
line(iydata2,data2(iydata2,2)+200,...
     'Color',[0.1 0.5 0.8],...
     'LineWidth',1)
xlabel('Index')
legend('Core 1','Core 2')
legend('boxoff')
```

We can also display the warping path, together with a straight-line fit between the two records.

```
figure('Position',[900 850 400 400])
axes('Box','On',...
     'FontSize',14,...
     'XGrid','On',...
     'YGrid','On'), hold on
line(ixdata1,iydata2,...
     'Color',[0.8 0.3 0.1],...
     'LineWidth',1)
line([ixdata1(1) ixdata1(end)],...
     [iydata2(1) iydata2(end)],...
     'Color','k',...
     'LineWidth',1,...
     'LineStyle',':')
xlabel('Index Core 1')
ylabel('Index Core 2')
legend('data1/2b')
legend('boxoff')
```

We then create new data array of aligned data values

```

clear data1i data2i
data1i(:,1) = iydata2;
data1i(:,2) = data1(ixdata1,2);
data2i(:,1) = iydata2;
data2i(:,2) = data2(iydata2,2);

```

and again display the results.

```

figure('Position',[50 150 800 300])
axes('Box','On',...
     'FontSize',14,...
     'XGrid','On',...
     'YGrid','On',...
     'YLim',[0 700]), hold on
line(data1i(:,1),data1i(:,2),...
     'Color',[0.8 0.3 0.1],...
     'LineWidth',1)
line(data2i(:,1),data2i(:,2)+200,...
     'Color',[0.1 0.5 0.8],...
     'LineWidth',1)
xlabel('Index')
legend('Core 1','Core 2')
legend('boxoff')

```

We then remove duplicate values after dynamic time warping.

```

for i = 1:length(data1i)-1
    if data1i(i,1) == data1i(i+1,1)
        data1i(i,:) = NaN;
    end
end
data1i(isnan(data1i(:,1))==1,:) = [];
for i = 1:length(data2i)-1
    if data2i(i,1) == data2i(i+1,1)
        data2i(i,:) = NaN;
    end
end
data2i(isnan(data2i(:,1))==1,:) = [];

```

We now interpolate the data. It is important to check the resolution of the new time vector; in our example we get about the same number of data points as in the raw data.

```

clear data1i_int data2i_int
data1i(:,1) = max(data1(:,1))*data1(:,1)/ ...
    length(data1(:,1));
data1i_int(:,1) = 0:mean(diff(data1(:,1))):...
    max(data1(:,1));
data1i_int(:,2) = interp1(data1(:,1),...
    data1(:,2),data1i_int(:,1),...
    'linear','extrap');
data2i(:,1) = max(data2(:,1))*data2(:,1)/ ...
    length(data2(:,1));
data2i_int(:,1) = data1i_int(:,1);
data2i_int(:,2) = interp1(data2i(:,1),...
    data2(:,2),data2i_int(:,1),...
    'linear','extrap');

```

We can now display the results by typing

```

figure('Position',[1400 950 800 300])
axes('Box','On',...
    'FontSize',14,...
    'XGrid','On',...
    'YGrid','On',...
    'YLim',[0 700]), hold on
line(data1i_int(:,1),data1i_int(:,2),...
    'Color',[0.8 0.3 0.1],...
    'LineWidth',1)
line(data2i_int(:,1),data2i_int(:,2)+200,...
    'Color',[0.1 0.5 0.8],...
    'LineWidth',1)
xlabel('Composite depth below surface (m)')
legend('Core 1','Core 2')
legend('boxoff')

```

We can then again calculate the piecewise cross correlation over a window of length w .

```

for i = w/2+1:length(data1i_int)-w/2
    [p,h] = corr(data1i_int(i-w/2:i+w/2,2),...
        data2i_int(i-w/2:i+w/2,2));
    corrt(1,i) = p;
end
corrt(1,1:w/2) = corrt(1,w/2+1) * ones(1,w/2);
corrt(1,length(data1i_int)-w/2+1:...
    length(data1i_int)) = ...
corrt(1,length(data1i_int)-w/2) * ones(1,w/2);
corrt = corrt';

```

Finally, we can display the aligned data with the piecewise cross correlation (Fig. 5.21).

```
figure('Position',[1400 150 800 500])
axes('Box','On',...
     'FontSize',14,...
     'XGrid','On',...
     'YGrid','On',...
     'YLim',[-500 700]), hold on
yyaxis left
line(data1i_int(:,1),data1i_int(:,2),...
     'Color',[0.8 0.3 0.1],...
     'LineWidth',1)
line(data2i_int(:,1),data2i_int(:,2)+200,...
     'Color',[0.1 0.5 0.8],...
     'LineWidth',1)
yyaxis right
line(data1i_int(:,1),corrt,...
     'Color',[0.5 0.6 0.1],...
     'LineWidth',1)
line(data1i_int(:,1),...
     zeros(size(data1i_int(:,1))),...
     'Color',[0.5 0.6 0.1],...
     'LineWidth',1,...
     'LineStyle',':')
set(gca,'YColor',[0.5 0.6 0.1],...
     'XLim',[0 max(data1(:,1))],...
     'YLim',[-1 5])
xlabel('Composite depth below surface (m)')
legend('Core 1','Core 2','RHO')
legend('boxoff')
```

As you can see from the code above, we introduce a +200 offset to the second curve in the y-direction in order to better display the data.

5.11 Nonlinear Time-Series Analysis (by N. Marwan)

The methods described in the previous sections detect linear relationships in data. However, natural processes on the Earth often complex and chaotic, and methods based on linear techniques may therefore yield unsatisfactory results. New techniques for nonlinear data analysis, derived from chaos theory, have become increasingly popular in recent decades. Such methods have been used to describe nonlinear behavior by, for example, defining the scaling laws and fractal dimensions of natural processes (Turcotte 1992; Kantz and Schreiber 1997). However,

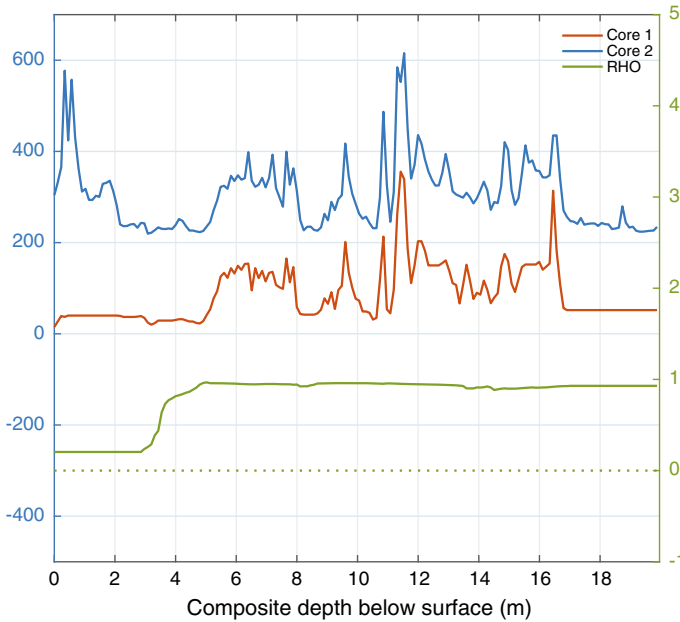


Fig. 5.21 Result of automated stratigraphic alignment of two hypothetical records using dynamic time warping with the MATLAB function `dtw`. After aligning the records the whole curve is interpolated linearly. The piecewise correlation of the aligned records helps to evaluate the result.

most methods of nonlinear data analysis require either long or stationary data series and these requirements are rarely satisfied in the earth sciences. While most nonlinear techniques work well on synthetic data, these methods are unable to describe nonlinear behavior in real data.

During recent decades *recurrence plots* have become very popular in science and engineering as a new method of nonlinear data analysis (Eckmann et al. 1987; Marwan et al. 2007). Recurrence is a fundamental property of dissipative dynamical systems. Although small disturbances in such systems can cause exponential divergence in their states, after some time the systems will return to a state that is close to a former state, before again passing through a similar evolution. Recurrence plots allow such recurrent behavior of dynamical systems to be visually portrayed. The method is now widely accepted as a useful tool for nonlinear analysis of short and nonstationary data sets.

Phase Space Portrait

The behavior of a complex dynamical system (such as the process of sedimentation in a lake) is determined by many variables, such as the properties of the water body and the sediment beneath the lake, the organisms living in the lake and its surroundings, precipitation (with high rainfall causing increased weathering and erosion in the

catchment area), evaporation (with higher evaporation resulting in, for example, higher potassium concentrations in the sediments), and wind speed (transporting more solid particles into the lake). Analysis of the properties of such a dynamical system and the time-varying interaction between its components requires a record of those state variables through time. Since there are usually no ancient records available of a system's state variables (e.g. precipitation, evaporation and wind speed), we instead have to rely on indirect indicators (proxies) that have been preserved in natural archives. A proxy record is obtained by sampling a single variable of a multi-dimensional dynamical system, which is equivalent to projecting the dynamics of a complex system onto a single axis. Considering the example of sediments from a paleolake, the sampled variable could be a time series $x(t)$ of potassium concentrations along a lake sediment core, representing a natural archive of past states of the lake system.

One way to unfold the dynamics of a multi-dimensional system from a one-dimensional time series $x(t)$ is through time-delay embedding, which preserves the dynamic characteristics of the system is preserved (Packard et al. 1980). Such a one-dimensional time series, which is based on a single variable, contains information about the system and its state variables from which a complete reconstruction of the entire system can be made. In other words, since the potassium concentration is the result of a complex interplay between unknown environmental parameters (e.g. precipitation, evaporation, and wind velocity), an analysis of the temporal variations in this environmental proxy helps us to understand the properties of the paleolake and the time-varying interactions between its components. The embedding of the time series $x(t)$ in a three-dimensional ($m = 3$) phase space, for example, means that three successive values $x(t)$, $x(t + \tau)$, and $x(t + 2\tau)$ with a temporal separation τ , are represented as a point in a three-dimensional coordinate system—the phase space (Packard et al. 1980). The phase space portrait displays the embedded time series of observations as a trajectory $\vec{y}(t) = [x(t), x(t + \tau), x(t + 2\tau)]^T$ within the phase space. The phase space trajectory represents the path over which the system's state evolves through time. The reconstructed phase space (reconstructed from embedding) will not be exactly the same as the original phase space (the true variables describing the lake), but its topological properties are preserved provided that the embedding dimension is sufficiently large (Packard et al. 1980; Takens 1981).

As an example we now explore the phase space portrait of a periodic process. We first create the position vector x_1 and the velocity vector x_2 .

```
clear, close all, clc

t = 0 : pi/10 : 3*pi;
x1 = sin(t);
x2 = cos(t);
```

The phase space portrait

```
plot(x1,x2)
xlabel('x_1')
ylabel('x_2')
```

is a circle, suggesting an exact recurrence of each state after one complete cycle (Fig. 5.22). Through time delay embedding we can reconstruct this phase space portrait using only a single observation (e.g. a velocity vector) and a time delay of five, which corresponds to a quarter of the period of a pendulum.

```
tau = 5;
plot(x2(1:end-tau),x2(1+tau:end))
xlabel('x_1')
ylabel('x_2')
```

As we can see, the reconstructed phase space is almost the same as the original phase space. We next compare this phase space portrait with one for a typical nonlinear system, the Lorenz system (Lorenz 1963). Weather patterns often do not change in a predictable manner; in 1963, Edward Lorenz introduced a simple three-dimensional model to describe the chaotic behavior exhibited by turbulence in the atmosphere. The variables defining the Lorenz system are the intensity of atmospheric convection, the temperature difference between ascending and descending currents, and the distortion of the vertical temperature profiles from linearity. Small variations in the initial conditions can cause dramatically divergent weather patterns, a behavior often referred to as the butterfly effect. The dynamics of the Lorenz system are described by three coupled nonlinear differential equations:

$$\begin{aligned}\frac{dx}{dt} &= s(y(t) - x(t)) \\ \frac{dy}{dt} &= -x(t)z(t) + rx(t) - y(t) \\ \frac{dz}{dt} &= x(t)y(t) - bz(t)\end{aligned}$$

Integrating the differential equations yields a simple MATLAB code for computing the xyz triplets of the Lorenz system. As system parameters controlling the chaotic behavior we use $s=10$, $r=28$ and $b=8/3$; the time delay is $dt=0.01$. The initial values for the position vectors are $x_1=8$, $x_2=9$ and $x_3=25$. These values, however, can be changed to any other values, which of course will change the behavior of the system.

```
clear

dt = .01;
s = 10;
r = 28;
```

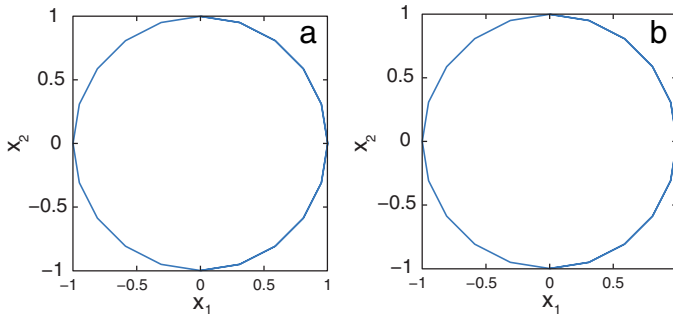



Fig. 5.22 **a** Original, and **b** reconstructed phase space portrait for a periodic system. The reconstructed phase space is almost the same as the original phase space.

```

b = 8/3;
x1 = 8; x2 = 9; x3 = 25;
for i = 1 : 5000
    x1 = x1 + (-s*x1*dt) + (s*x2*dt);
    x2 = x2 + (r*x1*dt) - (x2*dt) - (x3*x1*dt);
    x3 = x3 + (-b*x3*dt) + (x1*x2*dt);
    x(i,:) = [x1 x2 x3];
end

```

Typical traces of a variable (such as the first variable) can be viewed by plotting $x(:,1)$ over time (Fig. 5.23).

```

t = 0.01 : 0.01 : 50;
plot(t,x(:,1))
xlabel('Time')
ylabel('Temperature')

```

We next plot the phase space portrait for the Lorenz system (Fig. 5.24).

```

plot3(x(:,1),x(:,2),x(:,3))
grid, view(70,30)
xlabel('x_1')
ylabel('x_2')
zlabel('x_3')

```

In contrast to the simple periodic system described above, the trajectories of the Lorenz system obviously do not follow precisely the previous course, but are very close to it. Moreover, if we follow two very close segments of the trajectory, we see that they run into different regions of the phase space with time. The trajectory is obviously circling around one fixed point in the phase space and then, after a

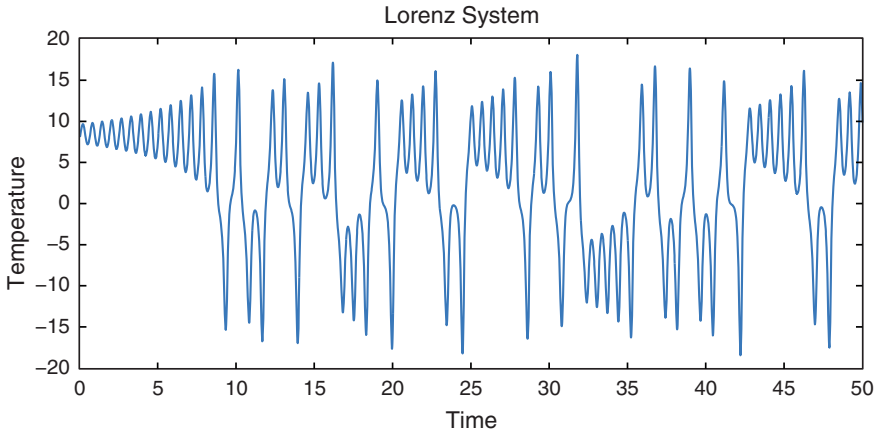


Fig. 5.23 The Lorenz system. As system parameters we used $s=10$, $r=28$ and $b=8/3$; the time delay is $dt=0.01$.

random time period, circling around another. The curious orbit of the phase states around fixed points is known as the Lorenz attractor.

These observed properties are typical of chaotic systems. While small disturbances in such a system cause exponential divergences in its state, the system returns approximately to a previous state through a similar course. The reconstruction of the phase space portrait using only the first state and a time delay of six

```
tau = 6;
plot3(x(1:end-2*tau,1),x(1+tau:end-tau,1),x(1+2*tau:end,1))
grid, view([100 60])
xlabel('x_1'), ylabel('x_2'), zlabel('x_3')
```

reveals a similar phase portrait with the two typical *ears* (Fig. 5.24). The characteristic properties of chaotic systems can also be observed in this reconstruction.

The time delay and embedding dimension need to be chosen from a previous analysis of the data. The delay can be estimated with the help of the autocovariance or autocorrelation sequence. For our example of a periodic oscillation,

```
clear
t = 0 : pi/10 : 3*pi;
x = sin(t);
```

we compute and plot the autocorrelation sequence

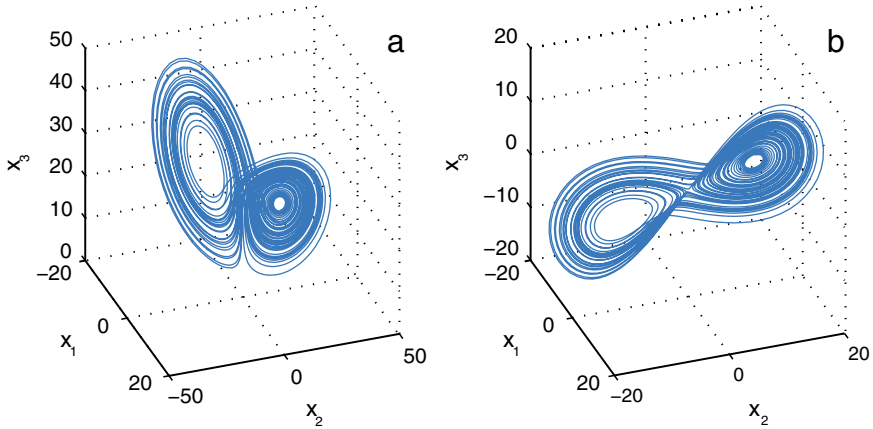


Fig. 5.24 **a** Phase space portrait for the Lorenz system. In contrast to the simple periodic system, the trajectories of the Lorenz system obviously do not follow precisely the previous course, but are very close to it. **b** The reconstruction of the phase space portrait using only the first state and a time delay of 6 reveals a topologically similar phase portrait to **a**, with the two typical ears.

```

for i = 1 : length(x) - 2
    r = corrcoef(x(1:end-i),x(1+i:end));
    C(i) = r(1,2);
end

plot(C)
xlabel('Delay'), ylabel('Autocorrelation')
grid on

```

We now choose a delay such that the autocorrelation sequence for the first time period equals zero. In our case this is five, which is the value that we have already used in our example of phase space reconstruction. The appropriate embedding dimension can be estimated using the false nearest neighbors method, or more simply, using recurrence plots, which are introduced in the next subsection. The embedding dimension is gradually increased until the majority of the diagonal lines are parallel to the line of identity.

The phase space trajectory or its reconstruction is the basis of several measures defined in nonlinear data analysis, such as *Lyapunov exponents*, *Rényi entropies*, or *dimensions*. The book on nonlinear data analysis by Kantz and Schreiber (1997) is recommended for more detailed information on these methods. Phase space trajectories or their reconstructions are also necessary for constructing recurrence plots (Fig. 5.25).

Recurrence Plots

A common feature of dynamical systems is the property of recurrence. Patterns of recurring states of a system reflect typical system characteristics whose descriptions contribute significantly to understanding of its dynamics. In our example it could be a recurrence of changes in the state variables precipitation, evaporation, and wind speed, that leads to similar (but not identical) conditions in the lake (e.g. depth and size of the lake, alkalinity and salinity of the lake water, species assemblage in the water body, diagenesis in the sediment). A recurrence plot (RP) is a graphical display of such recurring states of the system, calculated from the distance (e.g. Euclidean) between all pairs of phase space vectors $\vec{y}(t)$, within a threshold value ε (Marwan et al. 2007).

$$R_{i,j} = \begin{cases} 0, & \|\vec{y}_i - \vec{y}_j\| > \varepsilon \\ 1, & \|\vec{y}_i - \vec{y}_j\| \leq \varepsilon \end{cases}$$

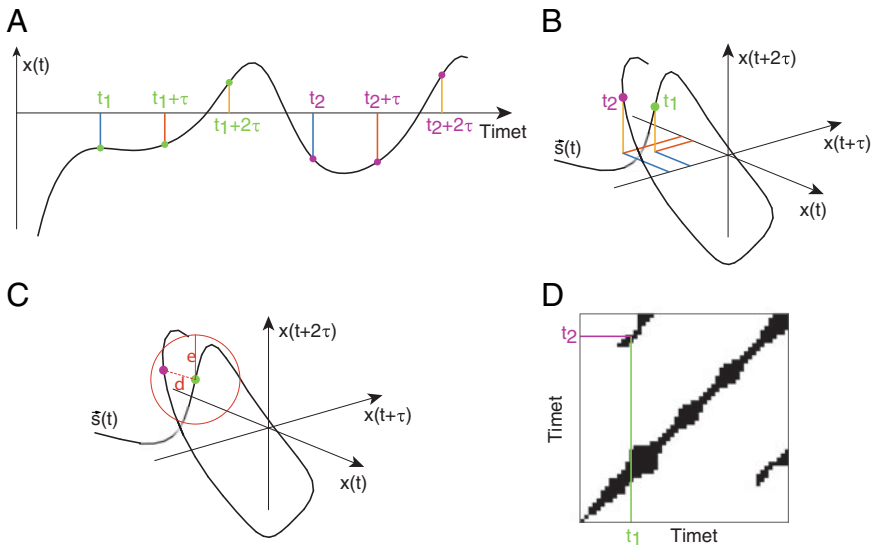


Fig. 5.25 Principle of recurrence plots (modified from Trauth et al. 2019). The dynamics of a multi-dimensional system from a one-dimensional time series $x(t)$ (black line, in panel A) is untangled by time-delay embedding. The embedding of the time series $x(t)$ in a three-dimensional ($m = 3$) coordinate system (a phase space, shown in panel B), for example, means that three successive values $x(t)$, $x(t + \tau)$, and $x(t + 2\tau)$ with a temporal separation of τ are represented as a single point (t) within the phase space (Iwanski and Bradley 1998; Webber and Zbilut 2005; Marwan et al. 2007). Recurrence plots (RPs, panel D), first introduced by Eckmann et al. (1987), are graphic displays of such recurring states within a system, calculated from the distance (e.g. the Euclidean distance d , shown in panel C) between all pairs of phase space vectors (t_1) and (t_2), below a threshold value ε (also shown in panel C) (Marwan et al. 2007).

If the distance between two states at times i and j is smaller than a given threshold ε , the value of the recurrence matrix R is one; otherwise it is zero. This analysis is therefore a pairwise test of all states. For N states we compute N^2 tests. The recurrence plot is then the two-dimensional display of the N -by- N matrix, where black pixels represent $R_{i,j}=1$ and white pixels indicate $R_{i,j}=0$, with a coordinate system representing two time axes. Such recurrence plots can help to find a preliminary characterization of the dynamics of a system or to find transitions and interrelationships within a system (cf. Fig. 5.26).

As a first example we load the synthetic time series containing 100 kyr, 40 kyr and 20 kyr cycles already used in the previous sections. Since the data are unevenly spaced, we need to linearly interpolate the data to an evenly-spaced time axis.

```
clear

series1 = load('series1.txt');
t = 0 : 3 : 996;
series1L = interp1(series1(:,1),series1(:,2),t,'linear');
```

We start with the assumption that the phase space is only one-dimensional. Calculating the distances between all points of the phase space trajectory using the function `pdist` produces the distance matrix S . Using `pdist` produces a list of pairwise distances of length equal to $N(N-1)/2$. The function `squareform` converts this list into the desired symmetric distance matrix.

```
N = length(series1L);

D = pdist(series1L(:));
S = squareform(D);
```

We can now plot the distance matrix for the data set

```
imagesc(t,t,S)
colormap(parula(64))
colorbar
xlabel('Time'), ylabel('Time')
axis xy
```

where a `colorbar` provides a quantitative measure of the distances between states (Fig. 5.27). We now apply a threshold ε to the distance matrix to generate the black/white recurrence plot (Fig. 5.28).

```
imagesc(t,t,S<1)
colormap([1 1 1;0 0 0])
xlabel('Time'), ylabel('Time')
axis xy
```

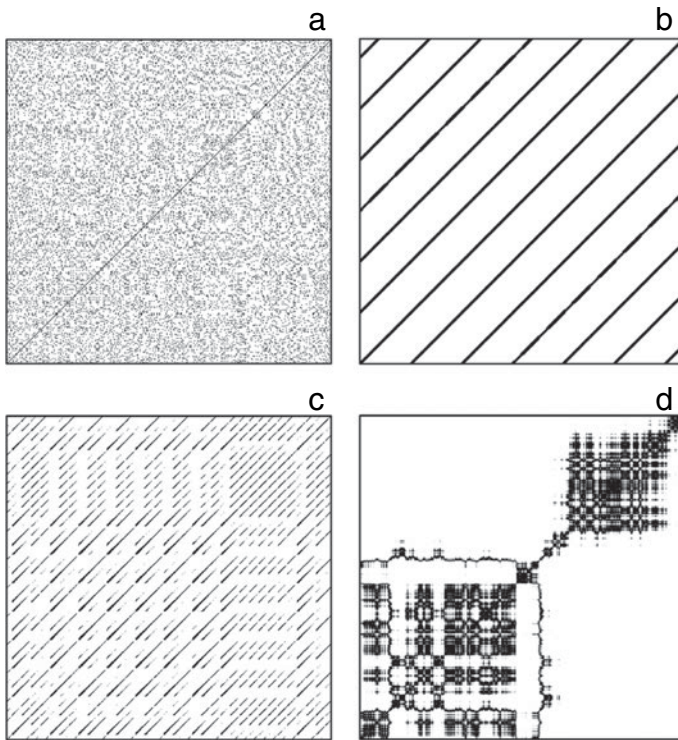


Fig. 5.26 Recurrence plots representing typical dynamical behaviors: **a** stationary uncorrelated data (white noise), **b** periodic oscillation, **c** chaotic data (Roessler system), and **d** non-stationary data with abrupt changes.

Both plots reveal periodically recurring patterns. The distances between these periodically recurring patterns represent the cycles contained in the time series. The most significant periodic patterns have periods of 200 and 100 kyrs. The 200 kyr period is the most significant because of the superposition of the 100 and 40 kyr cycles, which are common divisors of 200 kyrs. Moreover, there are smaller sub-structures within the recurrence plot that have periods of 40 and 20 kyrs.

As a second example we now apply the method of recurrence plots to the Lorenz system. We again generate xyz triplets from the coupled differential equations.

```
clear

dt = .01;
s = 10;
r = 28;
b = 8/3;
x1 = 21; x2 = 9; x3 = 25;
```

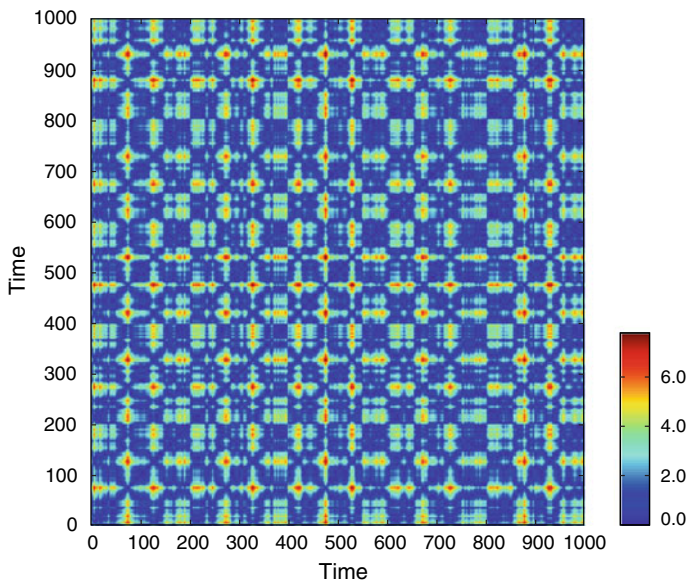


Fig. 5.27 Display of the distance matrix from the synthetic data, providing a quantitative measure for the distances between states at particular times; blue colors indicate small distances and red colors represent large distances.

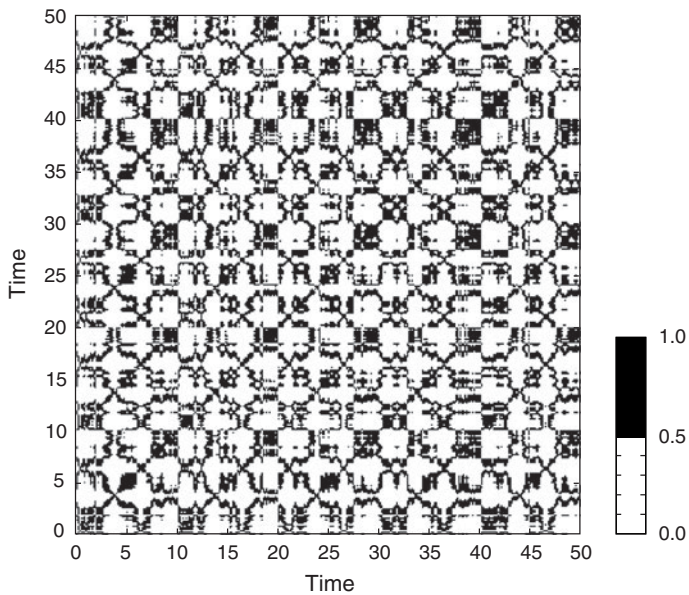


Fig. 5.28 Recurrence plot for synthetic data derived from the distance matrix, as shown in Fig. 5.23, after applying a threshold of $\varepsilon = 1$.

```

for i = 1 : 5000
    x1 = x1 + (-s*x1*dt) + (s*x2*dt);
    x2 = x2 + (r*x1*dt) - (x2*dt) - (x3*x1*dt);
    x3 = x3 + (-b*x3*dt) + (x1*x2*dt);
    x(i,:) = [x1 x2 x3];
end

```

We then choose the resampled first component of this system and reconstruct a phase space trajectory using an embedding of $m=3$ and $\tau=2$.

```

t = 0.01 : 0.05 : 40;
s = x(1:5:4000,1);
m = 3; tau = 2;

N = length(y);
N2 = N - tau*(m - 1);

```

The original data series had a length of 4,000 data points, which was reduced to 800 data points (equivalent to 40 s), but because of the time delay method the reconstructed phase space trajectory has an actual length of 796 data points. We can create the phase space trajectory with

```

for mi = 1:m
    y(:,mi) = s([1:N2] + tau*(mi-1));
end

```

and the distance matrix using `pdist` and `squareform`:

```

D = pdist(y);
S = squareform(D);

imagesc(t(1:N2),t(1:N2),S<10)
colormap([1 1 1;0 0 0])
xlabel('Time'), ylabel('Time')
axis xy

```

This recurrence plot reveals many short diagonal lines (Fig. 5.29). These lines represent periods of time during which the phase space trajectory runs parallel to earlier or later sequences in this trajectory, i.e. periods of time during which the states and dynamics were similar. The distances between these diagonal lines represent the periods of the cycles, which vary and are not constant, in contrast to those for a harmonic oscillation (Fig. 5.24).

Recurrence Quantification

The structure of recurrence plots can also be described by a suite of quantitative measures in a *recurrence quantification analysis* (RQA). Several measures are based on the distribution of the lengths of diagonal or vertical lines, as well as on the local proximity configuration. These measures can be used to trace hidden transitions within a dynamical process. As an example we will consider two measures: the recurrence rate and the transitivity coefficient. The recurrence rate is the density of points in the recurrence plot and corresponds to the recurrence probability of the system. The transitivity coefficient has its roots in graph theory and characterizes the regularity (or complexity) of the system.

For our example, we consider a time series of normally-distributed (Gaussian) noise, with a transition in the mean at time 1,000 and a change of the autocorrelation prior to this transition (starting at time 800). Importantly, neither the mean nor the variances change in the precursor region, so this change cannot be found using conventional methods.

```
rng(20);
N = 1000;
x = [zeros(N,1);ones(N,1)];
x = x + .3*randn(length(x),1);
for i = (N-round(N/5)):N
    x(i) = .9 * x(i-1) + .5 * x(i) + .02;
```

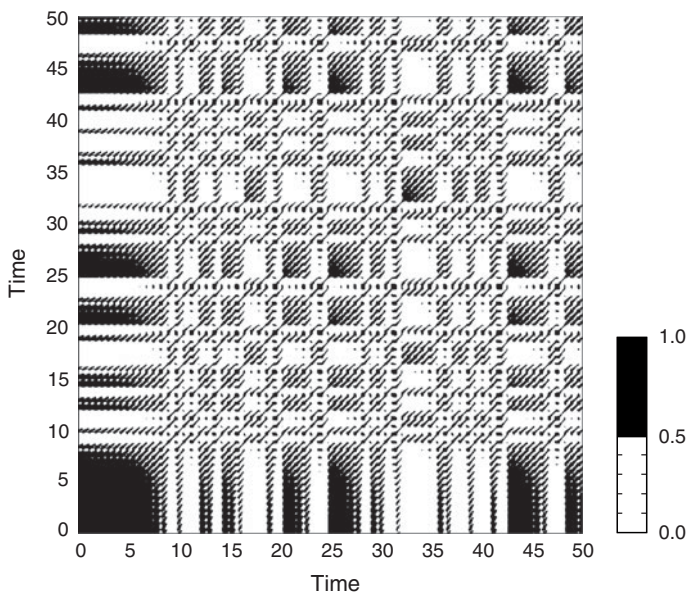


Fig. 5.29 Recurrence plot of the Lorenz system using a threshold of $\varepsilon = 2$. The regions with regular diagonal lines reveal unstable periodic orbits, typical of chaotic systems.

```

end
x = x/std(x);

t = 1:length(x);

plot(t, x)
xlabel('Time')

```

We first embed the time series and then calculate the recurrence plot:

```

N = length(x);
tau = 2; m=2;
N2 = N - tau*(m - 1);

s = zeros(N2,m);
for mi = 1:m
    s(:,mi) = x([1:N2] + tau*(mi-1));
end

D = pdist(s);
S = squareform(D);
R = S<.5;

imagesc(t(1:N2),t(1:N2),R)
colormap([1 1 1;0 0 0])
xlabel('Time'), ylabel('Time')
axis square xy

```

The RP shows two blocks of randomly distributed points, each of these points marking times when the system randomly returns to a similar state. In random noise, similar states frequently recur, but without any regularity, except for the main diagonal line, the line of identity. The two blocks of the RP mark the transition at time 1,000. The states before this change point differ strongly from the states after the change point, represented by the almost empty lower right and upper left quadrants of the RP. However, the preceding increase of autocorrelation between time 800 and 1,000 is not clearly visible in the RP, although a slightly denser appearance could be guessed. In this situation quantification of the structures in the RP can be of benefit.

To calculate the recurrence rate we can simply compute the mean of the matrix R

```
RR = mean(R(:))
```

which yields

```
RR =
0.1039
```

The probability that the system returns to a randomly selected previous state is therefore about 10%.

The transitivity coefficient is a graph-theoretical measure of the probability that three connected network nodes (triples) are completely interconnected, i.e. that they form a closed triangle:

$$\tau = \frac{\sum_{i,j,k}^N A_{ij}A_{ij}A_{ik}}{\sum_{i,j,k=1}^N A_{ij}A_{ik}(1 - \delta_{ik})}$$

This measure can be intuitively understood with respect to recurrences in the phase space. We identify a recurrence of states by a network link: close points on the phase space trajectory are connected by a link. Three connected points form a triple, but only if all three points recur close to each other, thus forming a triangle. Such a triangular configuration will remain along the phase space trajectory if the dynamic is very regular (recurring states remain recurring over a long period of time). However, if the dynamic is chaotic, then parts of the phase space trajectory that were initially close will subsequently diverge and the triangular configuration will break down, although the corresponding triple nodes might remain interconnected for some time. The probability of finding triangles is therefore higher for regular dynamics and lower for chaotic dynamics. This explanation is, of course, rather simplified but a theoretically substantiated explanation can be found in Donner et al. (2011).

In order to calculate the probability that triples also form triangles we need to compute the number of connected triples and the number of triangles, which can be achieved directly from the recurrence plot, excluding the main diagonal.

```
A = R - eye(size(R));
```

The number of triangles and triples is then

```
numTripl = sum(sum(A * A));
numTria = trace(A * A * A);
```

and finally, the transitivity coefficient is the fraction

```
Trans = numTria/numTripl
```

which yields

```
Trans =
0.6481
```

This number means that the system does not have regular dynamics (which would yield a transitivity coefficient close to one).

In order to detect changes in the dynamics, the RQA can be computed for moving windows. The RQA of moving windows can be calculated in two ways, either (1) by calculating the RQA measures for windows moving along a single, global RP or (2) by calculating individual RPs of windows moving along the original time series. If nonstationarities or trends are not in the main focus of the analysis, approach (2) helps to find transitions that ignore these nonstationarities. However, if detection of overall (global, long-scale) changes is of interest (e.g. to test for nonstationarity), the recurrence conditions should be kept constant over time (thus considering the global RP of the whole time series) and approach (1) should be selected. (Marwan 2011). The window size selected needs to be small enough to ensure a good temporal resolution but large enough to cover typical variations (e.g. several cycles) in order to find recurrences.

In our example we choose a moving window length of 150 data points and an overlap of 20% (Fig. 5.30).

```
w = 150;
```

We then calculate the recurrence rate (*RR*) and transitivity coefficient (*Trans*) within these moving windows (Figs. 5.31 and 5.32).

```
w = 150;
Trans = zeros(length(R)-w,1);
RR = zeros(length(R)-w,1);
for i = 1:w/5:length(R)-w
    subR = R(i:i+w,i:i+w);
    RR(i) = mean(subR(:));
    subA = A(i:i+w,i:i+w);
    numTripl = sum(sum(subA * subA));
    numClosTria = trace(subA * subA * subA);
    Trans(i) = numClosTria/numTripl;
end

plot(t(round(w/2) + (1:w/5:length(RR))), RR(1:w/5:end),...
     t(round(w/2) + (1:w/5:length(RR))), Trans(1:w/5:end))
xlabel('Time')
legend('Recurrence Rate','Transitivity Coeff')
```

The RQA measures show a clear transition within the higher autocorrelation section. At this point, they have higher values, reflecting the higher density of black dots, changing neighborhood configuration due to the autocorrelation, and a change in the dynamics of the system. This fact can be used to detect such a change in autocorrelation, e.g. as a precursor to a tipping point, and to ultimately predict the tipping point itself. Directly at the change point that occurs at time 1,000, the two empty quadrants interfere with the moving window, leading to a sudden decrease in the *RR* measure.

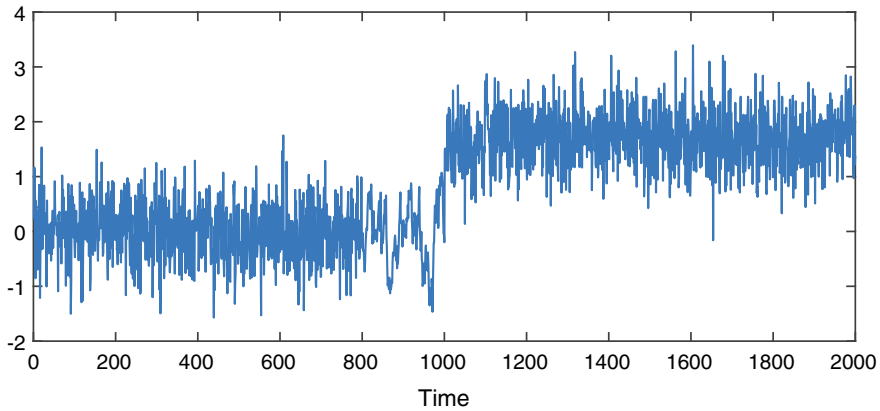


Fig. 5.30 Time series of the synthetic data used in the example of quantitative measures of recurrence plots.

For a reliable interpretation of the variations in the recurrence measures, a statistical test should be applied (Marwan 2011). Other more complex measures that quantify other aspects of the dynamics (e.g. predictability, or laminar phases) are included in the Cross Recurrence Plot Toolbox for MATLAB, available from

<http://tocsy.pik-potsdam.de/CRPtoolbox/>

Bivariate and multivariate extensions of recurrence plots allow nonlinear correlation tests and synchronization analyses to be carried out. A detailed introduction to methods based on recurrence plots can be found on the following web site:

<http://www.recurrence-plot.tk>

Recurrence plots have already been used to analyze many problems in earth sciences. The comparison of the dynamics of modern precipitation data with paleo-rainfall data inferred from annual-layered lake sediments in the northwestern Argentine Andes provides a good example of such analyses (Marwan et al. 2003). In this instance the recurrence plot method was applied to red-color intensity transects across varved lake sediments that were approximately 30 kyrs old (Sect. 8.7). Comparing the recurrence plots from the sediments with those from modern precipitation data revealed that the reddish layers document the more intense rainy seasons that occurred during La Niña years. Linear techniques were, however, unable to link the increased flux of reddish clays with either the El Niño or La Niña phase of the El Niño/Southern Oscillation. Moreover, recurrence plots helped to prove the hypothesis that longer rainy seasons, increased precipitation,

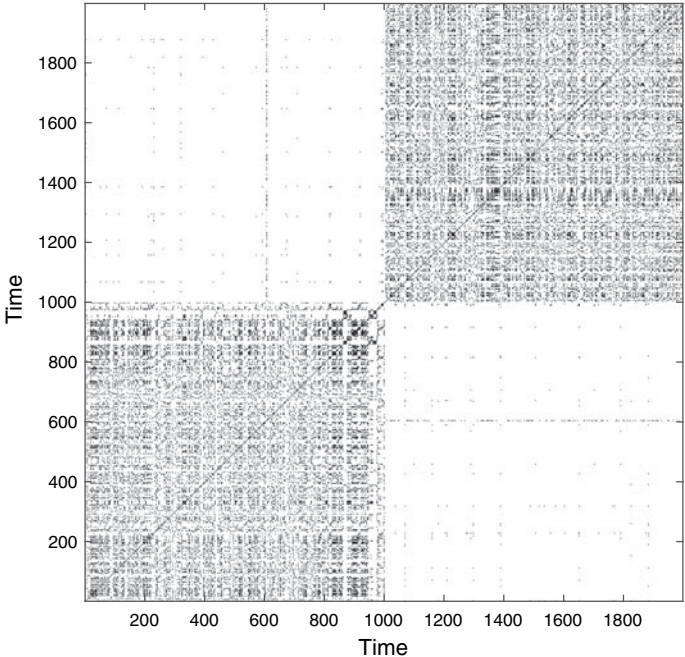


Fig. 5.31 Recurrence plot of the synthetic data shown in Fig. 5.30, using an embedding of $m = 5$ and $\tau = 3$ and applying a threshold of $\varepsilon = 1.2$.

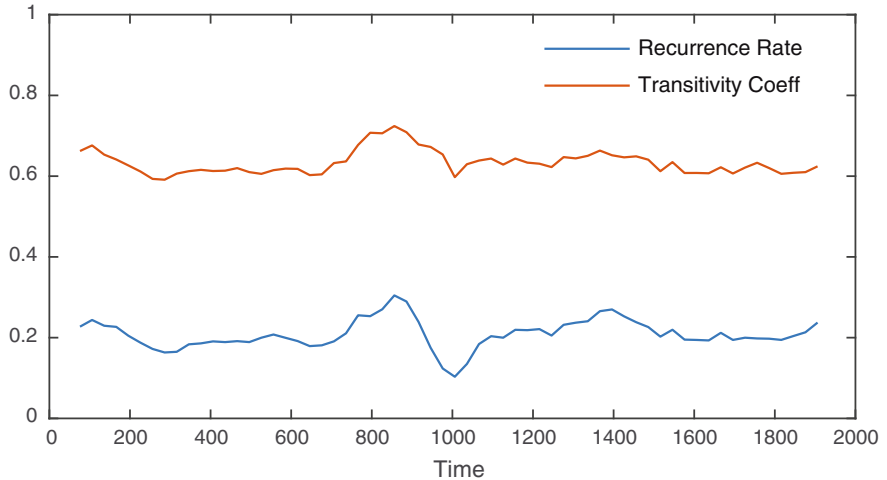


Fig. 5.32 Recurrence rate and transitivity coefficient for the synthetic data in Fig. 5.26, using a moving window of 150 data points and an overlap of 20%.

and the stronger influence of the El Niño/Southern Oscillation caused an increase in the number of landslides 30 kyrs ago (Marwan et al. 2003; Trauth et al. 2003). A second earth science example, including a comprehensive description of recurrence plots and recurrence quantification analysis, uses the technique to classify different types of variability and transitions in records from lake sediment cores to achieve an improved understanding of the response of the biosphere to climate change, and especially the response of humans in the southern Ethiopian rift (Trauth et al. 2019).

Recommended Reading

- Ansari AR, Bradley RA (1960) Rank-sum tests for dispersion. *Ann Math Stat* 31:1174–1189. [\[Open access\]](#)
- Blackman RB, Tukey JW (1958) *The measurement of power spectra*. Dover, New York
- Campisano CJ et al (2017) The Hominin Sites and Paleolakes Drilling Project: high-resolution paleoclimate records from the East African rift system and their implications for understanding the environmental context of hominin evolution. *PaleoAnthropology* 2017:1–43
- Cooley JW, Tukey JW (1965) An algorithm for the machine calculation of complex fourier series. *Math Comput* 19(90):297–301
- Donner RV, Heitzig J, Donges JF, Zou Y, Marwan N, Kurths J (2011) The geometry of chaotic dynamics—a complex network perspective. *Eur Phys J B* 84:653–672
- Eckmann JP, Kamphorst SO, Ruelle D (1987) Recurrence plots of dynamical systems. *Eur Lett* 5:973–977
- Grenander U (1958) Bandwidth and variance in estimation of the spectrum. *J Roy Stat Soc B* 20:152–157
- Holschneider M (1995) *Wavelets, an analysis tool*. Oxford University Press, Oxford
- Kantz H, Schreiber T (1997) *Nonlinear time series analysis*. Cambridge University Press, Cambridge
- Killick R, Fearnhead P, Eckley IA (2012) Optimal detection of changepoints with a linear computational cost. *J Am Stat Assoc* 107:1590–1598
- Lau KM, Weng H (1995) Climate signal detection using wavelet transform: how to make a time series sing. *Bull Am Meteor Soc* 76:2391–2402
- Lavielle M (2005) Using penalized contrasts for the change-point problem. *Sig Process* 85:1501–1510
- Lepage Y (1971) A combination of Wilcoxon's and Ansari-Bradley's statistics. *Biometrika* 58:213–271
- Lomb NR (1972) Least-squared frequency analysis of unequally spaced data. *Astro-Phys Space Sci* 39:447–462
- Lorenz EN (1963) Deterministic nonperiodic flow. *J Atmos Sci* 20:130–141
- Mackenzie D, Daubechies I, Kleppner D, Mallat S, Meyer Y, Ruskai MB, Weiss G (2001) Wavelets: seeing the forest and the trees. *Beyond Discovery*, National Academy of Sciences, December 2001. <http://www.beyonddiscovery.org>
- Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *Ann Math Stat* 18:50–60
- Marwan N, Thiel M, Nowaczyk NR (2002) Cross recurrence plot based synchronization of time series. *Nonlinear Process Geophys* 9(3/4):325–331
- Marwan N, Trauth MH, Vuille M, Kurths J (2003) Nonlinear time-series analysis on present-day and pleistocene precipitation data from the NW argentine andes. *Clim Dyn* 21:317–332

- Marwan N, Romano MC, Thiel M, Kurths J (2007) Recurrence plots for the analysis of complex systems. *Phys Rep* 438:237–329
- Marwan N (2011) How to avoid potential pitfalls in recurrence plot based data analysis. *Int J Bifurc Chaos* 21:1003–1017
- MathWorks (2020a) Signal processing toolbox—user’s guide. The MathWorks, Inc., Natick, MA
- MathWorks (2020b) Wavelet toolbox—user’s guide. The MathWorks, Inc., Natick, MA
- MathWorks (2020c) Bioinformatics toolbox—user’s guide. The MathWorks, Inc., Natick, MA
- Mudelsee M, Stattegger M (1997) Exploring the structure of the mid-Pleistocene revolution with advanced methods of time-series analysis. *Int J Earth Sci* 86:499–511
- Mudelsee M (2000) Ramp function regression: A tool for quantifying climate transitions. *Comput Geosci* 26:293–307
- Muller RA, MacDonald GJ (2000) Ice ages and astronomical causes—data spectral analysis and mechanisms. Springer, Berlin, Heidelberg, New York
- Packard NH, Crutchfield JP, Farmer D, Shaw RS (1980) Geometry from a time series. *Phys Rev Lett* 45:712
- Paliwal KK, Agarwal A, Sinha SS (1982) A modification over Sakoe and Chiba’s dynamic time warping algorithm for isolated word recognition. *Sig Process* 4:329–333
- Press WH, Teukolsky SA, Vetterling WT (2007) Numerical recipes: the art of scientific computing, 3rd edn. Cambridge University Press, Cambridge
- Romano M, Thiel M, Kurths J, von Bloh W (2004) Multivariate recurrence plots. *Phys Lett A* 330 (3–4):214–223
- Sakoe H, Chiba S (1978) Dynamic programming algorithm optimization for spoken word recognition. *IEEE Trans Acoust Speech Signal Process ASSP* 26:43–49
- Scargle JD (1981) Studies in astronomical time series analysis. I. Modeling random processes in the time domain. *Astrophys J Suppl Ser* 45:1–71
- Scargle JD (1982) Studies in astronomical time series analysis. II. Statistical aspects of spectral analysis of unevenly spaced data. *Astrophys J* 263:835–853
- Scargle JD (1989) Studies in astronomical time series analysis. III. Fourier transforms, autocorrelation functions, and cross-correlation functions of unevenly spaced data. *Astrophys J* 343:874–887
- Scargle JD (1990) Studies in astronomical time series analysis. IV. Modeling chaotic and random processes with linear filters. *Astrophys J* 359:469–482
- Schulz M, Stattegger K (1998) SPECTRUM: spectral analysis of unevenly spaced paleoclimatic time series. *Comput Geosci* 23:929–945
- Schuster A (1898) On the investigation of hidden periodicities with application to a supposed 26 day period of meteorological phenomena. *Terr Magnetism Atmos Electr* 3:13–41
- Takens F (1981) Detecting strange attractors in turbulence. *Lect Notes Math* 898:366–381
- Torrence C, Compo GP (1998) A practical guide to wavelet analysis. *Bull Am Meteor Soc* 79:61–78
- Trulla LL, Giuliani A, Zbilut JP, Webber CL Jr (1996) Recurrence quantification analysis of the logistic equation with transients. *Phys Lett A* 223(4):255–260
- Turcotte DL (1992) Fractals and chaos in geology and geophysics. Cambridge University Press, Cambridge
- Trauth MH, Bookhagen B, Marwan N, Strecker MR (2003) Multiple landslide clusters record quaternary climate changes in the NW Argentine Andes. *Palaeogeogr Palaeoclimatol Palaeoecol* 194:109–121
- Trauth MH, Larrasoana JC, Mudelsee M (2009) Trends, rhythms and events in Plio-Pleistocene African climate. *Quatern Sci Rev* 28:399–411
- Trauth MH (2015) MATLAB recipes for Earth sciences, 4th edn. Springer, Berlin, Heidelberg, New York

- Trauth MH, Foerster V, Junginger A, Asrat A, Lamb H, Schaebitz F (2018) Abrupt or gradual? Change point analysis of the late Pleistocene-Holocene climate record from Chew Bahir, Southern Ethiopia. *Quatern Res* 90:321–330
- Trauth MH, Asrat A, Duesing W, Foerster V, Kraemer KH, Marwan N, Maslin MA, Schaebitz F (2019) Classifying past climate change in the Chew Bahir basin, southern Ethiopia, using recurrence quantification analysis. *Clim Dyn* 53:2557–2572. code `ClimDyn_RP_RQA_vs3.zip` or <https://tinyurl.com/yyqjyoq4>
- Weedon G (2003) Time-series analysis and cyclostratigraphy—examining stratigraphic records of environmental change. Cambridge University Press, Cambridge
- Welch PD (1967) The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms. *IEEE Trans. Audio Electroacoustics* AU-15:70–73

6.1 Introduction

Signal processing involves techniques for manipulating a signal in order to minimize the effects of noise, to correct all kinds of unwanted distortions, and to separate out various components of interest. Most signal processing algorithms include the design and realization of filters. A *filter* can be described as a system that transforms signals. *System theory* provides the mathematical background for filter design and realization. A filter has an input and an output, with the *output signal* $y(t)$ being modified with respect to the *input signal* $x(t)$ (Fig. 6.1). The *signal transformation* can be carried out through a mathematical process known as *convolution* or, if filters are involved, as *filtering*.

This chapter deals with the design and realization of *digital filters* with the help of a computer. Many natural processes, however, resemble *analog filters* that act over a range of spatial dimensions. A single rainfall event is not recorded in lake sediments because short and low-amplitude events are smeared over a longer time span. Bioturbation also introduces serious distortions, for instance in deep-sea sediment records. In addition to such natural filters, the field collection and sampling of geological data alters and smoothes the data with respect to its original form. For example a finite sized sediment sample is integrated over a certain period of time and therefore smoothes the natural signal. Similarly, the measurement of magnetic susceptibility in a sediment core with the help of a loop sensor introduces significant smoothing since the loop response is integrated over a section of the core.

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_6) contains supplementary material, which is available to authorized users.

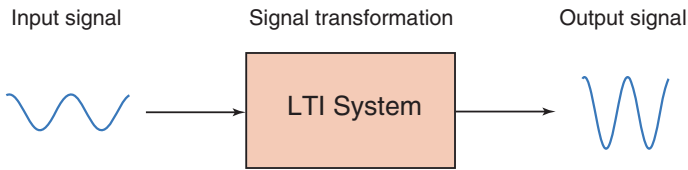


Fig. 6.1 Schematic of a linear time-invariant (LTI) system. The input signal is transformed into an output signal.

The characteristics of these natural filters are often difficult to determine, whereas numerical filters are designed with well-defined characteristics. In addition, artificial filters are time invariant in most cases, whereas natural filters, such as mixing within the water body of a lake or bioturbation at the water-sediment interface, may vary with time. An easy way to describe or predict the effect of a filter is to explore the filter output from a simple input signal as a sine wave, a square wave, a sawtooth function, a ramp function, or a step function. Although there is an endless variety of such input signals, most systems or filters are described by their impulse response, i.e. the output resulting from the input of a unit impulse.

This chapter starts with a technical section (Sect. 6.2) on generating periodic signals, trends, and noise, following on from Sect. 5.2 of the previous chapter. Section 6.3 then considers linear time-invariant systems, providing the mathematical background for filters. The succeeding Sects. 6.4–6.9 deal with the design, the realization, and the application of linear time-invariant filters. Section 6.10 then considers the use of adaptive filters originally developed for use in the telecommunication industry. Adaptive filters automatically extract noise-free signals from duplicate measurements on the same object. Such filters can be used in a large number of applications, for example to remove noise from duplicate paleoceanographic time series, or to improve the signal-to-noise ratio of parallel color-intensity transects across varved lake sediments (see Chap. 5, Fig. 5.1). Adaptive filters are also widely used in geophysics for noise canceling. We use the Signal Processing Toolbox (MathWorks 2020), which contains all the necessary routines for time-series analysis.

6.2 Generating Signals

MATLAB provides numerous tools for generating basic signals that can be used to illustrate the effects of filters. In Chap. 5 we generated a signal by adding together three sine waves with different amplitudes and periods. In the following example the time vector is transposed in order to generate column vectors.

```
clear, close all, clc

t = (1:100)';
x = 2*sin(2*pi*t/50) + sin(2*pi*t/10) + 0.5*sin(2*pi*t/5);

plot(t,x), axis([0 100 -4 4])
```

Frequency-selective filters are very common in earth sciences. They are used to remove specific frequency bands from the data. As an example we can design a filter to suppress that portion of the signal that has a periodicity of $\tau = 10$, leaving the other two cycles unaffected. The effects of such filters on simple periodic signals can also be used to predict signal distortions of natural filters.

A *step function* is another basic input signal that can be used to explore filter characteristics. It describes the transition from a value of one towards a value of zero at a specific time. The function `stairs` draws a stairstep graph of the elements of `x`.

```
t = (1:100)';
x = [ones(50,1);zeros(50,1)];

stairs(t,x), axis([0 100 -2 2])
```

This signal can be used to study the effects of a filter on a sudden transition. An abrupt climate change could be regarded as an example. Most natural filters tend to smooth such a transition and smear it over a longer time period.

A *unit impulse* is a third important signal type that we will use in the following examples. This signal equals zero at all times except at a single data point, where it equals one. The function `stem` plots the data sequence `x` as stems from the t -axis, using circles for the data values.

```
t = (1:100)';
x = [zeros(49,1);1;zeros(50,1)];

stem(t,x), axis([0 100 -4 4])
```

The unit impulse is the most popular synthetic signal used to study the performance of a filter. The output of the filter, i.e. the impulse response, describes the characteristics of a filter very well. Moreover, the output of a linear time-invariant filter can be described by the superposition of impulse responses that have been scaled by multiplying the output of the filter by the amplitude of the input signal.

6.3 Linear Time-Invariant Systems

Filters can be described as systems with an input $x(t)$ and output $y(t)$. We will therefore first describe the characteristics of systems in general before then considering filters. Important characteristics of a system are

- *Continuity*—A system with continuous inputs $x(t)$ and outputs $y(t)$ is a continuous system. Most natural systems are continuous. However, after sampling natural signals we obtain discrete data series and model these natural systems as discrete systems, with discrete inputs and outputs.
- *Linearity*—For linear systems, the output $y(t)$ of the linear combination of several input signals $x_i(t)$, where

$$x(t) = k_1x_1(t) + k_2x_2(t)$$

is the same as the linear combination of the outputs $y_i(t)$:

$$y(t) = k_1y_1(t) + k_2y_2(t)$$

Important properties of linearity are scaling and additivity (*superposition*), which allow the input and output to be multiplied by a constant k_i , either before or after transformation. Superposition allows additive components of the input to be extracted and transformed separately. Fortunately, many natural systems follow a linear pattern of behavior. Complex linear signals such as additive harmonic components can be separated out and transformed independently. Milankovitch cycles provide an example of linear superposition in paleoclimate records, although there is an ongoing debate about the validity of this theory. Numerous nonlinear systems, which do not possess the properties of scaling and additivity, also exist in nature. An example of a linear system is

```
clear, close all, clc
```

```
x = (1:100)';
```

```
y = 2*x;
```

```
plot(x,y)
```

where x is the input signal and y is the output signal. An example of a nonlinear system is

```
x = (-100:100)';
```

```
y = x.^2;
```

```
plot(x,y)
```

- *Time invariance*—The system output $y(t)$ does not change as a result of a delay in the input $x(t+i)$: the system characteristics are constant with time. Unfortunately, natural systems often change their characteristics with time. For instance, benthic mixing or bioturbation depends on various environmental parameters such as nutrient supply, and the system's properties consequently vary significantly with time. In such a case it is difficult to determine the actual input of the

system from the output, e.g. to extract the actual climate signal from a bioturbated sedimentary record.

- *Invertibility*—An invertible system is a system in which the original input signal $x(t)$ can be reproduced from the system's output $y(t)$. This is an important property if unwanted signal distortions are to be corrected, in which case the known system is inverted and the output then used to reconstruct the undisturbed input. For example, a core logger measuring magnetic susceptibility with a loop sensor integrates the signal over a specific core interval, with the sensitivity highest at the position of the loop and decreasing down-core and up-core. This system is invertible, i.e. we can compute the input signal $x(t)$ from the output signal $y(t)$ by inverting the system. The inverse of the above linear system is

```
x = (1:100)';
y = 0.5*x;

plot(x,y)
```

where x is the input signal and y is the output signal. A nonlinear system

```
x = (-100:100)';
y = x.^2;

plot(x,y)
```

is not invertible. Since this system yields equal responses for different inputs, such as $y = 4$ for inputs $x = -2$ and $x = +2$, the input x cannot be reconstructed from the output y . A similar situation can also occur in linear systems, such as

```
x = (1:100)';
y = zeros(size(x));

plot(x,y)
```

The output y is zero for all inputs x , and the output therefore does not contain any information about the input.

- *Causality*—The system response only depends on present and past inputs $x(0)$, $x(-1)$, ..., whereas future inputs $x(+1)$, $x(+2)$, ... have no effect on the output $y(0)$. All real-time systems, such as telecommunication systems, must be causal since they cannot have future inputs available to them. All systems and filters in MATLAB are indexed as causal. In earth sciences, however, numerous non-causal filters are used. The filtering of images and signals extracted from sediment cores are examples where the future inputs are available at the time of filtering. Output signals have to be delayed after filtering in order to compensate for the differences between causal and non-causal indexing.

- *Stability*—A system is stable if the output $y(t)$ of a finite input $x(t)$ is also finite. Stability is critical in filter design, where filters often have the disadvantage of provoking divergent outputs. In such cases, the filter design has to be revised and improved.

Linear time-invariant (LTI) systems are very popular as a special type of filter. Such systems have all the advantages that have been described above, as well as being easy to design and use in many applications. The following Sects. 6.4–6.9 describe the design, realization and application of LTI-type filters to extract specific frequency components from signals. These filters are mainly used to reduce the noise level in signals. Unfortunately, however, many natural systems do not behave as LTI systems in that the signal-to-noise ratio often varies with time. Section 6.10 describes the application of adaptive filters that automatically adjust their characteristics in a time-variable environment.

6.4 Convolution, Deconvolution and Filtering

Convolution is a mathematical description of a system transformation. Filtering is an application of the convolution process. A running mean of length five provides an example of such a simple filter. The output of an arbitrary input signal is

$$y(t) = \frac{1}{5} \sum_{k=-2}^2 x(t-k)$$

The output $y(t)$ is simply the average of the five input values $x(t-2)$, $x(t-1)$, $x(t)$, $x(t+1)$ and $x(t+2)$. In other words, all the five consecutive input values are multiplied by a factor of $1/5$ and summed to form $y(t)$. In this example all input values are multiplied by the same factor, i.e. they are equally weighted. The five factors used in the above operation are therefore called filter weights b_k . The filter can be represented by the vector

```
clear, close all, clc
```

```
b = [0.2 0.2 0.2 0.2 0.2]
```

consisting of the five identical filter weights. Since this filter is symmetric, it does not shift the signal on the time axis: the only function of this filter is to smooth the signal. Running means of a given length are often used to smooth signals, mainly for cosmetic reasons. Modern spreadsheet software usually contains running means as a function for smoothing data series. The effectiveness of a smoothing filter increases with the filter length.

The weights that a filter of arbitrary length uses can be varied. As an example let us consider an asymmetric filter of five weights.

```
b = [0.05 0.08 0.14 0.26 0.47]
```

The sum of all of the filter weights is one and therefore it does not introduce any additional variance into the signal. However, since it is highly asymmetric, it shifts the signal along the time axis, i.e. it introduces a phase shift.

The general mathematical representation of the filtering process is the *convolution*:

$$y(t) = \sum_{k=-N_1}^{N_2} b_k \cdot x(t-k)$$

where b_k is the vector of *filter weights*, and N_1+N_2 is the *order of the filter*, which is the length of the filter reduced by one. Filters with five weights, as in our example, have an order of four. In contrast to this format, MATLAB uses the engineering standard for indexing filters, i.e. filters are always defined as causal. The convolution used by MATLAB is therefore

$$y(t) = \sum_{k=0}^N b_k \cdot x(t-k)$$

where N is the order of the filter. A number of the frequency-domain tools provided by MATLAB cannot simply be applied to non-causal filters that have been designed for applications in earth sciences. Phase corrections are therefore commonly carried out in order to simulate non-causality. For example frequency-selective filters, as will be introduced in Sect. 6.9, can be applied using the function `filtfilt`, which provides zero-phase forward and reverse filtering.

The functions `conv` and `filter` that provide digital filtering in MATLAB are best illustrated in terms of a simple running mean. The n elements of the vector $x(t_1)$, $x(t_2)$, $x(t_3)$, ..., $x(t_n)$ are replaced by the arithmetic means of subsets of the input vector. For instance, a running mean over three elements computes the mean of inputs $x(t_{n-1})$, $x(t_n)$, $x(t_{n+1})$ to obtain the output $y(t_n)$. We can illustrate this simply by generating a random signal

```
clear
t = (1:100)';
rng(0)
x1 = randn(100,1);
```

designing a filter that averages three data points of the input signal

```
b1 = [1 1 1]/3;
```

and convolving the input vector with the filter


```
y1 = conv(x1,b1);
```

The elements of $b1$ are the weights of the filter. In our example all filter weights are the same and equal to $1/3$. Note that the `conv` function yields a vector that has a length of $n+m-1$, where m is the length of the filter.

```
m1 = length(b1);
```

We can explore the contents of our workspace to check the length of the input and output of `conv`. Typing

```
whos
```

yields

Name	Size	Bytes	Class	Attributes
b1	1x3	24	double	
m1	1x1	8	double	
t	100x1	800	double	
x1	100x1	800	double	
y1	102x1	816	double	

Here, we see that the actual input series $x1$ has a length of 100 data points, whereas the output $y1$ has two additional elements. Convolution generally introduces $(m-1)/2$ data points at each end of the data series. To compare input and output signals, we therefore clip the output signal at either end.

```
y1 = y1(2:101,1);
```

A more general way to correct the phase shifts of `conv` is

```
y1 = y1(1+(m1-1)/2:end-(m1-1)/2,1);
```

which of course only works for an odd number of filter weights. An alternative is to use `same` for the `shape` parameter in `conv(a, b, shape)` in order to return the most central 100 data points of the convolution that have the same size as the input signal $x1$.

```
y1 = conv(x1,b1, 'same');
```

We can then plot both input and output signals for comparison, using `legend` to display a legend for the plot.

```
plot(t,x1,'b-',t,y1,'r-')
legend('x1(t)', 'y1(t)')
```

This plot illustrates the effect that the running mean has on the original input series. The output y_1 is significantly smoother than the input signal x_1 . If we increase the length of the filter we obtain an even smoother signal output y_2 .

```
b2 = [1 1 1 1 1]/5;
m2 = length(b2);

y2 = conv(x1,b2,'same');

plot(t,x1,'b-',t,y1,'r-',t,y2,'g-')
legend('x1(t)', 'y1(t)', 'y2(t)')
```

The reverse of convolution is deconvolution, which is often used in signal processing to reverse the effects of a filter. We use the first example of a random signal x_1 , design a filter that averages three data points of the input signal, and convolve the input vector with the filter, which yields the output y_1 .

```
clear

t = (1:100)';
rng(0)
x1 = randn(100,1);

b1 = [1 1 1]/3;
y1 = conv(x1,b1);
```

We use `deconv` to reverse the convolution and compare the deconvolution result x_{1d} with the original signal x_1 .

```
x1d = deconv(y1,b1);

plot(t,x1,'b:',t,x1d,'r')
```

As we can see, there is no difference. There is a difference, however, if we add noise to the signal and deconvolve the result by typing

```
y1n = y1 + 0.05*randn(size(y1));
x1nd = deconv(y1n,b1);

plot(t,x1,'b:',t,x1nd,'r')
```

The next section provides a broader definition of filters.

6.5 Comparing Functions for Filtering Data Series

The filters described in the previous section were very simple examples of *non-recursive filters*, in which the filter output $y(t)$ depends only on the filter input $x(t)$ and the filter weights b_k . Prior to introducing a broader description of linear time-invariant filters, we replace the function `conv` by `filter`, which can also be used for *recursive filters*. In this case, the output $y(t_n)$ depends not only on the filter input $x(t)$, but also on previous elements of the output $y(t_{n-1})$, $y(t_{n-2})$, $y(t_{n-3})$ and so on (Sect. 6.6). We will first use `conv` for nonrecursive filters in order to compare the results of `conv` and `filter`.

```
clear, close all, clc
```

```
t = (1:100)';
rng(0)
x3 = randn(100,1);
```

We design a filter that averages five data points of the input signal.

```
b3 = [1 1 1 1 1]/5;
m3 = length(b3);
```

The input signal can be convolved using the function `conv`.

```
y3 = conv(x3,b3,'same');
```

We next follow a similar procedure with the function `filter` and compare the result with that obtained using the function `conv`. In contrast to the function `conv` without using `same`, the function `filter` yields an output vector with the same length as the input vector. Unfortunately, the function `filter` assumes that the filter is causal. The filter weights are indexed n , $n-1$, $n-2$ and so on, and therefore no future elements of the input vector, such as $x(n+1)$, $x(n+2)$ etc. are needed to compute the output $y(n)$. This is of great importance in electrical engineering, the classic field of MATLAB application, where filters are often applied in real time. In earth sciences, however, the entire signal is, in most applications, available at the time of processing the data. The data series is filtered by

```
y4 = filter(b3,1,x3);
```

and the phase correction is then carried out using

```
y4 = y4(1+(m3-1)/2:end-(m3-1)/2,1);
y4(end+1:end+m3-1,1) = zeros(m3-1,1);
```

which works only for an odd number of filter weights. This command simply shifts the output by $(m-1)/3$ towards the lower end of the t -axis, and then fills the data to the end with zeros. Comparing the ends of both outputs illustrates the effect of this correction, where

```
y3(1:5,1)
y4(1:5,1)
```

yields

```
ans =
    0.0225
    0.1950
    0.2587
   -0.1103
   -0.5638

ans =
    0.0225
    0.1950
    0.2587
   -0.1103
   -0.5638
```

This was the lower end of the output. We can see that both vectors y_3 and y_4 contain the same elements. We now explore the upper end of the data vector, where

```
y3(end-5:end,1)
y4(end-5:end,1)
```

yields the output

```
ans =
    0.5491
    0.0181
   -0.5868
   -0.8124
   -0.8498
   -0.8333
```

```
ans =
    0.5491
    0.0181
         0
         0
         0
         0
```

The vectors are identical up to element $y(\text{end}-m3+1)$, but then the second vector $y4$ contains zeros instead of true data values. Plotting the results with

```
subplot(2,1,1), plot(t,x3,'b-',t,y3,'r-')
subplot(2,1,2), plot(t,x3,'b-',t,y4,'r-')
```

or in a single plot,

```
plot(t,x3,'b-',t,y3,'g-',t,y4,'r-')
```

shows that the results from using `conv` are identical with those from using `filter`, except at the upper end of the data vector. These observations are important for our next steps in signal processing, particularly if we are interested in leads and lags between various components of signals.

6.6 Recursive and Nonrecursive Filters

We now expand the nonrecursive filters by a recursive component, such that the output $y(t_n)$ depends not only on the filter input $x(t)$, but also on previous output values $y(t_{n-1})$, $y(t_{n-2})$, $y(t_{n-3})$, and so on. This filter requires not only the nonrecursive filter weights b_i , but also the recursive filters weights a_i (Fig. 6.2), and can be described by the *difference equation*:

$$y(t) = \sum_{k=-N_1}^{N_2} b_k \cdot x(t-k) - \sum_{k=1}^M a_k \cdot y(t-k)$$

Although this is a non-causal version of the difference equation, MATLAB again uses the causal indexing,

$$y(t) = \sum_{k=0}^N b_k \cdot x(t-k) - \sum_{k=1}^M a_k \cdot y(t-k)$$

with the known problems in the design of zero-phase filters. The larger of the two quantities M , and N_1+N_2 or N , is the order of the filter.

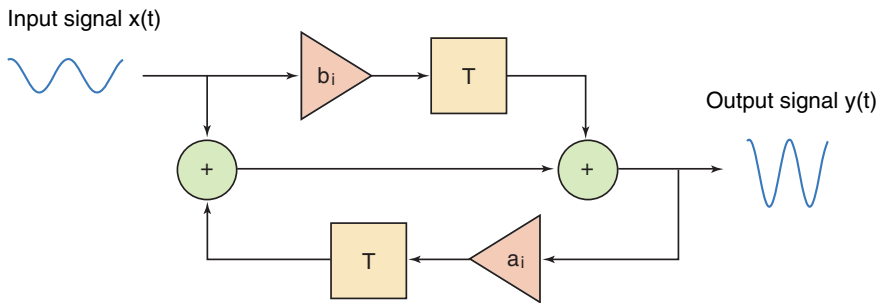


Fig. 6.2 Schematic of a linear time-invariant filter with an input $x(t)$ and an output $y(t)$. The filter is characterized by its weights a_i and b_i , and the delay elements T . Nonrecursive filters require only nonrecursive weights b_i whereas recursive filters also require the recursive filter weights a_i .

We use the same synthetic input signal x_5 as in the previous example to illustrate the performance of a recursive filter.

```
clear, close all, clc
t = (1:100)';
rng(0)
x5 = randn(100,1);
```

This input is then filtered using a recursive filter with a set of weights a_5 and b_5 ,

```
b5 = [0.0048  0.0193  0.0289  0.0193  0.0048];
a5 = [1.0000 -2.3695  2.3140 -1.0547  0.1874];
m5 = length(b5);
y5 = filter(b5,a5,x5);
```

and the output y_5 corrected for the phase

```
y5 = y5(1+(m5-1)/2:end-(m5-1)/2,1);
y5(end+1:end+m5-1,1) = zeros(m5-1,1);
```

We can now plot the results.

```
plot(t,x5,'b- ',t,y5,'r-')
```

This filter clearly changes the signal dramatically. The output contains only low-frequency components and all higher frequencies have been eliminated. A comparison of the periodograms for the input and the output reveals that all frequencies above $f = 0.1$ (corresponding to a period of $\tau = 10$) have been suppressed.

```
[Pxx,f] = periodogram(x5,[],128,1);
[Pyx,f] = periodogram(y5,[],128,1);

plot(f,Pxx,f,Pyx)
```

We have now designed a frequency-selective filter, i.e. a filter that eliminates certain frequencies while leaving other frequencies relatively unaffected. The next section introduces tools that are used to characterize a filter in the time and frequency domains and to predict the effect of a frequency-selective filter on arbitrary signals.

6.7 Impulse Response

The impulse response is a very convenient way of describing the characteristics of a filter (Fig. 6.3). The impulse response h is useful in LTI systems where the convolution of the input signal $x(t)$ with h is used to obtain the output signal $y(t)$.

$$y(t) = \sum_{k=-N_1}^{N_2} h_k \cdot x(t-k)$$

It can be shown that the values of the impulse response h are identical to the filter weights in nonrecursive filters, but not in recursive filters. The above convolution equation is often written in a short form:

$$y(t) = h(t) * x(t)$$

In many examples convolution in the time domain is replaced by a simple multiplication of the Fourier transforms $H(f)$ and $X(f)$ in the frequency domain.

$$Y(f) = H(f) \cdot X(f)$$

The output signal $y(t)$ in the time domain is then obtained by a reverse Fourier transform of $Y(f)$. Signals are often convolved in the frequency domain rather than the time domain because of the relative simplicity of the multiplication. However, the Fourier transformation itself introduces a number of artifacts and distortions, and convolution in the frequency domain is therefore not without problems. In the following examples we apply the convolution only in the time domain.

First, we generate a unit impulse:

```
clear, close all, clc

t = (0:20)';
x6 = [zeros(10,1);1;zeros(10,1)];

stem(t,x6), axis([0 20 -4 4])
```

The function `stem` plots the data sequence `x6` as stems from the x -axis, terminated with circles for the data value. This can be a better way to plot digital data than using the continuous lines generated by `plot`. We now feed this into the filter and explore the output `y6`. The impulse response is identical to the weights of nonrecursive filters.

```
b6 = [1 1 1 1 1]/5;
m6 = length(b6);

y6 = filter(b6,1,x6);
```

We again correct this for the phase shift of the function `filter`, although this might not be important in this example.

```
y6 = y6(1+(m6-1)/2:end-(m6-1)/2,1);
y6(end+1:end+m6-1,1) = zeros(m6-1,1);
```

We obtain an output vector `y6` of the same length and phase as the input vector `x6`. We now plot the results for comparison.

```
stem(t,x6)
hold on
stem(t,y6,'filled','r')
axis([0 20 -2 2])
hold off
```

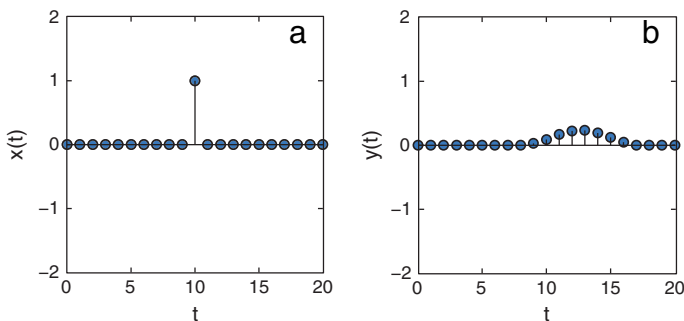


Fig. 6.3 Transformation of **a** a unit impulse to compute **b** the impulse response of a system. The impulse response is often used to describe and predict the performance of a filter.

In contrast to `plot`, the function `stem` accepts only one data series and the second series `y6` is therefore overlaid on the same plot using the function `hold`. The effect of the filter is clearly seen on the plot: it averages the unit impulse over a length of five elements. Furthermore, the values of the output `y6` equal the filter weights of `a6`; in our example these values are 0.2 for all elements of `a6` and `y6`.

For a recursive filter, however, the output `y6` does not match the filter weights. Once again, we first generate an impulse:

```
clear
t = (0:20)';
x7 = [zeros(10,1);1;zeros(10,1)];
```

An arbitrary recursive filter with weights of `a7` and `b7` is then designed.

```
b7 = [0.0048  0.0193  0.0289  0.0193  0.0048];
a7 = [1.0000 -2.3695  2.3140 -1.0547  0.1874];
m7 = length(b7);
y7 = filter(b7,a7,x7);
y7 = y7(1+(m7-1)/2:end-(m7-1)/2,1);
y7(end+1:end+m7-1,1) = zeros(m7-1,1);
```

The stem plot of the input `x2` and the output `y2` shows an interesting impulse response:

```
stem(t,x7)
hold on
stem(t,y7,'filled','r')
axis([0 20 -2 2])
hold off
```

The signal is smeared over a broader area, and is also shifted towards the right. This filter therefore not only affects the amplitude of the signal, but also shifts the signal towards lower or higher values. Such phase shifts are usually unwanted characteristics of filters, although in some applications shifts along the time axis might be of particular interest.

6.8 Frequency Response

We next investigate the frequency response of a filter, i.e. the effect of a filter on the amplitude and phase of a signal (Fig. 6.4). The frequency response $H(f)$ of a filter is the Fourier transform of the impulse response $h(t)$. The absolute value of the complex frequency response $H(f)$ is the magnitude response of the filter $A(f)$.

$$A(f) = |H(f)|$$

The argument of the complex frequency response $H(f)$ is the phase response of the filter.

$$\Phi(f) = \arg(H(f))$$

Since MATLAB filters are all causal it is difficult to explore the phase of signals using the corresponding functions included in the Signal Processing Toolbox. The user's guide for this toolbox simply recommends that the filter output be delayed in the time domain by a fixed number of samples, as we have done in the previous examples. As another example a sine wave with a period of 20 and an amplitude of 2 is used as an input signal.

```
clear, close all, clc

t = (1:100)';
x8 = 2*sin(2*pi*t/20);
```

A running mean over eleven elements is designed and this filter applied to the input signal `x8`.

```
b8 = ones(1,11)/11;
m8 = length(b8);
y8 = filter(b8,1,x8);
```

The phase of the output `y8` is corrected for causal indexing.

```
y8 = y8(1+(m8-1)/2:end-(m8-1)/2,1);
y8(end+1:end+m8-1,1) = zeros(m8-1,1);
```

Both input and output of the filter are plotted.

```
plot(t,x8,t,y8)
```

The filter clearly reduces the amplitude of the sine wave. Whereas the input signal `x8` has an amplitude of 2, the output `y8` has an amplitude of

```
max(y8)

ans =

    1.1480
```

The filter reduces the amplitude of a sine with a period of 20 by

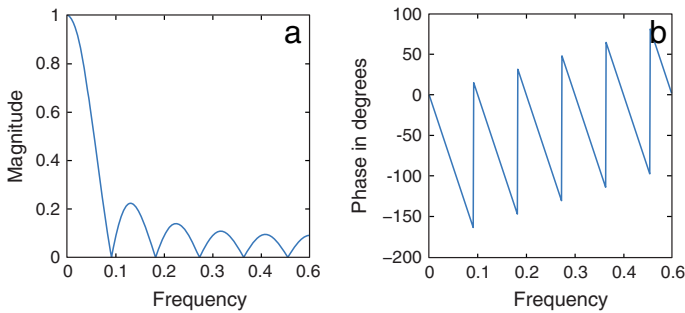


Fig. 6.4 **a** Magnitude and **b** phase response of a running mean over eleven elements.

```
1-max(y8(40:60))/2
```

```
ans =
```

```
0.4260
```

i.e. by approximately 43%. The elements 40 to 60 are used for computing the maximum value of $y8$, in order to avoid edge effects. Nevertheless, the filter does not affect the phase of the sine wave, i.e. both input and output are in phase.

The same filter, however, has a different impact on a different input signal. Let us design another sine wave $x9$, with a similar amplitude but with a different period of 15.

```
clear
```

```
t = (1:100)';
```

```
x9 = 2*sin(2*pi*t/15);
```

Applying a similar filter and correcting the output $y9$ for the phase shift of the function `filter` yields

```
b9 = ones(1,11)/11;
```

```
m9 = length(b9);
```

```
y9 = filter(b9,1,x9);
```

```
y9 = y9(1+(m9-1)/2:end-(m9-1)/2,1);
```

```
y9(end+1:end+m9-1,1) = zeros(m9-1,1);
```

The output $y9$ is again in phase with the input $x9$, but the amplitude is dramatically reduced compared to that of the input.

```
plot(t,x9,t,y9)
```

```
1-max(y9(40:60))/2
```

```
ans =
```

```
0.6768
```

The running mean over eleven elements reduces the amplitude of this signal by $\sim 68\%$. More generally, the filter response clearly depends on the frequency of the input. The frequency components of a more complex signal containing multiple periodicities are affected in a different way. The frequency response of a filter

```
clear
b10 = ones(1,11)/11;
```

can be computed using the function `freqz`.

```
[h,w] = freqz(b10,1,512);
```

The function `freqz` returns the complex frequency response `h` of the digital filter `b10`. The frequency axis is normalized to π . We transform the frequency axis `w` to the true frequency values `f`. The true frequency values `f` are `w` times the sampling frequency (which is one in our example) divided by 2π .

```
f = 1*w/(2*pi);
```

Next, we calculate and display the magnitude of the frequency response.

```
magnitude = abs(h);
plot(f,magnitude)
xlabel('Frequency'), ylabel('Magnitude')
title('Magnitude')
```

This plot can be used to predict the effect of the filter for any frequency of an input signal. We can interpolate the magnitude of the frequency response to calculate the increase or reduction in a signal's amplitude for a specific frequency. As an example the interpolation of magnitude for a frequency of $1/20$

```
1-interp1(f,magnitude,1/20)

ans =

    0.4260
```

results in the expected $\sim 43\%$ reduction in the amplitude of a sine wave with a period of 20. The sine wave with a period of 15 experiences an amplitude reduction of

```
1-interp1(f,magnitude,1/15)

ans =

    0.6751
```

i.e. approximately 68%, which is similar to the value observed previously. It is very important that such a running mean wipes out certain frequencies, e.g. those for which $\text{magnitude}=0$. As an example, applying the filter to a signal with a period of approximately $1/0.09082$ completely eliminates that signal. Furthermore, since the magnitude of the frequency response is the absolute value of the complex frequency response h , the magnitude response is actually negative between ~ 0.09082 and ~ 0.1816 , between ~ 0.2725 and ~ 0.3633 , and between ~ 0.4546 and the Nyquist frequency. All signal components having frequencies within these intervals are mirrored on the t -axis. As an example we try a sine wave with a period of 7 (e.g. a frequency of approximately 0.1429), which is within the first interval with a negative magnitude response:

```
t = (1:100)';
x10 = 2*sin(2*pi*t/7);

b10 = ones(1,11)/11;
m10 = length(b10);

y10 = filter(b10,1,x10);

y10 = y10(1+(m10-1)/2:end-(m10-1)/2,1);
y10(end+1:end+m10-1,1) = zeros(m10-1,1);

plot(t,x10,t,y10)
```

The sine wave with a period of 7 experiences an amplitude reduction of

```
1-interp1(f,magnitude,1/7)

ans =

    0.7957
```

i.e. approximately 80%, but also changes the sign as we can see from the plot. The elimination of certain frequencies and flipping of the signal have important consequences when interpreting causality in earth sciences. These filters should therefore be avoided completely, even though they are offered as standards in spreadsheet programs. As an alternative, filters with a specific frequency response should be used, such as a Butterworth lowpass filter (Sect. 6.9).

The frequency response can be calculated for all kinds of filters. It is a valuable tool for predicting the effects of a filter on signals in general. The phase response can also be calculated from the complex frequency response h of the filter (Fig. 6.4).

```
phase = 180*angle(h)/pi;

plot(f,phase)
xlabel('Frequency'), ylabel('Phase in degrees')
title('Phase')
```

The phase angle phase is plotted in degrees. We observe frequent jumps in this plot that are an artifact of the function arctangent within the function angle. We can unwrap the phase response to eliminate those jumps that are equal to or larger than 180° , with the help of the function unwrap.

```
plot(f,180*unwrap(angle(h))/pi)
xlabel('Frequency'), ylabel('Phase in degrees')
title('Phase')
```

In our example this has no effect since no jumps occur that are equal to or larger than 180° . Since the filter has a linear phase response phase, no shifts occur in the frequency components of the signals relative to each other. We would therefore not expect any distortions of the signal in the frequency domain. The phase shift of the filter on a specific period can be computed using

```
interp1(f,180*unwrap(angle(h))/pi,1/20) * 20/360
ans =
-5.0000
```

and

```
interp1(f,180*unwrap(angle(h))/pi,1/15) * 15/360
ans =
-5.0000
```

for the sine waves with periods of 20 and 15, respectively. Since MATLAB uses causal indexing for filters, the phase needs to be corrected in a similar way to the delayed output of the filter. In our example we used a filter with a length of eleven. We therefore have to correct the phase by $(11-1)/2 = 5$, which suggests a zero phase shift for the filter for both frequencies.

This also works for recursive filters. Consider a simple sine wave with a period of 8 and the previously employed recursive filter.

```
clear
t = (1:100)';
x11 = 2*sin(2*pi*t/8);

b11 = [0.0048  0.0193  0.0289  0.0193  0.0048];
a11 = [1.0000 -2.3695  2.3140 -1.0547  0.1874];

m11 = length(b11);

y11 = filter(b11,a11,x11);
```

We correct the output for the phase shift introduced by causal indexing and plot both input and output signals.

```

y11= y11(1+(m11-1)/2:end-(m11-1)/2,1);
y11(end+1:end+m11-1,1) = zeros(m11-1,1);

plot(t,x11,t,y11)

```

The magnitude is reduced by

```

1-max(y11(40:60))/2

ans =

    0.6465

```

which is also supported by the magnitude response

```

[h,w] = freqz(b11,a11,512);

f = 1*w/(2*pi);

magnitude = abs(h);

plot(f,magnitude)
xlabel('Frequency'), ylabel('Magnitude')
title('Magnitude Response')

1-interp1(f,magnitude,1/8)

ans =

    0.6462

```

The phase response

```

phase = 180*angle(h)/pi;

f = 1*w/(2*pi);

plot(f,180*unwrap(angle(h))/pi)
xlabel('Frequency'), ylabel('Phase in degrees')
title('Magnitude Response')

interp1(f,180*unwrap(angle(h))/pi,1/8) * 8/360

ans =

   -5.0557

```

must again be corrected for causal indexing. Since the sampling interval was one and the filter length is five, we have to add $(5-1)/2 = 2$ to the phase shift of -5.0557 . This suggests a corrected phase shift of -3.0557 , which is exactly the delay seen on the plot.

```
plot(t,x11,t,y11), axis([30 40 -2 2])
```

The next section gives an introduction to the design of filters with a desired frequency response. These filters can be used to amplify or suppress different components of arbitrary signals.

6.9 Filter Design

We now aim to design filters with a specific frequency response. We first generate a synthetic signal `x12` with two periods, 50 and 5. The power spectrum of the signal shows the expected peaks at frequencies of 0.02 and 0.20.

```
clear, close all, clc

t = 0 : 1000;
x12 = 2*sin(2*pi*t/50) + sin(2*pi*t/5);

plot(t,x12), axis([0 200 -4 4])

[Pxx,f] = periodogram(x12,[],1024,1);

plot(f,Pxx)
xlabel('Frequency')
ylabel('Power')
```

The Butterworth filter design technique is widely used in order to create filters of any order with a lowpass, highpass, bandpass and bandstop configuration (Fig. 6.5). In our example we would like to design a five-order lowpass filter with a cutoff frequency of 0.10. The inputs of the function `butter` are the order of the filter and the cutoff frequency normalized to the Nyquist frequency, which in our example is 0.5 (i.e. half of the sampling frequency). The outputs from `butter` are the filter weights `a12` and `b12`.

```
[b12,a12] = butter(5,0.1/0.5);
```

The frequency characteristics of the filter show a relatively smooth transition from the passband to the stopband, but the advantage of the filter is its low order.

```
[h,w] = freqz(b12,a12,1024);
f = 1*w/(2*pi);

plot(f,abs(h)), grid
xlabel('Frequency')
ylabel('Magnitude')
```


We can again apply the filter to the signal by using the function `filter`. Frequency-selective filters such as lowpass, highpass, bandpass and bandstop filters are designed to suppress certain frequency bands, but phase shifts should be avoided. The function `filtfilt` provides zero phase-shift forward and reverse digital filtering. After filtering in the forward direction, the filtered sequence is reversed and runs back through the filter. The magnitude of the signal is not affected by this operation, since it is either 0 or 100% of the initial amplitude, depending on the frequency. Any phase shifts introduced by the filter are canceled out by the forward and reverse application of the same filter. This function also helps to overcome the problems with causal indexing of filters in MATLAB by eliminating the phase differences between the causal and non-causal versions of the same filter. Filtering, and then plotting the results clearly illustrates the effects of the filter.

```
xf12 = filtfilt(b12,a12,x12);

plot(t,x12,'b-',t,xf12,'r-')
axis([0 200 -4 4])
```

We might now wish to design a new filter with a more rapid transition from passband to stopband. Such a filter requires a higher order, i.e. it needs a larger number of filter weights `a12` and `b12`. We now create a 15-order Butterworth filter as an alternative to the above filter.

```
[b13,a13] = butter(15,0.1/0.5);

[h,w] = freqz(b13,a13,1024);

f = 1*w/(2*pi);

plot(f,abs(h)), grid
xlabel('Frequency')
ylabel('Magnitude')
```

The frequency response is clearly improved. The entire passband is relatively flat at a value of 1.0, whereas the stopband is approximately zero everywhere. We next modify our input signal by introducing a third period of 5. This signal is then used to illustrate the operation of a Butterworth bandstop filter.

```
clear

t = 0 : 1000;
x14 = 2*sin(2*pi*t/50) + sin(2*pi*t/10) + 0.5*sin(2*pi*t/5);
plot(t,x14), axis([0 200 -4 4])

[Pxx,f] = periodogram(x14,[],1024,1);

plot(f,Pxx)
```

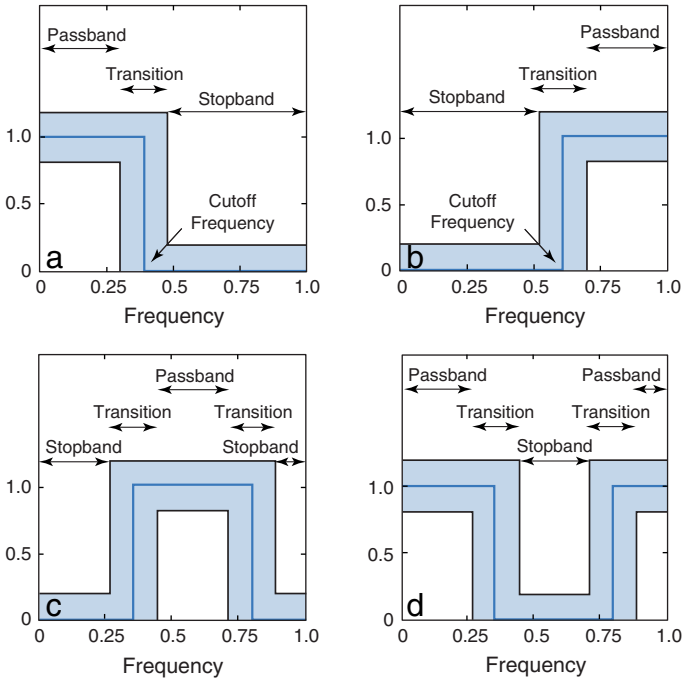


Fig. 6.5 Frequency responses for the fundamental types of frequency-selective filters. **a** Lowpass filter to suppress the high-frequency component of a signal. In earth sciences, such filters are often used to suppress high-frequency noise in a low-frequency signal. **b** Highpass filters to remove all low frequencies and trends in natural data. **c, d** Bandpass and bandstop filters to extract or suppress a certain frequency band. The solid line in all graphics depicts the ideal frequency response of a frequency-selective filter, while the gray band shows the tolerance for a low-order design of such a filter. In practice, the frequency response lies within the gray band. Modified from von Grünigen (2014).

The new Butterworth filter is a bandstop filter. The stopband of the filter is between the frequencies 0.05 and 0.15. It can therefore be used to suppress the period of 10, corresponding to a frequency of 0.1.

```
rng(0)
xn14 = x14 + randn(1,length(t));

[b14,a14] = butter(5,[0.05 0.15]/0.5,'stop');
xf14 = filtfilt(b14,a14,x14);

[Pxx,f] = periodogram(xf14,[],1024,1);

plot(f,Pxx)

figure
plot(t,xn14,'b- ',t,xf14,'r- '), axis([0 200 -4 4])
```

The plots show the effect of this filter. The frequency band between 0.05 and 0.15, and therefore also the frequency of 0.1, have been successfully removed from the signal.

6.10 Adaptive Filtering

The fixed filters used in the previous sections make the basic assumption that the signal degradation is known and does not change with time. However, *a priori* knowledge of the signal and noise statistical characteristics is not usually available in most applications. In addition, both the noise level and the variance of the genuine signal can be highly nonstationary with respect to time, e.g. stable isotope records during a glacial-interglacial transition. Fixed filters cannot therefore be used in a nonstationary environment where there is no knowledge of the signal-to-noise ratio.

Adaptive filters, widely used in the telecommunication industry, could help to overcome these problems. An adaptive filter is an inverse modeling process that iteratively adjusts its own coefficients automatically without requiring any *a priori* knowledge of the signal and the noise. The operation of an adaptive filter includes (1) a filtering process, the purpose of which is to produce an output in response to a sequence of data, and (2) an adaptive process, providing a mechanism for adaptive control of the filter weights (Haykin 2003).

In most practical applications the adaptive process is oriented towards minimizing an estimation error. The estimation error e at an instant i is defined as the difference between the desired response d_i and the actual filter output y_i , which is the filtered version of a signal x_i , as shown by

$$e_i = d_i - y_i$$

where $i = 1, 2, \dots, N$ and N is the length of the input data vector. In the case of a nonrecursive filter characterized by a vector of filter weights W with f elements, the filter output y_i is given by the inner product of the transposed vector W and the input vector X_i .

$$y_i = W^T \cdot X_i$$

The choice of desired response d that is used in the adaptive process depends on the application. Traditionally, d is a combination signal that is comprised of a signal s and random noise n_0 . The signal x contains noise n_1 that is uncorrelated with the signal s but correlated in some unknown way with the noise n_0 . In noise canceling systems the practical objective is to produce a system output y that is a best fit in the least-squares sense to the desired response d .

Different approaches have been developed to solve this multivariate minimum error optimization problem (e.g. Widrow and Hoff 1960; Widrow et al. 1975; Haykin 2003). The selection of one algorithm over another is influenced by various factors including the rate of convergence (the number of adaptive steps required for the algorithm to converge closely enough to an optimum solution), the misadjustment (the measure of the amount by which the final value of the mean-squared error deviates from the minimum squared error of an optimal filter, e.g. Wiener 1949; Kalman and Bucy 1961), and the tracking (the capability of the filter to work in a nonstationary environment, i.e. to track changing statistical characteristics of the input signal) (Haykin 2003).

The simplicity of the least-mean-squares (LMS) algorithm, originally developed by Widrow and Hoff (1960), has made it the benchmark against which other adaptive filtering algorithms are tested. For applications in earth sciences we use this filter to extract the noise from two signals S and X , both containing the same signal s , but with uncorrelated noise n_1 and n_2 (Hattingh 1988). As an example, consider a simple duplicate set of measurements on the same material, e.g. two parallel stable isotope records from the same foraminifera species. You would expect two time-series, each with N elements, containing the same desired signal overlain by different, uncorrelated noise. The first record is used as the primary input S

$$S = (s_1, s_2, \dots, s_N)$$

and the second record as the reference input X .

$$X = (x_1, x_2, \dots, x_N)$$

As demonstrated by Hattingh (1988), the desired noise-free signal can be extracted by filtering the reference input X using the primary input S as the desired response d . The minimum error optimization problem is solved by the least-mean-square norm. The mean-squared error e_i^2 is a second-order function of the weights in the nonrecursive filter. The dependence of e_i^2 on the unknown weights W may be seen as a multidimensional paraboloid with a uniquely defined minimum point. The weights corresponding to the minimum point on this error performance surface define the optimal Wiener solution (Wiener 1949). The value computed for the weight vector W using the LMS algorithm represents an estimator whose expected value approaches the Wiener solution as the number of iterations approaches infinity (Haykin 2003). Gradient methods are used to reach the minimum point on the error performance surface. To simplify the optimization problem, Widrow and Hoff (1960) developed an approximation for the required gradient function that can be computed directly from the data. This leads to a simple relationship for updating the filter-weight vector W .

$$W_{i+1} = W_i + 2 \cdot u \cdot e_i \cdot X_i$$

The new parameter estimate W_{i+1} is based on the previous set of filter weights W_i plus a term that is the product of a bounded step size u , a function of the input state X_i and a function of the error e_i . In other words, error e_i calculated from the previous step is fed back into the system to update filter coefficients for the next step (Fig. 6.6). The fixed convergence factor u regulates the speed and stability of adaption. A low value of u ensures a higher level of accuracy, but more data are needed to enable the filter to reach the optimum solution. In the modified version of the LMS algorithm by Hattingh (1988), this problem is overcome by feeding the data back so that the filter can have another chance to improve its own coefficients and adapt to the changes in the data.

In the following example introducing the use of the function `canc`, each of these loops is called an iteration, and many of these loops are required if optimal results are to be achieved. This algorithm extracts the noise-free signal from two vectors, x and s , containing the correlated signals and uncorrelated noise. As an example we generate two signals $yn1$ and $yn2$ containing the same sine wave but different Gaussian noise.

```
clear, close all, clc

x = 0 : 0.1 : 100; x = x';
y = sin(x);
rng(0)
yn1 = y + 0.5*randn(size(y));
yn2 = y + 0.5*randn(size(y));

plot(x,yn1,x,yn2)
```

The algorithm `canc` formats both signals, feeds them into the filter loop, corrects the signals for phase shifts, and formats the signals for the output. The required inputs are the signals x and s , the step size u , the filter length l and the number of iterations `iter`. In our example the two noisy signals are $yn1$ and $yn2$. We make an arbitrary choice of a filter with $l=5$ filter weights. A value of u in the range of $0 < u < 1/\lambda_{max}$, where λ_{max} is the largest eigenvalue of the autocorrelation matrix for the reference input, leads to reasonable results (Haykin 2003) (Fig. 6.7). The value of u is computed using

```
k = kron(yn1,yn1');
u = 1/max(eig(k))
```

where `kron` returns the Kronecker tensor product of $yn1$ and $yn1'$ (which is a matrix formed by taking all possible products between the elements of $yn1$ and $yn1'$), and `eig` returns the eigenvector of k . This yields

```
u =
0.0019
```

We now run the adaptive filter `canc` for 20 iterations and use the above value of u .

```
[z,e,mer,w] = canc(yn1,yn2,0.0019,5,20);
```

The output variables from `canc` are the filtered primary signal z , the extracted noise e , the mean-squared error mer for the number of iterations it performed with step size u , and the filter weights w for each data point in $yn1$ and $yn2$. The plot of the mean-squared error mer

```
plot(mer)
```

illustrates the performance of the adaptive filter, although the chosen step size $u = 0.0019$ clearly results in a relatively rapid convergence. In most examples a smaller step size decreases the rate of convergence but improves the quality of the final result. We therefore reduce u and run the filter again with further iterations.

```
[z,e,mer,w] = canc(yn1,yn2,0.0001,5,20);
```

The plot of the mean-squared error mer against the iterations

```
plot(mer)
```

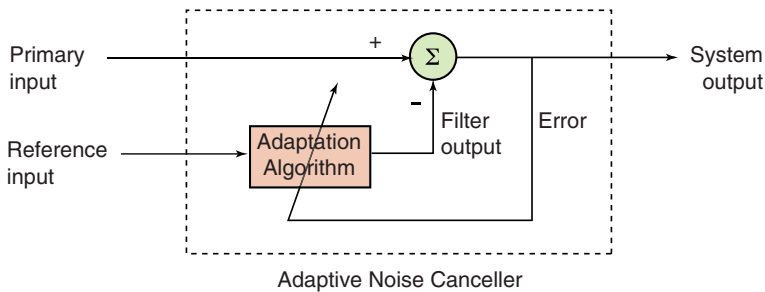


Fig. 6.6 Schematic of an adaptive filter. Each iteration involves a new estimate of the filter weights W_{i+1} based on the previous set of filter weights W_i plus a term that is the product of a bounded step size u , a function of the filter input X_i , and a function of the error e_i . In other words, error e_i calculated from the previous step is fed back into the system to update filter coefficients for the next step (modified from Trauth 1998).

now converges after about six iterations. In practice, the user should vary the parameters u and l to obtain the optimum result. We can now compare the filter output with the original noise-free signal.

```
plot(x,y,'b',x,z,'r')
```

This plot shows that the noise level of the signal has been reduced dramatically by the filter. Next, the plot

```
plot(x,e,'r')
```

shows the noise extracted from the signal. Using the last output from `canc` we can calculate and display the mean filter weights of the final iteration from w .

```
wmean = mean(w)
plot(wmean)
```

The frequency characteristic of the filter provides a more illustrative representation of the effect of the filter.

```
[h,w] = freqz(www,1,1024);
f = 1*w/(2*pi);
plot(f,abs(h))
```

which clearly shows that the filter is a lowpass filter with a relatively smooth transition band. This means that it does not have the quality of a recursive filter designed, for example, using the Butterworth approach. However, the filter weights are calculated in an optimization process rather than chosen arbitrarily.

The strength of an adaptive filter is in filtering a time series with a variable signal-to-noise ratio along the time axis. Since the filter-weight vector W is updated for each individual data point, these filters are even used in real-time applications such as telecommunication systems. We examine this behavior through an example in which the signal-to-noise ratio in the middle of the time series ($x = 500$) is reduced from about 10% to zero.

```
clear
x = 0 : 0.1 : 100; x = x';
y = sin(x);
rng(0)
yn1 = y + 0.5*randn(size(y));
yn2 = y + 0.5*randn(size(y));

yn1(501:1001) = y(501:1001);
yn2(501:1001) = y(501:1001);

plot(x,yn1,x,yn2)
```

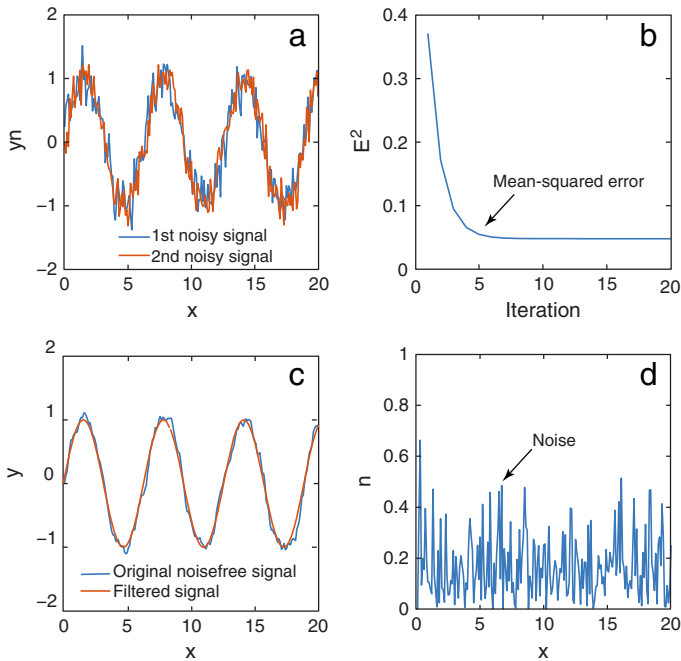


Fig. 6.7 Output of the adaptive filter. **a** The duplicate records corrupted by uncorrelated noise are fed into the adaptive filter with 5 weights with a convergence factor of 0.0019. After 20 iterations, the filter yields **b** the learning curve, **c** the noise-free record, and **d** the noise extracted from the duplicate records.

The value of u is again computed by

```
k = kron(yn1,yn1');
u = 1/max(eig(k))
```

which yields

```
u =
    0.0016
```

We now run the adaptive filter `canc` for 20 iterations and use the above value of u .

```
[z,e,mer,w] = canc(yn1,yn2,0.0016,5,20);
```


The plot of the mean-squared error `mer` versus the number of performed iterations `it` with step size `u`

```
plot(mer)
```

illustrates the performance of the adaptive filter, although the chosen step size $u = 0.0016$ clearly results in a relatively rapid convergence. Again, we can now compare the filter output with the original noise-free signal.

```
plot(x,y,'b',x,z,'r')
```

This plot shows that the filter output `y` is almost the same as the noise-free signal `x`. The plot

```
plot(x,e,'r')
```

shows the noise extracted from the signal. Here we can observe some signal components that have been removed by the filter in error, from within the noise-free segment of the time series beyond $x = 500$. Using the last output of `canc`, we can calculate and display the filter weights `w` of the final iteration

```
surf(w(3:999,:)), shading interp
```

which shows nicely the adaptation of the filter weights before and after the change in the signal-to-noise ratio. We plot only the middle part of `w`, as the edges `1:(1-1)/2` and `end-(1-1)/2:end` are all zero due to the length of the filter `1`. We can also use this example to demonstrate the effect that the values of `u` and `1` have on the performance of the adaptive filter. In theory, a smaller `u` leads to more accurate results, but the rapid adaptation to a changing signal-to-noise ratio does not work well. Larger values of `1` also give better results but the number of data points lost through the filtering process increases by $(1-1)/2$.

The graphical user interface (GUI) version `canctool` can be used as an alternative to `canc` (Fig. 6.8). We use the same example data set as before; this function was created using the *GUI Design Environment* (GUIDE) (see Sect. 2.10). We again generate two signals, `yn1` and `yn2`, containing the same sine wave but different Gaussian noise.

```
clear
x = 0 : 0.1 : 100; x = x';
y = sin(x);
```

```
rng(0)
yn1 = y + 0.5*randn(size(y));
yn2 = y + 0.5*randn(size(y));
```

We next run `canctool`

```
canctool(yn1,yn2)
```

and watch the filter iteratively adjusting its filter weights. The function `canctool` accepts the primary and reference signals as inputs. The GUI provides default values for the filter length, the convergence rate, and the number of iterations. The user must adjust these values to the specific data, for example, using the formula for a suitable convergence rate that has been described above. As an example, we choose a filter length of 11, a convergence rate of 0.00001, and 30 iterations. Once the calculation is complete, `canctool` exports the filtered primary signal `yy`, the filtered reference signal `zz` (using the same set of filter weights), the noise `ee` extracted from the primary signal, and the mean-squared error `mer` for each iteration, to the workspace using the function `assignin`. The application of this algorithm has been demonstrated on duplicate oxygen-isotope records from ocean sediments (Trauth 1998). This work by MH Trauth illustrates the use not only of the modified LMS algorithm, but also of another type of adaptive filter, the recursive least-squares (RLS) algorithm, in various different environments (Haykin 2003; Trauth 1998).

Recommended Reading

- Alexander ST (1986) *Adaptive Signal Processing: Theory and Applications*. Springer, Berlin Heidelberg New York
- Buttkus B (2000) *Spectral analysis and filter theory in applied geophysics*. Springer, Berlin, Heidelberg, New York
- Cowan CFN, Grant PM (1985) *Adaptive filters*. Prentice Hall, Englewood Cliffs, NJ
- von Grünigen DH (2014) *Digitale Signalverarbeitung mit einer Einführung in die kontinuierlichen Signale und Systeme, Fünfte, neu bearbeitete*. Fachbuchverlag Leipzig im Carl Hanser Verlag, Leipzig
- Hattingh M (1988) A new data adaptive filtering program to remove noise from geophysical time- or space series data. *Comput Geosci* 14(4):467–480
- Haykin S (2003) *Adaptive filter theory*. Prentice Hall, Englewood Cliffs, NJ
- Kalman R, Bucy R (1961) New results in linear filtering and prediction theory. *ASME trans Ser D J Basic Eng* 83:95–107
- MathWorks (2020) *Signal processing toolbox—user’s guide*. The MathWorks, Natick, MA
- Sibul LH (1987) *Adaptive signal processing*. IEEE Press
- Trauth MH (1998) Noise removal from duplicate Paleooceanographic time-series: the use of adaptive filtering techniques. *Math Geol* 30(5):557–574
- Weeks M (2007) *Digital signal processing using MATLAB and wavelt*. Infinity Science Press, Jones and Bartlett Publishers, Boston, Toronto, London, Singapore
- Widrow B, Hoff M Jr (1960) Adaptive switching circuits. *IRE WESCON Conv. Rev.* 4:96–104

- Widrow B, Glover JR, McCool JM, Kaunitz J, Williams CS, Hearn RH, Zeidler JR, Dong E, Goodlin RC (1975) Adaptive noise cancelling: principles and applications. *Proc IEEE* 63 (12):1692–1716
- Wiener N (1949) Extrapolation, interpolation and smoothing of stationary time series, with engineering applications. MIT Press, Cambridge, Mass (reprint of an article originally issued as a classified National Defense Research Report, February, 1942)

7.1 Types of Spatial Data

Most data in earth sciences are spatially distributed, either as vector data, (points, lines, polygons) or as raster data (gridded topography). Vector data are generated by digitizing map objects such as drainage networks or outlines of lithologic units. Raster data can be obtained directly from a satellite sensor output, but gridded data can also be interpolated from irregularly-distributed field sampling (gridding).

The following section introduces the use of vector data, using coastline data as an example (Sect. 7.2). The acquisition and handling of raster data are then illustrated using digital topographic data (Sects. 7.3–7.5). The availability and use of digital elevation data has increased considerably since the early 90s. With a resolution of 5 arc minutes (about 9 km), ETOPO5 was one of the first data sets available for topography and bathymetry. In October 2001 it was replaced by ETOPO2, which has a resolution of 2 arc minutes (about 4 km), and in March 2009 ETOPO1 became available, which has a resolution of 1 arc minutes (about 2 km). There is also a data set for topography called GTOPO30, completed in 1996, that has a horizontal grid spacing of 30 arc seconds (about 1 km). More recently, the 30 and 90 m resolution data from the Shuttle Radar Topography Mission (SRTM) have replaced the older data sets in most scientific studies. Section 7.6 demonstrates how to export 3D graphics objects such as digital terrain models to create interactive Adobe PDF documents, in particular for use with tablet computers.

The second part of this chapter deals with the estimation of continuous surfaces from unevenly-spaced data and the statistics of spatial data (Sects. 7.7–7.8). In earth sciences, most data are collected in an irregular pattern. Access to rock samples is

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_7) contains supplementary material, which is available to authorized users.

often restricted to natural outcrops such as shoreline cliffs and the walls of a gorge, or anthropogenic outcrops such as road cuttings and quarries. The sections on interpolating such unevenly-spaced data illustrate the use of the most important gridding routines and outline the potential pitfalls when using these methods. Sections 7.9–7.11 introduce various methods for statistically analyzing spatial data, including the application of statistical tests to point distributions (Sect. 7.9), the spatial analysis of digital elevation models (Sect. 7.10), and an overview of geostatistics and kriging (Sect. 7.11).

This chapter requires the Mapping Toolbox (MathWorks 2020a), although most graphics routines used in our examples can be easily replaced by standard MATLAB functions. Section 7.6 requires Simulink 3D Animation to create animated 3D graphics objects (MathWorks 2020b). An alternative and useful mapping toolbox by Rich Pawlowicz (Earth and Ocean Sciences, at the University of British Columbia) is available from

<http://www.eos.ubc.ca/~rich/>

An popular alternative tool for analyzing digital terrain models is the TopoToolbox by Wolfgang Schwanghart (Schwanghart and Kuhn 2010, Schwanghart and Scherler 2014).

<https://topotoolbox.wordpress.com>

This toolbox is being continually expanded and current topics and applications are discussed in a blog. The handling and processing of large spatial data sets requires a computing system with at least 4 GB physical memory.

7.2 The Global Geography Database GSHHG

The *Global Self-consistent, Hierarchical, High-resolution Geography* (GSHHG) database is an amalgamation of two public domain databases by Paul Wessel (SOEST, University of Hawaii, Honolulu, HI) and Walter Smith (NOAA Laboratory for Satellite Altimetry, Silver Spring, MD) (Wessel and Smith 1996). The GSHHG database consists of the older GSHHS shoreline database (Soluri and Woodson 1990; Wessel and Smith 1996) with the poor quality Antarctica data replaced by more accurate data from Bohlander and Scambos (2007) and with rivers and borders taken from the CIA World Data Bank II (WDBII) (Gorny 1977). The GSHHG data can be downloaded from the SOEST server

<http://www.soest.hawaii.edu/pwessel/gshhg/index.html>

in three different formats (as Generic Mapping Tools (GMT) files, as ESRI shapefiles, or as native binary files) and in a wide range of spatial resolutions. This example uses version 2.3.7 of the native binary file *gshhg-bin-2.3.7.zip*. We first use the MATLAB `unzip` function to decompress this file

```
clear, close all, clc

unzip('gshhg-bin-2.3.7.zip')
```

which creates files containing the shorelines (*gshhs*), borders (*wdb_borders*) and rivers (*wdb_rivers*), each at different resolutions ranging from *f* (full resolution, 95.8 MB) to *c* (crude resolution, 183 KB), as explained on Paul Wessel's SOEST webpage. As an example we import the high-resolution shoreline data set from file *gshhs_h.b* using.

```
S = gshhs('gshhs_h.b');
```

We obtain a structure array *S* that includes the longitude/latitude coordinates of the shoreline segments, together with various other types of information. The elements of *S* contain the shorelines of the continents and islands: for example *S(2)* contains the longitude/latitude coordinates of Africa. We can extract the shoreline coordinates by typing.

```
data(:,1) = S(2).Lon;
data(:,2) = S(2).Lat;
```

We save the longitude/latitude coordinates in an ASCII file and a binary *.mat* file using

```
save coastline_data.txt data -ascii
save coastline_data.mat
```

and load it after clearing the workspace using

```
clear

load coastline_data.mat
```

There are different ways of displaying the data. As an example, we can use the `line` command to create a line plot, after having first defined the figure and axes properties, as follows:

```
figure('Color',[1 1 1]);
axes('Box','on',...
     'DataAspectRatio',[1 1 1]);
line(data(:,1),data(:,2),...
     'Color',[0 0 0]);
xlabel('Longitude');
ylabel('Latitude');
```

More advanced plotting functions are contained in the Mapping Toolbox, which allows alternative versions of this plot to be generated (Fig. 7.1),

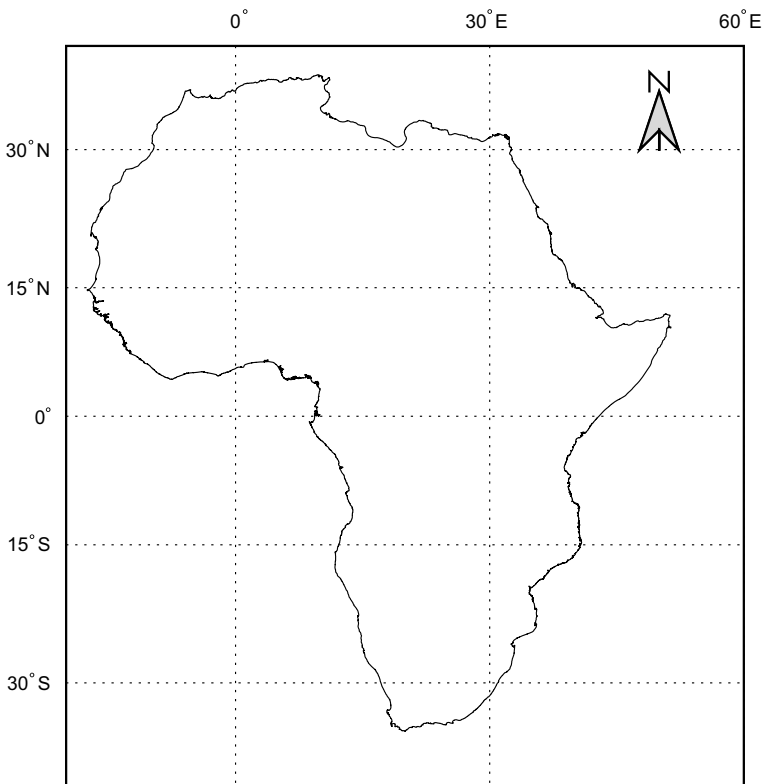


Fig. 7.1 Shoreline display from the GSHHS data set. The map shows an area between latitudes 0 and 15° north, and longitudes 40 and 50° east. This simple map is made using the function `plot`, with equal axis aspect ratios (data from Wessel and Smith 1996).

```

figure('Color',[1 1 1]);
axes('Visible','off')
axesm('MapProjection','mercator', ...
      'MapLatLimit',[-40 40], ...
      'MapLonLimit',[-20 60], ...
      'Frame','on', ...
      'FLineWidth',1,...
      'MeridianLabel','on', ...
      'ParallellLabel','on',...
      'Grid','on');
northarrow('Latitude',30,...
          'Longitude',50,...
          'FaceColor',[0.8 0.8 0.8])
plotm(data(:,2),data(:,1),'k');

```

or using `geoshow` instead of `plotm`.

```

figure('Color',[1 1 1]);
axes('Visible','off')
axesm('MapProjection','mercator', ...
      'MapLatLimit',[-40 40], ...
      'MapLonLimit',[-20 60], ...
      'Frame','on', ...
      'FLineWidth',1,...
      'MeridianLabel','on', ...
      'ParallellLabel','on',...
      'Grid','on');
geoshow(data(:,2),data(:,1),...
        'Color', [0 0 0])
northarrow('Latitude',30,...
          'Longitude',50,...
          'FaceColor',[0.8 0.8 0.8])

```

Note that the input for `plotm` must be in the correct order, with the *latitude* entered first, followed by the *longitude* (i.e. the second column of the data matrix must be entered first). In contrast, the function `plot` requires an *xy* input, i.e. input must be in the opposite order, with the first column of the matrix entered first. The function `axesm` defines the map axis and sets various map properties such as the map projection, the map limits, and the axis labels. The function `northarrow` adds a graphic element pointing to geographic north.

7.3 The 1 Arc-Minute Gridded Global Relief Data ETOPO1

The *1 arc-minute global relief model of the Earth's surface* (ETOPO1) is a global database of topographic and bathymetric data on a regular 1 arc-minute grid (about 2 km) (Amante and Eakins 2009). Older ETOPO2 and ETOPO5 global relief grids have been superseded but are still available. ETOPO1 is a compilation of data from a variety of sources. It can be downloaded from the NOAA National Centers for Environmental Information (NCEI) webpage.

<https://www.ngdc.noaa.gov/mgg/global/global.html>

We can download either the whole-world grids, or custom grids for ice surface or bedrock. We create and extract a custom grid online from ETOPO1 using NCEI's WCS Client.

<http://maps.ngdc.noaa.gov/viewers/wcs-client/>

We first choose the *ETOPO1* (bedrock) layer. We then select *Select with Coordinates* by clicking the *xy* icon in the upper-left corner of the map and specify an area of interest; for example, a latitude from *North: 20* (20°N) and *South: -20* (20°S), and a longitude between *East: 30* (30°E) and *East: 60* (60°E) covering the East African coast. Negative latitude and longitude values are assigned to the southern hemisphere and for the western hemisphere, respectively. Clicking *OK* marks the area of interest as a transparent yellow rectangle on the map. Next, we choose *ArcGIS ASCII Grid* as the Output Format and download the file *etopo1_bedrock.asc*, which has the following content:

```
ncols      1801
nrows      2401
xllcorner  29.991666666667
yllcorner  -20.008333333333
cellsize    0.016666666667
294 299 293 288 285 282 ...
237 241 245 266 264 274 ...
259 263 267 262 263 266 ...
310 306 300 294 296 291 ...
348 346 352 356 353 353 ...
381 383 381 381 382 381 ...
(cont'd)
```

The headers document the size of the data matrix (e.g. 1801 columns and 2401 rows in our example), the coordinates of the lower-left corner (e.g. approximately $x = 20$ and $y = -20$), and the cell size (e.g. $\sim 0.0167 = 1/60^\circ$ latitude and longitude, or 1 arc minute). Older versions of the ETOPO1 data set, as well as other similar data sets, may also contain the -32768 flag for data voids. We comment the header by typing % at the beginning of the first five lines

```
%ncols      1801
%nrows      2401
%xllcorner  29.991666666667
%yllcorner  -20.008333333333
%cellsize   0.016666666667
294 299 293 288 285 282 ...
237 241 245 266 264 274 ...
259 263 267 262 263 266 ...
310 306 300 294 296 291 ...
348 346 352 356 353 353 ...
381 383 381 381 382 381 ...
(cont'd)
```

save the file as *etopo1_data.txt*, and then load the data into the workspace.

```
clear, close all, clc

ETOP01 = load('etopo1_data.txt');
```

We flip the matrix up and down using `flipud` to change the indexing of the data array according to MATLAB conventions. The `-32768` flag for data voids, if any, must then be replaced by the MATLAB representation for Not-a-Number `NaN`.

```
ETOP01 = flipud(ETOP01);
ETOP01(ETOP01 == -32768) = NaN;
```

Finally, we check that the data are now correctly stored in the workspace by printing the minimum and maximum elevations for the area.

```
max(ETOP01(:))
min(ETOP01(:))
```

In this example the maximum elevation for the area is 5,677 m and the minimum elevation is -5,859 m. The reference level is the sea level at 0 m. We now define a coordinate system using the information that the lower-left corner is at latitude 20°S and longitude 30°E. The resolution is 1 arc minute, corresponding to 1/60°.

```
[LON,LAT] = meshgrid(30:1/60:60,-20:1/60:20);
```

We now generate a colored surface from the elevation data using the function `surf`.

```
surf(LON,LAT,ETOP01)
colormap jet
shading interp
axis equal, view(0,90)
colorbar
```

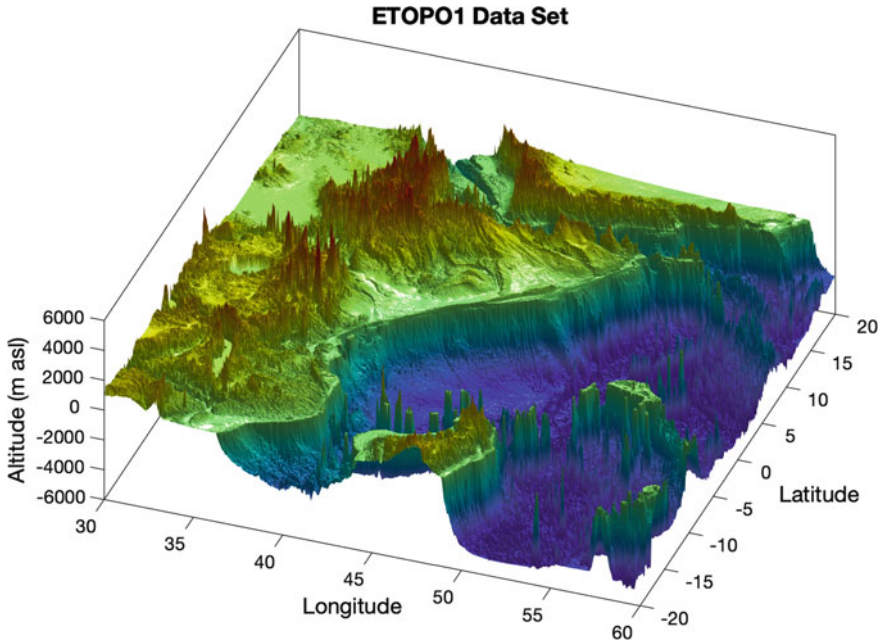


Fig. 7.2 Surface plot of the ETOPO1 elevation data using `light`. The plot uses Phong as the lighting type, which is very popular in 3D computer graphics, creating a combined diffuse and specular reflection on surfaces. The specular exponent of 20 defines the size of the highlight spot on the surface, which can vary between 0 and 500 (data from Amante and Eakins 2009).

This script opens a new Figure Window and generates a colored surface. The surface is highlighted by a set of color shades, and is displayed in an overhead view. A spectacular way to plot digital topography is as a surface plot with angled lighting (Fig. 7.2).

```
figure('Color',[1 1 1]);
axes('Box','on',...
     'View',[20 60],...
     'LineWidth',0.6,...
     'FontSize',8), hold on
surf(LON,LAT,ETOPO1,...
     'SpecularExponent',20,...
     'FaceLighting','phong',...
     'FaceColor','interp',...
     'EdgeColor','none')
light('Style','local',...
     'Position',[145 70 155801]);
title('ETOPO1 Data Set',...
     'FontSize',10)
```

```
xlabel('Longitude',...
      'FontSize',8)
ylabel('Latitude',...
      'FontSize',8)
zlabel('Altitude (m asl)',...
      'FontSize',8)
colormap(jet)
```

We can now export the graph in a PNG format with 300 dpi resolution using the command.

```
print -dpng -r300 etopo1.png
```

7.4 The 30 Arc-Seconds Elevation Model GTOPO30

The *Global 30 Arc-Second Elevation Data* (GTOPO30) is a 30 arc second (approximately 1 km) global digital elevation data set that contains only elevation data and no bathymetry. The data set has been developed by the Earth Resources Observation System Data Center and is available from the USGS EarthExplorer website.

<https://earthexplorer.usgs.gov>

The EarthExplorer website can be used to search, preview, and download satellite images, aerial photographs and cartographic products through the US Geological Survey (USGS). The GTOPO30 data set is located under the Digital Elevation category. On this webpage we first select the desired map section in the *Search Criteria*, either by entering the coordinates of the four corners of the map or by zooming into the area of interest and selecting *Use Map*. As an example we enter the coordinates of the Suguta Valley in the Northern Kenya Rift: 2°8'37.58"N 36°33'47.06"E. We then choose *GTOPO30* from the *Digital Elevation* collection as the *Data Set* and click *Results*. Clicking *Results* produces a list of records, together with a toolbar for previewing and downloading data. In our example we find a single data set *GT30E020N40*, acquired on 01 December 1996, centered at 15°N, 40°E. We need to register with the USGS website, log on, and then download the data set as a 55.0 MB GeoTIFF and a 24.9 MB DEM, which is provided as a 26.1 MB compressed zip file. We can use the MATLAB unzip function to decompress the file *gt30e020n40_dem.zip*, as follows.

```
clear, clc, close all

unzip('gt30e020n40_dem.zip')
```

After decompressing the zip file we obtain eight files containing the raw data and header files in various formats. The file also provides a GIF image of a shaded relief display of the data.

Importing the GTOPO30 data into the workspace is simple. The Mapping Toolbox provides an import routine `gtopo30` that reads the data and stores it onto a regular data grid. We import only a subset of the original matrix using the `gtopo30` command included in the Mapping Toolbox. Unfortunately, the GTOPO30 data have been renamed on the USGS EarthExplorer server and the function `gtopo30` has not yet been updated to take into account this change. The binary file containing the digital elevation model extracted from *gt30e020n40_dem.zip* is named *gt30e020n40.dem* but `gtopo30` expects the file to be named *E020N40.dem*. We therefore rename the files by removing *gt30* from the file name in the unzipped collection and use

```
latlim = [-5 5]; lonlim = [30 40];
GTOPO30 = gtopo30('E020N40',1,latlim,lonlim);
```

to import the data. This script reads the data from the tile *e020n40* (without file extension) at full resolution (scale factor = 1) into the matrix `gtopo30`, which has dimensions of 1,200 by 1,200 cells. The coordinate system is defined using the *lon/lat* limits listed above. The resolution is 30 arc seconds, corresponding to 1/120 of a degree.

```
[LON,LAT] = meshgrid(30:1/120:40-1/120,-5:1/120:5-1/120);
```

We need to reduce the limits by a factor of 120 in order to obtain a matrix of similar dimensions to the GTOPO30 matrix. A grayscale image can be generated from the elevation data using the function `surf`. The fourth power of the colormap gray is used to darken the map and the colormap is then flipped vertically in order to obtain dark colors for high elevations and light colors for low elevations, instead of the other way around.

```
surf(LON,LAT,GTOPO30)
shading interp
colormap(flipud(gray.^4))
axis equal, view(0,90)
colorbar
```

This script opens a new Figure Window and generates the gray surface using interpolated shading, displayed in an overhead view. We again plot the digital topography as a surface plot with angled lighting (Fig. 7.3).

```
clf
axes('Visible','off')
```

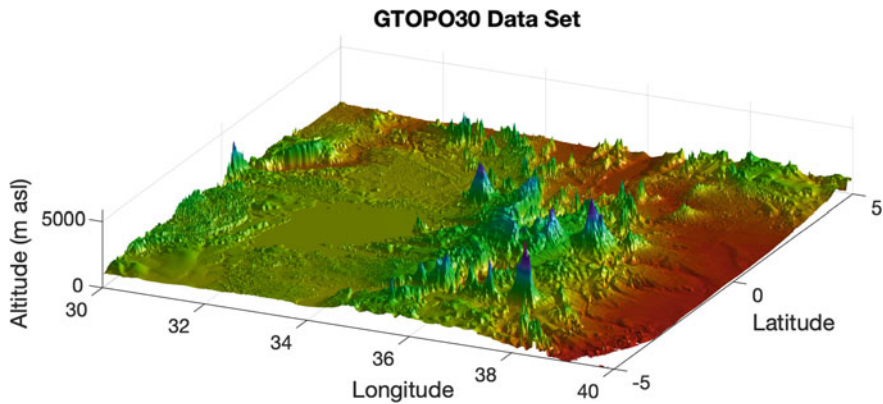


Fig. 7.3 Surface plot of the GTOPO30 elevation data using `light`. The plot uses *Phong* as the lighting type, which is very popular in 3D computer graphics, creating a combined diffuse and specular reflection on surfaces (data from the U.S. Geological Survey).

```
surf (LON,LAT,GTOPO30,...
      'SpecularExponent',20,...
      'FaceLighting','phong',...
      'FaceColor','interp',...
      'EdgeColor','none');
light('Style','local',...
      'Position',[145 70 155801])
set(gca,'View',[25 20])
colormap hsv
daspect([1 1 4000])
title('GTOPO30 Data Set',...
      'FontSize',10)
xlabel('Longitude',...
      'FontSize',8)
ylabel('Latitude',...
      'FontSize',8)
zlabel('Altitude (m asl)',...
      'FontSize',8)
```

We again export the graph as a PNG file

```
print -dpng -r300 gtopo30.png
```

with a resolution of 300 dpi.

7.5 The Shuttle Radar Topography Mission SRTM

The *Shuttle Radar Topography Mission* (SRTM) was a radar system on board the Space Shuttle *Endeavour* during an 11-day mission in February 2000 (Farr and Kobrick 2000; Farr et al. 2008). SRTM was an international project spearheaded by the National Geospatial-Intelligence Agency (NGA) and the National Aeronautics and Space Administration (NASA). Detailed information on the SRTM project, including a gallery of images and a user's forum, can be accessed through the NASA web page:

<http://www2.jpl.nasa.gov/srtm/>

The data were processed at the Jet Propulsion Laboratory. They are distributed through the United States Geological Survey's (USGS) National Map Viewer and Download Platform:

<https://www.usgs.gov/core-science-systems/ngp/tnm-delivery/>

Alternatively, the raw data files can be downloaded from

<http://dds.cr.usgs.gov/srtm/>

This directory contains zipped files of SRTM DEMs from various areas of the world, processed by the SRTM global processor and sampled at resolutions of 1 arc second (SRTM-1, 30 meter grid) and 3 arc seconds (SRTM-3, 90 m grid). As an example we download the 1.7 MB file *s01e036.hgt.zip* from

http://dds.cr.usgs.gov/srtm/version2_1/SRTM3/Africa/

containing SRTM-3 data for the Kenya Rift Valley in East Africa. All elevations are in meters referenced to the WGS84 EGM96 geoid, as documented at

<http://earth-info.nga.mil/GandG/wgs84/index.html>

The name of this file refers to the longitude and latitude of the lower-left (south-west) pixel of the tile, i.e. latitude one degree south and longitude 36° east. SRTM-3 data contain 1,201 lines and 1,201 samples, with similar numbers of overlapping rows and columns. After having downloaded *S01E036.hgt.zip*, we use the MATLAB `unzip` function to decompress this file

```
clear, close all, clc
unzip('S01E036.hgt.zip')
```

and save *S01E036.hgt* in our working directory. The digital elevation model is provided as 16-bit signed integer data in a simple binary raster. The bit order is *big-endian* (Motorola's standard) with the most significant bit first. The data are imported into the workspace using

```
fid = fopen('S01E036.hgt','r');
SRTM = fread(fid,[1201,inf],'int16','b');
fclose(fid);
```

This script opens the file *s01e036.hgt* for read only access using `fopen` and defines the file identifier `fid`, which is then used to read the binaries from the file (using `fread`) and to write them into the matrix `SRTM`. Function `fclose` closes the file defined by `fid`. The matrix first needs to be transposed and flipped vertically.

```
SRTM = SRTM'; SRTM = flipud(SRTM);
```

The `-32768` flag for data voids can be replaced by `NaN`, which is the MATLAB representation for *Not-a-Number*.

```
SRTM(SRTM == -32768) = NaN;
```

The SRTM data contain numerous gaps that might cause spurious effects during statistical analysis or when displaying the digital elevation model in a graph. A popular way to eliminate gaps in digital elevation models is by filling gaps with the arithmetic means of adjacent elements. We use the function `nanmean` since it treats `NaNs` as missing values and returns the mean of the remaining elements that are not `NaNs`. The following double `for` loop averages `SRTM(i-1:i+1,j-1:j+1)` arrays, i.e. averages over three-by-three element wide areas of the digital elevation model.

```
for i = 2 : 1200
    for j = 2 : 1200
        if isnan(SRTM(i,j)) == 1
            SRTM(i,j) = nanmean(nanmean(SRTM(i-1:i+1,j-1:j+1)));
        end
    end
end
clear i j
```

If there are still `NaNs` in the data set causing errors when importing the data set into a *Virtual Reality Modeling Language* (VRML) client (Sect. 7.6), the double `for` loop can be run a second time. Finally, we check whether the data are now correctly stored in the workspace by printing the minimum and maximum elevations of the area.

```
max(SRTM(:))
min(SRTM(:))
```

In our example the maximum elevation of the area is 3,992 m above sea level and the minimum is 1,504 m. A coordinate system can be defined using the information that the lower-left corner is *s01e036*. The resolution is 3 arc seconds, corresponding to 1/1,200 of a degree.


```
[LON,LAT] = meshgrid(36:1/1200:37,-1:1/1200:0);
```

A shaded grayscale map can be generated from the elevation data using the function `surf1`. This function displays a shaded surface with simulated lighting.

```
surf1(LON,LAT,SRTM)
shading interp
colormap gray
view(0,90)
```

This script opens a new Figure Window and generates the shaded-relief map using interpolated shading, as well as a gray colormap, displayed in an overhead view. Since SRTM data contain a large amount of noise, we first smooth the data using an arbitrary 9-by-9 pixel moving average filter. Contrary to the discussion of running means in Sect. 6.8, the side effects of the running means are not important at this point. The new matrix is then stored in the matrix `SRTM_FILTERED`.

```
B = 1/81 * ones(9,9);
SRTM_FILTERED = filter2(B,SRTM);
```

The corresponding shaded-relief map is generated by

```
surf1(LON,LAT,SRTM_FILTERED)
shading interp
colormap gray
view(0,90)
```

After having generated the shaded-relief map (Fig. 7.4), the plot must be exported to a graphics file. For instance, the figure may be written into a JPEG format with a 70% quality level and 300 dpi resolution.

```
print -djpeg70 -r300 srtmimage
```

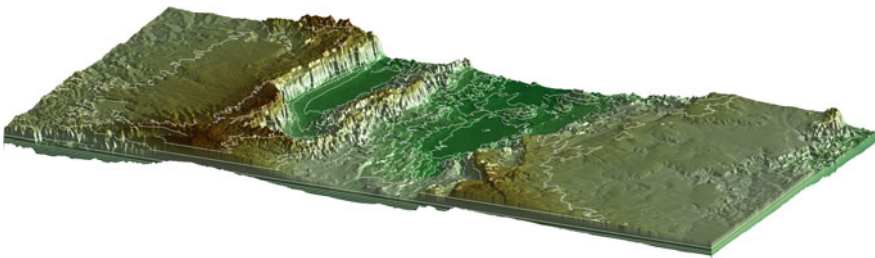


Fig. 7.4 Display from the filtered SRTM elevation data set. The function `surf1` has been used to generate a shaded-relief map with simulated lighting, `contour3` to create white elevation contours and the `demcmap` colormap from the Mapping Toolbox (data from Farr and Kobrick 2000; Farr et al. 2008).

The new file *srtmimage.jpg* has a size of 320 KB; the decompressed image has a size of 16.5 MB.

Neighboring SRTM tiles overlap one line or column, which we must delete when merging SRTM data sets. We use the Baringo basin in Kenya as an example, which is covered by two SRTM tiles. We first load the two SRTM data sets from files *N00E036.hgt* and *N00E035.hgt*, as described previously:

```
clear

fid = fopen('N00E035.hgt','r');
SRTM1 = fread(fid,[1201,inf],'int16','b');
fclose(fid);
fid = fopen('N00E036.hgt','r');
SRTM2 = fread(fid,[1201,inf],'int16','b');
fclose(fid);
```

We next transpose and flip the data sets up and down, before replacing the 16-bit signed integer no-data identifier -32768 with the MATLAB no-data identifier NaN.

```
SRTM1 = SRTM1'; SRTM1 = flipud(SRTM1);
SRTM2 = SRTM2'; SRTM2 = flipud(SRTM2);
SRTM1(SRTM1 == -32768) = NaN;
SRTM2(SRTM2 == -32768) = NaN;
```

We eliminate the overlapping (identical) column in the first tile SRTM1 and concatenate it horizontally with the second tile SRTM2 using *horzcat*.

```
SRTM1(:,1201) = [];
SRTM = horzcat(SRTM1,SRTM2);
clear SRTM1 SRTM2
```

We fill any gaps that may occur with the arithmetic means of adjacent elements:

```
for i = 2 : 1200
    for j = 2 : 2400
        if isnan(SRTM(i,j)) == 1
            SRTM(i,j) = nanmean(nanmean(...
                SRTM(i-1:i+1,j-1:j+1)));
        end
    end
end
clear i j
```

and smooth the data using a running mean.

```
B = 1/25 * ones(5,5);
SRTM = filter2(B,SRTM);
```

We then define a coordinate grid and contour levels (See Sect. 7.7).

```
[LON,LAT] = meshgrid(35:1/1200:37,-1:1/1200:0);
v = [700 800 900 1000 1100 1200 ...
     1300 1500 2000 2500 3000];
```

The script below yields of a great display of the data. Please note the use of the function `daspect` to reduce the length of the z-axis.

```
figure1 = figure('Color',[1 1 1],...
    'Position',[50 50 1200 600]);
axes1 = axes('Visible','off',...
    'Units','centimeters',...
    'FontSize',8);
hold(axes1,'all');
surf1 = surf(LON,LAT,SRTM,...
    'SpecularExponent',20,...
    'FaceLighting','phong',...
    'FaceColor','interp',...
    'EdgeColor','none');
light1 = light('Parent',axes1,...
    'Style','local',...
    'Position',[145 70 900000]);
set(gca,'View',[25 20])
daspect([1 1 20000])
colormap(flipud(hsv))
```

The Mapping Toolbox includes the function `demcmap`, which creates and assigns a colormap appropriate for elevation data since it relates land and sea colors to hypsometry and bathymetry (Fig. 7.4).

```
figure1 = figure('Color',[1 1 1],...
    'Position',[50 50 1200 600]);
axes1 = axes('Visible','off',...
    'Units','centimeters',...
    'FontSize',8);
```

```

hold(axes1,'all');
surf1 = surf(LON,LAT,SRTM,...
    'SpecularExponent',20,...
    'FaceLighting','phong',...
    'FaceColor','interp',...
    'EdgeColor','none');
light1 = light('Parent',axes1,...
    'Style','local',...
    'Position',[145 70 90000]);
set(gca,'View',[25 20])
daspect([1 1 20000])
colormap(flipud(hsv))
contour3(LON,LAT,SRTM,v,...
    'Color','w')
demcmap(SRTM)

```

After having generated the shaded-relief map, the plot must be exported to a graphics file. For instance, the graphics may be written into a PNG format with 300 dpi resolution.

```
print -dpng -r300 srtmimage.png
```

The new file *srtmimage.png* has a size of 3.1 MB. This file can now be imported into another software package such as Adobe Photoshop.

7.6 Exporting 3D Graphics to Create Interactive Documents

Since the introduction of electronic devices with touch controls, interactive 3D graphics objects have become increasingly popular in multimedia electronic books (*ebooks*). The Simulink 3D Animation available from MathWorks provides the necessary tools to create and export 3D graphics objects for inclusion in documents such as multimedia ebooks, interactive webpages, and presentations (MathWorks 2020b). The VRML files are text files with the file extension *.wrl* for *world*, short for *Virtual Reality World*. Opening such a file in, for example, the MATLAB editor reveals that it contains vertices and edges for 3D polygons, together with parameters such as surface colors, textures, transparency and so on. The VRML files can be viewed using a browser plugin (of which there are many different types available online), or using 3D software such as the open-source *MeshLab* software.

<http://www.meshlab.net>

or the free *Blender* software available at

<http://www.blender.org>

These tools (as well as others) can be used to convert the VRML format into other 3D graphics object file formats such as the *Universal 3D .u3d* and *COLLADA .dae* formats. As an example the U3D format is the format required to place a 3D graphics object onto a PDF page with Adobe Acrobat. The DAE format is used to include 3D graphics objects in Apple iBooks Author files, for use with Apple iPads and other iOS devices with touch controls.

The digital terrain models created in previous sections are great examples of such interactive 3D objects. We use a modified script to import and display an SRTM data set from the previous section as an example. The data are imported into the workspace using

```
clear, close all, clc

fid = fopen('S01E036.hgt', 'r');
SRTM = fread(fid, [1201, inf], 'int16', 'b');
fclose(fid);
```

The matrix first needs to be transposed and flipped vertically.

```
SRTM = SRTM'; SRTM = flipud(SRTM);
```

The SRTM data contain numerous gaps that might cause spurious effects during statistical analysis or when displaying the digital elevation model in a graph. We can use the method described in the previous section to eliminate the gaps. In this case, however, we have to search the data set for additional possible NaNs, even after having searched for NaNs and replaced them with the means of the surrounding pixels, because the VRML clients cannot handle the character string *NaN* and therefore produces an error message when importing the file into software such as MeshLab. Since it is for display only we can also use a simplified method of replacing the gaps marked by -32768 with the mean of the entire DEM.

```
SRTM(SRTM == -32768) = mean(SRTM(:));
```

A coordinate system can then be defined using the information that the lower-left corner is *s01e036*. The resolution is 3 arc seconds, corresponding to 1/1,200 of a degree.

```
[LON, LAT] = meshgrid(36:1/1200:37, -1:1/1200:0);
```

We also need to reduce the size of the array by a factor of 10 because most VRML clients limit the number of vertices in a VRML file. You can run an experiment with a larger number of vertices and see whether it works with your VRML software.

```
LON = LON(1:10:end,1:10:end);
LAT = LAT(1:10:end,1:10:end);
SRTM = SRTM(1:10:end,1:10:end);
```

We also eliminate the edges of the data set, as these may cause problems when writing the VRML files.

```
LON = LON(2:end-1,2:end-1);
LAT = LAT(2:end-1,2:end-1);
SRTM = SRTM(2:end-1,2:end-1);
```

3D graphics can be generated from the elevation data using the function `trimesh` following Delaunay triangulation of the data set using `delaunay`.

```
tri = delaunay(LON,LAT);
trimesh(tri,LON,LAT,SRTM)
axis([35.5 37.5 -1.5 0.5 -500 4500]), axis off
```

Next, we determine the size of the array `SRTM` and reshape the 119-by-119 array `SRTM` to a `n`-by-1 array, where `n = xdim*ymdim = 14161`.

```
[xdim ydim] = size(SRTM);
SRTM = SRTM(:);
```

We then determine the range of the z -values and define the spacing in x and y directions.

```
zrange = range(SRTM);
xspace = 10;
yspace = 10;
```

We again use the colormap `demcmap` to display the `SRTM` data set.

```
cmap = demcmap(SRTM,256);
```

Alternatively, we can use any other colormap, even one that we have created ourselves, to display the digital terrain model. We then normalize the elevation data to the `[1,length(cmap)]` range.

```
cmap = cmap(round((SRTM-min(SRTM)) ...
    .*(size(cmap,1)-1)./zrange)+1,:);
```

The Simulink 3D Animation User's Guide (MathWorks [2020b](#)) contains a great introduction in its chapter *Interact with Virtual Reality Worlds*, which explains the following steps in greater detail than here. We first need to make sure that all virtual reality worlds are closed and deleted.

```
out = vrwho;
for i=1:length(out)
    while (get(out(i),'opencount')~=0)
        close(out(i));
    end
    delete(out(i));
end
```

We then create a new virtual reality world `myworld` using `vrworld`.

```
myworld = vrworld('');
open(myworld)
```

Our world then needs a shape and a grid. We create the shape `newShape` with the name `Landscape` and the grid `newGrid` using `vrnode`. The function `vrnode` creates a handle to either an existing or a new node.

```
shapeName = ['Landscape'];
newShape = vrnode(myworld,shapeName,'Shape');
newGrid = vrnode(newShape,'geometry','DEM','ElevationGrid');
```

We can access the contents of the variable `newShape` from the `vrworld` class in a similar manner to structure arrays, e.g. by typing

```
getfield(newShape.geometry)
```

which lists a number of the defining parameters of `newShape` such as, for example, the color. The nodes of the world can be accessed using one of the two commands

```
nodes(myworld)
mynodes = get(myworld,'Nodes')
```

We can access the fields of the node `Landscape` using one of the two commands

```
fields(myworld.Landscape)
fields(mynodes.DEM)
```

and the DEM using

```
fields(myworld(1))
fields(mynodes(2))
```

both with a detailed output of all the parameter settings of the nodes. We can also list the fields in `newShape` and `newGrid` using

```
fields(newShape)
fields(newGrid)
```

and modify the values of the various parameters of `newShape` and `newGrid`. As an example we can reduce the height values to 20% of the original SRTM values in order to change the proportions of the 3D graphics.

```
setfield(newGrid, ...
    'xDimension',xdim,...
    'zDimension',ydim,...
    'xSpacing',xspace,...
    'zSpacing',yspace,...
    'height',0.2*SRTM);
```

We then use the same colormap `cmap` that was created previously (before using `demcmap`), based on elevation

```
GridColor = vrnnode(newGrid,...
    'color','TerrainColor',...
    'Color');
GridColor.color = cmap;
getfield(newGrid,'color')
```

and save the world `myworld` created with Simulink 3D Animation to a new VRML file `srtm_2.wrl`.

```
save(myworld,'srtm.wrl')
close(myworld)
delete(myworld)
```

We then close and delete the world `myworld`. Importing the resulting VRML file into a VRML client such as MeshLab or Blender reveals the terrain model does indeed have colors, as defined by `demcmap`. We can use the file in the VRML format

to create an interactive document. However, the software to create such interactive documents, such as Adobe Acrobat, may require different file formats; these can easily be converted with the MeshLab or Blender software.

7.7 Gridding and Contouring

The previous data sets were all stored in evenly-spaced two-dimensional arrays. Most data in earth sciences are, however, obtained from irregular sampling patterns. The data are therefore unevenly-spaced and need to be interpolated in order to allow a smooth and continuous surface to be computed from our measurements in the field. *Surface estimation* is typically carried out in two major steps (Fig. 7.5). Firstly, the number of *control points* needs to be selected, and secondly, the value of the variable of interest needs to be estimated for the *grid points*.

Control points are the unevenly-spaced field measurements, such as the thicknesses of sandstone units at different outcrops or the concentrations of a chemical tracer in water wells. The data are generally represented as xyz triplets, where x and y are spatial coordinates and z is the variable of interest. In such cases most gridding methods require continuous and unique data. However, spatial variables in earth sciences are often discontinuous and not spatially unique: for example, the sandstone unit may be faulted or folded. Furthermore, gridding requires spatial autocorrelation, i.e. the neighboring data points should be correlated with each other through a specific relationship. There is no point in making a surface estimation if the z variables are random and have no autocorrelation. Having selected the control points, a number of different methods are available for calculating the z -values at the evenly-spaced grid points.

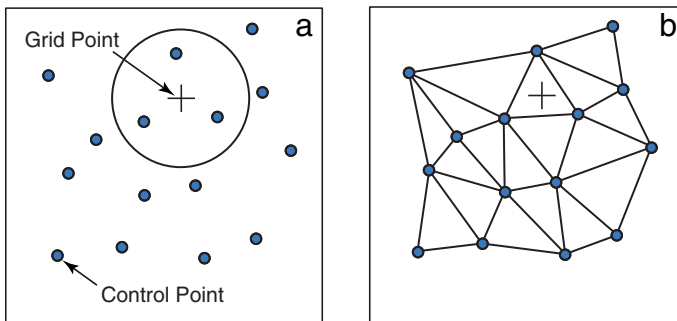


Fig. 7.5 Methods for selecting the control points to use for estimating the values at grid points. **a** Construction of a circle around the grid point (plus sign) with a radius defined by spatial autocorrelation of the z -values at the control points (small circles). **b** Triangulation: the control points are selected from the vertices of the triangle surrounding the grid point, with the option of also including the vertices of the adjoining triangles. Modified from Swan and Sandilands (1995).

Various techniques exist for selecting the control points. Most methods make arbitrary assumptions on the autocorrelation of the z -variable. The *nearest-neighbor criterion* includes all control points within a circular neighborhood of the grid point, where the radius of the circle is specified by the user (Fig. 7.5a). Since the degree of spatial autocorrelation is likely to decrease with increasing distance from the grid point, considering too many distant control points is likely to lead to erroneous results when computing values for the grid points. On the other hand, using radii that are too small may limit the number of control points used in calculating the grid point values to a very small number, resulting in a noisy estimate of the modeled surface.

It is perhaps due to these difficulties that *triangulation* is often used as an alternative method for selecting the control points (Fig. 7.5b). In this technique, all control points are connected in a triangular network. Every grid point is located within the triangular area formed by three control points. The z -value of the grid point is computed from the z -values of the three grid points. A modification of this form of gridding also uses the three points at the apices of the three adjoining triangles. The *Delaunay triangulation* method uses a triangular net in which the acuteness of the triangles is minimized, i.e. the triangles are as close as possible to equilateral.

Kriging, introduced in Sect. 7.11, is an alternative approach to select control points. It is often regarded as the ultimate method of gridding. Some people even use the term *geostatistics* synonymously with kriging. Kriging is a method for quantifying the spatial autocorrelation and hence the circle's dimension. More sophisticated versions of kriging use an elliptical area instead of a circle.

As mentioned above, the second step in surface estimation is the actual computation of the z -values for the grid points. The *arithmetic mean* of the measured z -values at the control points

$$\bar{z} = \frac{1}{N} \sum_{i=1}^N z_i$$

provides the easiest way of computing the values at the grid points. This is a particularly useful method if there are only a limited number of control points. If the study area is well-covered by control points and the distance between these points is highly variable, the z -values of the grid points should be computed using a *weighted mean*. This involves weighting the z -values at the control points by the inverse of the distance d_i from the grid points.

$$\bar{z} = \frac{\sum_{i=1}^N (z_i/d_i)}{\sum_{i=1}^N (1/d_i)}$$

Depending on the spatial scaling relationship of the variable z , the inverse of the square root of the distance may be used to weight the z -values, rather than simply the inverse of distance. The fitting of 3D *splines* to the control points offers another method for computing the grid point values that is commonly used in the earth sciences. Most routines used in surface estimation involve *cubic polynomial splines*, i.e. a third-degree 3D polynomial is fitted to at least six adjacent control points. The final surface is a composite comprising different portions of these splines.

MATLAB has, from the start, provided a biharmonic spline interpolation method, which was developed by Sandwell (1987). This gridding method is particularly well suited for producing smooth surfaces from noisy data sets with unevenly-distributed control points. As an example we use synthetic xyz data representing the vertical distance between the surface of an imaginary stratigraphic horizon that has been displaced by a normal fault, and a reference surface. The foot wall of the fault shows roughly horizontal strata, whereas the hanging wall is characterized by the development of two large sedimentary basins. The xyz data are irregularly distributed and so need to be interpolated onto a regular grid. The xyz data are stored as a three-column table in a file named *normalfault.txt*.

```
4.3229698e+02  7.4641694e+01  9.7283620e-01
4.4610209e+02  7.2198697e+01  6.0655065e-01
4.5190255e+02  7.8713355e+01  1.4741054e+00
4.6617169e+02  8.7182410e+01  2.2842172e+00
4.6524362e+02  9.7361564e+01  1.1295175e-01
4.5526682e+02  1.1454397e+02  1.9007110e+00
4.2930233e+02  7.3175896e+01  3.3647807e+00
(cont'd)
```

The first and second column contain the coordinates x (between 420 and 470 of an arbitrary spatial coordinate system) and y (between 70 and 120), while the third column contains the vertical z -values. The data are loaded using

```
clear, close all, clc
data = load('normalfault.txt');
```

Initially, we wish to create an overview plot of the spatial distribution of the control points. In order to label the points in the plot, numerical z -values of the third column are converted into character string representations with a maximum of two digits.

```
labels = num2str(data(:,3),2);
```

The 2D plot of our data is generated in two steps. Firstly, the data are displayed as empty circles using the `plot` command. Secondly, the data are labeled using the function `text(x,y,'string')`, which adds text contained in `string` to the xy locations. The value 1 is added to all x coordinates in order to produce a small offset between the circles and the text.

```
plot(data(:,1),data(:,2),'o'), hold on
text(data(:,1)+1,data(:,2),labels), hold off
```

This plot helps us to define the axis limits for gridding and contouring: `xlim=[420 470]` and `ylim=[70 120]`. The function `meshgrid` transforms the domain specified by vectors `x` and `y` into arrays `XI` and `YI`. The rows of the output array `XI` are copies of the vector `x` and the columns of the output array `YI` are copies of the vector `y`. We choose 1.0 as the grid interval.

```
x = 420:1:470; y = 70:1:120;
[XI,YI] = meshgrid(x,y);
```

The biharmonic spline interpolation is used to interpolate the irregular-spaced data at the grid points specified by `XI` and `YI`.

```
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI, 'v4');
```

The option `v4` selects the biharmonic spline interpolation, which was the sole gridding algorithm available until MATLAB4 was replaced by MATLAB5. MATLAB provides various tools with which to display the results. The simplest way to display the gridding results is as a contour plot using `contour`. By default, the number of contour levels and the values of the contour levels are chosen automatically. The choice of the contour levels depends on the minimum and maximum values of z .

```
contour(XI,YI,ZI)
```

Alternatively, the number of contours can be chosen manually, e.g. ten contour levels.

```
contour(XI,YI,ZI,10)
```

Contouring can also be performed at values specified in a vector `v`. Since the maximum and minimum values of z are

```
min(data(:,3))
ans =
-27.4357
```

```
max(data(:,3))
ans =
    21.3018
```

we choose

```
v = -40 : 10 : 20;
```

The command

```
[c,h] = contour(XI,YI,ZI,v);
```

yields contour matrix `c` and a handle `h` that can be used as input to the function `clabel`, which labels contours automatically.

```
clabel(c,h)
```

Alternatively, the plot can be labeled manually by selecting the `manual` option in the function `clabel`. This function places labels onto locations that have been selected with the mouse. Labeling is terminated by pressing the return key.

```
[c,h] = contour(XI,YI,ZI,v);
clabel(c,h,'manual')
```

Filled contours are an alternative to the empty contours used above. This function is used together with `colorbar`, which displays a legend for the plot. In addition, we can plot the locations (small circles) and z -values (contour labels) of the true data points (Fig. 7.6).

```
contourf(XI,YI,ZI,v), colorbar, hold on
plot(data(:,1),data(:,2),'ko')
text(data(:,1)+1,data(:,2),labels), hold off
```

A pseudocolor plot is generated using the function `pcolor`. Black contours are also added at the same levels as in the above example.

```
pcolor(XI,YI,ZI), shading flat, hold on
contour(XI,YI,ZI,v,'k'), hold off
```

The third dimension is added to the plot using the `mesh` command. We can also use this example to introduce the function `view(az,el)` to specify the direction of viewing, where `az` is the azimuth or horizontal rotation and `el` is the elevation (both in degrees). The values `az=-37.5` and `el=30` define the default view for all 3D plots,

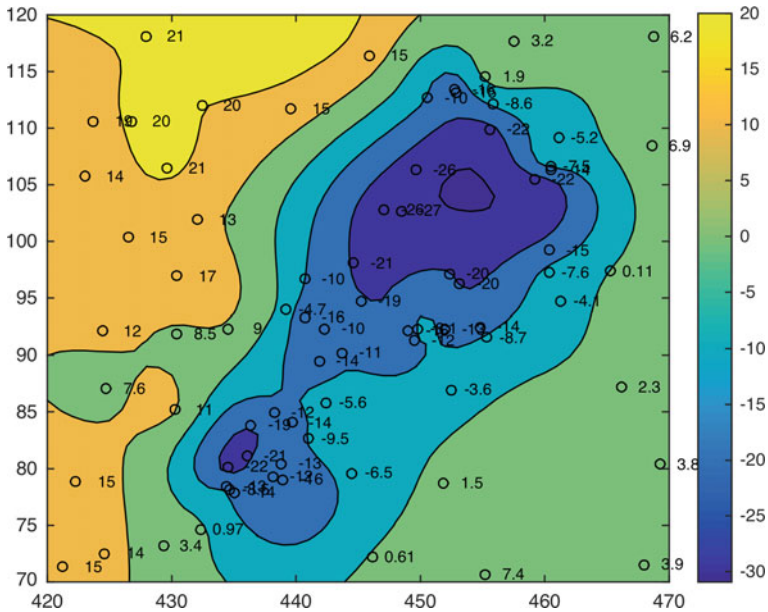


Fig. 7.6 Contour plot with the locations (small circles) and z -values (contour labels) of the true data points.

```
mesh(XI,YI,ZI), view(-37.5,30)
```

whereas $az = 0$ and $el = 90$ is directly overhead and the default 2D view:

```
mesh(XI,YI,ZI), view(0,90)
```

The function `mesh` provides one of many methods available in MATLAB for 3D presentation, another commonly used function being `surf`. The figure can be rotated by selecting the *Rotate 3D* option on the *Edit Tools* menu. We also introduce the function `colormap`, which uses predefined color look-up tables for 3D graphics. Typing `help graph3d` lists a number of built-in colormaps, although colormaps can also be arbitrarily modified and generated by the user. As an example we use the colormap *hot*, which is a *black-red-yellow-white* colormap.

```
surf(XI,YI,ZI), colormap('hot'), colorbar
```

Using *Rotate 3D* only rotates the 3D plot, not the colorbar. The function `surf` combines both a surface and a 2D contour plot in one graph.

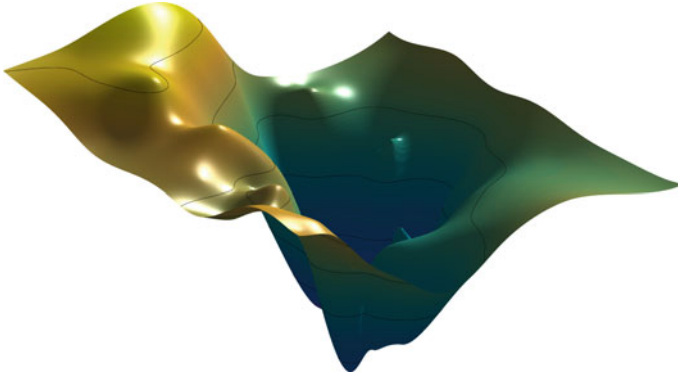


Fig. 7.7 Three-dimensional colored surface with interpolated shading and simulated lighting. The axis labeling, ticks and background are turned off. The plot also contains 3D contours, in black.

```
surf(XI,YI,ZI)
```

The function `surf1` can be used to illustrate an advanced application for 3D visualization, generating a 3D colored surface with interpolated shading and lighting. The axis labeling, ticks, and background can be turned off by typing `axis off`. In addition, black 3D contours can be added to the surface, as above. The grid resolution is increased prior to data plotting in order to obtain smooth surfaces (Fig. 7.7).

```
[XI,YI] = meshgrid(420:0.25:470,70:0.25:120);
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI,'v4');

surf(XI,YI,ZI), shading interp, light, axis off, hold on
contour3(XI,YI,ZI,v,'k'), hold off
```

The biharmonic spline interpolation described in this section provides a solution to most gridding problems. It was therefore, for some time, the only gridding method that came with MATLAB. However, different applications in earth sciences require different methods of interpolation, although they all have their problems. The next section compares biharmonic spline interpolation with other gridding methods and summarizes their strengths and weaknesses.

7.8 Comparison of Methods and Potential Artifacts

The first example in this section illustrates the use of the *bilinear interpolation* technique for gridding irregular-spaced data. Bilinear interpolation is an extension of the one-dimensional technique of linear interpolation introduced in Sect. 5.5. In

the two-dimensional case, linear interpolation is first performed in one direction, and then in the other direction. The bilinear method would appear to be one of the simplest interpolation techniques, which might intuitively not be expected to produce serious artifacts or distortions in the data. The opposite is true, however, as this method has a number of disadvantages and other methods are therefore preferred in many applications.

The sample data used in the previous section can again be loaded to study the effects of a bilinear interpolation.

```
clear, close all, clc

data = load('normalfault.txt');
labels = num2str(data(:,3),2);
```

We now choose the option linear while using the function `griddata` to interpolate the data.

```
[XI,YI] = meshgrid(420:0.25:470,70:0.25:120);
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI,'linear');
```

The results are plotted as contours. The plot also includes the locations of the control points.

```
v = -40 : 10 : 20;
contourf(XI,YI,ZI,v), colorbar, hold on
plot(data(:,1),data(:,2),'o'), hold off
```

The new surface is restricted to the area that contains control points: by default, bilinear interpolation does not extrapolate beyond this region. Furthermore, the contours are rather angular compared to the smooth shape of the contours from the biharmonic spline interpolation. The most important character of the bilinear gridding technique, however, is illustrated by a projection of the data in a vertical plane.

```
plot(XI,ZI,'k'), hold on
plot(data(:,1),data(:,3),'ro')
text(data(:,1)+1,data(:,3),labels)
title('Linear Interpolation'), hold off
```

This plot shows the projection of the estimated surface (vertical lines) and the labeled control points. The z -values at the grid points never exceed the z -values of the control points. As with the linear interpolation of time series (Sect. 5.5), bilinear interpolation causes significant smoothing of the data and a reduction in high-frequency variations.

Biharmonic spline interpolations are, in many ways, the other extreme. They are often used for extremely irregular-spaced and noisy data.

```
[XI,YI] = meshgrid(420:0.25:470,70:0.25:120);
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI,'v4');

v = -40 : 10 : 20;
contourf(XI,YI,ZI,v), colorbar, hold on
plot(data(:,1),data(:,2),'o'), hold off
```

The contours suggest an extremely smooth surface. This solution is very useful in many applications but the method also produces a number of artifacts. As we can see from the next plot, the estimated values at the grid points are often beyond the range of the measured z -values.

```
plot(XI,ZI,'k'), hold on
plot(data(:,1),data(:,3),'o')
text(data(:,1)+1,data(:,3),labels)
title('Biharmonic Spline Interpolation'), hold off
```

This can sometimes be appropriate and does not smooth the data in the way that bilinear gridding does. However, introducing very close control points with different z -values can cause serious artifacts. As an example we introduce one reference point with a z -value of +5 close to a reference point with a negative z -value of around -26.

```
data(79,:) = [450 105 5];
labels = num2str(data(:,3),2);
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI,'v4');

v = -40 : 10 : 20;
contourf(XI,YI,ZI,v), colorbar, hold on
plot(data(:,1),data(:,2),'ko')
text(data(:,1)+1,data(:,2),labels), hold off
```

The extreme gradient at the location (450, 105) results in a paired *low* and *high* (Fig. 7.8). In such cases, it is recommended that one of the two control points be deleted and the z -value of the remaining control point be replaced by the arithmetic mean of both z -values.

Extrapolation beyond the area supported by control points is a common feature of spline interpolation (see also Sect. 5.5). Extreme local trends combined with large areas with no data often result in unrealistic estimates. To illustrate these edge effects we eliminate all control points in the upper-left corner.

```
[i,j] = find(data(:,1)<435 & data(:,2)>105);
data(i,:) = [];

labels = num2str(data(:,3),2);
```

```
plot(data(:,1),data(:,2),'ko'), hold on
text(data(:,1)+1,data(:,2),labels), hold off
```

We again use the biharmonic spline interpolation technique.

```
[XI,YI] = meshgrid(420:0.25:470,70:0.25:120);
ZI = griddata(data(:,1),data(:,2),data(:,3),XI,YI,'v4');

v = -40 : 10 : 40;
contourf(XI,YI,ZI,v)
caxis([-40 40])
colorbar
hold on
plot(data(:,1),data(:,2),'ko')
text(data(:,1)+1,data(:,2),labels)
hold off
```

As can be seen from the plot, this method extrapolates the gradients beyond the area with control points, up to the edge of the map (Fig. 7.9). Such an effect is particular undesirable when gridding closed data, such as percentages, or data that have only positive values. In such cases it is recommended that the estimated z -values be replaced by NaN. For instance, we delete the areas with z -values larger than 20, which are regarded as unrealistic values. The resulting plot now contains a sector with no data.

```
ZID = ZI;
ZID(find(ZID > 20)) = NaN;

contourf(XI,YI,ZID,v)
caxis([-40 40])
colorbar
hold on
plot(data(:,1),data(:,2),'ko')
text(data(:,1)+1,data(:,2),labels)
hold off
```

Alternatively, we can eliminate a rectangular area with no data.

```
ZID = ZI;
ZID(131:201,1:71) = NaN;

contourf(XI,YI,ZID,v)
caxis([-40 40])
colorbar
hold on
plot(data(:,1),data(:,2),'ko')
text(data(:,1)+1,data(:,2),labels)
hold off
```

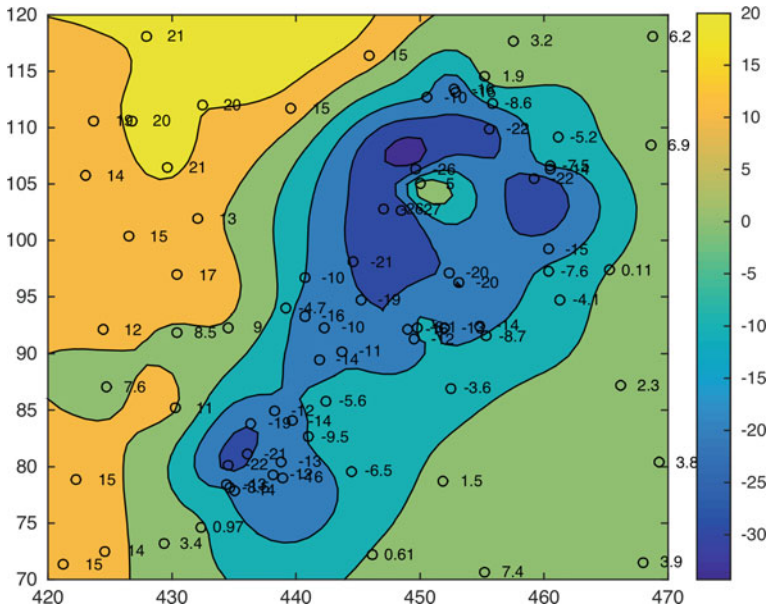


Fig. 7.8 Contour plot of a data set gridded using a biharmonic spline interpolation. At the location (450, 105), very close control points with different z -values have been introduced. Interpolation causes a paired low and high, which is a common artifact in spline interpolation of noisy data.

In some examples the area with no control points is simply concealed by placing a legend over this part of the map.

Other interpolation methods available with `griddata` are the cubic (`cubic`), natural neighbor (`natural`), and nearest neighbor (`nearest`) interpolation methods. The cubic option interpolates the data in two dimensions using a spline, i.e. a third-degree 3D polynomial is fitted to at least six adjacent control points, resulting in a surface (and its first derivative) that is continuous. The resulting surface is much smoother than those calculated using linear (`linear`), natural (`natural`), or nearest neighbor (`nearest`) techniques but not as smooth as that resulting from a biharmonic spline interpolation (`v4`). For this reason the typical artifacts of splines do not occur to the same extent. We can compare all of these methods in the next example. We first clear the workspace and reload the data from *normalfault.txt*.

```
clear

data = load('normalfault.txt');
data(79,:) = [450 105 5];
labels = num2str(data(:,3),2);
```

We then create titles for the results from the different interpolation methods.

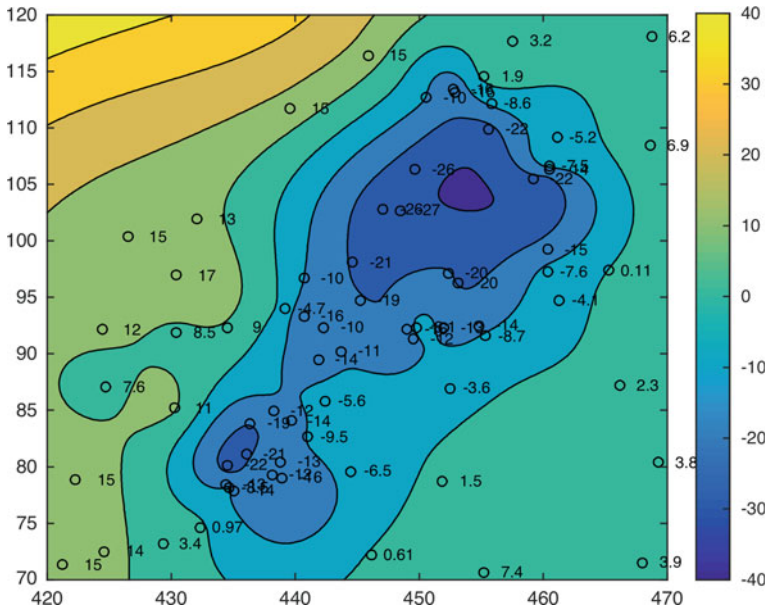


Fig. 7.9 Contour plot of a data set gridded using a biharmonic spline interpolation. No control points are available in the upper left corner. The spline interpolation then extrapolates beyond the area with control points using the gradients at the map edges, resulting in unrealistic z estimates at the grid points.

```
titles = ['linear '; 'nearest'; 'natural'; 'cubic '; 'biharmo'];
```

Since we store the titles in a single character array, we use spaces to expand the names of the methods so that they are all the same length. We again define the axis limits for gridding and contouring: `xlim = [420 470]` and `ylim = [70 120]`. The function `meshgrid` transforms the domain specified by vectors `x` and `y` into arrays `XI` and `YI`. The rows of the output array `XI` are copies of the vector `x` and the columns of the output array `YI` are copies of the vector `y`. We choose 1.0 as the grid interval.

```
x = 420:1:470; y = 70:1:120;
[XI,YI] = meshgrid(x,y);
```

We then use `griddata` with all available options and store the results in a three-dimensional array `ZI`.

```

ZI(:, :, 1) = griddata(data(:, 1), data(:, 2), data(:, 3), ...
    XI, YI, 'linear');
ZI(:, :, 2) = griddata(data(:, 1), data(:, 2), data(:, 3), ...
    XI, YI, 'nearest');
ZI(:, :, 3) = griddata(data(:, 1), data(:, 2), data(:, 3), ...
    XI, YI, 'natural');
ZI(:, :, 4) = griddata(data(:, 1), data(:, 2), data(:, 3), ...
    XI, YI, 'cubic');
ZI(:, :, 5) = griddata(data(:, 1), data(:, 2), data(:, 3), ...
    XI, YI, 'v4');

```

We compare the results in five different graphics in separate Figure Windows, slightly offset on the computer display, using `figure`. The data are displayed as filled contours at values specified in a vector `v`.

```

v = -40 : 10 : 20;

for i = 1 : 5
    figure('Position', [50 i*100-50 500 300])
    contourf(XI, YI, ZI(:, :, i), v), colorbar, hold on
    plot(data(:, 1), data(:, 2), 'ko')
    text(data(:, 1)+1, data(:, 2), labels), hold off
    title(titles(i, :))
end

```

In MATLAB R2013a release the linear (`linear`), the natural (`natural`) and the nearest neighbor (`nearest`) techniques were moved to the new function `scatteredInterpolant`, while the biharmonic (`v4`) and cubic spline (`cubic`) options remain in the `griddata` function. In fact the code of `griddata` invokes the `scatteredInterpolant` function instead of the original codes of `linear`, `natural`, or `nearest`. The function works in a very similar manner to `griddata`, as we can easily explore by typing

```

FLIN = scatteredInterpolant(data(:, 1), data(:, 2), data(:, 3), ...
    'linear', 'linear');
FNEA = scatteredInterpolant(data(:, 1), data(:, 2), data(:, 3), ...
    'nearest', 'nearest');
FNAT = scatteredInterpolant(data(:, 1), data(:, 2), data(:, 3), ...
    'natural', 'none');

ZI(:, :, 6) = FLIN(XI, YI);
ZI(:, :, 7) = FNEA(XI, YI);
ZI(:, :, 8) = FNAT(XI, YI);

titles(6:8, :) = ['scatlin'; 'scatnea'; 'scatnat'];

```

```

for i = 6:8
    figure('Position',[350 (i-5)*100-50 500 300])
    contourf(XI,YI,ZI(:, :, i),v), colorbar, hold on
    plot(data(:,1),data(:,2),'ko')
    text(data(:,1)+1,data(:,2),labels), hold off
    title(titles(i,:))
end

```

Another very useful MATLAB gridding method is *splines with tension* by Wessel and Bercovici (1998), available for download from the author's webpage:

<http://www.soest.hawaii.edu/wessel/tspline/>

The *tsplines* use biharmonic splines in tension t , where the parameter t can vary between 0 and 1. A value of $t = 0$ corresponds to a standard cubic spline interpolation. Increasing t reduces undesirable oscillations between data points, e.g. the paired *lows* and *highs* observed in one of the previous examples. The limiting situation $t \rightarrow 1$ corresponds to linear interpolation.

7.9 Statistics of Point Distributions

This section is about the statistical distribution of objects within an area, which may help explain the relationship between these objects and properties of the area (Fig. 7.10). For instance, the spatial concentration of hand-axes in an archaeological site may suggest that a larger population of hominins lived in that part of the area, the clustered distribution of fossils may document environmental conditions that were favorable to those particular organisms, and the alignment of volcanoes can help map tectonic structures concealed beneath the Earth's surface.

Various methods for the statistical analysis of point distributions are introduced below. We first consider a test for a uniform spatial distribution of objects, followed by a test for a random spatial distribution and finally, a simple test for a clustered distribution of objects.

Test for Uniform Distribution

In order to illustrate the test for a uniform distribution we first need to compute some synthetic data (Fig. 7.10b). The function `rand` computes uniformly-distributed pseudo-random numbers drawn from a uniform distribution within the interval [0, 1]. We compute *xy* data using `rand` and multiply the data by ten to obtain data within the interval [0, 10].

```

clear, close all, clc

rng(0)
data = 10 * rand(100,2);

```

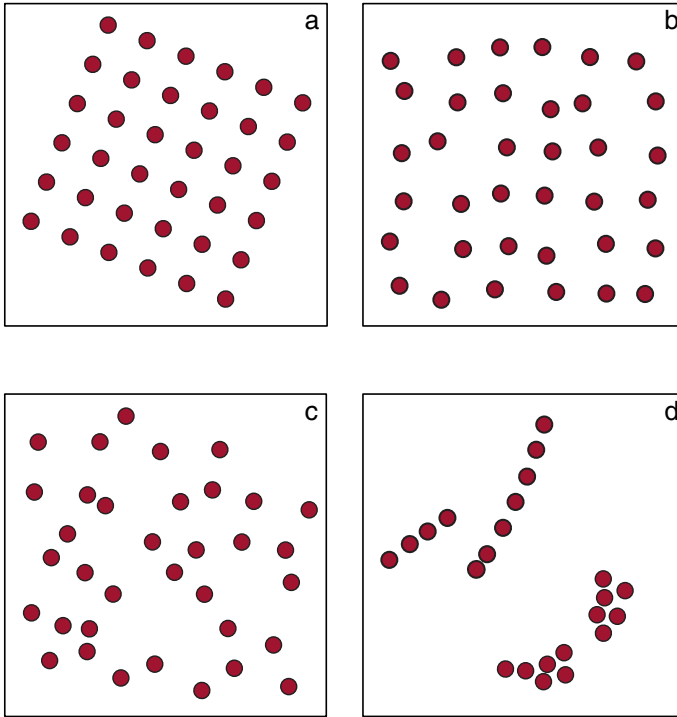


Fig. 7.10 Spatial distribution of objects. **a** Regular point distribution, **b** uniform point distribution, **c** random point distribution, and **d** anisotropic and clustered point distribution.

We can use the χ^2 -test introduced in Sect. 3.8 to test the null hypothesis that the data have a uniform distribution. The xy data are now organized into 25 classes that are square subareas with dimensions of 2-by-2. This definition of the classes ignores the rule of thumb that the number of classes should be close to the square root of the number of observations (see Sect. 3.3). Our choice of classes, however, does not result in any empty classes, which should be avoided when applying the χ^2 -test. Furthermore, 25 classes produce integer values for the expected number of observations that are easier to work with. We display the data as blue circles in a plot of y versus x . The rectangular areas are outlined with red lines (Fig. 7.11).

```

plot(data(:,1),data(:,2),'o')
hold on
x = 0:10; y = ones(size(x));
for i = 1:4, plot(x,2*i*y,'r-'), end
for i = 1:4, plot(2*i*y,x,'r-'), end
hold off

```

A three-dimensional version of a histogram `hist3` is used to display the spatial data, organized in classes (Fig. 7.12).

```

hist3(data,[5 5]), view(30,70)

```

As with the equivalent two-dimensional function, the function `hist3` can be used to compute the frequency distribution `n_obs` of the data.

```

n_obs = hist3(data,[5 5]);
n_obs = n_obs(:);

```

The theoretical frequencies for the different classes in a uniform distribution are all identical. The expected number of objects in each square area is the size of the total area $10 \cdot 10 = 100$ divided by the 25 subareas or classes, which comes to four. To compare the theoretical frequency distribution with the actual spatial distribution of objects, we generate a 5-by-5 array with an identical number of objects.

```

n_exp = 4 * ones(25,1);

```

The χ^2 -test explores the squared differences between the observed and expected frequencies (Sect. 3.9). The quantity χ^2 is defined as the sum of the squared differences divided by the expected frequencies.

```

chi2_data = sum((n_obs - n_exp).^2 ./n_exp)

chi2 =

    22.5000

```

The critical χ^2 can be calculated using `chi2inv`. The χ^2 -test requires the degrees of freedom Φ . In our example we test the hypothesis that the data are uniformly distributed, i.e. we estimate only one parameter (Sect. 3.5). The number of degrees of freedom is therefore $\Phi = 25 - (1+1) = 23$. We test the hypothesis at a $p = 95\%$ significance level. The function `chi2inv` computes the inverse of the χ^2 CDF with parameters specified by Φ for the corresponding probabilities in p .

```

chi2_theo = chi2inv(0.95,25-1-1)

ans =

    35.1725

```

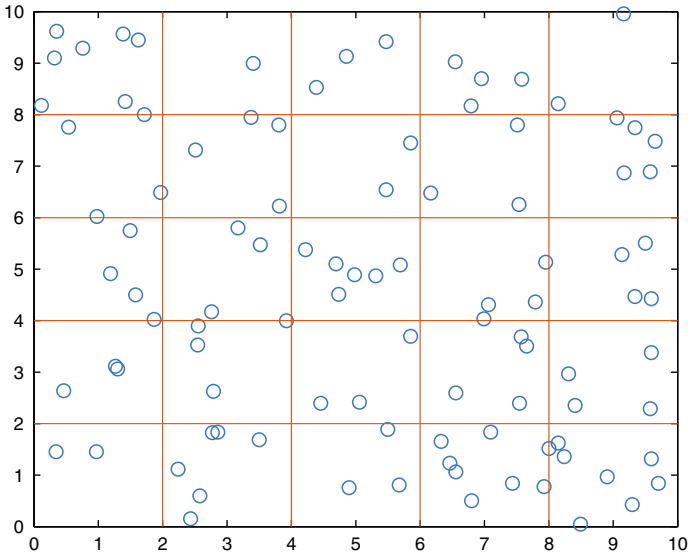



Fig. 7.11 Two-dimensional plot of a point distribution. The distribution of objects in the field is tested for uniform distribution using the χ^2 -test. The xy data are organized into 25 classes that are subareas with dimensions of 2-by-2.

Since the critical χ^2 of 35.1725 is well above the measured χ^2 of 22.5000, we cannot reject the null hypothesis and therefore conclude that our data follow a uniform distribution.

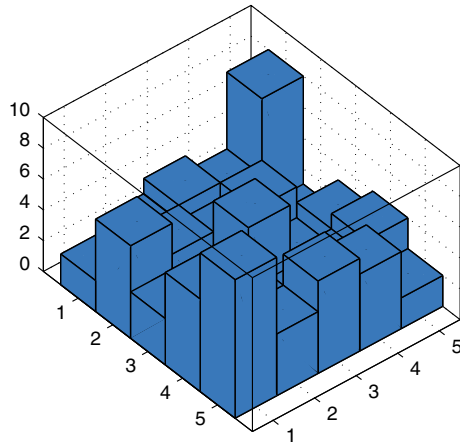
Test for Random Distribution

The following example illustrates the test for random distribution of objects within an area (Fig. 7.10c). We use the uniformly-distributed data generated in the previous example and display the point distribution.

```
clear

rng(5)
data = 10 * rand(100,2);
plot(data(:,1),data(:,2),'o')
hold on
x = 0:10; y = ones(size(x));
for i = 1:9, plot(x,i*y,'r-'), end
for i = 1:9, plot(i*y,x,'r-'), end
hold off
```

Fig. 7.12 Three-dimensional histogram displaying the numbers of objects for each subarea. The histogram was created using `hist3`.



We then generate the three-dimensional histogram and use the function `hist3` to count the objects per class. In contrast to the previous test, we now count the subareas containing a certain number of observations. The number of subareas is larger than would normally be used for the previous test. In our example we use 49 subareas or classes.

```
hist3(data,[7 7])
view(30,70)

counts = hist3(data,[7 7]);
counts = counts(:);
```

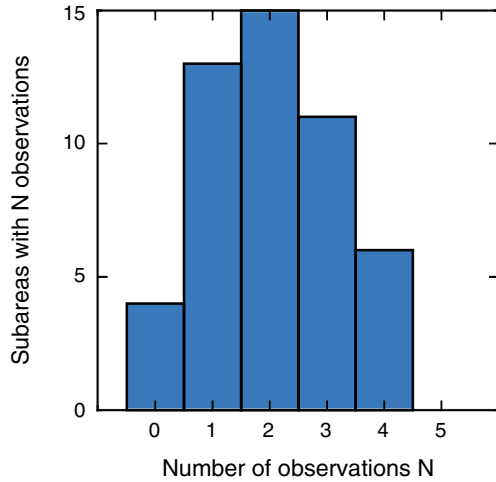
The frequency distribution of those subareas that contain a specific number of objects follows a Poisson distribution (Sect. 3.5) if the objects are randomly distributed. First, we compute a frequency distribution of the subareas containing N objects. In our example we count the subareas with 0, ..., 5 objects. We also display the histogram of the frequency distribution as a two-dimensional histogram using `histogram`, after having first calculated the bin edges E from the bin centers N (Sect. 3.3) (Fig. 7.13).

```
N = 0 : 5;
E = -0.5 : 1 : 5.5;

h = histogram(counts,E);
title('Histogram')
xlabel('Number of observations N')
ylabel('Subareas with N observations')

v = h.BinWidth * 0.5 + h.BinEdges(1:end-1);
n_obs = h.Values;
```

Fig. 7.13 Frequency distribution of subareas with N objects. In our example the subareas with 0, ..., 5 objects are counted. The histogram of the frequency distribution is displayed as a two-dimensional histogram, using `histogram`.



The midpoints of the histogram intervals v correspond to the $N = 0, \dots, 5$ objects contained in the subareas. The expected number of subareas E_j with a certain number of objects j can be computed using

$$E_j = T e^{-n/T} \frac{(n/T)^j}{j!}$$

where n is the total number of objects and T is the number of subareas. For $j = 0$, $j!$ is taken to be 1. We compute the expected number of subareas, i.e. the theoretical frequency distribution `n_exp`, using the equation shown above,

```
for i = 1 : 6
    n_exp(i) = 49*exp(-100/49)*(100/49)^i/factorial(N(i));
end
n_exp = sum(n_obs)*n_exp/sum(n_exp);
```

and display both the empirical and theoretical frequency distributions in a single plot.

```
h1 = bar(v,n_obs);
hold on
h2 = bar(v,n_exp);
hold off

set(h1,'FaceColor','none','EdgeColor','r')
set(h2,'FaceColor','none','EdgeColor','b')
```

The χ^2 -test is again used to compare the empirical and theoretical distributions, i.e. to test the null hypothesis that the frequency distribution of the subareas that contain a

specific number of objects follows a Poisson distribution. The test is performed at a $p = 95\%$ significance level. Since the Poisson distribution is defined by only one parameter (Sect. 3.4), the number of degrees of freedom is $\Phi = 6 - (1 + 1) = 4$. The measured χ^2 of

```
chi2 = sum((n_obs - n_exp).^2 ./ n_exp)

chi2 =

    3.7615
```

is well below the critical χ^2 , which is

```
chi2inv(0.95,6-1-1)

ans =

    9.4877
```

We therefore cannot reject the null hypothesis and conclude that our data follow a Poisson distribution and the point distribution is random.

Test for Clustering

Point distributions in geosciences are often clustered (Fig. 7.10d). We use a *nearest-neighbor criterion* to test a spatial distribution for clustering. Davis (2002) has published an excellent summary of the nearest-neighbor analysis, summarizing the work of a number of other authors. Swan and Sandilands (1995) presented a simplified description of this analysis. The test for clustering computes the distances d_i separating all possible pairs of nearest points in the field. The *observed mean nearest-neighbor distance* is

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n d_i$$

where n is the total number of points or objects in the field. The arithmetic mean of all distances between possible pairs is related to the area covered by the map. This relationship is expressed by the *expected mean nearest-neighbor distance*, which is

$$\bar{\delta} = \frac{1}{2} \sqrt{A/n}$$

where A is the area covered by the map. Small values for this ratio then suggest significant clustering, whereas large values indicate regularity or uniformity. The test uses a Z statistic (Sect. 3.5), which is

$$Z = \frac{\bar{d} - \bar{\delta}}{s_e}$$

where se is the standard error of the mean nearest-neighbor distance, which is defined as

$$se = \frac{0.27136}{\sqrt{n^2/A}}$$

The null hypothesis *randomness* is tested against two alternative hypotheses, *clustering* and *uniformity or regularity* (Fig. 7.10). The Z statistic has critical values of 1.96 and -1.96 at a significance level of 95%. If $-1.96 < Z < +1.96$, we cannot reject the null hypothesis that the data are randomly distributed. If $Z < -1.96$, we reject the null hypothesis and accept the first alternative hypothesis of clustering. If $Z > +1.96$, we also reject the null hypothesis, but accept the second alternative hypothesis of uniformity or regularity.

As an example we again use the synthetic data analyzed in the previous examples.

```
clear
rng(5)
data = 10 * rand(100,2);
plot(data(:,1),data(:,2),'o')
```

We first compute the pairwise Euclidian distance between all pairs of observations using the function `pdist` (Sect. 9.5). The resulting distance matrix `distances` is then converted into a symmetric, square format, so that `distmatrix(i, j)` denotes the distance between `i` and `j` objects in the original data.

```
distances = pdist(data,'Euclidean');
distmatrix = squareform(distances);
```

The following `for` loop finds the nearest neighbors, stores the nearest-neighbor distances and computes the mean distance.

```
for i = 1 : 100
    distmatrix(i,i) = NaN;
    k = find(distmatrix(i,:) == min(distmatrix(i,:)));
    nearest(i) = distmatrix(i,k(1));
end
observednearest = mean(nearest)

observednearest =

    0.5078
```

In our example the mean nearest distance `observednearest` comes to 0.5078. We next calculate the area of the map. The expected mean nearest-neighbor distance is half the square root of the map area divided by the number of observations.

```
maparea = (max(data(:,1))-min(data(:,1))) ...
          *(max(data(:,2))-min(data(:,2)));
expectednearest = 0.5 * sqrt(maparea/length(data))

expectednearest =

    0.4875
```

In our example the expected mean nearest-neighbor distance `expectednearest` is 0.4875. Finally, we compute the standard error of the mean nearest-neighbor distance `se`

```
se = 0.26136/sqrt(length(data).^2/maparea)

se =

    0.0255
```

and the test statistic `Z`.

```
Z = (observednearest - expectednearest)/se

Z =

    0.7954
```

Since $-1.96 < Z < +1.96$, we cannot reject the null hypothesis and conclude that the data are randomly distributed, but not clustered.

7.10 Analysis of Digital Elevation Models (by R. Gebbers)

Digital elevation models (DEMs) and their derivatives (e.g. slope and aspect) can indicate surface processes such as lateral water flow, solar irradiation, or erosion. The simplest derivatives of a DEM are the slope and the aspect. The *slope* (or *gradient*) is a measure of the steepness, the incline or the grade of a surface measured in either percentages or degrees. The *aspect* (or *exposure*) refers to the direction in which a slope faces.

We use the SRTM data set introduced in Sect. 7.5 to illustrate the analysis of a digital elevation model for slope, aspect and other derivatives. The data are loaded by

```
clear, close all, clc

fid = fopen('S01E036.hgt','r');
SRTM = fread(fid,[1201,inf],'int16','b');
fclose(fid);
```

```
SRTM = SRTM';
SRTM = flipud(SRTM);
SRTM(SRTM== -32768) = NaN;
```

These data are elevation values in meters above sea level, sampled on a 3 arc second or 90 grid. The SRTM data contain small-scale spatial disturbances and noise that could cause problems when computing a drainage pattern. We therefore filter the data with a two-dimensional moving-average filter, using the function `filter2`. The filter calculates a spatial running mean of 3-by-3 elements. We use only the subset `SRTM(400:600, 650:850)` of the original data set, in order to reduce computation time. We also remove the data at the edges of the DEM to eliminate filter artifacts.

```
F = 1/9 * ones(3,3);
SRTM = filter2(F, SRTM(750:850,700:800));
SRTM = SRTM(2:99,2:99);
```

The DEM is displayed as a pseudocolor plot using `pcolor` and the colormap `demcmap` included in the Mapping Toolbox. The function `demcmap` creates and assigns a colormap appropriate for elevation data since it relates land and sea colors to hypsometry and bathymetry.

```
h = pcolor(SRTM);
demcmap(SRTM), colorbar
set(h,'LineStyle','none')
axis equal
title('Elevation [m]')
[r c] = size(SRTM);
axis([1 c 1 r])
set(gca,'TickDir','out');
```

The DEM indicates a horseshoe-shaped mountain range surrounding a valley that slopes down towards the south-east (Fig. 7.16a).

The SRTM subset is now analyzed for slope and aspect. When we are working with DEMs on a regular grid, slope and aspect can be estimated using centered finite differences in a local 3-by-3 neighborhood. Figure 7.14 shows a local neighborhood using the MATLAB cell indexing convention. For calculating slope and aspect, we need two finite differences in the DEM elements z , in x and y directions:

$$z_x = \frac{z_{r,c-1} - z_{r,c+1}}{2h}$$

and

Fig. 7.14 Local neighborhood showing the MATLAB cell number convention.

Z(1)	Z(4)	Z(7)
Z(2)	Z(5)	Z(8)
Z(3)	Z(6)	Z(9)

$$z_y = \frac{z_{r-1,c} - z_{r+1,c}}{2h}$$

where h is the cell size, which has the same units as the elevation. Using the finite differences, the slope is then calculated by

$$slp_{DF} = \sqrt{z_x^2 + z_y^2}$$

Other primary relief attributes such as the *aspect*, the *plan*, the *profile* and the *tangential curvature* can be derived in a similar way using finite differences (Wilson and Galant 2000). The function `gradientm` in the Mapping Toolbox calculates the slope and aspect of a data grid z in degrees above the horizontal and degrees clockwise from north. The function `gradientm(z, refvec)` requires a three-element reference vector `refvec`. The reference vector contains the number of cells per degree as well as the latitude and longitude of the upper-left (northwest) element of the data array. Since the SRTM digital elevation model is sampled on a 3 arc second grid, $60 \cdot 60/3 = 1200$ elements of the DEM correspond to one degree of longitude or latitude. For simplicity we ignore the actual coordinates of the SRTM subset in this example and use the indices of the DEM elements instead.

```
refvec = [1200 0 0];
[asp, slp] = gradientm(SRTM, refvec);
```

We display a pseudocolor map of the DEM slope in degrees (Fig 7.15b).

```
h = pcolor(slp);
colormap(jet), colorbar
set(h, 'LineStyle', 'none')
axis equal
title('Slope [°]')
[r c] = size(slp);
axis([1 c 1 r])
set(gca, 'TickDir', 'out');
```

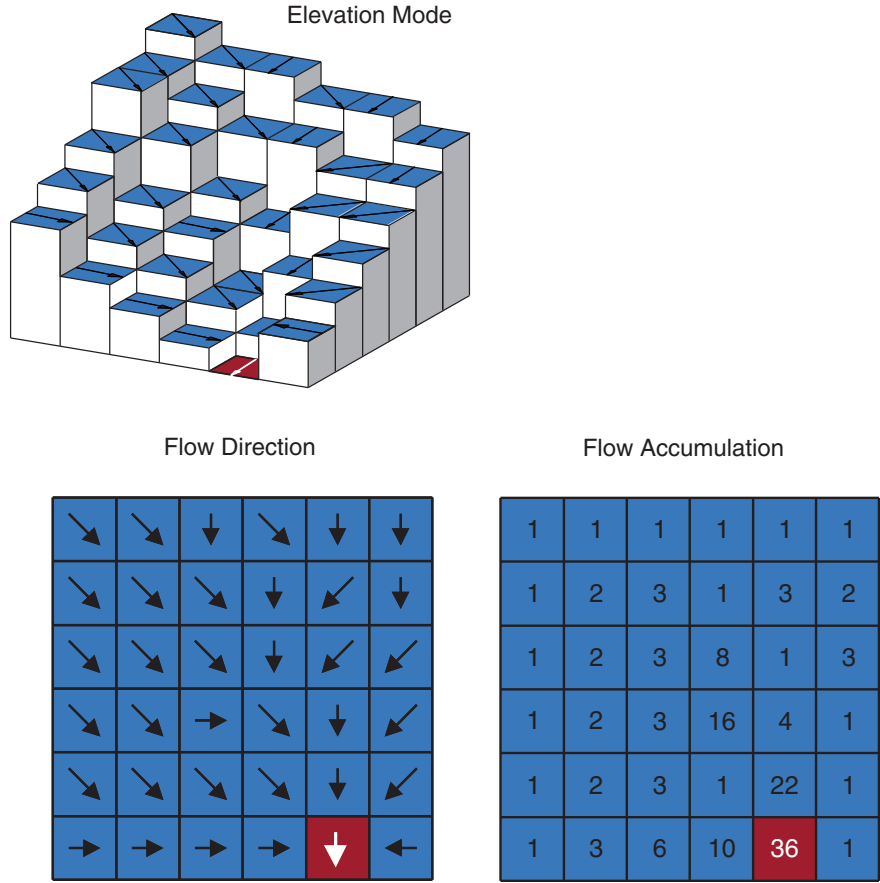



Fig. 7.15 Schematic of calculation of flow accumulation by the D8 method.

Flat areas are common on the summits and on the valley floors. The south-eastern and south-south-western sectors are also relatively flat. The steepest slopes are concentrated in the center of the area and in the south-western sector. Next, a pseudocolor map of the aspect is generated (Fig. 7.16c).

```

h = pcolor(asp);
colormap(hsv), colorbar
set(h,'LineStyle','none')
axis equal
title('Aspect')
[r c] = size(asp);
axis([1 c 1 r])
set(gca,'TickDir','out');
    
```

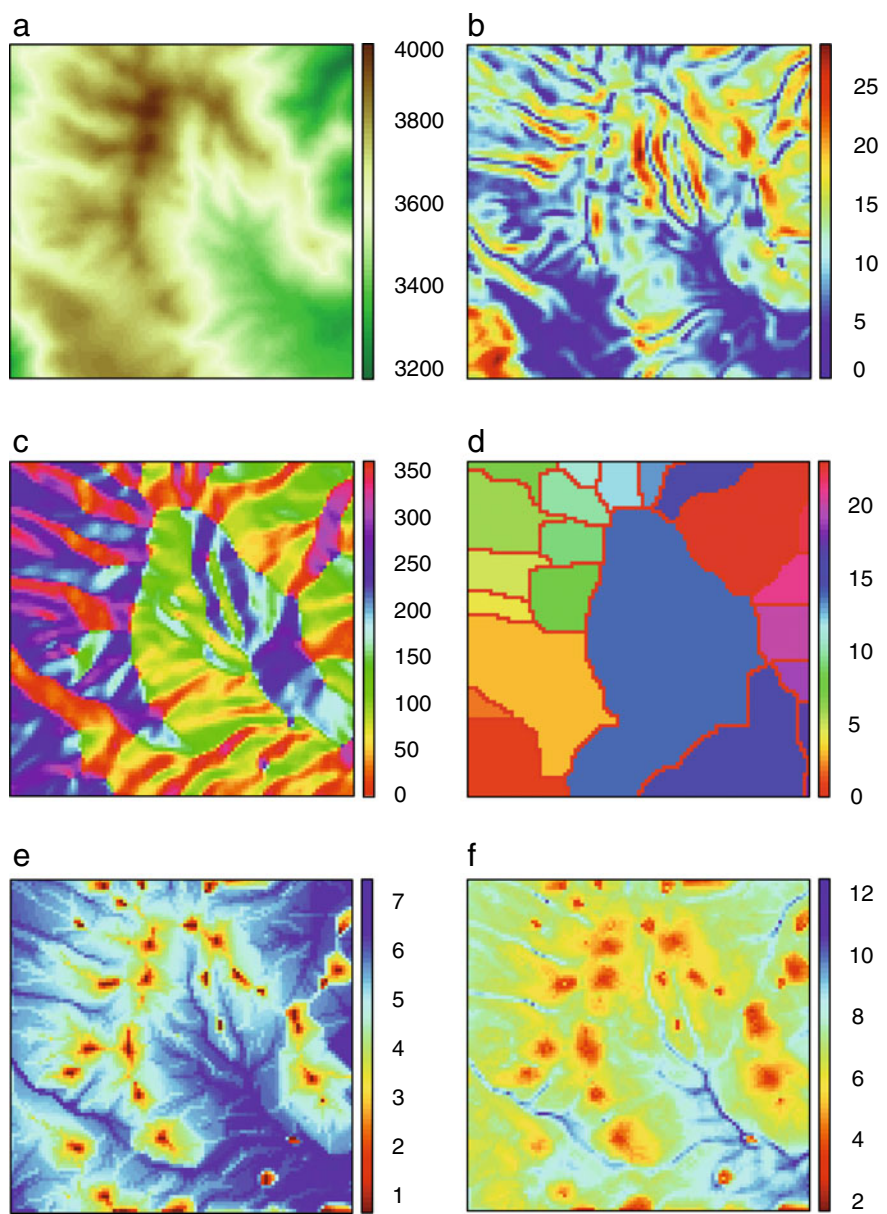


Fig. 7.16 Display of a subset of the SRTM data set used in Sect. 7.5 and primary and secondary attributes of the digital elevation model; **a** elevation, **b** slope, **c** aspect, **d** watershed, **e** flow accumulation, and **f** wetness index (data from Farr and Kobrick 2000; Farr et al. 2008).

This plot displays the aspect in degrees, clockwise from north. For instance, mountain slopes facing north are displayed in red and east-facing slopes in green.

The aspect changes abruptly along the ridges of the mountain ranges where neighboring drainage basins are separated by *watersheds*. The Image Processing Toolbox includes the function `watershed` to detect these drainage divides and to ascribe numerical labels to each catchment area, starting with one.

```
watersh = watershed(SRTM);
```

The catchment areas are displayed in a pseudocolor plot in which each area is assigned a color from the color table `hsv` (Fig. 7.16d), according to its numerical label.

```
h = pcolor(watersh);
colormap(hsv), colorbar
set(h,'LineStyle','none')
axis equal
title('Watershed')
[r c] = size(watersh);
axis([1 c 1 r])
set(gca,'TickDir','out');
```

The watersheds are represented by a series of red pixels. The largest catchment area corresponds to the medium blue region in the center of the map. To the north-west this large catchment area appears to be bordered by three catchments areas (represented by green colors) with no outlets. As in this example, `watershed` often generates unrealistic results since watershed algorithms are sensitive to local minima that act as spurious sinks. We can detect such sinks in the SRTM data using the function `imregionalmin`. The output of this function is a binary image in which pixels containing local maxima are marked by a value of one and all other pixels are marked by a zero.

```
sinks = 1*imregionalmin(SRTM);

h = pcolor(sinks);
colormap(gray)
set(h,'LineStyle','none')
axis equal
title('Sinks')
[r c] = size(sinks);
axis([1 c 1 r])
set(gca,'TickDir','out');
```

The pseudocolor plot of the binary image shows twelve local sinks, represented by white pixels, that are potential locations for spurious areas of internal drainage and should be borne in mind during any subsequent computation of hydrological characteristics from the DEM.

Flow accumulation, also called *specific catchment area* or *upslope contributing area*, is defined as the number of cells (or area) contributing runoff to a particular cell (Fig. 7.15). In contrast to the local parameters of slope and aspect, flow accumulation can only be determined from the global neighborhood. The principal operation is to add cell inflows from topographically higher neighboring cells, starting from the specified cell and working up to the watersheds. Before adding together the outflows from each cell we need to determine the gradient of each individual cell towards each neighboring cell, indexed by N . The array N contains indices for the eight adjacent cells, according to the MATLAB convention, as shown in Fig. 7.14. We make use of the `circshift` function to access the neighboring cells. For a two-dimensional matrix Z , the function `circshift(Z, [r c])` circularly shifts the values in the matrix Z by r rows and c columns. For example `circshift(Z, [1 1])` will circularly shift Z one row down and one column to the right. The individual gradients are calculated by

$$grad = \frac{z_{r+y,c+x} - z_{r,c}}{h}$$

for the eastern, southern, western, and northern neighbors (known as the *rook's case*) and by

$$grad = \frac{z_{r+y,c+x} - z_{r,c}}{\sqrt{2}h}$$

for the diagonal neighbors (known as the *bishop's case*). In these formulae, h is the cell size, z_r, c is the elevation of the central cell, and $z_{r+y, c+x}$ is the elevation of the neighboring cells. The cell indices x and y are obtained from the matrix N . The gradients are stored in a three-dimensional matrix `grads`, where `grads(:, :, 1)` contains the gradient towards the neighboring cells to the east, `grads(:, :, 2)` contains the gradient towards the neighboring cells to the south-east, and so on. Negative gradients indicate outflow from the central cell towards the relevant neighboring cell. To obtain the surface flow between cells, gradients are transformed using the inverse tangent of `grads` divided by 0.5π .

```
N = [0 -1;-1 -1;-1 0;+1 -1;0 +1;+1 +1;+1 0;-1 +1];
[a b] = size(SRTM);
grads = zeros(a,b,8);
for c = 2 : 2 : 8
    grads(:, :, c) = (circshift(SRTM, [N(c,1) N(c,2)]) ...
        - SRTM)/sqrt(2*90);
end
```

```

for c = 1 : 2 : 7
    grads(:, :, c) = (circshift(SRTM, [N(c,1) N(c,2)]) ...
        - SRTM)/90;
end
grads = atan(grads)/pi*2;

```

Since a central cell can have several downslope neighbors, water can flow in several directions. This phenomenon is called *divergent flow*. Early flow accumulation algorithms were based on the single-flow-direction method (known as the D8 method, Fig. 7.15), which allows flow to only one of the cell's eight neighboring cells. This method cannot, however, model divergences in ridge areas and tends to produce parallel flow lines in some situations. In our example we illustrate the use of a multiple-flow-direction method, which allows flow from a central cell to multiple neighboring cells. The proportion of the total outflow that is assigned to a neighboring cell is dependent on the gradient between the central cell and that particular neighboring cell. Even though multiple-flow methods produce more realistic results in most situations, they tend to result in dispersion in valleys, where the flow should be more concentrated. A weighting factor w is therefore introduced, which controls the relationship between the outflows.

$$flow_i = \frac{grad_i^w}{\sum_{i=1}^8 grad_i^w} \text{ for } grad_i^w < 0$$

A recommended value for w is 1.1; higher values would concentrate the flow in the direction of the steepest slope, while $w = 0$ would result in extreme dispersion. In the following sequence of commands, we first select those gradients that are less than zero and then multiply the gradients by the weighting.

```

w = 1.1;
flow = (grads.*(-1*grads<0)).^w;

```

We then sum up the upslope gradients along the third dimension of the `flow` matrix. Replacing all upslope gradient values of 0 by a value of 1 avoids the problems created by trying to divide by zero.

```

upssum = sum(flow,3);
upssum(upssum==0) = 1;

```

We divide the flows by `upssum` to obtain fractional weights that add up to a total of one. This is achieved separately for each layer of the 3D `flow` matrix using a `for` loop:

```

for i = 1:8
    flow(:, :, i) = flow(:, :, i).*(flow(:, :, i)>0)./upssum;
end

```

The 2D matrix `inflowsum` will store the intermediate inflow totals for each iteration. These intermediate totals are then summed to reach a figure for the total accumulated flow `flowac` at the end of each iteration. The initial values for `inflowsum` and `flowac` are obtained through `upssum`.

```
inflowsum = upssum;
flowac = upssum;
```

Another 3D matrix `inflow` is now needed, in which to store the total intermediate inflow from all neighbors:

```
inflow = grads*0;
```

Flow accumulation is terminated when there is no inflow or, translating this into MATLAB code, we use a conditional while loop that terminates if `sum(inflowsum(:)) == 0`. The number of non-zero entries in `inflowsum` will decrease during each loop. This is achieved by alternately updating `inflow` and `inflowsum`. Here, `inflowsum` is updated with the intermediate `inflow` of the neighboring cells weighted by `flow`, under the condition that all of the neighboring cells are contributing cells, i.e. where `grads` are positive. Where not all neighboring cells are contributing cells, the intermediate `inflowsum` is reduced, as also is `inflow`. The flow accumulation `flowac` increases through consecutive summations of the intermediate `inflowsum`.

```
while sum(inflowsum(:))>0
    for i = 1:8
        inflow(:, :, i) = circshift(inflowsum, [N(i,1) N(i,2)]);
    end
    inflowsum = sum(inflow.*flow.*grads>0,3);
    flowac = flowac + inflowsum;
end
```

We display the result as a pseudocolor map with log-scaled values (Fig. 7.16e).

```
h = pcolor(log(1+flowac));
colormap(flipud(jet)), colorbar
set(h, 'LineStyle', 'none')
axis equal
title('Flow accumulation')
[r c] = size(flowac);
axis([1 c 1 r])
set(gca, 'TickDir', 'out');
```

The plot displays areas with high flow accumulation in shades of blue and areas with low flow accumulation, usually corresponding to ridges, in shades of red. We used a logarithmic scale for mapping the flow accumulation in order to obtain a better representation of the results. The simplified algorithm introduced here for calculating flow accumulation can be used to analyze sloping terrains in DEMs. In flat terrains, where the slope approaches zero, no flow direction can be generated by our algorithm and thus flow accumulation stops. Such situations require more sophisticated algorithms to perform analyses on completely flat terrain. These more advanced algorithms also include sink-filling routines to avoid spurious sinks that interrupt flow accumulation. Small depressions can be filled by smoothing, as we did at the beginning of this section.

The first part of this section was about primary relief attributes. Secondary attributes of a DEM are functions of two or more primary attributes. Examples of secondary attributes are the wetness index and the stream power index. The *wetness index* for a cell is the log of the ratio between the area of the catchment for that particular cell and the tangent of its slope:

$$weti = \log\left(\frac{1 + flowac}{\tan(slp)}\right)$$

The term $1 + flowac$ avoids the problems associated with calculating the logarithm of zero when $flowac = 0$. The wetness index is used to predict the soil water content (*saturation*) resulting from lateral water movement. The potential for water logging is usually highest in the lower parts of catchments, where the slopes are more gentle. Flat areas with a large upslope area have a high wetness index compared to steep areas with small catchments. The wetness index *weti* is computed and displayed by

```
weti = log((1+flowac)./tand(slp));

h = pcolor(weti);
colormap(flipud(jet)), colorbar
set(h,'LineStyle','none')
axis equal
title('Wetness index')
[r c] = size(weti);
axis([1 c 1 r])
set(gca,'TickDir','out');
```

In this plot blue colors indicate high values for the wetness index while red colors represent low values (Fig. 7.16f). In our example soils in the south-east are the most likely to have a high water content due to the runoff from the large central valley and the flatness of the terrain.

The *stream power index* is another important secondary relief attribute that is frequently used in hillslope hydrology, geomorphology, soil science, and related disciplines. As a measure of stream power it provides an indication of the potential

for sediment transport and erosion by water. It is defined as the product of the catchment area for a specific cell and the tangent of the slope of that cell:

$$spi = flowac \cdot \tan(slp)$$

The potential for erosion is high when large quantities of water (calculated by flow accumulation) are fast flowing due to an extreme slope. The following series of commands compute and display the stream power index:

```
spi = flowac.*tand(slp);

h = pcolor(log(1+spi));
colormap(jet), colorbar
set(h, 'LineStyle', 'none')
axis equal
title('Stream power index')
[r c] = size(spi);
axis([1 c 1 r])
set(gca, 'TickDir', 'out');
```

The wetness and stream power indices are particularly useful in high resolution terrain analysis, i.e. digital elevation models sampled at intervals of less than 30 meters. In our terrain analysis example we have calculated *weti* and *spi* from a medium resolution DEM, and must expect a degree of scale dependency in these attributes.

This section has illustrated the use of basic tools for analyzing digital elevation models. A more detailed introduction to digital terrain modeling is given in the book by Wilson and Galant (2000). Furthermore, the article by Freeman (1991) provides a comprehensive summary of digital terrain analysis, including an introduction to the use of advanced algorithms for flow accumulation.

7.11 Geostatistics and Kriging (by R. Gebbers)

Geostatistics describes the autocorrelation of one or more variables in 1D, 2D, or 3D space, or even in 4D space-time, in order to make predictions for unobserved locations, to obtain information on the accuracy of the predictions, and to reproduce spatial variability and uncertainty. The shape, range, and direction of the spatial autocorrelation are described by a *variogram*, which is the main tool in linear geostatistics. The origins of geostatistics can be traced back to the early 1950s when the South African mining engineer Daniel G. Krige first published an interpolation method based on the spatial dependency of samples. In the 60s and 70s, the French mathematician George Matheron developed the *theory of regionalized variables*, which provides the theoretical foundations for Krige's more practical methods. This theory forms the basis of several procedures for the analysis and estimation of spatially dependent variables, which Matheron called *geostatistics*. Matheron also coined the term *kriging* for spatial interpolation by geostatistical methods.

Theoretical Background

A basic assumption in geostatistics is that a spatiotemporal process is composed of both deterministic and stochastic components (Fig. 7.17). The deterministic components can be global and local trends (sometimes called *drifts*). The stochastic component comprises a purely random part and an autocorrelated part. The autocorrelated component suggests that on average, closer observations are more similar to each other than more widely separated observations. This behavior is described by the variogram in which squared differences between observations are plotted against their separation distances. Krige's fundamental idea was to use the variogram for interpolation, as a means of determining the amount of influence that neighboring observations have when predicting values for unobserved locations. Basic linear geostatistics includes two main procedures: variography for modeling the variogram, and kriging for interpolation.

Preceding Analysis

Because linear geostatistics as presented herein is a parametric method, the underlying assumptions need to be checked by a preceding analysis. As with other parametric methods, linear geostatistics is sensitive to outliers and deviations from a normal distribution. We first open the data file *geost_dat.mat* containing *xyz* data triplets, and then plot the sampling locations. This allows us to check the point distribution and detect any major errors in the data coordinates, *x* and *y*.

```
clear, close all, clc  
  
load geost_dat.mat  
  
plot(x,y, '.')
```

The range of the observations *z* can be checked by

```
min(z)  
  
ans =  
  
3.7199  
  
max(z)  
  
ans =  
  
7.8460
```

For linear geostatistics, the observations *z* should be Gaussian distributed. This is usually only tested by visual inspection of the histogram because statistical tests are often too sensitive if the number of samples exceeds about 100. One can also calculate the skewness and kurtosis of the data.

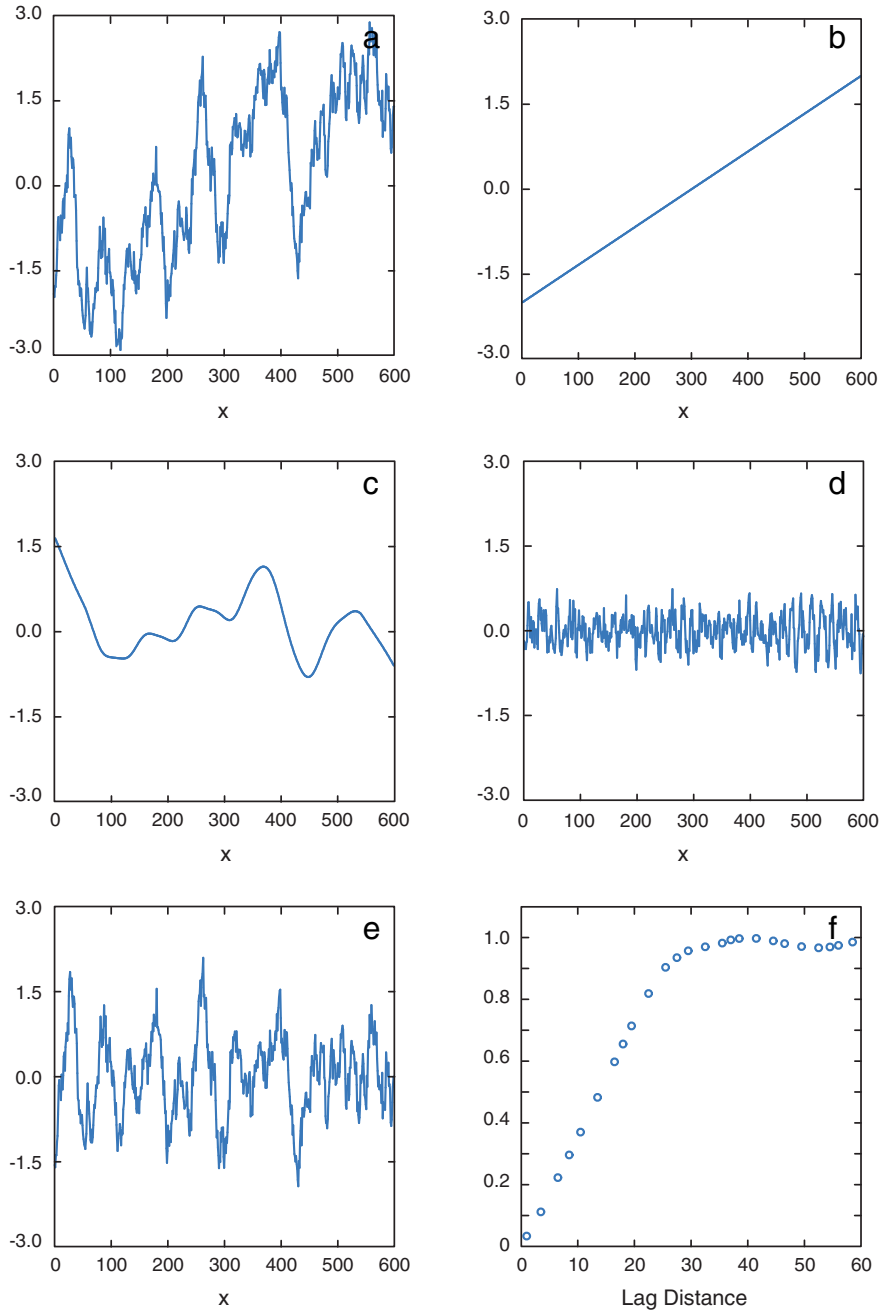


Fig. 7.17 Components of a spatiotemporal process and the variogram: **a** spatiotemporal process, **b** global trend component, **c** local trend component, **d** random component, **e** autocorrelation component, and **f** variogram. The variogram should only be derived from the autocorrelated component.

```

histogram(z)

skewness(z)

ans =

    0.2568

kurtosis(z)

ans =

    2.5220

```

A flat-topped or multiple-peaked distribution suggests that there is more than one population present in the data set. If these populations can be related to particular areas they should be treated separately. Another reason for multiple peaks can be preferential sampling of areas with high and/or low values. This usually happens as a result of some *a priori* knowledge and is known as a cluster effect. Dealing with a cluster effect is described in Deutsch and Journel (1998) and in Isaaks and Srivastava (1989).

Most problems arise from positive skewness, i.e. if the distribution has a long tail to the right. According to Webster and Oliver (2007), one should consider root transformation if the skewness is between 0.5 and 1, and logarithmic transformation if the skewness exceeds 1. A general transformation formula is:

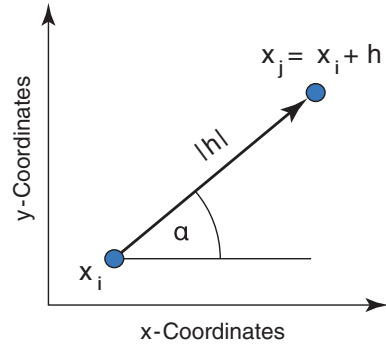
$$z^* = \begin{cases} ((z+m)^2 - 1)/k, & \text{for } k \neq 0 \\ \log(z+m), & \text{for } k = 0 \end{cases}$$

for $\min(z) + m > 0$. This is known as the Box-Cox transform, with the special case $k = 0$ when a logarithm transformation is used. In the logarithm transformation, m should be added if z is zero or negative. Interpolation results of power-transformed values can be back-transformed directly after kriging. The back-transformation of log-transformed values is slightly more complicated and will be explained later. The procedure is known as *lognormal kriging*. It can be important because lognormal distributions are not uncommon in geology.

Variography with the Classic Variogram

A variogram describes the spatial dependency of referenced observations in a unidimensional or multidimensional space. Since the true variogram of the spatial process is usually unknown, it has to be estimated from observations. This procedure is called variography. Variography starts by calculating the *experimental variogram* from the raw data. In the next step, the experimental variogram is summarized by the *variogram estimator*. The variography then concludes by fitting a variogram model to the variogram estimator. The experimental variogram is calculated as the differences between pairs of observed values and is dependent on the *separation vector* h (Fig. 7.18). The classic experimental variogram is defined by the *semivariance*,

Fig. 7.18 Separation vector h between two points.



$$\gamma(h) = 0.5 \cdot (z_x - z_{x+h})^2$$

where z_x is the observed value at location x and z_{x+h} is the observed value at another point at a distance h from x . The length of the separation vector h is called the *lag distance*, or simply the *lag*. The correct term for $\gamma(h)$ is the *semivariogram* (or *semivariance*), where *semi* refers to the fact that it is half of the variance in the differences between z_x and z_{x+h} . It is, nevertheless, the variance per point when points are considered in pairs (Webster and Oliver 2007). By convention, however, $\gamma(h)$ is termed a *variogram* instead of a semivariogram, a convention that we shall follow for the rest of this section. To calculate the experimental variogram we first need to group pairs of observations. This is achieved by typing

```
[X1,X2] = meshgrid(x);
[Y1,Y2] = meshgrid(y);
[Z1,Z2] = meshgrid(z);
```

The matrix of separation distances D between the observation points is

$$D = \text{sqrt}((X1 - X2).^2 + (Y1 - Y2).^2);$$

We then obtain the experimental variogram G , as half the squared differences between the observed values:

$$G = 0.5 * (Z1 - Z2).^2;$$

In order to speed up the processing we use the MATLAB capability to vectorize commands instead of using for loops to run faster. However, we have computed n^2 pairs of observations although only $n(n-1)/2$ pairs are required. For large data sets (e.g. more than 3,000 data points) the software and physical memory of the computer may become limiting factors. In such case one should use loops and store aggregated results in a variogram grid instead of generating a matrix with individual pairs as used in SURFER (Golden Software Inc.). The plot of the experimental

variogram is called the *variogram cloud* (Fig. 7.19), which we obtain by extracting the lower triangular portions of the D and G arrays.

```
indx = 1:length(z);
[C,R] = meshgrid(indx);
I = R > C;

plot(D(I),G(I),'.')
xlabel('lag distance')
ylabel('variogram')
```

The variogram cloud provides a visual impression of the dispersion of values at the different lags. It can be useful for detecting outliers or anomalies, but it is hard to judge from this presentation whether there is any spatial correlation, and if so, what form it might have and how we could model it (Webster and Oliver 2007). To obtain a clearer view and to prepare a variogram model the experimental variogram is now replaced by the variogram estimator.

The variogram estimator is derived from the experimental variograms in order to summarize their central tendency (similar to the descriptive statistics derived from univariate observations, Sect. 3.2). The classic variogram estimator is the averaged empirical variogram within certain distance classes or bins defined by multiples of

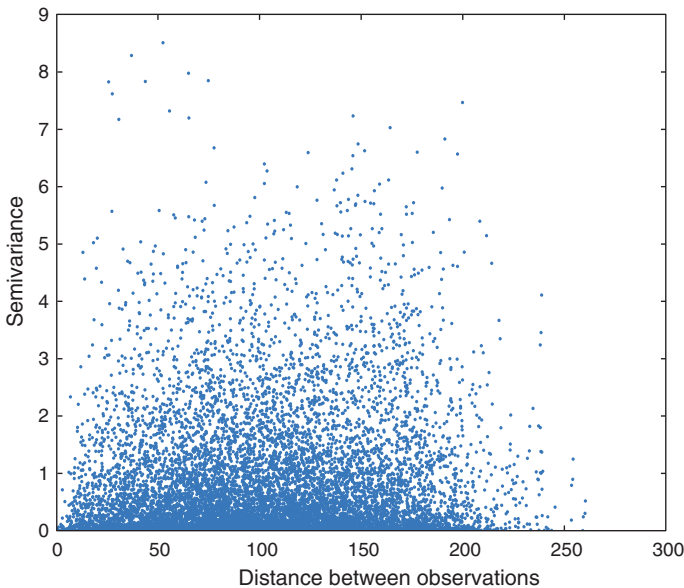


Fig. 7.19 Variogram cloud: plot of the experimental variogram (half the squared difference between pairs of observations) versus the lag distance (separation distance between the two components of a pair).

the lag interval. The classification of separation distances is illustrated in Fig. 7.20. The variogram estimator is calculated by:

$$\gamma_E(h) = \frac{1}{2 * N(h)} \cdot \sum_{i=1}^{N(h)} (z_{xi} - z_{xi+h})^2$$

where $N(h)$ is the number of pairs within the lag interval h .

We first need to decide on a suitable lag interval h . If sampling has been carried out on a regular grid, the length of a grid cell can be used. If the samples are unevenly spaced, as in our case, the mean minimum separation distance of pairs is a good starting point for the lag interval (Webster and Oliver 2007). To calculate the mean minimum separation distance of pairs we need to replace the zeros in the diagonal of the lag matrix D with NaNs, otherwise the mean minimum separation distance will be zero:

```
D2 = D.*(diag(x*NaN)+1);
lag = mean(min(D2))

lag =

    8.0107
```

Since the estimated variogram values tend to become more erratic with increasing separation distances, it is important to place a maximum separation distance limit on the calculation. As a rule of thumb, half of the maximum separation distance is a suitable limit for variogram analysis. We obtain the half maximum distance and the maximum number of lags by:

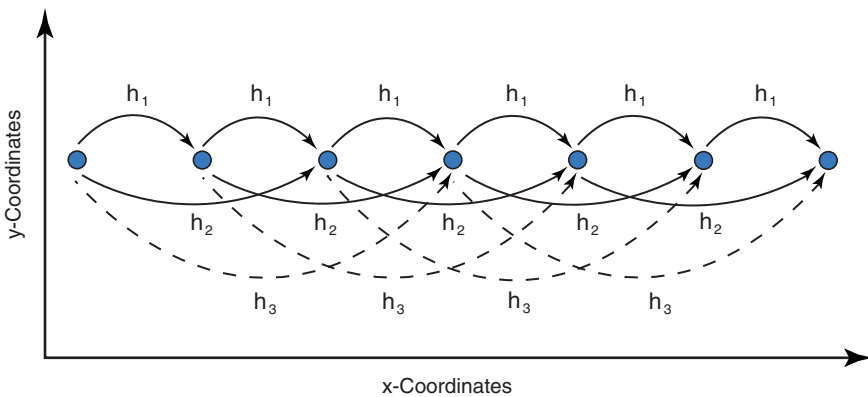


Fig. 7.20 Classification of separation distances for observations that are equally spaced. The lag interval is h_1 , and h_2 , h_3 etc., are multiples of the lag interval.

```

hmd = max(D(:))/2

hmd =

    130.1901

max_lags = floor(hmd/lag)

max_lags =

    16

```

The separation distances are then classified and the classic variogram estimator calculated:

```

LAGS = ceil(D/lag);

for i = 1 : max_lags
    SEL = (LAGS == i);
    DE(i) = mean(mean(D(SEL)));
    PN(i) = sum(sum(SEL == 1))/2;
    GE(i) = mean(mean(G(SEL)));
end

```

where SEL is the selection matrix defined by the lag classes in LAG, DE is the mean lag, PN is the number of pairs, and GE is the variogram estimator. We can now plot the classic variogram estimator (variogram versus mean separation distance), together with the population variance:

```

plot(DE,GE, ' . ')
var_z = var(z);
b = [0 max(DE)];
c = [var_z var_z];
hold on

plot(b,c, '--r')
yl = 1.1 * max(GE);
ylim([0 yl])
xlabel('Averaged distance between observations')
ylabel('Averaged semivariance')
hold off

```

The variogram in Fig. 7.21 exhibits a typical pattern of behavior. Values are low at small separation distances (near the origin) and then increase with increasing distance until reaching a plateau (*sill*), which is close to the population variance. This indicates that the spatial process is correlated over short distances but that is no spatial dependency over longer distances. The extent of the spatial dependency is called the *range* and is defined as the separation distance at which the variogram reaches the sill.

The *variogram model* is a parametric curve fitted to the variogram estimator. This is similar to frequency distribution fitting (see Sect. 3.5), where the frequency distribution is modeled by a distribution type and its parameters (e.g. a normal

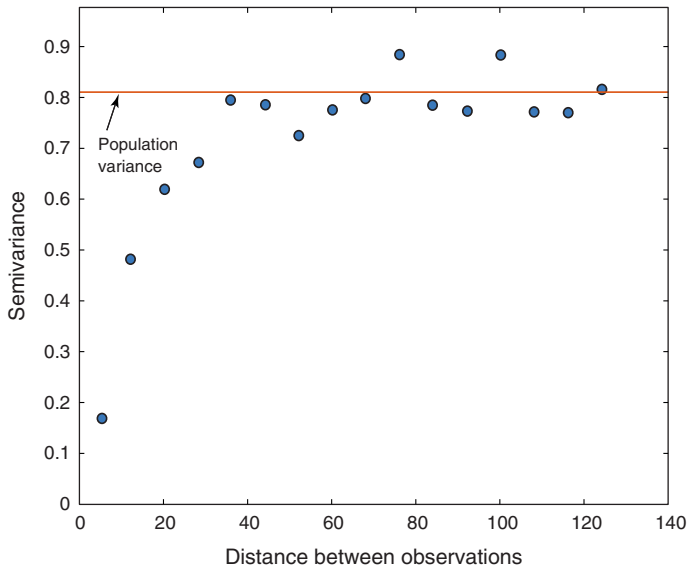


Fig. 7.21 The classic variogram estimator (blue) and the population variance (dashed line).

distribution with its mean and variance). For theoretical reasons, only functions with certain properties should be used as variogram models. Common *authorized models* are the spherical, the exponential, and the linear model (more models can be found in the relevant published literature).

Spherical model:

$$\gamma_{sph}(h) = \begin{cases} c \cdot \left(1.5 \cdot \frac{h}{a} - 0.5 \left(\frac{h}{a}\right)^3\right), & \text{for } 0 \leq h \leq a \\ c, & \text{for } h > a \end{cases}$$

Exponential model:

$$\gamma_{exp}(h) = c \cdot \left(1 - \left(-3 \cdot \frac{h}{a}\right)\right)$$

Linear model:

$$\gamma_{lin}(h) = b \cdot h$$

where c is the sill, a is the range, and b is the slope (for a linear model). The parameters c and either a or b must be modified if a variogram model is fitted to the variogram estimator. The *nugget effect* is a special type of variogram model. In practice, when extrapolating the variogram towards a separation distance of zero we often observe a positive intercept on the y-axis. This is called the nugget effect and

it is explained by measurement errors and by small scale fluctuations (*nuggets*) that are not captured due to the sampling intervals being too large. We sometimes have expectations about the minimum nugget effect from the variance of repeated measurements in the laboratory, or from other prior knowledge. More details concerning the nugget effect can be found in Cressie (1993) and Kitanidis (1997). If there is a nugget effect, it can be added into the variogram model. An exponential model with a nugget effect looks like this:

$$\gamma_{exp+nug}(h) = c_0 + c \cdot \left(1 - \exp\left(-3 \cdot \frac{h}{a}\right)\right)$$

where c_0 is the nugget effect.

We can even combine variogram models, e.g. two spherical models with different ranges and sills. These combinations are called *nested models*. During variogram modeling the components of a nested model are regarded as spatial structures that should be interpreted as being the results of geological processes. Before we discuss further aspects of variogram modeling let us just fit some models to our data. We begin with a spherical model with no nugget effect, and then add an exponential model and a linear model, both with nugget variances:

```
plot(DE,GE,'o','MarkerFaceColor',[.6 .6 .6])
var_z = var(z);
b = [0 max(DE)];
c = [var_z var_z];
hold on
plot(b,c,'--r')
xlim(b)
yl = 1.1*max(GE);
ylim([0 yl])

% Spherical model with nugget
nugget = 0;
sill = 0.803;
range = 45.9;
lags = 0:max(DE);
Gsph = nugget + (sill*(1.5*lags/range - 0.5*(lags/...
    range).^3).*(lags<=range) + sill*(lags>range));
plot(lags,Gsph,'g')

% Exponential model with nugget
nugget = 0.0239;
sill = 0.78;
range = 45;
Gexp = nugget + sill*(1 - exp(-3*lags/range));
plot(lags,Gexp,'-.b')
```

```
% Linear model with nugget
nugget = 0.153;
slope = 0.0203;
Glin = nugget + slope*lags;
plot(lags,Glin,'-m')
xlabel('Distance between observations')
ylabel('Semivariance')
legend('Variogram estimator','Population variance',...
       'Spherical model','Exponential model','Linear model')
hold off
```

The techniques of variogram modeling are still very much under discussion. Some advocate *objective* variogram modeling by automated curve fitting, using a weighted least squares, maximum likelihood, or maximum entropy method. In contrast, it is often argued that geological knowledge should be included in the modeling process and visual fitting is therefore recommended. In many cases the problem with variogram modeling is much less a question of whether the appropriate procedure has been used than a question of the quality of the experimental variogram. If the experimental variogram is good, both procedures will yield similar results.

Another important question in variogram modeling is the intended use of the model. In our case the linear model does not at first appear to be appropriate (Fig. 7.22). Following a closer look, however, we can see that the linear model fits reasonably well over the first three lags. This can be sufficient if we use the variogram model only for kriging, because in kriging the nearby points are the most important points for the estimate (see discussion of kriging below). Different variogram models with similar fits close to the origin will therefore yield similar kriging results if the sampling points are regularly distributed. If, however, the objective is to describe the spatial structures then the situation is quite different. It then becomes important to find a model that is suitable over all lags and to accurately determine the sill and the range. A collection of geological case studies in Rendu and Readdy (1982) show how process knowledge and variography can be interlinked. Good guidelines for variogram modeling are provided by Gringarten and Deutsch (2001) and Webster and Oliver (2007).

We will now briefly discuss a number of other aspects of variography:

- *Sample size*—As in any statistical procedure, as large a sample as possible is required in order to obtain a reliable estimate. For variography it is recommended that the number of samples should be in excess of 100, and preferably more than 150 (Webster and Oliver 2007). For smaller sample numbers a maximum likelihood variogram should be computed (Pardo-Igúzquiza and Dowd 1997).
- *Sampling design*—In order to obtain a good estimation close to the origin of the variogram, the sampling design should include observations over small distances. This can be achieved by means of a nested design (Webster and Oliver 2007). Other possible designs have been evaluated by Olea (1984).
- *Anisotropy*—Thus far we have assumed that the structure of spatial correlation is independent of direction. We have calculated *omnidirectional variograms*

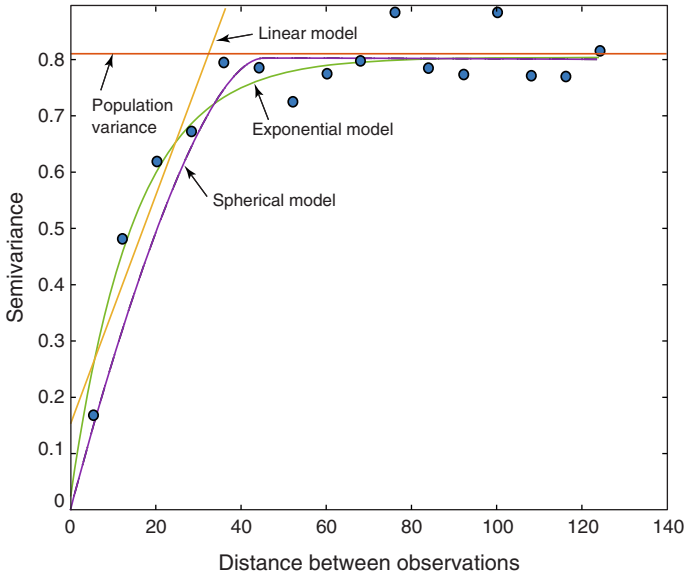


Fig. 7.22 Variogram estimator (blue circles), population variance (solid line), and spherical, exponential, and linear models (dotted and dashed lines).

ignoring the direction of the separation vector h . In a more thorough analysis, the variogram should be discretized not only in distance but also in direction (directional bins). Plotting *directional variograms*, usually in four directions, we are sometimes able to observe different ranges (*geometric anisotropy*), different scales (*zonal anisotropy*), and different shapes (indicating a trend). The treatment of anisotropy requires a highly interactive graphical user interface, which is beyond the scope of this book (see the VarioWin software by Pannatier 1996).

- *Number of pairs and the lag interval*—When calculating the classic variogram estimator it is recommended that more than 30 pairs, and preferably more than 50 pairs of points be used per lag interval (Webster and Oliver 2007). This is due to the sensitivity to outliers. If there are fewer pairs, the lag interval should be increased. The lag spacing does not necessarily need to be uniform but can be chosen individually for each distance class. It is also possible to work with overlapping classes, in which case the *lag width* (*lag tolerance*) must be defined. However, increasing the lag width can cause unnecessary smoothing, with a resulting loss of detail. The separation distance and the lag width must therefore be chosen with care. Another option is to use a more robust variogram estimator (Cressie 1993; Deutsch and Journel 1998).
- *Calculation of separation distance*—If the observations cover a large area, for example more than 1,000 km², spherical distances should be calculated instead of Pythagorean distances from a planar Cartesian coordinate system.

Kriging

We will now interpolate the observations onto a regular grid by *ordinary point kriging*, which is the most popular kriging method. Ordinary point kriging uses a weighted average of the neighboring points to estimate the value of an unobserved point:

$$\hat{z}_{x0} = \sum_i^N \lambda_i \cdot z_{xi}$$

where λ_i are the weights that have to be estimated. The sum of the weights should be equal to one in order to guarantee that the estimates are unbiased:

$$\sum_i^N \lambda_i = 1$$

The expected (average) error for the estimation must be zero. That is:

$$E(\hat{z}_{x0} - z_{x0}) = 0$$

where z_{x0} is the true, but unknown value. We can use the above equations to compute algebraically the mean-squared error in terms of the variogram:

$$E\left((\hat{z}_{x0} - z_{x0})^2\right) = 2 \sum_{i=1}^N \lambda_i \gamma(x_i, x_0) - \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j \gamma(x_i, x_j)$$

where E is the estimation or *kriging variance* (which must be minimized), $\gamma(x_i, x_0)$ is the variogram (semivariance) between the data points and the unobserved points, $\gamma(x_i, x_j)$ is the variogram between the data points x_i and x_j , and λ_i and λ_j are the weights of the i th and j th data points.

For kriging we must minimize this equation (a quadratic objective function), satisfying the condition that the sum of the weights should be equal to one (linear constraint). This optimization problem can be solved using a Lagrange multiplier v , resulting in a *linear kriging system* of $N + 1$ equations and $N + 1$ unknowns:

$$\sum_{i=1}^N \lambda_i \gamma(x_i, x_j) + v = \gamma(x_i, x_0)$$

After obtaining the weights λ_i , the kriging variance is given by

$$\sigma^2(x_0) = \sum_{i=1}^N \lambda_i \gamma(x_i, x_0) + v(x_0)$$

The kriging system can be presented in a matrix notation:

$$G_{mod} \cdot E = G_R$$

where

$$G_{mod} = \begin{bmatrix} 0 & \gamma(x_1, x_2) & \dots & \gamma(x_1, x_N) & 1 \\ \gamma(x_2, x_1) & 0 & \dots & \gamma(x_2, x_N) & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ \gamma(x_N, x_1) & \gamma(x_N, x_2) & \dots & 0 & 1 \\ 1 & 1 & \dots & 1 & 0 \end{bmatrix}$$

is the matrix of the coefficients: these are the modeled variogram values for the pairs of observations. Note that on the diagonal of the matrix, where separation distance is zero, the value of γ disappears.

$$E = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_N \\ v \end{bmatrix}$$

is the vector of the unknown weights and the Lagrange multiplier.

$$G_R = \begin{bmatrix} \gamma(x_1, x_0) \\ \gamma(x_2, x_0) \\ \vdots \\ \gamma(x_N, x_0) \\ 1 \end{bmatrix}$$

is the right-hand-side vector. To obtain the weights and the Lagrange multiplier the matrix G_{mod} is inverted:

$$E = G_{mod}^{-1} \cdot G_R$$

The kriging variance is given by

$$\sigma^2 = G_R^{-1} \cdot E$$

For our calculations using MATLAB we need the matrix of coefficients derived from the distance matrix D and a variogram model. D was calculated in the variography section above and we use the exponential variogram model with a nugget, sill, and range from the previous section:

$$G_mod = (nugget + sill*(1 - \exp(-3*D/range))).*(D>0);$$

We then take the number of observations and add a column and row vector of all values of one to the `G_mod` matrix and a zero in the lower left corner:

```
n = length(x);
G_mod(:,n+1) = 1;
G_mod(n+1,:) = 1;
G_mod(n+1,n+1) = 0;
```

The `G_mod` matrix now has to be inverted:

```
G_inv = inv(G_mod);
```

A grid with the locations of the unknown values is needed. Here we use a grid cell size of five within a quadratic area ranging from 0 to 200 in x and y directions. The coordinates are created in matrix form by:

```
R = 0 : 5 : 200;
[Xg1,Xg2] = meshgrid(R,R);
```

and converted to vectors by:

```
Xg = reshape(Xg1,[],1);
Yg = reshape(Xg2,[],1);
```

We then allocate memory for the kriging estimates `Zg` and the kriging variance `s2_k` by:

```
Zg = Xg * NaN;
s2_k = Xg * NaN;
```

We now krig the unknown values at each grid point:

```
for k = 1 : length(Xg)
    DOR = ((x - Xg(k)).^2 + (y - Yg(k)).^2).^0.5;
    G_R = (nugget + sill*(1 - exp(-3*DOR/range))).*(DOR>0);
    G_R(n+1) = 1;
    E = G_inv * G_R;
    Zg(k) = sum(E(1:n,1).*z);
    s2_k(k) = sum(E(1:n,1).*G_R(1:n,1))+E(n+1,1);
end
```

The first command computes the distance between the grid points (Xg, Yg) and the observation points (x, y). We then create the right-hand-side vector of the kriging system using the variogram model `G_R` and adding one to the last row. We next obtain the matrix `E` with the weights and the Lagrange multiplier. The estimate `Zg` at

each point k is the weighted sum of the observations z . Finally, the kriging variance $s2_k$ of the grid point is computed and we can plot the results. We first create a grid of the kriging estimate and the kriging variance:

```
r = length(R);
Z = reshape(Zg,r,r);
SK = reshape(s2_k,r,r);
```

A subplot on the left presents the kriged values:

```
subplot(1,2,1)
h = pcolor(Xg1,Xg2,Z);
set(h,'LineStyle','none')
axis equal
ylim([0 200])
title('Kriging Estimate')
xlabel('x-Coordinates')
ylabel('y-Coordinates')
colormap(jet)
colorbar
```

The left hand subplot presents the kriging variance:

```
subplot(1,2,2)
h = pcolor(Xg1,Xg2,SK);
set(h,'LineStyle','none')
axis equal
ylim([0 200])
title('Kriging Variance')
xlabel('x-Coordinates')
ylabel('y-Coordinates')
colormap(jet)
colorbar
hold on
```

and we overlay the sampling positions:

```
plot(x,y,'ok')
hold off
```

The kriged values are shown in Fig. 7.23a. The kriging variance depends only on the distance from the observations and not on the observed values (Fig. 7.23b). Kriging reproduces the population mean when observations are beyond the range of

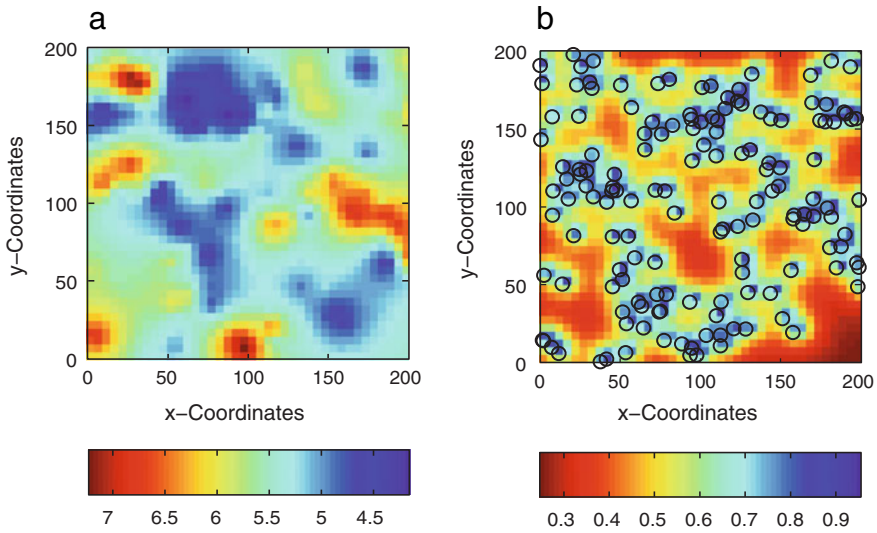


Fig. 7.23 Values interpolated on a regular grid by ordinary point kriging using **a** an exponential variogram model, and **b** the kriging variance as a function of the distance from the observation (empty circles).

the variogram; at the same time, the kriging variance increases (lower right corner of the maps in Fig. 7.23). The kriging variance can be used as a criterion to improve sampling design and it is needed for back-transformation in lognormal kriging. Back-transformation for lognormal kriging is achieved by:

$$y(x_0) = \exp(z(x_0) + 0.5 \cdot \sigma^2(x_0) - v)$$

Discussion of Kriging

Point kriging as presented here is an exact interpolator. It reproduces exactly the values at an observation point, even though a variogram with a nugget effect is used. Smoothing can be achieved by including the variance of the measurement errors (see Kitanidis 1997), and by *block kriging*, which averages the observations within a certain neighborhood (or block). While kriging variance depends only on the distance between the observed and the unobserved locations, it is primarily a measure of the density of information (Wackernagel 2003). The accuracy of kriging is better evaluated by cross-validation using a resampling method or a surrogate test (Sects. 4.6 and 4.7). The influence of the neighboring observations on the estimation depends on their configuration, as summarized by Webster and Oliver (2007): “Near points carry more weight than more distant ones; the relative weight of a point decreases when the number of points in the neighborhood increases;

clustered points carry less weight individually than isolated ones at the same distance; data points can be screened by ones lying between them and the target.” Sampling design for kriging is different from the design that might be optimal for variography. A regular grid, triangular or quadratic, can be regarded as optimal.

The MATLAB code presented here is a straightforward implementation of the above formulae. In professional programs the number of data points entering the G_mod matrix is restricted and the inversion of G_mod is avoided by working with the covariances instead of the variograms (Webster and Oliver 2007; Kitanidis 1997). For those who are interested in programming and in a deeper understanding of algorithms, the publication by Deutsch and Journel (1992) is essential reading. The best internet source is the homepage for AI-GEOSTATISTICS:

https://wiki.52north.org/AI_GEOSTATS/WebHome

Recommended Reading

- Amante C, Eakins BW (2009) ETOPO1 1 Arc-minute global relief model: procedures, data sources and analysis. NOAA Technical Memorandum NESDIS NGDC-24
- Bohlander J, Scambos T (2007) Antarctic coastlines and grounding line derived from MODIS Mosaic of Antarctica (MOA). National Snow and Ice Data Center, Boulder, Colorado
- Cressie N (1993) statistics for spatial data, revised edition. Wiley, New York
- Davis JC (2002) Statistics and data analysis in geology, 3rd edn. Wiley, New York
- Deutsch CV, Journel AG (1998) GSLIB—geostatistical software library and user's guide, 2nd edn. Oxford University Press, Oxford
- Farr TG, Rosen P, Caro E, Crippen R, Duren R, Hensley S, Kobrick M, Paller M, Rodriguez E, Roth L, Seal D, Shaffer S, Shimada J, Umland J, Werner M, Oskin M, Burbank D, Alsdorf D (2008) The shuttle radar topography mission. *Rev Geophys* 45:RG2004
- Farr TG, Kobrick M (2000) Shuttle Radar Topography Mission produces a wealth of data. *Am Geophys Union Eos* 81:583–585
- Freeman TG (1991) Calculating catchment area with divergent flow based on a regular grid. *Comput Geosci* 17:413–422
- Gorny AJ (1977) World data bank II general user guide Rep. PB 271869. Central Intelligence Agency, Washington, DC
- Gringarten E, Deutsch CV (2001) Teacher's aide variogram interpretation and modeling. *Math Geol* 33:507–534
- Isaaks E, Srivastava M (1989) An introduction to applied geostatistics. Oxford University Press, Oxford
- Kitanidis P (1997) Introduction to geostatistics—applications in hydrogeology. Cambridge University Press, Cambridge
- MathWorks (2020a) Mapping toolbox—user's guide. The MathWorks, Natick, MA
- MathWorks (2020b) Simulink 3D animation—user's guide. The MathWorks, Natick, MA
- Olea RA (1984) Systematic sampling of spatial functions. Kansas series on spatial analysis 7. Kansas Geological Survey, Lawrence, KS
- Pannatier Y (1996) VarioWin—software for spatial data analysis in 2D. Springer, Berlin, Heidelberg, New York

- Pardo-Igúzquiza E, Dowd PA (1997) AMLE3D: a computer program for the interference of spatial covariance parameters by approximate maximum likelihood estimation. *Comput Geosci* 23:793–805
- Rendu JM, Readdy L (1982) Geology and semivariogram—a critical relationship. In: Johnson TB, Barns RJ (eds) 17th international symposium american institute of mining application of computer & operation research in the mineral industry. Metallurgical and Petroleum Engineers, New York, pp 771–783
- Sandwell DT (1987) Biharmonic spline interpolation of GEOS-3 and SEASAT altimeter data. *Geophys Res Lett* 2:139–142
- Schwanghart W, Scherler D (2014) TopoToolbox 2—MATLAB-based software for topographic analysis and modeling in Earth surface sciences. *Earth Surf Dyn* 2:1–7
- Schwanghart W, Kuhn NJ (2010) TopoToolbox: a set of Matlab functions for topographic analysis. *Environ Modell Softw* 25:770–781
- Soluri EA, Woodson VA (1990), World vector shoreline. *Int Hydrogr Rev* 67(1):27–35
- Swan ARH, Sandilands M (1995) Introduction to geological data analysis. Blackwell Sciences, Oxford
- Wackernagel H (2003) Multivariate geostatistics: an introduction with applications, 3rd edn. Springer, Berlin, Heidelberg, New York
- Webster R, Oliver MA (2007) Geostatistics for environmental scientists, 2nd edn. Wiley, New York
- Wessel P, Bercovici D (1998) Gridding with splines in tension: a green function approach. *Math Geol* 30:77–93
- Wessel P, Smith WHF (1996) A global self-consistent, hierarchical, high-resolution shoreline database. *J Geophys Res* 101(B4):8741–8743
- Wilson JP, Gallant JC (2000) Terrain analysis, Principles and Applications. Wiley, New York

8.1 Introduction

Computer graphics are stored and processed as either vector or raster data. Most of the data encountered in the previous chapter were vector data, i.e. points, lines and polygons. Drainage networks, the outlines of geologic units, sampling locations, and topographic contours are all examples of vector data. In Chap. 7, coastlines are stored in a vector format while bathymetric and topographic data are saved in a raster format. Vector and raster data are often combined in a single data set, for instance to display the course of a river on a satellite image. Raster data are often converted to vector data by digitizing points, lines or polygons. Conversely, vector data are sometimes transformed to raster data.

Images are generally represented as raster data, i.e. as a 2D array of color intensities. Images are everywhere in geosciences. Field geologists use aerial photos and satellite images to identify lithologic units, tectonic structures, landslides and other features within a study area. Geomorphologists use such images to analyze drainage networks, river catchments, and vegetation or soil types. The analysis of images from thin sections, the automated identification of objects, and the measurement of varve thicknesses all make use of a great variety of image processing methods.

This chapter is concerned with the analysis and display of image data. The various ways that raster data can be stored on the computer are first explained (Sect. 8.2). The main tools for importing, manipulating and exporting image data are then presented in Sect. 8.3. This knowledge is then used to process and to georeference satellite images (Sects. 8.4–8.6). On-screen digitization techniques are discussed in Sect. 8.7. Sections 8.8 and 8.9 deal with importing, enhancing, and analyzing images from laminated lake sediments, including color-intensity

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_8) contains supplementary material, which is available to authorized users.

measurements on transects across the laminae. Sections 8.10–8.12 deal with automated grain size analysis, charcoal quantification in microscope images, and the detection of objects in microscope images on the basis of their shapes. Section 8.13 demonstrates how to calculate the normalized difference vegetation index (NDVI) from both handheld cameras and satellite images.

The Image Processing Toolbox is used for the specific examples throughout this chapter (MathWorks 2020). While the MATLAB User's Guide to the Image Processing Toolbox provides an excellent general introduction to the analysis of images, this chapter provides an overview of typical applications in earth sciences.

8.2 Data Storage

Vector and raster graphics are the two fundamental methods for storing pictures. The typical format for storing *vector data* has already been introduced in the previous chapter. In the following example the two columns in the file *coastline.txt* represent the longitudes and latitudes of the points of a polygon.

```
NaN      NaN
42.892067 0.000000
42.893692 0.001760
NaN      NaN
42.891052 0.001467
42.898093 0.007921
42.904546 0.013201
42.907480 0.016721
42.910414 0.020828
42.913054 0.024642
(cont'd)
```

The NaNs help to identify break points in the data (Sect. 7.2).

The *raster data* are stored as 2D arrays. The elements of these arrays represent variables such as the altitude of a grid point above sea level, the annual rainfall or, in the case of an image, the color intensity values.

```
174 177 180 182 182 182
165 169 170 168 168 170
171 174 173 168 167 170
184 186 183 177 174 176
191 192 190 185 181 181
189 190 190 188 186 183
```

Raster data can be visualized as 3D plots. The x and y figures are the indices of the 2D array or any other reference frame, and z is the numerical value of the elements of the array (see also Chap. 7). The numerical values contained in the 2D array can be displayed as a pseudocolor plot, which is a rectangular array of cells with colors determined by a *colormap*. A colormap is an m -by-3 array of real numbers between 0.0 and 1.0. Each row defines a red, green, or blue (RGB) color. An example is the above array, which could be interpreted as grayscale intensities ranging from 0 (black) to 255 (white). More complex examples include satellite imagery acquired from different bands of the electromagnetic spectrum, including the visible and near-infrared (VNIR, 400–1,400 nm), short-wave infrared (SWIR, 1,400–3,000 nm), and thermal (or long-wave) infrared (TIR, 3,000–15,000 nm) bands. The imagery is then stored in 3D arrays whose 3rd dimension corresponds to the number of spectral bands.

As previously discussed, a computer stores data as bits that have one of two states, represented by either a one or a zero (Chap. 2). If the elements of the 2D array represent the color intensity values of the *pixels* (short for *picture elements*) of an image, 1-bit arrays contain only ones and zeros.

```
0  0  1  1  1  1
1  1  0  0  1  1
1  1  1  1  0  0
1  1  1  1  0  1
0  0  0  0  0  0
0  0  0  0  0  0
```

This 2D array of ones and zeros can be simply interpreted as a black-and-white image, where the value of one represents white and zero corresponds to black. Alternatively, the 1-bit array could be used to store an image consisting of any two different colors, such as red and blue.

In order to store more complex types of data, the bits are joined together to form larger groups, such as bytes consisting of eight bits. Since the earliest computers could only process eight bits at a time, early computer code was written in sets of eight bits, which came to be called bytes. Each element of the 2D array or pixel therefore contains a vector of eight ones or zeros.

```
1  0  1  0  0  0  0  1
```

These 8 bits (or 1 byte) allow $2^8 = 256$ possible combinations of the eight ones or zeros, and are therefore able to represent 256 different intensities, such as grayscales. The 8 bits can be read in the following way, reading from right to left: a single bit represents two numbers, two bits represent four numbers, three bits represent eight numbers, and so forth up to a byte (or eight bits), which represents 256 numbers. Each added bit doubles the count of numbers. Here is a comparison of binary and decimal representations of the number 161:

128	64	32	16	8	4	2	1	(value of the bit)
1	0	1	0	0	0	0	1	(binary)

$$128 + 0 + 32 + 0 + 0 + 0 + 0 + 1 = 161 \text{ (decimal)}$$

The end members of the binary representation of grayscales are

0 0 0 0 0 0 0 0

which is black, and

1 1 1 1 1 1 1 1

which is pure white. In contrast to the above 1-bit array, the 1-byte array allows a grayscale image of 256 different levels to be stored. Alternatively, the 256 numbers could be interpreted as 256 discrete colors. In either case, the display of such an image requires an additional source of information concerning how the 256 intensity values are converted into colors. Numerous global colormaps for the interpretation of 8-bit color images exist that allow the cross-platform exchange of raster images, while local colormaps are often embedded in a graphics file.

The disadvantage of 8-bit color images is that the 256 discrete colorsteps are not enough to simulate smooth transitions for the human eye. A 24-bit system is therefore used in many applications, with 8 bits of data for each RGB channel giving a total of $256^3 = 16,777,216$ colors. Such a 24-bit image is stored in three 2D arrays, or one 3D array, of intensity values between 0 and 255.

195 189 203 217 217 221
 218 209 187 192 204 206
 207 219 212 198 188 190
 203 205 202 202 191 201
 190 192 193 191 184 190
 186 179 178 182 180 169

209 203 217 232 232 236
 234 225 203 208 220 220
 224 235 229 214 204 205
 223 222 222 219 208 216
 209 212 213 211 203 206
 206 199 199 203 201 187

```

174 168 182 199 199 203
198 189 167 172 184 185
188 199 193 178 168 172
186 186 185 183 174 185
177 177 178 176 171 177
179 171 168 170 170 163

```

Compared to the 1-bit and 8-bit representations of raster data, 24-bit storage certainly requires a lot more computer memory. In the case of very large data sets such as satellite images and digital elevation models the user should therefore think carefully about the most suitable way to store the data. The default data type in MATLAB is the 64-bit array, which allows storage of the sign of a number (bit 63), the exponent (bits 62–52) and roughly 16 significant decimal digits between approximately 10^{-308} and 10^{+308} (bits 51–0). However, MATLAB also works with other data types such as 1-bit, 8-bit and 24-bit raster data, to save memory.

The amount of memory required for storing a raster image depends on the data type and the image's dimensions. The dimensions of an image can be described by the number of pixels, which is the number of rows multiplied by the number of columns of the 2D array. Let us assume an image of 729-by-713 pixels, such as the one we will use in the following section. If each pixel needs 8 bits to store a grayscale value, the memory required by the data is $729 \cdot 713 \cdot 8 = 4,158,216$ bits or $4,158,216/8 = 519,777$ bytes. This number is exactly what we obtain by typing `whos` in the command window. Common prefixes for bytes are kilo-, mega-, giga- and so forth.

```

bit = 1 or 0 (b)
8 bits = 1 byte (B)
1024 bytes = 1 kilobyte (KB)
1024 kilobytes = 1 megabyte (MB)
1024 megabytes = 1 gigabyte (GB)
1024 gigabytes = 1 terabyte (TB)

```

Note that in data communication 1 kilobit = 1,000 bits, while in data storage 1 kilobyte = 1,024 bytes. A 24-bit or *true color image* then requires three times the memory required to store an 8-bit image, or 1,559,331 bytes = 1,559,331/1,024 kilobytes (KB) $\approx 1,523$ KB $\approx 1,559,331/1,024^2 = 1.487$ megabytes (MB).

However, the dimensions of an image are often given, not by the total number of pixels, but by the length and height of the image and its resolution. The resolution of an image is the number of *pixels per inch* (ppi) or *dots per inch* (dpi). The standard resolution of a computer monitor is 72 dpi although modern monitors often have a higher resolution such as 96 dpi. For instance, a 17 inches monitor with 72 dpi resolution displays 1,024-by-768 pixels. If the monitor is used to display images at a different (lower, higher) resolution, the image is resampled to match the

monitor's resolution. For scanning and printing, a resolution of 300 or 600 dpi is enough in most applications. However, scanned images are often scaled for large printouts and therefore have higher resolutions such as 2,400 dpi. The image used in the next section has a width of 25.2 cm (or 9.92 inches) and a height of 25.7 cm (10.12 inches). The resolution of the image is 72 dpi. The total number of pixels is therefore $72 \cdot 9.92 \approx 713$ in a horizontal direction, and $72 \cdot 10.12 \approx 729$ in a vertical direction.

Numerous formats are available for saving vector and raster data into a file, each with their own particular advantages and disadvantages. Choosing one format over another in an application depends on the way the images are to be used in a project and whether or not the images are to be analyzed quantitatively. The most popular formats for storing vector and raster data are:

- *Compuserve Graphics Interchange Format (GIF)*—This format was developed in 1987 for raster images using a fixed 8-bit colormap of 256 colors. The GIF format uses compression without loss of data. It was designed for fast transfer rates over the Internet. The limited number of colors means that it is not the right format for the smooth color transitions that occur in aerial photos or satellite images. It is, however, often used for line art, maps, cartoons and logos (<http://www.compuserve.com>).
- *Portable Network Graphics (PNG)*—This is an image format developed in 1994 that is used as an alternative to the GIF. It is similar to the GIF in that it also uses a fixed 8-bit colormap of 256 colors. Alternatively, grayscale images of 1–16 bits can be stored, as well as 24 and 48 bit color images. The PNG format uses compression without loss of data, with the method employed being better than that used for GIF images.
- *Microsoft Windows Bitmap Format (BMP)*—This is the default image format for computers running Microsoft Windows as the operating system. However, numerous converters also exist to read and write BMP files on other platforms. Various modifications of the BMP format are available, some of them without compression and others with effective and fast compression (<http://www.microsoft.com>).
- *Tagged Image File Format (TIFF)*—This format was designed by the Aldus Corporation and Microsoft in 1986 to become an industry standard for image-file exchange. A TIFF file includes an image file header, a directory, and the data in all available graphics and image file formats. Some TIFF files even contain vector and raster versions of the same picture, as well as images at different resolutions and with different colormaps. The main advantage of TIFF files was originally their portability. A TIFF should perform on all computer platforms; unfortunately, however, numerous modifications of the TIFF have evolved in subsequent years, resulting in incompatibilities. The TIFF is therefore now often called the *Thousands of Incompatible File Formats*.

- *PostScript* (PS) and *Encapsulated PostScript* (EPS)—The PS format was developed by John Warnock at PARC, the Xerox research institute. Warnock was also co-founder of Adobe Systems, where the EPS format was created. The PostScript vector format would never have become an industry standard without Apple Computers. In 1985 Apple needed a typesetter-quality controller for the new Apple LaserWriter printer and the Macintosh operating system and adopted the PostScript format. The third partner in the history of PostScript was the company Aldus, the developer of the PageMaker software and now part of Adobe Systems. The combination of Aldus PageMaker software, the PS format and the Apple LaserWriter printer led to the creation of Desktop Publishing. The EPS format was then developed by Adobe Systems as a standard file format for importing and exporting PS files. Whereas a PS file is generally a single-page format containing either an illustration or a text, the purpose of an EPS file is to also allow the inclusion of other pages, i.e. a file that can contain any combination of text, graphics and images (<http://www.adobe.com>).
- In 1986 the *Joint Photographic Experts Group* (JPEG) was founded for the purpose of developing various standards for image compression. Although JPEG stands for the committee, it is now widely used as the name for an image compression and a file format. This compression involves grouping pixel values into 8-by-8 blocks and transforming each block with a discrete cosine transform. As a result, all unnecessary high-frequency information is deleted, which makes this compression method irreversible. The advantage of the JPEG format is the availability of a three-channel, 24-bit, true color version. This allows images with smooth color transitions to be stored. The new JPEG-2000 format uses a wavelet transform instead of the cosine transform (Sect. 5.8) (<http://www.jpeg.org>).
- *Portable Document Format* (PDF)—The PDF designed by Adobe Systems is now a true self-contained cross-platform document. PDF files contain the complete formatting of vector illustrations, raster images and text, or a combination of all these, including all necessary fonts. These files are highly compressed, allowing a fast internet download. Adobe Systems provides the free-of-charge Acrobat Reader for all computer platforms to read PDF files (<http://www.adobe.com>).

8.3 Importing, Processing and Exporting Images

We first need to learn how to read an image from a graphics file into the workspace. As an example we use a satellite image showing a 10.5 km by 11 km subarea in northern Chile:

<http://asterweb.jpl.nasa.gov/gallery/images/unconform.jpg>

The file *unconform.jpg* is a processed TERRA-ASTER satellite image that can be downloaded free-of-charge from the NASA web page. We save this image in the working directory. The command

```
clear, close all, clc
```

```
I1 = imread('unconform.jpg');
```

reads and decompresses the JPEG file, imports the data as a 24-bit RGB image array and stores it in a variable *I1*. The command

```
whos
```

shows how the RGB array is stored in the workspace:

Name	Size	Bytes	Class	Attributes
I1	729x713x3	1559331	uint8	

The details indicate that the image is stored as a 729-by-713-by-3 array, representing a 729-by-713 array for each of the colors red, green and blue. The listing of the current variables in the workspace also gives the information *uint8* array, i.e. each array element representing one pixel contains 8-bit integers. These integers represent intensity values between 0 (minimum intensity) and 255 (maximum). As an example, here is a sector in the upper-left corner of the data array for red:

```
I1(50:55,50:55,1)
```

```
ans =
```

```
6×6 uint8 matrix
```

```
174 177 180 182 182 182
165 169 170 168 168 170
171 174 173 168 167 170
184 186 183 177 174 176
191 192 190 185 181 181
189 190 190 188 186 183
```

We can now view the image using the command

```
imshow(I1)
```

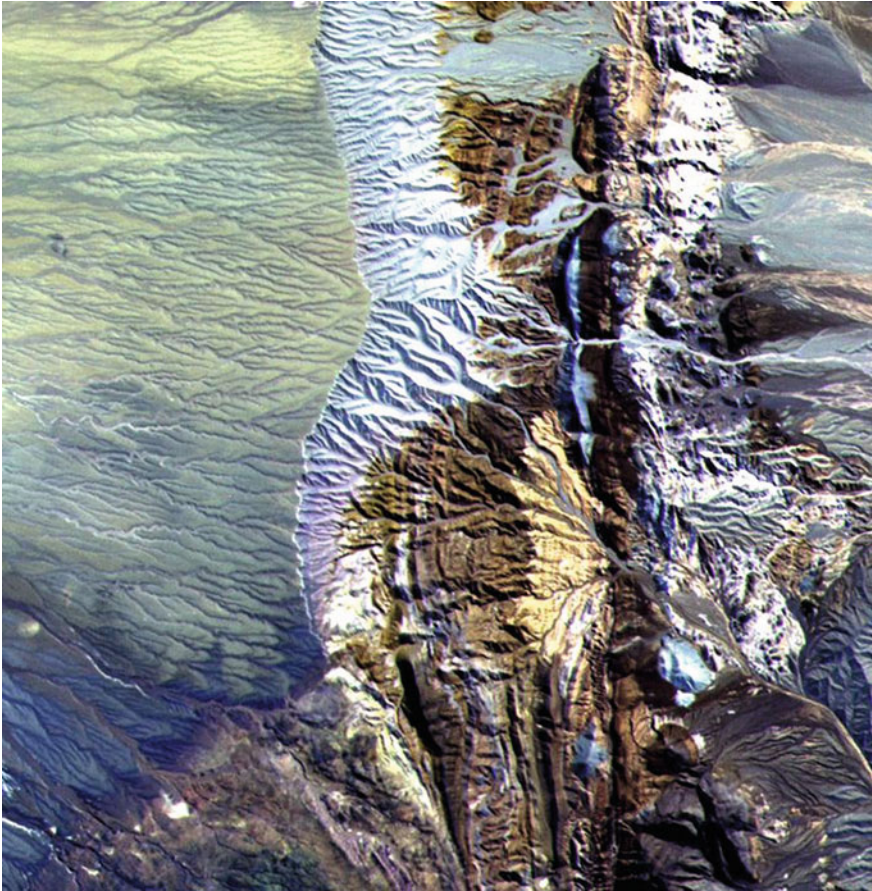


Fig. 8.1 RGB true color image contained in the file *unconform.jpg*. After decompressing and reading the JPEG file into a 729-by-713-by-3 array, MATLAB interprets and displays the RGB composite using the function `imshow`. Original image courtesy of NASA/GSFC/METI/ERSDAC/JAROS and U.S./Japan ASTER Science Team.

which opens a new Figure Window showing an RGB composite of the image (Fig. 8.1). In contrast to the RGB image, a grayscale image needs only a single array to store all the necessary information. We therefore convert the RGB image into a grayscale image using the command `rgb2gray` (RGB to gray):

```
I2 = rgb2gray(I1);
```

The new workspace listing now reads

Name	Size	Bytes	Class	Attributes
I1	729x713x3	1559331	uint8	
I2	729x713	519777	uint8	
ans	6x6	36	uint8	

in which the difference between the 24-bit RGB and the 8-bit grayscale arrays can be observed. The variable `ans` for *Most recent answer* was created above using `I1(50:55,50:55,1)`, without assigning the output to another variable. The commands

```
imshow(I2)
```

display the result. It is easy to see the difference between the two images in separate Figure Windows. Let us now process the grayscale image. First, we compute a histogram of the distribution of intensity values.

```
imhist(I2)
```

A simple technique to enhance the contrast in such an image is to transform this histogram to obtain an equal distribution of grayscales.

```
I3 = histeq(I2);
```

We can again view the difference using

```
imshow(I3)
```

and save the results in a new file.

```
imwrite(I3,'unconform_gray.jpg')
```

We can read the header of the new file by typing

```
imfinfo('unconform_gray.jpg')
```

which yields

```
ans =  
    struct with fields:  
        Filename: [1x40 char]  
        FileModDate: '18-Dec-2013 11:26:53'  
        FileSize: 138419  
        Format: 'jpg'  
        FormatVersion: ''  
        Width: 713  
        Height: 729  
        BitDepth: 8  
        ColorType: 'grayscale'  
        FormatSignature: ''  
        NumberOfSamples: 1  
        CodingMethod: 'Huffman'  
        CodingProcess: 'Sequential'  
        Comment: {}
```

The command `imfinfo` can therefore be used to obtain useful information (name, size, format, and color type) concerning the newly-created image file.

There are many ways of transforming the original satellite image into a practical file format. The image data could, for instance, be stored as an *indexed color image*, which consists of two parts: a colormap array and a data array. The colormap array is an m -by-3 array containing floating-point values between 0 and 1. Each column specifies the intensity of the red, green and blue colors. The data array is an x -by- y array containing integer elements corresponding to the lines m of the colormap array, i.e. the specific RGB representation of a certain color. Let us transfer the above RGB image into an indexed image. The colormap of the image should contain 16 different colors. The result of

```
[I4,map] = rgb2ind(I1,16);  
imshow(I1), figure, imshow(I4,map)
```

saved as another JPEG file using

```
imwrite(I4,map,'unconform_ind.jpg')
```

clearly shows the difference between the original 24-bit RGB image (256^3 or about 16.7 million different colors) and a color image of only 16 different colors. The display of the image uses the default colormap of MATLAB. Typing

```
imshow(I4,map)
cmap = colormap
```

actually retrieves the 16-by-3 array of the current colormap

```
cmap =
    0.0588    0.0275    0.0745
    0.5490    0.5255    0.4588
    0.7373    0.7922    0.7020
    0.3216    0.2706    0.2667
    0.6471    0.6784    0.6157
    0.7961    0.8549    0.9176
    0.4510    0.3922    0.3333
    0.2000    0.1451    0.1451
    0.4824    0.5412    0.5843
    0.4039    0.4078    0.4784
    0.6667    0.7020    0.7451
    0.8980    0.8745    0.7255
    0.2824    0.2902    0.4039
    0.9569    0.9647    0.9608
    0.1765    0.1686    0.2902
    0.5843    0.5843    0.6078
```

We can replace the default colormap by any other built-in colormap. Typing

```
help graph3d
```

lists the predefined colormaps:

```
parula      - Blue-green-orange-yellow color map
hsv         - Hue-saturation-value color map.
hot         - Black-red-yellow-white color map.
gray        - Linear gray-scale color map.
bone        - Gray-scale with tinge of blue color map.
copper      - Linear copper-tone color map.
pink        - Pastel shades of pink color map.
white       - All white color map.
flag        - Alternating red, white, blue, and black color map.
lines       - Color map with the line colors.
colorcube   - Enhanced color-cube color map.
vga         - Windows colormap for 16 colors.
jet         - Variant of HSV.
```

prism	- Prism color map.
cool	- Shades of cyan and magenta color map.
autumn	- Shades of red and yellow color map.
spring	- Shades of magenta and yellow color map.
winter	- Shades of blue and green color map.
summer	- Shades of green and yellow color map.

The default MATLAB colormap is parula, named after a former genus of New World warblers with blue, green and yellow feathers. The advantages of parula over the previous default colormap jet are discussed in blog posts entitled *A New Colormap for MATLAB* by Steve Eddins of The MathWorks, Inc. at

<https://blogs.mathworks.com/steve/>

An introduction to the history of colormaps entitled *Origins of Colormaps* which includes the history of jet is provided Cleve Moler, the creator of MATLAB, at

<https://blogs.mathworks.com/cleve/>

As an example of colormaps we can use hot from the list above by typing

```
imshow(I4,map)
colormap(hot)
```

to display the image with a black-red-yellow-white colormap. Typing

```
edit hot
```

reveals that hot is a function that creates an m -by-3 array containing floating-point values between 0 and 1. We can also design our own colormaps, either by manually creating an m -by-3 array or by creating another function similar to hot. As an example the colormap `precip.m` (which is a yellow-blue colormap included in the book's file collection) was created to display precipitation data, with yellow representing low rainfall and blue representing high rainfall. Alternatively, we can also use random numbers

```
rng(0)
map = rand(16,3);
imshow(I4,map)
```

to display the image with random colors. Finally, we can create an indexed color image of three different colors, displayed with a simple colormap of full intensity red, green and blue.

```
[I5,map] = rgb2ind(I1,3);  
imshow(I5,[1 0 0;0 1 0;0 0 1])
```

Typing

```
imwrite(I4,map,'unconform_rgb.jpg')
```

saves the result as another JPEG file.

8.4 Importing, Processing and Exporting LANDSAT Images

The Landsat project is a satellite remote sensing program jointly managed by the US National Aeronautics and Space Administration (NASA) and the US Geological Survey (USGS), which began with the launch of the Landsat 1 satellite (originally known as the Earth Resources Technology Satellite 1) on 23rd July 1972. The latest in a series of successors is the Landsat 8 satellite, launched on 11th February 2013 (Ochs et al. 2009; Irons et al. 2011). It has two sensors, the Operational Land Imager (OLI) and the Thermal Infrared Sensor (TIRS). These two sensors provide coverage of the global landmass at spatial resolutions of 30 meters (visible, NIR, SWIR), 100 meters (thermal), and 15 meters (panchromatic) (Ochs et al. 2009; Irons et al. 2011). General information concerning the Landsat program can be obtained from the webpage

http://landsat.gsfc.nasa.gov/?page_id=7195

Landsat data, together with data from other NASA satellites, can be obtained from the webpage

<https://earthexplorer.usgs.gov>

On this webpage we first select the desired map section in the *Search Criteria*, either by entering the coordinates of the four corners of the map or by zooming into the area of interest and selecting *Use Map*. As an example we enter the coordinates 4°42'40.72" N 36°51'10.47" E for the center of the Chew Bahir Basin in the Southern Ethiopian Rift. We then choose *L8 OLI/TIRS* from the *Landsat Archive* as the *Data Set* and click *Results*. Clicking *Results* produces a list of records, together with a toolbar for previewing and downloading data. By clicking the *Show Browse Overlay* button we can examine the images for cloud cover. We find the cloud-free image


```
Entity ID: LC81690572013358LGN00
Coordinates: 4.33915,36.76225
Acquisition Date: 24-DEC-13
Path: 169
Row: 57
```

taken on 24th December 2013. We need to register with the USGS website, log on, and then download the Level 1 GeoTIFF Data Product (897.5 MB), which is then stored on the hard drive in the file *LC81690572013358LGN00.tar.gz*. The *.tar.gz* archive contains separate files for each spectral band as well as a metadata file containing information about the data. We use band 4 (Red, 640–670 nm), band 3 (Green, 530–590 nm) and band 2 (Blue, 450–510 nm), each of which has a 30 meters resolution. We can import the 118.4 MB TIFF files using

```
clear, close all, clc

I1 = imread('LC81690572013358LGN00_B4.TIF');
I2 = imread('LC81690572013358LGN00_B3.TIF');
I3 = imread('LC81690572013358LGN00_B2.TIF');
```

Typing

```
whos
```

reveals that the data are in an unsigned 16-bit format `uint16`, i.e. the maximum range of the data values is from 0 to $2^{16} = 65,536$.

Name	Size	Bytes	Class	Attributes
I1	7771x7611	118290162	uint16	
I2	7771x7611	118290162	uint16	
I3	7771x7611	118290162	uint16	

For quantitative analyses these digital number (DN) values need to be converted to radiance and reflectance values, which is beyond the scope of the book. The radiance is the power density scattered from the Earth in a particular direction and has the units of watts per square meter per steradian ($\text{Wm}^{-2} \text{sr}^{-1}$) (Richards 2013). The radiance values need to be corrected for atmospheric and topographic effects to obtain Earth surface reflectance percentages. The Landsat 8 Handbook provides the necessary information on these conversions:

<https://www.usgs.gov/media/files/landsat-8-data-users-handbook>

We will instead use the Landsat 8 data to create an RGB composite of bands 4, 3, and 2 to be used for fieldwork. Since the image has a relatively low level of contrast, we use `adapthisteq` to perform a contrast-limited adaptive histogram equalization (CLAHE) (Zuiderveld 1994). Unlike `histeq` used in the previous section, the `adapthisteq` algorithm works on small regions (or tiles) of the image, rather than on the entire image. The neighboring tiles are then combined using bilinear interpolation to eliminate edge effects.

```
I1 = adapthisteq(I1,'ClipLimit',0.1,'Distribution','Rayleigh');
I2 = adapthisteq(I2,'ClipLimit',0.1,'Distribution','Rayleigh');
I3 = adapthisteq(I3,'ClipLimit',0.1,'Distribution','Rayleigh');
```

Using `ClipLimit` with a real scalar between 0 and 1 limits the contrast enhancement, while higher numbers result in increased contrast; the default value is 0.01. The `Distribution` parameter sets the desired histogram shape for the tiles by specifying a distribution type, such as `Uniform`, `Rayleigh`, or `Exponential`. Using a `ClipLimit` of 0.1 and a `Rayleigh` distribution yields good results. The three bands are concatenated to a 24-bit RGB images using `cat`.

```
I = cat(3,I1,I2,I3);
```

We only display that section of the image containing the Chew Bahir Basin (using axes limits) and hide the coordinate axes. We scale the images to 10% of the original size to fit the computer screen.

```
axes('XLim',[3000 5000],...
     'YLim',[1000 4000],...
     'Visible','Off'), hold on
imshow(I,'InitialMagnification',10)
```

Exporting the processed image from the Figure Window, we only save the image at the monitor's resolution. To obtain an image of the basins at a higher resolution, we use the command

```
imwrite(I(1000:4000,3000:5000,:),...
       'chewbahirbasin.tif','tif')
```

This command saves the RGB composite as a TIFF-file *chewbahirbasin.tif* (about 36.3 MB) in the working directory, which can then be processed using other software such as Adobe Photoshop.

According to the USGS Landsat webpage, Landsat data are amongst the most geometrically and radiometrically corrected data available. Data anomalies do occasionally occur, however, of which the most common types are listed on the USGS webpage:

<https://www.usgs.gov/land-resources/nli/landsat/landsat-known-issues>

We explore one of these types of anomaly as an example, i.e. artifacts known as *Single Event Upsets* (SEUs) that cause anomalously high values in the image, similar to the *Impulse Noise* (IN) that is also described on the same webpage. These anomalies occur in some, but not all, Landsat images and similarly anomalous high or low values can also occur in other satellite images. We therefore use a part of a Landsat 7 image covering an area in the southern Ethiopian Rift, acquired by the Enhanced Thematic Mapper (ETM+) instrument of that satellite. We can load and display the image using

```
clear

I1 = imread('ethiopianrift_blue.tif');
imshow(I1, 'InitialMagnification', 200)
title('Original Image')
```

The parameter `InitialMagnification` is a numeric scalar that scales the image to, as an example, 200% magnification. The image `I1` shows numerous randomly-distributed anomalously high or low values, as well as a parallel track of paired anomalies in the right half of the image. We first apply a 10-by-10 pixel median filter to the image (see Sect. 8.8):

```
I2 = medfilt2(I1, [10,10], 'symmetric');
imshow(I2, 'InitialMagnification', 200)
title('Median Filtered Image')
```

Using the option `symmetric` with the function `medfilt2` extends `I2` symmetrically at the image boundaries by reflecting it across its boundaries, instead of padding the image with zeros at the boundaries (by default). The median-filtered version of the `I2` image is, of course, very smooth compared to the original `I1` image. We would, however, lose a lot of detail if we used this version of the image. We next subtract the median-filtered image `I2` from original image `I1`, which yields the image `I3`.

```
I3 = imsubtract(I1, I2);
imshow(I3, 'InitialMagnification', 200)
title('I1-I2')
```

We then subtract the original image `I1` from the median-filtered image `I2`, which yields the image `I4`.

```
I4 = imsubtract(I2, I1);
imshow(I4, 'InitialMagnification', 200)
title('I2-I1')
```

We next replace the original pixels with their median-filtered versions if the difference between the median-filtered image `I2` and the original image `I1` is greater than 10 in both directions (as it is in our example).

```
I5 = I1;
I5(I3>10 | I4>10) = I2(I3>10 | I4>10);
imshow(I5, 'InitialMagnification', 200)
title('Despeckled Image')
```

The image `I5` obtained using this approach is the despeckled version of image `I1`. We can also explore the pixel values of both versions of the image (`I1` and `I5`) in a 3D surface plot, using

```
subplot(1,2,1)
I1S = im2double(I1);
surface(I1S), colormap jet, caxis([0 1])
shading interp, view(120,33), axis off
axis([1 size(I1,1) 1 size(I1,2) min(I1S(:)) max(I1S(:))])

subplot(1,2,2)
I5S = im2double(I5);
surface(I5S), colormap jet, caxis([0 1])
shading interp, view(120,33), axis off
axis([1 size(I1,1) 1 size(I1,2) min(I1S(:)) max(I1S(:))])
```

We need convert the image data to class `double` using `im2double` in order to be able to display the data using `surface`. Finally, we can display both images in the same figure window

```
subplot(1,2,1), imshow(I1)
title('Original Image')
subplot(1,2,2), imshow(I5)
title('Despeckled Image')
```

to see the result of despeckling the image `I1` (Fig. 8.2).

8.5 Importing and Georeferencing TERRA ASTER Images

In Sect. 8.3 we used a processed ASTER image that we downloaded from the ASTER webpage. In this section we will use raw data from this sensor. The ASTER sensor is mounted on the TERRA satellite launched in 1999, part of the Earth

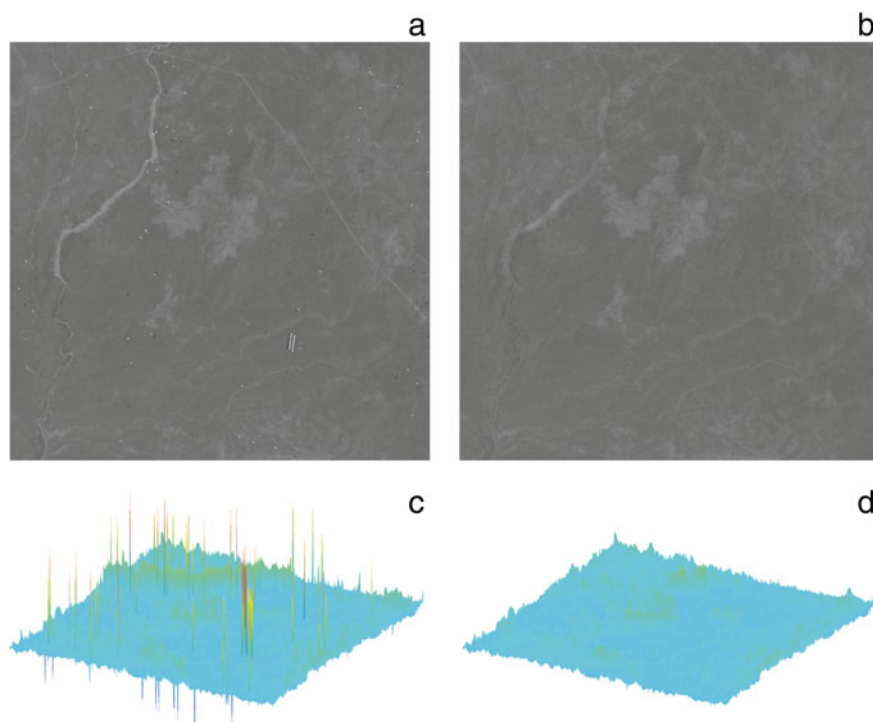


Fig. 8.2 Despeckled section of the blue band of a Landsat image covering the Chew Bahir catchment in southern Ethiopia: **a** original image, **b** despeckled image, **c** surface plots of the original image, and **d** surface plot of the image after despeckling. Original image courtesy of the Landsat Program of the US National Aeronautics and Space Administration (NASA) and the US Geological Survey (USGS).

Observing System (EOS) series of multi-national NASA satellites (Abrams and Hook 2002). ASTER stands for *Advanced Spaceborne Thermal Emission and Reflection Radiometer*, providing high-resolution (15–90 meters) images of the Earth in 14 bands, including three visible to near-infrared bands (VNIR bands 1–3), six short-wave infrared bands (SWIR bands 4–9), and five thermal (or long-wave) infrared bands (TIR bands 10–14). ASTER images are used to map the surface temperature, emissivity, and reflectance of the Earth’s surface. The 3rd VNIR band is recorded twice: once with the sensor pointing directly downwards (band 3N, where *N* stands for *nadir* from the Arabic word for *opposite*), as it does for all other channels, and a second time with the sensor angled backwards at 27.6° (band 3B, where *B* stands for backward looking). These two bands are used to generate ASTER digital elevation models (DEMs).

The ASTER instrument produces two types of data: Level-1A (L1A) and Level-1B (L1B) data (Abrams and Hook 2002). Whereas the L1A data are reconstructed, unprocessed instrument data, the L1B data are radiometrically and geometrically corrected. Any data that ASTER has already acquired are available; they can be obtained from the webpage

<https://earthexplorer.usgs.gov>

As an example we process an image from an area in Kenya showing Lake Naivasha (0°46'31.38" S 36°22'17.31" E). The Level-1A data are stored in two files

```
AST_L1A_003_03082003080706_03242003202838.hdf
AST_L1A_003_03082003080706_03242003202838.hdf.met
```

The first file (116 MB) contains the actual raw data while the second file (102 KB) contains the header, together with all sorts of information about the data. We save both files in our working directory. Since the file name is very long, we first save it in the `filename` variable and then use `filename` instead of the long file name. We then need to modify only this single line of MATLAB code if we want to import and process other satellite images.

```
clear, close all, clc

filename = 'AST_L1A_003_03082003080706_03242003202838.hdf';
```

The Image Processing Toolbox contains various tools for importing and processing files stored in the hierarchical data format (HDF). The graphical user interface (GUI) based import tool for importing certain parts of the raw data is

```
hdftool('filename')
```

This command opens a GUI that allows us to browse the content of the HDF-file, which obtains all information on the contents and imports certain frequency bands of the satellite image. Unfortunately, `hdftool` has been removed and causes an error message. The information that was previously provided by the function `hdftool` can now be obtained with the function `hdfinfo`

```
hdfinfo('filename')
```

which yields

```
info =

struct with fields:

    Filename: 'naivasha.hdf'
    Attributes: [1×10 struct]
    Vgroup: [1×4 struct]
    SDS: [1×1 struct]
```

We can use

```
info.Vgroup
info.Vgroup.Name
info.Vgroup(2).Vgroup.Name
info.Vgroup(2).Vgroup(1).Vgroup.Name
info.Vgroup(2).Vgroup(1).Vgroup(2).SDS.Name
```

to dive into the hierarchical structure of the file to find out the names of the data fields that we want to import. We then use `hdfread` as a quicker way of accessing image data. The *vnir_Band3n*, *vnir_Band2*, and *vnir_Band1* typically contain much information about lithology (including soils), vegetation, and surface water on the Earth's surface. These bands are therefore usually combined into 24-bit RGB images. We first read the data

```
I1 = hdfread(filename, 'VNIR_Band3N', 'Fields', 'ImageData');
I2 = hdfread(filename, 'VNIR_Band2', 'Fields', 'ImageData');
I3 = hdfread(filename, 'VNIR_Band1', 'Fields', 'ImageData');
```

These commands generate three 8-bit image arrays, each representing the intensity within a certain infrared (IR) frequency band of a 4200-by-4100 pixel image. We are not using the data for quantitative analyses and therefore do not need to convert the digital number (DN) values into radiance and reflectance values. The *ASTER User Handbook* provides the necessary information on these conversions (Abrams and Hook 2002). Instead, we will process the ASTER image to create a georeferenced RGB composite of bands 3N, 2 and 1, to be used in fieldwork. We first use a contrast-limited adaptive histogram equalization method to enhance the contrast in the image by typing

```
I1 = adapthisteq(I1);
I2 = adapthisteq(I2);
I3 = adapthisteq(I3);
```

and then concatenate the result to a 24-bit RGB image using `cat`.

```
naivasha_rgb = cat(3,I1,I2,I3);
```

As with the previous examples, the 4200-by-4100-by-3 array can now be displayed using

```
imshow(naivasha_rgb, 'InitialMagnification', 10)
```

We set the initial magnification of this very large image to 10%. MATLAB scales images to fit the computer screen. Exporting the processed image from the Figure Window, we only save the image at the monitor's resolution. To obtain an image at a higher resolution, we use the command

```
imwrite(naivasha_rgb, 'naivasha.tif', 'tif')
```

This command saves the RGB composite as a TIFF-file *naivasha.tif* (ca. 52 MB) in the working directory, which can then be processed using other software such as Adobe Photoshop. The processed ASTER image does not yet have a coordinate system and therefore needs to be tied to a geographical reference frame (*georeferencing*). The header of the HDF file can be used to extract the geodetic coordinates of the four corners of the image contained in *scenefourcorners*. We collect the coordinates of the four scene corners into a single array *movingpoints*:

```
movingpoints(1,:) = [36.214332 -0.319922]; % upper left corner
movingpoints(2,:) = [36.096003 -0.878267]; % lower left corner
movingpoints(3,:) = [36.770406 -0.400443]; % upper right corner
movingpoints(4,:) = [36.652213 -0.958743]; % lower right corner
```

It is important to note that the coordinates contained in *productmetadata.0* need to be flipped in order to have $x = \text{longitudes}$ and $y = \text{latitudes}$. The four corners of the image correspond to the pixels in the four corners of the image, which we store in a variable named *fixedpoints*.

```
fixedpoints(1,:) = [1,1]; % upper left pixel
fixedpoints(2,:) = [1,4200]; % lower left pixel
fixedpoints(3,:) = [4100,1]; % upper right pixel
fixedpoints(4,:) = [4100,4200]; % lower right pixel
```

The function *fitgeotrans* now takes the pairs of control points, *movingpoints* and *fixedpoints*, and uses them to infer a spatial transformation matrix *tform*. We use an affine transformation using *affine* which preserves points, straight lines and planes, and parallel lines remain parallel after transforming the image.

```
tform = fitgeotrans(movingpoints, fixedpoints, 'affine');
```

We next determine the limits of the input for georeferencing (i.e. of the original image *naivasha_rgb*) using *size*, which yields *xLimitsIn* and *yLimitsIn*. Adding a value of 0.5 to both *xLimitsIn* and *yLimitsIn* prevents the edges of the image from being truncated during the affine transformation. We then determine the limits of the output (i.e. of the georeferenced image, which is subsequently called *new-naivasha_rgb*) using *outputLimits*, which yields *XBounds* and *YBounds*.


```
xLimitsIn = 0.5 + [0 size(naivasha_rgb,2)];
yLimitsIn = 0.5 + [0 size(naivasha_rgb,1)];
[XBounds,YBounds] = outputLimits(tform,xLimitsIn,yLimitsIn);
```

We then use `imref2d` to reference the image to a world (or global) coordinate system.

```
Rout = imref2d(size(naivasha_rgb),XBounds,YBounds);
```

An `imref2d` object encapsulates the relationship between the intrinsic coordinates anchored to the rows and columns of the image, and the spatial location of the same row and column locations within a world coordinate system. Finally, the affine transformation can be applied to the original RGB composite `naivasha_rgb` in order to obtain a georeferenced version of the satellite image `newnaivasha_rgb` with the same size as `naivasha_rgb`.

```
newnaivasha_rgb = imwarp(naivasha_rgb,tform,'OutputView',Rout);
```

An appropriate grid for the image can now be computed. The grid is typically defined by the minimum and maximum values for the longitude and latitude. The vector increments are then obtained by dividing the ranges of the longitude and latitude by the array's dimensions and then subtracting one from the results. Note the difference between the MATLAB numbering convention and the common coding of maps used in published literature. The north/south suffix is generally replaced by a negative sign for south, whereas MATLAB coding conventions require negative signs for north.

```
X = 36.096003 : (36.770406 - 36.096003)/4100 : 36.770406;
Y = -0.958743 : ( 0.958743 - 0.319922)/4200 : -0.319922;
```

The georeferenced image is displayed with coordinates on the axes and a superimposed grid (Fig. 8.3). By default, the function `imshow` inverts the latitude axis when images are displayed by setting the `YDir` property to `Reverse`. To invert the latitude axis direction back to normal, we need to set the `YDir` property to `Normal` by typing

```
imshow(newnaivasha_rgb,...
    'XData',X,...
    'YData',Y,...
    'InitialMagnification',10)
axis on, grid on, set(gca,'YDir','Normal')
xlabel('Longitude'), ylabel('Latitude')
title('Georeferenced ASTER Image')
```

Exporting the image is possible in many different ways, for example using

```
print -djpeg70 -r600 naivasha_georef.jpg
```

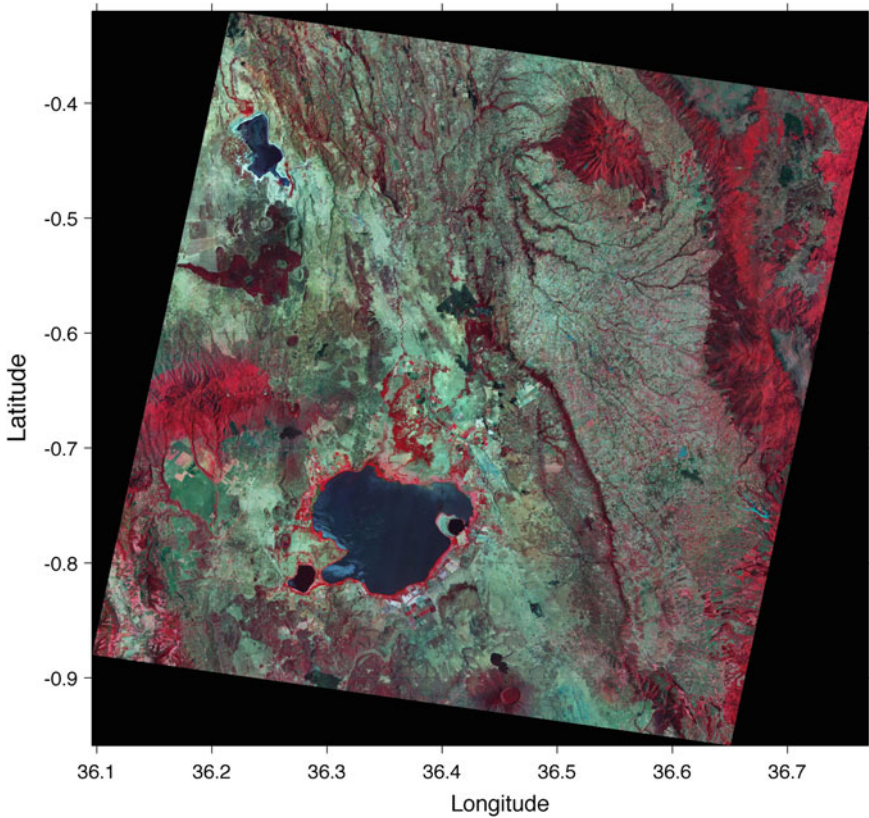


Fig. 8.3 Georeferenced RGB composite of a TERRA-ASTER image using the infrared bands *vnir_Band3n*, 2 and 1. The result is displayed using `imshow`. Original image courtesy of NASA/GSFC/METI/ERSDAC/JAROS and U.S./Japan ASTER Science Team.

to export it as a JPEG file *naivasha_georef.jpg*, compressed to 70% and with a resolution of 600 dpi.

In the previous example we used the geodetic coordinates of the four corners to georeference the ASTER image. The Image Processing Toolbox also includes functions to automatically align two images that are shifted and/or rotated with respect to each other, cover slightly different areas, or have different resolutions. We use two ASTER images of the Suguta Valley in the Northern Kenya Rift as an example. The images have been processed in the same way as described for the image of Lake Naivasha and exported as TIFF files using `imwrite`. The image in the file *sugutavalley_1.tif* was acquired on 20th February 2003 and the second image, in *sugutavalley_2.tif*, was acquired on 31st August 2003, both just after 8 o'clock in the morning. Lake Logipi, in the center of the images, is much larger in the second image than in the first image. The original images are otherwise almost identical,

except for the second image being shifted slightly towards the east. To demonstrate the automatic alignment of the images, the second image has been rotated counterclockwise by five degrees. Furthermore, both images have been cropped to the area of the Suguta Valley, including a small section of the rift shoulders to the west and to the east. We import both images using

```
clear

image1 = imread('sugutavalley_1.tif');
image2 = imread('sugutavalley_2.tif');
```

The size of `image1` is 666-by-329-by-3, while `image2` is slightly smaller: 614-by-270-by-3. We display the images by typing

```
subplot(1,2,1), imshow(image1)
subplot(1,2,2), imshow(image2)
```

The function `imregconfig` creates optimizer and metric configurations that we transfer into `imregister` to perform intensity-based image registration,

```
[optimizer, metric] = imregconfig('monomodal');
```

where `monomodal` assumes that the images were captured by the same sensor. We can use these configurations to calculate the spatial transformation matrix `tform` using the transformation type `affine`, as in the previous example.

```
tform = imregtform(image2(:,:,1), image1(:,:,1), ...
    'affine', optimizer, metric);
```

This transformation can be applied to `image2` in order to automatically align it with `image1`.

```
image2_reg = imwarp(image2, tform, 'OutputView', ...
    imref2d(size(image1)));
```

We can compare the result with the original images using

```
subplot(1,3,1), imshow(image1)
subplot(1,3,2), imshow(image2)
subplot(1,3,3), imshowpair(image1, image2_reg, 'blend')

print -djpeg70 -r600 sugutavalley_aligned.jpg
```

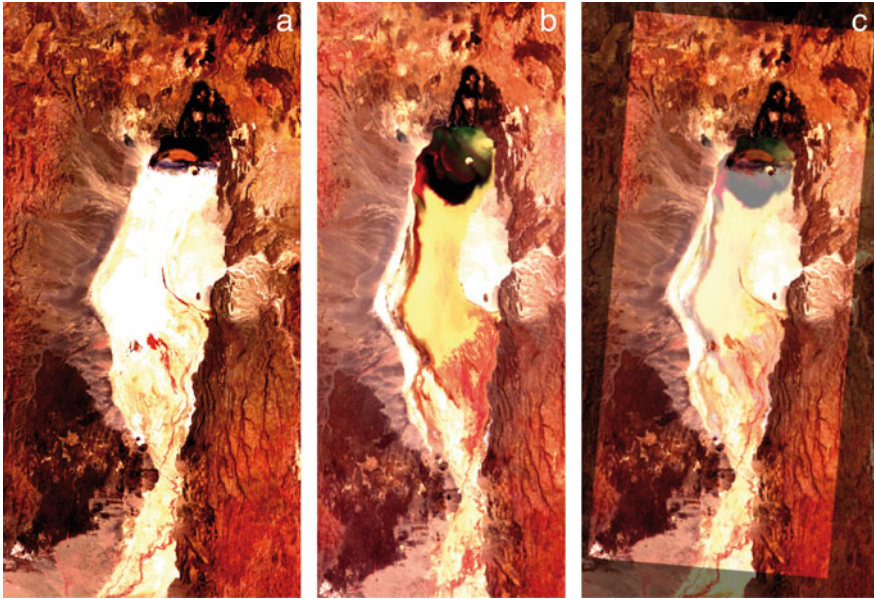


Fig. 8.4 Automatically aligned TERRA-ASTER images of the Suguta Valley in the Northern Kenya Rift: **a** first image taken on 20th February 2003, **b** second image taken on 31st August 2003, and **c** overlay of the second image aligned with the first image. Original image courtesy of NASA/GSFC/METI/ERSDAC/JAROS and U.S./Japan ASTER Science Team.

As we can see, the second image is now nicely aligned with the first image (Fig. 8.4). The two images can now be used to map changes in the area (e.g. in the size of the lake) between 20th February and 31st August 2003. This script can also be used to automatically align other images, in particular those captured by different sensors.

8.6 Processing and Exporting EO-1 Hyperion Images

The Earth Observing-1 Mission (EO-1) satellite was part of the New Millennium Program of the US National Aeronautics and Space Administration (NASA) and the US Geological Survey (USGS), which began with the launch of this satellite on 21 November 2000 and was deactivated on 30 March 2017. EO-1 has two sensors: the Advanced Land Image (ALI) has nine multispectral bands with a 30 meters

spatial resolution and a panchromatic band with a 10-m resolution, and the hyperspectral sensor (Hyperion) has 220 bands between 430 and 2,400 nm (Mandl et al. 2002; Lein 2012). General information about the EO-1 program can be obtained from the webpage

<http://eo1.gsfc.nasa.gov>

Hyperion data (together with data from of other NASA satellites) are freely available from the webpage

<https://earthexplorer.usgs.gov>

On this webpage we first select the desired map section in the *Search Criteria*, either by entering the coordinates of the four corners of the map or by zooming into the area of interest and selecting *Use Map*. As an example we enter the coordinates 2°8'37.58" N 36°33'47.06" E for the center of the Suguta Valley in the Northern Kenya Rift. We then choose *Hyperion* from the *EO-1* collection as the *Data Set* and click *Results*. Clicking *Results* produces a list of records, together with a toolbar for previewing and downloading data. Clicking the *Show Browse Overlay* button allows us to examine the images for cloud cover. We find the cloud-free image

```
Entity ID: E01H1690582013197110KF_PF2_01
Acquisition Date: 16-JUL-13
Target Path: 169
Target Row: 58
```

taken on 16th July 2013. As before, we need to register with the USGS website, log on, and then download the radiometrically corrected (but not geometrically corrected) Level 1R (L1R) product (215.3 MB), which is then stored on the hard drive in the file *LEO1H1690582013197110KF_1R.ZIP*. The *.ZIP* archive consists of a metadata file (*.MET*), a Federal Geographic Data Committee (FGDC) metadata file (*.fgdc*), an HDF data set file (*.L1R*), and multiple auxiliary files. The EO-1 User's Guides provides some useful information on the data formats of these files (Barry 2001; Beck 2003). We can import the data from the *EO1H1690582013197110KF.L1R* file using

```
clear, close all, clc

HYP = hdfread('E01H1690582013197110KF.L1R', ...
    '/E01H1690582013197110KF.L1R', ...
    'Index', {[1 1 1],[1 1 1],[3189 242 256]});
```

The parameter `Index` is a three-element cell array, `{start, stride, edge}`, specifying the location, range, and values to be read from the data set. The value of `start` specifies the position in the file at which to start reading. In our example it specifies to start reading at the beginning of the file, i.e. starting from the `[1 1 1]` element. The value of `stride` defines the interval between the values to be read, which in our example is `[1 1 1]`, i.e. every element of the data set is to be read. The value of `edge` specifies the size of the data in the file; in our example the dimensions of the data are 3,189-by-242-by-256. Typing

```
whos
```

shows how the hyperspectral image is stored in the workspace:

Name	Size	Bytes	Class	Attributes
HYP	3189x242x256	395129856	int16	

The details indicate that the image is stored as a 3,189-by-242-by-256 array, representing a 3,189-by-256 array for each of the 242 spectral bands. The listing of the current variables in the workspace also gives the information *int16* array, i.e. each array element representing one pixel contains signed 16-bit integers. We need to permute the array to move the bands to the third dimension by typing

```
HYP = permute(HYP, [1 3 2]);
```

We next need to determine the radiance values from the digital number (DN) values in `HYP`. The radiance is the power density scattered from the Earth in a particular direction and has the units of watts per square meter per steradian ($\text{Wm}^{-2} \text{sr}^{-1}$) (Richards 2013). The EO-1 User Guide (v. 2.3) provides the necessary information on these conversions in its *Frequently Asked Questions* (FAQ) section (Beck 2003). According to this document, the radiance `HYPR` for the visible and near-infrared (VNIR) bands (bands 1–70) is calculated by dividing the digital number in `HYP` by 40. The radiance for the shortwave infrared (SWIR) bands (bands 71–242) is calculated by dividing `HYP` by 80.

```
HYPR(:, :, 1:70) = HYP(:, :, 1:70)/40;
HYPR(:, :, 71:242) = HYP(:, :, 71:242)/80;
```

For quantitative analyses, the radiance values `HYPR` need to be corrected for atmospheric and topographic effects. This correction, which yields earth surface reflectance values (in percentages), is beyond the scope of the book. The EO-1 User

Guide (v. 2.3) again explains several methods to convert radiance values to reflectance values (Beck 2003).

We will instead process the Hyperion image to create a georeferenced RGB composite of bands 29, 23 and 16, to be used in fieldwork. The header file *O1H1690582013197110KF.HDR* contains (among other things) the wavelengths corresponding to the 242 spectral bands. We can read the wavelengths from the file using `textscan`:

```
fid = fopen('E01H1690582013197110KF.hdr');
C = textscan(fid, '%f %f %f %f %f %f %f %f', ...
    'Delimiter', ',', ...
    'Headerlines', 257, ...
    'CollectOutput', 1)
fclose(fid);
```

This script opens the header file for read only access using `fopen` and defines the file identifier `fid`, which is then used to read the text from the file with `textscan` and to write it into the array `C`. The character string `%f %f %f %f %f %f %f %f` enclosed in single quotation marks defines the conversion specifiers, where `%f` stands for the double-precision floating-point 64-bit output class. The parameter `Headerlines` is set to 257, which means that the first 257 lines are ignored when reading the file. If the parameter `CollectOutput` is 1 (true), `textscan` concatenates output cells of the same data type into a single array. Function `fclose` closes the file defined by the file identifier `fid`. The array `C` is a cell array, which is a data type with indexed containers called cells. We can easily obtain the wavelengths from `C` using

```
wavelengths = C{1};
wavelengths = wavelengths';
wavelengths = wavelengths(isnan(wavelengths(:))==0);
```

We can now plot the radiance `HYPR` of the VNIR bands (in blue) and SWIR bands (in red) in a single plot.

```
plot(wavelengths(1:60), squeeze(HYPR(536, 136, 1:60)), 'b', ...
    wavelengths(71:242), squeeze(HYPR(536, 136, 71:242)), 'r')
```

According to v. 2.3 of the EO-1 User Guide (Beck 2003), Hyperion records 220 unique spectral channels collected from a complete spectrum covering 357–2,576 nm. The L1R product has 242 bands but only 198 bands are calibrated. Because of an overlap between the VNIR and SWIR focal planes, there are only 196 unique

channels. Calibrated channels are 8–57 for the VNIR, and 77–224 for the SWIR. The bands that are not calibrated are set to zero in those channels.

In order to create an RGB composite of bands 29, 23 and 16, we can extract the bands from the radiance values data HYP1 by typing

```
HYP1 = HYP1(:, :, 29);
HYP2 = HYP1(:, :, 23);
HYP3 = HYP1(:, :, 16);
```

To display the data with `imshow` we need to convert the signed integer 16-bit (`int16`) data to unsigned integer 8-bit data (`uint8`). For this purpose, we first obtain an overview of the range of the data using a histogram plot with 100 classes.

```
subplot(1,3,1), histogram(double(HYP1(:)),100), title('Band 29')
subplot(1,3,2), histogram(double(HYP2(:)),100), title('Band 23')
subplot(1,3,3), histogram(double(HYP3(:)),100), title('Band 16')
```

As we see, the radiance values of most pixels from the spectral bands 29, 23 and 16 lie between 0 and $200 \text{ Wm}^{-2} \text{ sr}^{-1}$. Several functions are available for converting the data from `int16` to `uint8`. The function `im2uint8` rescales the data to the range [0, 255] and offsets it if necessary. The function `uint8` simply rounds all values in the range [0, 255] to the nearest integer; all values less than 0 are set to 0 and all values greater than 255 are set to 255. The function `mat2gray(A, [AMIN AMAX])` converts an arbitrary array `A` to an intensity image `I` containing values in the range 0 (black) to 1.0 (full intensity or white), where `[AMIN AMAX]` can be used to limit the range of the original data, which is scaled to the range [0, 1]. Since most of our radiance values are within the range [0, 255], we use `uint8` to convert our data to the `uint8` data type without losing much information.

```
HYP1 = uint8(HYP1);
HYP2 = uint8(HYP2);
HYP3 = uint8(HYP3);
```

Again, displaying the radiance values of the three bands in a histogram using

```
subplot(1,3,1), histogram(single(HYP1(:)),30), title('Band 29')
subplot(1,3,2), histogram(single(HYP2(:)),30), title('Band 23')
subplot(1,3,3), histogram(single(HYP3(:)),30), title('Band 16')
```

reveals that most radiance values are actually within the range [20, 80]. Instead of using `histogram` we can also use `imhist` to display the `uint8` data.


```
subplot(1,3,1), imhist(HYP1(:)), title('Band 29')  
subplot(1,3,2), imhist(HYP2(:)), title('Band 23')  
subplot(1,3,3), imhist(HYP3(:)), title('Band 16')
```

We then use `histeq` to enhance the contrast in the image and concatenate the three bands to a 3,189-by-242-by-3 array.

```
HYP1 = histeq(HYP1);  
HYP2 = histeq(HYP2);  
HYP3 = histeq(HYP3);  
  
subplot(1,3,1), imhist(HYP1(:)), title('Band 29')  
subplot(1,3,2), imhist(HYP2(:)), title('Band 23')  
subplot(1,3,3), imhist(HYP3(:)), title('Band 16')  
  
HYPC = cat(3,HYP1,HYP2,HYP3);
```

Finally, we can display the entire image using

```
imshow(HYPC)
```

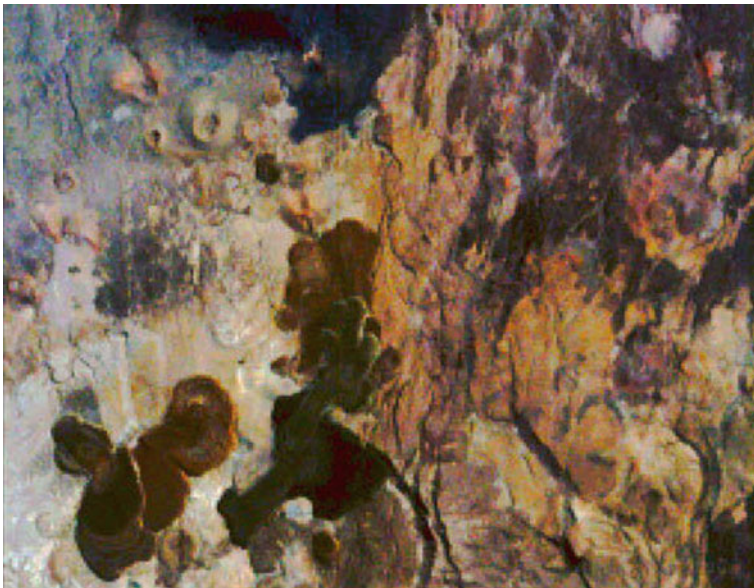


Fig. 8.5 RGB composite of an EO-1 Hyperion image using VNIR bands 29, 23 and 16, showing the Barrier Volcanic Complex in the Suguta Valley of the Northern Kenya Rift. The image was acquired on 16th July 2013. Original image courtesy of NASA EO-1 Mission.

or, as an alternative, just that part of the image covering the Barrier Volcanic Complex in the northern Suguta Valley (Fig. 8.5).

```
imshow(HYPC(900:1100, :, :))
```

Exporting the image is possible in many different ways, for example using

```
print -r600 -dtiff barrier.tif
```

to export it as a TIFF file *barrier.tif* with a resolution of 600 dpi.

8.7 Digitizing from the Screen

On-screen digitizing is a widely-used image processing technique. While practical digitizer tablets exist in all formats and sizes, most people prefer digitizing vector data from the screen. Examples of this type of application include the digitizing of river networks and catchment areas on topographic maps, of the outlines of lithologic units on geological maps, of landslide distributions on satellite images, and of mineral grain distributions in microscope images. The digitizing procedure consists of the following steps. The image is first imported into the workspace. A coordinate system is then defined, allowing the objects of interest to be entered by moving a cursor or cross hair onto it and clicking the mouse button. The result is a two-dimensional array of *xy* data, such as longitudes and latitudes of the corner points of a polygon, or the coordinates of the objects of interest in a particular area.

The function `ginput` included in the standard MATLAB toolbox allows graphical input from the screen, using a mouse. It is generally used to select points, such as specific data points, from a figure created by an arbitrary graphics function such as `plot`. The function `ginput` is often used for interactive plotting, i.e. the digitized points appear on the screen after they have been selected. The disadvantage of the function is that it does not provide coordinate referencing on an image. We therefore use a modified version of the function, which allows an image to be referenced to an arbitrary rectangular coordinate system. Save the following code for this modified version of the function `ginput` in a text file *minput.m*.

```
function data = minput(imagefile)
% Specify the limits of the image
xmin = input('Specify xmin! ');
xmax = input('Specify xmax! ');
ymin = input('Specify ymin! ');
ymax = input('Specify ymax! ');
```

```
% Read image and display
B = imread(imagefile);
a = size(B,2); b = size(B,1);
imshow(B);

% Define lower left and upper right corner of image
disp('Click on lower left and upper right corner, then <return>')
[xcr,ycr] = ginput;
XMIN = xmin-((xmax-xmin)*xcr(1,1)/(xcr(2,1)-xcr(1,1)));
XMAX = xmax+((xmax-xmin)*(a-xcr(2,1))/(xcr(2,1)-xcr(1,1)));
YMIN = ymin-((ymax-ymin)*ycr(1,1)/(ycr(2,1)-ycr(1,1)));
YMAX = ymax+((ymax-ymin)*(b-ycr(2,1))/(ycr(2,1)-ycr(1,1)));

% Digitize data points
disp('Click on data points to digitize, then <return>')
[xdata,ydata] = ginput;
XDATA = XMIN + ((XMAX-XMIN)*xdata/size(B,2));
YDATA = YMIN + ((YMAX-YMIN)*ydata/size(B,1));
data(:,1) = XDATA; data(:,2) = YDATA;
```

The function `minput` has four stages. In the first stage the user enters the limits of the coordinate axes as reference points for the image. The image is then imported into the workspace and displayed on the screen. The third stage uses `ginput` to define the upper left and lower right corners of the image. In the fourth stage the relationship between the coordinates of the two corners on the figure window and the reference coordinate system is then used to compute the transformation for all of the digitized points.

As an example we use the image stored in the file *naivasha_georef.jpg* and digitize the outline of Lake Naivasha in the center of the image. We activate the new function `minput` from the Command Window using the commands

```
clear

data = minput('naivasha_georef.jpg')
```

The function first asks for the coordinates of the limits of the *x*-axis and the *y*-axis, for the reference frame. We enter the corresponding numbers and press *return* after each input.

```
Specify xmin! 36.1
Specify xmax! 36.7
Specify ymin! -1
Specify ymax! -0.3
```

The function then reads the file *naivasha_georef.jpg* and displays the image. We ignore the warning

```
Warning: Image is too big to fit on screen; displaying at 33%
```

and wait for the next response

```
Click on lower left and upper right corner, then <return>
```

The image window can be scaled according to user preference. Clicking on the lower left and upper right corners defines the dimensions of the image. These changes are registered by pressing *return*. The routine then references the image to the coordinate system and waits for the input of the points we wish to digitize from the image.

```
Click on data points to digitize, then <return>
```

We finish the input by again pressing *return*. The *xy* coordinates of our digitized points are now stored in the variable *data*. We can now use these vector data for other applications.

8.8 Image Enhancement, Correction and Rectification

This section introduces some fundamental tools for image enhancement, correction and rectification. As an example we use an image of varved sediments deposited around 33 kyrs ago in a landslide-dammed lake in the Quebrada de Cafayate of Argentina (25°58.900' S 65°45.676' W) (Trauth and Strecker 1999; Trauth et al. 2003). The diapositive was taken on 1st October 1996 with a film-based single-lens reflex (SLR) camera. A 30-by-20 cm print was made from the slide, which has been scanned using a flatbed scanner and saved as a 394 KB JPEG file. We use this as an example because it demonstrates some problems that we can solve with the help of image enhancement (Fig. 8.6). We then use the image to demonstrate how to measure color-intensity transects for use in time series analysis (Sect. 8.9).

We can read and decompress the file *varves_original.jpg* by typing

```
clear, close all, clc
```

```
I1 = imread('varves_original.jpg');
```

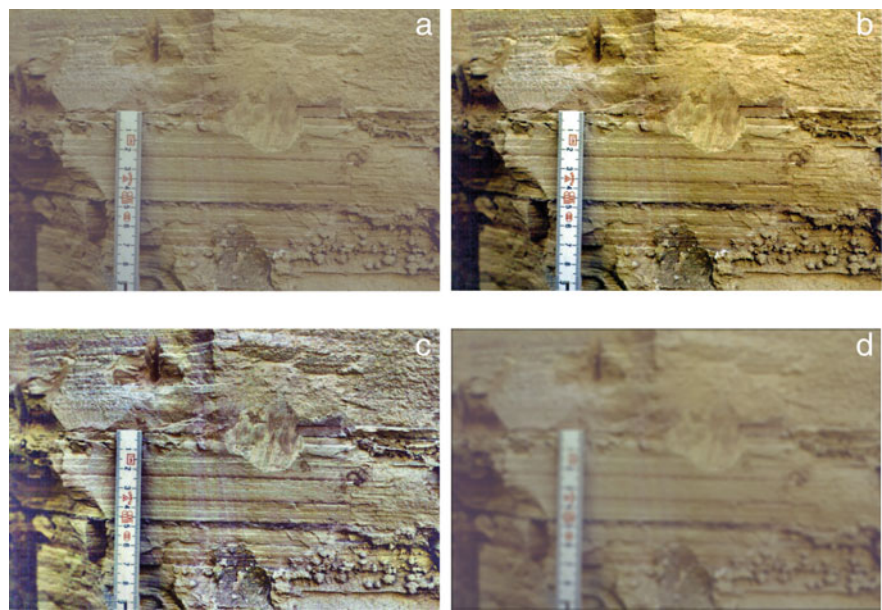


Fig. 8.6 Results of image enhancements: **a** original image, **b** image with intensity values adjusted using `imadjust` and `gamma=1.5`, **c** image with contrast enhanced using `adapthisteq`, **d** image after filtering with a 20-by-20 pixel filter with the shape of a Gaussian probability density function with a mean of zero and a standard deviation of 10, using `fspecial` and `imfilter`.

which yields a 24-bit RGB image array `I1` in the MATLAB workspace. Typing

```
whos

yields

Name      Size      Bytes    Class    Attributes

I1      1096x1674x3  5504112  uint8
```

revealing that the image is stored as an `uint8` array of the size 1,096-by-1,674-by-3, i.e. 1,096-by-1,674 arrays for each color (red, green and blue). We can display the image using the command

```
imshow(I1)
```

which opens a new Figure Window showing an RGB composite of the image. As we see, the image has a low level of contrast and very pale colors, and the sediment layers are not exactly horizontal. These are characteristics of the image that we want to improve in the following steps.

First, we adjust the image intensity values or colormap. The function `imadjust(I1,[li;hi],[lo ho])` maps the values of the image `I1` to new values in `I2`, such that values between `li` and `hi` are adjusted to values between `lo` and `ho`. Values below `li` and above `hi` are clipped, i.e. these values are adjusted to `lo` and `ho`, respectively. We can determine the range of the pixel values using

```
lh = stretchlim(I1)
```

which yields

```
lh =
    0.3255    0.2627    0.2784
    0.7020    0.7216    0.7020
```

indicating that the red color ranges from 0.3255 to 0.7020, green ranges from 0.2627 to 0.7216, and blue ranges from 0.2784 to 0.7020. We can utilize this information to automatically adjust the image with `imadjust` by typing

```
I2 = imadjust(I1,lh,[]);
```

which adjusts the ranges to the full range of `[0, 1]`, and then display the result.

```
imshow(I2)
```

We can clearly see the difference between the very pale image `I1` and the more saturated image `I2`. The parameter `gamma` in `imadjust(I1,[li;hi],[lo;ho],gamma)` specifies the shape of the curve describing the relationship between `I1` and `I2`. If `gamma<1` the mapping is weighted toward higher (brighter) output values. If `gamma>1` the mapping is weighted toward lower (darker) output values. The default value of `gamma=1` causes linear mapping of the values in `I1` to new values in `I2`.

```
I3 = imadjust(I1,lh,[],0.5);
I4 = imadjust(I1,lh,[],1.5);

subplot(2,2,1), imshow(I1)
title('Original Image')
subplot(2,2,2), imshow(I2)
title('Adjusted Image,Gamma=1.0')
```

```
subplot(2,2,3), imshow(I3)
title('Adjusted Image,Gamma=0.5')
subplot(2,2,4), imshow(I4)
title('Adjusted Image,Gamma=1.5')
```

We can use `imhist` to display a histogram showing the distribution of intensity values for the image. Since `imhist` only works for two-dimensional images, we examine the histogram of the red color only.

```
subplot(2,2,1), imhist(I1(:, :, 1))
title('Original Image')
subplot(2,2,2), imhist(I2(:, :, 1))
title('Adjusted Image, Gamma=1.0')
subplot(2,2,3), imhist(I3(:, :, 1))
title('Adjusted Image, Gamma=0.5')
subplot(2,2,4), imhist(I4(:, :, 1))
title('Adjusted Image, Gamma=1.5')
```

The result obtained using `imadjust` differs from that obtained using `histeq` (which we used in Sect. 8.3 to enhance the contrast in the image). The function `histeq(I1, n)` transforms the intensity of image `I1`, returning in `I5` an intensity image with `n` discrete levels. A roughly equal number of pixels is ascribed to each of the `n` levels in `I5`, so that the histogram of `I5` is approximately flat. Since `histeq` only works for two-dimensional images, histogram equalization (using `histeq`) has to be carried out separately for each color. We use `n = 50` in our exercise, which is slightly below the default value of `n = 64`.

```
I5(:, :, 1) = histeq(I1(:, :, 1), 50);
I5(:, :, 2) = histeq(I1(:, :, 2), 50);
I5(:, :, 3) = histeq(I1(:, :, 3), 50);
```

```
subplot(2,2,1), imshow(I1)
title('Original Image')
subplot(2,2,3), imhist(I1(:, :, 1))
title('Original Image')
subplot(2,2,2), imshow(I5)
title('Enhanced Image')
subplot(2,2,4), imhist(I5(:, :, 1))
title('Enhanced Image')
```

The resulting image looks quite disappointing and we therefore use the improved function `adapthisteq` instead of `histeq`. This function uses the contrast-limited adaptive histogram equalization (CLAHE) by Zuiderveld (1994). Unlike `histeq` and `imadjust`, the algorithm works on small regions (or tiles) of the image, rather

than on the entire image. The neighboring tiles are then combined using bilinear interpolation to eliminate edge effects.

```
I6(:,:,1) = adapthisteq(I1(:,:,1));
I6(:,:,2) = adapthisteq(I1(:,:,2));
I6(:,:,3) = adapthisteq(I1(:,:,3));

subplot(2,2,1), imshow(I1)
title('Original Image')
subplot(2,2,3), imhist(I1(:,:,1))
title('Original Image')
subplot(2,2,2), imshow(I6)
title('Enhanced Image')
subplot(2,2,4), imhist(I6(:,:,1))
title('Enhanced Image')
```

The result looks slightly better than that obtained using `histeq`. However, all three functions for image enhancement (`imadjust`, `histeq` and `adapthisteq`) provide numerous ways to manipulate the final outcome. The *Image Processing Toolbox—User's Guide* (MathWorks 2020) and the excellent book by Gonzalez and others (2009) provide more detailed introductions to the use of the various parameters available and the corresponding values of the image enhancement functions.

The Image Processing Toolbox also includes numerous functions for 2D filtering of images. Many of the methods we have looked at in Chap. 6 for one-dimensional data also work with two-dimensional data, as we have already seen in Chap. 7 when filtering digital terrain models. The most popular 2D filters for images are Gaussian filters and median filters, as well as filters for image sharpening. Both Gaussian and median filters are used to smooth an image, mostly with the aim of reducing the amount of noise. In most examples the signal-to-noise ratio is unknown and adaptive filters (similar to those introduced in Sect. 6.10) are therefore used for noise reduction. A Gaussian filter can be designed using

```
h = fspecial('gaussian',20,10);
I7 = imfilter(I1,h);
```

where `fspecial` creates predefined 2D filters such as moving average, disk, or Gaussian filters. The Gaussian filter weights `h` are calculated using `fspecial('gaussian',20,10)`, where 20 corresponds the size of a 20-by-20 pixel filter following the shape of a Gaussian probability density function with a standard deviation of 10. Next, we calculate `I8`, which is a median-filtered version of `I1`.


```
I8(:,:,1) = medfilt2(I1(:,:,1),[20 20]);  
I8(:,:,2) = medfilt2(I1(:,:,2),[20 20]);  
I8(:,:,3) = medfilt2(I1(:,:,3),[20 20]);
```

Since `medfilt2` only works for two-dimensional data, we again apply the filter separately to each color (red, green and blue). The filter output pixels are the medians of the 20-by-20 neighborhoods around the corresponding pixels in the input image.

The third filter example demonstrates sharpening an image using `imsharpen`.

```
I9 = imsharpen(I1);
```

This function calculates the Gaussian lowpass filtered version of the image, which is used as an unsharp mask, i.e. the sharpened version of the image is calculated by subtracting the blurred filtered version from the original image. The function comes with several parameters that control the ability of the Gaussian filter to blur the image and the strength of the sharpening effect, and a threshold specifying the minimum contrast required for a pixel to be considered an edge pixel and sharpened by unsharp masking. Comparing the results of the three filtering exercises with the original image

```
subplot(2,2,1), imshow(I1)  
title('Original Image')  
subplot(2,2,2), imshow(I7)  
title('Gaussian Filter')  
subplot(2,2,3), imshow(I8)  
title('Median Filter')  
subplot(2,2,4), imshow(I9)  
title('Sharpening Filter')
```

clearly demonstrates the effect of the 2D filters. As an alternative to these time-domain filters, we can also design 2D filters with a specific frequency response, such as the 1D filters described in Sect. 6.9. Again, the book by Gonzalez et al. (2009) provides an overview of 2D frequency-selective filtering for images, including functions used to generate such filters. The authors also demonstrate the use of a 2D Butterworth lowpass filter in image processing applications.

We next rectify the image, i.e. we correct the image distortion by transforming it to a rectangular coordinate system using a script that is similar to that used for georeferencing satellite images in Sect. 8.5. This we achieve by defining four points within the image, which are actually at the corners of a rectangular area (which is our reference area). We first define the *upper left*, *lower left*, *upper right*, and *lower right* corners of the reference area, and then press return. Note that it is important to

pick the coordinates of the corners in this particular order. In this instance we use the original image `I1`, but we could also use any other enhanced version of the image from the previous exercises. As an example we can click the left side of the ruler at 1.5 cm and at 4.5 cm, where two thin white sediment layers cross the ruler, for use as the upper-left and lower-left corners. We then choose the upper-right and lower-right corners, further to the right of the ruler but also lying on the same two white sediment layers,

```
close all
imshow(I1)
movingpoints = ginput
```

and click return which yields

```
movingpoints =

    517.0644    508.9059
    511.5396    733.5792
    863.2822    519.9554
    859.5990    739.1040
```

or similar values. The image and the reference points are then displayed in the same figure window.

```
close all
imshow(I1)
hold on
line(movingpoints(:,1),movingpoints(:,2),...
      'LineStyle','none',...
      'Marker','+',...
      'MarkerSize',48,...
      'Color','b')
hold off
```

We arbitrarily choose new coordinates for the four reference points, which are now on the corners of a rectangle. To preserve the aspect ratio of the image, we select numbers that are the means of the differences between the x - and y -values of the reference points in `movingpoints`.

```
dx = (movingpoints(3,1)+movingpoints(4,1))/2- ...
      (movingpoints(1,1)+movingpoints(2,1))/2
dy = (movingpoints(2,2)+movingpoints(4,2))/2- ...
      (movingpoints(1,2)+movingpoints(3,2))/2
```

which yields

```
dx =  
347.1386  
  
dy =  
221.9109
```

We therefore choose

```
fixedpoints(1,:) = [0 0];  
fixedpoints(2,:) = [0 dy];  
fixedpoints(3,:) = [dx 0];  
fixedpoints(4,:) = [dx dy];
```

The function `fitgeotrans` now takes the pairs of control points, `fixedpoints` and `movingpoints`, and uses them to infer a spatial transformation matrix `tform` using the transformation type `projective`.

```
tform = fitgeotrans(movingpoints,fixedpoints,'projective');
```

We next need to estimate the spatial limits for the outputs, `XBounds` and `YBounds`, corresponding to the projective transformation `tform`, and a set of spatial limits for the inputs `xLimitsIn` and `yLimitsIn`.

```
xLimitsIn = 0.5 + [0 size(I1,2)]  
yLimitsIn = 0.5 + [0 size(I1,1)]  
  
[XBounds,YBounds] = outputLimits(tform,xLimitsIn,yLimitsIn)
```

Then we use `imref2d` to reference the image to world coordinates.

```
Rout = imref2d(size(I1),XBounds,YBounds)
```

An `imref2d` object `Rout` encapsulates the relationship between the intrinsic coordinates anchored to the rows and columns of the image and the spatial location of the same row and column locations in a world coordinate system. Finally, the projective transformation can be applied to the original RGB composite `I1` in order to obtain a rectified version of the image (`I10`).

```
I10 = imwarp(I1,tform,'OutputView',Rout);
```

We now compare the original image I1 with the rectified version I10.

```
subplot(2,1,1), imshow(I1)
title('Original Image')
subplot(2,1,2), imshow(I10)
title('Rectified Image')
```

We see that the rectified image has black areas at the corners. We can remove these black areas by cropping the image using `imcrop`.

```
I11 = imcrop(I10);

subplot(2,1,1), imshow(I1)
title('Original Image')
subplot(2,1,2), imshow(I11)
title('Rectified and Cropped Image')
```

The function `imcrop` creates displays of the image with a resizable rectangular tool that can be interactively positioned and manipulated using the mouse. After manipulating the tool into the desired position, the image is cropped by either double clicking on the tool or choosing *Crop Image* from the tool's context menu. The result of our image enhancement experiment can now be used in the next section to analyze the colors of individual sediment layers.

8.9 Color-Intensity Transects Across Varved Sediments

High-resolution core logging has, since the early 1990s, become popular as an inexpensive tool for investigating the physical and chemical properties of marine and lacustrine sediments. During the early days of nondestructive core logging, magnetic susceptibility and grayscale intensity transects were measured on board research vessels to generate a preliminary stratigraphy of marine cores, since the cyclic recurrence of light and dark layers seemed to reflect glacial-interglacial cycles during the Pleistocene. Paleolimnologists adopted these techniques to analyze annually-layered (varved) lake sediments and to statistically detect short-term variabilities such as the 11 year sunspot cycle, the 3–7 year El Niño cycle, or the 78 year Gleissberg cycle. Modern multi-sensor core loggers are now designed to log a great variety of physical and chemical properties using optical scanners, radiograph imaging, and x-ray fluorescence elemental analyzers.

As an example we explore varved sediments deposited around 33 kyrs ago in a landslide-dammed lake in the Quebrada de Cafayate of Argentina (Trauth and Strecker 1999; Trauth et al. 2003). These lake sediments have been intensively studied for paleoclimate reconstructions since they document episodes of a wetter and more variable climate that may have fostered mass movements in the NW

Argentine Andes during the Late Pleistocene and Holocene. In addition to various sedimentological, geochemical and micropaleontological analyses, the colors of the sediments have also been studied as a proxy for rainfall intensities at the time of deposition. Color-intensity transects have been analyzed to detect interannual variations in precipitation caused by the El Niño/Southern Oscillation (ENSO, 3–7 year cycles) and the Tropical Atlantic Sea-Surface Temperature Variability (TAV, 10–15 year cycles) using linear and nonlinear methods of time-series analysis (e.g. Trauth et al. 2000; Marwan et al. 2003).

The El Paso section in the Quebrada de Cafayate contains well-developed annual layers in most parts of the profile (Fig. 8.7). The base of each of these mixed clastic and biogenic varves consists of reddish silt and clay, with a sharp lower boundary. Towards the top of the varves, reddish clay and silt are gradually replaced by light-brown to greenish clay. The change from reddish hues correlates with a slight decrease in grain size. This clastic portion of the varves is capped by a thin layer of pure white diatomite. Diatomites are sediments comprised mainly of porous siliceous skeletons of single-cell algae, i.e. diatoms. This internal structure of the laminae is typical of annual-layered sediments. The recurrence of these layers and the distribution of diatoms, together with the sediment coloration and provenance, all provide additional evidence that rhythmic sedimentation in this region was controlled by a well-defined annual cycle. The provenance of the sediments contained in the varved layers can be traced using index minerals characteristic of the

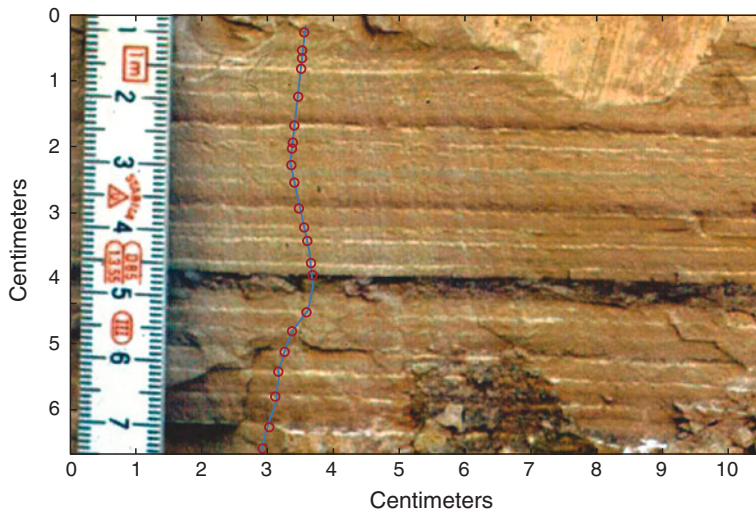


Fig. 8.7 Photograph of varved lake sediments from the Quebrada de Cafayate in the Santa Maria Basin, with cyclic occurrences of intense dark-red coloration reflecting enhanced precipitation and sediment input during ENSO- and TAV-type periodicities (350 cm above the base of the El Paso section). The solid blue line denotes the course of the digitized color-intensity transect. The red circles mark positions of the diatomite layers, representing annual layers.

various possible source areas within the catchment. A comparison of the mineral assemblages in the sediments with those of potential source rocks within the catchment area indicates that Fe-rich Tertiary sedimentary rocks exposed in the Santa Maria Basin were the source of the red-colored basal portion of the varves. In contrast, metamorphic rocks in the mountainous parts of the catchment area were the most likely source of the drab-colored upper part of the varves.

In nearly every second to fifth, and every tenth to fourteenth varve, respectively, the varve thickness increases by a factor of 1.5 to 2 and the basal reddish coloration is more intense, suggesting a greater fluvial input of Fe-rich Tertiary material. Exceptionally well-preserved sections containing 70–250 varves were used for more detailed statistical analysis of the observed cyclicities (see Chap. 5). High-quality photographs from these sections were scanned and subjected to standardized color and illumination corrections. Pixel-wide representative red-color intensities were subsequently extracted from transects across the images of these varves. The resolution of these time series was of the order of ten intensity values per varve.

We will now analyze a 22-year package of varved lake sediments from the Quebrada de Cafayate as an example (Fig. 8.6). The photo was taken during a field expedition in the late 1990s using an analog camera. It was then scanned and the contrast levels adjusted to heighten details using standard photo processing software. We import the image from the file *varves_original.tif* as a 24-bit RGB image array and store the data in a variable *I*.

```
clear, close all, clc

I = imread('varves_original');
```

We display the image using *imshow* and turn the axis labeling, tick marks and background back on.

```
imshow(I), axis on
```

The image is scaled to pixel indices or coordinates, so we first need to scale the image to a centimeter scale. While keeping the figure window open we use *ginput* to count the number of pixels per centimeter. The function *ginput* provides a crosshair with which to gather an unlimited number of points forming a polygon, until the *return* key is pressed. We place the crosshair at 1 and 6 cm on the scale in the image and click to gather the pixel coordinates of the 5-cm interval.

```
[x,y] = ginput;
```

The image is `size(I,2)=1830` pixels wide and `size(I,1)=1159` pixels high. We convert the width and the height of the image into centimeters using the conversion $5/\sqrt{((y(2,1)-y(1,1))^2+(x(2,1)-x(1,1))^2)}$, where 5 corresponds to the 5 cm

interval equivalent to the $\text{sqrt}((y(2,1)-y(1,1))^2+(x(2,1)-x(1,1))^2)$ pixels gathered using `ginput`.

```
ix = 5 * size(I,2) / sqrt((y(2,1)-y(1,1))^2+(x(2,1)-x(1,1))^2);
iy = 5 * size(I,1) / sqrt((y(2,1)-y(1,1))^2+(x(2,1)-x(1,1))^2);
```

We can now display the image using the new coordinate system where `ix` and `iy` are the width and height of the image, respectively (in centimeters).

```
imshow(I,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
```

We now digitize the color-intensity transect from the top of the image to bottom. The function `improfile` determines the RGB pixel values `C` along line segments defined by the coordinates `[CX,CY]`.

```
[CX,CY,C] = improfile;
```

The scaled image and the polygon are displayed in the same figure window. The three color-intensity curves are plotted in a separate window.

```
imshow(I,...
    'XData',[0 ix],...
    'YData',[0 iy]), hold on
plot(CX,CY), hold off

figure
plot(CY,C(:,1),'r',...
    CY,C(:,2),'g',...
    CY,C(:,3),'b')
xlabel('Centimeters')
ylabel('Intensity')
```

The image and the color-intensity profiles are on a centimeter scale. To detect the interannual precipitation variability, as recorded in the color intensity of the sediments, we need to convert the length scale to a time scale. We use the 22 white diatomite layers as time markers to define individual years in the sedimentary history. We again use `ginput` to mark the diatomite layers from top to bottom along the color-intensity transect and store the coordinates of the laminae in the new variable `laminae`.

```

imshow(I,'XData',[0 ix],...
       'YData',[0 iy]), hold on
plot(CX,CY), hold off
laminae = ginput;

```

To inspect the quality of the age model we plot the image, together with the polygon and the marked diatomite layers.

```

imshow(I,'XData',[0 ix],...
       'YData',[0 iy])
hold on
plot(CX,CY)
plot(laminae(:,1),laminae(:,2),'ro')
xlabel('Centimeters')
ylabel('Centimeters')
hold off

```

We define a new variable `newlaminae` that contains the vertical y-component of `laminae` as the first column and the years 1–22 (counting backwards in time) as the second column. The 22 years are equivalent to the length of `laminae`. The function `interp1` is used to interpolate the color-intensity transects over an evenly-spaced time axis stored in the variable `age`.

```

newlaminae(:,1) = laminae(:,2);
newlaminae(:,2) = 1 : length(laminae(:,2));
age = interp1(newlaminae(:,1),newlaminae(:,2),CY);

```

We complete the analysis by plotting the color-intensity curves on both a length and a time scale for comparison (Fig. 8.8).

```

subplot(2,1,1)
plot(CY,C(:,1),CY,C(:,2),CY,C(:,3))
xlabel('Centimeters')
ylabel('Intensity')
title('Color vs. Length')
subplot(2,1,2)
plot(age,C(:,1),age,C(:,2),age,C(:,3))
xlabel('Years')
ylabel('Intensity')
title('Color vs. Age')

```

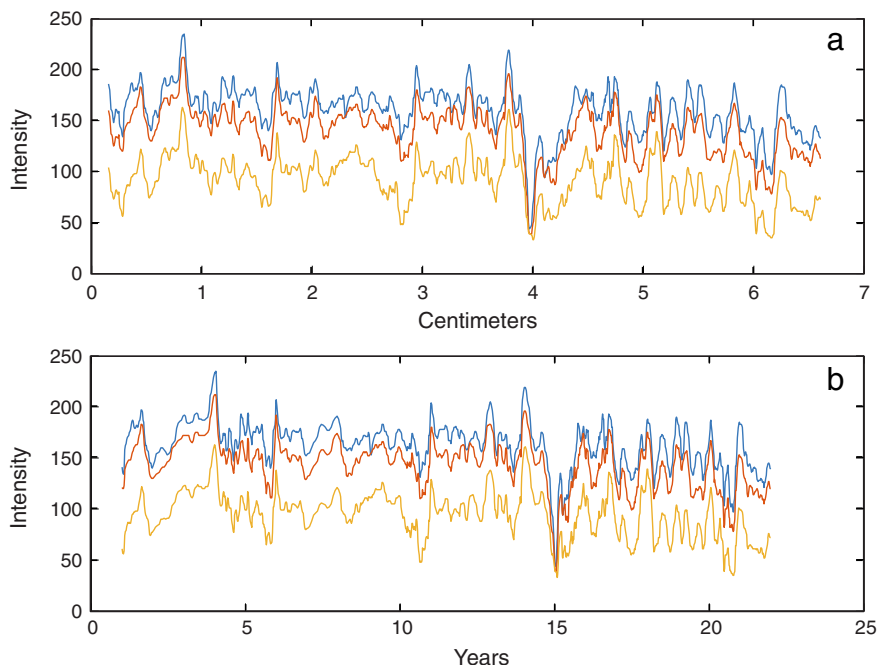



Fig. 8.8 Color-intensity curves (red, green and blue) plotted against **a** depth, and **b** age.

The interpolated color-intensity transects can now be further analyzed using the time-series analysis tools. The analysis of a representative red-color intensity transect across 70–250 varves during the project described above revealed significant peaks at about 13.1, 3.2, 2.2 and 1.0 yrs, suggesting both ENSO and TAV influences in the area at around 33,000 years ago (see Chap. 5 and Fig. 5.1).

8.10 Grain Size Analysis from Microscope Images

Identifying, measuring and counting particles in an image are the classic applications of image analysis. Examples from the geosciences include grain size analysis, counting pollen grains, and determining the mineral composition of rocks from thin sections. For grain size analysis the task is to identify individual particles, measure their sizes, and then count the number of particles per size class. The motivation to use image analysis is the ability to perform automated analyses of large sets of samples in a short period of time and at relatively low costs. Three different approaches are commonly used to identify and count objects in an image: (1) region-based segmentation using the watershed segmentation algorithm,

(2) object detection using the Hough transform, and (3) thresholding using color differences to separate objects. Gonzalez et al. (2009) describe these methods in great detail in the 2nd edition of their excellent book, which also provides numerous MATLAB recipes for image processing. The book has a companion webpage at

<http://www.imageprocessingplace.com/>

that offers additional support in a number of important areas (including classroom presentations, M-files, and sample images) as well as providing numerous links to other educational resources. We will use two examples to demonstrate the use of image processing for identifying, measuring, and counting particles. In this section we will demonstrate an application of watershed segmentation in grain size analysis and then in Sect. 8.9 we will introduce thresholding as a method for quantifying charcoal in microscope images.

The following example the segmentation, measuring, and counting of objects using the watershed segmentation algorithm (Fig. 8.9). We first read an image of coarse lithic grains of different sizes and store it in the variable `I1`. The size of the image is 284×367 pixels and, since the width is 3 cm, the height is $3 \text{ cm} \cdot 284/367 = 2.32 \text{ cm}$.

```
clear, close all, clc

I1 = imread('grainsize.tif');
ix = 3; iy = 284 * 3 / 367;
imshow(I1,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Original Image')
```

in which `ix` and `iy` denote the coordinate axes used to calibrate the image `I1` to a centimeter scale. The true number of objects counted in this image is 236, including three grains that overlap the borders of the image and will therefore be ignored in the following exercise. We reject the color information of the image and convert `I1` to grayscale using the function `rgb2gray`.

```
I2 = rgb2gray(I1);
imshow(I2,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Grayscale Image')
```

This grayscale image `I2` has a relatively low level of contrast. We therefore use the function `imadjust` to adjust the image intensity values. The function `imadjust` maps the values in the intensity image `I2` to new values in `I3`, such that 1% of the

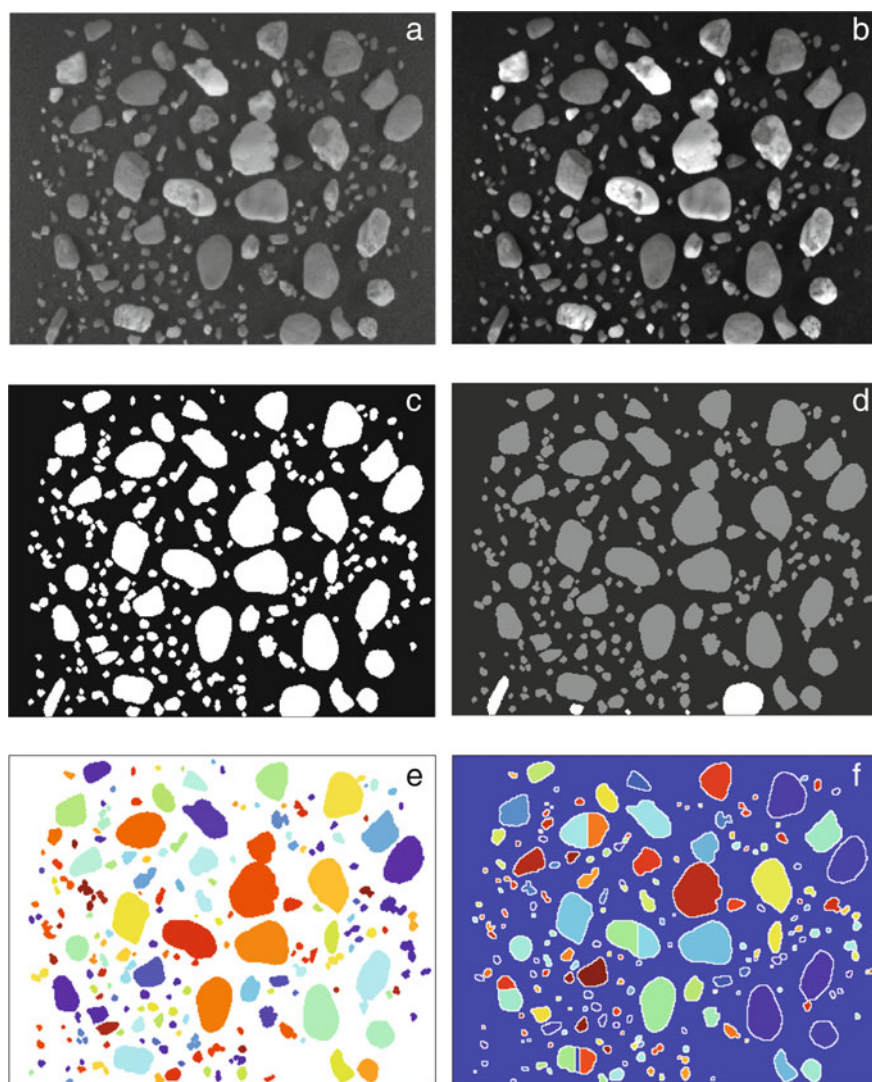


Fig. 8.9 Results from automated grain size analysis of a microscope image: **a** original grayscale image, **b** image after removal of background, **c** image after conversion to binary image, **d** image after eliminating objects overlapping the image border, **e** image with objects detected by tracing the boundaries of connected pixels, and **f** image with objects detected using a watershed segmentation algorithm.

data is saturated at low and high intensities. This increases the contrast in the new image I3.

```
I3 = imadjust(I2);
imshow(I3,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Adjusted Intensity Values')
```

We next determine the background of the I3 image, which means basically the texture of the black foil on which the grains are located. The function `imopen(im, se)` determines objects in an image `im` with a specific shape `se` (a flat structuring element such as a circular disk) and size (expressed as a specific number of pixels), as defined by the function `strel`. We then produce a background-free image, I4.

```
I4 = imopen(I3,strel('disk',1));
imshow(I4,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('No Background')
```

We subtract the background-free image I4 from the original grayscale image I3 to observe the background I5 that has been eliminated.

```
I5 = imsubtract(I3,I4);
imshow(I5,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Background')
```

The function `im2bw` converts the background-free image I4 to a binary image I6 by thresholding. If the threshold is 1.0 the image is all black, corresponding to a pixel value of 0. If the threshold is 0.0 the image is all white, corresponding to a pixel value of 1. We manually change the threshold value until we get a reasonable result and find 0.2 to be a suitable threshold.

```
I6 = im2bw(I4,0.2);
imshow(I6,...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Binary Image')
```

We next eliminate objects in `I6` that overlap the image border, since they are actually larger than shown in the image and will result in false estimates. We eliminate these using `imclearborder` and generate image `I7`.

```
I7 = imclearborder(I6);
himage1 = imshow(I6,...
    'XData',[0 ix],...
    'YData',[0 iy]); hold on
set(himage1, 'AlphaData', 0.7);
himage2 = imshow(imsubtract(I6,I7),...
    'XData',[0 ix],...
    'YData',[0 iy]);
set(himage2, 'AlphaData', 0.4);
title('Image Border'), hold off
```

We then trace the boundaries using `bwboundaries` in a binary image where non-zero pixels belong to an object and zero pixels are background. By default, the function also traces the boundaries of holes in the `I7` image. We therefore choose the option `noholes` to suppress the tracing of the holes. Function `label2rgb` converts the label matrix `L` resulting from `bwboundaries` to an RGB image. We use the colormap `jet`, the zerocolor `w` for white, and the color order `shuffle` (which simply shuffles the colors of `jet` in a pseudorandom manner).

```
[B,L] = bwboundaries(I7,'noholes');
imshow(label2rgb(L,@jet,'w','shuffle'),...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Define Objects')
```

The function `bwlabeln` is used to obtain the number of connected objects found in the binary image. The integer 8 defines the desired connectivity, which can be either 4 or 8 in two-dimensional neighborhoods. The elements of `L` are integer values greater than or equal to 0. The pixels labeled 0 are the background. The pixels labeled 1 make up one object, the pixels labeled 2 make up a second object, and so on.

```
[labeled,numObjects] = bwlabeln(I7,8);
numObjects
```

In our example the method identified 192 grains, which is significantly less than the 236 grains counted manually, reduced by the three objects that overlap the borders of the image. Visual inspection of the color-coded image generated by `bwboundaries` reveals the reason for the underestimated number of grains. Two large grains in the middle of the image have been observed as being connected,

giving a single, very large grain in the final result. Reducing the disk size with `strel` from `disk=1` to `disk=5` can help separate connected grains. Larger disks, on the other hand, reduce the number of grains because smaller grains are lost by filtering. We now determine the areas of each of the grains.

```
graindata = regionprops(labeled,'basic');
grainareas= [graindata(:).Area];
objectareas = 3^2 * grainareas * 367^(-2);
```

We then find the maximum, minimum and mean areas for all grains in the image, in cm^2 .

```
max_area = max(objectareas)
min_area = min(objectareas)
mean_area = mean(objectareas)
```

The connected grain in the middle of the image has a size of 0.16 cm^2 , which represents the maximum size of all grains in the image. Finally, we plot the histogram of all the grain areas.

```
clf
e = 0 : 0.0005 : 0.15;
histogram(objectareas,e)
xlabel('Grain Size in Millimeters^2')
ylabel('Number of Grains')
axis([0 0.1 0 30])
```

Several methods exist that partly overcome the artifact of connected grains in grain size analyses. The most popular technique for region-based segmentation is the watershed segmentation algorithm. Watersheds in geomorphology are ridges that divide areas contributing to the hydrological budget of adjacent catchments (see Sect. 7.10). Watershed segmentation applies to grayscale images the same methods used to separate catchments in digital elevation models. In this application, the grayscale values are interpreted as elevations in a digital elevation model, where the watershed then separates the two objects of interest.

The criterion commonly used to identify pixels that belong to a particular object is the nearest-neighbor distance. We use the distance transform performed by `bwdist`, which assigns to each pixel a number that is the distance between a pixel and the nearest non-zero pixel in `I7`. In an image in which objects are identified by pixel values of zero and the background by pixel values of one, the distance transform has zeros in the background areas and non-zero values that increase progressively with increasing distances from the edges of the objects. In our example however, the objects have pixel values of one and the background has

pixels with values of zero. We therefore have to apply `bwdist` to the complement of the binary image `I7` instead of to the image itself.

```
D = bwdist(~I7, 'cityblock');
```

The function `bwdist` provides several methods for computing the nearest-neighbor distances including *Euclidean distances*, *cityblock distances*, *chessboard distances* and *quasi-Euclidean distances*. We choose the *cityblock* option in this particular example, but other methods might be more appropriate for separating objects in other images. The distance matrix now contains positive non-zero values in the object pixels and zeros elsewhere. We then complement the distance transform and ascribe a value of `-Inf` to each pixel that does not belong to an object.

```
D = -D;
D(~I7) = -Inf;
```

We compute the watershed transform for the distance matrix and display the resulting label matrix.

```
L2 = watershed(D);
imshow(label2rgb(L2,@jet,'w','shuffle'),...
    'XData',[0 ix],...
    'YData',[0 iy])
title('Watershed Segmentation')
```

After having displayed the results from watershed segmentation, we determine the number of pixels for each object using the recipe described above, except for index `i` running from 2 to `max(objects)` since the value 1 denotes the background and 0 denotes the boundaries of the objects. The first true object is therefore marked by the value of 2.

```
objects = sortrows(L2(:),1);
max(objects)
clear objectsizes
for i = 2 : max(objects)
    clear individualobject
    individualobject = objects(objects == i);
    objectsizes(i) = length(individualobject);
end
objectsizes = objectsizes';
objectsizes = sortrows(objectsizes,1);
objectsizes = objectsizes(objectsizes~=0);
```

We have now recognized 205 objects, i.e. more objects than were identified in the previous exercise without watershed segmentation. Visual inspection of the result, however, reveals some oversegmentation (due to noise or other irregularities in the image) in which larger grains are divided into smaller pieces. On the other hand, very small grains have been eliminated by filtering the image with the morphological structuring element `strel1`. We scale the object sizes such that the area of one pixel is $(3 \text{ cm}/367)^2$.

```
objectareas = 3^2 * objectsizes * 367^(-2);
```

We now determine the areas for each of the grains. We again find the maximum, minimum and mean areas for all grains in the image, in cm^2 .

```
max_area = max(objectareas)
min_area = min(objectareas)
mean_area = mean(objectareas)
```

The largest grain in the center of the image has a size of 0.09 cm^2 , which represents the maximum size of all grains in the image. Finally, we plot the histogram of all the grain areas.

```
clf
e = 0 : 0.0005 : 0.15;
histogram(objectareas,e)
xlabel('Grain Size in Millimeters^2'),...
ylabel('Number of Grains')
axis([0 0.1 0 70])
```

As a check of the final result we digitize the outline of one of the larger grains and store the polygon in the variable `data`.

```
figure
imshow(I1,...
    'XData',[0 ix],...
    'YData',[0 iy])
data = ginput;
```

We close the polygon by copying the first row of coordinates to the end of the array. We then display the polygon on the original image.


```
data(end+1,:) = data(1,:)

imshow(I1,...
    'XData',[0 ix],...
    'YData',[0 iy]), hold on
plot(data(:,1),data(:,2)), hold off
```

The function `polyarea` yields the area of the large grain.

```
polyarea(data(:,1),data(:,2))

ans =

    0.0951
```

The calculated area corresponds approximately to the result from the grain size analysis. If oversegmentation is a major problem when using segmentation to count objects in an image, the reader is referred to the book by Gonzalez et al. (2009) that describes marker-controlled watershed segmentation as an alternative method that can be used to avoid oversegmentation.

8.11 Quantifying Charcoal in Microscope Images

Quantifying the composition of substances in geosciences, such as the mineral composition of a rock in thin sections, or the amount of charcoal in sieved sediment samples, is facilitated by the use of image processing methods. Thresholding provides a simple solution to problems associated with segmenting objects within an image that have different coloration or grayscale values. During the thresholding process, pixels with an intensity value greater than a threshold value are marked as object pixels (e.g. pixels representing charcoal in an image) and the rest as background pixels (e.g. all other substances). The threshold value is usually defined manually through visual inspection of the image histogram, but numerous automated algorithms are also available.

As an example we analyze an image of a sieved lake-sediment sample from Lake Nakuru, Kenya (Fig. 8.10). The image shows abundant light-gray oval ostracod shells and some mineral grains, as well as gray plant remains and black charcoal fragments. We use thresholding to separate the dark charcoal particles and count the pixels of these particles after segmentation. After having determined the number of pixels for all objects distinguished from the background by thresholding, we use a lower threshold value to determine the ratio of the number of pixels representing charcoal to the number of pixels representing all particles in the sample, i.e. to determine the percentage of charcoal in the sample.

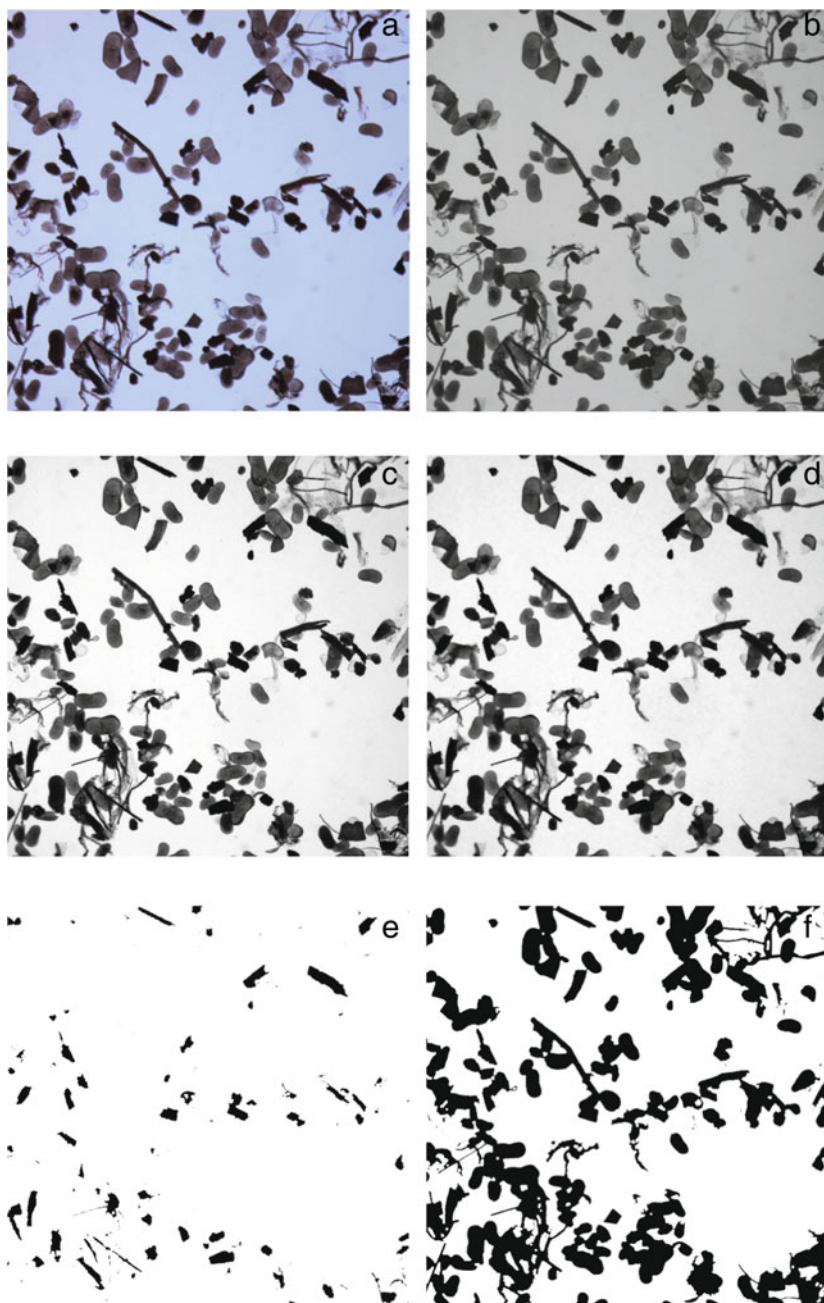


Fig. 8.10 Display of results from automatic quantification of charcoal in a microscope image: **a** original color image, **b** grayscale image, **c** image after enhancement of contrast, **d** image after removal of background, **e** image after thresholding to separate charcoal particles, and **f** image after thresholding to separate all objects.

We read the image of size 1,500-by-1,500 pixels and assume that the width and the height of the square image are both one centimeter.

```
clear, close all, clc

I1 = imread('lakesediment.jpg');
ix = 1; iy = 1;
imshow(I1,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeter')
ylabel('Centimeter')
title('Original Image')
```

The RGB color image is then converted to a grayscale image using the function `rgb2gray`.

```
I2 = rgb2gray(I1);
imshow(I2,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('Grayscale')
```

Since the image contrast is relatively low, we use the function `imadjust` to adjust the image intensity values. The function `imadjust` maps the values in the intensity image `I1` to new values in `I2`, such that 1% of the data is saturated at low and high intensities of `I2`. This increases the contrast in the new image `I2`.

```
I3 = imadjust(I2);
imshow(I3,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('Better Contrast')
```

We next determine the background of the lithic grains, which basically means the texture of the black foil on which the grains are located. The function `imopen(im, se)` determines objects in an image `im` below a certain pixel size and a flat structuring element `se`, such as a disk with a radius of 5 pixels generated by the function `strel`. The variable `I4` is the background-free image resulting from this operation.

```

I4 = imopen(I3,strel('disk',5));
imshow(I4,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('W/O Background')

```

We subtract the background-free image I4 from the original grayscale image I3 to observe the background I5 that has been eliminated.

```

I5 = imsubtract(I3,I4);
imshow(I5,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('Background')

```

The function `im2bw` converts the I4 image to a binary image (I6) by thresholding. If the threshold is 1.0 the image is all black, corresponding to a pixel value of 0. If the threshold is 0.0 the image is all white, corresponding to a pixel value of 1. We manually change the threshold value until we get a reasonable result. In our example a threshold of 0.03 gives good results for identifying charcoal fragments.

```

I6 = im2bw(I4,0.03);
imshow(I6,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('Only Charcoal')

```

Since we know the size of a pixel we can now simply count the number of pixels to estimate the total amount of charcoal in the image. Finally, we compute the area of all objects, including charcoal.

```

I7 = im2bw(I4,0.6);
imshow(I7,...
    'XData',[0 ix],...
    'YData',[0 iy]), axis on
xlabel('Centimeters')
ylabel('Centimeters')
title('All Objects')

```

We are not interested in the absolute areas of charcoal in the image but in the percentage of charcoal in the sample.

```
100*sum(sum(I6==0))/sum(sum(I7==0))
```

```
ans =
```

```
13.4063
```

The result suggests that approximately 13% of the sieved sample is charcoal. As a next step, we could quantify the other components in the sample, such as ostracods or mineral grains, by choosing different threshold values.

8.12 Shape-Based Object Detection in Images

The counting of objects within images on the basis of their shapes is a very time-consuming task. Examples of where this is carried out for round objects include the counting of planktonic foraminifera shells to infer past sea-surface temperatures, of diatom frustules to infer past chemical composition of lake water, and of pollen grains to determine assemblages that can be used to reconstruct regional air temperature and precipitation. Linear objects that are determined include faults in aerial photos and satellite images (to derive the present-day stress field of an area) and annual layers (varves) in thin sections (to establish an annually-resolved sedimentary history).

The Hough transform, named after the related 1962 patent of Paul VC Hough, is a popular technique with which to detect objects within images, based on their shapes. The Hough transform was originally used to detect linear features, but soon after being patented it was generalized to identify objects of any shape (Duda and Hart 1972; Ballard 1981). The book by Gonzalez et al. (2009) contains a comprehensive introduction to the Hough transform and its applications for detecting objects within images. According to their introduction to the method, the Hough transform is performed in two steps. In the first step an edge detector is used to extract edge features, such as distinct sediment layers or the outlines of pollen grains, from an image. In the second step lines (or objects of any other shape) that trace these edge features are identified. The Image Processing Toolbox (MathWorks 2020) contains functions that use the Hough transform to detect lines or circular objects.

The classic Hough transform is used to detect lines in images. After applying an edge detector of any kind we end up with a binary image that has black pixels on the edges and white pixels in between. We next describe the lines through a given black pixel by the Euclidean distance ρ between the line and the origin, and by the angle θ of the vector from the origin to the closest point on the line (Fig. 8.11a):

$$\rho = x \cos \theta + y \sin \theta$$

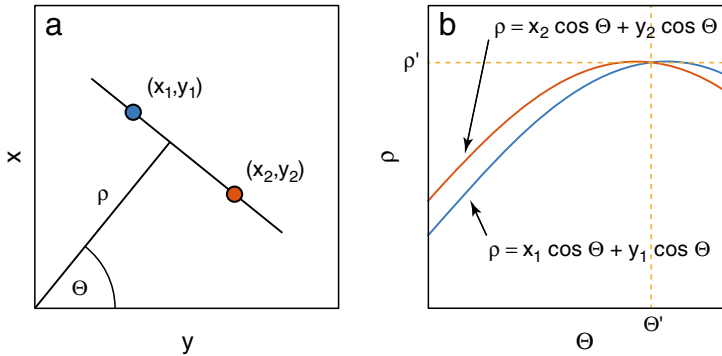


Fig. 8.11 The concept of the Hough transform: **a** parametrization of lines in the xy -plane, and **b** sinusoidal curves in the $\rho\theta$ parameter space, with the point of intersection corresponding to the line that passes through two different pixels of an edge feature (modified from Gonzalez et al. 2009).

The family of all lines passing through this particular pixel (x_i, y_i) of an edge feature are displayed as a sinusoidal curve in the (θ, ρ) parameter space (Fig. 8.11b). The intersection point (θ', ρ') of two such sinusoidal curves corresponds to the line that passes through two different pixels, (x_1, y_1) and (x_2, y_2) , of an edge feature. Next, we search for n points (x_i, y_i) in the Hough transform where many lines intersect, since these are points defining the line tracing an edge feature. Detecting circles instead of lines works in a similar manner, using the coordinates of the center of the circle and its radius instead of ρ and θ .

For our first example we use these functions to detect the thin layers of pure white diatomite within varved sediments exposed in the Quebrada de Cafayate of Argentina, which have already been used as examples in previous sections (Trauth and Strecker 1999; Trauth et al. 2003) (Fig. 8.12). The quality of the image is not perfect, which is why we cannot expect optimal results. We first read the cropped version of the laminated sediment from Sect. 8.8 and store it in the variable `I1`. The size of the image is 1,047-by-1,691 pixels, consisting of three colors (red, green and blue).

```
clear, close all, clc
```

```
I1 = imread('varves_cropped.tif');
imshow(I1, 'InitialMagnification', 30)
```

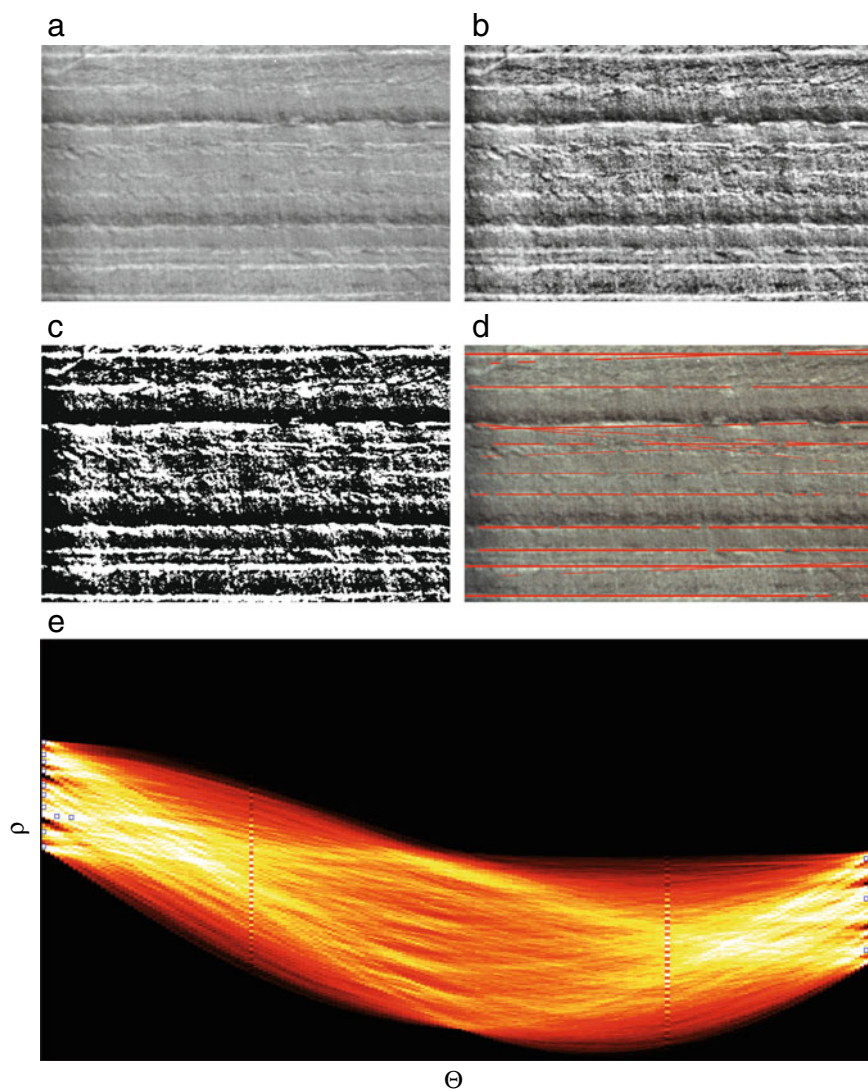


Fig. 8.12 Automated detection of thin layers of pure white diatomite within varved sediments exposed in the Quebrada de Cafayate of Argentina, using *houghlines* (Trauth and Strecker 1999; Trauth et al. 2003): **a** grayscale image, **b** enhanced image, **c** binary image, **d** image with diatomite layers marked by red lines, and **e** Hough transform of the image.

We reject the color information of the image and convert `I1` to grayscale using the function `rgb2gray`.

```
I2 = rgb2gray(I1);  
imshow(I2, 'InitialMagnification', 30)
```

We then use `adapthisteq` to perform a contrast-limited adaptive histogram equalization (CLAHE), in order to enhance the contrast in the image (Zuiderveld [1994](#)).

```
I3 = adapthisteq(I2,...  
    'ClipLimit', 0.1,...  
    'Distribution', 'Rayleigh');  
imshow(I3, 'InitialMagnification', 30)
```

In this case `ClipLimit` limits the contrast enhancement using a real scalar from 0 to 1, with higher numbers resulting in greater contrast; the default value is 0.01. The `Distribution` parameter defines the desired histogram shape for the tiles by specifying a distribution type, such as `Uniform`, `Rayleigh` or `Exponential`. Using a `ClipLimit` of 0.1 and a `Rayleigh` distribution yields good results. Using the function `im2bw` then converts the `I3` image to a binary image (`I4`) by thresholding. If the threshold is 1.0 the image is all black, corresponding to a pixel value of 0. If the threshold is 0.0 the image is all white, corresponding to a pixel value of 1. We manually change the threshold value until we get a reasonable result and find 0.55 to be a suitable threshold.

```
I4 = im2bw(I3, 0.55);  
imshow(I4, 'InitialMagnification', 30)
```

The function `hough` implements the Hough transform, `houghpeaks` finds the high-count accumulator cells in the Hough transform, and `houghlines` extracts lines in the original image, based on the other two functions. We determine the $n = 15$ lines corresponding to the first 15 maxima of the Hough transform and store fifteen of the lines (lines 1–5, 6–10, and 11–15) in three separate variables `lines1`, `lines2` and `lines3`.

```
[H, theta, rho] = hough(I4);  
peaks = houghpeaks(H, 15);  
  
lines1 = houghlines(I4, theta, rho, peaks(1:5, :));  
lines2 = houghlines(I4, theta, rho, peaks(6:10, :));  
lines3 = houghlines(I4, theta, rho, peaks(11:15, :));
```


We then display the Hough transform and mark the 15 maxima of the Hough transform with blue squares.

```
imshow(imadjust(mat2gray(H)),[], ...
    'XData',theta, ...
    'YData',rho, ...
    'InitialMagnification','fit')
colormap(hot), axis square, hold on
plot(theta(peaks(:,2)),rho(peaks(:,1))), ...
    'LineStyle','none', ...
    'Marker','s', ...
    'Color','b')
xlabel('\theta')
ylabel('\rho')
title('Hough Transform')
```

The variables `lines1`, `lines2` and `lines3` can now be used to display the lines on the image, with the line thickness decreasing from `lines3` to `lines1` depending on the rank of the lines in the Hough transform.

```
imshow(I1,'InitialMagnification',30), hold on
for k = 1:length(lines1)
xy = [lines1(k).point1; lines1(k).point2];
plot(xy(:,1),xy(:,2),...
    'LineWidth',3,...
    'Color',[1 0 0]);
end
hold on
for k = 1:length(lines2)
xy = [lines2(k).point1; lines2(k).point2];
plot(xy(:,1),xy(:,2),...
    'LineWidth',2,...
    'Color',[1 0 0]);
end
for k = 1:length(lines3)
xy = [lines3(k).point1; lines3(k).point2];
plot(xy(:,1),xy(:,2),...
    'LineWidth',1,...
    'Color',[1 0 0]);
end
```

The result shows that the clearly recognizable white layers are well detected whereas the less pronounced layers are not identified. The method also mistakenly marks non-existing lines on the image because of the low quality of the image.

Using a better quality image and carefully adjusting the parameters used with the Hough transform will yield better results.

In the second example the Hough transform is used to automatically count pollen grains in a microscope image of Argentine honey (Fig. 8.13). The quality of the image is again not perfect, which is why we can not expect optimum results. In particular, the image of three-dimensional objects was taken with a very large magnification, so it is slightly blurred. We first read the pollen image and store it in the variable `I1`. The size of the image is 968-by-1,060 pixels of three colors (red, green and blue). Since the image is relatively large, we reduce the image size by a factor of two.

```
clear

I1 = imread('pollen.jpg');
I1 = I1(1:2:end,1:2:end,:);
imshow(I1,'InitialMagnification',100)
```

We reject the color information of the image and use the red color only.

```
I2 = I1(:,:,1);
imshow(I2,'InitialMagnification',100)
```

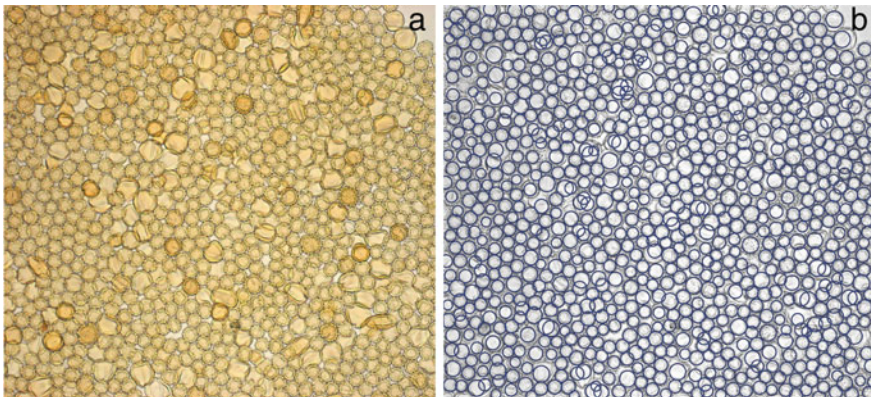


Fig. 8.13 Automated detection of pollen grains (mostly *Asteraceae*, with less abundant *Caesalpiniaceae* and *Lamiaceae* pollen) in a microscope image of Argentine honey, using `imfindcircles` (original image courtesy of K. Schitteck and F. Flores): **a** original RGB image, **b** pollen grains detected using the Hough transform.

Next, we use `adapthisteq` to perform a contrast-limited adaptive histogram equalization (CLAHE) in order to enhance the contrast in the image (Zuiderveld 1994).

```
I3 = adapthisteq(I2);
imshow(I3, 'InitialMagnification', 100)
```

The function `imfindcircles` implements the Hough transform and extracts circles from the original image.

```
[centers, radii] = imfindcircles(I3, [12 20], ...
    'Method', 'TwoStage', ...
    'ObjectPolarity', 'Bright', ...
    'Sensitivity', 0.92, ...
    'EdgeThreshold', 0.20);
num = length(centers);
nstr = ['Number of Pollen Grains: ', num2str(num)];
```

We use the `TwoStage` detection method for a two-stage circular Hough transform, following the procedure described by Yuen et al. (1990) and Davies (2005). The object polarity is set to `bright` as we are looking for bright rather than dark objects in our image. The sensitivity of 0.92 and the edge threshold of 0.20 are found by trial and error. We then use `viscircles` to display the circles on the grayscale image `I2`

```
imshow(I2, 'InitialMagnification', 100)
viscircles(centers, radii, 'EdgeColor', 'b')
title(nstr)
```

using the output `centers` and `radii` from `imfindcircles`. The edge color of the circles in the graphics is set to `b` for blue. The result shows that we have counted 884 pollen grains with the method. The algorithm identifies the majority of the objects, but some are not recognized and some of the larger objects are mistakenly identified as two or more pollen grains. Using a better quality image and carefully adjusting the parameters used with the Hough transform will yield better results. Plotting the histogram of the pollen radii using `histogram`

```
histogram(radii)
```

reveals that most of the grains have a radius of around 15 pixels.

8.13 The Normalized Difference Vegetation Index

The pigments of plants reflect near-infrared light (700–1100 nm), whereas they strongly absorb visible light (400–700 nm) for use in photosynthesis. The normalized difference vegetation index (NDVI) is an index used to map live green vegetation, calculated as

$$NDVI = \frac{(NIR - R)}{(NIR + R)}$$

using the spectral reflectance measurements acquired in the near-infrared (*NIR*) and red (*R*) spectral bands. The index can be measured with ground-based spectral cameras as well as from drones, aircrafts and satellites. The cameras used are normally very expensive but there are now some quite inexpensive alternatives available. Examples of such alternatives include the AgroCam (<https://www.agrocam.eu>) and the MAPIR Survey Camera (<https://www.mapir.camera>). These cameras are used in professional applications but because of their low prices they are also suitable for student exercises.

Since the MAPIR cameras, manufactured by Peau Productions Inc. in San Diego, CA, have the same size and shape as the GoPro camera system they can make use of all the accessories of the popular GoPro action-camcorder, such as memory cards, camera mounts, and housings. We built an inexpensive multispectral array consisting of four cameras recording NDVI Red + NIR (650 and 850 nm), Blue (450 nm), Green (550 nm) and Visible Light RGB (~370–650 nm).

The following example explains how to acquire images with MAPIR cameras, how to import and process the images with MATLAB, and how to calculate the NDVI in order to detect vegetation in an image. While editing the images from the MAPIR Survey2 NDVI camera, we learn a lot about how cameras actually capture and store images. This is important in order to be able to correctly import the images into MATLAB, process them, and compute corresponding NDVI images. Failure to do so correctly produce an image that can be immediately seen to be unsatisfactory.

The following script uses information about the camera provided on the manufacturer's webpage (<https://www.mapir.camera>). The script is also inspired by the example "Finding Vegetation in a Multispectral Image" from the MATLAB Image Processing Toolbox. The example does not include any calibration of the camera and uses an auto white balance, rather than a true white balance using a white reference. According to the MAPIR webpage about *Calibrating Images in QGIS with MAPIR Plugin*, the Survey2 cameras capture 16 bit RAW photos, which means there are 16 bit (65,536) pixels with intensity values ranging from 0 to 65,535. When the camera saves a JPEG file it compresses the image leaving a range of only 0 to 255. We first define a file name and the size of the image.

```
clear, close all, clc

filename = '2017_0328_110031_007.RAW';
row = 3456;
col = 4608;
```

According to MAPIR specification page for the Survey2 NDVI camera it uses a Sony Exmor IMX206 16MP (Bayer RGB), which is important information required for correct loading the images into MATLAB. However, the sensor specification page at Sony Inc. says that the sensor actually acquires 12 bit images, which is in agreement with information received by email from the software engineer at Peau Productions Inc. in March 2017, but not with what is stated on the company's webpage. With the support of this engineer, our own research, and some support from readers of Stack Overflow (<http://stackoverflow.com>), we have solved the problem. We read the data in a uint16 array in MATLAB and displaying the image suggests that we have the right solution:

```
id = fopen(filename);
I1 = fread(id,[col, row], 'uint16', 'ieee-le');
I1 = I1';

figure('Position',[100 700 800 600])
imshow(I1,'InitialMagnification',30)
```

The MAPIR Survey2 NDVI Camera measures red (660 nm) and near- infrared (850 nm) radiances for the NDVI calculation but where in the grayscale image above are our red and infrared data? The Sony sensor is a single-chip sensor, storing the three colors, red, green, and blue, in a single array I1, displayed as a grayscale image using the default gray colormap in MATLAB. Most cameras have single-chip sensors, even single-lens reflex cameras (SLRs) that can store images in RAW format in addition to the JPEG format, using lossy compression. More expensive three-sensor cameras also exist that use separate sensors for the filtered red, green, and blue color ranges.

Single-chip cameras store RGB images on their single chips is using the Bayer mosaic, named after Bryce Bayer (1929–2012), a research scientist at Eastman Kodak (see <http://en.wikipedia.org> for more information about Bayer mosaics) (Fig. 8.14). Bayer's idea was to use filters and direct the individual colors red, green, and blue onto individual sensor pixels, usually in an RGGB arrangement because the human eye reacts more sensitively to green than to red or blue. Because of the Bayer mosaic, SLRs with 16 MP resolution (such as that of our Sony chip) do not really take 16 MPs, but only 4 MPs for red and blue (25% each) and 8 MP for green (50%), and then interpolate each of the three colors each to 16 MP.

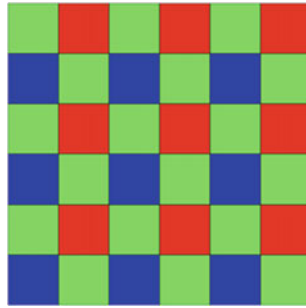


Fig. 8.14 The Bayer RGGB pattern helps to store RGB images on the sensor of a single-chip camera. The pattern contains twice as many green pixels as the red and blue because the human eye reacts more sensitively to green than to the other colors.

The RAW format provided by SLRs does not mean that the colors are stored in full resolution, but that the images are not compressed (for example, using the JPEG algorithm). Using MATLAB instead we can import and process the image with its original resolution, i.e. 4 MPs for red and blue, and 8 MPs for green. Alternatively, we can demosaic the image, in the same way an SLR does for using, for instance, the MATLAB function `demosaic`. Demosaicing the image, according to MATLAB Help, converts a Bayer pattern encoded image to a true color (RGB) image using gradient-corrected linear interpolation. According to the MAPIR webpage, the red (660 nm) radiance is stored in the red image channel and the near-infrared (850 nm) radiance is stored in the blue image channel, with no information stored in the green channel.

Using `imshow` to display `I2` as a true color RGB image, we find blue hues in areas with high reflection in the NIR spectral band, and hence denser live green vegetation, while all other areas with low NIR values are more red in color.

```
figure('Position',[100 300 800 600])
I2 = demosaic(I1,'rggb');
imshow(I2,'InitialMagnification',30)
```

To improve the contrast of the image we can, for example, apply a decorrelation stretch for multichannel images.

```
I3 = decorrstretch(I2, 'Tol', 0.01);
imshow(I3,'InitialMagnification',30)
```

We now display the Red and NIR channels,

```
figure('Position',[100 100 800 300])
subplot(1,2,1), imshow(I3(:,:,1)), title('Red')
subplot(1,2,2), imshow(I3(:,:,3)), title('NIR')
```

We then perform a white balance to correct any unrealistic color casts. For this we need a reference card (usually gray) which is placed within the image and then used as a reference for the white balance. Alternatively, we can perform a fully automated white balance (as we do here) by dividing the values of each individual channel by the mean of the values of the individual color meanR, meanG and meanB, and multiplying the values of each channel by the mean value of the values of all channels meanGray. To perform this operation, we must first convert the data to single precision and then convert it back to uint16.

```
I3G = rgb2gray(I3);
meanR = mean2(I3(:,:,1));
meanG = mean2(I3(:,:,2));
meanB = mean2(I3(:,:,3));
meanGray = mean2(I3G);
I3W(:,:,1) = uint16(single(I3(:,:,1))* ...
    meanGray / meanR);
I3W(:,:,2) = uint16(single(I3(:,:,2))* ...
    meanGray / meanG);
I3W(:,:,3) = uint16(single(I3(:,:,3))* ...
    meanGray / meanB);
```

Next, we calculate the NDVI after converting the image into single precision to perform the calculation using MATLAB:

```
I4 = single(I3W)/65536;
ndvi = (I4(:,:,3) - I4(:,:,1)) ./ ...
    (I4(:,:,3) + I4(:,:,1));

figure('Position',[100 700 800 600])
imshow(ndvi,...
    'DisplayRange',[-1 1],...
    'InitialMagnification',30)
```

Alternatively we can display the NDVI as a pseudocolor plot using the colormap jet.

```
figure('Position',[100 700 800 600])
imagesc(ndvi), axis off, colormap(jet), colorbar
```

Finally, we can define an arbitrary threshold (e.g. 0.4) to separate vegetation (displayed as white pixels) from other materials (gray pixels).

```
threshold = 0.4;
ndvit = (ndvi > threshold);

figure('Position',[100 700 800 600])
imshow(ndvit,...
    'DisplayRange',[-1 1],...
    'InitialMagnification',30)
```

Alternatively, we can display pixels with NDVI>0.4 in green, with lower values displayed in white (Fig. 8.15).

```
figure('Position',[100 700 800 600])
imshow(ndvit,'InitialMagnification',30)
colormap(gca,[1 1 1;0.5 0.6 0.3])
```

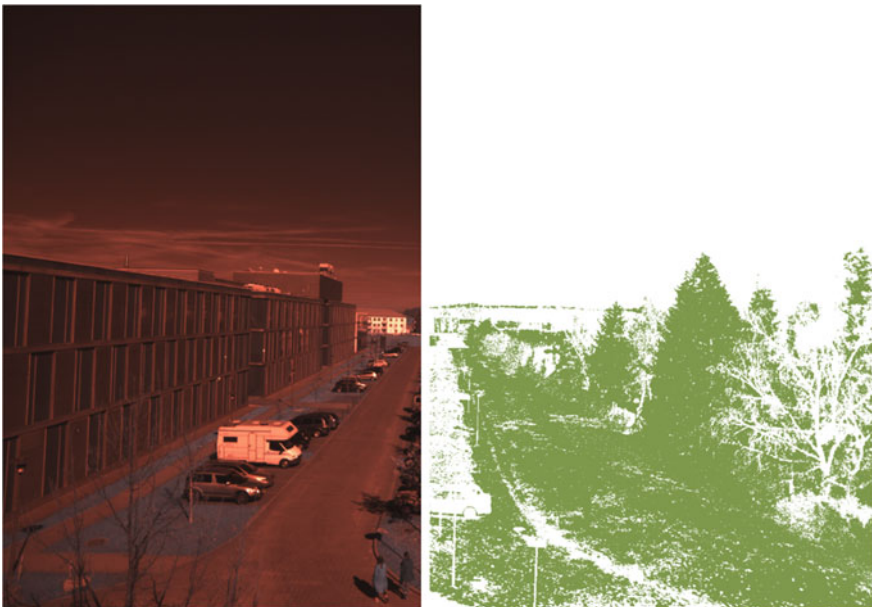


Fig. 8.15 RGB representation of the demosaiced image captured with the MAPIR Survey2 NDVI camera (left), and threshold image of the Normalized Difference Vegetation Index (NDVI) with green pixels indicating the areas with high reflection values in the near-infrared frequency band (right). The red (660 nm) is stored in the red image channel and the near-infrared (850 nm) is stored in the blue image channel. Since vegetation-rich areas reflect the near-infrared more strongly, they appear rather bluish, while vegetation-poor areas appear red.

We can also calculate the NDVI from satellite data that include the red and near-infrared spectral bands. Sentinel-2 is a system of four satellites designed to acquire terrestrial observations for a variety of tasks including forest monitoring, the mapping of land surface changes and the impact of natural disasters. The data can be downloaded free of charge from the USGS EarthExplorer (<https://earthexplorer.usgs.gov>) website. We use two images, acquired on 22 and 29 May 2018, from which we load the 650 nm (B04) and 850 nm (B08) spectral bands required to calculate the NDVI.

```
clear, clc, close all

filename1 = 'T32UQD_20180522T102031_B04_10m.jp2';
filename2 = 'T32UQD_20180522T102031_B08_10m.jp2';
filename3 = 'T32UQD_20180529T101031_B04_10m.jp2';
filename4 = 'T32UQD_20180529T101031_B08_10m.jp2';
B04_22 = imread(filename1);
B08_22 = imread(filename2);
B04_29 = imread(filename3);
B08_29 = imread(filename4);
```

We clip the image of the Berlin-Brandenburg area to the area of the city of Potsdam by typing

```
clip = [8500 9500 6500 7500];
B04_22 = B04_22(8500:9500,6500:7500);
B08_22 = B08_22(clip(1):clip(2),clip(3):clip(4));
B04_29 = B04_29(clip(1):clip(2),clip(3):clip(4));
B08_29 = B08_29(clip(1):clip(2),clip(3):clip(4));
```

Next we convert the uint16 data to single precision for the calculations.

```
B04_22 = single(B04_22);
B08_22 = single(B08_22);
B04_29 = single(B04_29);
B08_29 = single(B08_29);
```

To display the images we first calculate the 5% and 95% percentiles.

```
B04_22_05 = prctile(reshape(B04_22,1,...
    numel(B04_22)), 5);
B04_22_95 = prctile(reshape(B04_22,1,...
    numel(B04_22)),95);
B08_22_05 = prctile(reshape(B08_22,1,...
    numel(B08_22)), 5);
```

```

B08_22_95 = prctile(reshape(B08_22,1,...
    numel(B08_22)),95);
B04_29_05 = prctile(reshape(B04_29,1,...
    numel(B04_29)), 5);
B04_29_95 = prctile(reshape(B04_29,1,...
    numel(B04_29)),95);
B08_29_05 = prctile(reshape(B08_29,1,...
    numel(B08_29)), 5);
B08_29_95 = prctile(reshape(B08_29,1,...
    numel(B08_29)),95);

```

and then calculate the NDVI values for both images by typing

```

NDVI_22 = (B08_22-B04_22)./(B08_22+B04_22);
NDVI_29 = (B08_29-B04_29)./(B08_29+B04_29);

```

As an example we display the 650 and 850 nm spectral bands of the image acquired on 22 May 2018 as a grayscale images, and the NDVI map as a color image, using the colormap jet.

```

figure('Position',[100 100 1400 400])
A1 = axes('Position',[0.025 0.1 0.4 0.8]);
imagesc(B04_22,[B04_22_05 B04_22_95])
title('650 nm')
colormap(A1,'Gray'), colorbar
set(gca,'FontSize',14)
axis square tight, axis off
A2 = axes('Position',[0.325 0.1 0.4 0.8]);
imagesc(B08_22,[B08_22_05 B08_22_95])
title('850 nm')
colormap(A2,'Gray'), colorbar
set(gca,'FontSize',14)
axis square tight, axis off
A3 = axes('Position',[0.625 0.1 0.4 0.8]);
imagesc(NDVI_22,[0 1])
title('NDVI 22 May 2018')
colormap(A3,'Jet'), colorbar
set(gca,'FontSize',14)
axis square tight, axis off

```

and save the image in a PNG file.

```

print -dpng -r300 sentinel_ndiv_1_vs1.png

```

Then we calculate the difference between the NDVI maps of the two images acquired on 22 and 29 May 2018, and again save the image in a PNG file.

```
DNDVI = NDVI_22 - NDVI_29;
figure('Position',[100 800 1400 400])
A3 = axes('Position',[0.025 0.1 0.4 0.8]);
imagesc(NDVI_22,[0 1])
title('NDVI 22 May 2018')
colorbar, set(gca,'FontSize',14)
colormap(A3,jet)
axis square tight, axis off
A4 = axes('Position',[0.325 0.1 0.4 0.8]);
imagesc(NDVI_29,[0 1])
title('NDVI 29 May 2018')
colorbar, set(gca,'FontSize',14)
colormap(A4,jet)
axis square tight, axis off
A5 = axes('Position',[0.625 0.1 0.4 0.8]);
imagesc(DNDVI,[-0.2 0.2])
title('Difference NDVI 22 - 29 May 2018')
colormap(A5,'Jet'), colorbar
set(gca,'FontSize',14)
axis square tight, axis off

print -dpng -r300 sentinel_ndiv_2_vs1.png
```

The image very nicely displays the progression in seasonal greening.

Recommended Reading

- Abrams M, Hook S (2002) ASTER user handbook—version 2. Jet Propulsion Laboratory and EROS Data Center, Sioux Falls
- Ballard DH (1981) Generalizing the Hough transform to detect arbitrary shapes. *Pattern Recognit* 13:111–122
- Barry P (2001) EO-1/Hyperion science data user's guide. TRW Space, Defense & Information Systems, Redondo Beach, CA
- Beck R (2003) EO-1 user guide. USGS Earth Resources Observation Systems Data Center (EDC), Sioux Falls, SD
- Campbell JB (2002) Introduction to remote sensing. Taylor & Francis, London
- Davies ER (2005) Machine vision: theory, algorithms, practicalities, 3rd edn. Morgan Kaufman Publishers, Burlington, MA
- Duda RO, Hart PE (1972) Use of the Hough transform to detect lines and curves in pictures. *Commun ACM* 15:1–15
- Francus P (2005) Image analysis, sediments and paleoenvironments—developments in paleoenvironmental research. Springer, Berlin, Heidelberg, New York

- Gonzalez RC, Woods RE, Eddins SL (2009) Digital image processing using MATLAB, 2nd edn. Gatesmark Publishing, LLC
- Hough PVC (1962) Method and means for recognizing complex patterns. US Patent No. 3069654
- Irons J, Riebeek H, Loveland T (2011) Landsat data continuity mission—continuously observing your world. NASA and USGS (available online)
- Jensen JR (2013) Remote sensing of the environment: Pearson New International Edition. Pearson, London
- Lein JK (2012) Environmental sensing—analytical techniques for earth observation. Springer, Berlin, Heidelberg, New York
- Mandl D, Cruz P, Frye S, Howard J (2002) A NASA/USGS collaboration to transform earth observing-1 into a commercially viable mission to maximize technology infusion. In: SpaceOps 2002, Houston, TX, 9–12 Oct 2002
- MathWorks (2020) Image processing toolbox—user’s guide. The MathWorks, Natick, MA
- Marwan N, Trauth MH, Vuille M, Kurths J (2003) Nonlinear time-series analysis on present-day and Pleistocene precipitation data from the NW Argentine Andes. *Clim Dyn* 21:317–326
- Ochs B, Hair D, Irons J, Loveland T (2009) Landsat Data Continuity Mission, NASA and USGS (available online)
- Richards JA (2013) Remote sensing digital image analysis, 4th edn. Springer, Berlin, Heidelberg, New York
- Seelos K, Sirocko F (2005) RADIUS—rapid particle analysis of digital images by ultra-high-resolution scanning of thin sections. *Sedimentology* 52:669–681
- Trauth MH, Bookhagen B, Marwan N, Strecker MR (2003) Multiple landslide clusters record Quaternary climate changes in the NW Argentine Andes. *Palaeogeogr Palaeoclimatol Palaeoecol* 194:109–121
- Trauth MH, Alonso RA, Haselton KR, Hermanns RL, Strecker MR (2000) Climate change and mass movements in the northwest Argentine Andes. *Earth Planet Sci Lett* 179:243–256
- Trauth MH, Strecker MR (1999) Formation of landslide-dammed lakes during a wet period between 40000–25000 yr B.P. in northwestern Argentina. *Palaeogeogr Palaeoclimatol Palaeoecol* 153:277–287
- Yuen HK, Princen J, Illingworth J, Kittler J (1990) Comparative study of Hough transform methods for circle finding. *Image Vis Comput* 8:71–77
- Zuiderveld K (1994) Contrast limited adaptive histogram equalization. In: *Graphic gems IV*. Academic Press Professional, San Diego, pp 474–485

9.1 Introduction

Multivariate analysis is used to understand and describe the relationships between an arbitrary number of variables. Earth scientists often deal with multivariate data sets such as microfossil assemblages, geochemical fingerprints of volcanic ash layers, or the clay mineral content of sedimentary sequences. Such multivariate data sets consist of measurements of p variables on n objects, which are usually stored in n -by- p arrays:

$$X = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}$$

The rows of the array represent the n objects and the columns represent the p variables. The characteristics of the 2nd object in the suite of samples are described by the vector in the second row of the data array:

$$X_2 = (x_{21} \quad x_{22} \quad \dots \quad x_{2p})$$

As an example consider a set of microprobe analyses on glass shards from volcanic ash layers in a tephrochronology project. The variables then represent the p chemical elements and the objects are the n ash samples. The aim of the study is to correlate ash layers by means of their geochemical fingerprints.

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_9) contains supplementary material, which is available to authorized users.

Most multi-parameter methods simply try to overcome the main difficulty associated with multivariate data sets, which relates to data visualization. Whereas the character of univariate or bivariate data sets can easily be explored by visual inspection of a 2D histogram or an xy plot (Chap. 3), the graphical display of a three variable data set requires a projection of the 3D distribution of data points onto a two-dimensional display. It is impossible to imagine or display a higher number of variables. One solution to the problem of visualization of high-dimensional data sets is to reduce the number of dimensions. A number of methods group together highly-correlated variables contained within the data set and then explore the reduced number of groups.

The classic methods for reducing the number of dimensions are *principal component analysis* (PCA) and *factor analysis* (FA). These methods seek the directions of maximum variance in a data set and use these as new coordinate axes. The advantage of replacing the variables by new groups of variables is that the groups are uncorrelated. These methods are therefore used for unmixing (or separating) mixed variables. Moreover, the new variables can often assist in the interpretation of a multivariate data set since they often contain valuable information on the processes responsible for the distribution of the data points. In a geochemical analysis of magmatic rocks the groups defined by the method usually contain chemical elements with similar sized ions in similar locations within the lattices of certain minerals. Examples include Si^{4+} and Al^{3+} , and Fe^{2+} and Mg^{2+} , in silicates.

A second important suite of multivariate methods aims to group objects by their similarity. As an example *cluster analysis* (CA) is often used to correlate volcanic ash layers, such as those in the above example. Tephrochronology attempts to correlate tephra by means of their geochemical fingerprints. When combined with a few radiometric age determinations from the key ash layers this method allows correlation between different sedimentary sequences that contain these ash layers (e.g. Westgate et al. 1998; Hermanns et al. 2000). Cluster analysis is also used in the field of micropaleontology, for example, to compare the pollen, foraminifera, or diatom content of different microfossil assemblages (e.g. Birks and Gordon 1985).

A third group of methods is concerned with the classification of observations. Humans tend to want to classify the things around them, even though nature rarely falls into discrete classes. Classification (or categorization) is useful as it can, for example, help decision makers to take necessary precautions to reduce risk, to drill an oil well, or to assign fossils to a particular genus or species. Most classification methods make decisions based on Boolean logic with two options, true or false; an example is the use of a threshold value for identifying charcoal in microscope images (Sect. 8.11). Alternatively, fuzzy logic (which is not explained in this book) is a generalization of the binary Boolean logic with respect to many real world problems in decision-making, where gradual transitions are reasonable (Zadeh 1965; MathWorks 2020a).

The following sections introduce the most important techniques of multivariate statistics: principal component analysis (PCA) and cluster analysis (CA) in Sects. 9.2 and 9.5, and *independent component analysis* (ICA), which is a nonlinear

extension of PCA, in Sect. 9.3. Section 9.4 introduces *discriminant analysis* (DA), which is a popular method of classification in earth sciences. Section 9.6 introduces *multiple linear regression*. Section 9.7 demonstrates the use of Aitchison's (1984, 1986, 1999, 2003) *log-transformation* to overcome the closed-sum problem that is very common in multivariate data sets. These sections first provide introductions to the theory behind the various techniques and then demonstrate their use for analyzing earth sciences data, using MATLAB functions (MathWorks 2020b).

9.2 Principal Component Analysis

Principal component analysis (PCA) detects linear dependencies between p variables and replaces groups of linearly correlated variables with p uncorrelated variables, referred to as the *principal components* (PCs) (Fig. 9.1). PCA, which was first introduced by Pearson (1901) and further developed by Hotelling (1931), is the standard method for unmixing (or separating) mixed variables. PCA is equivalent to fitting a p -dimensional ellipsoid to a data set of p variables, where the p eigenvectors of the p -by- p covariance matrix of the data set are the axes of the ellipsoid. PCA is therefore one of the eigenvector methods. The p eigenvalues represent the distribution of the variance between each of the eigenvectors.

The performance of PCA is best illustrated with a bivariate data set of the two variables, x_1 and x_2 , that exhibit a strong linear correlation, by fitting a two-dimensional ellipse to the data with the two eigenvectors, PC_1 and PC_2 , of the 2-by-2 covariance matrix of the data set (Fig. 9.1). Since the first principal component PC_1 passes through the long axis of the data scatter and the new origin is the bivariate mean, PC_1 can be used to describe most of the variance, while PC_2 contributes only a small amount of additional information. Prior to this transformation two axes, x_1 and x_2 , were required to describe the data set, but it is now possible to reduce the dimensions of the data by dropping the second axis PC_2 without losing very much information, as shown in Fig. 9.1.

Since the method produces signals, PC_1 and PC_2 , that are linearly uncorrelated, this method is also called whitening since this property is characteristic of white noise. An important prerequisite for successful application of this method is that the data are Gaussian distributed. Although the separated signals are uncorrelated, they can still be interdependent, i.e. they can retain a nonlinear correlation. This phenomenon arises when, for example, the data are not Gaussian distributed and the PCA consequently does not yield useful results. The independent component analysis (ICA) was developed for this type of task; it separates p independent variables, which are then nonlinearly uncorrelated, from the p dependent variables (Sect. 9.3).

This process is now expanded to an arbitrary number of variables and samples. Assume a data set that consists of measurements of p variables on n samples, stored in an n -by- p array X .

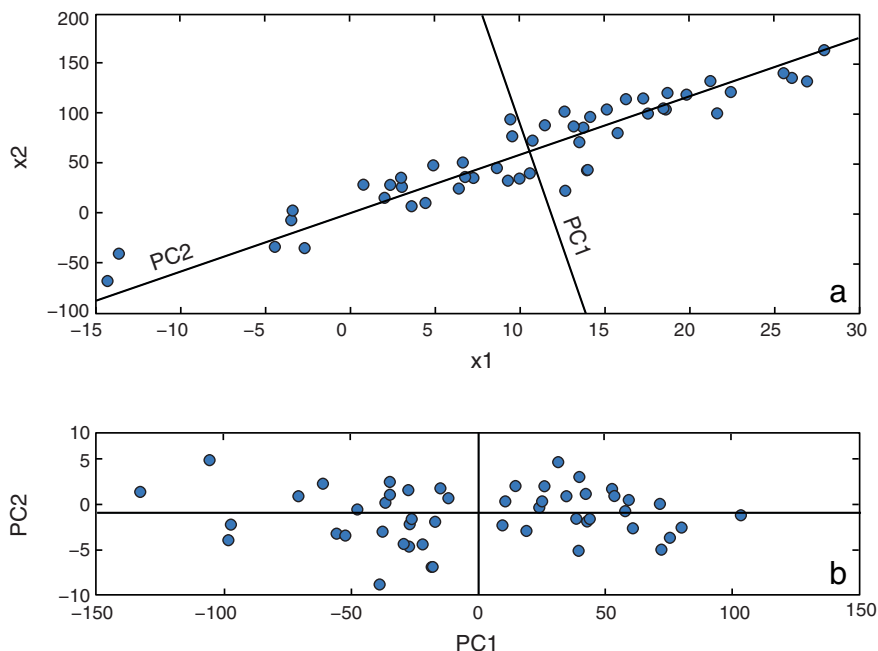


Fig. 9.1 Principal component analysis (PCA) illustrated for a bivariate scatter. The original x_1x_2 coordinate system is replaced by a new orthogonal system in which the 1st axis PC_1 passes through the long axis of the data scatter and the new origin is the bivariate mean. We can now reduce the number of dimensions by dropping the 2nd axis PC_2 with little loss of information. Modified from Swan and Sandilands (1995).

$$X = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & & \vdots \\ x_{n1} & x_{n2} & & x_{np} \end{pmatrix}$$

The columns of the array X represent the p variables and the rows represent the n samples. After rotating the axis and moving the origin, the new coordinates Y_j can be computed by

$$\begin{aligned} Y_1 &= a_{11}X_1 + a_{12}X_2 + \dots + a_{1p}X_p \\ Y_2 &= a_{21}X_1 + a_{22}X_2 + \dots + a_{2p}X_p \\ &\vdots \\ Y_p &= a_{p1}X_1 + a_{p2}X_2 + \dots + a_{pp}X_p \end{aligned}$$

The first principal component (PC_1), which is denoted by Y_1 , contains the highest variance, PC_2 contains the second highest variance, and so forth. All the PC s together contain the full variance of the data set. This variance is, however, largely concentrated in the first few PC s, which include most of the information content of the data set. The last PC s are therefore generally ignored to reduce the dimensions of the data. The factors a_{ij} in the above equations are the *principal component loads*; their values represent the relative contributions of the original variables to the new PC s. If the load a_{ij} of a variable X_j in PC_1 is close to zero then the influence of this variable is low, whereas a high positive or negative a_{ij} suggests a strong influence. The new values Y_j of the variables computed from the linear combinations of the original variables X_j , weighted by the loads, are called the *principal component scores*.

PCA is commonly used to unmix (or separate) variables X , which are a linear combination of independent source variables S

$$X = A^T \cdot S$$

where A is the mixing matrix. PCA tries to determine (although not quantitatively) both the source variables S (represented by the principal components scores Y_i) and the mixing matrix A (represented by the principal component loads a_{ij}). Unmixing such variables works best if the probability distribution of the original variables X is a Gaussian distribution, and only in such cases are the principal components completely decorrelated. However, data in earth sciences are often not Gaussian distributed and alternative methods, such as independent component analysis (ICA), should therefore be used instead (Sect. 9.3). For example, radiance and reflectance values from hyperspectral data are often not Gaussian distributed and ICA is therefore widely used in remote sensing applications to decorrelate the spectral bands, rather than PCA. Examples in which PCA is used include the assessment of sediment provenance (as described in the example below), the unmixing of peridotite mantle sources of basalts, and multispectral classification of satellite images.

The first example illustrates PCA without using the MATLAB function `pca`, but instead using the function `cov` to calculate the p -by- p covariance matrix of a $p = 2$ dimensional data set and then `eig` to calculate the p eigenvectors and p eigenvalues of this matrix. Assume a data set that consists of measurements of p variables on n samples, stored in an n -by- p array called `data`. As an example we can create a bivariate data set of 2-by-30 data points each, with a strong linear correlation, overlain by normally distributed noise.

```
clear, clc, close all

rng(0)
data(:,1) = randn(30,1);
data(:,2) = 3.4 + 1.2 * data(:,1);
data(:,2) = data(:,2) + 0.2*randn(size(data(:,1)));
data = sortrows(data,1);
```

We can display the data in a coordinate system in which the coordinate axes intersect at the origin. This is achieved using the axis properties `XAxisLocation` and `YAxisLocation` with the value `Origin` (Fig. 9.2a).

```
axes('XAxisLocation','Origin',...
     'YAxisLocation','Origin',...
     'XLim',[-6 6],...
     'YLim',[0 8]);
line(data(:,1),data(:,2),...
     'LineStyle','None',...
     'Marker','o');
xlabel('x-value')
ylabel('y-value')
```

The objective is to find the unit vector pointing in the direction of the largest variance within the bivariate data set. The solution to this problem is to calculate the largest eigenvalue D of the covariance matrix C and the corresponding eigenvector V .

$$C \cdot V = \lambda \cdot V$$

where

$$C = data' \cdot data$$

which is equivalent to

$$(C - D \cdot E) \cdot V = 0$$

where E is the identity matrix, which is a classic eigenvalue problem: it is a linear system of equations with the eigenvectors V as the solution. We first have to mean center the data, which involves subtracting the univariate means from the two columns (i.e. the two variables of data). In the following steps we therefore study the deviations from the mean(s) only.

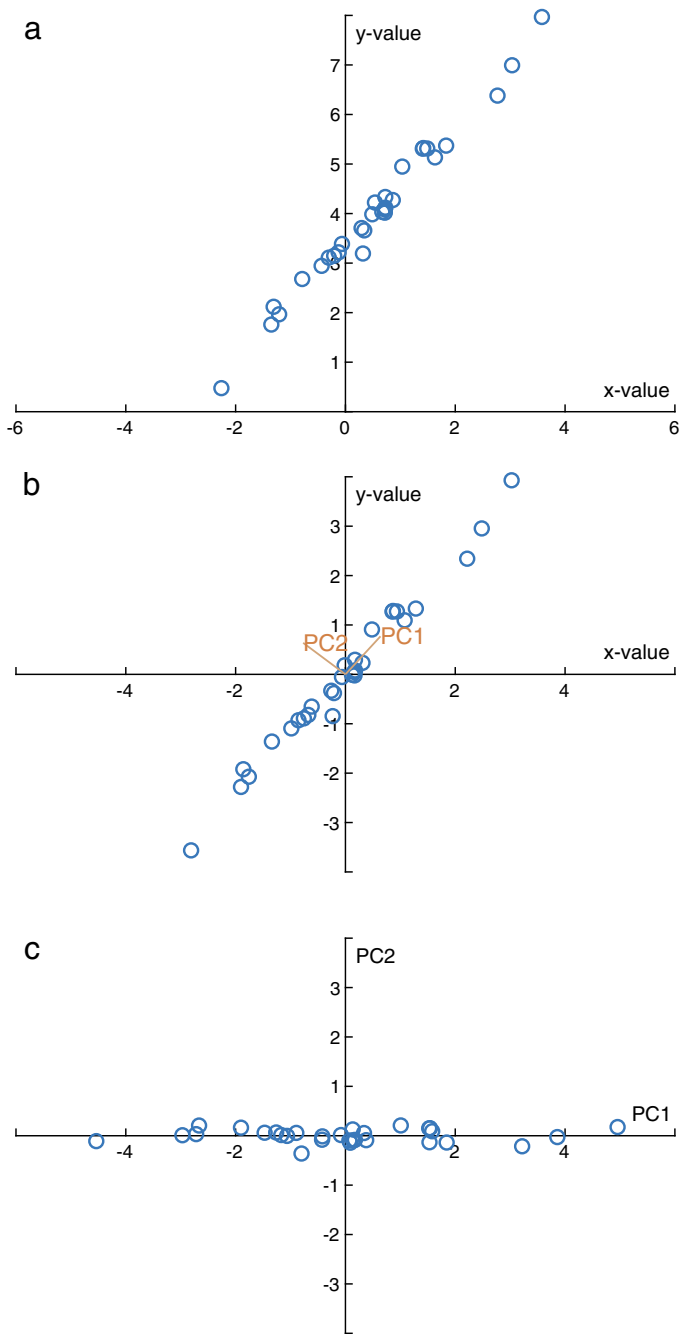


Fig. 9.2 Principal component analysis (PCA) of a bivariate data set: **a** original data set of two variables x_1 and x_2 , **b** mean-centered data set of two variables x_1 and x_2 together with the two principal components PC_1 and PC_2 of the covariance matrix, and **c** the data set described by the new coordinate system of the variables PC_1 and PC_2 .

```
data(:,1) = data(:,1)-mean(data(:,1));
data(:,2) = data(:,2)-mean(data(:,2));
```

We next calculate the covariance matrix C of `data`. The covariance matrix contains all information required to rotate the coordinate system.

```
C = cov(data)
```

which yields

```
C =
    1.6901    2.0186
    2.0186    2.4543
```

The rotation helps to create new variables that are uncorrelated, i.e. the covariance is zero for all pairs of the new variables. The decorrelation is achieved by diagonalizing the covariance matrix C . The eigenvectors V belonging to the diagonalized covariance matrix are a linear combination of the old base vectors, thus expressing the correlation between the old and the new time series. We find the eigenvalues of the covariance matrix C by solving the equation

$$\det(C - D \cdot E) = 0$$

The eigenvalues D of the covariance matrix, i.e. the diagonalized version of C , give the variance within the new coordinate axes, i.e. the principal components. We now calculate the eigenvectors V and eigenvalues D of the covariance matrix C .

```
[V,D] = eig(C)
```

which yields

```
V =
   -0.7701    0.6380
    0.6380    0.7701

D =
    0.0177    0
    0         4.1266
```

As we can see, the second eigenvalue of 4.1266 is larger than the first eigenvalue of 0.0177. We therefore need to change the order of the new variables after the transformation. We display the data, together with the eigenvectors representing the new coordinate system (Fig. 9.2b).

```

axes('XAxisLocation','Origin',...
     'YAxisLocation','Origin',...
     'XLim',[-6 6],...
     'YLim',[-4 4]);
line(data(:,1),data(:,2),...
      'LineStyle','None',...
      'Marker','o');
line([0 V(1,1)],[0 V(2,1)],...
      'Color',[0.8 0.5 0.3],...
      'LineWidth',0.75);
text(V(1,1),V(2,1),'PC2',...
      'Color',[0.8 0.5 0.3])
line([0 V(1,2)],[0 V(2,2)],...
      'Color',[0.8 0.5 0.3],...
      'LineWidth',0.75);
text(V(1,2),V(2,2),'PC1',...
      'Color',[0.8 0.5 0.3])
xlabel('x-value')
ylabel('y-value')

```

The eigenvectors are unit vectors and orthogonal, therefore the norm is one and the inner (scalar, dot) product is zero.

```

norm(V(:,1))
norm(V(:,2))
dot(V(:,1),V(:,2))

```

which yields

```

ans = 1
ans = 1
ans = 0

```

We now calculate the data set in the new coordinate system, `newdata`. We need to flip `newdata` left/right since the second column is the one with the largest eigenvalue, as stated previously,

```

newdata = V * data';
newdata = newdata';
newdata = fliplr(newdata)

```

which yields

```
newdata =
-4.5374  0.1091
-2.9687 -0.0101
-2.6655 -0.2065
-2.7186 -0.0324
-1.9021 -0.1632
-1.4716 -0.0606
-1.2612 -0.0659
-1.1692 -0.0142
(cont'd)
```

The eigenvalues of the covariance matrix indicate the variance in this (new) coordinate direction. We can use this information to calculate the relative variance for each new variable by dividing the variances according to the eigenvectors by the sum of the variances. In our example, the 1st and 2nd principal components contain 99.57 and 0.43% of the total variance in the data.

```
var(newdata)
var(newdata)/sum(var(newdata))
```

which yields

```
ans =
4.1266 0.0177

ans =
0.9957 0.0043
```

Alternatively, we can obtain the variances by normalizing the eigenvalues:

```
variance = D / sum(D(:))
```

which yields

```
variance =
0.0043  0
0      0.9957
```

and again indicating 99.57 and 0.43% of the variance for the two principal components. We now display the data in `newdata` in the new coordinate system (Fig. 9.2c).

```
axes('XAxisLocation','Origin',...
     'YAxisLocation','Origin',...
     'XLim',[-6 6],...
     'YLim',[-4 4]);
line(newdata(:,1),newdata(:,2),...
     'LineStyle','None',...
     'Marker','o');
xlabel('PC1')
ylabel('PC2')
```

We can now carry out the same experiment using the MATLAB function `pca`. The input for the function is `data`; the function then returns a number of variables, some of which we suppress by using `~`. We are only interested in the variables `pcs`, `newdata` and `variance`, where `pcs` are the principal components, i.e. the eigenvectors V of the covariance matrix C .

```
[pcs,newdata,~,~,variance] = pca(data)
```

As we can see, we get the same values as before.

```
pcs =
    0.6380    0.7701
    0.7701   -0.6380

newdata =
   -4.5374    0.1091
   -2.9687   -0.0101
   -2.6655   -0.2065
   -2.7186   -0.0324
   -1.9021   -0.1632
   -1.4716   -0.0606
   -1.2612   -0.0659
   -1.1692   -0.0142
(cont'd)
```

```
variance =
```

```
99.5720
```

```
0.4280
```

As expected, the data have been successfully decorrelated, which we can verify with `corrcoef`.

```
corrcoef(newdata)
```

which yields

```
ans =
```

```
1.0000    0.0000
```

```
0.0000    1.0000
```

The second synthetic example demonstrates linear unmixing of a data set of sediment samples. Thirty samples were taken from thirty different levels in a sedimentary sequence containing varying proportions of the three different minerals stored in the columns of the array `x`. The sediments were derived from three distinct rock types (with unknown mineral compositions) whose relative contributions to each of the thirty sediment samples are represented by `s1`, `s2` and `s3`. Variations in these relative contributions (as represented by the thirty values in `s1`, `s2` and `s3`) could, for example, reflect climatic variability within the catchment area of the sedimentary basin. It may therefore be possible to use the sediment compositions in the array `x` (from which we calculate `s1`, `s2` and `s3` using a PCA) to derive information on past variations in climate

We need to create a synthetic data set consisting of three measurements representing the proportions of each of the three minerals in the each of the thirty sediment samples. We first clear the workspace, reset the random number generator with `rng(0)` and create thirty values `s1`, `s2` and `s3`. We use random numbers with a Gaussian distribution generated using `randn`, with means of zero and standard deviations of 10, 7 and 12.

```
clear
```

```
rng(0)
```

```
s1 = 10*randn(30,1);
```

```
s2 = 7*randn(30,1);
```

```
s3 = 12*randn(30,1);
```


We then calculate the varying proportions of each of the three minerals in the thirty sediment samples by summing up the values in s_1 , s_2 and s_3 , after first multiplying them by a weighting factor.

```
x(:,1) = 15.4+7. 2*s1+10.5*s2+2.5*s3;
x(:,2) = 124.0-8.73*s1+ 0.1*s2+2.6*s3;
x(:,3) = 100.0+5.25*s1- 6.5*s2+3.5*s3;
```

The weighting factors, which together represent the mixing matrix in our exercise, reflect not only differences in the mineral compositions of the source rocks, but also differences in the weathering, mobilization, and deposition of minerals within sedimentary basins. Hence if two minerals have weighting factors with different signs, one could be (for example) the weathering product of the other mineral, which would explain why their proportions in the thirty sediment samples were anti-correlated. Alternatively, the different signs could indicate a dilution effect, i.e. if the proportions of one of the minerals in the sediment samples remain constant but the proportions of all other minerals vary in a similar way, they will be anti-correlated with the proportions of the first mineral. To complete the generation of the data set, we add some Gaussian noise with a standard deviation of 3.8 to the proportions of the minerals and save the result as *sediment_1.txt*.

```
x = x + 3.8*randn(size(x));

save sediments_1.txt x -ascii
```

Having seen how the sedimentary record was created and the dependencies that exist within the data set, we will now pretend that we do not know the relative contributions that the three source rocks made to the thirty sediment samples. Instead we have a record of the mineral contents of thirty sediment samples stored in the file *sediment_1.txt*.

```
clear

x = load('sediments_1.txt');
```

The aim of the PCA is now to decipher the statistically independent contribution of the three source rocks to the sediment compositions. We can display the histograms of the data and see that they are not perfectly Gaussian distributed, which means that we cannot expect a perfect unmixing result.

```
subplot(1,3,1), histogram(x(:,1))
subplot(1,3,2), histogram(x(:,2))
subplot(1,3,3), histogram(x(:,3))
```

We display the proportions of the three minerals in the thirty samples along the sedimentary section. In this graphic we can see weak correlations and anti-correlations between the proportions of the three minerals.

```
plot(x(:,1:3)), grid
legend('Min1', 'Min2', 'Min3')
xlabel('Sample ID')
ylabel('Quantity')
```

Before running the PCA we define labels for the various graphics created during the exercise. We number the samples 1 to 30, with the minerals being identified by four-character abbreviations Min1, Min2 and Min3.

```
for i = 1 : 30
    samples(i,:) = [sprintf('%02.0f',i)];
end
minerals = ['Min1'; 'Min2'; 'Min3'];
```

We can explore the correlations between the minerals in pairwise bivariate scatter plots. We observe a strong negative correlation between the first and second mineral, a weak positive correlation between the first and third mineral, and a moderate negative correlation between the second and third mineral.

```
subplot(1,3,1),
plot(x(:,1),x(:,2),'o')
xlabel('Mineral 1')
ylabel('Mineral 2')
subplot(1,3,2),
plot(x(:,1),x(:,3),'o')
xlabel('Mineral 1')
ylabel('Mineral 3')
subplot(1,3,3),
plot(x(:,2),x(:,3),'o')
xlabel('Mineral 2')
ylabel('Mineral 3')
```

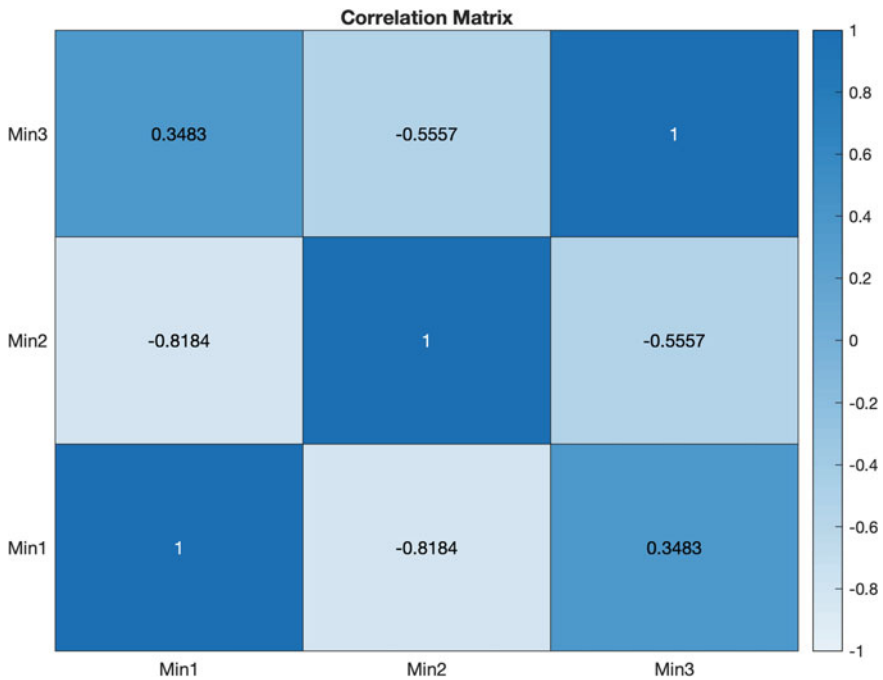


Fig. 9.3 Correlation matrix containing Pearson’s correlation coefficients for each pair of variables (for example, minerals in a sediment sample). Light colors represent strong positive linear correlations, while dark colors represent negative correlations. Orange suggests no correlation.

The *correlation matrix* provides a technique for exploring such dependencies between the variables in the data set (i.e. the three minerals in our example). The elements of the correlation matrix are Pearson’s correlation coefficients (Chap. 4) for each pair of variables, as shown in Fig. 9.3.

```
corrmatrix = corrcoef(x);  
corrmatrix = flipud(corrmatrix);
```

We can display the correlation matrix using a heatmap chart.

```
heatmap(corrmatrix,...
        'Title','Correlation Matrix',...
        'XDisplayLabels',minerals,...
        'YDisplayLabels',flipud(minerals))
```

This heatmap of the correlation coefficients confirms the correlations between the minerals revealed in the pairwise bivariate scatter plots, again showing a strong negative correlation between the first and second minerals ($r = -0.8184$), a weak positive correlation between minerals 1 and 3 ($r = 0.3483$), and a moderate negative correlation between minerals 2 and 3 ($r = -0.5557$). These observed dependencies would lead us to expect interesting results from the application of a PCA.

Various methods exist for scaling the original data before applying a PCA, such as *mean centering* (using a mean equal to zero) or *standardizing* (using a mean equal to zero and a standard deviation equal to one). We will, however, use the original data for the PCA. The output of the function `pca` includes the principal component loads `pcs`, the scores `newx`, and the variances `variances`. The loads `pcs` are weights (or weighting factors) that indicate the extent to which the old variables (the minerals) contribute to the new variables (the principal components, or PCs). The principal component scores are the coordinates of the thirty samples in the new coordinate system defined by the three principal components, PC_1 to PC_3 (stored in the three columns of `pcs`), which we interpret as the three source rocks.

```
[pcs,newx,variances] = pca(x);
```

The loads of the three principal components PC_1 to PC_3 can be shown by typing

```
pcs(:,1:3)

ans =
    0.6342   -0.5085    0.5825
   -0.6215    0.1130    0.7753
    0.4600    0.8536    0.2444
```

We observe that PC_1 (first column) has high positive loads in variables 1 and 3 (first and third rows), and a high negative load in variable 2 (second row). PC_2 (second column) has a high negative load in variable 1 and a high positive load in variable 3, while the load in variable 2 is close to zero. PC_3 (third column) has high loads in variables 1 and 2, with the load in variable 3 being relatively low but also positive. We create a number of plots to visualize the PCs:

```

subplot(1,3,1)
plot(1:3,pcs(:,1),'o'), axis([0.5 3.5 -1 1])
text((1:3)+0.2,pcs(:,1),minerals,'FontSize',14), hold
plot(0.5:3.5,zeros(4,1),'r'), title('PC 1')
subplot(1,3,2)
plot(1:3,pcs(:,2),'o'), axis([0.5 3.5 -1 1])
text((1:3)+0.2,pcs(:,2),minerals,'FontSize',14), hold
plot(0.5:3.5,zeros(4,1),'r'), title('PC 2')
subplot(1,3,3)
plot(1:3,pcs(:,3),'o'), axis([0.5 3.5 -1 1])
text((1:3)+0.2,pcs(:,3),minerals,'FontSize',14), hold
plot(0.5:3.5,zeros(4,1),'r'), title('PC 3')

```

The loads of the minerals and their relationships to the *PCs* can be used to interpret the relative (not absolute) influences of the different source rocks. PC_1 is characterized by strong positive contributions of minerals 1 and 3, reflecting a relatively strong influence of the first rock type as a source of the sediments. An opposite sign for the contribution of mineral 2 to the sediment reflects either different mobilization processes for this mineral within the catchment area, or a dilution effect. The second principal component PC_2 is also dominated by minerals 1 and 3, but with opposite signs, while mineral 2 has relatively little influence. The third principal component PC_3 is influenced by all three minerals, with the same sign. An alternative way to plot the loads is as a bivariate plot of two principal components. We ignore PC_3 at this point and concentrate on PC_1 and PC_2 . Remember to either close the figure window before plotting the loads or clear the figure window using `clf`, in order to avoid integrating the new plot as a fourth subplot in the previous figure window.

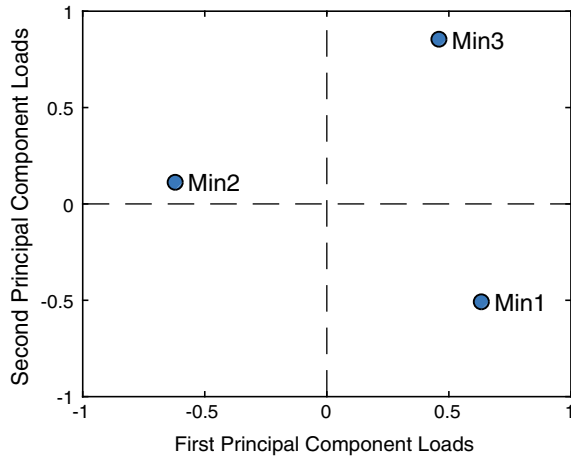
```

plot(pcs(:,1),pcs(:,2),'o'), hold on
text(pcs(:,1)+0.02,pcs(:,2),minerals,'FontSize',14)
plot([-1 1],[0 0],'r')
plot([0 0],[-1 1],'r')
xlabel('First Principal Component Loads')
ylabel('Second Principal Component Loads')
hold off

```

We can now observe in a single plot the same relationships that have previously been shown in several graphics (Fig. 9.4). It is also possible to plot the data set as functions of the new variables (the source rocks). This requires the second output of `pca`, containing the principal component scores.

Fig. 9.4 Principal component loads suggesting that the *PCs* are influenced by different minerals. See text for detailed interpretation of the *PCs*.



```
plot(newx(:,1),newx(:,2),'+'), hold on
text(newx(:,1)+2,newx(:,2),samples,'FontSize',14)
plot([-400 400],[0 0], 'r')
plot([0 0],[-150 300], 'r')
xlabel('First Principal Component Scores')
ylabel('Second Principal Component Scores')
hold off
```

This plot clearly defines groups of samples with similar influences, e.g. samples 19, 20 and 26, and possibly also sample 18. We next use the third output from the function `pca` to compute the variances of the *PCs*.

```
percent_explained = 100*variances/sum(variances)

percent_explained =
    72.4362
    22.7174
     4.8463
```

We see that more than 72% of the total variance is contained in PC_1 , and about 23% is contained in PC_2 , while PC_3 contributes very little to the total variance of the data set ($\sim 5\%$). This means that most of the variability in the data set can be described by just two new variables. As would be expected, the two new variables do not correlate with each other, as illustrated by a correlation coefficient between `newx(:,1)` and `newx(:,2)` that is close to zero.

```
corrcoef(newx(:,1),newx(:,2))

ans =

    1.0000    0.0000
    0.0000    1.0000
```

We can therefore plot the time series of the thirty samples as two independent variables, PC_1 and PC_2 , in a single plot.

```
plot(1:30,newx(:,1),1:30,newx(:,2))
grid, legend('PC1','PC2')
xlabel('Sample ID'), ylabel('Value')
```

This plot displays approximately 95% (72% + 23%) of the variance contained in the multivariate data set. According to our interpretation of PC_1 and PC_2 , this plot shows the variability in the relative contributions from the two sources to the sedimentary column under investigation. Since we have worked with a synthetic data set and the actual contribution of the three source rocks to the sediment is known from

```
rng(0)
s1 = 10*randn(30,1);
s2 = 7*randn(30,1);
s3 = 12*randn(30,1);
```

we can estimate the quality of the result by comparing the initial variations in s_1 , s_2 and s_3 with the (more or less) independent variables PC_1 , PC_2 and PC_3 stored in the three columns of $newx$.

```
subplot(3,1,1)
plotyy(1:30,newx(:,1),1:30,s1), title('PC1')
subplot(3,1,2)
plotyy(1:30,-newx(:,2),1:30,s2), title('PC2')
subplot(3,1,3)
plotyy(1:30,newx(:,3),1:30,s3), title('PC3')
```

The sign and the amplitude cannot be determined quantitatively and therefore, in this case, we change the sign of the second PC and use `plotyy` to display the data on different axes in order to compare the results. As we can see, we have successfully unmixed the varying contributions of the source rocks s_1 , s_2 and s_3 to the mineral composition of the sedimentary sequence.

The approach described above has been used to study the provenance of the varved lake sediments described in the previous chapter (Sect. 8.9), which were deposited around 33 kyrs ago in a landslide-dammed lake in the Quebrada de Cafayate (Trauth et al. 2003). The provenance of the sediments contained in the varved layers can be traced using index minerals characteristic of the various possible source areas within the catchment. A comparison of the mineral assemblages in the sediments with those of potential source rocks within the catchment area indicates that Fe-rich Tertiary sedimentary rocks exposed in the Santa Maria Basin were the source of the red-colored basal portion of the varves. In contrast, metamorphic rocks in the mountainous parts of the catchment area were the most likely source of the relatively drab-colored upper part of the varves.

9.3 Independent Component Analysis (by N. Marwan)

Principal component analysis (PCA) is the standard method for unmixing (or separating) mixed variables (Sect. 9.2). Such analyses produce signals that are linearly uncorrelated, and this method is also called *whitening* since this property is characteristic of white noise. Although the separated signals are uncorrelated, they can still be interdependent, i.e. they may retain a nonlinear correlation. This phenomenon arises when, for example, the data are not Gaussian distributed and the PCA consequently does not yield good results. The *independent component analysis* (ICA) was developed for this type of task; it separates the variables X into independent variables S , which are then nonlinearly uncorrelated. The basis of an ICA, according to the central limit theorem, is that the mixture of standardized random numbers is Gaussian distributed. The ICA algorithms therefore use a criterion that estimates how Gaussian the combined distribution of the independent components is (Hyvärinen 1999). The less Gaussian this distribution, the more independent the individual components.

According to the linear mixing model, p independent variables X are linearly mixed in n measurements,

$$X = A^T \cdot S$$

in which we are interested in the source variables S and the mixing matrix A (see Sect. 9.2). This is perhaps analogous to being at a party in which a lot of people are carrying on independent conversations: we can hear a mixture of these conversations but cannot distinguish them individually. We could perhaps install some microphones and use these to separate out the individual conversations. This dilemma is therefore sometimes known as the *cocktail party problem*; its correct term is *blind source separation*, which is defined by

$$S = W^T X$$

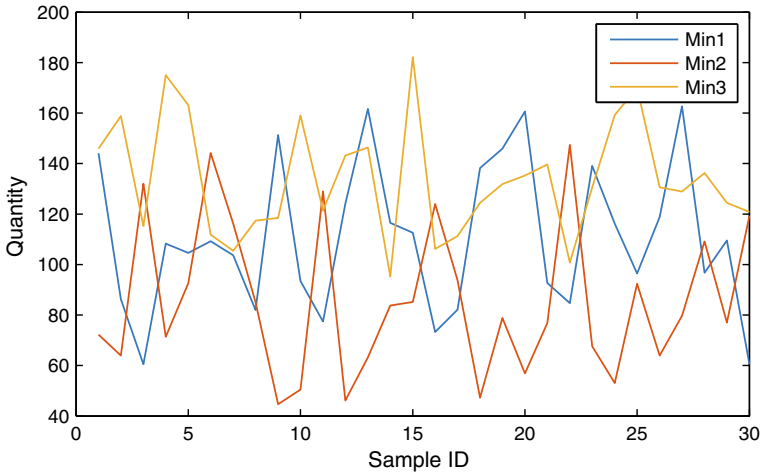


Fig. 9.5 Sample input for independent component analysis. The relative proportions of three minerals in 30 sediment samples reflect temporal variations in the contributions from three source rocks within the catchment of a sedimentary basin.

where W^T is the separation matrix required to reverse the mixing and obtain the original signals. In earth sciences we encounter similar problems, for example if we want to determine the relative contributions of different source rocks to basin sediments, as we did with the PCA, but this time with the possibility that there are nonlinear dependencies in the data and that these are not Gaussian distributed (Sect. 9.2).

We again create a synthetic data set consisting of thirty measurements (the proportions of each of the three minerals) from each of the thirty sediment samples. In contrast to the PCA example, however, the temporal variation in the source rocks is not Gaussian distributed but is uniformly distributed, since we use `rand` instead of `randn` to create the pseudorandom numbers.

```
clear, clc, close all
```

```
rng(0)
s1 = 10*rand(30,1);
s2 = 7*rand(30,1);
s3 = 12*rand(30,1);
```

We use the same mixing equation as in the PCA example to create the three columns of x (corresponding to the three minerals) by linearly mixing the source rocks $s1$, $s2$ and $s3$

```

x(:,1) = 15.4+7.2 *s1+10.5*s2+2.5*s3;
x(:,2) = 124.0-8.73*s1+ 0.1*s2+2.6*s3;
x(:,3) = 100.0+5.25*s1- 6.5*s2+3.5*s3;

x = x + 3.8*randn(size(x));

save sediments_2.txt x -ascii

```

and save the data in the file *sediments_2.txt* (Fig. 9.5). Having created the synthetic data set, the data stored in *sediment_2.txt* are loaded by typing

```

clear

x = load('sediments_1.txt');

```

We can display the histograms of the data to see that they are not Gaussian distributed.

```

subplot(1,3,1), hist(x(:,1))
subplot(1,3,2), hist(x(:,2))
subplot(1,3,3), hist(x(:,3))

```

We then display the proportions of the minerals in the thirty samples as a time series, in two separate plots.

```

plot(x(:,1:3)), grid
legend('Min1','Min2','Min3')
xlabel('Sample ID')
ylabel('Quantity')

```

We begin with unmixing the variables using PCA. We calculate the principal components *pcs*, the mixing matrix *a_pca*, and the whitening matrix *w_pca* using

```

[pcs,newx,variances] = pca(x);
newx = newx./repmat(std(newx),length(newx),1);

a_pca = pcs*sqrt(diag(variances));
w_pca = inv(sqrt(diag(variances)))*pcs';

```

The pre-whitening reduces the unmixing into independent components (*S*) to a problem of finding a suitable rotation matrix *B* that can be applied to the variables X_{PC} :

$$S = B^T X_{PC}$$

We need to find a rotation matrix B such that the variables S have a completely non-Gaussian distribution. There are several possibilities for implementing such a non-Gaussian criterion, such as minimizing or maximizing the excess kurtosis,

$$\gamma = E(x^4) - 3(E(x^2))^2$$

because the excess kurtosis γ for normally distributed data is zero. Please note that the excess kurtosis differs from the kurtosis: the excess kurtosis is the kurtosis minus three, since the kurtosis of Gaussian-distributed data is three (Sect. 3.2) but the excess kurtosis of such a distribution is zero. To find a B that ensures a minimum or maximum excess kurtosis, a learning algorithm can be used that finds the fixed points of the learning rule

$$E(x(B^T x)^3) - 3 \|B\|^2 B + f(\|B\|^2)B = 0$$

The FastICA algorithm by Hyvärinen (1999), which is based on a fixed-point iteration scheme, is an efficient algorithm for solving this problem. The learning rule is reduced in this example to the form

$$B(k) = E(x(B(k-1)^T x)^3) - 3B(k-1)$$

We begin with an initial rotation matrix B , which consists only of random numbers.

```
B = orth(rand(3,3)-0.5);
B0ld = zeros(size(B));
```

The iteration is continued until a divergence criterion `div` is reached. We choose a value of zero for this criterion.

```
div = 0;
```

The fixed-point iteration scheme consists of two steps: a symmetric orthogonalization, and the application of the learning rule. The divergence criterion `div` is updated with each iteration step and checked in order to stop the iteration process as soon as the divergence criterion is reached. Since MATLAB works with floating-point numbers, `div` will not actually reach zero and we therefore allow $(1-\text{div})$ to fall below the floating-point relative accuracy `eps(1)`, or `eps` without input parameter 1 (since the default input for `eps` is 1) which is about $2.2204\text{e-}16$ and therefore close to zero (see Sect. 2.5).

```

while (1-div) > eps
    B = B*real(inv(B'*B)^(1/2));
    div = min(abs(diag(B'*B0ld)));
    B0ld = B;
    B = (newx'*(newx*B).^3)/length(newx)-3*B;
    sica = newx*B;
end

```

Finally, we compare the synthetic source rocks s_1 , s_2 and s_3 with the unmixed variables IC1, IC2 and IC3 (Fig. 9.6).

```

rng(0)
s1 = 10*rand(30,1);
s2 = 7*rand(30,1);
s3 = 12*rand(30,1);

subplot(3,1,1)
plotyy(1:30,-sica(:,3),1:30,s1)
title('IC1')
subplot(3,1,2)
plotyy(1:30,sica(:,2),1:30,s2)
title('IC2')
subplot(3,1,3)
plotyy(1:30,sica(:,1),1:30,s3)
title('IC3')

```

The ICA has identified the source signals almost perfectly. We notice that the descending order of the ICs is different from the initial order of s_1 , s_2 and s_3 , which is due to the commutativity of addition. In real-world examples, of course, the order of the ICs is not relevant. Furthermore, the exact sign and amplitude does not match the original values and the ICA therefore yields only semi-quantitative results, as was also the case for the PCA. Finally, we can actually compute the mixing matrix a_{ica} and the separation matrix w_{ica} using

```

a_ica = a_pca*B;
w_ica = B'*w_pca;

```

The mixing matrix a_{ica} can be used to estimate the proportions of the separated variables in our measurements. The components a_{ij} of the mixing matrix a_{ica} correspond to the principal component loads, as introduced in Sect. 9.2. The

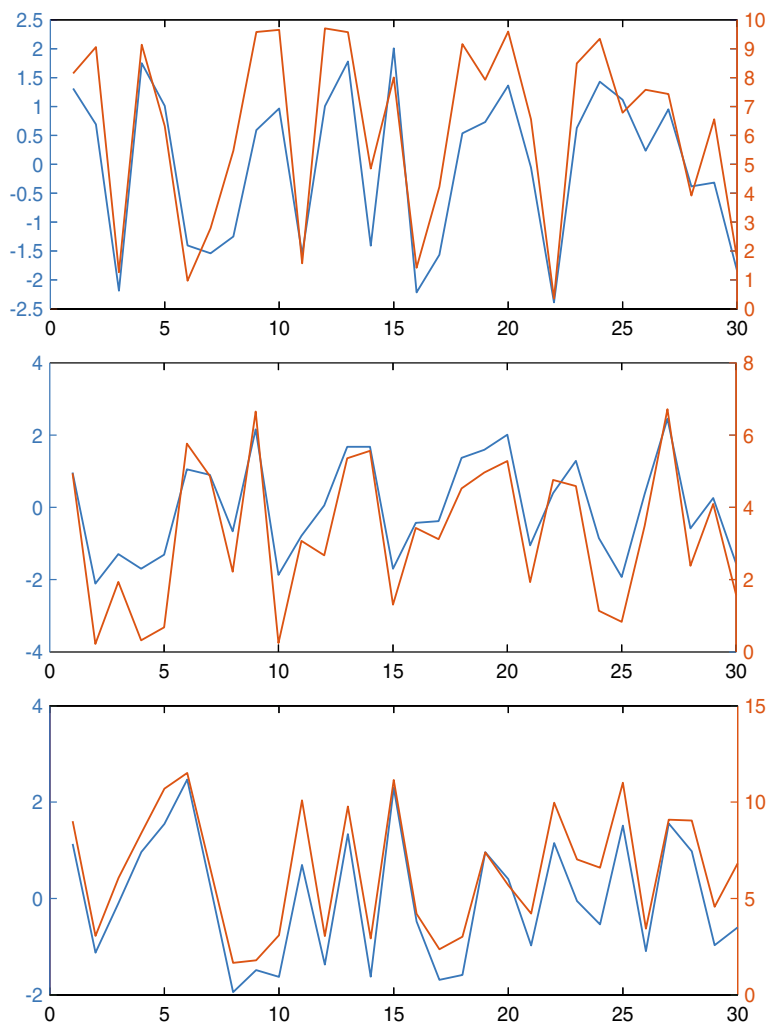


Fig. 9.6 Independent component analysis output. The ICA has identified the source signals almost perfectly, as the pairwise similarity of the original inputs and the independent components suggests.

FastICA package is available for MATLAB and can be found on A. Hyvärinen's webpage:

<http://research.ics.aalto.fi/ica/fastica/>

9.4 Discriminant Analysis

Discriminant analysis helps to assign objects to established categories or groups. Examples include the assignment of fossil specimens to established genera or species, the identification of rock types following mineralogical (or chemical) analysis, and the mapping of vegetation types from satellite images. Discriminant analysis is different from simple classification, which does not define the number of groups or categories prior to the analysis.

The classic example of a discriminant analysis in petrography is the QAPF or Streckeisen diagram (Streckeisen 1976). This diagram categorizes igneous rocks (especially plutonic rocks) by their percentages of quartz (Q), alkali feldspar (including albite) (A), plagioclase (P), and feldspathoids (F), normalized a total of 100%. The QAPF diagram displays the percentages of the four minerals in a double ternary plot, with QAP percentages in the upper half of the graphics and FAP percentages in the lower half. The QAPF diagram is commonly used by the International Union of Geological Sciences (IUGS) to classify plutonic rocks: e.g. a plutonic rock with 50% quartz, 30% alkali feldspar, 20% plagioclase and 0% feldspathoids is termed a granite. Whereas Albert Streckeisen's definition of plutonic rocks represents compromises between established usages in different parts of the world (Streckeisen 1976), discriminant analysis is based solely on mathematical constraints for the classification of objects. Furthermore, discriminant analysis assumes normality for the measured values within a class, which is probably not a valid assumption for the Streckeisen classification. Since normality is important for the success of the method, the user must be careful to ensure that this condition is actually met, especially when analyzing compositional (closed) data (see also Sect. 9.5).

Discriminant analysis was first introduced by Fisher (1936) to discriminate between two or more populations of flowering plant. The first step involves determining the discriminant function Y_i that best separate two groups of objects described by the normally-distributed variables X_i

$$Y = a_0 + a_1X_1 + a_2X_2 + \cdots + a_nX_n$$

The parameters a_i are determined to maximize the distance between the multivariate means of the individual groups. In other words, we determine a_i such that the ratio of the distances between the means of the groups to the distances between group members is high. In the second step, having determined the discriminant function from a training set of objects, new objects can be assigned to one group or the other. Using the Streckeisen diagram, a rock sample is assigned to an established rock type, such as granite, on the basis of the percentages of Q, A, P and F.

As an example we first create a synthetic data set of granite rock samples, described by two variables, x_1 and x_2 . These two variables could represent the percentages of two chemical elements expressed as oxides (in weight percent). Let us assume that we know from preliminary studies that these rock samples come from three different granites that were formed at different times during three

separate magmatic events. Apart from natural inhomogeneities within a granite intrusion, we can assume that the measured values from the granite samples are normally distributed. In this example we will first determine the discriminant functions separating the three groups (or types of granite). We then use the discriminant functions to assign rock samples (which were collected during a subsequent field campaign) to one of the three types of granite.

We first clear the workspace

```
clear, clc, close all
```

and then reset the random number generator.

```
rng(0)
```

Next, we generate a data set from the chemical compositions of the three types of granite. Sixty granite samples were collected from each rock type and chemically analyzed. The percentages of two chemical elements, x_1 and x_2 , we store in three variables `data1`, `data2` and `data3`.

```
data1 = randn(60,2);
data1(:,1) = 3.4 + 1.2*data1(:,1);
data1(:,2) = 1.7 + 0.4*data1(:,2);

data2 = randn(60,2);
data2(:,1) = 5.5 + 1.8*data2(:,1);
data2(:,2) = 2.9 + 0.6*data2(:,2);

data3 = randn(60,2);
data3(:,1) = 3.0 + 1.8*data2(:,1);
data3(:,2) = 0.3 + 1.2*data2(:,2);
```

In order to define the established categories (or classes) we create a character array containing the labels for each pair of measurements using `repmat`, which creates 60 copies of each string `Granite_1`, `Granite_2`, and `Granite_3` and stores them in `classes`.

```
classes(1:60,1:9) = repmat('Granite_1',60,1);
classes(61:120,1:9) = repmat('Granite_2',60,1);
classes(121:180,1:9) = repmat('Granite_3',60,1);
```

We then vertically concatenate the three variables `data1`, `data2` and `data3` to a single variable `data`.

```
data = [data1(:,1:2);data2(:,1:2);data3(:,1:2)];
```

We save the resulting synthetic data set in the binary file *granite.mat*.

```
save granite.mat
```

We have thus generated a synthetic data set from three groups of normally distributed data. We can load the data set (as we would with any other real data set) from the file *granite.mat* by typing

```
clear
```

```
load granite.mat
```

We then create a linear discriminant analysis classifier using `ClassificationDiscriminant.fit`.

```
cls = ClassificationDiscriminant.fit(data,classes);
```

The function returns a discriminant analysis model for predictors `data` and class labels `classes`. The layers of `cls` can be listed by typing

```
cls
```

resulting in the output

```
cls =  
  
ClassificationDiscriminant  
    ResponseName: 'Y'  
    CategoricalPredictors: []  
          ClassNames: [3×9 char]  
    ScoreTransform: 'none'  
    NumObservations: 180  
    DiscrimType: 'linear'  
          Mu: [3×2 double]  
    Coeffs: [3×3 struct]
```

Properties, Methods

The meaning of most fields in this output is obvious, except for `ScoreTransform` which is either a function handle for transforming scores or a string representing a built-in transformation function. We are not using the score transform which is why it is set to `none`. The field `Mu` contains the means of the two variables from the three

classes. The 3-by-3 structure `Coeffs` of the three classes contains the coefficients of the linear classification boundaries between classes 1 and 2, and between classes 2 and 3. Typing

```
cls.Coeffs
```

results in the output

```
ans =  
3x3 struct array with fields:  
    DiscrimType  
    Const  
    Linear  
    Class1  
    Class2
```

which lists the discriminant type (linear in our example), the class names of the pairs of classes *i* and *j*, and the two parameters `Const` and `Linear` describing the classification boundaries separating the two pairs of classes. We can extract the parameters of the classification boundaries by typing

```
K1 = cls.Coeffs(2,3).Const;  
L1 = cls.Coeffs(2,3).Linear;  
K2 = cls.Coeffs(1,2).Const;  
L2 = cls.Coeffs(1,2).Linear;
```

and store them in new variables `K1`, `L1`, `K2` and `L2`. We then store the bivariate means by typing

```
Mu1 = cls.Mu(1,:);  
Mu2 = cls.Mu(2,:);  
Mu3 = cls.Mu(3,:);
```

Finally, we display the result in a graphic using

```
h1 = axes('XLim',[-5 25],...  
    'YLim',[0 5],...  
    'Box','On');  
hold on  
  
line(data1(:,1),data1(:,2),...  
    'Marker','.',...  
    'MarkerSize',8,...
```

```

        'LineStyle','None',...
        'MarkerEdgeColor','r')
line(data2(:,1),data2(:,2),...
      'Marker','.',...
      'MarkerSize',8,...
      'LineStyle','None',...
      'MarkerEdgeColor','b')
line(data3(:,1),data3(:,2),...
      'Marker','.',...
      'MarkerSize',8,...
      'LineStyle','None',...
      'MarkerEdgeColor','m')
line(Mu1(:,1),Mu1(:,2),...
      'Marker','o',...
      'MarkerEdgeColor','k',...
      'MarkerSize',8,...
      'MarkerFaceColor','k')
line(Mu2(:,1),Mu2(:,2),...
      'Marker','o',...
      'MarkerEdgeColor','k',...
      'MarkerSize',8,...
      'MarkerFaceColor','k')
line(Mu3(:,1),Mu3(:,2),...
      'Marker','o',...
      'MarkerEdgeColor','k',...
      'MarkerSize',8,...
      'MarkerFaceColor','k')

h2 = legend('Granite 1','Granite 2','Granite 3',...
            'Location','SouthEast');

set(h2,'Box','Off')

f1 = @(x1,x2) K1 + L1(1)*x1 + L1(2)*x2;
h3 = ezplot(f1,[-5 12 0 5]);
set(h3,'Color','k')

f2 = @(x1,x2) K2 + L2(1)*x1 + L2(2)*x2;
h4 = ezplot(f2,[-5 10 0 5]);
set(h4,'Color','k')

title('Discriminant Analysis')

hold off

```

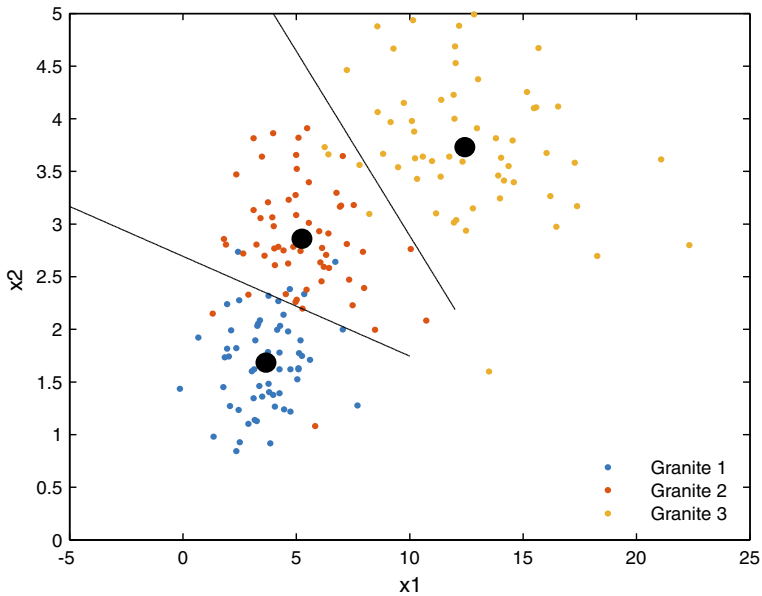


Fig. 9.7 Result of linear discriminant analysis, separating three types of granite (1, 2 and 3).

The graphic shows the members of the three classes (or types of granite) in three different colors, the two lines separating the three classes, and the bivariate means of the classes (marked as black filled circles) (Fig. 9.7). A new sample with the composition $x_1=5.2$ and $x_2=3.5$ can easily be assigned to the class Granite 2.

```
h1 = axes('XLim', [-5 25], ...
          'YLim', [0 5], ...
          'Box', 'On');
hold on

line(5.2, 3.5, ...
     'Marker', 'o', ...
     'MarkerEdgeColor', 'b', ...
     'MarkerSize', 8, ...
     'MarkerFaceColor', 'b')

f1 = @(x1, x2) K1 + L1(1)*x1 + L1(2)*x2;
h3 = ezplot(f1, [-5 12 0 5]);
set(h3, 'Color', 'k')
```

```

f2 = @(x1,x2) K2 + L2(1)*x1 + L2(2)*x2;
h3 = ezplot(f2, [-5 10 0 5]);
set(h3, 'Color', 'k')

text(0,2, 'Granite 1', 'Color', 'r')
text(-0.5,4, 'Granite 2', 'Color', 'b')
text(10,4.5, 'Granite 3', 'Color', 'm')

title('Discriminant Analysis')

hold off

```

The function `ClassificationDiscriminant.fit` also allows a quadratic classifier to be fitted instead of the linear classifier used in the example with three types of granite (MathWorks [2020b](#)).

9.5 Cluster Analysis

Cluster analysis creates groups of objects that are very similar to each other, compared to other individual objects or groups of objects. It first computes the similarity or (alternatively) the dissimilarity (or distance) between all pairs of objects and then ranks the groups according to their similarity or distance, finally creating a hierarchical tree visualized as a dendrogram. The grouping of objects can be useful in the earth sciences, for example when making correlations within volcanic ash layers (Hermanns et al. [2000](#)) or comparing different microfossil assemblages (Birks and Gordon [1985](#)).

There are numerous methods for calculating the similarity or (alternatively) the dissimilarity (or distance) between two data vectors. Let us define two data sets consisting of multiple measurements on the same object. These data can be described by vectors:

$$\begin{aligned}
 X_1 &= (x_{11}, x_{12}, \dots, x_{1p}) \\
 X_2 &= (x_{21}, x_{22}, \dots, x_{2p})
 \end{aligned}$$

The most popular measures of dissimilarity (or distance) between the two sample vectors are:

- the *Euclidian distance*—This is simply the shortest distance between the two points describing two measurements in the multivariate space:

$$\Delta_{12} = \sqrt{(x_{11} - x_{21})^2 + (x_{12} - x_{22})^2 + \cdots + (x_{1p} - x_{2p})^2}$$

The Euclidian distance is certainly the most intuitive measure for similarity. However, in heterogeneous data sets consisting of a number of different types of variables, a better alternative would be.

- the *Manhattan* (or *city block*) *distance*—In the city of Manhattan, one must follow perpendicular avenues rather than crossing blocks diagonally. The Manhattan distance is therefore the sum of all differences:

$$\Delta_{12} = |x_{11} - x_{21}| + |x_{12} - x_{22}| + \cdots + |x_{1p} - x_{2p}|$$

Measures of similarity include

- the *correlation similarity coefficient*—This uses Pearson's linear product-moment correlation coefficient to compute the similarity between two objects:

$$r_{x_1x_2} = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{(n-1)s_{x_1}s_{x_2}}$$

This measure is used if one is interested in the ratios between the variables measured on the objects. However, Pearson's correlation coefficient is highly sensitive to outliers and should be used with care (see also Sect. 4.2).

- the *inner-product similarity index*—Normalizing the length of the data vectors to a value of one and computing their inner product yields another important similarity index that is often used in transfer function applications. In this example a set of modern flora or fauna assemblages with known environmental preferences is compared with a fossil sample, in order to reconstruct past environmental conditions.

$$s_{12} = \frac{1}{|X_1|} \frac{1}{|X_2|} (x_{11}x_{12} \cdots x_{1p}) \begin{pmatrix} x_{21} \\ x_{22} \\ \vdots \\ x_{2p} \end{pmatrix}$$

The inner-product similarity varies between 0 and 1. A zero value suggests no similarity and a value of one represents maximum similarity.

The second step in performing a cluster analysis is to rank the groups by their similarity and to build a hierarchical tree, visualized as a dendrogram. Most clustering algorithms simply link the two objects with the highest level of similarity or dissimilarity (or distance). In the following steps, the most similar pairs of objects or clusters are linked iteratively. The difference between clusters, each made up of groups of objects, is described in different ways depending on the type of data and the application:

- *K-means clustering* uses the Euclidean distance between the multivariate means of a number of K clusters as a measure of the difference between the groups of objects. This distance is used if the data suggest that there is a true mean value surrounded by random noise. Alternatively,
- *K-nearest-neighbors clustering* uses the Euclidean distance of the nearest neighbors as a measure of this difference. This is used if there is a natural heterogeneity in the data set that is not attributed to random noise.

It is important to evaluate the data properties prior to the application of a clustering algorithm. The absolute values of the variables should first be considered. For example a geochemical sample from volcanic ash might show an SiO_2 content of around 77% and an Na_2O content of only 3.5%, but the Na_2O content may be considered to be of greater importance. In such a case the data need to be transformed so that they have means equal to zero (*mean centering*). Differences in both the variances and the means are corrected by *standardizing*, i.e. the data are standardized to means equal to zero and variances equal to one. Artifacts arising from closed data, such as artificial negative correlations, are avoided by using *Aitchison's log-ratio transformation* (Aitchison 1984, 1986) (see Sect. 9.7). This ensures data independence and avoids the constant sum normalization constraints. The log-ratio transformation is

$$x_{tr} = \log(x_i/x_d)$$

where x_{tr} denotes the transformed score ($i = 1, 2, 3, \dots, d-1$) of some raw data x_i . The procedure is invariant under the group of permutations of the variables, and any variable can be used as the divisor x_d .

As an exercise in performing a cluster analysis, the sediment data stored in *sediment_3.txt* are loaded. This data set contains the percentages of various minerals contained in sediment samples. The sediments are sourced from three rock types: a

magmatic rock containing amphibole (*amp*), pyroxene (*pyr*) and plagioclase (*pla*), a hydrothermal vein characterized by the presence of fluorite (*flu*), sphalerite (*sph*) and galena (*gal*), some feldspars (plagioclase and potassium feldspars, *ksp*) and quartz, and a sandstone unit containing feldspars, quartz and clay minerals (*cla*). Ten samples were taken from various levels in this sedimentary sequence, each containing varying proportions of these minerals. First, the distances between pairs of samples can be computed. The function `pdist` provides many different measures of distance, such as the Euclidian or Manhattan (or city block) distance. We use the default setting which is the Euclidian distance.

```
clear, clc, close all

data = load('sediments_3.txt');
Y = pdist(data);
```

The function `pdist` returns a vector `Y` containing the distances between each pair of observations in the original data matrix. We can visualize the distances in a heatmap.

```
heatmap(squareform(Y));
title('Euclidean distance between pairs of samples')
xlabel('First Sample No.')
ylabel('Second Sample No.')
```

The function `squareform` converts `Y` into a symmetric, square format, so that the elements (*i*, *j*) of the matrix denote the distance between the *i* and *j* objects in the original data. We next rank and link the samples with respect to the inverse of their separation distances using the function `linkage`.

```
Z = linkage(Y)

Z =

    2.0000    9.0000    0.0564
    8.0000   10.0000    0.0730
    1.0000   12.0000    0.0923
    6.0000    7.0000    0.1022
   11.0000   13.0000    0.1129
    3.0000    4.0000    0.1604
   15.0000   16.0000    0.1737
    5.0000   17.0000    0.1764
   14.0000   18.0000    0.2146
```

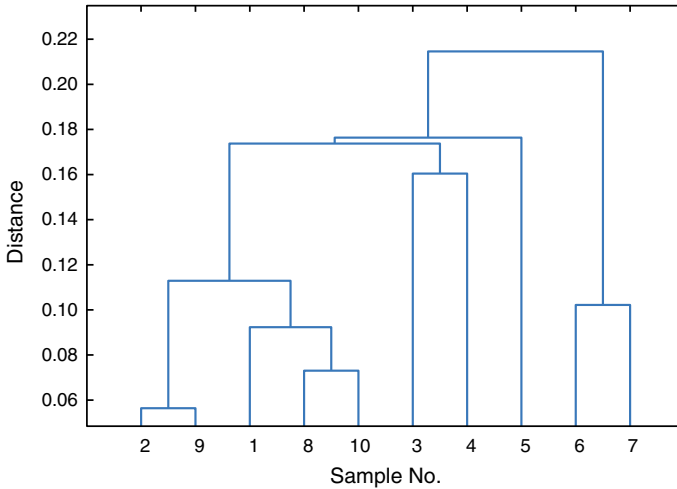


Fig. 9.8 Cluster analysis output. The dendrogram shows clear groups consisting of samples 1, 2, 8, 9 and 10 (the magmatic source rocks), samples 3, 4 and 5 (the magmatic dyke containing ore minerals), and samples 6 and 7 (the sandstone unit).

In this 3-column array Z , each row identifies a link. The first two columns identify the objects (or samples) that have been linked, while the third column contains the separation distance between these two objects. The first row (link) between objects (or samples) 1 and 2 has the smallest distance, corresponding to the greatest similarity. In our example samples 2 and 9 have the smallest separation distance of 0.0564 and are therefore grouped together and given the label 11, i.e. the next available index higher than the highest sample index 10. Next, samples 8 and 10 are grouped to 12 since they have the second smallest separation difference of 0.0730. The next row shows that the new group 12 is then grouped with sample 1, which have a separation difference of 0.0923, and so forth. Finally, we visualize the hierarchical clusters as a dendrogram, which is shown in Fig. 9.8.

```

clf, dendrogram(Z);
xlabel('Sample No.')
ylabel('Distance')
box on

```

Clustering finds the same groups as the principal component analysis. We observe clear groups consisting of samples 1, 2, 8, 9 and 10 (the magmatic source rocks), samples 3, 4 and 5 (the hydrothermal vein), and samples 6 and 7 (the sandstone). One way to test the validity of our clustering result is to use the *cophenet correlation coefficient*:


```
cophenet(Z,Y)
```

```
ans =
```

```
0.7579
```

The result is convincing since the closer this coefficient is to one, the better the cluster solution.

9.6 Multiple Linear Regression

In Chap. 4 linear regression models were introduced as a way of describing the relationship between a dependent variable y and an independent variable x . The dependent variable is also known as the *response variable*, and the independent variable as the *predictor variable*. A multiple linear regression model describes the relationship between a dependent (or response) variable y , and n independent (or predictor) variables x_i

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

where b_i are the $n+1$ regression coefficients of the linear model. The linearity in the term multiple linear regression refers to the fact that the response variable is a linear function of the predictor variables. The regression coefficients are estimated by minimizing the mean-squared difference between the predicted and true values of the response variable y . An example that is commonly used in the earth sciences is the quality of crude oil, which is assumed to be linearly dependent on the age of the sediment, the burial depth, and temperature. In practice, the plausibility of the assumption of linearity must first be examined. If this assumption is probably true then there are several methods of multiple linear regression available, some of which are included in the Statistics and Machine Learning Toolbox (MathWorks 2020b).

As a first example we create a noise-free synthetic data set with three variables Var1, Var2 and Var3. We wish to find the influence of variables Var1 and Var2 on variable Var3. The variables Var1 and Var2 are therefore the predictor variables and the variable Var3 is the response variable. The linear relationship between the response variable and the predictor variables is $\text{Var3} = 0.2 - 52.0 \cdot \text{Var1} + 276.0 \cdot \text{Var2}$. The three variables Var1, Var2 and Var3 are stored as columns 1, 2 and 3 in a single array data.

```
clear, clc, close all

rng(0)
data(:,1) = 0.3 + 0.03*randn(50,1);
data(:,2) = 0.2 + 0.01*randn(50,1);
data(:,3) = 0.2 ...
    - 52.0*data(:,1) ...
    + 276.0*data(:,2);
```

We create labels for the names of the samples and the names of the variables, as we did in Sect. 9.2.

```
for i = 1 : size(data,1)
    samples(i,:) = [sprintf('%02.0f',i)];
end

variables = ['Var1';
            'Var2';
            'Var3'];
```

We then calculate the coefficients beta of the multiple linear regression model using `fitlm`.

```
beta = fitlm(data(:,1:2),data(:,3),...
    'ResponseVar',variables(3,:),...
    'PredictorVars',variables(1:2,:))
```

The function `fitlm` uses a least mean-squares criterion to calculate beta. The method calculates an F -statistic to test the null hypothesis that all regression coefficients are zero and there is no relationship between the response and predictor variables. The output of the function `fitlm`

```
beta =
```

```
Linear regression model:
    Var3 ~ 1 + Var1 + Var2
```

```
Estimated Coefficients:
```

	Estimate	SE	tStat	pValue
(Intercept)	0.2	0	Inf	0
Var1	-52	0	-Inf	0
Var2	276	0	Inf	0

```
Number of observations: 50, Error degrees of freedom: 47
```

```
R-squared: 1, Adjusted R-Squared: 1
```

```
F-statistic vs. constant model: Inf, p-value = 0
```

first recalls the linear regression model using the names of the response variable and the predictor variables. It then displays a table with the regression coefficient estimates for each predictor variable in the first column, the standard error, the F -statistic, and the p -values of the coefficient estimates. The quality of the model is excellent as the p -value of the constant model, i.e. with no influence of predictor variables on the response variable, is zero, as suggested in the last line of the comments below the table.

Since we have only three variables we can display the results in a three-dimensional plot. We first create a rectangular grid for the first two variables and then calculate the predicted values for the second variable using the estimated regression coefficients. We then use `surf` to display the linear regression plane of the model and `line` to plot the measurements as red points, with a marker size of 30 points.

```
[Var1,Var2] = meshgrid(0.20:0.01:0.45,0.17:0.01:0.23);
Var3 = beta.Coefficients.Estimate(1) + ...
       beta.Coefficients.Estimate(2)*Var1 + ...
       beta.Coefficients.Estimate(3)*Var2;

surf(Var1,Var2,Var3,'FaceAlpha',0.2), hold on
line(data(:,1), data(:,2), data(:,3),...
      'LineStyle','none',...
      'Marker','.',...
      'MarkerSize',30)
grid on
view(70,30)
hold off
```

Since the data set is noise free the data points all lie on the linear regression plane. This changes if we introduce normally-distributed noise with a standard deviation of 2.4.

```
clear

rng(0)
data(:,1) = 0.3 + 0.03*randn(50,1);
data(:,2) = 0.2 + 0.01*randn(50,1);
data(:,3) =    0.2 ...
             - 52.0*data(:,1) ...
             + 276.0*data(:,2) ...
             +  2.4*randn(50,1);

for i = 1 : size(data,1)
    samples(i,:) = [sprintf('%02.0f',i)];
end
```

```
variables = ['Var1';
            'Var2';
            'Var3'];
```

Using `fitlm` again

```
beta = fitlm(data(:,1:2),data(:,3),...
            'ResponseVar',variables(3,:),...
            'PredictorVars',variables(1:2,:))
```

We see from the output

```
beta =

Linear regression model:
  Var3 ~ 1 + Var1 + Var2

Estimated Coefficients:

```

	Estimate	SE	tStat	pValue
(Intercept)	5.0222	6.9235	0.72538	0.47181
Var1	-45.307	9.1424	-4.9557	9.7694e-06
Var2	239.8	33.273	7.2071	3.969e-09

```

Number of observations: 50, Error degrees of freedom: 47
Root Mean Squared Error: 2.41
R-squared: 0.596, Adjusted R-Squared: 0.579
F-statistic vs. constant model: 34.6, p-value = 5.68e-10
```

that the estimates of the regression coefficients (5.0222, -45.307, and 239.8) do not exactly match those that were used to create the synthetic data (0.2, -52.0, and 276.0). Furthermore, the p -value of the constant model is significantly higher than in the previous example, although it is still very good, as suggested in the last line of the comments below the table. The p -values of the individual regression coefficient remain low, however, suggesting that both predictor variables Var1 and Var2 contribute equally to the response variable Var3 at a 5% significance level. We can again display the results in a three-dimensional plot (Fig. 9.9)

```
[Var1,Var2] = meshgrid(0.20:0.01:0.45,0.17:0.01:0.23);
Var3 = 5.0222 - 45.307*Var1 + 239.8*Var2;

surf(Var1,Var2,Var3,'FaceAlpha',0.2), hold on
line(data(:,1), data(:,2), data(:,3),...
     'LineStyle','none',...
     'Marker','.',...
     'MarkerSize',30)
grid on
xlabel('x1'), ylabel('x2'), zlabel('y')
view(70,30)
hold off
```

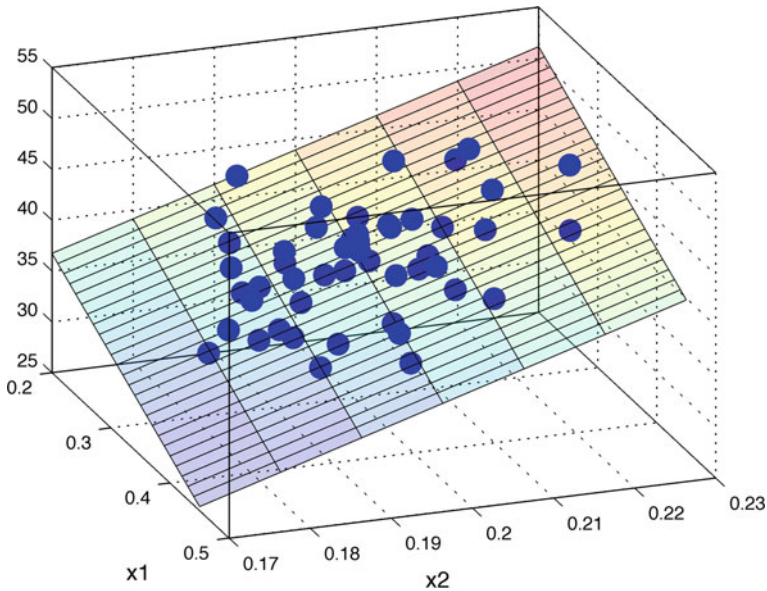


Fig. 9.9 Linear regression model for a synthetic data set with three variables and random noise.

and observe some differences between the data and the regression plane if we rotate the graph in three dimensions using the *Rotate 3D* tool on the toolbar of the Figure Window.

As a second example we use laboratory data from the Sherwood Sandstone, England, that suggest that intergranular permeability (as response variable) is mainly influenced by porosity and matrix conductivity (as predictor variables), while other variables have less influence. This example has been previously discussed in detail in an article by Olorunfemi (1985) and was used as an example in the textbook by Swan and Sandilands (1995). The data for 40 rock samples from the Sherwood Sandstone were taken from M.O. Olorunfemi's publication and are stored in the file *sherwoodsandstone.txt*. We load the data from the file by typing

```
clear

data = load('sherwoodsandstone.txt');
```

The five columns of the array data contain the numerical values of the petrophysical parameters permeability (in m/s), porosity (without physical unit), matrix conductivity (in S/m), true formation factor (without physical unit), and induced polarization (in %). We also load the sample IDs (from Olorunfemi 1985) from the file *sherwoodsandstone_samples.txt* using `textscan`:

```
fid = fopen('sherwoodsandstone_samples.txt');
samples = textscan(fid,'%s','Headerlines',8);
fclose(fid);
```

The *sherwoodsandstone_samples.txt* file contains 40 sample IDs, each with five characters including spaces that are used to make the length of the sample IDs identical so that they fit into the cell array `samples`. This cell array contains only one cell, `samples{1}`, which in turn contains a 40-by-5 character array of the sample IDs. We next create another character array that contains the variable names: `Perm` for permeability, `Por` for porosity, `MaCo` for matrix conductivity, `TrFF` for true formation factor, and `InPo` for induced polarization.

```
variables = ['Perm';
             'Por';
             'MaCo';
             'TrFF';
             'InPo'];
```

We then display the variables in a plot that includes a legend and has the data points labeled with the sample IDs. We use random colors for the data points, with the random number generator first set to a seed value of 0. Since the values of the variables are very small we plot the logarithm of the values instead of the true values.

```
rng(0)
for i = 1:size(data,2)
    rcolor = rand(1,3);
    line(log(data(:,i)),1:40,...
         'Marker','Square','MarkerEdgeColor',rcolor,...
         'MarkerSize',8,'MarkerFaceColor',rcolor,...
         'LineStyle',':', 'Color',rcolor)
    text(log(data(:,i))+0.1,1:40,samples{1})
end
legend(variables)
```

We then calculate the coefficients beta of the multiple linear regression model using `fitlm`.

```
beta = fitlm(data(:,2:5),data(:,1),...
    'ResponseVar',variables(1,:),...
    'PredictorVars',variables(2:5,:))
```

The function `fitlm` uses a least mean-squares criterion to calculate `beta`. The output of the function `fitlm`

```
beta =

Linear regression model:
    Perm ~ 1 + Poro + MaCo + TrFF + InPo

Estimated Coefficients:

```

	Estimate	SE	tStat	pValue
(Intercept)	0.15868	7.5546	0.021005	0.98336
Poro	50.636	18.714	2.7058	0.010458
MaCo	-330.9	150.51	-2.1986	0.034609
TrFF	-0.17857	0.18537	-0.96329	0.34201
InPo	-0.45263	0.50101	-0.90344	0.37247

```

Number of observations: 40, Error degrees of freedom: 35
Root Mean Squared Error: 2.68
R-squared: 0.483, Adjusted R-Squared: 0.424
F-statistic vs. constant model: 8.18, p-value = 9.07e-05
```

first recalls the linear regression model using the names of the response variable and the predictor variables. It then displays a table with the regression coefficient estimates for each predictor variable in the first column, the standard error, the *t*-statistic, and the *p*-values of the coefficient estimates. The quality of the model is good as the *p*-value of the constant model (i.e. with no influence of predictor variables on the response variable) is very low, as the last line of the comments below the table suggests. The *p*-values of the individual regression coefficient estimates suggests that only the porosity and matrix conductivity influence the permeability of the Sherwood Sandstone, at a 5% significance level.

There are multiple ways to examine the quality of the result. The Statistics and Machine Learning Toolbox includes several functions for displaying diagnostic plots to identify and remove outliers from the data set. We can, for instance, produce a leverage plot of the data and the model by typing

```
plotDiagnostics(beta)
```

The leverage plot displays the leverage that an individual measurement has when added to the regression model, assuming that all other measurements are already in the model. The plot also displays a horizontal dashed line that is not explained in

the User's Guide of the Statistics and Machine Learning Toolbox (MathWorks 2020b). Information regarding this dashed line can, however, be found in the R2013 Release Notes and the M-file of the function `LinearModel`

```
edit LinearModel.fit
```

where the regression diagnostics are explained in detail from line 126 onward. According to this information, the average value of leverage is the ratio between the number of regression coefficients and the number of measurements

```
size(data,2)/size(data,1)
```

which in this particular example yields

```
ans =  
0.1250
```

Observations with leverage values larger than twice this ratio

```
2*size(data,2)/size(data,1)  
ans =  
0.2500
```

are considered to have high leverage, and are all measurements above the dashed line at 0.250. In our example there are two points with leverages greater than 0.250, one in row 11 of the array and the other in row 22. We can obtain the IDs of these samples by typing

```
samples{1}(11)  
samples{1}(22)
```

which yields

```
ans =  
1×1 cell array  
{'HF14'}  
  
ans =  
1×1 cell array  
{'HF49'}
```


The leverage plot does not reveal whether or not the high-leverage points are outliers. We can instead calculate and display the Cook's distance of each data point

```
plotDiagnostics(beta, 'cookd')
```

which is the change in the residuals of a model if this specific data point is removed. The larger the Cook's distance of a particular data point the more likely it is to be an outlier. The diagnostic plot again displays a dashed line, which marks a recommended threshold of three times the mean Cook's distance

```
3*mean mdl.Diagnostics.CooksDistance)
```

which yields

```
ans =  
0.1144
```

According to this plot there are three data points above the critical value marked by the dashed line, one each in rows 22, 27 and 35. Since the data point HF49 in the 22nd row also has a high leverage on the model it should be removed as an outlier. We can also examine the residuals of the model using

```
plotResiduals(beta)
```

which suggests that the measurements above 9 are outliers. As the last diagnostic plot in this exercise we display the normal probability plot of the residuals

```
plotResiduals(beta, 'probability')
```

where the black line represents the probability of the residuals in the case of an ideal normal distribution. In our example, the tails of the normal distribution plot of the residuals do not match a normal distribution and our linear regression model is therefore good (but not perfect) without having removed the outliers. The regression coefficients calculated from the regression analysis (including the outliers) are, however, similar to those reported on page 347 of Swan and Sandilands (1995).

The linear regression model can now be used to predict the permeability of a new sample from the values of the other petrophysical parameters. The effect that each predictor variable has on the model can be examined using one of several functions included in the Statistics and Machine Learning Toolbox. As an example

```
plotSlice(beta)
```

yields a slice plot of the responses displaying each predictor variable separately. Within this plot we can drag the lines or insert values for the individual predictor variables and examine the resulting value of the response variable.

9.7 Aitchison's Log-Ratio Transformation

Anyone who has ever dealt with the statistical analysis of compositional data must have stumbled across John Aitchison's (1926–2016) log-ratio transformation. The Scottish statistician spent much of his career on the statistics of such data, wrote the famous book *The Statistical Analysis of Compositional Data* (Aitchison 1984, 1986, 1999), and published multiple papers on the same topic. The paper was associated with the MATLAB 5 software package CODA available from the author, of which updated versions are now available for download at CoDAWeb:

<http://www.compositionaldata.com>

Aitchison's log-ratio transformation overcomes the closed-sum problem of closed data, i.e. data expressed as proportions and adding up to a fixed total such as 100%. The closed-sum problem causes spurious negative correlations between pairs of variables in multivariate data sets, which are avoided by logarithmizing ratios of the variables. As an example, multiproxy data in paleoclimate studies are often closed data, i.e. they are very much influenced by dilution effects. In the simplest example, this could be a mixture of three components that together make up 100% of the sediment. Imagine that one of the components is now constant, i.e. the same amount of this component is always delivered per unit time. Since the total of all three components is always 100%, the other two components are necessarily anticorrelated: if one goes up then the other one must go down by the same amount.

Using ratios helps to overcome the problem that the magnitudes of compositional data are actually ratios, whose denominators are the sums of the concentrations all constituents (see Davis 2002 for a short summary of Aitchison's log-ratio transformation). Product-moment covariances of ratios, however, are awkward to manipulate and result in complicated products and quotients. John Aitchison therefore suggested to take the logarithms of those ratios, taking advantage of the simple properties of the logarithmic function such as $\log(x/y) = \log(x) - \log(y)$, and hence $\log(x/y) = -\log(y/x)$, which allows a significant reduction in the number of covariances necessary to describe the data set (Aitchison 1986, reprinted 2003, pp. 65–66).

Here is a simple example illustrating the closed-sum problem of a system with three variables, and the use of ratios as well as log-ratios to overcome the problem of spurious correlations between pairs of variables. First, we clear the workspace and choose colors for the plots.

```
clear, clc, close all
```

```
colors = [  
0 114 189  
217 83 25  
237 177 32  
126 47 142  
]./255;
```

We are interested in `element1a` and `element1b` contributing to a sediment, which are diluted by `element1c`. We simply create three variables with magnitudes measured in milligrams and with a sinusoidal variation of 200 samples down core, making sure that all absolute values are >0 .

```
t = 0.1 : 0.1 : 20; t = t';  
  
element1a = sin(2*pi*t/2) + 5;  
element1b = sin(2*pi*t/5) + 5;  
element1c = 2*sin(2*pi*t/20) + 5;  
  
figure('Position',[100 1000 600 300])  
a1 = axes('Box','On');  
line(a1,t,element1a,...  
      'Color',colors(1,:))  
line(a1,t,element1b,...  
      'Color',colors(2,:))  
line(a1,t,element1c,...  
      'Color',colors(4,:));  
legend('1a','1b','1c',...  
       'Box','Off',...  
       'Location','northwest'), grid  
title('Absolute amount of 1a-c (mg)')
```

We calculate the percentages of `element1a` to `element1c`, i.e. we create ratios of the concentrations of the individual elements and the sum of the concentrations of all elements. This process creates closed data, i.e. the data are now expressed as proportions and add up to a fixed total of 100%: `elements1a+elements1b`

+element1c = 100%. Element1a and element1b are now affected by the dilution due to element1c. These elements show a significant sinusoidal long-term trend that is not real, as the first figure shows.

```

element1a_perc = element1a./...
    (element1a+element1b+element1c);
element1b_perc = element1b./...
    (element1a+element1b+element1c);
element1c_perc = element1c./...
    (element1a+element1b+element1c);

figure('Position',[100 1000 600 300])
a2 = axes('Box','On');
line(a2,t,element1a_perc,...
    'Color',colors(1,:))
line(a2,t,element1b_perc,...
    'Color',colors(2,:))
line(a2,t,element1c_perc,...
    'Color',colors(4,:))
legend('1a','1b','1c',...
    'Box','Off',...
    'Location','northwest'), grid
title('Relative concentration 1a-c (%)')

```

We have now established ratios of element1a to element1b, and element1b to element1a, respectively, which are both independent of element1c. Note the change of sign and difference in amplitude. The ratios of element1a/element1b and element1b/element1a do not show any evidence of a trend caused by the dilution effect of element1c. However, the two curves are not identical, i.e. element1a/element1b and element1b/element1a are not symmetric (see Weltje and Tjallingii 2008, p. 426).

```

ratio12 = element1a_perc./element1b_perc;
ratio21 = element1b_perc./element1a_perc;

figure('Position',[100 1000 600 300])
a3 = axes('Box','On');
yyaxis left, line(a3,t,ratio12,...
    'Color',colors(1,:))
yyaxis right, line(a3,t,ratio21,...
    'Color',colors(2,:))
set(a3,'YDir','Reverse')
legend('1a/1b',...
    '1b/1a',...
    'Box','Off',...
    'Location','northwest'), grid
title('Ratios elements 1a/1b and 1b/1a')

```

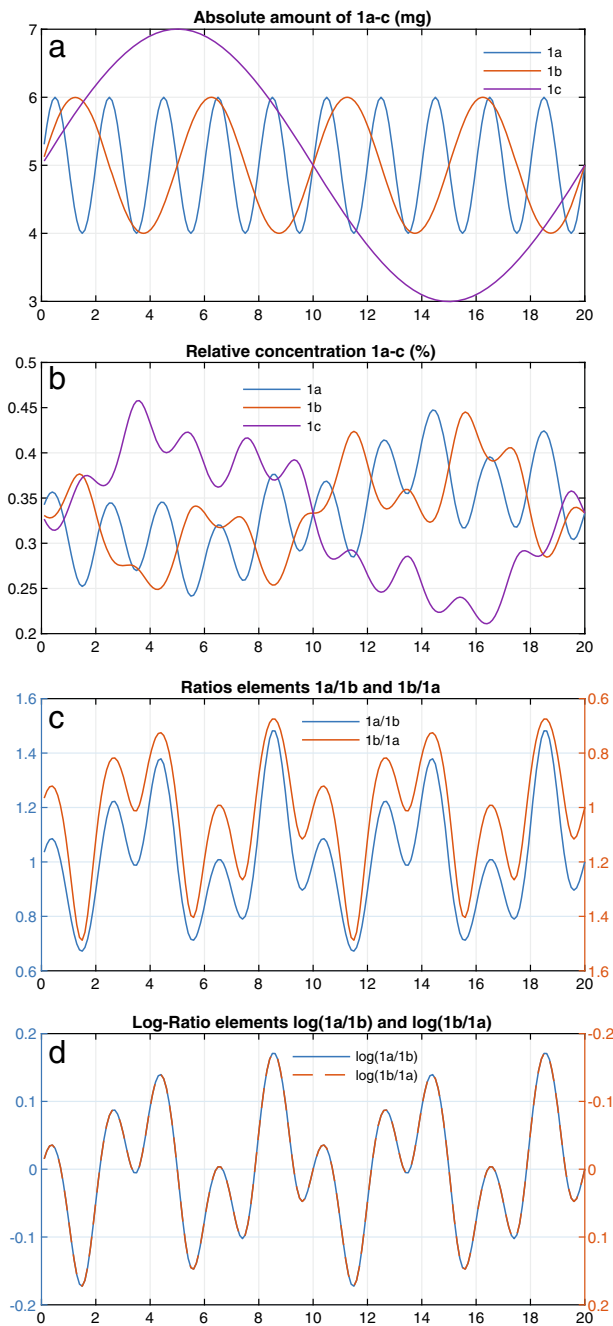


Fig. 9.10 Demonstration of Aitchison’s log-ratio transformation to overcome the closed-sum problem of closed data: **a** absolute values of element1a to element1c, **b** relative amounts of element1a to element1c showing the closed-sum problem, **c** ratios of element1a/element1b and element1b/element1a, which are both independent of element1c, **d** log-ratios according to Aitchison (1986), which are identical except for the sign, overcoming the closed-sum problem.

We now display log-ratios according to Aitchison (1986). The log-ratios $\log(\text{element1a}/\text{element1b})$ and $\log(\text{element1b}/\text{element1a})$ are identical except for the sign. It therefore makes no difference whether we use $\log(\text{element1a}/\text{element1b})$ or $\log(\text{element1b}/\text{element1a})$, for instance, when running further statistical analyses on the data.

```
ratio12log = log10(ratio12);
ratio21log = log10(ratio21);

figure('Position',[100 1000 600 300])
a4 = axes('Box','On');
yyaxis left
line(a4,t,ratio12log,...
     'Color',colors(1,:))
yyaxis right
line(a4,t,ratio21log,...
     'Color',colors(2,:),...
     'LineStyle','-')
set(a4,'YDir','Reverse')
legend('log(1a/1b)',...
      'log(1b/1a)',...
      'Box','Off',...
      'Location','northwest'), grid
title('Log-Ratio elements log(1a/1b) and log(1b/1a)')
```

Aitchison's log-ratio transformation must be used in all multivariate statistical methods that analyze closed data (e.g. Sects. 9.2–9.6, Fig. 9.10).

Recommended Reading

- Aitchison J (1984) The statistical analysis of geochemical composition. *Math Geol* 16(6):531–564
- Aitchison J (1986, reprinted 2003) The statistical analysis of compositional data. The Blackburn Press, Caldwell
- Aitchison J (1999) Logratios and natural laws in compositional data analysis. *Math Geol* 31 (5):563–580
- Birks HJB, Gordon AD (1985) numerical methods in quaternary pollen analysis. Academic Press, London
- Brown CE (1998) Applied Multivariate Statistics in Geohydrology and Related Sciences. Springer, Berlin Heidelberg New York
- Davis JC (2002) Statistics and Data Analysis in Geology, 3rd edn. Wiley, New York
- Fisher RA (1936) The use of multiple measurements in taxonomic problems. *Ann Eugen* 7:179–188
- Härdle WK, Simar L (2012) Applied multivariate statistical analysis. Springer, Berlin Heidelberg, New York
- Hermanns R, Trauth MH, McWilliams M, Strecker M (2000) Tephrochronologic constraints on temporal distribution of large landslides in NW-Argentina. *J Geol* 108:35–52

- Hotelling H (1931) Analysis of a complex of statistical variables with principal components. *J Educ Psychol* 24(6):417–441
- Hyvärinen A (1999) Fast and Robust fixed-point algorithms for independent component analysis. *IEEE Trans Neural Netw* 10(3):626–634
- MathWorks (2020a) Fuzzy logic toolbox—users guide. The MathWorks, Inc., Natick, MA
- MathWorks (2020b) Statistics and machine learning toolbox—user’s guide. The MathWorks, Inc., Natick, MA
- Olorunfemi MO (1985) Statistical relationships among some formation parameters for Sherwood Sandstone, England. *Math Geol* 17:845–852
- Pawlowsky-Glahn V (2004) Geostatistical analysis of compositional data—studies in mathematical geology. Oxford University Press, Oxford
- Pearson K (1901) On lines and planes of closest fit to a system of points in space. *Philos Mag J Sci* 6(2):559–572
- Reyment RA, Savazzi E (1999) Aspects of multivariate statistical analysis in geology. Elsevier Science, Amsterdam
- Streckeisen A (1976) To each plutonic rock its proper name. *Earth-Sci Rev* 12:1–33
- Swan ARH, Sandilands M (1995) Introduction to geological data analysis. Blackwell Sciences, Oxford
- Trauth MH, Bookhagen B, Mueller A, Strecker MR (2003) Erosion and climate change in the Santa Maria Basin, NW Argentina during the last 40,000 yrs. *J Sediment Res* 73(1):82–90
- Weltje GJ, Tjallingii R (2008) Calibration of XRF core scanners for quantitative geochemical logging of sediment cores: theory and application. *Earth Planet Sci Lett* 274:423–438
- Westgate JA, Shane PAR, Pearce NJG, Perkins WT, Korisettar R, Chesner CA, Williams MAJ, Acharyya SK (1998) All toba tephra occurrences across peninsular India belong to the 75,000 yr BP eruption. *Quat Res* 50:107–112
- Zadeh L (1965) Fuzzy sets. *Information. Control* 8:338–353



10.1 Introduction

Methods for analyzing circular and spherical data are widely used in earth sciences. Structural geologists, for example, measure and analyze the orientation of slickensides (or striae) on fault planes. Circular statistics is also common in paleomagnetic applications. Microstructural investigations include the analysis of grain shapes and quartz *c*-axis orientations in thin sections. Paleoenvironmentalists also reconstruct paleocurrent directions from fossil alignments (Fig. 10.1). There are theoretically two types of directional data in earth sciences: directional data *sensu stricto*, and oriented data. Directional data, such as the paleocurrent direction of a river as documented by flute marks, or the flow direction of a glacier as indicated by glacial striae, have a true polarity. Oriented data describe axial data and lines, such as the orientation of joints, without any sense of direction.

There are a number of useful publications available on the statistical analysis of directional data, such as the books by Fisher (1995) and Mardia and Jupp (1999), and the chapters on the subject in books by Swan and Sandilands (2000), Davis (2002), and Borradaile (2003). Furthermore, Chap. 9 of the book by Middleton (1999), two journal articles by Jones (2006a, b), and this chapter of the book by Marques de Sá (2007) discuss the use of MATLAB® for the statistical analysis of directional data. MATLAB is not the first choice for analyzing directional data since it does not provide the relevant functions, such as algorithms that can be used to compute the probability distribution function of a von Mises distribution, or to run a Rayleigh's test for the significance of a mean direction. Earth scientists have therefore developed numerous stand-alone programs with which to analyze such data, e.g. the excellent desktop and smartphone software developed by Rick

Electronic supplementary material The online version of this chapter (https://doi.org/10.1007/978-3-030-38441-8_10) contains supplementary material, which is available to authorized users.



Fig. 10.1 *Orthoceras* fossils from an outcrop at Neptuni Åkrar near Byxelkrok on Öland, Sweden. *Orthoceras* is a cephalopod with a straight shell that lived in the Ordovician era, about 450 million years ago. Such elongated, asymmetric objects tend to orient themselves in the hydrodynamically most stable position and therefore indicate paleocurrent directions. The statistical analysis of cephalopod orientations at Neptuni Åkrar reveals a significant southerly paleocurrent direction, which is in agreement with the paleogeographic reconstructions.

Allmendinger, structural geologist at Cornell University (Allmendinger et al. 2017; Allmendinger 2019)

<http://www.geo.cornell.edu/geology/faculty/RWA/programs.html>

and Néstor Cardozo, who did his PhD with Allmendinger and is now at the University of Stavanger

<http://www.ux.uis.no/~nestor/work/programs.html>

On Cardozo's website you can also find numerous MATLAB functions from the book *Structural Geology Algorithms* by Allmendinger and Cardozo together with Donald M. Fisher (Allmendinger et al. 2012). Another interesting collection of MATLAB functions for directional data can be found in Philipp Berens' Circular Statistics Toolbox, available for download from the MATLAB File Exchange (Berens 2009).

The following tutorial on the analysis of directional data is independent of these tools. It provides simple MATLAB codes for displaying directional data, computing the von Mises distribution, and running simple statistical tests. The first

section (Sect. 10.2) introduces rose diagrams as the most widely used method to display directional data. With a similar concept to Chap. 3 on univariate statistics, the following two sections (10.3 and 10.4) are then on the use of empirical and theoretical distributions to describe directional data. The next three sections (10.5–10.7) then describe the three most important tests for directional data, these being tests for the randomness of circular data, for the significance of a mean direction, and for the difference between two sets of circular data. The last two sections (10.8 and 10.9) describe the graphical representation and statistics of spherical data.

10.2 Graphical Representation of Circular Data

The classic way to display directional data is in a rose diagram. A rose diagram is a histogram for measurements of angles. In contrast to a bar histogram with the height of the bars proportional to frequency, a rose diagram comprises segments of a circle, with the radius of each sector being proportional to the frequency. We use synthetic data to illustrate two types of rose diagram that are used to display directional data. We first load a set of directional data from the file *directional_1.txt*.

```
clear, clc, close all

data_degrees_1 = load('directional_1.txt');
```

The data set contains two hundred measurements of angles, in degrees. The function `rose(az,nb)` has been used to display the data in a rose diagram since the 2nd edition of this book, when this chapter first appeared. This function plots an angle histogram for the angles `az` in radians, where `nb` is the number of classes. However, since the original data are in degrees we need to convert all measurements to radians before plotting the data.

```
data_radians_1 = pi*data_degrees_1/180;
rose(data_radians_1,12)
```

The function `rose` counts in a counterclockwise direction, with zero degrees lying along the x -axis of the coordinate graph. In the geosciences, the angles increase in a clockwise direction with 0° indicating due north, 90° indicating due east, and so on. The command `view` rotates the plot by $+90^\circ$ (the azimuth) and mirrors the plot by -90° (the elevation) (Fig. 10.2).

```
rose(data_radians_1,12)
view(90,-90)
```

The area of the arc segments increases with the square of the frequency. In a modification the rose diagram is therefore scaled to the square root of the class frequency. The function `rose` does not allow plotting of the square root of the frequencies by default, but the corresponding file `rose.m` can be easily modified as

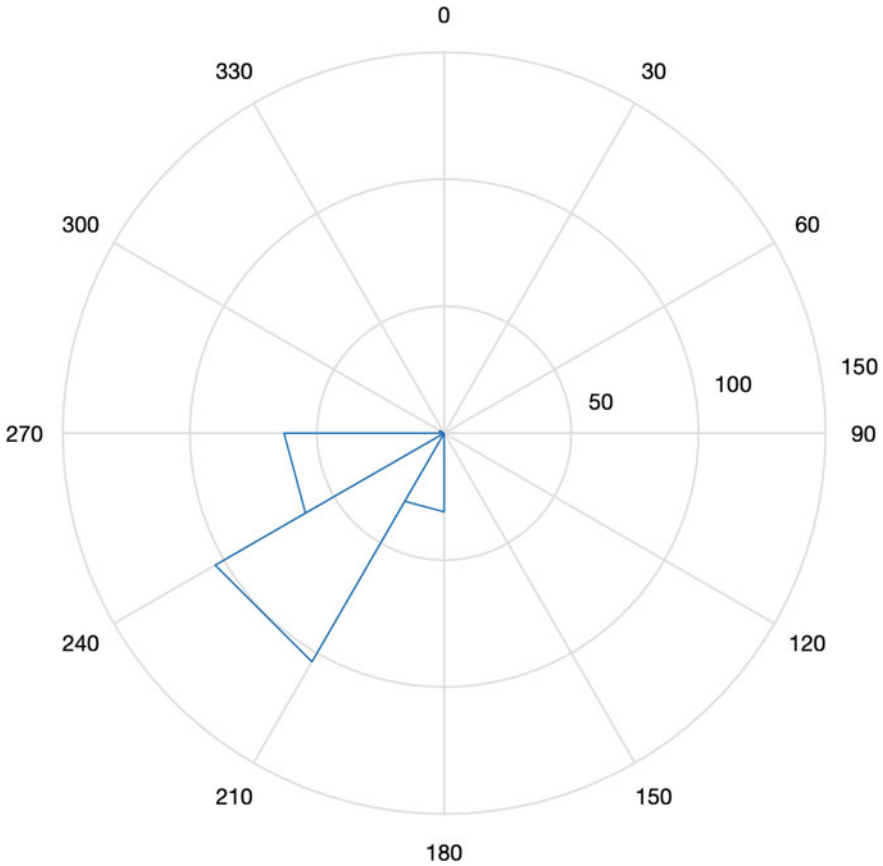


Fig. 10.2 Rose diagram to display directional data using the function `rose`. The radii of the area segments are proportional to the frequencies for each class. We use `rose` with `view(90, -90)`, so that 0° points due north and 90° points due east, i.e. the angles increase in a clockwise direction.

follows. After the histogram of the angles is computed in line 57 using the function `histc`, add a line with the command `nn = sqrt(nn)`, which computes the square root of the frequencies `nn`. Save the modified function as file `rose_sqrt.m` and apply the new function to the data set.

```
rose_sqrt(data_radians_1,12)
view(90,-90)
```

This plot satisfies all conventions in the geosciences. Since the MATLAB R2016b release, use of the function `rose` is no longer recommended and has been replaced by the `polarhistogram` function. As with `rose`, this function creates a histogram chart in polar coordinates using radians.

```
polarhistogram(data_radians_1,12)
```

However, in contrast to `rose`, this function does not select classes distributed over the entire circle, but only over the range of the data, which results in a completely different graphic. We can use the `BinEdges` property to force `polarhistogram` to use the same bins as `rose`:

```
polarhistogram(data_radians_1,...
    'BinEdges',0:2*pi/12:2*pi-2*pi/12)
```

In order to change the graphic so that 0° indicates due north and 90° indicates due east, we use `polarhistogram` together with `polaraxes` to change the location of the zero reference axis to the top of the diagram and the direction of increasing angles to clockwise (Fig. 10.3).

```
polaraxes('ThetaZeroLocation','top',...
    'ThetaDir','clockwise',...
    'ThetaLim',[0 360]), hold on
polarhistogram(data_radians_1,...
    'BinEdges',0:2*pi/12:2*pi-2*pi/12)
```

We can also change the histogram display style to `stairs` instead of `bar`, which then displays just the outline of the histogram without any fill.

```
polaraxes('ThetaZeroLocation','top',...
    'ThetaDir','clockwise'), hold on
polarhistogram(data_radians_1,...
    'BinEdges',0:2*pi/12:2*pi-2*pi/12,...
    'DisplayStyle','Stairs')
```

10.3 Empirical Distributions of Circular Data

This section introduces statistical measures used to describe empirical distributions of directional data. The characteristics of directional data are described by measures of central tendency and dispersion, similar to the statistical characterization of univariate data sets (Chap. 3). Assume that we have collected a number of angular measurements such as fossil alignments. The collection of data can be written as

$$\theta_1, \theta_2, \theta_3, \dots, \theta_N$$

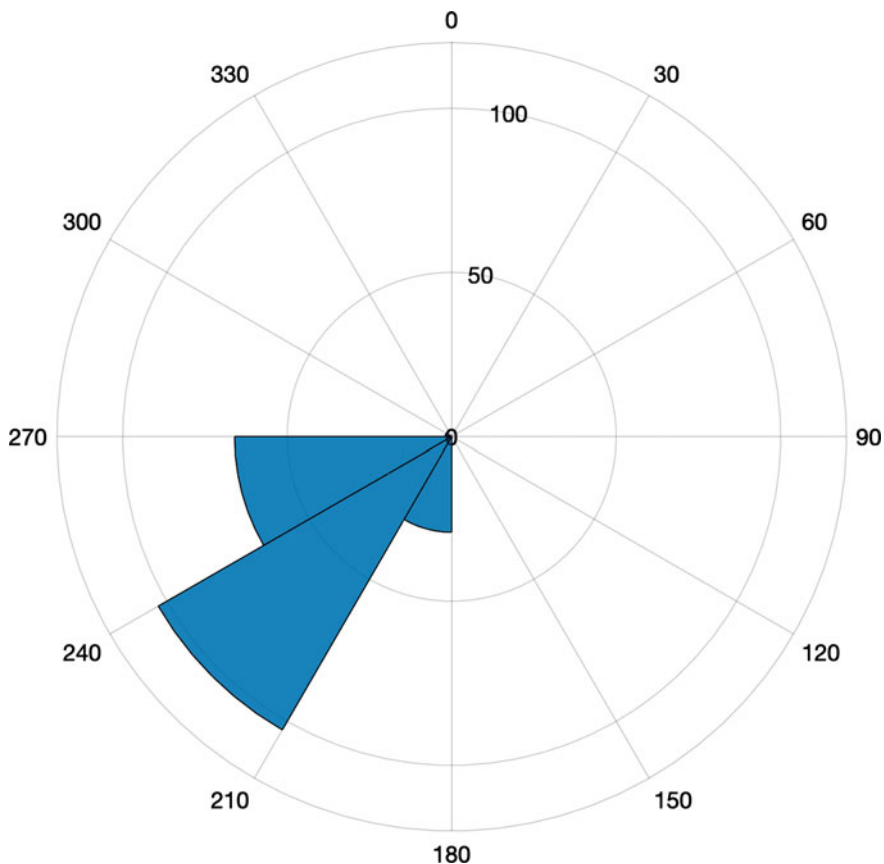


Fig. 10.3 Polar histogram to display directional data using the function `polarhistogram`. The radii of the area segments are proportional to the frequencies for each class. We use `polarhistogram` with `polaraxes` to change the location of the zero reference axis to the top of the diagram and the direction of increasing angles to clockwise, so that 0° points due north and 90° points due east.

containing N observations θ_i . Sine and cosine values are computed for each direction θ_i and then used to compute the *resultant* or *mean direction* for the set of angular data (Fisher 1995; Mardia and Jupp 1999).

$$x_r = \sum \sin \theta_i$$

$$y_r = \sum \cos \theta_i$$

The resultant direction of the data set is

$$\bar{\theta} = \tan(x_r/y_r)$$

The resultant length is

$$R = \sqrt{(x_r^2 + y_r^2)}$$

The resultant length clearly depends on the dispersion of the data. Normalizing the resultant length to the number of observations yields the mean resultant length.

$$\bar{R} = R/N$$

The value of the mean resultant length decreases with increasing dispersion (Fig. 10.4). The difference between one and the mean resultant length, which is the *circular variance*, is therefore often used as a measure of dispersion for directional data

$$\sigma_0 = 1 - \bar{R}$$

The following example illustrates the use of these parameters by means of synthetic directional data. We first load the data from the file *directional_1.txt* and convert all measurement to radians.

```
clear, clc, close all

data_degrees_1 = load('directional_1.txt');
data_radians_1 = pi*data_degrees_1/180;
```

We now calculate the resultant vector R . Firstly, we compute the x and y components of the resultant vector:

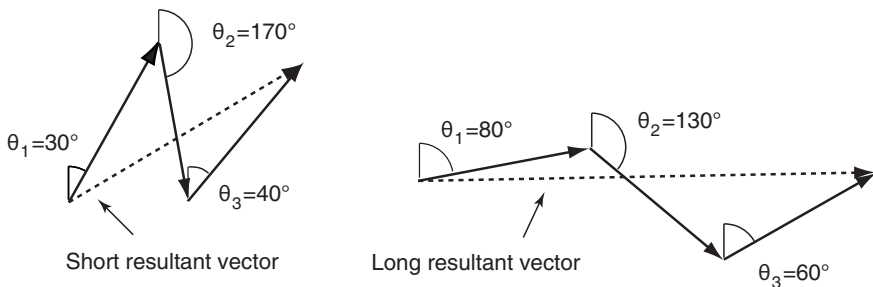


Fig. 10.4 The resultant length R of a sample decreases with increasing dispersion of the data θ_i . Modified from Swan and Sandilands (1995).

```

x_1 = sum(sin(data_radians_1))
y_1 = sum(cos(data_radians_1))

x_1 =

-146.5480

y_1 =

-120.3454

```

The mean direction is the inverse tangent of the ratio of x and y .

```

mean_radians_1 = atan(x_1/y_1)
mean_degrees_1 = 180*mean_radians_1/pi

mean_radians_1 =

0.8833

mean_degrees_1 =

50.6071

```

This result suggests that the resultant vector R is around 0.88 radians or $\sim 51^\circ$. However, since both x and y are negative, the true value of `mean_degrees` is located in the third quadrant and we therefore add 180°

```

mean_degrees_1 = mean_degrees_1 + 180

mean_degrees_1 =

230.6071

```

which results in a mean direction of around 231° . The length of this vector is the absolute value of the vector, which is

```

R_1 = sqrt(x_1^2 + y_1^2)

R_1 =

189.6294

```

The resultant length depends on the dispersion of the directional data. Normalizing the resultant length to the sample size yields the mean resultant length R_m of

```
Rm_1 = R_1 / (length(data_radians_1))
```

```
Rm_1 =
```

```
0.9481
```

A high R_m value suggests less variance. We then compute the circular variance σ , which is

```
sigma_1 = 1 - Rm_1
```

```
sigma_1 =
```

```
0.0519
```

10.4 Theoretical Distributions of Circular Data

As in Chap. 3, the next step in a statistical analysis is to find a suitable theoretical distribution, which we then fit to the empirical distribution visualized and described in the previous section. The classic theoretical distribution to describe directional data is the *von Mises distribution*, named after the Austrian mathematician Richard

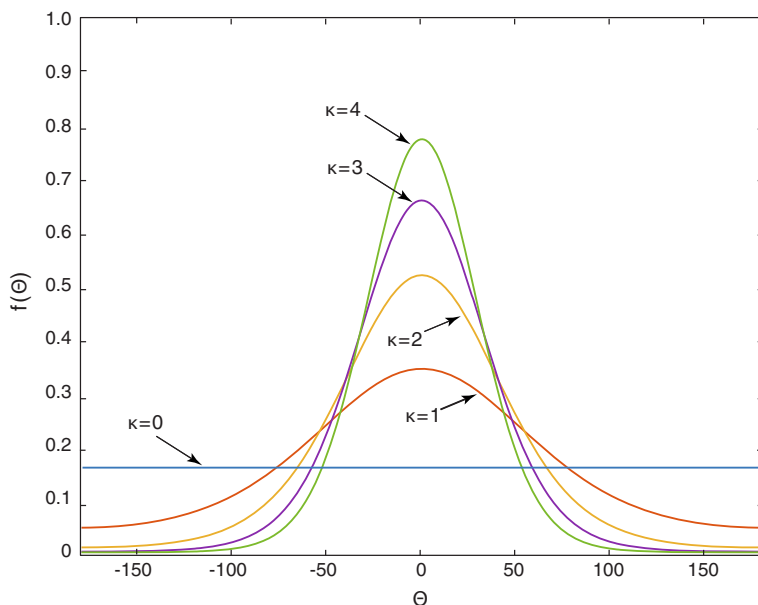


Fig. 10.5 Probability density function $f(x)$ of a von Mises distribution with $\mu = 0$ and five different values for κ .

Edler von Mises (1883–1953). The probability density function of a von Mises distribution is

$$f(\theta) = \frac{1}{2\pi I_0(\kappa)} e^{\kappa \cos(\theta - \mu)}$$

where μ is the mean direction and κ is the concentration parameter (Mardia and Jupp 1999) (Fig. 10.5). $I_0(\kappa)$ is the zero-order modified Bessel function of the first kind. The Bessel functions are solutions of a second-order differential equation (Bessel's differential equation) and are important in many problems of wave propagation in a cylindrical waveguide, and of heat conduction in a cylindrical object. The von Mises distribution is also known as the circular normal distribution since it has similar characteristics to a normal distribution (Sect. 3.4). This distribution is used when the mean direction is the most frequent direction. The probability of deviations is equal on either side of the mean direction and decreases with increasing distance from the mean direction.

As an example let us assume a mean direction of $\mu=0$ and five different values for the concentration parameter κ .

```
clear, clc, close all
```

```
mu = 0; kappa = [0 1 2 3 4]';
```

We first define an angle scale for a plot that runs from -180 to 180 degrees, with intervals of one degree.

```
theta = -180:1:180;
```

All angles are converted from degrees to radians.

```
mu_radians = pi*mu/180;
theta_radians = pi*theta/180;
```

We then compute the von Mises distribution for these values. The formula uses the zero-order modified Bessel function of the first kind, which can be calculated using the function `besseli`. We compute the probability density function for the five values of κ .

```
for i = 1:5
    mises(i,:) = (1/(2*pi*besseli(0,kappa(i))))* ...
        exp(kappa(i)*cos(theta_radians-mu_radians));
    theta(i,:) = theta(1,:);
end
```

The results are plotted by

```
for i = 1:5
    plot(theta(i,:),mises(i,:))
    axis([-180 180 0 max(mises(i,:))])
    hold on
end
```

The mean direction and concentration parameters of such theoretical distributions can be easily modified for comparison with empirical distributions.

We can create the MATLAB code to create a pseudorandom number generator for von Mises distributed data from the algorithm described on Page 155 of Best and Fisher (1979). We first define the sample size n , the mean direction μ converted into radians, and the concentration parameter κ .

```
clear

n = 200;
mu = 230;
mu = pi*mu/180;
kappa = 5;
```

We then reset the random number generator `rand`, and calculate n pseudorandom numbers (in degrees) using the algorithm from Best and Fisher (1979).

```
rng(0)
tau = 1 + (1 + 4*kappa^2)^0.5;
rho = (tau - (2*tau)^0.5)/(2*kappa);
r = (1 + rho^2)/(2*rho);
i = 1;
while i <= n
    u1 = rand(1);
    u2 = rand(1);
    u3 = rand(1);
    z = cos(pi*u1);
    f = (1+r*z)/(r+z);
    c = kappa*(r-f);
    if c*(2-c)-u2 > 0
        theta(i)=(sign(u3-0.5)*acos(f));
        i=i+1;
    elseif log(c/u2)+1-c >= 0
        % Nothing happens, return to
        % beginning of while loop
    end
end
end
```

```

data_radians_1 = theta + mu;
data_radians_1 = data_radians_1';
data_degrees_1 = 180*data_radians_1/pi;

```

Please note that the `elseif` condition does not contain any conditional expressions. In the algorithm of Best and Fisher (1979) is simply says *return to step 1*, which is the beginning of the while loop, without evaluating any line of code. Instead of using this code we can also use the function `vmises` that is included in electronic supplement of this book.

```

rng(0)
data_radians_1 = vmises(n,mu,kappa);
data_degrees_1 = 180*data_radians_1/pi;

```

We can display the data using the `polarhistogram` function, as in the previous section.

```

figure('Position',[400 500 300 300])
polaraxes('ThetaZeroLocation','top',...
    'ThetaDir','clockwise'), hold on
polarhistogram(data_radians_1)

```

10.5 Test for Randomness of Circular Data

The first test for directional data compares the data set with a uniform distribution. Directional data following a uniform distribution are purely random, i.e. there is no preference for any one direction. We use the χ^2 -test (Sect. 3.8) to compare the empirical frequency distribution with the theoretical uniform distribution. We first load our sample data.

```

clear, clc, close all

data_degrees_1 = load('directional_1.txt');

```

We then use the function `histogram` to count the number of observations within 12 classes, each with a width of 30 degrees.

```

h = histogram(data_degrees_1,0:30:360);
counts = h.Values;

```

The expected number of observations is $200/12$, where 200 is the total number of observations and 12 is the number of classes.

```
expect = 40/12 * ones(1,12);
```

The χ^2 -test explores the squared differences between the observed and expected frequencies. The quantity χ^2 is defined as the sum of these squared differences divided by the expected frequencies.

```
chi2 = sum((counts - expect).^2 ./expect)
```

```
chi2 =
```

```
748.6000
```

The critical χ^2 value can be calculated using `chi2inv`. The χ^2 -test requires the degrees of freedom Φ . In our example we test the hypothesis that the data are uniformly distributed, i.e. we estimate one parameter, which is the number of possible values N . Since the number of classes is 12, the number of degrees of freedom is $\Phi = 12 - 1 - 1 = 10$. We test our hypothesis on a $p = 95\%$ significance level. The function `chi2inv` computes the inverse of the cumulative distribution function (CDF) of the χ^2 distribution with numbers of parameters specified by Φ for the corresponding probabilities in p .

```
chi2inv(0.95,12-1-1)
```

```
ans =
```

```
18.3070
```

Since the critical χ^2 of 18.3070 is well below the measured χ^2 of 748.6000 we reject the null hypothesis and conclude that our data do not follow a uniform distribution, i.e. they are not randomly distributed.

10.6 Test for the Significance of a Mean Direction

Having measured a set of directional data in the field, we may wish to know whether there is a prevailing direction documented in the data. We can find out using the Rayleigh's test for the significance of a mean direction (Mardia 1972; Mardia and Jupp 1999). This test uses the mean resultant length introduced in Sect. 10.3, which increases as the preferred direction becomes more significant.

Table 10.1 Critical values of mean resultant length for Rayleigh's test for the significance of a mean direction of N samples (Mardia 1972).

N	Level of Significance, α				
	0.100	0.050	0.025	0.010	0.001
5	0.677	0.754	0.816	0.879	0.991
6	0.618	0.690	0.753	0.825	0.940
7	0.572	0.642	0.702	0.771	0.891
8	0.535	0.602	0.660	0.725	0.847
9	0.504	0.569	0.624	0.687	0.808
10	0.478	0.540	0.594	0.655	0.775
11	0.456	0.516	0.567	0.627	0.743
12	0.437	0.494	0.544	0.602	0.716
13	0.420	0.475	0.524	0.580	0.692
14	0.405	0.458	0.505	0.560	0.669
15	0.391	0.443	0.489	0.542	0.649
16	0.379	0.429	0.474	0.525	0.630
17	0.367	0.417	0.460	0.510	0.613
18	0.357	0.405	0.447	0.496	0.597
19	0.348	0.394	0.436	0.484	0.583
20	0.339	0.385	0.425	0.472	0.569
21	0.331	0.375	0.415	0.461	0.556
22	0.323	0.367	0.405	0.451	0.544
23	0.316	0.359	0.397	0.441	0.533
24	0.309	0.351	0.389	0.432	0.522
25	0.303	0.344	0.381	0.423	0.512
30	0.277	0.315	0.348	0.387	0.470
35	0.256	0.292	0.323	0.359	0.436
40	0.240	0.273	0.302	0.336	0.409
45	0.226	0.257	0.285	0.318	0.386
50	0.214	0.244	0.270	0.301	0.367
100	0.150	0.170	0.190	0.210	0.260

$$\bar{R} = \frac{1}{n} \sqrt{\left(\sum \sin \theta_i\right)^2 + \left(\sum \cos \theta_i\right)^2}$$

The data show a preferred direction if the calculated mean resultant length is below the critical value (Mardia 1972) (Table 10.1). As an example we again load the data contained in the file *directional_1.txt*.

```
clear, clc, close all
```

```
data_degrees_1 = load('directional_1.txt');
data_radians_1 = pi*data_degrees_1/180;
```

We then calculate the mean resultant vector `Rm_1_calc` by typing

```
x_1 = sum(sin(data_radians_1));
y_1 = sum(cos(data_radians_1));

mean_radians_1 = atan(x_1/y_1);
mean_degrees_1 = 180*mean_radians_1/pi;
mean_degrees_1 = mean_degrees_1 + 180;

Rm_1_calc = 1/length(data_degrees_1) ...
  .*(x_1.^2+y_1.^2).^0.5
```

which yields

```
Rm_1_calc =

    0.9481
```

The mean resultant length `Rm_1_calc` in our example is 0.9481. There are a number of approximations involved in the calculation of the critical `Rm_1_crit`, for example Wilkie's approximation (Wilkie 1983), according to which `Rm_1_crit` can be calculated by typing

```
n = length(data_degrees_1);

alpha = 0.050;

Rm_1_crit = sqrt((-log(alpha)-(2*log(alpha)+ ...
  (log(alpha))^2)/(4*n))/n)
```

which yields

```
Rm_1_crit =

    0.1223
```

The critical R_m for $\alpha = 0.05$ and $n = 200$ is 0.1223. Since this value is lower than the `Rm_1_calc` from the data we reject the null hypothesis and conclude that there is a preferred single direction, which is

```
theta_1 = 180 * atan(x_1/y_1) / pi
```

```
theta_1 =
```

```
50.6071
```

The negative signs of the sine and cosine, however, suggest that the true result is in the third sector (180–270°), and the correct result is therefore $180 + 50.6071 = 230.6071$:

```
theta_1 = 180 + theta_1
```

```
theta_1 =
```

```
230.6071
```

10.7 Test for the Difference between Two Sets of Directions

Let us consider two sets of measurements in two files, *directional_1.txt* and *directional_2.txt*. We wish to compare the two sets of directions and test the hypothesis that they are significantly different. We use the Watson-William test to test the similarity between two mean directions

$$F = \left(1 + \frac{3}{8\kappa}\right) \frac{(n-2)(R_A + R_B - R_T)}{n - R_A - R_B}$$

where κ is the concentration parameter, R_A and R_B are the resultant lengths of samples A and B , respectively, and R_T is the resultant length of the combined samples (Watson and Williams 1956; Mardia and Jupp 1999). The concentration parameter can be obtained from tables using R_T (Batschelet 1965; Gumbel et al. 1953, Table 10.2). The calculated F is compared with critical values from the standard F tables (Sect. 3.8). The two mean directions are not significantly different if the calculated F -value is less than the critical F -value, which depends on the degrees of freedom $\Phi_a = 1$ and $\Phi_b = n - 2$, and also on the significance level α . Both samples must follow a von Mises distribution (Sect. 10.4).

We use two synthetic data sets of directional data to illustrate the application of this test. We first load the data and convert the degrees to radians.

```
clear, clc, close all

data_degrees_1 = load('directional_1.txt');
data_degrees_2 = load('directional_2.txt');

data_radians_1 = pi*data_degrees_1/180;
data_radians_2 = pi*data_degrees_2/180;
```

We then compute the lengths of the resultant vectors.

```
x_1 = sum(sin(data_radians_1));
y_1 = sum(cos(data_radians_1));
x_2 = sum(sin(data_radians_2));
y_2 = sum(cos(data_radians_2));
```

Table 10.2 Maximum likelihood estimates of concentration parameter κ for calculated mean resultant length (adapted from Batschelet 1965; Gumbel et al. 1953).

R	κ	R	κ	R	κ	R	κ
0.000	0.000	0.260	0.539	0.520	1.224	0.780	2.646
0.010	0.020	0.270	0.561	0.530	1.257	0.790	2.754
0.020	0.040	0.280	0.584	0.540	1.291	0.800	2.871
0.030	0.060	0.290	0.606	0.550	1.326	0.810	3.000
0.040	0.080	0.300	0.629	0.560	1.362	0.820	3.143
0.050	0.100	0.310	0.652	0.570	1.398	0.830	3.301
0.060	0.120	0.320	0.676	0.580	1.436	0.840	3.479
0.070	0.140	0.330	0.700	0.590	1.475	0.850	3.680
0.080	0.161	0.340	0.724	0.600	1.516	0.860	3.911
0.090	0.181	0.350	0.748	0.610	1.557	0.870	4.177
0.100	0.201	0.360	0.772	0.620	1.600	0.880	4.489
0.110	0.221	0.370	0.797	0.630	1.645	0.890	4.859
0.120	0.242	0.380	0.823	0.640	1.691	0.900	5.305
0.130	0.262	0.390	0.848	0.650	1.740	0.910	5.852
0.140	0.283	0.400	0.874	0.660	1.790	0.920	6.539
0.150	0.303	0.410	0.900	0.670	1.842	0.930	7.426
0.160	0.324	0.420	0.927	0.680	1.896	0.940	8.610
0.170	0.345	0.430	0.954	0.690	1.954	0.950	10.272
0.180	0.366	0.440	0.982	0.700	2.014	0.960	12.766
0.190	0.387	0.450	1.010	0.710	2.077	0.970	16.927
0.200	0.408	0.460	1.039	0.720	2.144	0.980	25.252
0.210	0.430	0.470	1.068	0.730	2.214	0.990	50.242
0.220	0.451	0.480	1.098	0.740	2.289	0.995	100.000
0.230	0.473	0.490	1.128	0.750	2.369	0.999	500.000
0.240	0.495	0.500	1.159	0.760	2.455	1.000	5000.000
0.250	0.516	0.510	1.191	0.770	2.547		


```

mean_radians_1 = atan(x_1/y_1);
mean_degrees_1 = 180*mean_radians_1/pi;
mean_radians_2 = atan(x_2/y_2);
mean_degrees_2 = 180*mean_radians_2/pi;

mean_degrees_1 = mean_degrees_1 + 180
mean_degrees_2 = mean_degrees_2 + 180

R_1 = sqrt(x_1^2 + y_1^2);
R_2 = sqrt(x_2^2 + y_2^2);

mean_degrees_1 =

    230.6071

mean_degrees_2 =

    212.1886

```

The orientations of the resultant vectors are approximately 223° and 201° . We also need the resultant length for both samples combined, so we combine both data sets and again compute the resultant length.

```

data_radians_T = [data_radians_1;data_radians_2];

x_T = sum(sin(data_radians_T));
y_T = sum(cos(data_radians_T));

mean_radians_T = atan(x_T/y_T);
mean_degrees_T = 180*mean_radians_T/pi;

mean_degrees_T = mean_degrees_T + 180;

R_T = sqrt(x_T^2 + y_T^2)
Rm_T = R_T / (length(data_radians_T))

R_T =

    372.4449

Rm_T =

    0.9311

```

We apply the F -statistic to the data for $\kappa = 7.426$ for $R_{m_T} = 0.9311$ (Table 10.2). The computed value for F is

```
n = length(data_radians_T);

F = (1+3/(8*7.426)) * (((n-2)*(R_1+R_2-R_T))/(n-R_1-R_2))

F =

    89.6054
```

Using the F -statistic, we find that for 1 and $400 - 2 = 398$ degrees of freedom and $\alpha = 0.05$, the critical value is

```
finv(0.95,1,n-2)

ans =

    3.8649
```

which is well below the observed value of $F = 89.6054$ (Sect. 3.8). We therefore reject the null hypothesis and conclude that the two samples could not have been drawn from populations with the same mean direction.

10.8 Graphical Representation of Spherical Data

There are many ways to display spherical data. However, since we always present the data in two dimensions, either on a sheet of paper or on a computer screen, we need to convert it into one of many available projections. MATLAB does not offer many functions for displaying and processing spherical data, especially not in a form commonly used in the geosciences. However, some functions from the Mapping Toolbox (MathWorks 2020) that were developed for geographic data can be used. These functions for spherical data in general, but with some modifications and limitations. Alternatively, the book by Middleton (1999) includes a number of MATLAB functions for displaying and processing circular and spherical data, e.g. for displaying spherical data on a Wulff net. It also includes a very short section on the statistics of directional data.

As a first example, we display the data using the map projection of the `globe` option in the `axesm` function creating the axes for our graphic. As quite rightly stated in the `globe` documentation, this is not a true map projection but it is created by calculating a three-dimensional frame and displaying the data points on the surface of this frame. Nevertheless, `globe` is true in scale, equal area, and

equidistant. The resulting graphic, however, looks very similar to an orthographic azimuthal projection, which is also included as an option in the MATLAB function `axesm`.

We use synthetic data for longitude `lon` and latitude `lat`, calculated with the function `vmises`. We first reset the random number generator using `rng(0)` and define the sample size `n`. We then define the mean of the `lon` and `lat` values in degrees, convert both values into radians, define the concentration parameter `kappa`, and then use `vmises` (included in the electronic supplement of this book) to calculate `n = 100` values for `lon` and `lat`. Finally, we convert the `lon` and `lat` values into degrees.

```
clear, clc, close all

rng(0)
n = 100;
mu_lon_degrees = 10;
mu_lat_degrees = 35;
mu_lon_radians = pi*mu_lon_degrees/180;
mu_lat_radians = pi*mu_lat_degrees/180;
kappa_lon = 10;
kappa_lat = 10;
lon_radians = vmises(n,mu_lon_radians,kappa_lon);
lat_radians = vmises(n,mu_lat_radians,kappa_lat);
lon = 180*lon_radians/pi;
lat = 180*lat_radians/pi;
```

In order that our data points can later be colored according to the data density, we first calculate the data density with `ksdensity`. We then use a two functions from the Mapping Toolbox to display our data in a specific map projection, namely `axesm` to create the axes and define the projection, and `scatterm` to display the data points as filled circles (MathWorks 2020) (Fig. 10.6).

```
c = ksdensity([lon,lat], [lon,lat]);

axesm('MapProjection','globe',...
      'Grid','on',...
      'Frame','off',...
      'MeridianLabel','on',...
      'MLabelLocation',30,...
      'MLabelParallel','equator',...
      'ParallelLabel','on',...
      'PLabelLocation',30,...
      'PLabelMeridian','prime',...
      'LabelFormat','signed');
```

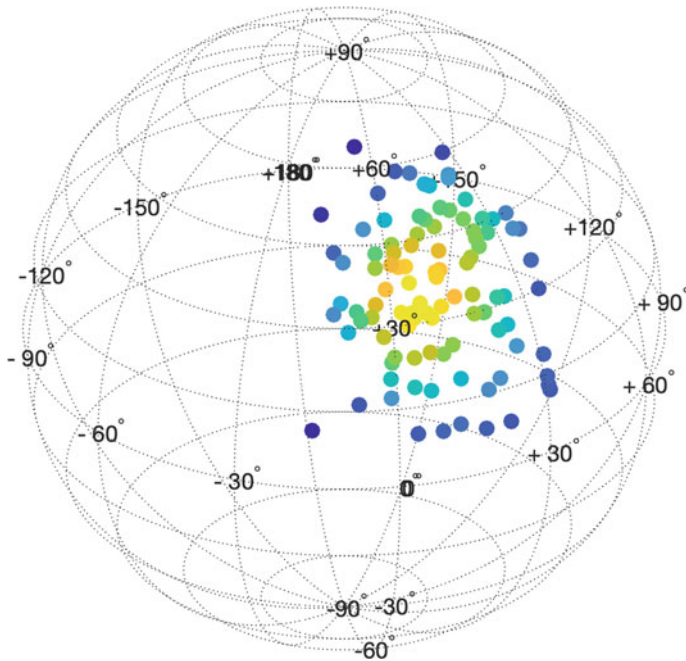


Fig. 10.6 Display of spherical data using the map projection globe with axesm.

```
scatterm(lat,lon,50,c,'filled')
view(80,30)
set(gca,'Visible','off')
```

The function `scatterm` allows us to define a size of 50 points of the circles, color the circles using the density `c`, and fill the circles using `filled`. Alternatively, we can display the data in a stereographic projection, which is a very popular way to display data in the earth sciences (Fig. 10.7).

```
clf
axesm('MapProjection','stereo',...
      'Grid','on',...
      'Frame','off',...
      'MeridianLabel','on',...
      'MLabelLocation',30,...
      'MLabelParallel','equator',...
      'ParallelLabel','on',...
      'PLabelLocation',30,...
      'PLabelMeridian','prime',...
      'LabelFormat','signed');
set(gca,'Visible','off')
scatterm(lat,lon,50,c,'filled')
```

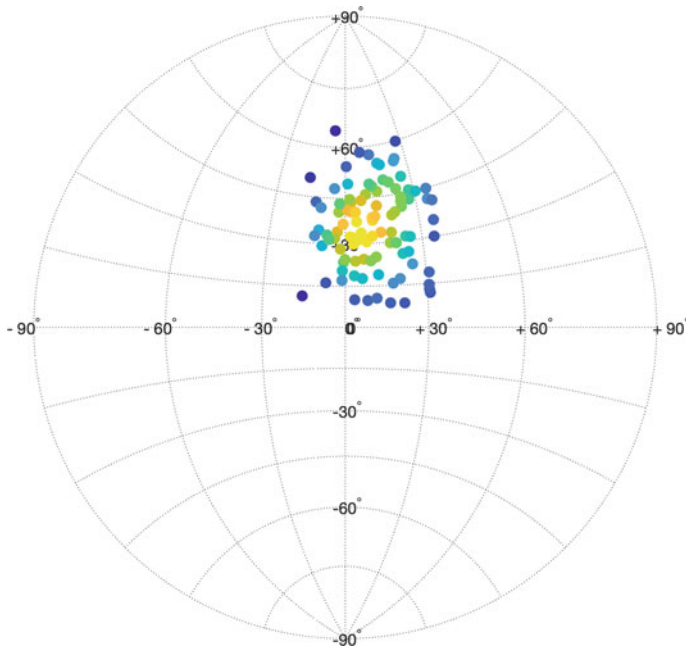


Fig. 10.7 Display of spherical data in a stereographic projection using the map projection `stereo` with `axesm`.

The last example displays the data in a simple rectangular coordinate system without using any projection (Fig 10.8):

```
axes('XLim',[-25 50],...
     'YLim',[0 75],...
     'Box','on',...
     'XGrid','on',...
     'YGrid','on'), hold on
scatter(lon,lat,50,c,'filled'), hold on
```

This simple method of presentation might be the preferred option in some applications, especially if the Mapping Toolbox or similar tools are not available.

10.9 Statistics of Spherical Data

The statistical analysis of spherical data is a big topic. Since MATLAB offers very little support for analyzing this type of data (with the exception of a few functions in the Mapping Toolbox), we will be very brief here. More detailed articles on this topic can be found in the books by Mardia and Jupp (1999) and by Borradaile (2003). The following section demonstrates how to calculate the mean, standard deviation and standard distance of geographic points. It also introduces a MATLAB function for calculating a frequency distribution for geographic points using equirectangular bins from the Mapping Toolbox (MathWorks 2020).

First, we again create synthetic data using the function `vmises` included in the electronic supplement of this book.

```
clear, clc, close all

rng(0)
n = 500;
mu_lon_degrees = 10;
mu_lat_degrees = 35;
mu_lon_radians = pi*mu_lon_degrees/180;
mu_lat_radians = pi*mu_lat_degrees/180;
kappa_lon = 10;
kappa_lat = 10;
lon_radians = vmises(n,mu_lon_radians,kappa_lon);
lat_radians = vmises(n,mu_lat_radians,kappa_lat);
lon = 180*lon_radians/pi;
lat = 180*lat_radians/pi;
```

Next, we calculate the data density with `ksdensity`.

```
c = ksdensity([lon,lat], [lon,lat]);
```

We then use the functions `meanm`, `stdm` and `stdist` from the Mapping Toolbox to calculate the mean location, the standard deviation, and the standard distance of geographic points.

```
[latmean,lonmean] = meanm(lat,lon)
[latstd,lonstd] = stdm(lat,lon)
sdist = stdist(lat,lon)
dist = stdist(lat,lon)
```

We then display the data points, together with the mean of the data in a geodensity plot using the `geodensityplot` function and axes created with `geoaxes`.

```

gx = geoaxes;
geodensityplot(gx,lat,lon,...
    'FaceColor','interp')
gx.Basemap = 'none';
gx.Scalebar.Visible = 'off';
gx.LatitudeLabel.String = ' ';
gx.LongitudeLabel.String = ' ';
gx.TickLabelFormat = '-dd';
line(latmean,lonmean,...
    'Marker','o',...
    'MarkerSize',5,...
    'MarkerFaceColor','r',...
    'MarkerEdgeColor','r')

```

We then calculate the histogram for geographic points using equirectangular bins.

```
[lats,lons,num,wnum] = histr(lat,lon,0.2);
```

As we can see the function `histr` does not actually display a 3D bar plot, it only calculates the frequency distribution which we then have to represent in a bar plot. We first create a coordinate grid

```

llon = unique(lons);
llat = unique(lats);

[X,Y] = meshgrid(llon,llat);

Z = zeros(size(X));

for i = 1 : length(num)
    Z(X == lons(i) & Y == lats(i)) = num(i);
end

```

and labels for such a bar plot by typing

```

latstring = num2str(llat,'%4.1f');
lonstring = num2str(llon,'%4.1f');

```

Then we use `imagesc` to display the frequencies in a pseudocolor plot with a red dot in the center showing the location of the mean of the geographic data.

```

imagesc(llon,llat,flipud(Z)), hold on
line(lonmean,latmean,max(Z(:)),...
    'Marker','o',...
    'MarkerSize',10,...

```

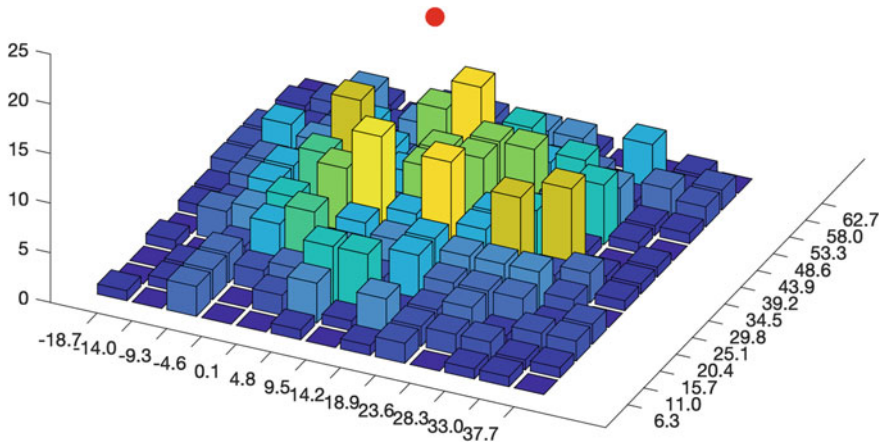


Fig. 10.8 Three-dimensional histogram showing the frequency distribution of geographic data. The MATLAB code is inspired by the function `scatterbar3` by Manu Raghavan available online from the File Exchange.

```
'MarkerFaceColor','r',...
'MarkerEdgeColor','r')
caxis([min(Z(:)) max(Z(:))])
set(gca,'YTick',llat,...
'YTickLabel',flipud(latstring),...
'XTick',llon,...
'XTickLabel',lonstring)
colorbar
```

Alternatively we can create our own script to display a 3D bar plot. Here is a possible function which is inspired by the function `scatterbar3` by Manu Raghavan, available online from the MATLAB File Exchange (Fig. 10.8).

```
close all

w = 2;
c = [0.3 0.5 0.8];
Z = Z';
axes('XLim',[-25 50],...
'YLim',[0 75],...
'View',[25 50]), hold on
for i = 1 : size(Z,1)
    for j = 1 : size(Z,2)
        d(1)=patch([-w -w w w]+llon(i),...
[-w w w -w]+llat(j),...
[0 0 0 0],c);
```



```

d(2)=patch(w.*[-1 -1 1 1]+llon(i),...
           w.*[-1 -1 -1 -1]+llat(j),...
           Z(i,j).*[0 1 1 0],c);
d(3)=patch(w.*[-1 -1 -1 -1]+llon(i),...
           w.*[-1 -1 1 1]+llat(j),...
           Z(i,j).*[0 1 1 0], 'b');
d(4)=patch([-w -w w w]+llon(i),...
           [-w w w -w]+llat(j),...
           [Z(i,j) Z(i,j) Z(i,j) Z(i,j)],c);
d(5)=patch(w.*[-1 -1 1 1]+llon(i),...
           w.*[1 1 1 1]+llat(j),...
           Z(i,j).*[0 1 1 0],c);
d(6)=patch(w.*[1 1 1 1]+llon(i),...
           w.*[-1 -1 1 1]+llat(j),...
           Z(i,j).*[0 1 1 0],c);
set(d, 'facecolor', 'flat',...
     'FaceVertexCData', Z(i,j))
end
end
caxis([min(Z(:)) max(Z(:))])
set(gca, 'YTick', llat,...
     'YTickLabel', latstring,...
     'XTick', llon,...
     'XTickLabel', lonstring)
line(lonmean, latmean, max(Z(:))+10,...
     'Marker', 'o',...
     'MarkerSize', 10,...
     'MarkerFaceColor', 'r',...
     'MarkerEdgeColor', 'r')

```

Recommended Reading

- Allmendinger RW, Cardozo N, Fisher DM (2012) Structural geology algorithms: vectors and tensors. Cambridge University Press, New York
- Allmendinger RW, Siron CR, Scott CP (2017) Structural data collection with mobile devices: accuracy, redundancy, and best practices. *J Struct Geol* 102:98–112
- Allmendinger RW (2019) Modern structural practice: a structural geology laboratory manual for the 21st century. <http://www.geo.cornell.edu/geology/faculty/RWA/structure-lab-manual/downloads.html>
- Batschelet E (1965) Statistical methods for the analysis of problems in animal orientation and certain biological rhythms. American Institute of Biological Sciences Monograph, Washington, DC
- Berens P (2009) A MATLAB toolbox for circular statistics. *J Stat Softw* 31:1–21
- Best DJ, Fisher NI (1979) Efficient simulation of the von Mises distribution. *Appl Stat* 28:152–157
- Borradaile G (2003) Statistics of earth science data—their distribution in time. Space and orientation. Springer, Berlin Heidelberg, New York
- Davis JC (2002) Statistics and data analysis in geology, 3rd edn. Wiley, New York
- Fisher NI (1995) Statistical analysis of circular data. Cambridge University Press, New York

- Gumbel EJ, Greenwood JA, Durand D (1953) The circular normal distribution: tables and theory. *J Am Stat Assoc* 48:131–152
- Jones TA (2006a) MATLAB functions to analyze directional (azimuthal) data—I: single sample inference. *Comput Geosci* 32:166–175
- Jones TA (2006b) MATLAB functions to analyze directional (azimuthal) data—II: correlation. *Comput Geosci* 32:176–183
- Mardia KV (1972) *Statistics of directional data*. Academic Press, London
- Mardia KV (1975) *Statistics of directional data*. *J R Stat Society Ser B (Methodol)* 37:349–393
- Mardia KV, Jupp PE (1999) *Directional statistics*. Wiley, Chichester
- Marques de Sá JP (2007) *Applied statistics using SPSS, STATISTICA, MATLAB and R*. Springer, Berlin Heidelberg, New York
- MathWorks (2020) *Mapping toolbox users's guide—user's guide*. The MathWorks, Inc., Natick, MA
- Middleton GV (1999) *Data analysis in the earth sciences using MATLAB*. Prentice Hall, New Jersey
- Swan ARH, Sandilands M (1995) *Introduction to geological data analysis*. Blackwell Sciences, Oxford
- Watson GS, Williams EJ (1956) On the construction of significance tests on a circle and the sphere. *Biometrika* 43:344–352
- Wilkie D (1983) Rayleigh Test for randomness of circular data. *Appl Stat* 32:311–212